

Rich Communication Suite - Advanced Communications Services and Client Specification

Version 18.0

17 July 2026

Security Classification: Non-Confidential

Access to and distribution of this document is restricted to the persons permitted by the security classification. This document is subject to copyright protection. This document is to be used only for the purposes for which it has been supplied and information contained in it must not be disclosed or in any other way made available, in whole or in part, to persons other than those permitted under the security classification without the prior written approval of the Association.

Copyright Notice

Copyright © 2026 GSM Association

Disclaimer

The GSMA makes no representation, warranty or undertaking (express or implied) with respect to and does not accept any responsibility for, and hereby disclaims liability for the accuracy or completeness or timeliness of the information contained in this document. The information contained in this document may be subject to change without prior notice.

Compliance Notice

The information contain herein is in full compliance with the GSMA Antitrust Compliance Policy.

This Permanent Reference Document is classified by GSMA as an Industry Specification, as such it has been developed and is maintained by GSMA in accordance with the provisions set out GSMA AA.35 - Procedures for Industry Specifications.

Table of Contents

1 Introduction 7
1.1 RCS Principles and Vision 7
1.2 Scope 7
1.3 Definition of Terms 8
1.4 Document Cross-References 14
1.5 Differences to previous specifications 24
1.5.1 New features and procedures 24
1.5.2 Removed features and procedures 25
1.5.3 Modified features and procedures 25
2 RCS General Procedures 26
2.1 RCS architecture 26
2.2 RCS devices and client types 28
2.3 Configuration Procedures 29
2.3.1 Client configuration parameters 29
2.3.2 RCS client Auto-Configuration mechanisms 30
2.4 IMS registration 36
2.4.1 General 36
2.4.2 Procedures for multidevice handling 38
2.4.3 Telephony feature tag 38
2.4.4 Services feature tags 39
2.4.5 P-CSCF discovery 40
2.4.6 IMS Flow Set Management 41
2.4.7 Loss of Registration 43
2.4.8 Connection management for Push Notification Mechanism 43
2.5 Addressing and identities 44
2.5.1 Overview 44
2.5.2 Sender Identification for Device Incoming SIP Request 44
2.5.3 Device Outgoing SIP Requests 45
2.5.4 Addressing related to Chatbots 46
2.5.5 RCS Addressing Model 49
2.6 Capability and new user discovery mechanisms 50
2.6.1 Capability Discovery 50
2.6.2 Handling of Capabilities 68
2.7 RCS protocols 69
2.7.1 void 71
2.7.2 MSRP session matching 71
2.7.3 SIP Issues 71
2.7.4 CPIM Body Compression 72
2.8 RCS and Access Technologies 73
2.8.1 RCS and Cellular/EPC-integrated Wi-Fi Access 73
2.8.2 Other access networks 77
2.9 End User Confirmation Requests 77

2.10 Multidevice support 77
2.10.1 Overview 77
2.10.2 Addressing of individual clients 78
2.11 Interconnect principles and guidelines 78
2.12 Access Security 79
2.12.1 IMS Security 79
2.12.2 OpenID Connect 86
2.12.3 Common Message Store Authentication and Security 87
2.13 Emergency Services 87
2.13.1 General 87
2.13.2 RCS Service Feature List 88
2.14 CPIM header extension support 88
2.14.1 CPIM header extension support feature tag 88
2.14.2 Procedures in the client 88
2.14.3 Procedures in the Messaging Server 89
2.15 Push Notification Mechanism 90
2.15.1 RCS client enabled for Push Notification Mechanism 90
2.15.2 Procedures in the Messaging Server 90
2.15.3 Client behaviour on receiving Push Notification 93
2.15.4 Capability Discovery for Push Notification 93
2.15.5 RCS client state alignment 94
2.15.6 Reconfiguration to persistent connection 95
3 RCS Services 96
3.1 General Service Overview 96
3.2 Messaging 96
3.2.1 1-to-1 Messaging Technology Selection 96
3.2.2 Standalone messaging 99
3.2.3 1-to-1 Chat 106
3.2.4 Group Chat 122
3.2.5 File Transfer 133
3.2.6 Geolocation Push services 146
3.2.7 Audio Messaging 153
3.2.8 Extended Messaging 155
3.2.9 Spam and Other Inappropriate User Behaviour Handling 169
3.2.10 Rich Text Message 174
3.2.11 User Profile sharing 177
3.3 Void 181
3.4 Void 181
3.5 Void 181
3.6 Chatbots 181
3.6.1 Architecture 181
3.6.2 Chatbot Feature tags 184
3.6.3 Discovery and specific management of Chatbots 186
3.6.4 Chatbot Information 206

3.6.5 Privacy Protection 234
3.6.6 Spam and other Inappropriate Chatbot Behaviour Handling 242
3.6.7 Traffic identification 248
3.6.8 Chatbot Service 249
3.6.9 Deferred Messaging 260
3.6.10 Rich Cards and Suggested Chip Lists 261
3.6.11 Critical Chatbots 301
3.6.12 Menu 301
3.7 MIVCs 306
3.7.1 Introduction 306
3.7.2 Provisioning 306
3.7.3 Technology Selection 306
3.7.4 ViLTE 307
3.7.5 RCS Video Chat 308
4 Cross-service functionality 358
4.1 Common Message Store 358
4.1.1 Overview 358
4.1.2 Support of GBA in the Common Message Store 359
4.1.3 Support of OpenID Connect in the Common Message Store 359
4.1.4 Support for Digest Authentication 360
4.1.5 Support for Basic Authentication 360
4.1.6 RESTful Web Service Calls 360
4.1.7 Folder Structure 362
4.1.8 Common Message Store and pager/multimedia-messages 364
4.1.9 Correlating SMS/MMS messages with messages stored in the Common Message Store 366
4.1.10 Correlation Algorithm for SMS 367
4.1.11 Dealing with Collisions 371
4.1.12 Recording of SMS messages 372
4.1.13 Recording of MMS messages 380
4.1.14 Optimisations for UNI operations to Common Message Store 390
4.1.15 A Common File Store for File Transfer via HTTP 391
4.1.16 Client behaviour 404
4.2 HTTP Content Server 410
4.2.1 Overview 410
4.2.2 File Upload Procedures 410
4.2.3 File Download Procedure 423
4.2.4 Schema Definition 426
4.2.5 HTTP Content Server URL 428
4.2.6 HTTP State Management 429
4.2.7 Security considerations 430
4.3 User Profile servers 430
4.3.1 Architecture 431
4.3.2 Inter-Profile Server Interface wireframe 432

4.3.3 Creation of a User Profile 433
4.3.4 Recipient procedures 433
4.3.5 User Profile Server Procedures 434
Annex A Managed objects and configuration parameters 437
A.1 Management objects parameters overview 437
A.1.1 Configuration parameters for the management of RCS services 437
A.1.2 Presence related configuration 438
A.1.3 Messaging related configuration 439
A.1.4 File Transfer related configuration 446
A.1.5 Push Notification Mechanism Parameter 448
A.1.6 IMS Core / SIP related configuration 448
A.1.7 Geolocation related configuration 453
A.1.8 Configuration related with Address book Back-up/Restore 453
A.1.9 Capability Discovery related configuration 453
A.1.10 Void 455
A.1.11 Void 455
A.1.12 Service Provider specific extensions 455
A.1.13 Void 455
A.1.14 Data Off 455
A.2 Provisioning Document of the RCS Management tree 459
A.2.1 Application characteristic type for the RCS Management tree 459
A.2.2 Services sub tree additions 460
A.2.3 Presence sub tree 468
A.2.4 Messaging sub tree additions 468
A.2.5 Capability Discovery MO sub tree 490
A.2.6 Service Provider Extensions MO sub tree 495
A.3 Other Management Objects 496
A.3.1 Overview 496
A.3.2 IMS sub tree additions 496
A.4 Configuration XML document structure 499
A.4.1 HTTP configuration XML structure 499
A.4.2 Void 500
Annex B Additional diagrams 501
B.1 Chat and store and forward diagrams 501
B.1.1 Store and forward: Receiver offline 501
B.1.2 Store and forward: Message deferred delivery with sender still on an active Chat session 502
B.1.3 Store and forward: Message deferred delivery with sender online 503
B.1.4 Store and forward: Message deferred delivery with sender offline (delivery notifications) 504
B.1.5 Store and forward: Notifications deferred delivery 505
B.1.6 Network Interworking to SMS/MMS 506
B.1.7 Message Revoke: Successful Request 507
B.1.8 Message Revoke: Failed Request 508

B.1.9 Deliver Stored Group Chat Messages while Chat is idle 509
B.1.10 Multi-device 510
B.1.11 Chat and store and forward diagrams: Notes 511
B.2 Restful Message Store Flows (informative) 513
B.2.1 Client Initialization and Synchronization using RESTful Approach 513
B.2.2 RESTful Notification Events 517
B.2.3 Notification Channel Setup 517
B.2.4 Object Upload 518
B.2.5 Example Object Download 518
B.2.6 Example RESTful Search operation 519
Annex C Special Procedures 524
C.1 SIP/TCP and NAT traversal 524
C.2 Errata for RFC 5438 525
C.3 Definition of RCS CPIM Header Extensions 526
C.3.1 RCS CPIM Extension Name Space 526
C.3.2 Definition of rcs.Service-Centre-Address header 526
C.3.3 Definition of rcs.Reply-Path header 526
C.3.4 Definition of rcs.Replace-Short-Message-Type header 527
C.3.5 Definition of rcs.Mms-Message-Class header 527
C.3.6 Definition of rcs.Message-Correlator header 527
C.3.7 Definition of rcs.Message-Context header 528
C.3.8 Definition of rcs.Reference-ID header 528
C.3.9 Definition of rcs.Reference-Type header 529
C.3.10 Definition of rcs.advised-action header 529
C.3.11 Definition of rcs.source header 530
C.3.12 Example CPIM Message with a Suspicion Indicator 530
C.3.13 Definition of rcs. Welcome-ID 530
C.3.14 Definition of rcs. Deep-Link-Context 531
C.3.15 Definition of rcs.User-Profile-Config-Data 531
C.4 Definition of SIP Header Extensions 532
C.4.1 User-Agent and Server Header Extensions 532
C.5 Void 534
C.6 IMDN Extensions 534
C.6.1 Definition of Delivery IMDN Extension 534
C.6.2 Delivery IMDN Example 535
C.7 Additional MIME Headers Definitions 535
C.7.1 Content-Encoding 535
C.7.2 Original-Content-Length 535
Annex D Document Management 536
D.1 Document History 536
D.2 Other Information 537

1 Introduction

1.1 RCS Principles and Vision

RCS (Rich Communication Suite) provides a framework for discoverable and interoperable advanced communication services and detailed specifications for a basic set of such advanced communication services. The functional requirements for these services and the framework components as well as their integration in the client's User Experience (UX) are described in [GSMA PRD-RCC.71].

The cornerstone mechanism that enables RCS is a service or Capability Discovery framework. For example, when a user scrolls through their address book, they will see their contacts with the RCS services that are available to communicate.

This mechanism will result in one of three types of response:

1. The contact is registered for service resulting in the contact's current service capabilities being received and logged.
2. The contact is not registered (they are provisioned but not registered).
3. The contact is not found (they are not provisioned for service).

This discovery mechanism is important since it ensures User A can determine what services are available before communicating and allows Service Providers to roll out new agreed services based on their own deployment schedule. These same mechanisms can be used to initially discover the service capabilities of all the contacts within an address book when the user first registers for the service.

1.2 Scope

This document focuses mainly on the User Network Interface (UNI) which to a large extent also determines the Network-Network Interface (NNI). The interconnect-specific aspects of the NNI are described in a separate document (see [GSMA PRD-IR.90]). Also the functional requirements, the UX and client local aspects are described in a separate document (see [GSMA PRD-RCC.71]).

It should be noted that the aim of this document is to only specify functionality that can be validated in standard compliant Internet Protocol (IP) Multimedia Subsystem (IMS) pre-production and production environments without major customisation or changes. Service Providers can still introduce customisations and changes to optimise or differentiate their networks however.

It should be noted that all pictures and flow diagrams are for informative purposes only.

1.3 Definition of Terms

Term	Description
2G	2nd Generation of Global System for Mobile Communications (GSM)
5GC	5G Core
5GS	5G System
ABNF	Augmented Backus–Naur Form
ACK	Acknowledgement
ACS	Auto-Configuration Server
AF	Anonymization Function
ALG	Application Layer Gateway
AMR	Adaptive Multi-Rate
APN	Access Point Name
AP	Authentication Proxy
API	Application Programming Interface
AS	Application Server
AuC	Authentication Centre
BA	Broadband Access
B2BUA	Back-to-Back User Agent
bool	Boolean
bps	Bits per second (used with Mbps: Mega-, kbps: kilo-)
BSF	Bootstrapped Security Function
B-TID	Bootstrapping Transaction Identifier
CAB	Converged Address Book
CFS	Client Fallback to SMS
Chatbot	See [GSMA PRD-RCC.71]
Chatbot Information	meta information on a Chatbot, amongst others a description and contact information
Chatbot Information Function	A function providing meta information on a Chatbot. The provided information will include amongst others a description and contact information
Chatbot Platform	See [GSMA PRD-RCC.71]
CMS	Common Message Store
CPIM	Common Profile for Instant Messaging
CPM	Converged IP Messaging
CRLF	Carriage Return Line Feed
CS	Circuit Switched
CSS	Cascading Style Sheets
DER	Distinguished Encoding Rules (defined in ITU-T X.690)
DNS	Domain Name System

Term	Description
DNS SRV	Domain Name System Service record
DRX	Discontinuous Reception
DTM	Dual Transfer Mode
DTX	Discontinuous Transmission
e2ae	end-to-access edge
e2e	end-to-end
E2EE	End-to-End Encryption
eIMS-AGW	Enhanced IP Multimedia Subsystem-Access GateWay
eP-CSCF	Enhanced Proxy-Call Session Control Function
EPSG	European Petroleum Survey Group
EUCR	End User Confirmation Request
FIFO	First IN First Out
FIR	Full Intra Request
Foreign User Profile Server	A User Profile Server that the client is communicating with through their Home Profile Server.
FQDN	Fully Qualified Domain Name
FTTH	Fibre To The Home
GAA	Generic Authentication Architecture
GBA	Generic Bootstrapping Architecture
GBR	Guaranteed Bitrate
GML	Geography Markup Language
GPRS	General Packet Radio Service
Group Chat Session Identity	Public Service Identity assigned to a Group Chat. It is assigned by the Messaging Server and used by the client for addressing the Messaging Server.
GRUU	Globally Routable User agent URI
GSM	Global System for Mobile Communications
GSMA	GSM Association
GSO	Group State Object
Home User Profile Server	A User Profile Server that the client is directly communicating with.
HOS	Home Operator Services
HPLMN	Home Public Land Mobile Network
HPSI	Home Profile Server Interface, proprietary interface that the RCS Client uses to share and fetch shared User Profiles from the Home User Profile Server
HSPA	High Speed Packet Access

Term	Description
HTTP	Hyper-Text Transfer Protocol NOTE: This specification uses the term “HTTP POST” and “HTTP GET” as a generic reference to the action of using the POST or GET methods of HTTP. Whether for a particular request HTTP or HTTPS is used depends on the scheme specified in the definition of that request.
HTTPS	Hyper-Text Transfer Protocol Secure
HW	HardWare
IARI	IMS Application Reference Identifier
I-CSCF	Interworking Call Session Control Function
ICSI	IMS Communication Service Identifier
ID	IDentifier
IEI	Information Element Identifier
IETF	Internet Engineering Task Force
IFC	initial Filter Criteria
IM	Instant Messaging. The term chat is also applied in this document to the same concept.
IMDN	Instant Message Disposition Notification
IMEI	International Mobile Station Equipment Identity
IMPI	Internet Protocol Multimedia Subsystem Private Identity
IMPU	Internet Protocol Multimedia Subsystem PUblic identity
IMS	Internet Protocol Multimedia Subsystem
IMS AKA	IMS Authentication and Key Agreement
Int	Integer
IP	Internet Protocol
IPsec	Internet Protocol Security
IPSI	Inter-Profile Server Interface, interface used by the Foreign and Home User Profile Server to exchange profiles.
IP-SM-GW	Internet Protocol Short Message Gateway
IPX	Internet Protocol Packet eXchange
ISIM	Internet Protocol Multimedia Services SIM
ISF	Interworking Selection Function
IWF	InterWorking Function
JSON	JavaScript Object Notation
JWK	JSON Web Key
JWS	JSON Web Signature
KB	KiloByte (i.e. 1024 bytes)
kB	Kilobyte 1 kilobyte = 10^3bytes = 1000bytes.
KDS	Key Distribution Service

Term	Description
LSB	Least Significant Bit
LTE	Long Term Evolution
MaaP	Messaging as a Platform
MAP	Mobile Application Part
Messaging Server	A server providing support for the standalone messaging service (see section 3.2.2) and/or Chat and Group Chat (see sections 3.2.3 and 3.2.4) according to [GSMA PRD-RCC.11]
MIME	Multipurpose Internet Mail Extensions
MIVC	Messaging-Initiated Video Call. See [GSMA PRD-RCC.71]
MMS	Multimedia Message Service
MMS-C	Multimedia Messaging Service Centre
MMTEL	MultiMedia TELephony
MNO	Mobile Network Operator
MO	Management Object
MSB	Most Significant Bit
MSISDN	Mobile Subscriber Integrated Services Digital Network Number
MSRP	Message Session Relay Protocol
MSRPoTLS	Message Session Relay Protocol over Transport Layer Security
Multimedia Telephony	Voice or Video Calling following [GSMA PRD-IR.92], [GSMA PRD-IR.94], [GSMA PRD-IR.51] or [GSMA PRD-NG.114] depending on the used access and packet core network
NAL	Network Abstraction Layer
NAT	Network Address Translation
NFS	Network Fallback to SMS
NGBR	Non-Guaranteed Bitrate
NNI	Network Interface
NR	New Radio
NW	NetWork
OEM	Original Equipment Manufacturer
OMA	Open Mobile Alliance
OS	Operating System
P-CSCF	Proxy-Call Session Control Function
PC	Personal Computer
PCC	Personal Contact Card
PDP	Packet Data Protocol
PIDF	Presence Information Data Format
PKI	Public Key Infrastructure
POSIX	Portable Operating System Interface

Term	Description
PPS	Picture Parameter Set
PRD	Permanent Reference Document
PS	Packet Switched
Push Controlled Idle State	A state of the RCS client where the client is waiting for Push Notification
Push Notification	A notification to the RCS client, when not connected to the RCS Service Provider, that a new incoming communication event (e.g. new Chat Message, File Transfer, Audio Message, Message or File Transfer Status / State notifications) is available for the user. The Push Notification is sent by the RCS Service Provider to wake up the RCS client for establishing a connection to the RCS Service Provider for the new incoming communication event.
Push Notification Mechanism	A mechanism that enables the RCS Service Provider to enable non-persistent connections between the RCS Service Provider and the RCS client (in contrast to the mechanism that uses persistent connections) for communication event delivery. The mechanism is not visible to the user.
Push Service Provider	An entity which provides Push Notification Mechanism to the RCS client.
QCI	Quality of Service Class Identifier
QoS	Quality of Service
RCS	Rich Communication Suite
RCS User	An end user that has device or client (and the corresponding Service Provider subscription) supporting the RCS Capability Discovery framework and at least one of the services defined in the current specification.
RCS Video Chat	A WebRTC [RFC8825] implementation of MIVC
RFC	Request For Comments
RLC	Radio Link Control
RLS	Resource List Server
RRAM	RCS Recorded Audio Message
RTCP	Real-time Transport Control Protocol
RTP	Real-time Transport Protocol
SBC	Session Border Controller
S-CSCF	Serving Call Session Control Function
SDP	Session Description Protocol
SGs interface	3GPP defined reference point between the Mobility Management Entity and the Mobile Switching Centre
SIM	Subscriber Identity Module
SIMPLE	Session Initiation Protocol for Instant Messaging and Presence Leveraging Extensions
SIO	Session Info Object
SIP	Session Initiation Protocol

Term	Description
SIPoTLS	Session Initiation Protocol over Transport Layer Security
SMPP	Short Message Peer-to-Peer
SMS	Short Message Service
SMS-C	Short Message Service Centre
SMSoIP	Short Message Service over Internet Protocol
SP	Service Provider
SPN	Service Provider Network
SFU	Selective Forwarding Unit
SSO	Single Sign On
STUN	Simple Traversal of User Datagram Protocol through Network Address Translations
Suggested Chip List	See [GSMA PRD-RCC.71]
SUPL	Secure User Plane Location
TCP	Transmission Control Protocol
tel URI	telephone Uniform Resource Identifier
TID	Transaction IDentifier
TLS	Transport Layer Security
TON	Type Of Number
TPDU	Transfer Protocol Data Unit
Trust Store	A trusted repository for establishing the chain of trust when validating Root certificates and Certificate Authorities
UA	User Agent
UCS2	2-byte Universal Character Set
UDP	User Datagram Protocol
UE	User Equipment
UI	User Interface
UID	Unique IDentifier
UNI	User Network Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
User Profile	Personal information related to a user (e.g. name and picture).
User Profile Server	A server used for storing and retrieving User Profiles.
USIM	Universal Subscriber Identity Module
UTC	Coordinated Universal Time
UUID	Universally Unique IDentifier
UX	User Experience

Term	Description
vCard	A format for electronic business cards
Verification Authority	A provider of verification for Chatbots (as defined in [GSMA PRD-NG.131])
VoLTE	Voice over Long Term Evolution
VoWiFi	Voice over EPC-integrated Wi-Fi as specified in [GSMA PRD-IR.51]
WebRTC	Web Real Time Communication
Wi-Fi	Trademark of Industry Consortium "Wi-Fi Alliance" used as synonym for WLAN (Wireless Local Area Network)
WLAN	Wireless Local Area Network
XML	Extensible Markup Language
xSIM	Generic reference to different types of SIMs (e.g. USIM, ISIM, etc.)

1.4 Document Cross-References

Ref	Document Number	Title
1	[3GPP TS 23.038]	3GPP TS 23.038 Release 10, 3rd Generation Partnership Project; Alphabets and language-specific information http://www.3gpp.org
2	[3GPP TS 23.040]	3GPP TS 23.040 Release 10, 3rd Generation Partnership Project; Technical realization of the Short Message Service (SMS) http://www.3gpp.org
3	[3GPP TS 23.140]	3GPP TS 23.140 Release 6, 3rd Generation Partnership Project; Multimedia Messaging Service (MMS);Functional description; Stage 2 http://www.3gpp.org
4	[3GPP TS 23.167]	3GPP TS 23.167 Release 11, 3rd Generation Partnership Project; IP Multimedia Subsystem (IMS) emergency sessions http://www.3gpp.org
5	[3GPP TS 23.228]	3GPP TS 23.228 Release 12, 3rd Generation Partnership Project; IP Multimedia Subsystem (IMS) Stage 2, http://www.3gpp.org
6	[3GPP TS 24.008]	3GPP TS 24.008 Release 12, 3rd Generation Partnership Project, Mobile radio interface Layer 3 specification; Core network protocols http://www.3gpp.org
7	[3GPP TS 24.109]	3GPP TS 24.109 Release 10, 3rd Generation Partnership Project; Bootstrapping interface (Ub) and network application function interface (Ua); Protocol details http://www.3gpp.org
8	[3GPP TS 24.167]	3GPP TS 24.167 Release 10, 3rd Generation Partnership Project; Technical Specification Group Core Network and Terminals; 3GPP IMS Management Object (MO) http://www.3gpp.org

Ref	Document Number	Title
9	[3GPP TS 24.229]	3GPP TS 24.229 Release 10, 3rd Generation Partnership Project; IP multimedia call control protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP) http://www.3gpp.org
10	[3GPP TS 24.229-rel11]	3GPP TS 24.229 Release 11, 3rd Generation Partnership Project; IP multimedia call control protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP) http://www.3gpp.org
11	[3GPP TS 24.229-rel12]	3GPP TS 24.229 Release 12, 3rd Generation Partnership Project; IP multimedia call control protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP) http://www.3gpp.org
12	[3GPP TS 24.229-rel15]	3GPP TS 24.229 Release 15, 3rd Generation Partnership Project; IP multimedia call control protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP) http://www.3gpp.org
13	[3GPP TS 24.301]	3GPP TS 24.301 Release 11, 3rd Generation Partnership Project; Non-Access-Stratum (NAS) protocol for Evolved Packet System (EPS); Stage 3 http://www.3gpp.org
14	[3GPP TS 24.368]	3GPP TS 24.368, 3rd Generation Partnership Project; Non-Access Stratum (NAS) configuration Management Object (MO) http://www.3gpp.org
15	[3GPP TS 24.605]	3GPP TS 24.368, 3rd Generation Partnership Project; Conference (CONF) using IP Multimedia (IM) Core Network (CN) subsystem; Protocol specification http://www.3gpp.org
16	[3GPP TS 26.143]	3GPP TS 26.143, Release 19, Messaging Media profiles. http://www.3gpp.org/DynaReport/26143.htm
17	[3GPP TS 33.203]	3GPP TS 33.203 Release 10, 3rd Generation Partnership Project; 3G security; Access security for IP-based services http://www.3gpp.org
18	[3GPP TS 33.220]	3GPP TS 33.220 Release 10, 3rd Generation Partnership Project; Generic Authentication Architecture (GAA);Generic Bootstrapping Architecture (GBA) http://www.3gpp.org
19	[3GPP TS 33.328]	3GPP TS 33.328 Release 10, 3rd Generation Partnership Project; IP Multimedia Subsystem (IMS) media plane security http://www.3gpp.org
20	[IETF-DRAFT-RKEEP]	Indication of support for reverse keep-alive, Version 00, June 21, 2012, http://tools.ietf.org/html/draft-holmberg-sipcore-rkeep-00

Ref	Document Number	Title
21	[GSMA PRD-IR.51]	GSMA PRD IR.51 - "IMS Profile for Voice, Video and SMS over untrusted Wi-Fi access" Version 6.0, 01 May 2018 http://www.gsma.com/
22	[GSMA PRD-IR.61]	GSMA PRD IR.61 - "Wi-Fi Roaming Guidelines" Version 12.0, 27 September 2017 http://www.gsma.com/
23	[GSMA PRD-IR.65]	GSMA PRD IR.65 - "IMS Roaming, Interconnection and Interworking Guidelines" Version 28.0, 02 May 2018 http://www.gsma.com/
24	[GSMA PRD-IR.67]	GSMA PRD IR.67 - "DNS and ENUM Guidelines for Service Providers and GRX and IPX Providers" Version 15.0, 29 June 2018 http://www.gsma.com/
25	[GSMA PRD-IR.88]	GSMA PRD IR.88 - "LTE and EPC Roaming Guidelines" 18.0, 07 June 2018 http://www.gsma.com/
26	[GSMA PRD-IR.90]	GSMA PRD IR.90 - "RCS Interworking Guidelines" Version 14.0, 13 October 2017 http://www.gsma.com/
27	[GSMA PRD-IR.92]	GSMA PRD IR.92 - "IMS Profile for Voice and SMS" Version 12.0, 02 May 2018 http://www.gsma.com/
28	[GSMA PRD-IR.94]	GSMA PRD IR.94 - "IMS Profile for Conversational Video Service" Version 13.0, 21 June 2018 http://www.gsma.com/
29	[GSMA PRD-NG.102]	GSMA PRD NG.102 - "IMS Profile for Converged IP Communications" Version 5.0, 29 June 2018 http://www.gsma.com/
30	[GSMA PRD-NG.110]	GSMA PRD NG.110 – "Multi-Device" Version 3.0, 13 May 2020 http://www.gsma.com/
31	[GSMA PRD-NG.113]	GSMA PRD NG.113 - "5GS Roaming Guidelines" Version 1.0 http://www.gsma.com/
32	[GSMA PRD-NG.114]	GSMA PRD NG.114 - "IMS Profile for Voice, Video and Messaging over 5GS" http://www.gsma.com/
33	[GSMA PRD-NG.131]	GSMA PRD NG.131 – "RCS Verification Authority API" Version 1.0 http://www.gsma.com
34	[GSMA PRD-RCC.11]	GSMA PRD RCC.11 RCS Endorsement of OMA CPM 2.2 Conversation Functions, Version 13.0, 28 Feb 2025 http://www.gsma.com/

Ref	Document Number	Title
35	[GSMA PRD-RCC.10]	GSMA PRD RCC.10 RCS Endorsement of OMA CPM 2.2 Interworking, Version 9.0, 16 October 2019 http://www.gsma.com/
36	[GSMA PRD-RCC.08]	GSMA PRD RCC.08 RCS Endorsement of 3GPP TS 29.311 Service level Interworking for Messaging Services, Version 9.0, 16 October 2019 http://www.gsma.com/
37	[GSMA PRD-RCC.07v8.0]	GSMA PRD RCC.07 Rich Communication Suite 7.0 Advanced Communications Services and Client Specification, Version 8.0, 28 June 2017 http://www.gsma.com/
38	[GSMA PRD-RCC.07v13.0]	GSMA PRD RCC.07 Rich Communication Suite - Advanced Communications Services and Client Specification, Version 13.0, 19 December 2022 http://www.gsma.com/
39	[GSMA PRD-RCC.14]	GSMA PRD RCC.14 HTTP-based Service Provider Device Configuration, Version 13.0, 19 February 2026 http://www.gsma.com/
40	[GSMA PRD-RCC.15]	GSMA PRD RCC.15 IMS Device Configuration and Supporting Services, Version 8.0, 04 June 2024 http://www.gsma.com/
41	[GSMA PRD-RCC.16]	GSMA PRD RCC.16 Rich Communication Suite – End-to-End Encryption Specification, Version 4.0, 17 July 2026 http://www.gsma.com/
42	[GSMA PRD-RCC.20]	GSMA PRD RCC.20, Enriched Calling Technical Specification, Version 8.0, 19 December 2022 http://www.gsma.com
43	[GSMA PRD-RCC.30]	GSMA PRD RCC.30, Rich Communication Suite – RCS gRPC, Version 1.0, 17 July 2026 http://www.gsma.com
44	[GSMA PRD-RCC.71]	GSMA PRD RCC.71 RCS Universal Profile Service Description Document, Version 4.1, 17 July 2026 http://www.gsma.com
45	[GSMA PRD-RCC.53]	RCS Device API 1.5.1 Specification, Version 3.0, 23 June 2016 http://www.gsma.com/
46	[RFC2045]	MIME (Multipurpose Internet Mail Extensions) Part One: Format of Internet Message Bodies Text IETF RFC http://tools.ietf.org/html/rfc2045
47	[RFC2047]	MIME (Multipurpose Internet Mail Extensions) Part Three: Message Header Extensions for Non-ASCII Text IETF RFC http://tools.ietf.org/html/rfc2047

Ref	Document Number	Title
48	[RFC2396]	Uniform Resource Identifiers (URI): Generic Syntax IETF RFC http://tools.ietf.org/html/rfc2396
49	[RFC2425]	A MIME Content-Type for Directory Information IETF RFC http://tools.ietf.org/html/rfc2425
50	[RFC2426]	vCard MIME Directory Profile IETF RFC http://tools.ietf.org/html/rfc2426
51	[RFC2617]	HTTP Authentication: Basic and Digest Access Authentication IETF RFC http://tools.ietf.org/html/rfc2617
52	[RFC3261]	SIP (Session Initiation Protocol) IETF RFC http://tools.ietf.org/html/rfc3261
53	[RFC3263]	Session Initiation Protocol (SIP): Locating SIP Servers IETF RFC http://tools.ietf.org/html/rfc3263
54	[RFC3264]	An Offer/Answer Model Session Description Protocol IETF RFC http://tools.ietf.org/html/rfc3264
55	[RFC3323]	A Privacy Mechanism for the Session Initiation Protocol (SIP) IETF RFC http://tools.ietf.org/html/rfc3323
56	[RFC3326]	The Reason Header Field for the Session Initiation Protocol (SIP) IETF RFC http://tools.ietf.org/html/rfc3326
57	[RFC3329]	Security Mechanism Agreement for the Session Initiation Protocol (SIP) IETF RFC http://tools.ietf.org/html/rfc3329
58	[RFC3403]	Dynamic Delegation Discovery System (DDDS) Part Three: The Domain Name System (DNS) Database IETF RFC
59	[RFC3458]	Message Context for Internet Mail IETF RFC http://tools.ietf.org/html/rfc3458
60	[RFC3501]	Internet Message Access Protocol - Version 4rev1, IETF RFC http://tools.ietf.org/html/rfc3501
61	[RFC3840]	Indicating User Agent Capabilities in the Session Initiation Protocol (SIP), IETF RFC http://tools.ietf.org/html/rfc3840
62	[RFC3862]	Common Presence and Instant Messaging (CPIM): Message Format, IETF RFC http://tools.ietf.org/html/rfc3862
63	[RFC3863]	Presence Information Data Format (PIDF), IETF RFC http://tools.ietf.org/html/rfc3863

Ref	Document Number	Title
64	[RFC3903]	Session Initiation Protocol (SIP) Extension for Event State Publication IETF RFC http://tools.ietf.org/html/rfc3903
65	[RFC3966]	The tel URI for Telephone Numbers, IETF RFC http://tools.ietf.org/html/rfc3966
66	[RFC3986]	Uniform Resource Identifier (URI): Generic Syntax, IETF RFC http://tools.ietf.org/html/rfc3986
67	[RFC4122]	The Universally Unique IDentifier (UUID) URN Namespace, IETF RFC http://tools.ietf.org/html/rfc4122
68	[RFC4479]	A Data Model for Presence, IETF RFC http://tools.ietf.org/html/rfc4479
69	[RFC4480]	RPID: Rich Presence Extensions to the Presence Information Data Format (PIDF), IETF RFC http://tools.ietf.org/html/rfc4480
70	[RFC4572]	Connection-Oriented Media Transport over the Transport Layer Security (TLS) Protocol in the Session Description Protocol (SDP), IETF RFC http://tools.ietf.org/html/rfc4572
71	[RFC4575]	A Session Initiation Protocol (SIP) Event Package for Conference State, IETF RFC http://tools.ietf.org/html/rfc4575
72	[RFC4826]	Extensible Markup Language (XML) Formats for Representing Resource Lists, IETF RFC http://tools.ietf.org/html/rfc4826
73	[RFC4867]	RTP Payload Format and File Storage Format for the Adaptive Multi-Rate (AMR) and Adaptive Multi-Rate Wideband (AMR-WB) Audio Codecs, IETF RFC http://tools.ietf.org/html/rfc4867
74	[RFC4975]	The Message Session Relay Protocol (MSRP), IETF RFC http://tools.ietf.org/html/rfc4975
75	[RFC5196]	Session Initiation Protocol (SIP) User Agent Capability Extension to Presence Information Data Format (PIDF), IETF RFC http://tools.ietf.org/html/rfc5196
76	[RFC5280]	Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile, IETF RFC http://tools.ietf.org/html/rfc5280
77	[RFC5322]	Internet Message Format IETF RFC http://tools.ietf.org/html/rfc5322
78	[RFC5365]	Multiple-Recipient MESSAGE Requests in the Session Initiation Protocol (SIP), IETF RFC http://tools.ietf.org/html/rfc5365

Ref	Document Number	Title
79	[RFC5438]	Instant Message Disposition Notification (IMDN), IETF RFC http://tools.ietf.org/html/rfc5438
80	[RFC5438Errata]	Instant Message Disposition Notification (IMDN), IETF RFC 5438 Errata ID 3013 http://www.rfc-editor.org/errata_search.php?rfc=5438 (see also section C.2)
81	[RFC5491]	GEOPRIV Presence Information Data Format Location Object (PIDF-LO) Usage Clarification, Considerations, and Recommendations, IETF RFC http://tools.ietf.org/html/rfc5491
82	[RFC5547]	A Session Description Protocol (SDP) Offer/Answer Mechanism to Enable File Transfer, IETF RFC http://tools.ietf.org/html/rfc5547
83	[RFC5626]	Managing Client-Initiated Connections in the Session Initiation Protocol (SIP), IETF RFC http://tools.ietf.org/html/rfc5626
84	[RFC5724]	URI Scheme for Global System for Mobile Communications (GSM) Short Message Service (SMS), IETF RFC http://tools.ietf.org/html/rfc5724
85	[RFC5764]	Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP) http://tools.ietf.org/html/rfc5764
86	[RFC5766]	Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN) http://tools.ietf.org/html/rfc5766
87	[RFC5870]	A Uniform Resource Identifier for Geographic Locations ('geo' URI), IETF RFC http://tools.ietf.org/html/rfc5870
88	[RFC6068]	The 'mailto' URI Scheme IETF RFC http://tools.ietf.org/html/rfc6068
89	[RFC6135]	Alternative Connection Model for the Message Session Relay Protocol (MSRP), IETF RFC http://tools.ietf.org/html/rfc6135
90	[RFC6223]	Indication of Support for Keep-Alive, IETF RFC http://tools.ietf.org/html/rfc6223
91	[RFC6265]	HTTP State Management Mechanism, IETF RFC http://tools.ietf.org/html/rfc6265
92	[RFC6350]	vCard Format Specification, IETF RFC http://tools.ietf.org/html/rfc6350
93	[RFC6665]	SIP-Specific Event Notification, IETF RFC http://tools.ietf.org/html/rfc6665

Ref	Document Number	Title
94	[RFC7065]	Traversal Using Relays around NAT (TURN) Uniform Resource Identifiers http://tools.ietf.org/html/rfc7065
95	[RFC7230]	Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing, IETF RFC http://tools.ietf.org/html/rfc7230
96	[RFC7232]	Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests IETF RFC http://tools.ietf.org/html/rfc7232
97	[RFC7233]	Hypertext Transfer Protocol (HTTP/1.1): Range Requests IETF RFC http://tools.ietf.org/html/rfc7233
98	[RFC7234]	Hypertext Transfer Protocol (HTTP/1.1): Caching IETF RFC http://tools.ietf.org/html/rfc7234
99	[RFC7235]	Hypertext Transfer Protocol (HTTP/1.1): Authentication IETF RFC http://tools.ietf.org/html/rfc7235
100	[RFC7515]	JSON Web Signature (JWS) IETF RFC http://tools.ietf.org/html/rfc7515
101	[RFC7518]	JSON Web Algorithms (JWA) IETF RFC http://tools.ietf.org/html/rfc7518
102	[RFC7616]	HTTP Digest Authentication IETF RFC http://tools.ietf.org/html/rfc7616
103	[RFC7638]	JSON Web Key (JWK) Thumbprint IETF RFC https://tools.ietf.org/html/rfc7638
104	[RFC7742]	WebRTC Video Processing and Codec Requirements https://tools.ietf.org/html/rfc7742
105	[RFC7874]	WebRTC Audio Codec and Processing Requirements https://tools.ietf.org/html/rfc7874
106	[RFC8030]	Generic Event Delivery Using HTTP Push IETF RFC http://tools.ietf.org/html/rfc8030
107	[RFC8155]	Traversal Using Relays around NAT (TURN) Server Auto Discovery http://tools.ietf.org/html/rfc8155
108	[RFC8291]	Message Encryption for Web Push IETF RFC http://tools.ietf.org/html/rfc8291
109	[RFC8292]	Voluntary Application Server Identification (VAPID) for Web Push IETF RFC http://tools.ietf.org/html/rfc8292
110	[RFC8445]	Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal http://tools.ietf.org/html/rfc8445

Ref	Document Number	Title
111	[RFC8760]	The Session Initiation Protocol (SIP) Digest Access Authentication Scheme IETF RFC http://tools.ietf.org/html/rfc8760
112	[RFC8825]	Overview: Real-Time Protocols for Browser-Based Applications http://tools.ietf.org/html/rfc8825
113	[RFC8827]	WebRTC Security Architecture http://tools.ietf.org/html/rfc8827
114	[RFC8832]	WebRTC Data Channel Establishment Protocol http://tools.ietf.org/html/rfc8832
115	[RFC8834]	Media Transport and Use of RTP in WebRTC http://tools.ietf.org/html/rfc8834
116	[RFC8835]	Transports for WebRTC http://tools.ietf.org/html/rfc8835
117	[RFC8841]	Session Description Protocol (SDP) Offer/Answer Procedures for Stream Control Transmission Protocol (SCTP) over Datagram Transport Layer Security (DTLS) Transport http://tools.ietf.org/html/rfc8841
118	[RFC8866]	SDP: Session Description Protocol http://tools.ietf.org/html/rfc8866
119	[RFC9420]	The Messaging Layer Security (MLS) Protocol https://www.rfc-editor.org/rfc/rfc9420.html
120	[RFC9577]	The Privacy Pass HTTP Authentication Scheme IETF RFC http://tools.ietf.org/html/rfc9577
121	[RFC9605]	Secure Frame (SFrame): Lightweight Authenticated Encryption for Real-Time Media http://tools.ietf.org/html/rfc9605
122	[GML3.1.1]	OpenGIS® Geography Markup Language (GML) Implementation Specification, Version 3.1.1, OGC 03-105r1 http://www.opengeospatial.org/
123	[CAB_TS]	OMA Converged Address Book (CAB) Specification, Approved Version 1.0, 13 November 2012 http://www.openmobilealliance.org
124	[CPM-SYS_DESC]	OMA Converged IP Messaging System Description, Candidate Version 1.0, 12 October 2010 http://www.openmobilealliance.org
125	[CPM-MSGSTOR-REST]	CPM Message Store using RestFul API, Draft Version 1.0, 25 Oct 2018, OMA-TS-CPM_Message_Storage_Using_RESTFul_API-V1_0-20181025-D http://member.openmobilealliance.org/ftp/Public_documents/COM/COM-CPM/Permanent_documents/OMA-TS-Message_Storage_Using_RESTFul_API-V1_0-20181025-D.zip

Ref	Document Number	Title
126	[MMSCONF]	OMA MMS Conformance Document, Approved Version 1.3, 13 September 2011 http://www.openmobilealliance.org
127	[MMSCTR]	OMA Multimedia Messaging Service Client Transactions, Approved Version 1.3, 13 September 2011 http://www.openmobilealliance.org
128	[MMSENC]	OMA Multimedia Messaging Service – Encapsulation Protocol, Approved Version 1.3 – 13 Sep 2011 http://www.openmobilealliance.org
129	[Presence]	OMA Presence SIMPLE Specification, 1.1, http://www.openmobilealliance.org/
130	[Presence2.0_DDS]	Presence SIMPLE Data Specification, Approved Version 2.0, 29 September 2009 http://www.openmobilealliance.org/
131	[Presence2.0_TS]	Presence SIMPLE Specification, Approved Version 2.0, 10 July 2012 http://www.openmobilealliance.org/
132	[PDE_14]	OMA Presence SIMPLE Data Extensions, Approved Version 1.4, 22 December 2015 http://www.openmobilealliance.org/
133	[PRESENCE2MO]	OMA Management Object for Presence SIMPLE 2.0, Approved Version 2.0, 10 July 2012 http://www.openmobilealliance.org
134	[vCard21]	vCard, The Electronic Business Card, A versit Consortium Specification, 18 September 1996 http://www.imc.org/pdi/vcard-21.doc
135	[ISO 639-1]	ISO 639-1:2002 Codes for the representation of names of languages -- Part 1: Alpha-2 code, July 2002 http://www.iso.org
136	[ISO8601]	ISO 8601:2004 Data elements and interchange formats -- Information interchange -- Representation of dates and times, 18 March 2008 http://www.iso.org
137	[POSIX]	IEEE Standard for Information Technology—Portable Operating System Interface IEEE Std 1003.1, 2013 Edition
138	[HTML-4.0]	HTML 4.01 Specification https://www.w3.org/
139	[CSS2.1]	Cascading Style Sheets Level 2 Revision 1 (CSS 2.1) Specification https://www.w3.org
140	[OpenID Connect]	OpenID Connect Core; OpenID Foundation http://openid.net/connect/

Ref	Document Number	Title
141	[WGS84]	NIMA TR8350.2, Department of Defense (DoD) World Geodetic System 1984 (WGS 84), Third Edition, US National Imagery and Mapping Agency, January 2000
142	[Emoji-Seq]	Davis, M., Ed. and P. Edberg, Ed., "Unicode Technical Standard #51: Unicode Emoji", September 2020, https://www.unicode.org/reports/tr51/#def_emoji_sequence
143	[Android-AppLinks]	Google for Developers, Android: Handling Android App Links, 20 December 2024 https://developer.android.com/training/app-links
144	[Apple-UnivLinks]	Apple Developer, Xcode: Allowing apps and websites to link to your content https://developer.apple.com/documentation/xcode/allowing-apps-and-websites-to-link-to-your-content
145	[gRPC]	gRPC Remote Procedure Calling https://grpc.io/

1.5 Differences to previous specifications

This current version of GSMA RCC.07 and the related specifications evolves on the functionality defined in GSMA RCC.07 v17.0.

The following sub-sections list the major differences.

1.5.1 New features and procedures

- End-to-End Encryption
    - Enhanced Self-heal procedure ([GSMA PRD-RCC.16] sections 6.2.4.1, 6.2.7, 6.3.2, 7.7.2.3, 7.10 and 10.1.2)
    - Allow for a maximum number of times that a client can advance the Era and Epoch ([GSMA PRD-RCC.16] sections 6.2.2, 6.2.5, 7.7.2.3, 11.2.2 and 11.2.3)
    - Procedure to inform the sender of MLS content included in a SIP request that no Negative-Delivery IMDN will be sent ([GSMA PRD-RCC.16] sections 7.2.5 and 7.7.2.1)
    - Procedure for handling Commits and Era creation in a Group Chat, including the use of a continuity token ([GSMA PRD-RCC.16] sections 7.5.1.2, 7.9.2, 7.9.3, 7.11.2.2; 7.11.12; 8.1, 8.3.1 and 10.1)
- Messaging
    - CPIM content compression (sections 2.4.1.3, 2.4.4.1, 2.7.4, A.1.3, A.2.4 and C.7)
    - User Profile sharing (sections 2.4.4.1, 2.6.1.3, 3.2.11, 4.3, C.3.15 and [GSMA PRD-RCC.16] section 7.5.7)
    - Disposition Notifications for Reaction, Recall Message and Delete Message messages (3.2.8.5.4, 3.2.8.6.3, 3.2.8.8.3 and 3.2.8.9.3)

  - Alternative gRPC-based transport for signalling and message payloads (including Messaging Initiated Video Call signalling) (section 2.1, 3.2.3.5, 3.2.3.8, 3.2.4, 3.2.6, 3.2.9, 3.6.1, 3.7.5.2, A.1.6, A.4.1 and [GSMA PRD-RCC.30])

- Chatbots
  - Provide Postback data when playing media elements (sections 3.6.10.4, 3.6.10.5.3.1 and 3.6.10.5.3.2)

1.5.2 Removed features and procedures

- Remove and Update Proposals ([GSMA PRD-RCC.16] section 6.2.3)

1.5.3 Modified features and procedures

Area	Section in RCC.07 or related specifications	Differences from RCC.07 v17.0 and related specifications
Messaging	3.2.8.5.6	Clarify that vendor-specific reaction content in proprietary CPIM headers is based on client implementation and not in scope of the specification
Messaging Initiated Video Calls	3.7.5.2.1, 3.7.5.2.1.1, A.1.3 and A.2.4	Allow to configure multiple TURN server URIs
End-to-End Encryption	2.3.2.2 and [GSMA PRD-RCC.16] section 4.1	Update format of participant_key_signed_nonce and introduce participant_key_hash for use in subsequent configuration requests
End-to-End Encryption	3.2.3.5 and [GSMA PRD-RCC.16] section 3.1	Clarify that Common Message Store-based multidevice is not used when End-to-End Encryption is enabled
End-to-End Encryption	[GSMA PRD-RCC.16] section 6.1.1	Updated retention requirements of unconsumed key material on the client
End-to-End Encryption	[GSMA PRD-RCC.16] sections 6.2.2, 6.2.5, 6.2.6, 6.2.7, 7.7.2.3 and 7.9.1	Updated server and client requirements on the validation of a Commit request (including external Commit requests, and the end_mls extension)
End-to-End Encryption	[GSMA PRD-RCC.16] sections 6.2.2, 7.5.1, 7.9.2; 7.11.10, 7.13, 9.2, 9.7.1 and 10.5	Updated procedures on the update of the Group Chat icon and subject
End-to-End Encryption	[GSMA PRD-RCC.16] sections 6.2.4 and 8.2.1	Updated procedures on confirming the acceptance of a Commit or Proposal message to its sender
End-to-End Encryption	[GSMA PRD-RCC.16] sections 7.5.1.2, 7.5.3, 7.6.3.2, 7.7.2.2 and 10.4, C.4.2 and C.4.3	Updated procedures on failure to decrypt and resending messages
End-to-End Encryption	[GSMA PRD-RCC.16] section 9.1.1	Clarified validation of encrypted messages by the Conversation Focus and client

Area	Section in RCC.07 or related specifications	Differences from RCC.07 v17.0 and related specifications
End-to-End Encryption	[GSMA PRD-RCC.16] sections 6.2.3, 7.11.8 and 9.4	Handling of self removal from a MLS group
End-to-End Encryption	[GSMA PRD-RCC.16] sections 10.1; 10.1.1, 10.2, 10.3 and C.6	Clarified Self-Healing Mechanism
End-to-End Encryption	[GSMA PRD-RCC.16] sections A.3.6	Clarified the Subject field of the Client Certificate Profile
End-to-End Encryption	[GSMA PRD-RCC.16] sections 9.5.4 and 11.2	Move of conversation to unencrypted state using the Unencrypted Singleton Era when failing to remove a client from a group
End-to-End Encryption	[GSMA PRD-RCC.16] Annex B	Updated Inter-KDS definitions (format of request_id and key_package_parameters)
End-to-End Encryption	[GSMA PRD-RCC.16] section C.1	Clarified the hash function to use for the Commitment for a value

Table 1: Modifications from RCC.07 version 17.0 and related specifications

2 RCS General Procedures

2.1 RCS architecture

For RCS, the base network element is either the IMS core system or the gRPC Messaging Server which enables peer-to-peer communication between RCS clients. Other network nodes can be deployed by the Service Provider to provide additional parts of the RCS feature set. Figure 1 illustrates a simplified example of the RCS architecture; a Service Provider may choose a different approach to implement a function within the Service Provider domain not influencing the interoperable Network-to-Network Interface (NNI) aspects.

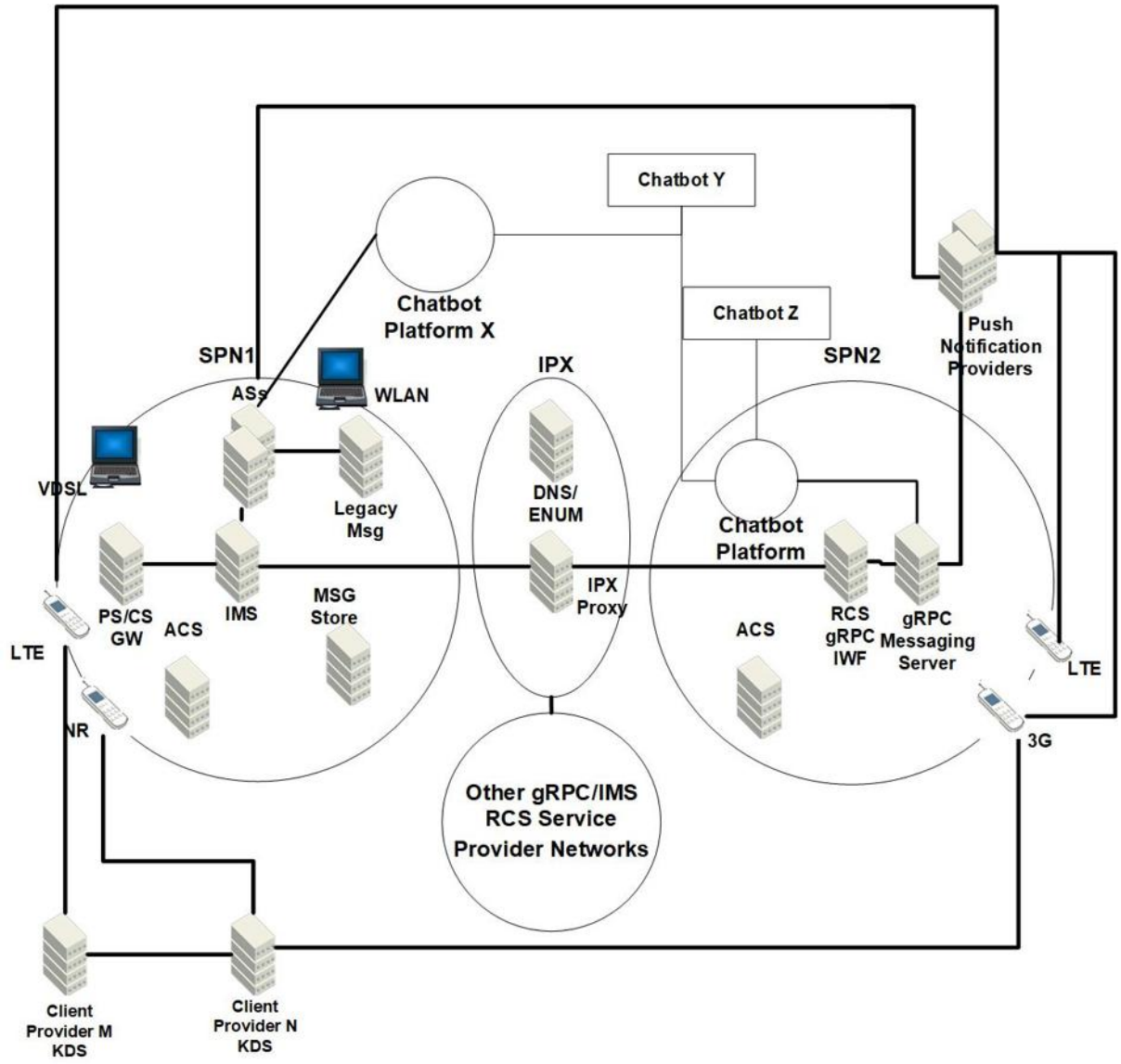


Figure 1: Simplified Example of RCS Architecture

SPN1's domain contains the SIP/MSRP related elements including the PS/CS gateway (GW) used for interworking between Circuit Switched (CS) and Packet Switched (PS) voice, for example, Voice over Long Term Evolution (VoLTE). MSG Store relates to the CPM (Converged IP Messaging) Message Store Server as illustrated in section 4.1. SPN2's domain contains the RCS gRPC related elements including the gRPC Messaging Server and the RCS gRPC IWF (interworking function). The RCS gRPC IWF is used as an interworking node between IMS-based and RCS gRPC-based implementations (the locations of the RCS gRPC IWF deployments can vary depending on the requirements of the SPN). Legacy Msg refers to the Short Message Service (SMS)/Multimedia Message Service (MMS) services that may be utilised via an IWF (Interworking Function) located in the group of Application Servers (ASs) which in addition to these IWF node(s) may also include various other nodes used by the RCS services, for example:

- Presence Server
- Messaging Server

- ASs for the support of Chatbot Functionality (see section 3.6)

An Auto-Configuration Server (ACS) is used to provide the clients with the configuration to support RCS services.

Figure 1 shows examples of two RCS Service Providers exchanging traffic with each other using the standard NNI mechanisms (IPX, IP Packet Exchange) as documented in [GSMA PRD-IR.90].

RCS compliant access networks include, but are not limited to, those illustrated in the Figure 1. Thus, deploying the RCS service does not indicate a 3G network should always be deployed. Further details of RCS services relating to particular access networks are found in section 2.6.1.3. It is recommended to follow the IMS node naming guidelines as defined in [GSMA PRD-IR.67] for naming the Messaging Server, since this server is required to be addressable across IPX.

RCS also provides support for Chatbot communications (see [GSMA PRD-RCC.71] for a definition and use cases) through the integration of Chatbot Platforms (see [GSMA PRD-RCC.71] for a definition) in the overall architecture. These platforms can either connect through the interconnect infrastructure or connect directly to an RCS Service Provider's network. Further details on Chatbot Platforms and their integration into the RCS architecture are provided in section 3.6 and section 3.6.1 in particular.

To enable End-to-End Encryption (E2EE), RCS clients rely on Client Provider-specific Key Distribution Services (KDS) that are interconnected to each other. For more details, see [GSMA PRD-RCC.16].

The RCS Service Provider may integrate with Push Service Providers to provide Push Notification Mechanism. The details are provided in section 2.15, [GSMA PRD-RCC.14] and [GSMA PRD-RCC.71]. The Push Notification Mechanism is mandatory for a Service Provider implementing the RCS gRPC protocol as per [GSMA PRD-RCC.30].

2.2 RCS devices and client types

Section 1.1.1 of [GSMA PRD-NG.110] defines two types of devices, a primary device and a secondary device. This section extends those definitions for RCS:

1. **Primary device**: a device carrying a Subscriber Identity Module (SIM) that is associated with the identity (i.e. IMPU/MSISDN) used for RCS. Two types of RCS clients exist for such a device that connect directly to the IMS:

    a) **RCS embedded client**: This is the client that is provided as part of the handset implementation and it is fully integrated with the native applications (address book, gallery/file browser application, calling application, etc.).
    b) **RCS downloadable client**: This is a client providing its own IMS connectivity that may be preinstalled or that has to be downloaded by the user. However, it is not part of the device base software, (i.e. it has no access to internal Application Programming Interfaces [APIs] and advanced Operating System [OS] functionality). The level of integration with the native applications is limited to the possibilities permitted by the corresponding mobile OS or OS platform API.

NOTE1: Next to this, there may also be downloadable clients that use terminal APIs (e.g. [PRD-RCC.53]) to access the RCS functionality that is provided by a device's RCS embedded client. This type of client is not considered in this document because it does not alter the UNI which is handled by the device's RCS embedded client.

2. **Secondary device**: a device that does not carry a SIM that is associated to the identity used for RCS

NOTE2: It may happen that a secondary device carries a SIM (e.g. a tablet or PC providing cellular data connectivity). That SIM will be associated to a different identity than the one used for RCS though.

RCS services can also be deployed using an identity that is not linked to a mobile network (e.g. a fixed-line telephone number or other identity). [PRD-RCC.14] describes in section 2.10 a generic mechanism for the configuration of such clients. Because it will be dependent on the use case, it is out of scope of this document how an identity is assigned to such devices and on which basis they are considered as a primary or secondary device.

2.3 Configuration Procedures

2.3.1 Client configuration parameters

The client shall offer individual RCS services to the user only if the Service Provider has authorised the use via the relevant client configuration parameters. In addition, the Service Provider need to provide IMS Core network configuration data to the client. RCS clients shall support the procedures for client configuration described in this section.

If the client is not configured with service configuration data for RCS or if RCS service are disabled via configuration, then the client shall not offer the relevant RCS service(s) to the user and shall disable the service specific entry points.

The set of client configuration parameters relevant for RCS services is defined in Annex A Managed objects and configuration parameters.

Some IMS Core network related configuration parameters are not applicable if the device registers services via the IMS well-known APN as defined in [GSMA PRD-RCC.15].

If the configuration data resulting from the client provisioning authorises the client to use RCS services, then the RCS client shall register with the network in accordance with definitions for the services described in this document. Once this registration process has successfully completed, the user is able to make use of the RCS services.

Client configuration parameters could also be updated and withdrawn by the Service Provider using the mechanism described in this section.

All the RCS client configuration parameters must be restricted from being modified by the user.

2.3.2 RCS client Auto-Configuration mechanisms

2.3.2.1 Overview

For the management of the configuration parameters controlling the RCS functionality in devices carrying the SIM associated with an RCS user's main identity, the mechanism defined in [GSMA PRD-RCC.14] including the enhancements described in section 2.1 of [GSMA PRD-RCC.15] shall be used.

For the configuration of additional RCS capable devices (i.e. devices not carrying the SIM associated with a subscriber's main identity), the HTTP(S) mechanism shall be used as described in [GSMA PRD-RCC.14] and [GSMA PRD-RCC.15] for such devices.

RCS devices supporting HTTP configuration as described in [GSMA PRD-RCC.14] shall support all enhancements described in section 2.1 of [GSMA PRD-RCC.15] including those relying on End User Configuration Requests (EUCR) covered in sections 2.1.2 and 2.1.3.1 of [GSMA PRD-RCC.15].

2.3.2.2 Configuration HTTP GET request parameters

The RCS client shall include in the configuration HTTP GET request the generic request parameters as defined in [GSMA PRD-RCC.14].

The RCS client shall indicate the support of RCS services by inclusion of an "app" HTTP GET request parameter as defined in [GSMA PRD-RCC.14] with the value set to "ap2002".

The RCS client shall include in the configuration HTTP GET request the parameters defined in [GSMA PRD-RCC.15].

If Enriched Calling is supported, the RCS client shall include in the configuration HTTP GET request the parameters defined in [GSMA PRD-RCC.20].

If supported, the RCS client shall indicate the support of the Non-Access Stratum Management Object defined [3GPP TS 24.368] by inclusion of an "app" HTTP GET request configuration parameter as defined in [GSMA PRD-RCC.14] with the value set to "urn:oma:mo:ext-3gpp-nas-config:1.0".

If the RCS client supports the Push Notification Mechanism as described in section 2.15.1, the RCS client shall include in the configuration HTTP GET request the parameters defined in [GSMA PRD-RCC.14] section 2.14 and follow the procedures in section 2.15.

The RCS client shall also include in the configuration HTTP GET request the parameters defined in Table 2:

Parameter	Description	Mandatory	Format
rcs_state	This is either -5, -4, -3, -2, -1, 0 or a positive integer. The parameter value shall be set to: 0 indicating that no configuration exists (e.g. out of the box, after a SIM change), or if the client received in the previous configuration	Y	Int (-5, -4,-3, -2, -1, 0 or a positive integer)

Parameter	Description	Mandatory	Format
	response a version parameter of the VERS characteristic set to "0" or a negative integer value, or if the client received in the previous configuration response a RCS DISABLED STATE parameter set to "0". A positive value indicates the version of the configuration document in which the last RCS configuration was received i.e. in most cases this will match the value of the generic *vers* parameter defined in [GSMA PRD-RCC.14]. -1 indicating that the client has received in the last configuration document the RCS DISABLED STATE configuration parameter set to "-1" (i.e. the operator has disabled the RCS services on the device/client). -2 indicating that the last configuration document included a RCS DISABLED STATE configuration parameter set to "-2" (i.e. RCS is disabled on the device, but a configuration query might be triggered on user action). -3 indicating that the last configuration document included a RCS DISABLED STATE configuration parameter set to "-3" (i.e. RCS is in a dormant state (i.e. no registration) in a way that is transparent to the user). -4 indicating that the user has explicitly disabled the RCS services on the device/client. -5 indicating that the device or RCS client has disabled RCS without user action due to an internal condition. A positive value indicates the version of the configuration document in which the last RCS configuration was received i.e. in most cases this will match the value of the generic *vers* parameter defined in [GSMA PRD-RCC.14].		
rcs_profile	String that identifies a fixed set of RCS services that are supported by the client. The services that are supported and the value to be used for the rcs_profile parameter to reference to this set are to be defined in external documents (e.g. a Service Provider's RCS Service definition document). In case multiple, (potentially overlapping) sets are supported the parameter shall be included multiple times	N	String (15 max), Case-Sensitive Multi-valued
client_vendor	String that identifies the vendor providing the RCS client.	Y	String (4 max), Case-Sensitive

Parameter	Description	Mandatory	Format
client_version	String that identifies the RCS client version. client_version_value = Platform "-" VersionMajor "." VersionMinor Platform = Alphanumeric (9 max) VersionMajor = Number (2 char max) VersionMinor = Number (2 char max) Example: client_version=RCSAndrd-1.0	Y	String (15 max), Case-Sensitive
default_sms_app	This is either 0,1 or 2 0 indicates that the OS does not allow user to select SMS application or the client cannot identify the selected SMS application 1 indicates that the RCS messaging client is selected as the default SMS application 2 indicates that the RCS messaging client is not selected as the default SMS application	N, only mandatory for client platforms supporting SMS (i.e. primary devices)	Int (0,1,2)
participant_public_key	If client has enabled End-to-End Encryption (E2EE), this is a base64 URL encoded of the Distinguished Encoding Rules (DER) representation of the public key for the participant.	N	String
participant_key_signed_nonce	If client has enabled E2EE and the Auto-Configuration Server returned a nonce as per section 2.11 in [GSMA PRD-RCC.14], client shall sign the returned nonce with the private key of the participant and the label “ACS_Participant_Key_Proof”, using the SignContent format specified in section 5.1.2 of [RFC9420]. This field is the base64 URL encoded signature.	N	String
home_kds	If client has enabled E2EE, this is the vendor ID of the Home Key Distribution Server (KDS)	N	Int
participant_key_hash	If the client has provided a Participant Public Key during initial configuration, this is a hash of the public key previously uploaded to be sent on subsequent configuration requests. This field is base64-URL encoded, using the format specified in section C.7 of [GSMA PRD-RCC.16] with label "ACS_Participant_Key_On_Config_Refresh"	N	String

Table 2: HTTP configuration: additional RCS specific HTTPS request GET parameters

2.3.2.3 Configuration request triggers

In addition to the triggers for client configuration defined in [GSMA PRD-RCC.14] and [GSMA PRD-RCC.15], the following additional triggers apply for RCS:

- if the RCS client is enabled (i.e. the last received configuration XML document

  - did include a positive integer value in the version parameter of the VERS characteristic defined in [GSMA PRD-RCC.14], and
  - did not include the RCS DISABLED STATE configuration parameter, see section A.1.1),

  and the user changes the selected SMS application resulting in a different value of the default_sms_app parameter, this shall also trigger a configuration query.

- If the user disables all RCS services in the client settings, then the client shall send a configuration request with the rcs_state set to “-4”. As a result of the processing of the configuration request, the client shall process any response in accordance with the client configuration response procedures. The client shall respect the user's request to disable RCS services.
- If the RCS client identifies an internal situation that prevents enabling RCS on the device permanently (e.g. eSIM transfer), then the client shall send a configuration request with the rcs_state set to “-5”. As a result of the processing of the configuration request, the client shall process any response in accordance with the client configuration response procedures. The client shall no longer connect to the RCS Service Provider disabling RCS services on the device.

NOTE: The recovery from rcs_state -5 is left to client implementation.

- If RCS services are disabled on the client as described above and the user enables all RCS services in the client settings, then the client shall send a configuration request with the rcs_state set to the value stored locally on the client.
- If the IMS registration fails as described in section 2.12.1.1.2, the device shall consider the device configuration invalid and perform a device configuration request as defined in section 2.12.1.1.2.
- If the HTTP Content Server returns a HTTP 401 AUTHENTICATION REQUIRED error response to the client's HTTP POST request using the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD for authentication as defined in section 4.2.2.
- If the HTTP Content Server or the Localisation function returns a HTTP 401 AUTHENTICATION REQUIRED error response to the client's HTTP GET request using the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD for authentication as defined in section 4.2.3.
- If the Common Message Store returns an authentication failure in result of the RESTful API authentication procedure using the values of the configuration parameters defined in Annex A.1.3.

2.3.2.4 RCS version management

The client configuration mechanism supports a mechanism for RCS version management to support changes of the functionality and configuration data version caused by RCS releases and set of RCS services profiles in external documents.

The configuration parameter RCS DISABLED STATE in conjunction with the SUPPORTED RCS PROFILE VERSION can be used for RCS version management as described in 2.3.2.4.

The client shall indicate the sets of profiled RCS services that it supports via the rcs_profile configuration request parameter.

The Auto-Configuration Server shall be able to indicate to the client the RCS profile versions it supports via the SUPPORTED RCS PROFILE VERSIONS configuration parameter as defined in section A.1.1.

If an Auto-Configuration Server cannot fully satisfy the client request for a specific RCS version or RCS profile versions, then it may disable RCS services by means of the RCS DISABLED STATE defined in section A1.1. The client shall use the values of the SUPPORTED RCS PROFILE VERSIONS configuration parameter to adapt its own behaviour and the client configuration request to enable RCS services via alternative versions.

The client shall inspect the values of the SUPPORTED RCS PROFILE VERSIONS configuration parameter for higher than the current negotiated version, if a valid configuration is provided by the Auto-Configuration Server. If a higher version is available and the client supports it, then the client shall change behaviour and the request parameters for a subsequent configuration request to update functionality.

2.3.2.5 RCS client management

During configuration the service provider may want to control the general state of the RCS client (e.g. disable it) without affecting the other services configured through the same configuration document. This shall be done through the generic RCS DISABLED STATE client configuration parameter defined in section A.1.1. Only if not included, other RCS settings shall be included in the configuration document. The client shall ignore any such settings if included when also the RCS DISABLED STATE parameter is included in the document.

If the Service Provider chooses to temporary disable RCS functionality on the device/client, the RCS DISABLED STATE configuration parameter shall be set to 0.

If the Service Provider chooses to permanently disable the RCS functionality on an RCS capable device/client, the RCS DISABLED STATE configuration parameter shall be set to -1.

If the SIM is swapped or the device is reset, the RCS capable device shall again set the rcs_state parameter defined in Table 2 to 0 in the queries for configuration settings.

If the Service Provider chooses to disable the RCS functionality on an RCS capable device/client until there is a User Interface (UI) dependent user action triggering a new query, the RCS DISABLED STATE parameter shall be set to -2.

If the SIM is swapped or the device is reset or the user triggers the UI dependent action, the RCS capable device shall initiate a configuration request including the rcs_state parameter set to 0.

If the Service Provider chooses to put the RCS functionality on an RCS capable device/client in a dormant state, the RCS DISABLED STATE parameter shall be set to -3.

The RCS client shall after receiving such a response behave as follows:

- It shall perform the configuration queries as if it were configured with a valid document (e.g. it performs a query at reboot, when an SMS requesting reconfiguration is received, etc.). In those queries it shall provide as value for the rcs_state parameter '-3'.
- The existing configuration document remains valid (i.e. a response with the RCS DISABLED STATE parameter set to '-3' shall be handled in this aspect as if the rcs_state parameter matched the current version available in the client).
- It shall not register into the IMS until a subsequent configuration query results in a configuration XML that does not contain the RCS DISABLED STATE configuration parameter. If it was registered when the document with the RCS DISABLED STATE configuration parameter set to '-3' is received, the client shall unregister.
- All RCS services entry points shall remain available (including those that base on cached capabilities). When the user activates RCS through one of those capabilities, the client shall:
    - Perform a configuration query providing as value for the version parameter the version of the latest configuration document that was received by the client (i.e. a positive value) including the rcs_state parameter with as value the version of the last configuration document that included a full RCS configuration (i.e. not including the RCS DISABLED STATE parameter).
    - If a new configuration document is received or the previously received document is still valid, apply that document, register into the IMS and perform a Capability Discovery to verify that the requested action is possible. If not the action is not possible, inform the user of this situation. Keep RCS active afterwards.
    - If an error or a document with a negative version is returned, inform the user that RCS is not available at that time.
    - Provide an indication to the user during these actions to show that RCS is being activated
- These actions to activate RCS shall also be performed when a SMS message requesting reconfiguration is received.

2.4 IMS registration

2.4.1 General

The device and IMS core network must follow the SIP registration procedures defined in [3GPP TS 24.229], complemented with the modifications described in this document (e.g. non-registration of some feature tags).

The device shall support the nonce storing procedures as defined in [3GPP TS 24.229-rel12] section 5.1 to allow some traffic reduction.

As specified in [3GPP TS 24.229], the SIP REGISTER request shall be sent to the IP address and port obtained via the discovery procedure (see section 2.4.5). If the device was unable to obtain a specific port, then the default port as specified in [RFC3261] shall be used.

The client shall send subsequent SIP REGISTER and non-REGISTER requests to the IP address and port that is used for the initial REGISTER, unless the security mechanism requires the use of negotiated ports for the exchange of protected messages.

In all cases, the device shall register in IMS indicating whether the device can receive SMSs associated with the identity used for RCS when the device is not registered in IMS for messaging by using the telephony feature tag described in section 2.4.3.

The device must use the authentication mechanisms as described in section 2.12.

A precondition to register is that all of the mandatory parameters presented in section A.1 shall be correctly configured. This includes those optional parameters that, due to their dependency on the configured value of a mandatory parameter, have become mandatory.

If SIP Content-Compression is supported by the client, it should include an Accept-Encoding header field set to "GZIP" in the SIP REGISTER request.

2.4.1.1 Devices Enabled for Multimedia Telephony

If the device is enabled for Multimedia Telephony then it must additionally follow the applicable procedures for registration specified in [GSMA PRD-NG.102] and [GSMA PRD-NG.114]. If the device is configured to not share a registration between Multimedia Telephony (including SMSoIP) and other RCS services (see the configuration parameter RCS VOLTE SINGLE REGISTRATION defined in section A.1.6), a User-Agent header field shall be included in the SIP REGISTER request for the registration for RCS services. This header field shall be formatted as specified in Annex C.4.1. If the device is configured to share a registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP), then the handling of the User-Agent and Server header is out of the scope of this document.

2.4.1.1.1 Use of non-cellular access

When the domain selection has selected IMS voice and the device is configured to share a registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP, see the configuration parameter RCS VOLTE SINGLE REGISTRATION defined in section A.1.6), the device is using Multimedia Telephony, and shall not directly register in non-cellular

networks (i.e. it shall not directly register over a Wi-Fi network). The client may register over EPC integrated Wi-Fi according to [GSMA PRD-NG.102].

When the domain selection has not selected IMS voice or the device is not configured to share a registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP), the device may register using non-cellular access as described in section 2.4.1.2.1.

As soon as the domain selection is again using IMS voice and a device is configured to share a registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP, see the configuration parameter RCS VOLTE SINGLE REGISTRATION defined in section A.1.6), the device shall attempt to de-register from IMS through the non-cellular access and shall register again using IMS over the cellular or EPC-integrated Wi-Fi network access.

2.4.1.2 Devices not enabled for Multimedia Telephony

If the device is not enabled for Multimedia Telephony as defined in section 2.8.1) and it is configured to support IP Voice Call and/or IP Video Call, it shall follow the procedures in [PRD-NG.110].

NOTE: Downloadable clients (see section 2.2) do not have sufficient device integration to behave as a device that is enabled for Multimedia Telephony and will, therefore, have to assume that the device is not enabled for Multimedia Telephony.

The client shall send a SIP REGISTER message to the network using the configuration parameters (SIP proxy and other IMS parameters as presented in section 2.2.1 of [GSMA PRD-RCC.15]). A User-Agent header field shall be included in this SIP REGISTER request. This header field shall be formatted as specified in Annex C.4.1.

2.4.1.2.1 Use of non-cellular access

Otherwise, the device may de-register from IMS on the cellular network and register again through direct non-cellular access when that is available. This switch to direct non-cellular access will interrupt any ongoing RCS sessions.

When registered over direct non-cellular access, all RCS traffic including the traffic from supporting protocols (i.e. XCAP and HTTP) shall use this direct non-cellular connection.

2.4.1.3 CPIM Content Compression

If the client supports CPIM content compression, the client shall include the CPIM content compression feature tag (defined in section 2.7.4.1) in the Contact header of the REGISTER requests that they send to communicate which CPIM body compression algorithms they support.

If the server supports CPIM content compression, the servers shall include the CPIM compression feature tag in the Contact header of the REGISTER response.

NOTE: It is for further study whether the Contact Header will be added by the Messaging Server or the S-CSCF.

The client and the server shall persist the received feature tag values for a given SIP registration. Both parties shall use only mutually supported compression algorithms for all subsequent SIP and MSRP sessions

If the CPIM content compression is supported by the client or the server, they shall, at minimum, advertise “gzip” as a supported algorithm.

2.4.2 Procedures for multidevice handling

A RCS client shall support the Instance ID to allow Application Servers to uniquely address clients residing on different devices as specified in [3GPP TS 24.229] based on the definitions below.

The client shall provide its instance ID in the Contact header field of REGISTER and non-REGISTER requests via the "*sip.instance*" feature tag as described in [3GPP TS 24.229].

If the device is enabled for Multimedia Telephony and other RCS services, then the device (i.e. for an embedded RCS client) shall set the value of sip.instance feature tag

- as specified in NG.114 when connected to a 5G Core (5GC)
- as specified in [GSMA PRD-NG.102] otherwise.

If the device is enabled for Multimedia Telephony, then a downloadable RCS client residing on the device shall set the value of sip.instance feature tag to a UUID (Universal Unique Identifier) value.

If the device is not enabled for Multimedia Telephony, then the client shall set the value of the sip.instance feature tag

- to the IMEI value as per [3GPP TS 24.229], if the client has access to the device IMEI, otherwise
- to a UUID value, if the client has no access to the device IMEI or no IMEI is available for the device.

If the client must provide as result of the procedures above a UUID value in the sip.instance feature tag, then the client shall use:

- the UUID value provided via the configuration parameter uuid_Value, as defined in [GSMA PRD-RCC.15]) if present in the IMS MO associated with RCS.
- otherwise, if the uuid_Value is absent in the IMS MO associated with RCS, the client shall generate a UUID value as defined in [RFC4122] section 4.2 at the time of the client instantiation, which must not be modified over the lifetime of the client instance.

2.4.3 Telephony feature tag

RCS defines a telephony feature tag used to indicate to the IMS network whether the device supports CS telephony services and hence can receive SMSs associated with the identity used for RCS when the device is not registered in IMS for messaging. The feature tag shall be included in the Contact header at registration with possible values to include “*none*” or “*cs "*. The use of any other value carried in the feature tag is out of scope for this

specification. If no value is included in the feature tag, it is treated by the IMS network as if it carried the value of “none”.

The feature tag is defined as +g.gsma.rcs.telephony=<values>.

Using the terms primary and secondary devices as defined in section 2.2:

- For a secondary device that by definition does not support CS telephony and thus does not support receiving SMSs for the identity used for RCS, the feature tag shall either not be present at all or be set as follows: +g.gsma.rcs.telephony=”none”.
- For a primary device that supports CS telephony and thus supports receiving SMSs for the identity used for RCS, the feature tag shall be set as follows even when the device is not currently in coverage conditions where CS telephony can be used: +g.gsma.rcs.telephony=”cs”.

2.4.4 Services feature tags

2.4.4.1 Service related feature tags at IMS registration as per service specifications endorsed by RCS

Based on the relevant service specifications, the client shall include the following feature tags in the SIP REGISTER request when the corresponding service has been the authorised/enabled for the used access (i.e. cellular or Wi-Fi):

RCS service	Tags
Standalone Messaging (section 3.2.2)	+g.3gpp.icsi-ref="urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.msg,urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.largemsg,urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.deferred";+g.gsma.rcs.cpm.pager-large
Chat (section 3.2.3); Group Chat (section 3.2.4)	+g.3gpp.icsi-ref="urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.session"
File Transfer (section 3.2.5)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp"
File Transfer via SMS (section 3.2.5.4)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.ftsms"
Geolocation PUSH (section 3.2.6)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.geopush" NOTE: This feature tag is included in the registration to support backward compatibility and to inform network elements of the support of the feature.
Geolocation PUSH via SMS (section 3.2.6.3)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.geosms"
Chatbot Communication using sessions (section 3.6.2.1)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot"
Chatbot Communication using Standalone Message (section 3.6.2.1)	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot.sa"

RCS service	Tags
Chatbot version supported (section 3.6.2.2)	+g.gsma.rcs.botversion="#=1,#=2,#=3"
CPIM header extension support (section 2.14)	+g.gsma.rcs.cpimext
Data Off (see section 2.8.1.5)	+g.3gpp.ps-data-off="active" or +g.3gpp.ps-data-off="inactive"
Extended Messaging (see section 3.2.8)	+g.3gpp.iari-ref=”urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended”
End-to-End Encryption (see [GSMA PRD-RCC.16])	+g.gsma.rcs.mls.mls-version=”v1”
End-to-End Encryption Encryption (see. [GSMA PRD-RCC.16])	+g.gsma.rcs.mls.mls-kds=”<home kds identifier>”
Rich Text Message (see section 3.2.10)	+g.gsma.rcs.msg.richtext="vN"
User Profile sharing (see section 3.2.11)	+g.gsma.rcs.userprofilesharing
CPIM content compression (see section 2.7.4.1)	+g.gsma.rcs.cpim-compression="<comma separated list of supported encodings>"

Table 3: RCS Services feature tags

NOTE: For a device enabled for Multimedia Telephony, the feature tags for the MMTEL and SMS over IP services may be included in addition, as per the procedures in [GSMA PRD-NG.102] and [GSMA PRD-NG.114].

NOTE: For the Rich Text Message feature tag, “vN” is a placeholder for a concrete version value (e.g. “v1”, “v2”, …).

If several IMS Application Reference Identifier (IARI) tag values or several IMS Communication Service Identifier (ICSI) tag values are included in a SIP REGISTER request, consistently with [RFC3840] and [3GPP TS 24.229], IARI tag values or ICSI tag values shall be concatenated using commas as shown in the example below:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp, urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.geopush"

Table 4: IARI tag concatenation format example

If the same ICSI or IARI tag value or feature tag is used by several services that are enabled, it shall be included only once in the SIP REGISTER request.

2.4.5 P-CSCF discovery

Prior to any initial IMS registration, the client shall discover the IP address of the P-CSCF as defined in [3GPP TS 23.228].

The P-CSCF discovery procedure shall be applied in accordance with the mode of operation of the RCS device:

- Devices enabled for Multimedia Telephony shall select the P-CSCF
  - as defined in [PRD-NG.114] when connected to a 5GC and
  - as defined in [GSMA PRD-NG.102] otherwise.
- Other devices shall select the P-CSCF from the LBO_P-CSCF_Address node of the IMS management object. If the P-CSCF AddressType of the P-CSCF address in the IMS management object indicates "FQDN", the device shall resolve the Fully Qualified Domain Name (FQDN) as defined in [RFC3263].
  For the protocol selection in [RFC3263], the device shall
  - Take the SIP transport protocol settings in the IMS device management object (as defined in section 2.2.1 of [GSMA PRD-RCC.15]) into account if SIP Digest is to be used for authentication, i.e. SIPoUDP (SIP over UDP), SIPoTLS (SIP over Transport Layer Security) or SIPoTCP (SIP over TCP) depending on the access network type (PS or Wi-Fi).
  - Ignore the SIP transport protocol settings in the IMS device management object (as defined in section 2.2.1 of [GSMA PRD-RCC.15]) if AKA is to be used for Authentication. The device selects either UDP or TCP as defined in [3GPP TS 24.229].

If the P-CSCF discovery results in a list of P-CSCF addresses, then the device shall select a new P-CSCF address for any initial registration in accordance with the priority indications (e.g. weight and priorities in Domain Name System Service [DNS SRV] records) to support load distribution in the network.

2.4.6 IMS Flow Set Management

IMS flow set is defined in [3GPP TS 24.229]. It refers to the "flow" defined by the combination of transport protocol, client IP address and port and P-CSCF IP address and port used by the client and the network to exchange all SIP signalling related to a single IMS registration. This section details the requirements for an RCS client to manage the IMS flow set (i.e. a single Registration) in the network.

2.4.6.1 Non REGISTER Request Handling

The RCS client shall make use of the procedures for methods excluding the REGISTER method as defined in [3GPP TS 24.229].

The following addition applies to [3GPP TS 24.229] section 5.1.2A.1.1:

- The proper preload route header for methods excluding the REGISTER method shall be built only with the IP address learnt through the P-CSCF discovery procedure, i.e. a FQDN must not be used.

2.4.6.2 IMS Flow Set Termination

The RCS client should ensure that an IMS flow set is released in the network before the conditions for the existence cease to exist, e.g. prior to the release of the bearer the IMS flow set makes use of.

The IMS flow set shall be terminated by the client by sending a de-registration request to the network using the IMS flow set to be terminated. If there is one or more ongoing session on the IMS flow set, these shall be released first.

2.4.6.3 Loss of Connection to P-CSCF

If the connection to the P-CSCF fails (e.g. TCP time-out), the RCS client should select another P-CSCF address from the list of addresses obtained during the P-CSCF discovery in accordance with their priority indication.

If the P-CSCF discovery is based on the IMS management object and it contains one or more FQDNs, then the client shall invoke the [RFC3263] FQDN resolution anew. A different P-CSCF address shall be selected from the name resolution result in accordance with their priorities and weights.

The client shall then send a new initial registration using the new discovered P-CSCF address.

2.4.6.4 Loss of Connectivity

If a RCS client discovers that connectivity has been lost, then it should attempt to re-establish the connection.

For a client that is not enabled for Multimedia Telephony or a client enabled for Multimedia Telephony that is not configured to share the registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP), when connectivity has been resumed then;

- If the IP address has been changed and the transport protocol setting for the new connection (as derived from the Management Object defined in section 2.2.1 of [GSMA PRD-RCC.15] for the new access network type) is the same as for the lost connection and the IMS registration is not yet expired, the client shall perform a new initial registration to the P-CSCF address of the last IMS flow set in use.
- If the IP address has not been changed and the IMS registration is not yet expired, the client shall perform a re-registration using the existing IMS flow set if the IP address has not been changed and the IMS registration is not yet expired. To minimize the network impact in cases of unstable connectivity conditions the client should hold a minimum re-registration time in which no such re-registration requests are sent. The minimum re-registration time should be typically in the range of 3-5 minutes.
- In all other cases, the client shall perform a new P-CSCF discovery and a new initial registration.

NOTE: The registration or re-registration may trigger delivery of messages stored in the network during the absence of connectivity.

2.4.6.5 Detection of Connection Loss in RCS Clients with no Bearer Control Capabilities

RCS client implementations may have no capability to identify the cause of a connection loss due to missing bearer control capabilities.

These clients should identify the cause of a loss of connectivity via the following procedure.

- If the client detects a connection loss during a P-CSCF signalling interaction (e.g. TCP time-out), then it shall attempt the procedure defined in section 2.4.6.3.
- Only if a new IMS flow set is established with an alternative P-CSCF the client shall release the IMS flow set used for the old P-CSCF locally.
- If the connection establishment to the alternative P-CSCF or other targets in the network fails (e.g. DNS Server), then the client shall assume loss of connectivity and act as defined in section 2.4.6.4.
- If the client detects a connection loss during network interactions other than signalling with the P-CSCF (e.g. media connection, Auto-Configuration Server) then the client shall assume loss of connectivity.

2.4.7 Loss of Registration

When the client receives a SIP response to a non-REGISTER request that is either:

- 403 FORBIDDEN without a warning header, or
- 504 Server Timeout containing a P-Asserted-Identity URI matching a URI received during registration in Service-Route or Path header field and containing a 3GPP IM CN subsystem XML body with the <alternative-service> child element with the <type> child element set to “restoration” and the <action> child element set to “initial-registration”

(indicating loss of registration due to change of IP, expiration, network problem), the client shall attempt to register again using the procedure in section 2.4.1. When successful the client shall resend the request that caused the error response. If this fails for 5 consecutive retries though, no further attempt shall be made and an error should be shown to the user. For all services except One-to-One Chat, the retry procedures will also be stopped if it takes longer than 5 seconds. Also in that case, an error message should be shown to the user.

NOTE: On receiving a 403 FORBIDDEN response, a client may before re-Registration first attempt to send a SIP request to his own URI and only re-Register if that request results in a 403 FORBIDDEN response.

2.4.8 Connection management for Push Notification Mechanism

An RCS client enabled for a Push Notification Mechanism may bring down (e.g. TCP FIN in case of SIP over TLS/TCP) the connection with the P-CSCF based on the client configuration parameter TimerIdle (Table 206).

Once all MSRP and SIP sessions are terminated, the client may start a local timer with an expiry equal to the value of the client configuration parameter TimerIdle. Any subsequent SIP request shall reset this timer. If this timer expires, the client may close the connection with the P-CSCF by using the appropriate transport layer protocol mechanism. The client

shall not send a SIP de-registration to the RCS Service Provider. This state of the client is referred to as Push Controlled Idle State. In this state client shall refresh its capabilities (see section 2.15.4.2) and its SIP registration (see section 2.4).

Other IMS-based services do not provide support for using a Push Notification Mechanism. Therefore, the RCS Service Provider shall not enable the Push Notification Mechanism on the RCS client (see the PUSH NOTIFICATION client configuration parameter defined in section A.1.5) when configuring the device to share a registration between Multimedia Telephony (including SMSoIP) and other RCS services (see the configuration parameter RCS VOLTE SINGLE REGISTRATION defined in section A.1.6).

2.5 Addressing and identities

2.5.1 Overview

Telephone numbers in the legacy address book must be usable (regardless of whether RCS contacts have been enriched or not) for the identification of contacts of incoming and outgoing SIP requests.

Also, RCS users, especially in Enterprise segments, may be assigned a non-Mobile Subscriber Integrated Services Digital Network Number (MSISDN) based identity. The RCS client would in that case be provisioned with only the appropriate SIP URI parameter as seen in section 2.2.1.1.2 of [GSMA PRD-RCC.15], leaving the tel URI parameter empty.

Consequently, an RCS enabled terminal's address book should also be able to store alphanumeric SIP URIs as part of a contact's details.

NOTE: The identification in Common Profile for Instant Messaging (CPIM) headers is discussed in section 3.2.3 and 3.2.4.

2.5.2 Sender Identification for Device Incoming SIP Request

For device incoming SIP requests, the address(es) of the contact are, depending on the type of request, provided as a URI in the body of a request or contained in the *P-Asserted-Identity* and/or the *From* headers. The Service Provider shall always provide the *P-Asserted-Identity* header field(s) towards the RCS client. The only exception to this rule is when a request for Chat or Standalone Messaging includes a *Referred-By* header (it is initiated by Messaging Server for example in a store and forward use case as described in 3.2.3.3), thereby the *Referred-By* header should be used to retrieve the originating user instead.

The receiving client will try to extract the contact's phone number out of the following types of URIs:

- tel URIs (telephone URIs, for example tel:+1234578901, or tel:2345678901;phone-context=<phonecontextvalue>)
- SIP URIs with a "user=phone" parameter, the contact's phone number will be provided in the user part (for example sip:+1234578901@operator.com;user=phone or sip:1234578901;phone-context=<phonecontextvalue>@operator.com;user=phone)

Once the MSISDN is extracted, it will be matched against the phone number of the contacts stored in the address book. If the received URI is a SIP URI but does not contain the "*user=phone*" parameter, the incoming identity should be checked against the SIP and tel URI address of the contacts in the address book instead.

If more than one *P-Asserted-Identity* is received in the message, all identities shall be processed until a matched contact is found.

If a matched contact is found for which the telephone number stored in the address book is not international format, the client shall cache for that contact the MSISDN extracted from the signalling for a duration equal to the value configured for the CAPABILITY INFO EXPIRY client configuration parameter defined in section A.1.9.

2.5.3 Device Outgoing SIP Requests

2.5.3.1 Identification of the target contact

If the target contact contains a SIP or tel URI, the value shall be used by the RCS client when generating the outgoing request even if an MSISDN is also present for the contact. This applies to the SIP Request-URI and the "*To*" header (as defined in [3GPP TS 24.229]) for 1-to-1 communication, including the URIs used in the recipient list and Refer-To header field included in outgoing SIP requests for Group Chat.

If no SIP or tel URI is present, the RCS client shall use the MSISDN extracted from SIP signalling that was cached as specified in section 2.5.2 or below. If that is not available, the RCS client shall use the telephone number (in local format for example *0234578901* or international format *+1234578901*) set in the address book or a dial string entered by the user.

If the target number is an international-format telephone number, the device shall be able to send it as tel URI (for example "*tel:+12345678901*") as defined in [RFC3966].

If the target number is a non-international format telephone number, the RCS client shall be able to send it as tel URI with a phone-context value set as defined in [3GPP TS 24.229] for home local numbers (for example *tel:0234578901;phone-context=<home-domain-name>*). In this case, if the response provides in the P-Asserted-Identity header field, the MSISDN of the contact in international format, the client shall cache for that contact the MSISDN extracted from the response for a duration equal to the value configured for the CAPABILITY INFO EXPIRY client configuration parameter defined in section A.1.9.

2.5.3.2 Self-Identification to the network and the addressed contact

When generating an outgoing non-REGISTER request, the RCS client shall populate the *From* header field and may populate the *P-Preferred-Identity* header field with a SIP or tel URI which has been received in the *P-Associated-URI* header field returned in the 200 OK to the SIP REGISTER. If both a SIP URI and a tel URI are available to the RCS client, the tel URI should be used.

2.5.3.3 User alias

The user shall be able to specify an alias or a username for RCS services. This information will be sent when establishing a communication service with another user so they are able to receive additional information (i.e. beyond than just a MSISDN), if the originating user is not in the receiver's address book. This scenario will likely be common with Group Chat sessions.

This alias information will be set in the *From* header of the SIP request as the display name and in a Group Chat also in the CPIM *From* header as the formal name.

When receiving a request, the RCS client device shall follow the rules explained in section 2.5.2 and extract the MSISDN or SIP URI. To avoid spam and identity manipulation, the receiver shall check the identity of the calling user against the address book. If the user is not in the address book, the alias information must then be used to provide more information about the calling user while clearly displaying in the UI that the identity is unchecked and it could be false. Otherwise, the name of the contact in the address book shall be used instead.

2.5.4 Addressing related to Chatbots

2.5.4.1 Chatbot Service Identifier

The Chatbot service identifier is a SIP URI identifying a Chatbot.

2.5.4.1.1 Discovery of the Chatbot Service Identifier for a tel URI

The Chatbot service identifier can be discovered for a tel URI, via the Chatbot Information procedures defined in Section 3.6.4.1.

Once the Chatbot service identifier has been discovered for a tel URI, it shall be used to address the Chatbot.

2.5.4.1.2 SIP URI format

The Chatbot service identifier SIP URI,

- may include a FQDN including the "botplatform" subdomain;
- may include a FQDN following the format the format botplatform.mnc<MNC>.mcc<MCC>.3gppnetwork.org,
  whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning (as defined in [GSMA PRD-IR.67]) for bots hosted on a Chatbot Platform hosted by a Service Provider. This enables the traceability of Chatbot transactions among operators;
- shall not include the URI parameter user=phone;
- may include URI parameters other than user=phone;
- unless derived from a Vanity-Service-ID as per section 2.5.4.1.3, shall include a user part set to the unique identifier assigned by the Chatbot Platform to the Chatbot. It shall be formatted according to the following definition:

```
bot-service-id-userpart = bot-identifier ["." bot-publisher-id]
```

```
bot-identifier    = 1* (bot-id-allowed)
bot-publisher-id  = 1* (bot-id-allowed / ".")
bot-id-allowed    = ALPHA / DIGIT / bot-id-unreserved / escaped-char
bot-id-unreserved = "-" / "_" / "!" / "~" / "*" / "'" / "(" / ")" /
                    "&" / "=" / "+" / "$" / "," / ";" / "?" / "/"
escaped-char      = "%" HEXDIG
```

where the different parts shall be used as follows:

- The `bot-identifier` part shall be an identifier for the Chatbot
- The `bot-publisher-id` part is, as indicated, optional. If provided, it shall be a unique identifier provided by the Chatbot Platform for the party (e.g. a business) providing the Chatbot. This will allow parties in the signalling path to determine that different Chatbots are provided by the same party.

Table 5 depicts the layout of the Chatbot service identifier based on the above rules.

sip:<bot-service-id-userpart>@botplatform.<botplatformdomain>

Table 5: Chatbot service identifier format

2.5.4.1.3 Vanity-Service-ID

Vanity-Service-IDs are used to simplify the input or representation of Chatbot service identifiers.

A contact address of a Chatbot shall be considered to be a Vanity-Service-ID, if a contact address matches the definitions of a Vanity-Service-ID but does not match the definitions of a phone number, as per RCS addressing model defined in section 2.5.5.

Prior to the use of the contact address for Capability Discovery, Chatbot Information requests and the establishment of a communication with the Chatbot, the client shall transform the Vanity-Service-ID to a fully qualified SIP URI in accordance with the definitions of a SIP URI in [RFC3261] as follows.

The client shall

- if the Vanity-Service-ID contains characters being reserved within the "userinfo" part of the SIP URI, escape these characters using the ""%" HEX" mechanism, as defined in [RFC3261], and
- prefix to the string the "sip:" scheme string, and
- suffix to the string a "@" character and a "host" part containing the FQDN composed from the service providers MCC and MNC as defined in section 2.5.4.1.2. When generating the FQDN, the client shall use its home network operator MCC and MNC values.

Examples:

If the home network operator of the client is assigned with the following MCC and MNC values:

MCC = 262, MNC = 01

then a Vanity-Service-ID:

"acme-corporation"

results in the Chatbot service identifier SIP URI:

"sip:acme-corporation@botplatform.mnc001.mcc262.3gppnetwork.org".

A Vanity-Service-ID:

"корпорация-acme"

results in the Chatbot service identifier SIP URI:

"sip:%D0%BA%D0%BE%D1%80%D0%BF%D0%BE%D1%80%D0%B0%D1%86%D0%B8%D1%8F-acme@botplatform.mnc262.mcc001.3gppnetwork.org".

When showing a Chatbot service identifier to the user, the client may omit the "sip:" scheme string, the "@" character of the "userinfo" part and the "host" part of a SIP URI identifying a Chatbot, if

- the SIP URI does not contain parameters, and
- the "host" part contains the FQDN composed from MCC and MNC as defined in section 2.5.4.1.2, and
- the values of MCC and MNC are the values of the client's home network operator.

If the client omits the "sip:" scheme string, the "@" character of the "userinfo" part and the "host" part of the SIP URI, and if the remaining part of the "userinfo" part contains escaped characters, then the client shall “unescape” these characters.

2.5.4.2 Anonymized Address for user of a Chatbot

An anonymized address used to hide the URI of a User shall follow the following syntax:

- sip:<token>@<routable hostname>;user=rcstk

NOTE: According to [RFC3261], if the URI contains a semicolon, the URI must be enclosed in angle brackets (< and >).

The ‘rcstk’ value in the ‘user’ URI parameter indicates that the user is using a token to hide their identity.

How the <token> is created is left to the Service Provider’s implementation. The token value shall be unique in the <routable hostname> realm.

2.5.4.3 Indication that anonymization was used for the User's identity

In order for the client to know if a token is used by a Chatbot to communicate with the user, the following URI parameter is defined:

- tk
  - Values
    - off : Anonymization is not used for the conversation
    - on : Anonymization is used for the conversation

  The '*tk'* URI parameter shall be added by the Anonymization Function (AF) to the Chatbot's SIP URI transported by the P-Asserted-Identity in SIP requests and responses going towards the client. The value of the '*tk'* SIP parameter indicates to the client whether a token is used between the AF and the Chatbot in the conversation. If the value is set to 'off', then no token is used in the conversation.

Usage example of a Chatbot SIP URI indicating that the Chatbot is using the token:

```
<sip:mybot@botplatform.foodomain;tk=on>
```

NOTE: According to [RFC3261], if the URI contains a semicolon, the URI must be enclosed in angle brackets (< and >).

2.5.4.4 Anonymized and Non-Anonymized Conversations

When comparing P-Asserted-Identity SIP URI values in incoming SIP requests/responses, and in Message Store synchronizations to existing Chatbot contacts in the address book, any added URI parameters shall be ignored by the client. This allows anonymized and non-anonymized messages to be interleaved in the same Chatbot conversation history. The client shall still use the URI parameters as defined in section 2.5.4.3 to indicate to the user which messages were part of an anonymized conversation and which were part of a non-anonymized conversation.

2.5.5 RCS Addressing Model

This section provides a RCS Addressing model for the determination of address types for user provided contact addresses. The following address types are defined:

- SIP URI
  to address a contact with a SIP URI, the client shall initiate SIP outgoing requests as defined in section 2.5.2 for the case where the contact is a SIP URI.
- phone number
  to address a contact with a phone number, the client shall initiate SIP outgoing requests as defined in section 2.5.2 for the case where the contact is a telephone number.
- Vanity-Service-ID
  to address a contact with a Vanity-Service-ID, the client shall apply the processing defined in section 2.5.4.1.3 prior to the initiation of SIP outgoing requests.

- e-mail
  to address a contact with an e-mail address, the client shall initiate a messaging request using the MMS service.

The following provides the formal definition of the address types of the RCS addressing model.

```
address-type = SIP-URI / phone-number / vanity-service-id / e-mail
  ; SIP-URI is defined in [RFC3261]

phone-number = [+] *(DIGIT / separator) DIGIT *(DIGIT / separator)

separator = "-" / "." / "(" / ")" / "*" / "#"

vanity-service-id = vanity-id-text

vanity-id-text =  1*allowed-utf8char

allowed-utf8char =  %x21-3F / %x41-7E / UTF8-NONASCII
  ; UTF8-NONASCII is defined in [RFC3261]

e-mail = mailbox
  ; mailbox is defined in [RFC5322]
```

A contact address matching both the phone number and the Vanity-Service-ID types shall be processed as a phone number.

2.6 Capability and new user discovery mechanisms

2.6.1 Capability Discovery

The capability or service discovery mechanism is a process which enhances service usability by allowing a user to understand the subset of RCS services available to access and/or communicate with their contacts, at certain points in time.

As contacts identified by a tel URI or SIP URI with a user=phone parameter might be Chatbots (see Section 3.6), an RCS client shall, for a contact that is identified by such a URI and that is not in the Local Address Book (and for which no cached capabilities exist), first attempt to retrieve Chatbot Information using the procedures defined in Section 3.6.4. If this procedure fails to return a valid Chatbot service identifier or fallback URI, the RCS client shall consider the contact as not being a Chatbot and apply the procedures in this section to determine the services available to the contact.

When available, the RCS specification provides two alternative mechanisms to perform the Capability Discovery:

- SIP OPTIONS exchange (section 2.6.1.1):
  - The SIP OPTIONS end-to-end message is used both to query the capabilities (services which the other user has available) of the target contact and to pass the information about some of the capabilities that are supported by the requester.

  - This method requires a specific application server (Options-AS) in the network to provide multidevice support and, potentially, include optimisations.

- Presence (section 2.6.1.2):

  - In this case, instead of performing an end-to-end transaction, the capabilities are queried against a server using the standard Open Mobile Alliance (OMA) SIP for Instant Messaging and Presence Leveraging Extensions (SIMPLE) Presence procedures which are described in detail in section 2.6.1.2.
  - Consistent with the previous paragraph and the OMA SIMPLE Presence procedures, this method requires a Presence Server and optionally (see section 2.6.1.2.4) a XDM server in the network.

The discovery mechanism that is to be used by the device is by using the configuration parameter CAPABILITY DISCOVERY MECHANISM (see Annex A section A.1.9).

The interoperability between the mechanisms is provided based on network interworking (section 2.6.1.4.1). Interoperability is achieved by deploying a network based interworking function which translates requests and responses between the SIP OPTIONS and presence-based Capability Discovery mechanisms.

2.6.1.1 Capability Discovery process through SIP OPTIONS message

This mechanism for Capability Discovery is based on the exchange of a SIP OPTIONS request as defined in [RFC3261], a peer-to-peer message exchanged between clients.

This mechanism is based on the use of tags corresponding to the different RCS services that are defined in section 2.6.1.3 that are transported in the *Contact* header field for the SIP OPTIONS request and its responses:

- The tags corresponding to the relevant set of functionalities that are supported by the requesting terminal at the time this request is made, shall be carried in the Contact header field of the SIP OPTIONS request. Only the tags that are related to services on which the client wants to obtain information, shall be considered relevant and should be included in the Contact header field (e.g. a request that is intended to obtain information on a contact's capabilities for messaging services should not include feature tags related to Call Composer).

NOTE1: The set of services that is considered relevant to be included in the request is left to client implementation given that it will be dependent on the specifics of the UI interaction that was the trigger for initiating the request.

NOTE2: The information on the relevant services might be used for routing decisions when the capabilities for the contact are split over different registrations (see [GSMA PRD-NG.102]) and would ensure a more consistent user experience in error scenarios.

- The tags corresponding to the subset of the functionalities that are supported by the receiver shall be included in the Contact header of the 200 OK responses.

In RCS, the SIP OPTIONS request shall NOT contain a Session Description Protocol (SDP) body.

Next to the relevant tags defined in section 2.6.1.3, a device should also add to the Contact header field the same feature tags used at SIP Registration (see section 2.4.4) if not already included in the SIP OPTIONS request/response for Capability Discovery and if they are part of the capabilities that are relevant to the device at this time.

When a SIP OPTIONS message is sent from User A to User B, User A shall handle the response as described in the following table:

Response	User B was a known RCS user before	User B was not a known RCS user before
200 OK including at least, one of the tags assigned to the RCS Services (see Table 10) Returned when User B is an RCS user and is currently registered	User B remains an RCS user The capabilities returned in the 200 OK response (using tags as described in Table 10) are considered as the current communication options with User B	User B is marked as an RCS user The capabilities returned in the 200 OK response (using tags as described in section Table 10) are considered as the current communication options with User B
200 OK not including any of the tags used by RCS services (see Table 10) Returned when User B is registered, but not with an RCS client	User B is not considered as an RCS user any longer Only the non-RCS communication services (e.g. voice calls, SMS, MMS, etc.) are indicated as available	No change in User B's status Only the non-RCS communication services (e.g. voice calls, SMS, MMS, etc.) are indicated as available
480 TEMPORARY UNAVAILABLE or 408 REQUEST TIMEOUT Returned by the network if User B is an IMS (and potentially thus an RCS) user, but is currently not registered	User B remains an RCS user but only the capabilities available to an offline contact are offered	No change in User B's status Only the non-RCS communication services (e.g. voice calls, SMS, MMS, etc.) are indicated as available
404 Not Found or 604 Does Not Exist Anywhere	User B is not considered as an RCS user any longer Only the non-RCS communication services (e.g. voice calls, SMS, MMS, etc.) are indicated as available	No change in User B's status Only the non-RCS communication services (e.g. voice calls, SMS, MMS, etc.) are indicated as available
Any other Final response returned by the network	User B remains an RCS user with unchanged capabilities. NOTE: The client treats the final response as described in [3GPP TS 24.229].	No change in User B's status

Table 6: Options response handling

The handling of a 200 OK response that does not include any of the tags used by RCS services defined in Table 6, means that an AS like the OPTIONS-AS described in section 2.6.1.1.1 would have to include the Chat capability defined in section 2.6.1.3 in the response if the user has multiple devices sharing the same IMS identity some of which are not RCS capable. When including this tag in situations where none of the RCS capable devices is online, the OPTIONS-AS shall also include the *automata* tag defined in [RFC3840] to indicate that this response does not originate from an end user device.

This is illustrated in Figure 2:

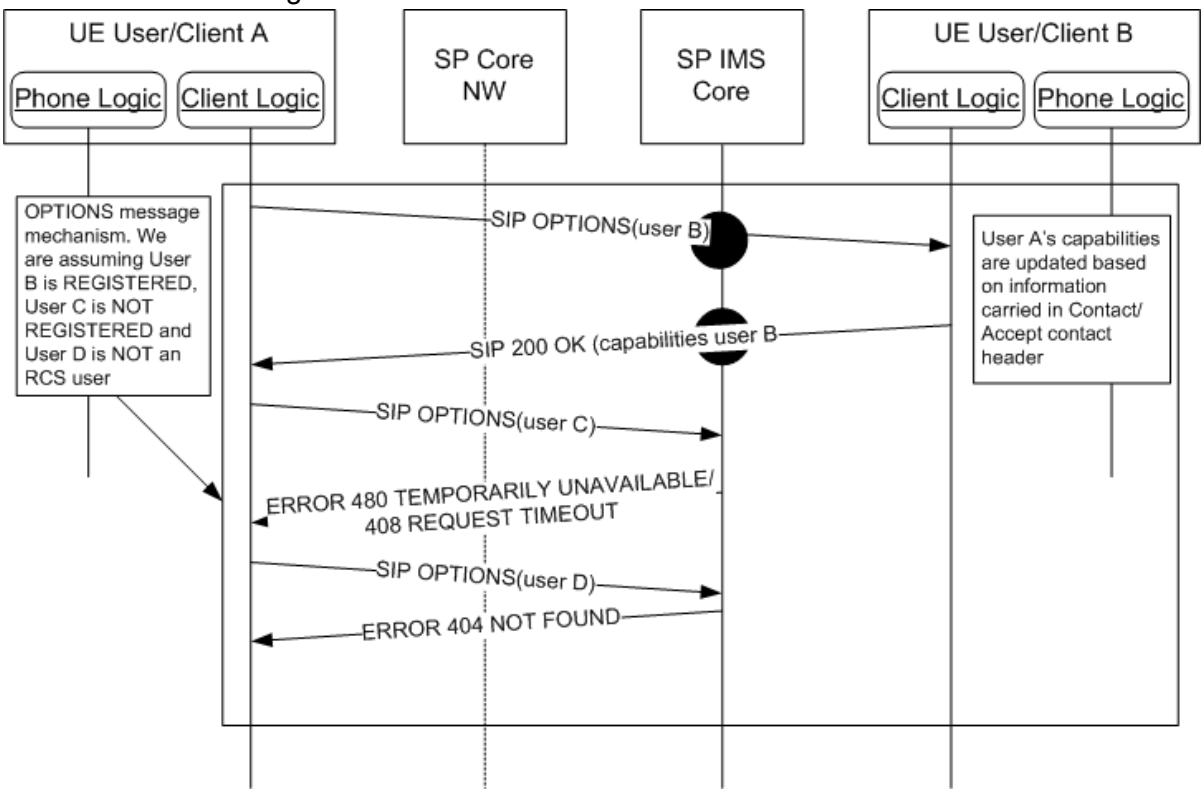


Figure 2: Capability Discovery via SIP OPTIONS message

2.6.1.1.1 Multidevice support: Options-AS

Ultimately, the choice of supporting multiple devices for a single user is decided by each Service Provider. The considerations contained in this section will only apply to those Service Providers willing to include RCS multidevice support in their networks.

In a multidevice scenario, when the user is registered to the IMS core with various devices using the same URI (that is the same implicit registration set) and nothing specific is done, the OPTIONS exchange will return incomplete information:

- The capabilities contained in the OPTIONS message refer only to the originating device (that is the originating user may be logged in with the same URI in several devices).
- The IMS core, depending on the configuration, either sends the OPTIONS message to the device that first registered to the IMS core or forks the OPTIONS to all of the registered devices. In any case, only the first response is passed back to the

requester, discarding the others. In other words, the capabilities returned in the OPTIONS response will be from only one of the user's devices.

The preferred implementation for handling the OPTIONS in a multidevice environment is left to the Service Provider's discretion. The only requirement is that it should not impact the terminal side (that is there will be no changes on the client side). A possible solution for extending the OPTIONS mechanism to a multidevice scenario is to include a custom AS implementing the following logic:

- A trigger will be setup in the IMS core to send all of the OPTIONS from an RCS user to the AS.
- The AS will fork the OPTIONS request to all of the RCS user's registered devices as specified in section 2.10.2.
- Once the responses from the different devices are received, the AS will aggregate all the capabilities from the replies of the different clients and send them back to the requester.
- Even if not all of the replies have been received in less than a configurable amount of time, the AS will return the aggregated information received so far.

NOTE: the recommendation is to set the value to optimise the User Experience (UX) on the terminal.

- Capabilities shall be aggregated to provide the response to an incoming SIP OPTIONS request. For outgoing requests, it is up to the Service Provider's policy to aggregate the capabilities.

NOTE: Similar procedures may at the service provider's discretion also be applied at originating side to aggregate the capabilities of all the user's devices in the OPTIONS request.

This is illustrated in Figure 3:

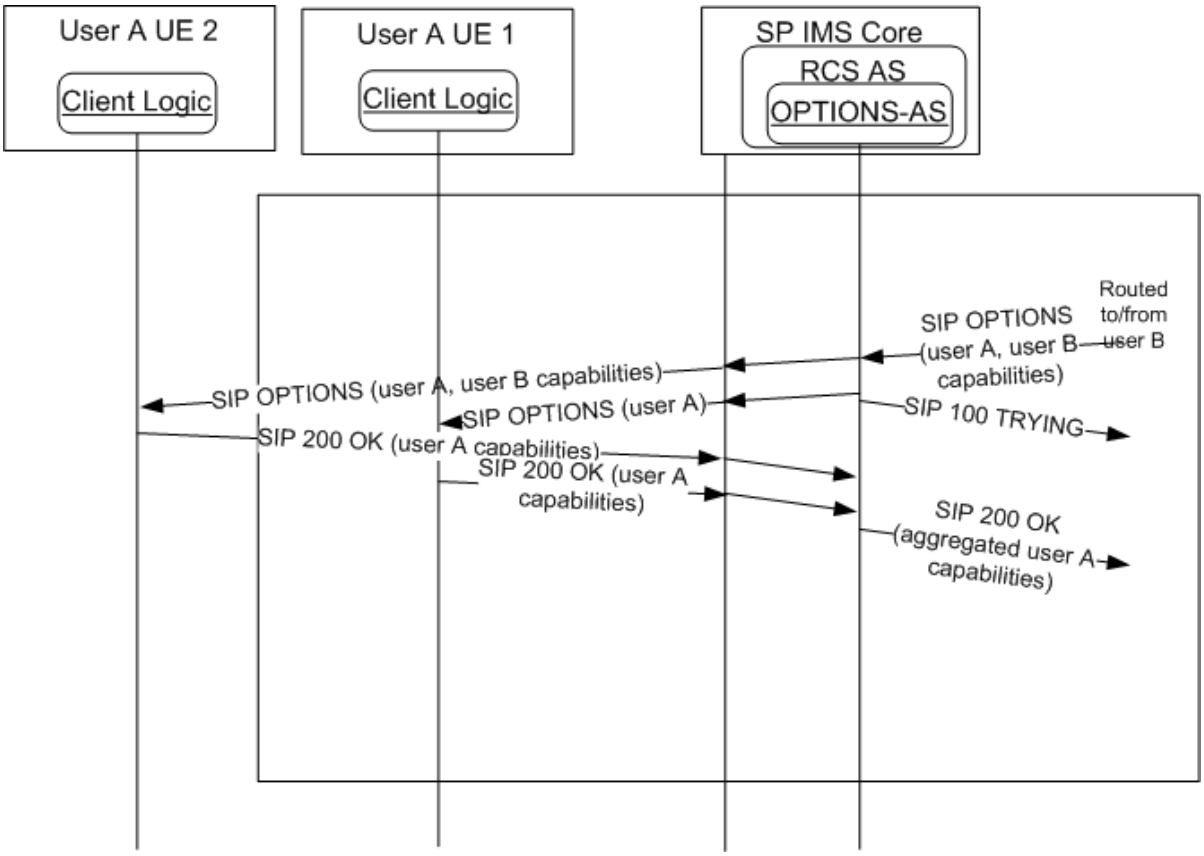


Figure 3: Options application server: Capability aggregation on SIP OPTIONS request

2.6.1.2 Capability Discovery via presence

2.6.1.2.1 General Overview

As an alternative to the SIP OPTIONS-based mechanism presented in the previous section, a Service Provider deploying a Presence Server may provide the Capability Discovery mechanism via presence. The service capabilities are then realised using the "*Service*" part of the Presence Data Model that is described in section 2.6.1.2.5.

2.6.1.2.2 Publication of the Service Capabilities

The capabilities are announced in a Presence document that is published by using the SIP PUBLISH method as defined in [Presence]. When the terminal is started, the client then sends a SIP PUBLISH request containing the capabilities (see section 2.6.1.2.5). This SIP PUBLISH request shall not include an *Expires* header field. Service capabilities publication through OMA Presence Enabler [Presence2.0_TS] or [Presence] must follow [PDE_14] rules.

The publication is maintained in the Presence Server by sending a refresh request before it expires.

If changes are required in the published capabilities (for example, due to the behaviour specified in section 2.6.1.3), a presence modify request is sent using the '*Sip-If-Match*' header according to [Presence]. When the client/device is switched off, it shall remove the published capabilities before unregistering according to the procedure defined in [RFC3903]

(i.e. by sending a SIP PUBLISH request without a body including the '*Sip-If-Match*' header and an *Expires* header set to 0).

2.6.1.2.3 Service Capabilities Retrieval

Service capabilities of an RCS user can be retrieved by another RCS user via a presence subscription issued by their client, providing the pertaining Presence Authorisation rules (see section 2.6.1.2.4) allow him to do so.

When using Presence as the enabler for Capability Discovery, RCS clients shall retrieve the service capability information of contacts by means of Anonymous Fetch operations (as described in section 7.1 of [PRESENCE]). This will result in a single NOTIFY request indicating the service capabilities of that contact. The contact shall be considered as an RCS user only if the response includes one of the service-IDs described in Table 10. This information shall then be cached in the client as described in section 2.6.2.

If an RLS-URI (Resource List Server URI, see Annex A section A.1.2.1) has been provisioned, a client shall use an Anonymous Fetch request using a request-contained list if the client has to query the capabilities of multiple users at once (e.g. during a poll). In this case, it shall do so according to section 5.2.1.2.2 of [Presence2.0_TS].

If only a single contact needs to be queried, an individual fetch shall be done instead even if an RLS-URI has been configured.

Both for an individual fetch and for one based on a list, the client shall add into the Contact header field of the Anonymous Fetch request the tags defined in section 2.6.1.3 that correspond to the relevant set of services supported by the client. Only the tags that are related to services on which the client wants to obtain information shall be considered relevant and should be included in the Contact header field (e.g. a request that is intended to obtain information on a contact's capabilities for messaging services should not include feature tags related to Call Composer).

NOTE1: The set of services that is considered relevant to be included in the request is left to client implementation given that it will be dependent on the specifics of the UI interaction that was the trigger for initiating the request.

NOTE2: The information on the relevant services might be used for routing decisions when the capabilities for the contact are split over different registrations (see [GSMA PRD-NG.102]) and would ensure a more consistent user experience in error scenarios.

2.6.1.2.3.1 General Processing Rules to Ensure Backwards Compatibility

To maintain enough flexibility and not to impose potentially sub-optimal technical choices on future RCS versions, the parsing of the capabilities in an RCS client should be sufficiently robust. First, the watcher should apply the processing rules defined in [Presence2.0_DDS] and if then there are still multiple elements the watcher shall follow the guidelines in the RCS presence parsing presented below:

- Unknown or unsupported elements and tuples could be present in the document. In that case, they should be ignored.

- Unknown service identifiers (Service-Id) could be present in the document. Tuples containing those should be ignored.
- Unknown service versions of known services could be present in the presence document. Tuples containing those should be ignored.
- The same service could occur multiple times in the presence document with different contact addresses. To cope with this case, the following behaviour shall be used for displaying and using the tuples:

    - If one of the tuples contains a contact address that corresponds to the presentity about which the presence document was received, all others shall be ignored.
    - Tuples that contain a contact (address) element which corresponds to another presentity (that is another contact in the contact-list of the user or another tel URI) shall be ignored.
    - Tuples containing contact elements with types of addresses that are not supported by the client for that service shall be ignored (for example messaging using an e-mail address while e-mail is not supported by the client).
    - If after applying the above rules, there are still multiple non-ignored tuples remaining for the service, all but the first shall be ignored.
    - If after applying the above rules, there is a non-ignored tuple remaining, the service behaviour shall be as follows

        - The capability to use the service for communication with the contact shall be announced to the user
        - If the remaining tuple contained no contact address or it matched the one of the presentity, the presentity's address will be used for setting up communication using that service
        - Otherwise the address contained in the contact element will be used for setting up the corresponding service

- The Watcher shall follow the procedures defined in section 6.2 "Default Watcher Processing" of [Presence2.0_DDS].

Regarding the use of the address provided in the contact, the communication addresses (contact) part of service tuples shall not be:

- Shown to the end-user, these addresses are handled locally by the terminal;
- Used to request presence subscription, an RCS client is NOT supposed to subscribe to the contact associated with a service capability tuple received in a presence document.

2.6.1.2.4 Authorisation for capabilities retrieval

To provide authorisation to retrieve the capabilities using an Anonymous Fetch request, an RCS Service Provider supporting the Capability Discovery using presence shall either provide a service provider policy on the presence server allowing anonymous subscriptions to retrieve the capabilities or set for every RCS subscriber a presence rules document in the presence XDMS providing such authorisation.

2.6.1.2.5 Service part of the presence Data Model

A service capability is provided according to the model described in Table 7:

Attribute	Specification	Comment
entity	[RFC3863]	The entity field should be populated with a tel URI provided that the device has received a tel URI in P-Associated-URI header of 200 OK response to REGISTER request.
Tuple: <presence> -> <tuple>	[RFC3863] and [Presence2.0_DDS]	According to the presence schema defined in the [Presence], services are presented with *tuple* elements.
Status <tuple> -> <status> -> <basic> -> Open	[RFC3863] and [Presence2.0_DDS]	Mandatory element in [RFC3863]. For every tuple element that is published the value 'open' shall be used. It does not have any particular meaning in RCS context.
Service-id <tuple> -> <service-description> -> <service-id>	[Presence2.0_DDS]	*Service-description* element identifies a service and is described by a *service-id* and *version*. *Service-id* element contains a string that identifies a single service. The Service-IDs that are used for the different services that are part of RCS are described in section 2.6.1.3.
Version <tuple> -> <service-description> -> <version>	[Presence2.0_DDS]	*Version* element contains the version number for the service, to identify different versions of the service (for example version number for specification number). The Version that are used for the different services that are part of RCS are described in section 2.6.1.3.
Media <tuple> -> <servcaps>	[RFC5196] and [Presence2.0_DDS]	Indicates the capabilities of the service. In RCS, this is only used to provide media capabilities for some specific services for which this is mentioned in section 2.6.1.3.

Attribute	Specification	Comment
Contact <tuple> -> <contact>	[RFC3863] and [Presence2.0_DDS]	Contact element contains Presentity's communication address for the service. Contact address can be for example a tel or SIP URI, depending on the service used. The use of the Contact element is optional (if used it has to be a global routable URI) since the watcher may use the URI stored in the address book when initiating communication with the presentity. RCS Presentities either do not insert any contact element or insert a contact element for which the address matches the one used for identifying itself in communication (see Section 2.5) NOTE1: According to [RFC3863], "tuples that contain a <*basic*> element SHOULD contain a <*contact*> address". Therefore, as a default- the <*contact*> element should be populated with a tel URI provided that: The device has received a tel URI in P-Associated-URI header of 200 OK response to REGISTER request. The service in question can utilise tel URIs.
Timestamp: <tuple> -> <timestamp>	[RFC3863] and [Presence2.0_DDS]	Timestamp when the presence information was published.

Table 7: Attributes of the Presence Service element

2.6.1.2.6 Handling of Permanent Presence and handling of Subscribe Response

Depending on the response received to the SIP SUBSCRIBE for presence information of a contact, the contact is considered to be:

- A non-RCS contact, if:
  - a SIP 404 Not Found error response is received when the contact is not an IMS user.
- An RCS contact if:
  - a SIP 200 OK response was received, followed by a SIP NOTIFY that included at least one RCS Service-ID element or that contact.
  - a SIP 200 OK response was received, followed by a SIP NOTIFY that only includes the "Capability Discovery via presence" tuple when the contact is an RCS contact that is not registered for RCS service.
- Inconclusive, if:
  - other non-2xx responses are received to the SIP SUBSCRIBE, or
  - a SIP 200 OK was received and the SIP NOTIFY did not include any of the RCS Service-IDs for that user (in single subscriptions, or in the RLMI body).

- a SIP 200 OK response was received, followed by a SIP NOTIFY that does not contain any RCS tuples for the contact.

In inconclusive cases, further SUBSCRIBE requests will be initiated and any existing RCS capabilities for the contact will be kept until a conclusive response above is received or the cached RCS capabilities expired.

A summary is captured in the tables below.

Response	Interpretation	Action
404	Conclusive (Non-RCS User)	Remove cached capabilities
Capabilities tuple in PIDF (status = "open")	Conclusive (RCS User)	Keep capabilities received
No RCS tuples	Inconclusive	Keep cached capabilities
Only contains Capability Discovery via presence tuple	Conclusive (RCS User)	Offline capabilities are available
non-2xx	Inconclusive	Keep cached capabilities
malformed/corrupt document NOTE: Includes: No Capabilities in PIDF (status = “closed”)	Inconclusive	Keep cached capabilities

Table 8: Individual Subscribe

Response	Interpretation	Action
No RCS tuples	Inconclusive	Keep cached capabilities
Only contains Capability discovery via presence tuple	Conclusive (RCS User)	Offline capabilities are available
Capabilities in PIDF (status = "open")	(Conclusive (RCS User)	Keep capabilities received
No PIDF and fullState = true	Inconclusive	Keep capabilities received
No PIDF and fullState = false		
non-2xx	Inconclusive	Keep capabilities received

Table 9: Individual Subscribe

2.6.1.2.7 Handling Offline Users

To avoid corner cases where RCS users are deemed as “non-RCS contact”, it is highly recommended that at least the RCS service Capability Discovery via presence is provisioned in the RCS user’s permanent presence.

NOTE 1: This can be done at the same time as the RCS service is provisioned for the user.

NOTE 2: Additional RCS services that are persisted in permanent presence data are subject to operator choices.

Terminating initial Filter Criteria (iFCs) shall be configured to route SUBSCRIBE requests to the correct Presence Server whenever the Request-URI identifies an RCS user, regardless of the IMS registration status of the presentity.

This ensures that the sender of the SUBSCRIBE will receive a 200 OK response and a NOTIFY from the Presence Server, which is enough to conclusively identify the presentity as an RCS user. Thus, there is no ambiguity when the B party is not IMS registered.

2.6.1.2.8 Optimizations for Presence-Based Capability Discovery

2.6.1.2.8.1 Full Address Book Scans (Capability Polling)

Capability Polling is optional and implementation specific and should be avoided unless the operator has a requirement for the first view of the address book to “identify the RCS capable contacts”. Operators who want to implement Capability Polling should consider the following challenges:

1. Capability Polling traffic load is sensitive to address book size. Operators who want the first view of the address book to “identify the RCS capable contacts” should be aware of the potential cost.
2. List SUBSCRIBEs give rise to large NOTIFY message bodies and bursts of Back End SUBSCRIBEs, which may make it difficult to control peak transaction load in the network.

2.6.1.2.8.2 Duration of SUBSCRIBEs

Long-lived SUBSCRIBEs have a major effect on scaling. For service capabilities:

3. Individual SUBSCRIBEs should include an Expires header with value 0.
4. List SUBSCRIBEs: If list SUBSCRIBEs are utilised, they should include an Expires header with a small non-zero value (e.g., 30-60 seconds). This allows for latency associated with Back End SUBSCRIBEs but limits the scaling impact on the Presence/Resource List Server

2.6.1.3 Service/capability indicators

The RCS capabilities represent the list of services that an RCS user/client can access at a certain point in time. The capabilities depend on four factors:

1. User Service Provider provisioning status: A Service Provider may choose to limit service to customers depending on subscription status (e.g. chat and file share, but not video).
2. The terminal hardware (HW): A terminal with limited HW (i.e. no capability to process video) may not be able to access all the RCS Services.
3. The terminal status: Even if a terminal HW supports all the services, it could be that the device status introduces a limitation (e.g. receiving files is not possible when the file storage is full).

4. Connectivity status: Some services may require a certain level of network Quality of Service (QoS). For example, streaming video over a 2G General Packet Radio Service (GPRS) is not possible with the used enablers.

In addition to the factors presented above and as presented in Annex A section A.1, it is possible for a Service Provider to select which services are available for a particular user. Therefore, the previous considerations shall only be taken into account assuming that the relevant RCS services are enabled via configuration.

Table 10 lists the feature tags and Service IDs that are used for indicating that a specific RCS service is available:

RCS service		Tag
Chat	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcse.im";+g.3gpp.icsi-ref="urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.session" The RCS Client shall assume that Chat is available if either of the two tags mentioned is present.
	Service ID	Service-id: org.openmobilealliance:IM-session Version: 1.0 Contact address type: tel / SIP URI Or Service-id: org.openmobilealliance:ChatSession Version: 2.0 Contact address type: tel / SIP URI
File Transfer	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp"
	Service ID	Service-id: org.openmobilealliance:File-Transfer-HTTP Version: 1.0 Contact address type: tel / SIP URI
File Transfer via SMS	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.ftsms"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.ftsms Version: 1.0 Contact address type: tel / SIP URI
Geolocation PUSH	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.geopush"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.geopush Version: 1.0 Contact address type: tel/ SIP URI
Geolocation PUSH via SMS	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.geosms"

RCS service		Tag
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.geosms Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Reply Message	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.reply"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.reply Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Reaction	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.reaction"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.reaction Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Multiple Reaction	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.multireaction"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.multireaction Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Custom Reaction	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.customreaction"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.customreaction Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Edit Message	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.edit"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.edit Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Message Recall	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.recall"
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.recall Version: 1.0 Contact address type: tel / SIP URI
Extended Messaging Delete Message	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended.delete"

RCS service		Tag
	Service ID	Service-id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.msg.extended.delete Version: 1.0 Contact address type: tel / SIP URI
E2EE Version	Tag	+g.gsma.rcs.mls.mls-version="v1"
	Service ID	Service-id: org.gsma.rcs.mls Version: 1.0 Contact address type: tel / SIP URI
E2EE Home KDS	Tag	+g.gsma.rcs.mls.mls-kds="<home kds identifier>"
	Service ID	Service-id: org.gsma.rcs.mls-kds.<home kds identifier> Version: 1.0 Contact address type: tel / SIP URI
Media Share Codecs	Tag	+g.gsma.rcs.mscodecs="#=<profile_version>[,<codec_name>]*" The format of the feature tag value is defined as `mscodecs-value` in section 3.2.5.6.
	Service ID	Not Applicable (see NOTE)
RCS Video Chat	Tag	+g.gsma.rcs.videochat
	Service ID	Service-id: org.gsma.rcs.videochat Version: 1.0 Contact address type: tel / SIP URI
Rich Text Message	Tag	+g.gsma.rcs.msg.richtext="vN"
	Service ID	Service-id: org.gsma.rcs.msg.richtext Version: N.0 Contact address type: tel / SIP URI
User Profile Sharing	Tag	+g.gsma.rcs.userprofilesharing
	Service ID	Service-id: org.gsma.rcs.userprofilesharing Version: N.0 Contact address type: tel / SIP URI

Table 10: Complete SIP OPTIONS tag and Presence Service ID usage for RCS

NOTE: E2EE does not work with Presence.

NOTE: Media Share Codecs capability does not work with Presence.

NOTE: Unless specified in other sections (e.g. section 2.4.4), the new tags defined in this section are defined for use in SIP OPTIONS exchanges only and the standard tags defined in the supporting PRDs and endorsement documents shall be used to identify the services in the rest of relevant SIP transactions. It should also be noted that in some cases, the tags employed in the SIP OPTIONS exchange match the standard tags.

NOTE: The existing base feature-tag defined in earlier versions of this document i.e urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.msg.extended was previously used as the base feature representing support for all Extended Messaging sub-features. This identifier is deprecated.

Finally, when several IARI tag values or several ICSI tag values are included in a SIP OPTIONS request, consistently with [RFC3840], IARI tag values or ICSI tag values shall be concatenated using commas as shown in the example below:

```
+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcse.im,urn%3Aurn-7%3A3gpp-application.ims.iari.rcse.ft"
```

Table 11: IARI tag concatenation format example

2.6.1.3.1 Future extensions to the mechanism

In addition to the aforementioned services and to allow:

- A Service Provider (or group of Service Providers) to deploy additional services which can benefit from the RCS discovery mechanism, an additional tag and Service ID format is defined.

NOTE: A Service Provider may deploy Extensions as a third-party.

For this purpose, following Capability Identifiers have been identified:

RCS service		Tag
Service Provider specific service	Tag	+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.mnc<mnc>.mcc<mcc>.<service name>"
	Service-ID (based on IARI)	Service-Id: org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.mnc<mnc>.mcc<mcc>.<service name> Version: Service Provider choice
	Service-ID (based on OMA scheme)	Service-Id: org.openmobilealliance:<RCS service name>.mnc<mnc>.mcc<mcc>.<service extension> Version: Service Provider choice

Table 12: Feature Tag and Presence service tuple proposal for future lines of work

When using the OMA scheme for the Service ID, Service extension patterns including “mnc<mnc>.mcc<mcc>” may be registered with OMNA, if a service provider wishes to reserve the values in order to avoid any future collisions with new services (extensions, or new OMA services).

For Service Provider Extensions, the service name is decided by the each Service Provider. The only requirement for a Service Provider following this approach is to include these tags in the relevant interoperability agreements with other Service Providers.

Examples of service extensions for Service Providers:

- OPTIONS tags:
  - +g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.mnc001.mcc214.serviceA"
  - +g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari rcs.mnc680.mcc310.serviceB"
- Service-id for extension(s)
  - to Chat with the OMA scheme:
    org.openmobilealliance:ChatSession.mnc072.mcc01
    OR
    org.openmobilealliance:ChatSession.mnc072.mcc01.myGCFlavor1 AND
    org.openmobilealliance:ChatSession.mnc072.mcc01.myGCFlavor2
  - Using IARI:
    org.3gpp.urn:urn-7:3gpp-application.ims.iari.rcs.mnc01.mcc072.sfgroupchatMyFlavor

2.6.1.3.2 void

2.6.1.4 Interworking between the different mechanisms

2.6.1.4.1 Coexistence between the discovery mechanisms via network interworking

When Service Providers use presence as the discovery mechanism, interoperability is achieved between such a Service Provider and those Service Providers who have selected SIP OPTIONS as the default discovery mechanism by bi-directional network based interworking.

Specific network interworking function requirements are contingent upon the service discovery modes and policies of each service provider. At the Service Provider’s discretion, an interworking function can be implemented in the network to:

- Answer incoming SIP OPTIONS requests based on the Presence Server information (Figure 4).
- Convert SIP ANONYMOUS SUBSCRIBE requests into SIP OPTIONS requests (Figure 5).

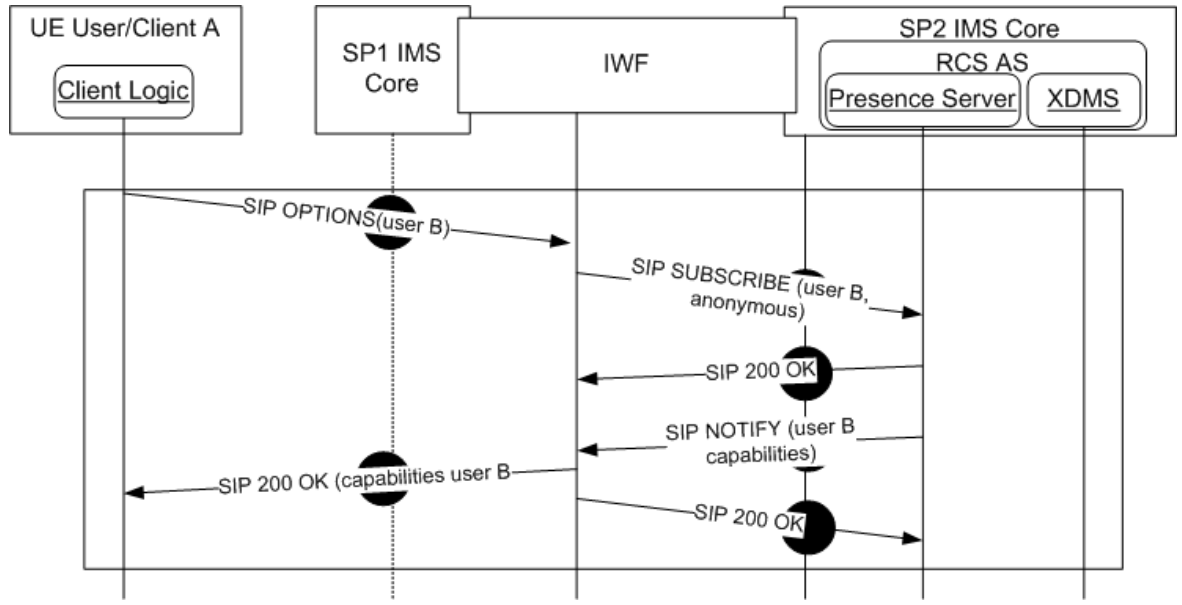


Figure 4: Capability Discovery interworking via network: Options request

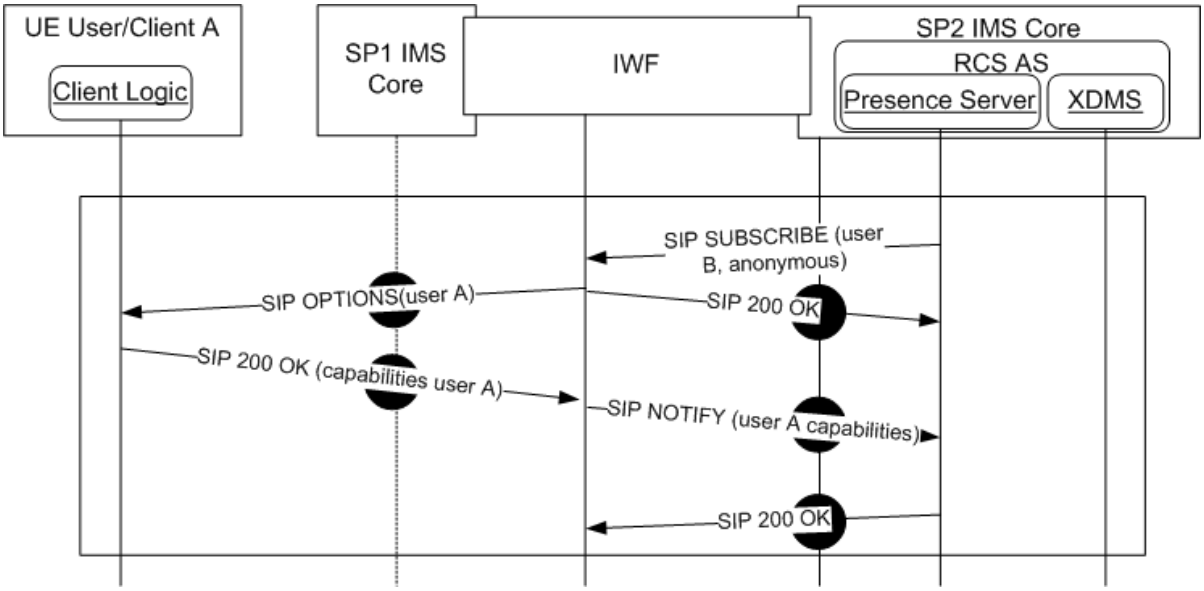


Figure 5: Capability Discovery interworking via network: Presence request

NOTE: Figure 4 and Figure 5 do not specify whether the IWF is deployed:

- In the Originating IMS network or
- In the Terminating IMS network or
- In the Inter-Network region.

All of the above are valid architectural options. NNI impact is not uniform and is a function of the architecture selected. While the details surrounding the specific architecture and functionality of an IWF are left to the Service Provider, it is recommended that impact at the UNI should be minimal and as transparent as possible.

The successful deployment of network IWF capabilities must provide an environment where all RCS devices exchange capabilities information without requiring additional functionality or logic at the client (i.e. no UNI impact).

The following additional guidelines are provided regarding the implementation of an IWF function:

- If either Service Provider has a heterogeneous network from a Capability Discovery mode perspective, this must be factored into the IWF architecture.
- The Service Provider implementing an IWF must consider policy aspects of the functionality. This includes any decisions to filter or transform service capabilities across the IWF.
    - Domain/Service Provider based policies; i.e. specific services are configured to be exposed based on the destination domain.
    - Service level policies: specific services, including Service Provider proprietary or other specialised services that may be filtered from exposure to any external domains.
    - User based policy; including privacy or other subscriber level policies.

2.6.1.4.2 Interworking with networks that disable Capability Discovery

When the Service Provider disables the Capability Discovery mechanism the network shall at the NNI still accept SIP OPTIONS requests for the capabilities of its users and return a SIP 200 OK including any of the agreed interworking service tags which are supported by User B as described in Table 10. If User B is currently not registered or is not a RCS user, the network shall respond in accordance with section 2.6.1.1 Capability Discovery process through SIP OPTIONS message.

2.6.2 Handling of Capabilities

The enablers discussed in section 2.6.1 provide indications on the following

- capability information (i.e. whether a contact supports a service) and
- service availability information (i.e. whether a contact is currently likely in conditions that allow successful establishment of the service)

The Presence based mechanism provides this information using the presence service descriptions as described in section 2.6.1.2 which would indicate just service availability from which capability information can be derived (i.e. if a service is currently available for use with a contact, that contact is assumed to support the service). For SIP OPTIONS, a similar approach shall be followed, but there the use of the “automata” tag allows to indicate that a contact supports a service (i.e. capability information) without indicating service availability as described in section 2.6.1.1.

Capability information and service availability information obtained through the Capability Discovery enablers will be cached on the device. Within this cache, different expiry policies are applicable for capability information and service availability. The configuration parameters controlling this are described in section A.1.9.

When encountering an event related to a contact that does not correspond to the cached capability information of that contact, a client shall refresh those cached capabilities by initiating a Capability Discovery and Service Availability request. This shall be done when:

- A 1-to-1 SIP request to an RCS enabled contact results in a SIP 404 response or
- A Geolocation Push, File Transfer request or a Rich Text Message is received from a contact for which the corresponding capability wasn't part of the cache or
- A SIP OPTIONS request carrying at least one RCS feature tag is received from a non-RCS contact.

NOTE: Reception of a Chat or Standalone Message from a non-RCS contact is not included because that could be the result of interworking.

2.6.2.1 Service Provider Controlled Service Capabilities Handling

The following items can be configured subject to the Service Provider's policies (see section A.1.9):

1. An expiry of the capabilities for a specific RCS or non-RCS contact through respectively the CAPABILITY INFO EXPIRY and the NON RCS CAPABILITY INFO EXPIRY client configuration parameters.

NOTE: Chatbots are considered to be RCS contacts.

2. An expiry of the service availability information for a specific RCS contact through the SERVICE AVAILABILITY INFO EXPIRY client configuration parameter.
3. The Contacts with a telephone number-based address considered for the Capability Discovery depending on their prefix through the CAPABILITY DISCOVERY ALLOWED PREFIXES client configuration parameter.

This will allow to control among others the maximum time before a client will discover that one of the contacts is now RCS capable

2.7 RCS protocols

The following table summarises the list of protocols employed by RCS clients. It must be noted that the choice among the options presented will not affect Service Provider interoperability:

Protocol name	Description	Transport layer	Secure transport layer/protocol
Session initiation protocol (SIP)	Client-IMS core signalling protocol	User Datagram Protocol (UDP) over IP or Transmission Control Protocol (TCP) over IP	SIP over Transport Layer Security (TLS) or IP Security (IPsec)
Message Session Relay Protocol (MSRP)	Chat messages, media (pictures) and file exchange protocol	TCP/IP	MSRP over TLS
Hyper Text Transfer Protocol (HTTP)	HTTP configuration mechanism File Transfer Access to Message Store Server Chatbot Information query Service Provider Chatbot directory query Retrieval of the lists of Chatbots requiring specific management	TCP/IP	Hyper Text Transfer Protocol Secure (HTTPS)

Table 13: RCS protocols

According to [RFC3261], RCS clients shall support both SIP/UDP (User Datagram Protocol) and SIP/TCP (Transmission Control Protocol). The choice of whether both are used or only TCP is used to transport the signalling data belongs to each Service Provider and is controlled by the configuration parameters "*psSignalling*", "*psSignallingRoaming*" and "*wifiSignalling*" in [GSMA PRD-RCC.15] section 2.2.2.2.

NOTE: The "psSignallingRoaming" parameter is defined as a temporary workaround to address PS roaming related issues identified in live deployments.

Regarding the impact of Network Address Translation (NAT) traversal in the different protocols involved in RCS, the following considerations shall be taken into account:

- Regarding the SIP protocol:
  - Carriage Return Line Feed (CRLF) keep-alive [RFC6223] support
    - shall be used when SIP/TCP or SIP/TLS is used by the RCS client and
    - May be used for SIP/UDP or when relying on the Push Notification Mechanism (section 2.15).

    Section C.1 describes how both client and server could initiate the sending of the keep alives.

  - Simple Traversal of UDP through NATs (STUN) keep-alive [RFC6223] should be used when SIP/UDP is used by the RCS client as it allows network capacity optimization.
  - An RCS client using SIP/UDP:
    - Shall support symmetric signalling (That is the IP and port combination used to send SIP messages is the same as the one used to receive SIP messages).
    - Shall perform TCP switchover for large SIP messages.

- For handling Message Session Relay Protocol (MSRP) sessions, the RCS MSRP endpoints shall support:
  - [RFC6135]: “The Alternative Connection Model for the Message Session Relay Protocol (MSRP)”
  - The mechanisms described in section 2.7.2 regarding session matching for MSRP.
  - For NAT traversal for MSRP, keep alives (i.e. empty MSRP packets) are not necessary. If the TCP connection is torn down because of inactivity, the MSRP session is torn down, and a new SIP INVITE request to set up a new MSRP session is sent the next time a message is to be sent.

- For HTTP, no specific mechanisms are mandated in the current specification to support NAT traversal.

The support of Transport Layer Security (TLS) based or IP Security (IPsec) based protocols to secure the signalling and TLS based for the MSRP protocol to secure media exchanges shall be used both for access over 3GPP networks (i.e. cellular access) and for those scenarios where the data is carried over a network outside the Service Provider domain (i.e. Wi-Fi access). For more information on access security, see section 2.12.

NOTE: To ensure interoperability of all RCS capable devices across different Service Provider networks, the list of preferred options for the transport and security for the signalling and media protocols is included in the configuration parameters as defined in [GSMA PRD-RCC.15], section 2.2.2.2. Consequently, a Service Provider provides this information as part of the first-time or re-configuration scenarios described in section 2.3.

2.7.1 void

2.7.2 MSRP session matching

For all services using MSRP, an RCS client shall set up MSRP sessions as per [GSMA PRD-RCC.11].

2.7.3 SIP Issues

1. An RCS client should use a random originating SIP signalling port of the range 1025-65535. If the selected port is not available, the next port number shall be used for this session.

2. An RCS client shall build its SIP contact address to be unique. A recommended way to do so is to use a hashed value of the +sip.instance tag as user part of the URI of the contact address.
3. For an incoming request, an RCS client should verify that the Request-URI matches the URI of its registered contact address. If not, the Request-URI shall be considered an unexpected address and the request shall be rejected as per [RFC3261] section 8.2.2.1.
4. The content in the body of SIP request or response could be compressed with the indication of Content-Encoding header as described in [RFC3261].
   When the size of the SIP request/response content exceeds the configured parameter CONTENT COMPRESS SIZE defined in Section A.1.3 (e.g. Chatbot messages with big JSON payload, Group Chat invitation/1-to-Many Messaging with big MIME resource-list body and CPM Group Session Participant Information via SIP Notify), and SIP Content-Compression is supported by the client, the SIP content (except SDP) should be compressed with the Content-Encoding header indication; an example is shown as below:
   Content-Encoding: gzip
   The receiver of the SIP request/response should handle the compressed content accordingly.

2.7.4 CPIM Body Compression

To reduce the size of large CPIM messages, clients and servers may compress the CPIM payload.

2.7.4.1 CPIM compression feature tag

The CPIM compression feature tag indicates an entity's support for specific compression algorithms for CPIM payloads. The tag is encoded as a feature parameter in accordance with [RFC 3840]. The feature tag name shall be set to `g.gsma.rcs.cpim-compression`. The value shall be a quoted string containing a comma-separated list of supported compression algorithms (e.g., "gzip")., with a following ABNF format:

```
cpim-compression-tag = "+g.gsma.rcs.cpim-compression" EQUAL DQUOTE
algorithm *(COMMA algorithm) DQUOTE
algorithm = token
```

2.7.4.2 Compression procedure

1. The client should use MINIMUM CPIM BODY SIZE FOR COMPRESSION client configuration parameter (defined in section A.1.3) to determine whether to attempt compression of a given payload.
2. The sender may compress the CPIM payload and shall only send the compressed payload if its resulting size is smaller than the uncompressed payload.
3. When embedding a compressed payload, the sender shall format it as binary data within the CPIM body and shall include the following MIME headers:

   a) `Content-Encoding`: Indicates the compression algorithm used (defined in section C.7.1).
   b) `Original-Content-Length`: Indicates the exact length of the uncompressed payload in bytes (defined in section C.7.2).
   c) `Content-Length`: Indicates the length of the compressed payload.

Example compressed text/plain message:

```
NS: imdn urn:ietf:params:imdn
imdn.Message-ID: 12345
To: tel:+67890
From: tel:+12345

Content-Type: text/plain
Content-Encoding: gzip
Content-Length: $(length of binary compressed Hello World!)
Original-Content-Length: 12

${binary compressed Hello World!}
```

2.7.4.3 Decompression procedure

Upon receiving a compressed CPIM message over SIP or MSRP, the RCS client or SPN receiving the CPIM message shall evaluate the headers and payload:

- If the Content-Encoding header specifies an algorithm that the decompressing party does not support, it shall reject the message with a SIP or MSRP 415 Unsupported Media Type response including a Warning header field with the warning text set to '415 Unsupported Content Coding'.

During the decompression, the decompressing party shall:

1. Verify that `Original-Content-Length` does not exceed the maximum Content-Length size the party is willing to accept,
2. Not exceed `Original-Content-Length` during the decompression. The decompressing party shall use a streaming decompression and terminate the decompression early if size limits are breached.
3. Verify that after the successful decompression, the `Original-Content-Length` matches the decompressed payload size.

If the decompressing party fails to decompress the content, or any of the steps above fail, the decompressing party shall reject the message with a SIP or MSRP 415 Unsupported Media Type response including a Warning header field with the warning text set to '415 Decompression Failed'.

2.8 RCS and Access Technologies

2.8.1 RCS and Cellular/EPC-integrated Wi-Fi Access

2.8.1.1 Access used by RCS in relation to Multimedia Telephony

To select the access used, a device providing RCS services and Multimedia Telephony services shall support

- the procedures in [GSMA PRD-NG.114] when connected to a 5GC
- the procedures described in [GSMA PRD-NG.102] otherwise.

These procedures are dependent on the RCS VOLTE SINGLE REGISTRATION configuration parameter defined in section A.1.6.

2.8.1.2 Radio Capabilities

2.8.1.2.1 LTE

Radio bearers, UE Discontinuous Reception (DRX) and Discontinuous Transmission (DTX) modes of operation, Radio Link Control (RLC) configurations, and Guaranteed Bitrate (GBR) and Non-Guaranteed Bitrate (NGBR) services and GBR Monitoring Function for devices enabled for Multimedia Telephony are all as specified in [GSMA PRD-IR.92] when connected to an EPC. None of this is applicable to other devices and access networks other than Long Term Evolution (LTE).

2.8.1.2.2 NR

Radio bearers, UE DRX and DTX modes of operation, RLC configurations and GBR and NGBR services and GBR Monitoring Function are all as specified in [GSMA PRD-NG.114] for devices enabled for Multimedia Telephony when connected to a 5GC. None of this is applicable to other devices and access networks other than NR.

2.8.1.3 Bearer aspects

For all IMS traffic, the following applies for an RCS device configured for Multimedia Telephony:

- For radio bearer management when connected to an EPC see section 4.3 of [GSMA PRD-IR.92].
- For radio bearer management when connected to a 5GC see section 4.3 of [GSMA PRD-NG.114].

For all RCS IMS traffic, the following applies:

- For a device enabled for Multimedia Telephony:
  - 5QI 8 and 9 shall be supported so that either may be used for MSRP traffic.
  - QCI (QoS class identifier) 8 and 9 shall be supported so that either may be used for MSRP traffic.
- For a device using the IMS APN for RCS (see section 2.8.1.4):
  - QCI (QoS class identifier) 8 and 9 shall be supported so that either may be used for MSRP traffic.
  - 5QI 8 and 9 shall be supported so that either may be used for MSRP traffic.
- For other devices: no requirements.

2.8.1.4 APN and roaming considerations

General technical guidelines on how roaming is handled for the RCS services shall follow [GSMA PRD-IR.65].

Guidance given for RCS and access technologies as documented in section 2.8 are applicable also in the roaming scenarios. Specific roaming considerations for the different RCS device types (as specified in section 2.2):

- All services on a primary device currently supporting multimedia telephony over a 5GC, shall follow the general rules as per [GSMA PRD-NG.113], APN usage as per [GSMA PRD-NG.114].
- All services on a primary device currently supporting Multimedia Telephony over an EPC, shall follow the general rules as per [GSMA PRD-IR.88], APN usage as per [GSMA PRD-NG.102].
- All services on a primary device enabled for VoWiFi shall follow the general rules as per [GSMA PRD-IR.61], APN usage as per [GSMA PRD-NG.102].
- Other devices: no specific requirements.

The APN to be used to access the RCS services depends on the capacity of the device and the network to support an IMS APN as per [GSMA PRD-IR.88], on the device configuration and on the client type (see section 2.2):

NOTE1: This section only covers the APN behaviour for RCS services. These settings shall not be taken into account for the usage of other APNs by non-RCS services.

- When the device is enabled for Multimedia Telephony and connected to a 5GC, an embedded client shall use the APN indicated in [GSMA PRD-NG.114].
- When the device is enabled for Multimedia Telephony and not connected to a 5GC, an embedded client shall use the APN indicated in [GSMA PRD-NG.102].
- For an embedded client on other devices, the behaviour shall depend on the setting of the RCS VOLTE SINGLE REGISTRATION configuration parameter (see section A.1.6):

NOTE2: The RCS VOLTE SINGLE REGISTRATION configuration parameter is used in this case even if there is no VoLTE registration from the device because the required behaviour is similar.

  - The IMS APN shall be used to access the RCS services when the device is configured through the RCS VOLTE SINGLE REGISTRATION configuration parameter defined in section A.1.6 to use a single registration approach.
  - The Home Operator Services (HOS) APN shall be used to access the RCS services when the device is configured through the RCS VOLTE SINGLE REGISTRATION configuration parameter defined in section A.1.6 to use a dual registration approach.

NOTE3: When roaming on a network where the device cannot access a local IMS APN (e.g. no VoLTE roaming agreement is in place), a client configured to use the IMS APN will, by using the IMS APN, automatically access RCS through the home network's IMS APN with the telephony service using the Circuit Switched network. If no IMS roaming agreement is in place, the visited network would degrade any requested QoS based on local configuration, as per normal procedures in the MME for any APN as

specified in [PRD-IR.88]. The operator could optionally have a QoS data roaming agreement to ensure that QCI=5 for the IMS APN is allowed in the visited network and that either QCI=8 or QCI=9 for MSRP will be allowed on the IMS APN. If the appropriate QCI bearer cannot be set up for MSRP when on LTE access, MSRP traffic will be on the default QCI=5 bearer unless prevented by the terminal or network.

To support traffic from non-RCS applications (e.g. generic internet access) in this case the device and network shall support other APNs to be active simultaneously.

For an embedded client, the APN to use for HTTP and XCAP shall be the HOS APN as defined in [GSMA PRD-IR.88]. The HOS APN can be configured to be, or by default be, the device's generic data access APN (i.e. the internet APN). The network should use the HOS APN when providing the RCS client with its configuration, in order to prevent unwanted data charging for client provisioning traffic.

NOTE4: By internet APN, we understand the default APN configured by the Service Provider to provide Internet connectivity on the device.

It is out of scope of this specification how the HOS APN is configured on the device.

- Downloadable clients shall use the internet APN.

2.8.1.5 Data Off

Users can switch cellular data usage off locally on their device. To allow the operator to offer RCS services to their customers even in these use cases, the data off switch shall have an operator configurable impact on the device connectivity. The service provider should ensure a good service experience if IP service usage is allowed although the data switch was set to off by the end user. The procedures for cellular data off are applicable for primary devices when RCS is not using the internet APN (see section 2.8.1.4).

For services using IMS protocols, the implementation of cellular data off shall be based on 3GPP PS Data Off defined in [3GPP TS 24.229-rel15] with the following additional clarifications and requirements.

The cellular data off exempt is configured for RCS services via the configuration parameters RCS MESSAGING DATA OFF and FILE TRANSFER DATA OFF as defined in section A.1.14.1.

When registering in the network, the client shall include in the contact header of any SIP REGISTER request the "data off" feature tag defined in Table 3 indicating the data-off status in accordance with section B.3.1.0 of [3GPP TS 24.229-rel15].

If cellular data off is active and if for a given SIP registration (i.e. a single registration Multimedia Telephony, SMS over IP and RCS services or a registration for RCS services only) no service is configured as a cellular data off exempt service, then the client shall not register in IMS.

If cellular data off is active and there are one or more services configured as cellular data off exempt services for a given SIP registration, then the client shall include in the Contact header of the SIP REGISTER only the service feature tags defined in Table 3 of the services being configured as cellular data off exempt services.

If cellular data off is active and there are one or more services configured as cellular data off exempt services for which Capability Discovery is required, then the client

- shall advertise capability information for services, including the ones not being cellular data off exempt services, and
- shall not advertise the service availability of services being not cellular data off exempt services.

For an overview of RCS services with capability information and service availability refer to section 3.3.1 of [GSMA PRD-RCC.71].

If cellular data off is active and there is no service configured as cellular data off exempt service for which Capability Discovery is required, then the client shall disable Capability Discovery.

For services using non-IMS protocols having no data connection at the time of disabling, no additional actions are required by the client. If disabled by Data off configuration the service will not be available at the time of invocation.

For services using non-IMS protocols and having a data connection active at the time of disabling, the ongoing session or transaction shall be terminated.

The procedures for the handling of cellular data off for the configuration procedures defined in section 2.3 are defined in [GSMA PRD-RCC.14].

2.8.1.6 void

2.8.2 Other access networks

In addition to the cellular PS access networks described in section 2.8.1, the RCS framework and services can be used over any IP access over which the Service Provider's IMS core and application servers can be reached, provided that it offers sufficient bandwidth and an acceptable latency. Section 2.4 details when such networks can be used and how to use RCS through them. Section 2.6.1.3 provides a guideline for which services can be used when connected through different types of access networks including broadband access.

2.9 End User Confirmation Requests

RCS clients shall support the End User Confirmation Request enabler as defined in [GSMA PRD-RCC.15] section 3.1.

2.10 Multidevice support

2.10.1 Overview

As shown in section 2.8.2, the use of a broadband access client leads to the possibility of the user having multiple devices that share the same (public) identity, a MSISDN for

instance. This multidevice environment allows a user to answer a call or respond to a message from a device/client that suits their purpose. The behaviour for voice/video calls is specified in [GSMA PRD-NG.110].

The general communication behaviour in this environment is that when the recipient has multiple devices/clients in use and a message is received every recipient's device will alert. The recipient may then respond to the message from any of their devices; whichever device is the best for the current situation.

To achieve this, an RCS client shall send a SIP 603 Decline response to the invite request when an RCS User explicitly declines a session invitation for a SIP session based service. According to [3GPP TS 24.229] and [RFC3261] both such a rejection and an acceptance will result in a SIP CANCEL request sent by the Serving Call Session Control Function (S-CSCF) to the other devices of the user that have not yet accepted nor rejected the invitation. In both cases, the requests may carry a Reason header field as specified in [RFC3326] that is populated with the proper SIP response code values (as per [3GPP TS 24.229]), in this case either the *cause=200* or *cause=603* values.

If the user device has accepted the INVITE with a 200 OK, then the S-CSCF should set the Reason header field with *SIP* protocol and the protocol-cause set to *200* along with an optional protocol-text (e.g. *SIP;cause=200;text="Call completed elsewhere"*).

In case one device has sent a 603 Decline then the S-CSCF should set the *cause=603* along with an optional protocol-text (e.g. *SIP;cause=603;text="Decline"*), in either SIP CANCEL and/or SIP BYE, towards the remaining user devices.

When a client receives a SIP CANCEL request containing a Reason Header field with the protocol set to "SIP" and the protocol-cause set to 200, a client may for example use this information to indicate to the user that the session was accepted on another device (rather than as for example a missed call).

As a fallback for legacy services where this general communication behaviour cannot be realised, a message might be directed to a certain device.

2.10.2 Addressing of individual clients

Any Application Server (e.g. a Messaging Server or OPTIONS AS) can address an individual RCS client using information received with the third party registration (using the sip.instance feature tag).

The sip.instance feature tag and value shall be used as the device identifier. The client shall include the sip.instance feature tag in the Contact header with the same instance-id value in any non-REGISTER request and responses that it sends where including a Contact header is possible, to allow identification by an application server.

2.11 Interconnect principles and guidelines

The Service Provider's IMS NNI shall follow the provisions in [GSMA PRD-IR.65] sections 3, 4, 5 and 6.

The Service Provider's RCS NNI shall follow the provisions in [GSMA PRD-IR.90]. The implementation could be any of the three connectivity options for RCS NNI defined in [GSMA PRD-IR.90].

2.12 Access Security

2.12.1 IMS Security

2.12.1.1 Access Signalling Security Methods

Several SIP signalling access security and authentication methods are specified in [3GPP TS 33.203] and [3GPP TS 24.229] for access to the IMS core and IMS based services such as RCS. The applicability and choice of method is highly dependent on the RCS client and access type (e.g. trusted or untrusted) including what is supported or required by the IMS core.

2.12.1.1.1 IMS AKA with IPsec

IMS AKA with IPsec is the preferred long-term approach in IMS for access signalling security from a cellular PS network. Such access requires the IMS client device to possess an AKA based credential (e.g. Universal SIM (USIM)/IP Multimedia Services SIM (ISIM)) and support the "*ipsec-3gpp*" procedures specified in [3GPP TS 33.203] and [3GPP TS 24.229].

2.12.1.1.2 SIP Digest Authentication and TLS

SIP Digest is a username and password challenge based authentication (based on HTTP Digest) which is suited for broadband access to IMS or for RCS clients which do not possess AKA based credentials (e.g. xSIM) or do not support IMS AKA based IPsec. SIP Digest is widely implemented in Internet Engineering Task Force (IETF) based SIP clients and is often deployed with TLS. Support for SIP Digest with and without TLS is specified in [3GPP TS 33.203] and [3GPP TS 24.229] for access to IMS from "non-3gpp" defined access networks (e.g. broadband/fixed access networks).

In RCS, a client is enabled for SIP Digest authentication by the Service Provider via the client configuration parameter IMS Mode Authentication Type defined in section 2.2.1.2 of [GSMA PRD-RCC.15] and in this case, shall use TLS always.

When an RCS client is enabled for SIP Digest authentication, then the client shall use the configuration parameters defined in section 2.1.2.2 of [GSMA PRD-RCC.15] and section A.1.6.1 of the current document to populate the Authorization header field parameters for authentication in accordance the rules and procedures of section 5.1.1 of [3GPP TS 24.229] for IMS registrations using SIP Digest as follows:

- The private user identity for the "username" Authorization header field parameter shall be taken from the value of the configuration parameter Realm User Name defined in section 2.2.1.2 of [GSMA PRD-RCC.15].
- The domain name of the home network for the "realm" Authorization header field parameter shall be taken from the value of the configuration parameter Realm defined in section 2.2.1.2 of [GSMA PRD-RCC.15].

- The SIP URI of the domain name of the home network for the "uri" Authorization header field directive shall be taken from the value of the configuration parameter Home_network_domain_name defined in section A.1.6.1.
- For the initial registration using SIP Digest the client shall populate the value of the Authorization header field "response" as defined in section 5.1.1.2.3 of [3GPP TS 24.229] using

    - the username taken from the value of the configuration parameter Realm User Name defined in section 2.2.1.2 of [GSMA PRD-RCC.15],
    - the password taken from the value of the configuration parameter Realm User Password defined in section 2.2.1.2 of [GSMA PRD-RCC.15],
    - the realm taken from the value of the configuration parameter Realm defined in section 2.2.1.2 of [GSMA PRD-RCC.15],
    - the URI taken from the value of the configuration parameter Home_network_domain_name defined in section A.1.6.1.

The IMS registration flow for SIP digest authentication is shown in Figure 6. In this example flow, the RCS client is connected to the IMS core over a Wi-Fi internet broadband connection.

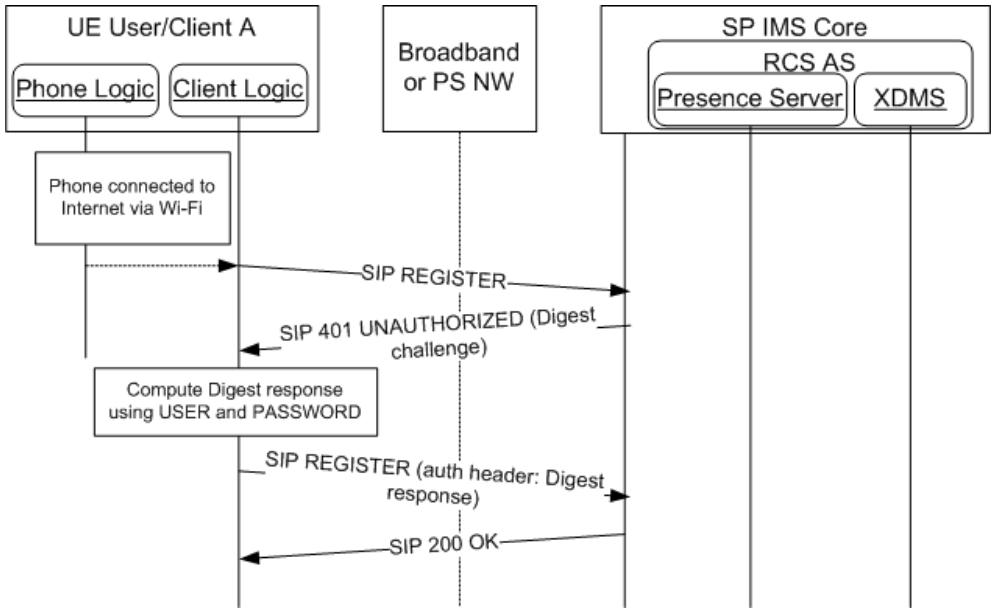


Figure 6: Registration with SIP Digest Authentication

If digest authentication fails two times with a SIP 401 Unauthorised response, the client shall not attempt further registration attempts, but rather consider the current configuration as invalid and force a reconfiguration using the procedures in chapter 2.3 the next time the handset is rebooted.

If the client is enabled for the SIP Digest authentication type, then the use of SIP/TLS is configured by the Service Provider via the access network specific transport signalling configuration parameters defined in section 2.2.1.2 of [GSMA PRD-RCC.15].

SIP Digest with TLS shall be used for access both from trusted and from untrusted access networks (including WLAN with no encryption). TLS provides per message authentication, integrity protection and encryption for SIP signalling. TLS with server side certificates also provides authentication of the IMS core to the RCS client.

NOTE: This requires the client to possess a root or intermediate certificate of a Certificate Authority (CA) that is in the certificate signing chain for the IMS core's (e.g. P-CSCF) TLS certificate.

Given that an RCS client shall use SIP/TLS when using SIP Digest, it should use the SIP TLS port obtained through P-CSCF discovery procedures (e.g. through DNS SRV records [Service records]) or configuration. However, if RCS client is not able to determine a SIP TLS port through these means, it shall use the default SIP port for TLS as specified in [RFC3261].

The RCS client enabled to use SIP Digest should first use the SIP security agreement (sec-agree) [RFC3329] as specified in [3GPP TS 24.229] to first negotiate the use of TLS with its SIP Proxy (P-CSCF). Alternatively, an RCS client may first try to establish a TLS session with the SIP proxy (P-CSCF) before sending an initial SIP Register message which does not include sec-agree for TLS. However, with this approach the S-CSCF may challenge subsequent non-Register messages with a 407 Proxy Authentication Required unless configured to trust SIP Digest without signalling security indicated or if the P-CSCF is able to provide this indication despite not using sec-agree.

NOTE: In both cases the SIP proxy (P-CSCF) authenticates to the RCS client using a TLS server certificate from a public certification authority.

If the RCS client is enabled and RCS Service Provider supports Push Notification Mechanism as specified in section 2.15, then the RCS Service Provider may optionally challenge the client to provide additional authorization using following procedure:

- In addition to the procedures described in [3GPP TS 24.229] section 5.4.1.2.1B, the RCS Service Provider may challenge by addition of a WWW-Authenticate header in SIP 401 Unauthorised response with a PrivateToken scheme including a token-key parameter as specified in [RFC9577] section 2.1.2.
- The client may respond to the challenge by following [3GPP TS 24.229] section 5.1.1.5.6 additionally responding to the Authorization header with a PrivateToken scheme in the subsequent SIP REGISTER request.
- The validation and attestation of the PrivateToken defined in [RFC9577] are beyond the scope of this document.
- On receiving an invalid PrivateToken the RCS Service Provider may choose the following:
    - A SIP 401 Unauthorised response with a PrivateToken scheme in a WWW-Authenticate header to re-challenge the client.
    - A SIP 403 Forbidden response including a Warning header field including the warning text set to "206 Spam Sender" to prevent the client accessing RCS services.

- A SIP 503 Internal Server Error response with a Retry-After header value to inform the client to retry the request as specified.

The RCS client and the RCS Service Provider may exchange additional information using the ‘extensions’ header in the WWW-Authenticate and Authorization header.

Optionally, If the RCS Service Provider receives an error from a Push Service Provider; specifically, a 410 error, the RCS Service Provider may reject the next registration attempt, with a SIP 403 Forbidden response containing a Warning header field with a warn-code value of 410 and a warn-text with the value "Push URI Expired". The RCS client shall cease further registration attempts then and consider the current Push URI as invalid. The RCS client shall then generate a new Push URI and force a reconfiguration using the procedures defined in section 2.3. This reconfiguration shall include the relevant forms for uploading a new Push URI to the HTTP Auto-Configuration Server.

NOTE: The generation of a Push URI by an RCS Client is out of scope of this document.

2.12.1.1.3 Signed SIP Digest Authentication

Signed SIP Digest authentication is a challenge-based authentication that extends the SIP Digest authentication by introducing the use of a client-side certificate.

In RCS, a client is enabled for Signed SIP Digest authentication by the Service Provider via the client configuration parameter IMS Mode Authentication Type defined in section 2.2.1.2 of [GSMA PRD-RCC.15].

The configuration parameters for Signed SIP Digest are identical to the ones described in 2.12.1.1.2. The format of the Authorization header and the values of its fields are also identical to those described in 2.12.1.1.2. The key difference is the addition of the “opaque” field as defined in [RFC3261]. The value of the “opaque” field is as follows:

- The “opaque” field shall contain the value of the “response” field, signed using the private key associated with the public key provided during the provisioning process described in section 2.12 of [GSMA PRD-RCC.14].

The IMS registration flow with Signed SIP digest is shown in Figure 7.

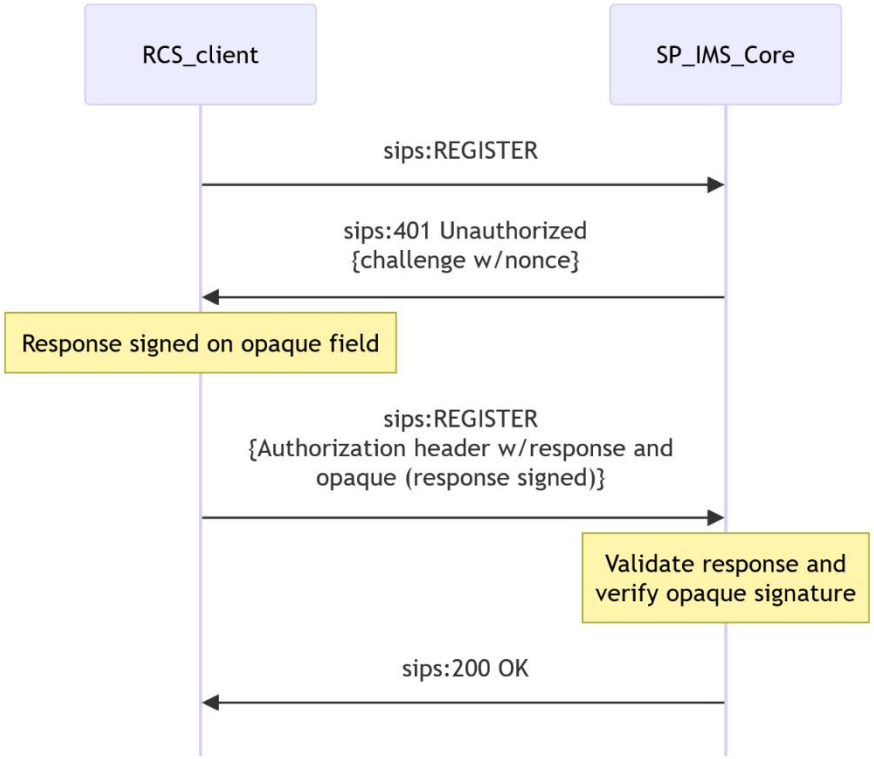


Figure 7: Registration with Signed SIP Digest Authentication

If Signed Digest authentication fails two times consecutively with a SIP 401 Unauthorised response, the client shall cease further registration attempts, consider the current configuration invalid, and force a reconfiguration using the procedures in section 2.3 the next time the handset is rebooted.

If Signed SIP Digest fails with a SIP 403 Forbidden response containing a Warning header with the value “Invalid Opaque Value”, the client shall cease further registration attempts, consider the current certificate invalid, and force a reconfiguration using the procedures in section 2.3. This reconfiguration should include uploading a new certificate to the HTTP Auto-Configuration Server.

NOTE: The procedure for obtaining the new certificate is outside the scope of this specification.

2.12.1.1.4 Access Signalling Security Profiles for RCS

As there are several considerations which access signalling security method should be used for access to RCS services, the following table defines authentication and access signalling security mechanisms as per RCS device and access type.

Device	Access	Applicable Security Methods	Applicability and Suitability
Mobile client not configured for Multimedia Telephony	Cellular PS Access	SIP Digest with TLS or IMS AKA with IPsec	IMS AKA with IPSec may be used when supported by both device and the network. SIP Digest with TLS is used depending on (pre-)configuration
	access over EPC-integrated Wi-Fi	SIP Digest with TLS or IMS AKA with IPsec	IMS AKA with IPSec may be used when supported by both device and the network. SIP Digest with TLS is used in cases when pre-configured
	Non-cellular broadband (Wi-Fi) access	SIP Digest with TLS or IMS AKA with IPsec	SIP Digest with TLS is used in cases when pre-configured or when the mobile device does not support IMS AKA for WLAN access
Multimedia Telephony enabled mobile client	Cellular PS Access	SIP Digest with TLS or IMS AKA with IPsec (see NOTE1) as specified in [GSMA PRD-NG.102] and [GSMA PRD-NG.114]	AKA credentials stored securely in a UICC such as an xSIM. The method used depends on the relation to the registration for Multimedia Telephony (see [GSMA PRD-NG.102] and [GSMA PRD-NG.114])
	EPC-integrated Wi-Fi	SIP Digest with TLS or IMS AKA with IPsec (see NOTE1) as specified in [GSMA PRD-NG.102] and [GSMA PRD-NG.114]	AKA credentials stored securely in a UICC such as an xSIM. The method used depends on the relation to the registration for Multimedia Telephony (see [GSMA PRD-NG.102] and [GSMA PRD-NG.114])
	Non-cellular broadband (Wi-Fi) access	SIP Digest with TLS or IMS AKA with IPsec (see NOTE1).	SIP Digest with TLS is used in cases when pre-configured or when the mobile device does not support IMS AKA for WLAN access.
Broadband Access Enabled		SIP Digest with TLS	SIP Digest with TLS is used for mobile devices which do not support IMS AKA for WLAN access.

Table 14: Access Signalling Security Profiles for RCS

NOTE1: The use of IPsec requires UDP encapsulation of IPsec for NAT traversal.

For RCS devices which can access the IMS core from both mobile and broadband/fixed networks (e.g. Wi-Fi) a separate access signalling security method and corresponding

authentication credential may be required. If the security mechanism is not pre-configured as per section 2.2.1.1.2 and 2.2.2.1.3 of [GSMA PRD-RCC.15], the RCS device negotiates the set of security mechanisms using the SIP security agreement [RFC3329] as specified for IMS in [3GPP TS 33.203] and [3GPP TS 24.229]. If the client is pre-configured with a specific access signalling security mechanism, the client uses the signalling corresponding to this security method in the initial registration procedure, and the IMS core determines (based on signalling) which mechanism is being used/requested and then determines (based on security policy) if the access signalling security method is allowed.

NOTE2: The RCS device shall support a configuration option for each of these profiles (where applicable).

2.12.1.2 Access Media Security

2.12.1.2.1 void

2.12.1.2.2 MSRP

MSRP is used in many RCS services which involve the exchange of images, files and instant messages (e.g. session based). Similar to RTP, MSRP is established through SDP exchanges in SIP signalling and it relies heavily on the security provided in signalling. The use of cryptographically strong random values appended to MSRP URIs exchanged within SDP provides binding between the SIP and MSRP sessions and any identities exchanged within SIP.

For RCS, the TLS mode as specified in [RFC4975] shall be used both for access over 3GPP networks (i.e. cellular access) and when MSRP is transported over an unsecure network (e.g. Wi-Fi). Consequently, a client configuration parameter to enable Message Session Relay Protocol over Transport Layer Security (MSRPoTLS) is specified in [GSMA PRD-RCC.15], and whether it is used or not can be configured differently for Wi-Fi access and cellular access.

The RCS client shall use self-signed TLS certificates to produce fingerprints (e.g. secure hash) of the certificate which are exchanged during the SDP negotiation associated with the invitation and MSRP establishment procedure. The certificate fingerprint used for MSRP shall follow the same fingerprint mechanism specified in [RFC4572]. This binding of the certificate fingerprint to SIP signalling relies on the underlying security and trust provided by SIP signalling (e.g. IPsec, SIPoTLS (SIP over TLS), etc.). As a consequence, it is assumed that MSRPoTLS connections shall only happen when combined with the use of encrypted SIP signalling.

When using MSRPoTLS, and with the following two objectives allow compliance with legal interception procedures, the TLS authentication shall be based on self-signed certificates and the MSRP encrypted connection shall be terminated in an element of the Service Provider network providing service to that UE. Mutual authentication shall be applied as defined in [RFC4572].

Since the alternative connection model for MSRP shall be supported as specified in [RFC6135] (see section 2.7) the network will in some cases take the active role, and in some cases take the passive role, in the establishment of the TCP connection. Each peer

(UE and network) shall take the same role (active or passive) in TLS as it took in TCP, so if the network has taken the passive role in TCP, it will also act as TLS server, as specified in [RFC6135]. When TLS is used, both endpoints shall exchange self-signed TLS certificates and fingerprints, as specified in [RFC4572].

In RCS, and in accordance with [RFC4975], all UEs are mandated to support MSRPoTLS as defined in [3GPP TS 24.229-rel12] with certificate fingerprints as defined in [3GPP TS 33.328]. For terminating sessions, the P-CSCF (IMS ALG) decides based on local policy whether to apply MSRPoTLS towards the UE. The P-CSCF (IMS-ALG) shall invoke procedures related to MSRPoTLS for all access (i.e. both Wi-Fi and cellular access).

2.12.2 OpenID Connect

2.12.2.1 Overview

OpenID Connect provides a generic solution for user authentication independent from the type of device and the access network. It encompasses authentication, authorization and consent management for primary and secondary devices.

The procedures of OpenID Connect shall be supported by the client for all RCS services and cross service functionality using HTTP as access technology, i.e. for

- the client configuration based on the procedures defined in [GSMA PRD-RCC.14]
- File Transfer as defined in section 3.2.5
- the access to the Common Message Store as defined in section 2.12.3

2.12.2.2 OpenID Connect Authentication Flow

The RCS client shall support the procedures of the OpenID Connect Authentication Flow using the Authorization Code Flow as defined in [OpenID Connect] as follows.

If the client has sent a HTTP request to an endpoint in the network providing one of the RCS services or cross service functionality above and the network endpoint returns a HTTP 302 Found response, then the client shall use the value of the HTTP Location header to connect to the Open ID Connect authorization endpoint. This request forms the OpenID Connect authentication request.

On reception of the OpenID Connect authentication request the OpenID Connect authorization endpoint performs the procedures for user authentication, authorization and consent.

If the procedures for authentication, authorization and consent succeed or fail, then the OpenID Connect authorization endpoint returns a HTTP 302 Found response. The client shall use the value of the HTTP Location header to connect to the network endpoint. This request forms the OpenID Connect authentication response.

On reception of the OpenID Connect authentication response the network endpoint performs the procedures as defined for the RCS service or cross service functionality. In result of the processing, the network endpoint returns a success or failure response to the client as defined for the respective RCS service or cross service functionality.

The network endpoint and the OpenID Connect authorization endpoint shall preserve the values of the client's initial HTTP request during the OpenID Connect authentication request. The path and query components of the URIs used for the OpenID Connect authentication request procedure are opaque for the client.

The RCS client shall support for the OpenID Connect authentication flow the procedures for HTTP state management defined in [RFC6265]. This allows an OpenID Connect authentication endpoint to return in HTTP responses a Set-Cookie header. The client shall apply the parsing and storage procedures of the Set-Cookie header as defined [RFC6265]. It shall send the cookie header in HTTP requests to the OpenID Connect authentication endpoint respecting the cookie attributes in the Set-Cookie header in accordance with [RFC6265].

2.12.2.3 Authentication, authorisation and user consent methods for OpenID Connect

The RCS client shall support the methods for user authentication, authorisation and user consent for OpenID Connect as defined for the Service Provider Device Configuration defined in [GSMA PRD-RCC.14].

In addition, if the client supports the procedures for authentication using a bootstrapped security association as defined in [3GPP TS 24.109], then the OpenID Authentication endpoint shall be able to invoke the authentication procedure accordingly.

2.12.3 Common Message Store Authentication and Security

The RCS client shall support the authentication and security mechanisms described in [CPM-MSGSTOR-REST] for access to the Common Message Store.

Authentication to the Message Store using RESTful API calls shall use one of the following methods:

1. Authentication using a bootstrapped security association as defined in section 4.1.2
2. HTTP Digest authentication as defined in section 4.1.4
3. HTTP Basic authentication as defined in section 4.1.5
4. OpenID Connect based authentication as defined in section 2.12.2 and 4.1.3

2.13 Emergency Services

2.13.1 General

In some markets, regulatory requirements are emerging for IMS Multimedia Emergency Services. UEs and the network in required markets must support the 3GPP Release 11 IMS Emergency Services as specified in [3GPP TS 24.229-rel11], [3GPP TS 23.167], Chapter 6 and Annex H, and 3GPP Release 11 emergency procedures specified in [3GPP TS 24.301].

Please note [GSMA PRD-IR.92] in section 5.2 and [GSMA PRD-IR.51] in section 5.3 specify Emergency Services support.

2.13.2 RCS Service Feature List

Emergency Services support is provided in the following RCS Service Feature:

- 1-to-1 Chat

2.14 CPIM header extension support

RCS service definitions make use of the CPIM header extensibility mechanism defined in section 3.4 of [RFC3862] beyond the header namespace defined in section 6.1 of [RFC5438]. The CPIM header extension support mechanism ensures interoperability with messaging clients not supporting these extensions.

2.14.1 CPIM header extension support feature tag

The CPIM header extension support feature tag is the indication of an entity to support the procedures of CPIM header extensibility defined in this section.

The indication of the CPIM header extension support is provided by a feature parameter as defined in section 9 of [RFC3840]. The feature parameter is encoded as an "other-tags" feature tag in accordance with the definitions of section 9 of [RFC3840].

The feature tag name shall be set to "g.gsma.rcs.cpimext". The media feature tag shall have no value.

Security considerations for this media feature tag are discussed in section 11.1 of [RFC3840].

The indication of the CPIM header extension support is represented as follows:

+g.gsma.rcs.cpimext

2.14.2 Procedures in the client

A client supporting CPIM header extensions shall advertise its support by means of the feature tag defined in section 2.14.1 in the Contact header of

- SIP REGISTER requests and
- SIP INVITE requests and 200 OK responses for
  - Chat,
  - Chatbot sessions,
  - Group Chat and
  - Large Message Mode sessions

A client supporting CPIM header extensions shall, on reception of a CPIM message, check the value of the CPIM "NS" headers contained in the message.

If

- the URI value and the associated namespace of a given CPIM "NS" header is known to the client and
- a header name containing the associated name prefix is known to the client,

then the client shall process the CPIM header in accordance with the definitions of the extension header.

If

- the URI value and the associated namespace of a given CPIM "NS" header is known to the client and
- a header name containing the associated name prefix is not known to the client,

then the client shall ignore the CPIM header.

If the URI value and the associated namespace of a given CPIM "NS" header is unknown to the client, then the client shall ignore all CPIM headers containing the associated name prefix.

2.14.3 Procedures in the Messaging Server

2.14.3.1 Session Initiation

An originating Messaging Server supporting the media plane handling of CPIM header extensions defined in section 2.14.3.2 shall advertise this by including the feature tag defined in section 2.14.1 in the Contact header field of all SIP INVITE requests for a Chat, Group Chat or Standalone Message session.

A terminating Messaging Server supporting the media plane handling of CPIM header extensions defined in section 2.14.3.2 shall advertise this by including the feature tag defined in section 2.14.1 in the Contact header field of the 200 OK response to a SIP INVITE for a Chat, Group Chat or Standalone Message session.

A Controlling Function supporting the media plane handling of CPIM header extensions defined in section 2.14.3.2 shall advertise this by including the feature tag defined in section 2.14.1 in the Contact header of the SIP INVITE for Group Chat or Standalone Message session and in the 200 OK response to a SIP INVITE for a Group Chat or Standalone Message session.

2.14.3.2 Media Plane Handling

The Messaging Server sending a CPIM message shall detect whether CPIM header extensions are supported for the Chat, Group Chat or Standalone Messaging session. CPIM header extensions are supported for a session, if the Messaging Server received the CPIM header extension support feature tag defined in section 2.14.1 in the Contact header of the SIP INVITE request or 200 OK response to establish the Chat, Group Chat or Standalone Messaging session.

If for the session the CPIM header extension is not supported and if the Messaging Server needs to send a CPIM message in such a session, then the Messaging Server shall parse the CPIM "NS" headers contained in the message.

If a given CPIM "NS" header contains a URI value different from the URI value defined in section 6.1 of [RFC5438] and there is no other compatibility rule defined for the namespace

or extension header in the corresponding service definitions, then when forwarding the CPIM Message the Messaging Server

- shall remove the "NS" header, and
- shall remove all CPIM headers containing the associated name prefix.

NOTE: an example of such other compatibility rules is provided in section 3.6.8.7.

If all CPIM "NS" headers have been processed, then the Messaging Server shall continue processing with the resulting CPIM message as defined for the corresponding RCS service.

2.15 Push Notification Mechanism

2.15.1 RCS client enabled for Push Notification Mechanism

If the RCS client is enabled for the Push Notification Mechanism, the client shall follow the procedures defined in section 2.14 of [GSMA PRD-RCC.14], sections 2.12.1.1.2 and 2.15.3, and use the client configuration parameter as defined in section A.1.5 provided in the configuration document.

2.15.2 Procedures in the Messaging Server

2.15.2.1 Session Termination for Incoming Communication

For a terminating SIP request, the Messaging Server shall support store and forward functionality defined in section 3.2.2, 3.2.3 and 3.2.4. The Messaging Server shall forward the request to P-CSCF to enable delivery of the SIP request. If the P-CSCF attempts to deliver the SIP request to the RCS client in Push Controlled Idle State as defined in section 2.4.8, then the RCS Service Provider Network shall notify the client of the incoming communication via a Push Notification.

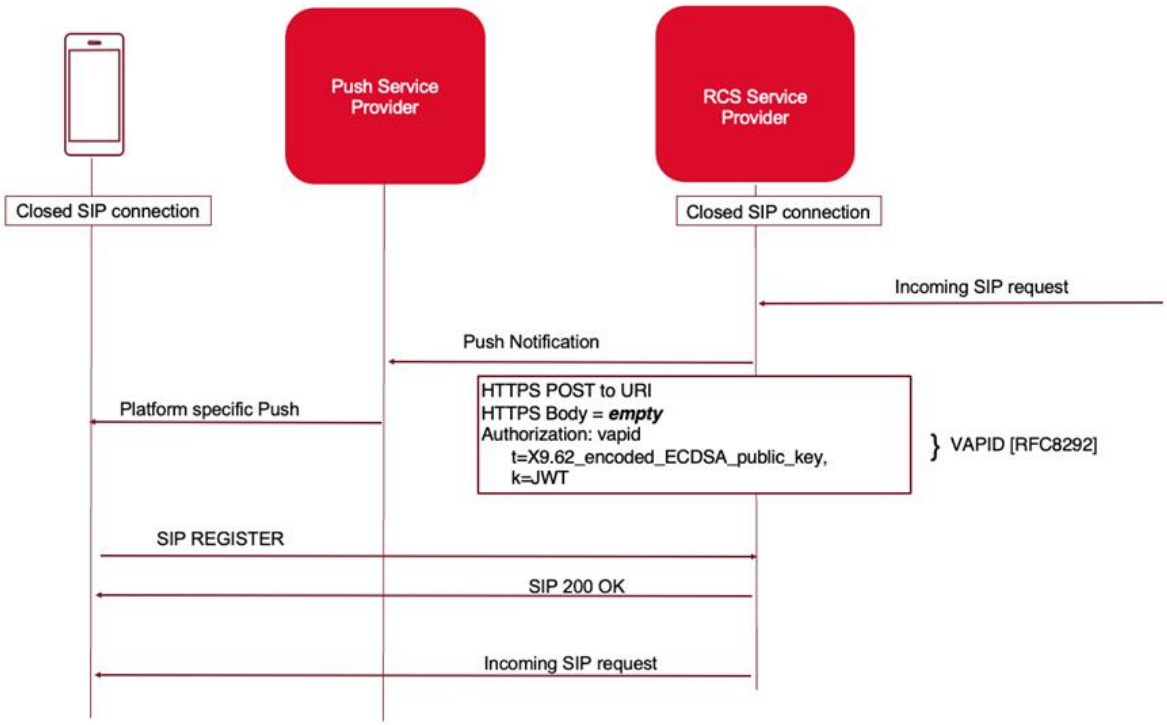


Figure 8: Push Notification Mechanism

2.15.2.2 RCS Service Provider sending Push Notification

If the P-CSCF receives an incoming SIP request and the RCS client is in the Push Controlled Idle State, then the RCS Service Provider shall follow the procedures defined in section 2.15.2.2.1 to send a Push Notification.

Additionally, if the RCS Service Provider sends a nonce header field parameter (as defined in [RFC7616] and [RFC8760]) in the HTTP body of the Push Notification to assist in the client in the SIP REGISTER, then the RCS Service Provider shall follow the procedures defined in section 2.15.2.2.2 to send a Push Notification. If there is content in HTTP body, then the RCS Service Provider shall follow procedures defined in section 2.15.2.2.1.

NOTE: The exact mechanism used to deliver the Push Notification by the Push Service Provider to the RCS client is vendor dependent.

2.15.2.2.1 Push Notification without content in HTTP body

If the RCS Service Provider is to transmit a Push Notification without content in HTTP body, then the RCS Service Provider shall send Push Notification using HTTP POST request to the URI received in the parameter Push URI as defined in [GSMA PRD-RCC.14] section 2.14.

The RCS Service Provider shall utilise the mechanisms specified in [RFC8030] section 5. and add an authorization header as defined in [RFC8292] and [GSMA PRD-RCC.14] section 2.14.

The parameter values of Push Message Urgency and TTL headers defined in [RFC8030] section 5.2 and 5.3 are left to implementation and are out of scope of this document.

If the Push Service Provider responds with a HTTP 2xx response code, there is no further action required on the RCS Service Provider.

NOTE: The RCS Service Provider is expected to work only with Push Service Providers that support deferred delivery

In case the Push Service Provider responds with non-2xx responses the following shall apply:

- If the Push Service Provider responds with HTTP error codes 429, 500, 503 with a Retry-After header, the RCS Service Provider shall honour the timer and retry the delivery attempt.

NOTE: The RCS Service Provider might choose to implement a backoff algorithm and set a maximum interval, instead of retrying indefinitely. If no value of the “Retry-After” header is received from the Push Service Provider, then the recommended value of minimum of 20 seconds could be used and the RCS Service Provider can retry up to a maximum of 60 mins.

2.15.2.2.2 Push Notification with nonce in HTTP Body

If the Push Notification contains a nonce header field parameter in HTTP body, then the RCS Service Provider, in addition to procedures described in section 2.15.2.2.1, shall encrypt the content as described in [RFC8291], using the public key of the RCS client as specified in [GSMA PRD-RCC.14] section 2.14.

On receiving the content, the client shall decrypt and extract the nonce. The client shall proceed to use this nonce in the subsequent SIP REGISTER and follow the procedures in section 2.4.

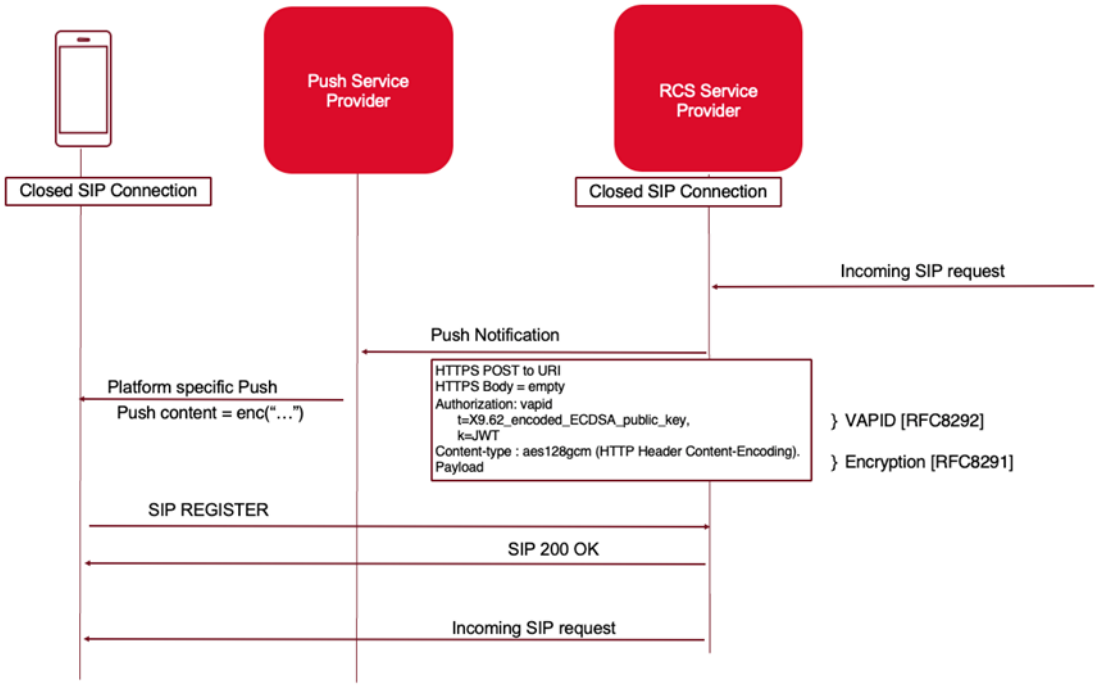


Figure 9: Push Notifications with content in HTTP body

2.15.3 Client behaviour on receiving Push Notification

If the RCS client receives a Push Notification from RCS Service Provider and the client is in Push Controlled Idle State, then the client shall send a SIP REGISTER as defined in section 2.4.

2.15.4 Capability Discovery for Push Notification

If the RCS client is enabled for Push Notification and is in Push Controlled Idle State as defined in section 2.4.8, then the RCS Service Provider may respond to the Capability Discovery request on behalf of the client.

The client and RCS Service Provider shall apply the mechanism described in sections 2.15.4.1 and may apply mechanisms described in 2.15.4.2.

2.15.4.1 Server behaviour for Capability Discovery

The RCS Service Provider, supporting Push Notification Mechanism, shall provide caching the capabilities of the RCS client, to support procedures defined in section 2.6.1.1.1 and 2.6.1.2.7.

NOTE: For example, the RCS Service Provider, might choose to store the contact with feature/media-tags in a local cache. If the information in cache is active and has not expired and client is in Push Controlled Idle state, then the RCS Service Provider might respond on behalf of the target client as described in section 2.6.1.1.

NOTE: if the cache is expired, the RCS Service Provider sends a Push Notification to the contact to refresh cached capabilities following the SIP REGISTER.

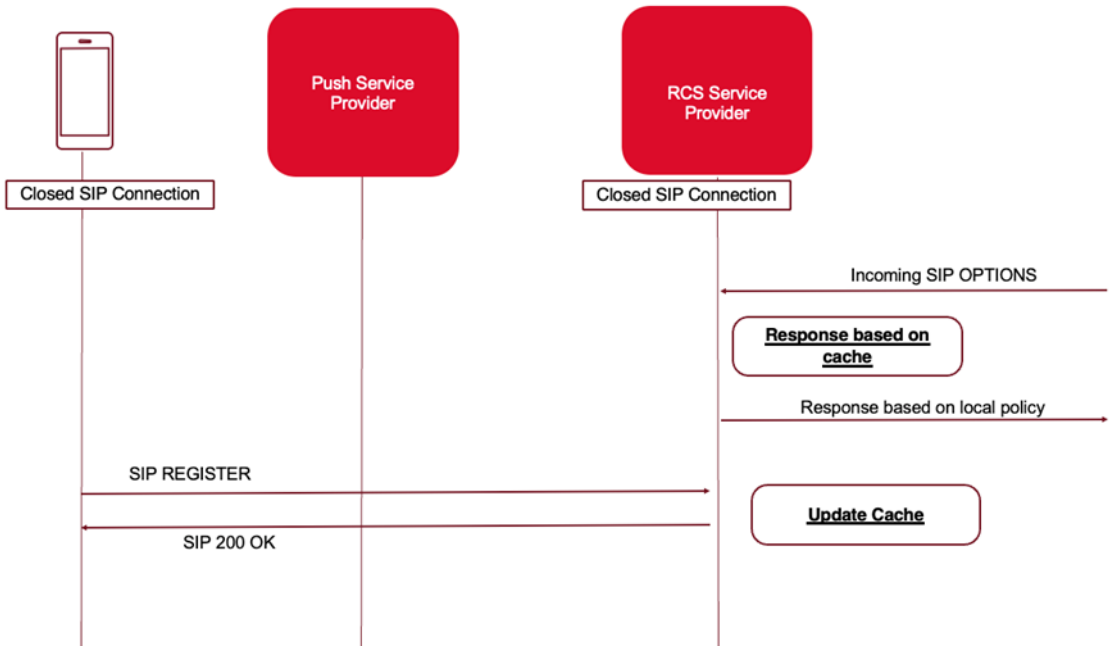


Figure 10: Capability discovery and cache behaviour for Push Notification

2.15.4.2 RCS client behaviour for CAPABILITY REFRESH

If the Push Notification Mechanism is enabled and the value of the client configuration parameter CAPABILITY REFRESH specified in section A.1.5 is less in value then the SIP REGISTER expiry, the client shall re-register based on value of the value CAPABILITY REFRESH client configuration parameter.

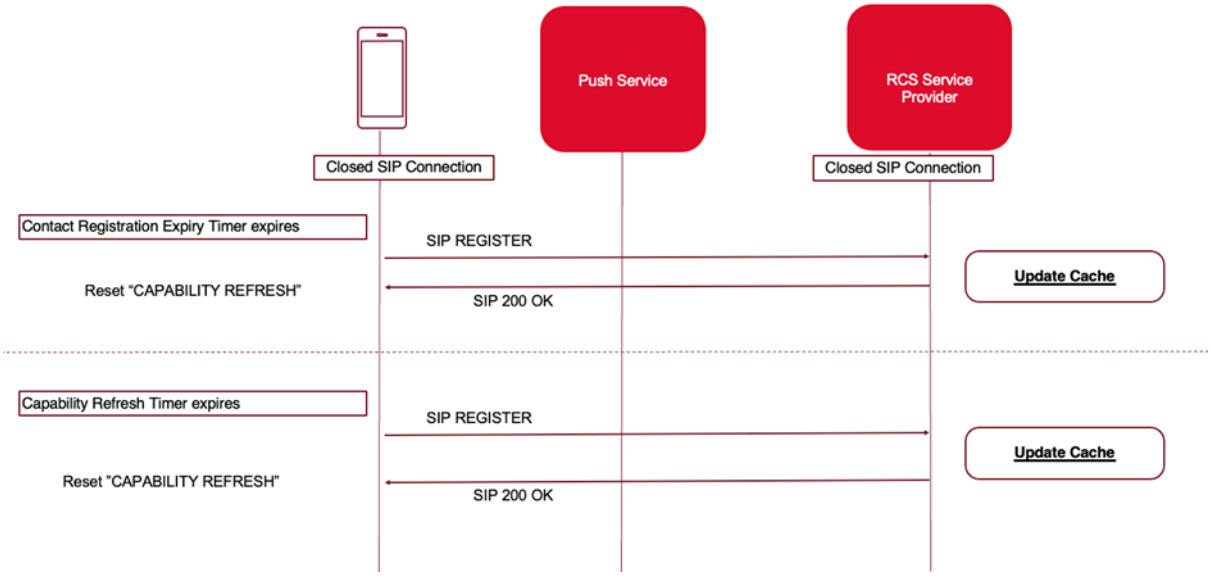


Figure 11: CAPABILITY REFRESH and cache

2.15.5 RCS client state alignment

The RCS client can have one of the following states:

- Registered state: A state of the RCS client where the client has a valid registration in the IMS and an active data connection
- Push Controlled Idle State: See section 1.3
- Un-Registered state: A state of the RCS client where the client has not registered in IMS.
- Disabled state: A state of the RCS client where the user has explicitly disabled the RCS service.

The RCS client and RCS Service Provider shall maintain a state of the IMS registration while using the Push Notification Mechanism. During transition from one state to other there may be mismatch of the RCS client state between the RCS client and the RCS Service Provider. The RCS client shall follow the actions listed in the following table.

NOTE: Specific actions on the RCS Service Provider side are not described because there are no differences from the case where the Push Notification Mechanism is not used.

Case	RCS client state on Device	RCS client Action on receiving Push Notification
1	Registered	No Action
2	Push Controlled Idle State	Register
3	Un-Registered	Register
4	Disabled	No Action

Table 15: Matrix of RCS client state on device and the action taken on receiving Push Notification

If an RCS client in Push Controlled Idle State or Un-Registered State receives a Push Notification, the RCS client shall perform a registration in the IMS as described in section 2.4.

If the RCS client is in Registered State, and it receives a Push Notification, it may initiate double CRLF keep-alive as described in section 2.7 and procedures defined in section 3.5.1 of [RFC5626] to the RCS Service Provider. Upon receiving the CRLF, the RCS Service Provider shall avoid using the Push Notification Mechanism further for as long as there is an active data connection with the RCS client.

NOTE: It is for further study whether the client needs to register or send a double CRLF keep-alive for case #1 in Table 15.

If the RCS client is in Disabled state by following procedures in section 2.3.2.2; the RCS Service Provider shall stop any further attempts to send a Push Notification to the RCS client.

NOTE: In such a case, the Push Service Provider can send back an invalid (non-retriable) response to the RCS Service Provider for that specific Push URI.

2.15.6 Reconfiguration to persistent connection

If there is a need to configure an RCS client from a non-persistent connection to a persistent connection, the RCS Service Provider shall reject two consecutive SIP REGISTER requests sent by the RCS client with a SIP 401 UNAUTHORISED response which will re-start the client configuration procedures as described in sections 2.12.1.1.2 and 2.3.2.3.

The RCS Service Provider shall use the PUSH NOTIFICATION configuration parameter to disable the Push Notification Mechanism as defined in section A.1.5.

3 RCS Services

3.1 General Service Overview

RCS provides several services that fit into the framework defined in section 1.

Section 3.2 describes messaging services and introduces services that enhance the user's messaging experience. Section 3.2.1 describes 1-to-1 Messaging and the selection of different technologies to provide this service. Section 3.2.2 describes the standalone messaging service based on OMA CPM that is considered as an evolution of the SMS/MMS messaging services providing fewer restrictions, allowing 1-to-Many Messaging and providing the interworking capability with those services. Section 3.2.3 introduces the 1-to-1 chat service that provides a more real-time experience through "isComposing" indications in addition to the store and forward functionality, including delivery and display notifications, that allows reaching users while they are offline. Section 3.2.4 describes the Group Chat service which provides multiparty scenarios. Section 3.2.5 describes the File Transfer service allowing a user to exchange any type of file with another user. A Geolocation Push service is introduced in section 3.2.6 which allows a user to share their location (or any other desired location) with another user. Section 3.2.7 describes the Audio Messaging service allowing a user to share an audio file with another user.

Section 3.6 describes the architecture and technical enablers supporting 1-to-1 communication with Chatbots.

Section 3.7 describes technical enablers for 1-to-1 and Group Messaging-Initiated Video Calls (MIVC).

All these services can be invoked either from within the address book provided that the contact has the corresponding capability (see section 2.6) and the current network connectivity allows using the service (see section 2.6.1.3) or directly from the device's menu. Additional entry points may be the chat and call history, the media gallery and camera application depending on what is suitable for the service.

Most of the NNI handling is done as described in section 2.11.

3.2 Messaging

3.2.1 1-to-1 Messaging Technology Selection

The technology selected for the first message in a 1-to-1 Conversation shall be based on:

- the RCS 1-to-1 Messaging technologies enabled i.e. RCS Standalone Messaging and/or RCS 1-to-1 Chat.
- the Chat capability of the contact (i.e. based on the last Capability Discovery that was executed according to the triggers defined in section 2.6.1)

The technology selection rules apply for the case where the originating client is registered for RCS services.

To a telephone number-based address, i.e. a tel URI or SIP URI with a user=phone parameter, RCS 1-to-1 Chat is the preferred service, if enabled and based on the Capability Discovery it is supported by a contact, then RCS 1-to-1 Chat is used. Otherwise, if Standalone Messaging is enabled and available in the current coverage then Standalone Messaging is used. If neither of those can be used, xMS is used. Error scenarios are further detailed in section 3.2.1.1.

To a SIP URI that is not known to be linked to a Chatbot, that means a SIP URI that doesn't match the criteria defined in section 3.6.2.4, then RCS 1-to-1 Chat is the preferred service, if enabled and based on the Capability Discovery it is supported by a contact, then RCS 1-to-1 Chat is used. Otherwise, if Standalone Messaging is enabled and available in the current coverage then Standalone Messaging is used. Error scenarios are further detailed in section 3.2.1.1.

For the technology selected towards a Chatbot, see section 3.2.1.2.

3.2.1.1 Messaging towards contacts not known as Chatbots

If the RCS 1-to-1 Chat Messaging technology is selected for a 1-to-1 conversation, the technology may change during the conversation based on the network fallback procedures defined in this document.

If the initiation of a RCS 1-to-1 Chat session fails with another SIP response code, then the client shall

- send an RCS Standalone Message if RCS Standalone Messaging is enabled, otherwise
- send a SMS if the destination address is a telephone number-based address.

If the RCS 1-to-1 Chat Messaging technology is selected for a 1-to-1 conversation and sending of a RCS 1-to-1 Chat message fails with one of the following MSRP response codes

- 400 request was unintelligible, or
- 403 attempted action is not allowed, or
- 501 recipient does not understand the request method, then

the client shall

- send an RCS Standalone Message if RCS Standalone Messaging is enabled, otherwise
- send a SMS if the destination address is a telephone number-based address.

If the RCS Standalone Messaging service is selected for a 1-to-1 conversation, originating or terminating network fallback may be applied based on Service Provider policies.

If the client receives one of the following responses:

- 380 Alternative Service, or
- 408 Request Timeout, or
- 486 Busy Here, or

- 487 Request Terminated, then

the client shall immediately resend the message as an xMS. For other error responses, the client will retry the message before falling back to xMS.

Table 16 provides an overview of the technology selection and fallback scenarios for 1-to-1 Messaging conversations.

Enabled technologies	Terminating network chat fallback mechanism indicated	Non -RCS recipient	RCS recipient with Chat capability	RCS recipient with no Chat capability
Only RCS Standalone Messaging	N/A	RCS Standalone Messaging	RCS Standalone Messaging	RCS Standalone Messaging
Only RCS 1-to-1 Chat Messaging	Revocation supported	SMS	RCS 1-to-1 Chat Messaging unless latched to SMS	SMS
	Interworking supported	SMS	RCS 1-to-Chat Messaging	SMS
Both RCS 1-to-1 Chat and RCS Standalone Messaging	Revocation supported	RCS Standalone Messaging	RCS 1-to-1 Chat Messaging unless latched to SMS	RCS Standalone Messaging
	Interworking supported	RCS Standalone Messaging	RCS 1-to-Chat Messaging	RCS Standalone Messaging

Table 16: Messaging technology selection for 1-to-1 conversation initiation when A party is online

3.2.1.2 Messaging towards contacts known as Chatbots

An RCS client willing to establish a messaging communication with a SIP URI that is known to be linked to a Chatbot, as defined in section 3.6.2.4, shall select the messaging technology based on enabled technologies, i.e. through the value of the configuration parameter CHATBOT MSG TECH, as follows:

Table 17 defines the technology that shall be used for Chatbot services.

Enabled technologies	Chatbot with Capabilities for		
	Chatbot Standalone Messaging only	1-to-1 Chatbot session only	Both 1-to-1 Chatbot session and Chatbot Standalone Messaging
Only Chatbot Standalone Messaging	Chatbot Standalone Messaging (see section 3.6.8.3.2)	Chatbot is not available	Chatbot Standalone Messaging (see section 3.6.8.3.2)
Only 1-to-1 Chatbot session	Chatbot is not available	1-to-1 Chatbot session (see section 3.6.8.3.1)	1-to-1 Chatbot session (see section 3.6.8.3.1)
Both 1-to-1 Chatbot session and Chatbot Standalone Messaging	If anonymization does not apply Chatbot Standalone Messaging (see section 3.6.8.3.2), otherwise Chatbot is not available	1-to-1 Chatbot session (see section 3.6.8.3.1)	1-to-1 Chatbot session (see section 3.6.8.3.1)

Table 17: Messaging technology selection for Chatbot conversations

If the RCS client is not registered when it is willing to establish a messaging communication or when the Chatbot services is not available in result of the technology selection procedure, then SMS may be used if the following conditions are fulfilled:

- An SMS address is known for the Chatbot through the Chatbot Information data (see section 3.6.4.1.6) and
- The user is sharing their MSISDN with the Chatbot (see section 3.6.5.1)
- The Chatbot is not known as a party sending spam (see section 3.6.6)
- The message to be sent is a regular text message i.e. not a File Transfer or one of the message types defined in section 3.6.10.1 and
- The RCS client is in coverage conditions where a SMS message can be sent.

3.2.2 Standalone messaging

3.2.2.1 Overview

The technical realisation of the RCS Standalone Messaging is based on the OMA CPM Pager Mode and Large Message Mode mechanisms as described in [GSMA PRD-RCC.11].

The client is enabled for Standalone Messaging via the configuration parameter STANDALONE MSG AUTH defined in section A.1.3. If Standalone Messaging is enabled i.e. the configuration parameter STANDALONE MSG AUTH is set to "1" or "2", the client shall include the feature tags in the SIP REGISTER request as defined in section 2.4.4.1.

The SDP of the SIP INVITE request and response in Large Message Mode should not be compressed even if SIP Content-Compression is supported by the client.

As per [GSMA PRD-RCC.71], Standalone Messaging is not enabled when E2EE is enabled.

3.2.2.2 Delivery and Display Notifications

The disposition status notifications for a sent Standalone Message will follow the reverse path of the sent message. The disposition notifications for Standalone Messaging could be used for the 1-to-1 or 1-to-Many messaging and for two types of notifications, delivery and display, as specified in [GSMA PRD-RCC.11].

The aggregation of IMDNs as specified in [RFC5438] shall not be used in RCS.

3.2.2.3 Deferred Messaging

The terminating Participating Function, amongst other procedures, performs the procedure for deferring messages if none of the RCS capable devices of the recipient is online.

When no RCS target recipient client is registered, the terminating Participating Function holding the message for delivery may decide to defer the standalone message for delivery at a later time. For the delivery of a deferred standalone message, the Participating Function shall, as specified in [GSMA PRD-RCC.11], push the deferred standalone messages once one of the clients of the target recipient RCS user becomes available.

If a deferred Standalone Message expires before it is delivered, the terminating Participating Function shall handle the deferred message by discarding it.

3.2.2.4 Multidevice handling

RCS supports delivering of Standalone Messages to multiple devices. As described in [GSMA PRD-RCC.11], the delivery of Standalone Messages will be done to all the user's RCS devices that are online. Also, when applicable, the message is delivered to a single non-RCS device of the user through interworking with either SMS or MMS as explained in section 3.2.2.5.1.

All procedures for sending and receiving standalone messages and their disposition notifications in an RCS multidevice environment, where the RCS user employs multiple devices, are performed as described in [CPM-SYS_DESC] and specified in [GSMA PRD-RCC.11].

3.2.2.5 Standalone Message Delivery Assurance

3.2.2.5.1 Interworking with Legacy Messaging services

Both the originating and the terminating network will offer interworking to xMS, where the former is used when sending a Standalone Message to a non-RCS user or a user of an operator with whom no interworking agreement for Standalone Messaging is offered. The latter option would be used towards RCS users whose primary device is offline, but reachable. This is considered to be providing the delivery assurance for Standalone Messaging except SIP MESSAGE request with the Chatbot IARI.

The [GSMA PRD-RCC.10] document describes general interworking procedures applicable to both SMS and MMS and the realisation details for the SMS and MMS interworking. The interworking procedures for the SMS include references to 3GPP's IP-SM-GW (IP Short Message Gateway) as described in [GSMA PRD-RCC.08].

When handling an incoming SIP MESSAGE request with the Chatbot IARI described in section 3.6.2, the Messaging Server shall not perform interworking to xMS even if interworking is supported in general, since the incoming request includes the require and explicit parameters on a dedicated Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1.

In that case, the Chatbot Platform can try to revoke the Chatbot messages as described in section 3.2.2.5.2 if required. Otherwise, the SIP MESSAGE will be deferred in the Messaging Server as per section 3.6.9.

When handling an incoming SIP MESSAGE request that carries a Rich Text Message payload as defined in section 3.2.10, the Messaging Server shall not perform interworking to xMS even if interworking is supported in general.

3.2.2.5.1.1 Interworking procedure

The procedures for the RCS standalone messaging service feature interworking to SMS and MMS legacy messaging services are performed by two interworking functional entities, the Interworking Selection Function (ISF) and the Interworking Function (IWF). After the Participating Function has decided that the message has to be interworked, the selection of whether to interwork to SMS or MMS is done in the ISF as described in [GSMA PRD-RCC.10]. The actual interworking procedure is performed by the SMS and MMS gateways described in [GSMA PRD-RCC.08] and [GSMA PRD-RCC.10]. These functions also interwork the delivery notifications received from the SMS and the delivery and display notifications received from the MMS message recipient(s) and forward them to the sending Participating Function to be passed on to the sending RCS client.

The interworking functions also interwork any incoming SMS or MMS messages to RCS messaging.

3.2.2.5.1.2 Interworking with SMS

When the target recipient device for an RCS Standalone Message is a non-RCS capable, SMS capable device, the process of interworking with legacy SMS is invoked according to [GSMA PRD-RCC.11].

When the SMS interworking function (IP-SM-GW or SMS-IWF) receives a SIP MESSAGE request with the OMA CPM ICSI "*3gpp-service.ims.icsi.oma.cpm.msg*", it checks the size of the received payload of the SIP MESSAGE request. If the size of the payload is too large to be sent as one SMS message, the payload will be divided into concatenated SMS messages. The SMS-IWF will send the request(s) generated based on the received SIP MESSAGE request towards the SMS-C (Short Message Service Centre) using either the SMPP (Short Message Peer-to-Peer) or MAP (Mobile Application Part) protocols, depending on the type of SMS network in which it is deployed, as specified in [GSMA PRD-RCC.10] or [GSMA PRD-RCC.08] respectively.

Breakout to SMS can be done at the originating side if the addressed user is not an IMS user. This is determined based on the standalone messaging capability information, on local information the Messaging Server may have about the recipient, or when the Messaging Server receives an error response. Otherwise, the breakout at the terminating side is done,

if either the addressed user is an RCS user using SMS instead of RCS standalone messaging service or the user is using a mixture of legacy and RCS devices.

The following error responses to the SIP MESSAGE (or, for the IP-SM-GW realisation, optionally for a Large Message Mode message the SIP INVITE) request indicate that the recipient is not an RCS contact and these responses can be used to trigger interworking:

- 404 Not Found;
- 405 Method Not Allowed;
- 410 Gone;
- 414 Request URI Too Long;
- 415 Unsupported Media Type;
- 416 Unsupported URI Scheme;
- 488 Not Acceptable Here;
- 606 Not Acceptable.

3.2.2.5.1.3 Interworking with MMS

When the target recipient device for standalone messaging is not an RCS device and the message to be sent is a multimedia message, the process of interworking with legacy MMS is invoked according to [GSMA PRD-RCC.11].

Depending on the size of the standalone message, it could be either a text message with a large payload or a multi-media standalone message. In the former case, the interworking with SMS would apply as described in section 3.2.2.5.1.2 if the message were small enough for a concatenated SMS. Otherwise, the interworking would be to the MMS service, hence sending a SIP INVITE request to the RCS MMS-IWF.

When the RCS MMS-IWF receives a SIP INVITE request containing the OMA CPM ICSI "*3gpp-service.ims.icsi.oma.cpm.largemsg*" for a Large Message Mode standalone message, it will send a 200 "OK" response if no errors are found in the SIP INVITE request or an appropriate error response. This is followed by the MMS-IWF's subsequent receiving of an MSRP SEND request for the establishment of the MSRP session, and the process then continues as described in [GSMA PRD-RCC.10].

Breakout to MMS can be done at the originating side if the addressee is not an IMS user either based on local information the Messaging Server may have about the recipient, or when it receives an error response. Otherwise, the breakout at the terminating side is done if either the addressee is an RCS user using MMS instead of RCS standalone messaging service or the user is using a mixture of legacy and RCS devices.

The following error responses to the INVITE request indicate the recipient is not an RCS contact and can be used to trigger interworking:

- 404 Not Found;
- 405 Method Not Allowed;
- 410 Gone;
- 414 Request URI Too Long;
- 415 Unsupported Media Type;

- 416 Unsupported URI Scheme;
- 488 Not Acceptable Here;
- 606 Not Acceptable.

3.2.2.5.2 Standalone Message Revocation

NOTE: Standalone Message Revocation is currently only used for SIP MESSAGE request with the Chatbot IARI described in section 3.6.2.

If required by the sender (i.e. currently only limited to the Chatbot Platform), the sender can generate MessageRevoke request as described in section 3.2.3.8.2.1. The MessageRevoke request is carried in the body of a SIP MESSAGE request that includes the same imdn.message-ID value of the Standalone Message that is intended to be revoked (as described in section 3.2.3.8.2.4). In this Standalone Message MessageRevoke SIP MESSAGE request, the following value shall be included instead of what required in section 3.2.3.8.2.1:

1. shall include an Accept-Contact header field with the CPM ICSI for Standalone Message, similarly to the case for IMDNs carried in SIP MESSAGE requests;
2. shall include a P-Preferred-Service header with the CPM ICSI for Standalone Message.

When the Messaging Server receives MessageRevoke requests, and Standalone Message Revocation is supported by it, the Messaging Server shall handle the request and shall send a MessageRevokeResponse request as described in section 3.2.3.8.2.2. The following value shall be included in the MessageRevokeResponse request instead of what required in section 3.2.3.8.2.2:

1. shall include an Accept-Contact header field with the CPM ICSI for Standalone Message, similarly to the case for IMDNs carried in SIP MESSAGE requests;
2. shall set the Request-URI of the MessageRevokeResponse request to the service-ids of the originating Chatbot defined as per section 2.5.4.1 that sent the message that is requested to be revoked;
3. shall include a P-Asserted-Service header with the CPM ICSI for Standalone Message;

3.2.2.6 Indicating of Capabilities in Pager Mode Messaging

This section defines a mechanism for the indication of client capabilities in a Pager Mode message. The sender client capabilities are conveyed in a SIP URI contained in the CPIM "From" header field of a Pager Mode Message.

The capability indication is added to the SIP URI using the "header" rule defined in section 25.1 of [RFC3261]. A "header" to convey capabilities is defined as follows:

```
capabilities = "Contact=" escaped-feature-param *("%3B" escaped-feature-
param)
     ; escaped-feature-param is feature-param defined in [RFC3840] with
     ; escaping of any characters not defined for use in [RFC3261] headers
```

This allows to include a Contact "header" in the URI the value of which shall be a list of feature tags as defined in [RFC3840] for which any characters in the feature tag name,

value and separators that are not allowed to be used in a "header" according to [RFC3261] shall be escaped according to the rules defined in [RFC2396].

Non-normative examples:

- A CPIM "From" header of a Pager Mode message to convey the feature collection foo=1, bar=2:

  ```
  From: <sip:bob@example.com?Contact=foo%3D1%26bar%3D2>
  ```

- A CPIM "From" header of a Pager Mode message sent from the Chatbot Platform:

  ```
  From:
  <sip:anonymous@anonymous.invalid?Contact=+g.gsma.rcs.isbot%3B+g.gsma.
  rcs.botversion%3D%22%23%3D1%2C%23%3D2%22>
  ```

3.2.2.7 Large Messages via OMA CPM Standalone Messaging Pager Mode

3.2.2.7.1 Procedures in the sending client

Rather than a fixed limit of 1300 bytes for switching over between CPM Pager Mode and Large Message Mode, for RCS the client shall select the Standalone Messaging mode based on the value of the configuration parameter STANDALONE SWITCHOVER SIZE defined in section A.1.3.

If the size of a Pager Mode message does not exceed the size provided in the configuration parameter value, then the OMA CPM Pager Mode mechanism shall be used, otherwise the client shall switch over to the OMA CPM Large Message Mode. If SIP content compression is supported, the size of a Pager Mode message shall be checked after the content compression.

3.2.2.7.2 Procedures in the originating Messaging Server

The procedures in the originating Messaging Server, if Standalone Messaging is used for interconnection with a terminating Messaging Server not supporting large pager mode messages, are based on bilateral agreement and are out of the scope of this document.

3.2.2.7.3 Procedures in the terminating Messaging Server

Prior to delivery of a Pager Mode message to a client, the terminating Messaging Server shall check the size of the message.

If the size of the message does not exceed 1300 bytes, then terminating Messaging Server shall invoke the procedures for the delivery of the message via the OMA CPM Pager Message mode mechanism.

If the size of the message exceeds 1300 bytes and the Messaging Server did not receive the large pager mode feature tag defined in section 3.2.2.7.5 in the SIP REGISTER from the client, then terminating Messaging Server shall invoke the procedures for the delivery of the message via the OMA CPM Large Message mode mechanism.

If the size of the message exceeds a size of 1300 bytes and the Messaging Server did receive the large pager mode feature tag defined in section 3.2.2.7.5 in the SIP REGISTER

from the client, then terminating Messaging Server shall invoke the procedures for the delivery of the message via the OMA CPM Pager Message mode mechanism.

3.2.2.7.4 Procedures in the receiving client

The client receiving a Standalone Message via the OMA CPM Pager Mode mechanism shall accept messages regardless of the size of their message body, i.e. the reception shall not be authorised against a message size limit.

The client shall advertise this capability by adding the large pager mode support feature tag defined in section 3.2.2.7.5 in the Contact header of SIP REGISTER requests.

3.2.2.7.5 Large Pager Mode support feature tag

The Large Pager Mode support feature tag is the indication of an OMA CPM client to support receiving a Standalone Message via the OMA CPM Pager Mode mechanism regardless of its size.

The indication of the large pager mode support is provided by a feature parameter as defined in section 9 of [RFC3840]. The feature parameter is encoded as an "other-tags" feature tag in accordance with the definitions of section 9 of [RFC3840].

The feature tag name shall be set to "g.gsma.rcs.cpm.pager-large". The media feature tag shall have no value.

Security considerations for this media feature tag are discussed in section 11.1 of [RFC3840].

The indication of the large pager mode support is represented as follows:

+g.gsma.rcs.cpm.pager-large

3.2.2.8 1-to-Many Messaging Technology Selection

1-to-Many Messaging builds on the 1-to-1 Messaging service and is based on OMA CPM procedures [GSMA PRD-RCC.11].

Technology selection rules are used to determine if OMA CPM based 1-to-Many Messaging is used or if it is provided using 1-to-1 Messaging per single recipient.

The availability of the 1-to-Many messaging service and the maximum number of participants allowed for 1-to-Many messaging services shall be controlled by the MAX 1 TO MANY RECIPIENTS parameter defined in sections A.1.3 and A.2.4. The MAX 1 TO MANY RECIPIENTS configuration parameter is applicable regardless of which technology is selected.

If the RCS Standalone Messaging service is enabled, the RCS Standalone Message service shall be selected based on procedures described in in sections 7.2.1 and 9.1 of [GSMA PRD-RCC.11] for sending a CPM Standalone Message to an ad-hoc group whereby the client shall set the copyControl attribute for all recipients to “BCC”.

If the RCS Standalone Messaging service is not enabled, the technology selected for the many 1-to-1 Messaging services is based on the 1 TO MANY SELECTED TECHNOLOGY

client configuration parameter value (see sections A.1.3 and A.2.4). If the Chat service is selected via the 1 TO MANY SELECTED TECHNOLOGY configuration parameter, the procedures for technology selection and delivery assurance apply on a single recipient basis as defined for the RCS 1-to-1 Chat service.

3.2.2.9 Extended Messaging

When Extended Messaging features apply, an RCS client sender client shall follow the procedures described in section 3.2.8.

3.2.2.10 Rich Text Message

Standalone Messages that carry a Rich Text Message payload as defined in section 3.2.10 shall follow the Rich Text Message procedures in that section together with the Standalone Messaging procedures. For Standalone Messaging, the SDP-based content negotiation in section 3.2.10.2 does not apply. The client shall only send a Standalone Message with a Rich Text Message payload when Capability Discovery (see section 2.6.1.3) shows that the destination supports Rich Text Messages.

3.2.3 1-to-1 Chat

3.2.3.1 Overview

At a technical level, the Chat service relies on the following concepts:

- SIP procedures for the setup of sessions using MSRP for the message exchange;
- In the SDP of the SIP INVITE request and response, the *a=accept-types* attribute shall include only *message/cpim* and *application/im-iscomposing+xml*, i.e., "*a=accept-types:message/cpim application/im-iscomposing+xml*".
- The SDP of the SIP INVITE request and response shall not be compressed even if SIP Content-Compression is supported by the client.
- Delivery assurance information is included in SIP signalling by the network as per section 3.2.3.8.
- If E2EE is enabled for the 1-to-1 Chat, the additional procedures in [GSMA PRD-RCC.16] shall be followed.
- Messages are transported in the MSRP session. Each MSRP SEND request contains a request to receive an Instant Messaging Disposition Notification (IMDN) 'delivery' notification, and possibly a request to receive an IMDN 'display' notification. A client should, therefore, always include "positive-delivery" and "negative-delivery" in the value for the CPIM/IMDN Disposition-Notification header field. That means that the value of the header field is either "positive-delivery,negative-delivery" or "positive-delivery,negative-delivery,display" depending on whether display notifications were requested.
  The receiving devices must generate an MSRP SEND request containing the IMDN status when the user message is delivered and if requested, another MSRP SEND request when the message is displayed.

NOTE: If there is not an already established MSRP session between sender and receiver, the Pager Mode (i.e. SIP MESSAGE) is used to transport IMDNs (delivery notification, display notifications)

- For the messages sent in a Chat session, as well as the notifications mentioned above, the client shall request in the Instant Message Disposition Notification (IMDN) Common Profile for Instant Messaging (CPIM) header an Interworking Disposition Notification as defined in Appendix O of [GSMA PRD-RCC.11], if during the setup of the session the terminating network has indicated support for interworking using the corresponding feature tag defined in Table 18.
- When receiving an Interworking Disposition Notification as defined in Appendix O of [GSMA PRD-RCC.11], the client shall consider the message to have been delivered and interworked. The client shall thus not assume that a Delivery and a Display notification will follow.
- In normal circumstances, between two users at most only a single session is active at a time. A client shall, therefore, not initiate a new Chat session towards a user with whom there is already an established Chat session.
- When users are allowed to have multiple devices, the Messaging Server is required to be able to fork the incoming 1-to-1 Chat session request to each of the receiving user's devices to set up an MSRP session with each of them.
- Multimedia content within a Chat session is not permitted. Therefore, in the SDP of the SIP INVITE request and response, the *a=accept-wrapped-types* attribute shall only include text/plain and message/imdn+xml and if File Transfer using HTTP, Geolocation PUSH or Rich Text Message is supported (see sections 3.2.5, 3.2.6.2 and 3.2.10) *application/vnd.gsma.rcs-ft-http+xml, application/vnd.gsma.rcspushlocation+xml* and *application/vnd.gsma.richtextmessage+proto* respectively, e.g., *a=accept-wrapped-types:text/plain message/imdn+xml*. This also applies to requests generated by the Participating Function, and to responses generated by the Participating Function even if a response from the terminating client has not yet been received. To transfer multimedia content during a chat, File Transfer is used.
- When receiving a Chat session invitation a 1-to-1 chat session is established by sending the 200 OK response immediately when receiving a SIP INVITE request for a 1-to-1 Chat session.
- IMDN [RFC5438]: RCS relies on the support of IMDN as defined in [RFC5438] and [RFC5438Errata] to request and forward disposition notifications of all the exchanged messages (See also section C.2 for the errata mentioned in [RFC5438Errata]).
- In MSRP requests, the client shall set both the CPIM From and CPIM To headers to *sip:anonymous@anonymous.invalid* to prevent revealing the user’s identity when transmitted over unprotected links. A client receiving a CPIM message in a one-to-one Chat should, therefore, ignore the identity indicated in the CPIM headers. Based on the SIP signalling at session setup the client has access to the asserted identity of the other party as described in section 2.5.
- The CPIM/IMDN wrapper shall be UTF-8 encoded to avoid any potential internationalisation issues.
- IMDN message identification for all messages (including notifications delivered via SIP MESSAGE) as defined in [RFC5438].
- The originating Messaging Server shall always set the CPIM DateTime header in the chat messages and notifications it receives by overwriting the value provided by the

client. A client receiving these requests should, therefore, rely on these headers rather than on locally available time information.

- Both the Originating and the Terminating function shall ensure that messages are received in correct order by the RCS client regardless if the messages are store and forwarded or not.
- The aggregation of IMDNs as specified in [RFC5438] shall not be used.
- Chat inactivity timeout: When a device or the network detects that there was no activity in a chat for IM SESSION TIMER, a configurable period of time (see Table 167), it will close the established Chat session.
- When reopening an older chat on the device, that contains messages for which a “display” notification should be sent, these notifications shall be sent according to the rules and procedures of [GSMA PRD-RCC.11].
- The "IsComposing" notification is generated and processed according to the rules and procedures of [GSMA PRD-RCC.11]. Consequently, the ‘IsComposing’ notification is not sent with CPIM headers, and as such, a delivery and/or displayed notification cannot be requested.
- If the client does not understand the content type or the content of the CPIM message, the client should return a “negative-delivery” disposition notification with a “failed” status as defined in [RFC5438] identifying the failure reason as defined in section C.6.1.

3.2.3.2 Store and Forward Mode

Since Store and Forward is mandatory, all Messaging Servers shall support it. The Messaging Server serving the receiver of the message has the responsibility to store and forward messages which are not delivered. The Messaging Server serving the sender of the message has the responsibility of storing the delivered/displayed notifications if the sender of the message is offline.

The Messaging Server stores undelivered messages for a period that is determined by local server policy. If at the end of this period the messages have not been delivered, the Messaging Server discards them. This applies to notifications as well as messages.

Stored messages and notifications are delivered to intended recipients as per the procedures defined in [GSMA PRD-RCC.11].

3.2.3.3 Delivering stored disposition notifications

Deferred disposition notification shall be delivered as described in section 8.3.5 of [GSMA PRD-RCC.11].

To support backward compatibility with Messaging Servers based on earlier versions of this specification, a client shall support receiving the store and forward functionality using a special session for the purpose of delivering these notifications. As specified in section 7.3.11 of [GSMA PRD-RCC.11], this special session shall be automatically accepted by the device. It shall be recognised by the device by means of the well-known username part of the URI (*rcse-standfw@<domain>*) uniquely identifying the store and forward service identity that is provided in the *P-Asserted-Identity* header field.

3.2.3.4 Interworking towards SMS/MMS

The functionality for interworking of the chat service to SMS/MMS is optional and it is the decision of each Service Provider whether to deploy it. This deployment involves:

- The Messaging Server described in [GSMA PRD-RCC.11].
- The ISF described in [GSMA PRD-RCC.10] which is responsible for selecting the appropriate interworking function for a new session.
- The IWF for SMS and MMS described in respectively [GSMA PRD-RCC.08] and [GSMA PRD-RCC.10] which are responsible for doing the actual interworking (that is the protocol conversions) between RCS based chat and SMS or MMS.

When a Chat session invitation needs to be interworked, the invitation will be first routed to the terminating network as described in previous sub-sections, and then the same procedures as for interworking of chat invitations on the originating side will apply.

As per [GSMA PRD-RCC.71], interworking towards SMS/MMS is not enabled when E2EE is enabled or when the message is a Rich Text Message (see section 3.2.10).

3.2.3.5 Multidevice handling

Multidevice support as specified in this section is applicable only for SIP-configured RCS clients.

Multidevice handling occurs when a user has more than one device (e.g., PC and mobile). As per [GSMA PRD-RCC.71], Common Message Store-based multidevice is not enabled when E2EE is enabled.

When a new 1-to-1 chat is initiated and a message is sent from User A to a User B with User B having multiple devices registered at the same time, the Messaging Server forks the Chat session invitation to the different devices. Forking on the Messaging Server is further elaborated in [GSMA PRD-RCC.11].

NOTE: It is assumed that the originating user uses one device per session.

3.2.3.6 Emoticons

Selected emoticons are displayed graphically but sent and received as text. The list of supported emoticons is defined in [GSMA PRD-RCC.11] Appendix L.

3.2.3.7 Chat message size limitations

The maximum size of a text Chat message in bytes that a user can enter is controlled through the MAX SIZE IM configuration parameter defined in Table 167 in section A.1.3.

3.2.3.8 1-to-1 Chat Delivery Assurance

For gRPC-configured RCS clients, only Client Fallback to SMS is supported. All 1-to-1 Chat procedures for that case are defined in section 5.2 of [GSMA PRD-RCC.30].

3.2.3.8.1 Network Fallback Support Capability

Delivery Assurance relies on a capability indication mechanism during the initiation of a 1-to-1 Chat session (see sections 3.2.3.8.3). The capability indication is used by the network

to indicate which fallback mechanism shall be applied for messages sent by a client in this 1-to-1 Chat session (see section 3.2.3.8.3). The capability indication is used by the network to indicate its support for either

- the network fallback procedure, where the network is responsible for providing the fallback (also called Network Fallback to SMS, or NFS), or
- the message revocation, where the client is responsible for the fallback (also called Client Fallback to SMS or CFS).

The indication is provided by the feature tags defined in Table 18.

Tag	Description
+g.gsma.rcs.msgrevoke	Message Revocation is supported (as defined in section 3.2.3.8.2)
+g.gsma.rcs.msgfallback	Network interworking is supported

Table 18: Feature tags used to indicate network support for chat fallback mechanisms

Only one of the network indications defined in Table 18 can be present in a 1-to-1 Chat session, i.e. they are mutually exclusive.

The indication and the support of a network fallback mechanism is mandatory. This includes service provider deployments supporting termination of 1-to-1 Chat sessions only.

As per [GSMA PRD-RCC.71], if the 1-to-1 Chat is E2EE enabled, the network must indicate Message Revocation support.

If the network indicates support of network fallback, the network shall take responsibility for delivering the message using the most suitable path. Therefore, if the terminating network has provided this indication, the client shall not apply procedures to monitor the delivery of chat messages and fallback.

When the CHAT REVOKE TIMER is set to a value higher than 0 and the network indicated capability is message revocation, the client shall apply procedures to monitor the delivery of chat messages and fallback.

When SMS latching is triggered, the client shall cache this event for future interactions with this contact unless Chat service is re-selected. This cached information shall affect the Capability Discovery triggers (see section 2.6) and the messaging technology selection (see section 3.2.1).

When the CHAT REVOKE TIMER parameter is set to 0 or the SMS fallback is disabled on the client and the network indicated capability is message revocation, then the client behaviour is based on the terminating network chat delivery policies (e.g. store and forward will be applied if the recipient cannot be reached).

3.2.3.8.2 Chat Message Revocation

Message revocation is a feature that allows a client to request a chat message to be revoked by the recipient's Messaging Server. The recipient's Messaging Server processes

MessageRevoke requests and responds with a MessageRevokeResponse request based on the chat message delivery status.

MessageRevoke requests and MessageRevokeResponse requests are not sent with CPIM headers, and a delivery and/or displayed notification shall not be requested.

There is no store and forward for the Message Revoke requests and the MessageRevokeResponse requests.

3.2.3.8.2.1 Generating Chat MessageRevoke Requests

The MessageRevoke request is generated by the client of the message sender. The MessageRevoke request is carried in the body of a SIP MESSAGE request that includes the same imdn.message-ID value of the chat message that is intended to be revoked (as described in section 3.2.3.8.2.4).

MessageRevoke requests shall be generated only towards networks where their Messaging Server can handle them as described in section 3.2.3.8.2.3 and are not meant to reach other clients. MessageRevoke requests shall not be generated in case the delivery notification pertaining to the original message has been received.

NOTE: Given that a revoke may be sent only if support has been indicated by the terminating network, it cannot be initiated when the INVITE transaction is still pending.

When message revocation is enabled (CHAT REVOKE TIMER, see section A.1.3), the client can generate MessageRevoke requests once the timer is expired. In order for the MessageRevoke requests to be transmitted, the client shall have data connectivity.

MessageRevoke requests shall be generated only if support for MessageRevoke requests has been indicated in the Contact header of the SIP INVITE request or in the response to the SIP INVITE request of the session to which the message intended to be revoked belongs. Specifically, this indication is in the form of a feature tag in the Contact header of the SIP INVITE request or response that is defined in section 3.2.3.8.2.3.

Message revocation is applicable for chat messages used for File Transfer via HTTP as defined in section 3.2.5 or Geolocation Push as defined in section 3.2.6 if an appropriate fallback mechanism is applicable for the message.

When a message is to be revoked, the client shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the client:

1. shall include an Accept-Contact header field with the CPM ICSI for Session Mode Messaging, similarly to the case for IMDNs carried in SIP MESSAGE requests;
2. shall add a dedicated Accept-Contact header field carrying the Message Revoke feature tag defined in section 3.2.3.8.2.3 along with the *require* and *explicit* parameters;

3. shall include the Content-Type header field with the value set to the message revocation content type *application/vnd.gsma.rcsrevoke+xml*, as described in section 3.2.3.8.2.4;
4. shall include in the P-Preferred-Identity header the address of the originating RCS client that has been authenticated as per [GSMA PRD-RCC.11];
5. shall include a User-Agent header field as specified in Annex C.4.1;
6. shall set the Request-URI of the MessageRevoke request to the address of the target contact of the message that is requested to be revoked;
7. shall not include the device identifier of the original sender of the message in the MessageRevoke request;
8. shall set the body of the MessageRevoke request, as described in section 3.2.3.8.2.4, as follows:

    a) The <Message-ID> element set to the value of the imdn.message-ID of the original message that is requested to be revoked,
    b) The <From> element set to the URI of the sender of the message,
    c) The <To> element set to the URI of the recipient of the message.

9. shall include a Conversation-ID header and a Contribution-ID header and shall

    a) If available, include the same Conversation-ID and Contribution-ID header field values that were used for the message being revoked; otherwise
    b) generate and include new Conversation-ID and Contribution-ID header field values;

10. shall include a P-Preferred-Service header with the CPM ICSI for Session Mode Messaging
11. shall send the SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11].

3.2.3.8.2.2 Handling MessageRevoke Requests

For a network, handling of MessageRevoke requests goes along with supporting clients that generate MessageRevoke requests.

The MessageRevokeResponse request shall indicate the result of the MessageRevoke request that can either have been successful or have failed. Similarly to the MessageRevoke request, the Messaging Server shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the Messaging Server handling the MessageRevoke request:

1. Shall include an Accept-Contact header field with the CPM ICSI for Session Mode Messaging, similarly to the case for IMDNs carried in SIP MESSAGE requests;
2. shall add a dedicated Accept-Contact header field carrying the Message Revoke feature tag defined in section 3.2.3.8.2.3 without the *require* and *explicit* parameters;
3. shall include the Content-Type header field with the value set to the message revocation content type *application/vnd.gsma.rcsrevoke+xml*, as described in section 3.2.3.8.2.4;

4. shall include in the P-Asserted-Identity header the address of the intended recipient RCS Client, where the Messaging Server initiates the MessageRevokeResponse on behalf of the intended recipient that has been authenticated as per [GSMA PRD-RCC.11];
5. shall include a User-Agent header field of the Messaging Server as specified in Annex C.4.1;
6. shall set the Request-URI of the MessageRevokeResponse request to the address of the contact that sent the message that is requested to be revoked;
7. shall set the body of the MessageRevokeResponse request, as follows:

    a) The <Message-ID> element set to the value of the imdn.message-ID of the original message that is requested to be revoked,
    b) The <From> element set to the URI of the sender of the message,
    c) The <To> element set to the URI of the recipient of the message,
    d) The <result> element set to the revoke result which shall be “success” if the MessageRevoke request was successful and “failure” if it has failed;

8. shall include a Conversation-ID header and a Contribution-ID header and shall

    a) if available, include the same Conversation-ID and Contribution-ID header field values that were used for the message being revoked; otherwise
    b) generate and include new Conversation-ID and Contribution-ID header field values;

9. shall include a P-Asserted-Service header with the CPM ICSI for Session Mode Messaging;
10. shall, when a 'tk' parameter set to ‘on’ is present in the P-Asserted-Identity of the corresponding MessageRevoke request, add a Privacy header as defined in [RFC3323] set to the value “tk”.
11. shall send the SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11].

The MessageRevokeResponse request shall be indicated as successful when the message to be revoked is removed from the deferred storage and will therefore not be delivered to the client.

The MessageRevokeResponse request shall indicate the request as having failed when any of the following conditions is met:

- Interworking towards SMS/MMS has occurred at originating or terminating side
- A successful delivery notification for which the MessageRevoke request has been generated has been received by the originating or terminating Messaging Server;
- Message revocation is not performed successfully by the terminating Messaging Server (e.g., due to Messaging Server failures);
- The message that the intended MessageRevoke request has been generated for is stored at the terminating side in the Common Message Store.
- The message is under retry delivery attempt due to Messaging Server store and forward functionality.

The client shall ignore any MessageRevokeResponse request for chat messages that it does not recognize based on the Message-ID (corner case).

MessageRevoke requests shall never be forwarded to the client and shall be processed right after being received by the Messaging Server.

3.2.3.8.2.3 Message Revoke feature tag

RCS defines a Message Revoke feature tag to indicate support of the message revocation feature. The RCS Client and originating Messaging Server shall make use of the message revocation feature only when the terminating Messaging Server has indicated its support through the Message Revoke feature tag. It can be used to indicate support for revoking of any message identified with a CPIM Message-ID. However, this release of RCS only allows it for chat messages.

The feature tag is set in the Contact header of the SIP INVITE request or response used to set up a 1-to-1 chat session and it is always attached by the Messaging Server that supports Message revocation feature. The client shall only include this feature tag in the MessageRevoke request.

The feature tag is defined as *+g.gsma.rcs.msgrevoke*.

3.2.3.8.2.4 Message Revoke content type

The Message Revoke XML schema is defined as shown on Table 19.

The associated Multipurpose Internet Mail Extensions (MIME) content type is *application/vnd.gsma.rcsrevoke+xml*.

This content type used in both the MessageRevoke request and in the MessageRevokeResponse request.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:rcsrevoke"
                xmlns:xs="http://www.w3.org/2001/XMLSchema"
                xmlns="urn:gsma:params:xml:ns:rcs:rcs:rcsrevoke"
                elementFormDefault="qualified"
                attributeFormDefault="unqualified">
        <xs:element name="imRevoke">
                <xs:complexType>
                        <xs:sequence>
                                <xs:element name="Message-ID" >
                                        <xs:simpleType>
                                                <xs:restriction base="xs:token"/>
                                        </xs:simpleType>
                                </xs:element>
                                <xs:element name="result" minOccurs="0" maxOccurs="1">
                                        <xs:simpleType>
                                                <xs:restriction base="xs:string">
                                                        <xs:enumeration value="success"/>
                                                        <xs:enumeration value="failure"/>
                                                </xs:restriction>
                                        </xs:simpleType>
                                </xs:element>
                                <xs:element name="From">
                                        <xs:simpleType>
                                                <xs:restriction base="xs:anyURI"/>
                                        </xs:simpleType>
                                </xs:element>
                                <xs:element name="To">
                                        <xs:simpleType>
                                                <xs:restriction base="xs:anyURI"/>
                                        </xs:simpleType>
                                </xs:element>
                                <xs:any namespace="##other" processContents="lax" minOccurs="0"
                                maxOccurs="unbounded"/>
                        </xs:sequence>
                </xs:complexType>
        </xs:element>
</xs:schema>
```

Table 19: RCS Revoke and RevokeResponse message body schema

The following is an example of the body of a SIP MESSAGE requesting that a specific chat message be revoked. In order to know whether the revoke was successful or not, the MessageRevoke request sender checks the result field in incoming MessageRevokeResponse requests.

Example of a MessageRevoke request:

```
Content-type: application/vnd.gsma.rcsrevoke+xml
Content-length: ...

<?xml version="1.0" encoding="UTF-8"?>
<imRevoke xmlns="urn:gsma:params:xml:ns:rcs:rcs:rcsrevoke">
<Message-ID>23499fuq34fu</Message-ID>
<From>tel:+1234578901</From>
<To>tel:+1234578902</To>
</imRevoke>
```

NOTE: The Message-ID, "23499fuq34fu", in the XML body refers to the CPIM Message-ID of the message to be revoked.

Example of a MessageRevokeResponse request where the revoke succeeded. If it had failed, the value of result would be “failure”:

```
Content-type: application/vnd.gsma.rcsrevoke+xml
Content-length: ...

<?xml version="1.0" encoding="UTF-8"?>
<imRevoke xmlns="urn:gsma:params:xml:ns:rcs:rcs:rcsrevoke">
<Message-ID>23499fuq34fu</Message-ID>
<result>success</result>
<From>tel:+1234578901</From>
<To>tel:+1234578902</To>
</imRevoke>
```

NOTE: The Message-ID, "23499fuq34fu", in the XML body refers to the CPIM Message-ID of the message that was revoked.

3.2.3.8.3 Chat Fallback Mechanism management in the Client

When initiating a 1-to-1 Chat session, the client shall monitor the delivery of the messages exchanged in the session based on the feature tags defined in Table 18:

- If a SIP 200 OK response is received as final response, the client shall behave as follows based on the presence of the feature tags defined in section 3.2.3.8.1.
    - If the Contact header in the 200 OK response included the message revocation feature tag defined in section 3.2.3.8.1 while the CHAT REVOKE TIMER client configuration parameter is configured with the value 0 or if applicable the SMS fall-back is disabled on the client, the client shall assume delivery for any messages based on the terminating network delivery policies (e.g. store and forward will be applied if the recipient cannot be reached).
    - If the Contact header in the 200 OK response includes the message revocation feature tag defined in section 3.2.3.8.1, the CHAT REVOKE TIMER client configuration parameter is set to a value higher than 0 and if applicable the SMS fall-back is enabled on the client, the client shall apply monitoring of the delivery for any messages sent within the session (i.e. through a timer based on the CHAT REVOKE TIMER client configuration parameter).
    - If the Contact header in the 200 OK response includes the interworking feature tag defined in section 3.2.3.8.1, the client shall not monitor the delivery of any messages that it sends in the Chat session.

For a client invited to a 1-to-1 Chat session (excluding a session for delivering stored messages or notifications), behaviour shall be as follows:

- If the Contact header field included in the SIP INVITE request included the message revocation feature tag defined in section 3.2.3.8.1 while the CHAT REVOKE TIMER client configuration parameter is configured with the value 0 or if applicable SMS fall-back is disabled on the client, the client shall assume the delivery of any messages that it sends in the Chat session according to the terminating network delivery policies (e.g. store and forward will be applied if the recipient cannot be reached).
- If the Contact header in the SIP INVITE request includes the message revocation feature tag defined in section 3.2.3.8.1 and the CHAT REVOKE TIMER client configuration parameter is set to a value higher than 0 and if applicable SMS fall-back is enabled on the client, the client shall monitor the delivery of any messages that it sends in the Chat session (i.e. through a timer based on the CHAT REVOKE TIMER client configuration parameter).
- If the Contact header of the SIP INVITE request includes the interworking feature tag defined in section 3.2.3.8.1, the client shall not monitor the delivery of the messages that it sends in the Chat session.

3.2.3.8.4 Chat Fallback Mechanism Management in the network

Service Providers supporting the RCS 1-to-1 Chat service shall support network fallback. The fallback is performed in the network serving the recipient of the message. RCS 1-to-1 Chat Fallback in the network serving the sender of the message is out of the scope of this document. The following procedures shall be implemented:

- When handling an SIP INVITE request for a 1-to-1 Chat session, the messaging server in the originating network shall add either the message revocation or the network interworking feature tag defined in section 3.2.3.8.1 in the Contact header field of the SIP INVITE request sent towards the terminating client based on the chat fallback mechanism that is supported.
- When handling an SIP INVITE request for a 1-to-1 Chat session, the messaging server in the terminating network shall add either the message revocation or the network interworking feature tag defined in section 3.2.3.8.1 in the Contact header field in every 200 OK to the SIP INVITE request sent towards the originating client.

NOTE: For networks with RCS Standalone messaging service enabled that enable only receiving 1-to-1 chat messages (i.e. CHAT AUTH is set to 0 and GROUP CHAT AUTH is set to 1) and that choose to interwork with Service Providers that enable only RCS 1-to-1 Chat service (option 2) by deploying an RCS Standalone message to RCS 1-to-1 Chat session interworking function, the above procedure can be fulfilled by the interworking function.

The terminating network shall always return a SIP 200 OK response to a SIP INVITE request for 1-to-1 chat session since the network takes responsibility for delivering the message in the best way possible regardless of the connectivity status of the client.

If the terminating network supports Message Revocation, this shall be implemented as defined in this document.

3.2.3.8.5 Network indicates delivery as SMS

When a network supports interworking as per section 3.2.3.4 and it delivers the message through interworking to SMS, the network shall generate an interworking notification as defined in Appendix O of [GSMA PRD-RCC.11] when such a notification was requested in the IMDN Disposition-Notification CPIM header of the message that was interworked. In that case, the network shall not generate a Delivery disposition notification. If IMDN Disposition-Notification CPIM header did not include a request for an interworking notification, the network shall generate a Delivery Notification instead.

NOTE: Since a client will always have requested an interworking notification, this requirement to generate a regular Delivery notification is intended to support clients supporting previous versions of this specification.

3.2.3.8.6 Procedures for Client SMS Fall-back

If according to the procedures defined in section 3.2.3.8.1 the client runs a timer based on the CHAT REVOKE TIMER client configuration parameter for messages in a conversation, and the client receives "delivery" disposition notifications for all messages in the conversation, then the client shall stop the timer.

The client shall start a timer based on the CHAT REVOKE TIMER client configuration parameter for the next message sent in the conversation considering the message revocation capability indicated in accordance with the procedures defined in section 3.2.3.8.1.

1. If the timer based on the CHAT REVOKE TIMER client configuration parameter is running for at least one conversation and if

    a) the client registers in IMS successfully due to a previous de-registration (e.g. due to user setting or data-off) and if the value of the RECONNECT GUARD TIMER configuration parameter defined in sections A.1.3 and A.2.4 is not set to "0", then the client shall start the reconnection guard timer with the value provided in the RECONNECT GUARD TIMER configuration parameter defined in sections A.1.3 and A.2.4, or
    b) the client re-connects to the Proxy Call Session Control Function (P-CSCF) due to a previous loss of connection to the P-CSCF and if the value of the RECONNECT GUARD TIMER configuration parameter defined in sections A.1.3 and A.2.4 is not set to "0", then the client shall start the reconnection guard timer with the value provided in the RECONNECT GUARD TIMER configuration parameter defined in sections A.1.3 and A.2.4.
    c) The reconnect guard timer shall start if

        i. a success response is received from the network for an initial registration or re-registration resulting from the client procedures to re-connect to the P-CSCF, otherwise
        ii. at the time of P-CSCF connection re-gain.

2. If the timer based on the CHAT REVOKE TIMER client configuration parameter expires, then processing commences with step 3.

3. The client shall check whether the reconnection guard timer is running. If yes, then processing commences with step 5, otherwise processing commences with step 4.
4. If the client

    a) is not registered in IMS due to missing data connection (e.g. data off, loss of connection to the P-CSCF and registration expired), then the client shall wait until data connection is regained.
    b) if the data connection is regained then the client shall send an initial registration as per procedures of section 2.4.
    c) has stored a valid IMS registration but it has previously detected a loss of connection to the P-CSCF, then the client shall wait until data connection is regained.
    d) If the data connection is regained

        i. and the IMS registration is valid (e.g. registration is not expired, the client IP address did not change, access network did not change), then the client shall send a re-registration as per procedures of section 2.4.
        ii. and the IMS registration is not valid (e.g. registration is expired, the client IP address did change), then the client shall send an initial registration as per procedures of section 2.4.

    e) If the initial registration or the re-registration is successful, then

        i. if the value of the RECONNECT GUARD TIMER configuration parameter defined in section A.1.3 and A.2.4 is set to "0" then processing commences with step 6.
        ii. otherwise, the client shall start the reconnection guard timer with the value provided in the RECONNECT GUARD TIMER configuration parameter defined in sections A.1.3 and A.2.4. Processing commences with step 5.

    f) otherwise, in all other cases where the client is registered in IMS, processing commences in step 6.

5. If the reconnection guard timer is running and

    a) if "delivery" disposition notifications have been received for all messages in the conversation, then the reconnection guard timer shall be stopped by the client. The client shall stop the message fall-back processing.
    b) if the client detects a loss of connection to the P-CSCF or it is de-registered, the reconnection guard timer is stopped and processing commences with step 4
    c) If the reconnection guard timer expires, the processing commences with step 6.

6. The client shall verify whether connectivity for sending SMS messages exists. If connectivity for sending of SMS messages exists, then processing commences with step 7. Otherwise, the client

    a) shall wait until connectivity for SMS is regained again.

b) If the connection to the P-CSCF is lost or the client is de-registered from IMS while waiting for connectivity for sending of SMS messages, the client shall stop waiting for SMS connectivity and continue processing in step 4.
c) If "delivery" disposition notifications have been received for all messages in the conversation, then the client shall stop waiting for SMS connectivity. The client shall stop the SMS fall-back processing.
d) Once connectivity for sending SMS is regained, then processing commences with step 7.

7. If a user authorisation is required for the SMS fall-back for this conversation, then the client shall invoke a user interaction. Otherwise processing commences with step 8.

   a) If the client receives "delivery" disposition notifications for all messages in the conversation, then the client shall abort the user interaction and stop the message fall-back processing.
   b) If the user interaction results in the user authorisation of the SMS fall-back, then the procedure commences with step 4. The client shall retain the user authorisation of the SMS fall-back for the subsequent processing, i.e. the client shall not invoke the user interaction again.
   c) If the user interaction results in rejection of the message fall-back, then the client shall stop message fall-back processing.

8. The client shall create a Message Revoke request for the oldest chat message of the conversation for which no "delivery" disposition notification has been received and which has not been processed for SMS fall-back. The client shall send the Message Revoke request to the network. Message revocation shall be implemented as defined in section 3.2.3.8.2. If the value of the configuration parameter CFS TRIGGER defined in sections A.1.3 and A.2.4

   a) is set to "1", then

      i. if the Message Revocation request fails due to loss of connection to the P-CSCF, then the client shall continue processing with step 4.
      ii. if the client receives a success response (200 OK) to the Message Revoke request, then the client shall
      iii. start an operation timer to supervise the processing of the Message Revocation, otherwise
      iv. not send a fall-back SMS and consider the chat message as processed for SMS fall-back. The client shall continue with the procedure in step 8 until all messages in the conversation have been processed.
      v. If the client receives a "delivery" disposition notification for the message to be revoked, it shall stop SMS fall-back handling.
      vi. If the client receives a Message Revocation response from the network with a "success" result, then the client shall stop the operation timer and shall send the fall-back SMS following the procedures for fall-back according to the type of message (e.g. chat message, file transfer).

vii. If the client receives a Message Revocation response from the network with a "failure" result, then the client shall stop the operation timer and shall not send the fall-back SMS and consider the chat message as processed for SMS fall-back. The client shall continue with the procedure in step 8 until all messages in the conversation have been processed.
viii. If the client detects a loss of connection to the P-CSCF or the client is de-registered from IMS (e.g. due to user settings), then the client shall stop the operation timer and shall send the fall-back SMS following the procedures for fall-back according to the type of message (e.g. chat message, file transfer). If there is at least one message in the conversation for which no "delivery" notification has been received and which has not been processed for SMS fall-back the client shall continue processing with step 4, otherwise is shall stop SMS fall-back handling.
ix. If the operation timer expires then the client shall send the fall-back SMS following the procedures for fall-back according to the type of message (e.g. chat message, file transfer).
x. If submission of the SMS to the network fails (e.g. no SMS connectivity), then the client shall suspend the processing of SMS fall-back and apply the client procedures for the handling of failed SMS message submissions.
xi. If the submission of the SMS is confirmed by the network with a success response, then the client shall consider the chat message as processed for SMS fall-back. The client shall trigger SMS latching and cache this status for the contact. The client shall continue with the procedure in step 8 until all messages in the conversation have been processed.
xii. If the submission of the SMS is rejected by the network with a failure response, then the client shall stop processing of the SMS fall-back and apply the client procedures for the handling rejected SMS message submissions

b) is set to "0" or is not present then the client shall send the fall-back SMS following the procedures for fall-back according to the type of message (e.g. chat message, file transfer) and the Message Revoke request at the same time.

i. If the Message Revoke request fails due to loss of connection to the P-CSCF and there is at least one message in the conversation for which no "delivery" notification has been received and which has not been processed for SMS fall-back, then the client shall continue processing with step 4, otherwise the client shall stop processing of SMS fall-back.
ii. If submission of the SMS to the network fails (e.g. no SMS connectivity), then the client shall suspend the processing of SMS fall-back and apply the client procedures for the handling failed SMS message submissions.

iii. If the submission of the SMS is confirmed by the network with a success response, then the client shall consider the chat message as processed for SMS fall-back. The client shall trigger SMS latching and cache this status for the contact. The client shall continue with the procedure in step 8 until all messages in the conversation have been processed.
iv. If the submission of the SMS is rejected by the network with a failure response, then the client shall stop processing of the SMS fall-back and apply the client procedures for the handling rejected SMS message submissions.
v. The outcome of the Message Revocation operation does not alter the processing requirements of the client with regard to SMS fall-back. However, the client may need to inspect the outcome of the Message Revocation operation to satisfy the requirements for the presentation of the message status to the user.

3.2.3.9 Message display and message store

All messages are stored in the participating devices, together with a time indication and an appropriate indication of the sender and the receiver of each message. This time indication shall be obtained from the CPIM *DateTime* header for received messages. Since according to section 3.2.3.1 these values should be set by the Messaging Server, this allows for a correct time based indication for those messages without depending on the device's own clock which may not have been set correctly. For sent messages however, the only clock available at transmission time is the device's own clock.

However, it is Messaging Server responsibility to deliver messages in the correct order, so the RCS Client is able to rely on the reception order to interleave the incoming and outgoing messages. Please note that the shown message time at the UX should be based on the network time (i.e. the CPIM DateTime header, when available) in order to correctly display the time of store and forwarded messages.

When a Common Message Store is available for the user, the messages are synchronised with the Message Store Server as specified in section 4.1.

When the storage limit is reached, deletion might occur on a first in/first out (FIFO) queue policy. It is open to OEM criteria how to implement other opt-in deletion mechanisms (e.g., ask always, delete always, delete any conversation/message from specific contacts, etc.).

3.2.3.10 Extended Messaging

When Extended Messaging features apply, an RCS client sender client shall follow the procedures described in section 3.2.8.

3.2.4 Group Chat

For Group Chat, gRPC-configured RCS clients shall follow the procedures defined in section 5.3 of [GSMA PRD-RCC.30]).

SIP-configured RCS clients shall follow the procedures in this section.

3.2.4.1 Overview

The technical realisation of Group Chat is based on the CPM Ad-hoc Group and the CPM Long-Lived Group session as defined in [GSMA PRD-RCC.11] with the additional requirements and clarifications defined in this document.

The support of CPM Long-Lived Group session is mandatory for the Messaging Server. This requires the conference focus of the Messaging Server to keep the Group Chat participant and meta-information beyond an active Group Chat session. The conference focus shall keep the participant information and meta-information for idle Group Chats for a minimum time as defined in [GSMA PRD-RCC.71].

The support of CPM Long-Lived Group session is mandatory for the client. The value of the Conversation-ID header defined in [GSMA PRD-RCC.11] provides the unique ID of an individual RCS Group Chat. It shall be used by the client to associate Group Chat sessions, Chat messages, participant and meta-information with the corresponding Group Chat conversation.

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] shall be followed.

3.2.4.2 Authorisation of Group Chat

The client is authorised to offer the Group Chat service to the user if the configuration parameter GROUP CHAT AUTH as defined in section A.1.3 is set to "1".

If the client is not authorised for Group Chat and it receives an invitation for a Group Chat session, then it shall reject it with a SIP 488 Not Acceptable Here response.

3.2.4.3 Initiation of the Group Chat

The client shall offer the user to initiate a Group Chat only with contacts being capable of the Chat service as defined in section 2.6.1.3.

NOTE: The caching of addresses as defined in section 2.5.2 and 2.5.3.1 in combination with the fact that the Chat capability of the contact must be known will result in the client always providing the addresses of invited participants in international format.

For the establishment of a Group Chat the client shall request the user to select at least 2 contacts capable of the Chat service. The client shall allow the user to select a number of Chat service capable contacts up to the limit defined in the configuration parameter MAX_AD-HOC_GROUP_SIZE, reduced by one (the initiator itself), as defined in section A.1.3.

Extending of a 1-to-1 Chat to a Group Chat as defined in [GSMA PRD-RCC.11] is not applicable for this version of the RCS specification.

The client shall allow the user to assign Group Chat meta-information such as subject and an icon to the Group Chat. The subject shall be conveyed to the invitees during Group session establishment as defined in [GSMA PRD-RCC.11]. The icon shall be conveyed to

the invitees via the procedures for managing and receiving Group Chat meta-information described in section 3.2.4.7 and 3.2.4.8 respectively.

Prior to initiation the client shall assign to the Group Chat a unique value for the Contribution-ID and Conversation-ID headers defined in [GSMA PRD-RCC.11]. Both Contribution-ID and Conversation-ID headers shall be set to the same value.

The client shall derive the Request-URI of the SIP INVITE from the configuration parameter CONF-FCTY-URI defined in section A.1.3.

The client shall initiate a Group Chat following the definitions of [GSMA PRD-RCC.11] for the initiation of a CPM Group session for a CPM Ad-hoc Group.

Once a 200 OK response is received for the Group Chat invitation, the client shall store the values of the Conversation-ID and Contribution-ID SIP headers and the Group Chat Session Identity from the Contact header along with the subject of the Group Chat.

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] for initiation of Group Chat shall be followed.

3.2.4.4 Invitation to a Group Chat

A client is invited to a new Group Chat if it receives a session invitation for a CPM Group session for a CPM Ad-hoc group as defined in [GSMA PRD-RCC.11] with a value of the Conversation-ID header which is not known by the client. The client shall accept the invitation to the Group Chat in accordance with the value of the configuration parameter IM SESSION AUTO ACCEPT GROUP CHAT as defined in section A.1.3.

Once the invitation to the Group Chat is accepted, the client shall store the values of the Contribution-ID and Conversation-ID SIP headers and the Group Chat Session Identity from the Contact header along with the subject of the Group Chat.

The SDP of the SIP INVITE request and response shall not be compressed even if SIP Content-Compression is supported by the client.

3.2.4.5 Content type negotiation

When establishing CPM Group sessions, the clients and the entities of the Messaging Server shall negotiate the supported content types for the CPM Group session as defined in [GSMA PRD-RCC.11] with the following additional clarifications and requirements:

- In the SDP of the SIP INVITE request and response, the a=accept-types attribute shall include *message/cpim*, and the content type for the conference-info object if supported.
- In the SDP of the SIP INVITE request and response, the a=accept-wrapped-types attribute shall only include *text/plain, message/imdn+xml and application/im-iscomposing+xml*.
- If File Transfer is supported (see section 3.2.5) then the a=accept-wrapped-types attribute shall also include *application/vnd.gsma.rcs-ft-http+xml*.
- If Geolocation PUSH is supported (see section 3.2.6.2), then the a=accept-wrapped-types attribute shall also include *application/vnd.gsma.rcspushlocation+xml*.

- If Rich Text Message is supported (see section 3.2.10), then the a=accept-wrapped-types attribute shall also include *application/vnd.gsma.richtextmessage+proto*.
- To transfer any other content (e.g. multimedia content) during a chat, File Transfer is used.
- The client and the Messaging Server shall take the procedures defined in section 3.2.5.2 for File Transfer into account.
- The client and the Messaging Server shall take the procedures defined in section 3.2.6.2 for Geolocation PUSH into account.

3.2.4.6 Adding participants to a Group Chat

The client shall allow the user to add additional participants to a Group Chat, if they are capable to support Chat and if they are not member of the Group Chat already.

NOTE: The caching of addresses as defined in section 2.5.2 and 2.5.3.1 in combination with the fact that the Chat capability of the contact must be known will result in the client always providing the addresses of invited participants in international format.

This includes participants listed in the most recent conference-info document received by the client if their endpoint status is set to "disconnected" with disconnection method set to "departed" or "failed".

The client shall allow the user to add more participants to a Group Chat if the value of the maximum-user-count element is greater than the value of the user-count element of the latest conference-info document defined in [RFC4575] and received by the client as per procedures defined in section 3.2.4.8. The client shall allow the user to only add a number of additional contacts that does not exceed the value of the maximum-user-count element.

Based on the client procedure above it is mandatory for the Messaging Server to provide the values of maximum-user-count and the user-count elements in all conference-info documents.

The client shall add new participants to a Group Chat in accordance with the procedures for the inviting other participants to a CPM Group session as defined in [GSMA PRD-RCC.11].

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] for adding of participants to a Group Chat shall be followed.

3.2.4.7 Managing Group Chat meta-information

Group Chat meta-information comprises additional characteristics of a Group Chat which can be managed by the participants.

The client and the Messaging Server shall support the procedures for the management of

- the subject
- the icon, and
- user roles

via the procedures defined in [GSMA PRD-RCC.11] for the Group Session Data Management.

For the request to change the icon, the client shall use the request operation to set the icon using the <icon-uri> element as defined in [GSMA PRD-RCC.11]. The use of the <file-info> element of the <file> element to set the icon is not applicable for this version of the RCS specification.

The client shall follow the procedures specified in section 4.2.2.1 to upload a new icon. Upon receiving the XML, the client shall extract the URL from <file-info> element and include it in the <icon-uri> element of the Group chat meta-information. The Controlling Function shall store the file locally upon receiving a request to change the icon. With this process, the Controlling Function shall store the file without an expiry and shall rewrite the icon URI to point to the file stored locally.

Upon receiving the <icon-url> change, the client shall download the file as described in section 6.8 in [GSMA PRD-RCC.11].

NOTE: The impact to the File Localization Function is for further study.

In addition, the client and the Messaging Server shall support the procedures related to the indication of the Group Chat policy as defined in [GSMA PRD-RCC.11].

If for a Group Chat session, the Group Chat Session Data Management is not available as defined in [GSMA PRD-RCC.11], then

- the client shall store the value of the subject assigned at the time of initiation of the Group Chat persistently. The client shall use the value of the subject in SIP INVITE and SIP REFER requests to restart a Group Chat or when adding participants to a Group Chat respectively. The client shall ignore the value of the subject header when receiving invitations for the stored CPM Group session.
- the client shall not offer functions related to the management of icon and user roles to the user.

The procedures for the client and the Messaging Server related to the user experience of user roles and the group policies are defined in [GSMA PRD-RCC.71].

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] for managing Group Chat metadata shall be followed.

3.2.4.8 Receiving Group Chat participant and meta-information

The client shall determine the Group Chat participant information (i.e. the list of participants) and Group Chat meta-information by the following means:

- During an active Group Chat session via
    - the conference-info document via MSRP SEND requests sent by the network automatically at the time of session set-up or after participant information or meta information has changed, or

  - via SIP NOTIFY as a result of the client's subscription to the conference event package

  as defined in [GSMA PRD-RCC.11].
  The policy for the client to receive Group Chat participant and meta-information via MSRP is determined by the network based on the negotiation of the content type for the conference-info object as defined in [GSMA PRD-RCC.11]. If the network supports delivery of Group Chat participant and meta-information via MSRP, then the client shall not subscribe to the conference event package. Otherwise, if the network does not support delivery of Group Chat participant and meta-information via MSRP, then the client shall subscribe to the conference event package and in this case, the network will not send any meta-information via MSRP.
  If SIP Content-Compression is supported by the client, the client should include an Accept-Encoding parameter set to GZIP in the SIP SUBSCRIBE request. A network may send the bodies of the SIP NOTIFY request using GZIP encoding if that parameter is provided.

- Outside of an active Group Chat session, if enabled, via the Group State Objects or Conference Information Objects stored in the Common Message Store using the procedures defined in section 4.1.

The client shall store the latest list of participant addresses locally.

If the client receives an update of participant or meta-information, then it shall compare the received list of participants or the meta-information event with the stored list of participants and meta-information and shall notify the user about changes in the participant list or meta-information only as a result of the comparison.

For the reception of icons assigned to the Group Chat, the client shall support both the <icon-uri> and the <file-info> element of the conference-info document extension defined [GSMA PRD-RCC.11].

To download the icon file using the URL contained in the <icon-uri> element, the client shall invoke the file download procedure as defined in section 4.2.3.

For the initiation of the file download procedure, the client shall provide the values of the download parameters defined in section 4.2.3 as follows:

Parameter	Value
url	URL contained in the <icon-uri> element
id	n/a
op	Group Chat Session Identity of the Group Chat, to which the icon applies
ci	Conversation-ID of the Group Chat to which the icon applies

Table 20: Group Chat Icon download parameters

3.2.4.9 Closing Group Chat sessions

3.2.4.9.1 Group Chat session idle time

The conference focus of the Messaging Server shall take responsibility to close idle Group Chat sessions. In this case, the participants of the Group Chat remain members of the Group Chat. A participant can restart the Group Chat session at any time using the procedure for the restart of a Group Chat. The conference focus of the Messaging Server shall end the Group Chat session using the procedure for the CPM Group Chat session ending request for session inactivity defined in [GSMA PRD-RCC.11]. The maximum allowed idle time in the conference focus shall not be larger than 300 seconds. When measuring the idle time in the conference focus, MSRP messages for disposition notifications shall not be taken into account.

A Messaging Server not acting as the conference focus may also monitor Group Chat sessions for becoming idle. Such a Messaging Servers shall apply a significantly larger idle timer value than the timer value defined for the conference focus.

The client can leave the Group Chat session involuntarily, e.g. at the time to shut down or power off. In this case, the client or an entity in the network detecting the event shall generate a SIP BYE request including a Reason Header field set to SIP and the protocol-cause set to a value other than 200. It is recommended to use a Reason header field with the protocol set to SIP and the protocol-cause set to 503 (e.g. SIP;cause=503;text="Service Unavailable") in accordance with the definitions in [3GPP TS 24.229] for bearer loss detected by the P-CSCF.

3.2.4.9.2 Group Chat termination Service Provider policies

The conference focus of the Messaging Server may close a Group Chat permanently based on Service Provider policy if

- less than the minimum number of participants is reached or
- the Group Chat has not been used by the participants for a time defined by the Service Provider.

If a Service Provider Policy applies and the condition for Group Chat permanent termination is met, then the conference focus of the Messaging Server shall remove all participants of the Group Chat using the procedure for the CPM Group Chat session ending request defined in [GSMA PRD-RCC.11]. If no Group Chat session exists, then the conference focus of the Messaging Server shall create one first, using the procedure to create a session with the participant(s) as defined in section 9.2.10 of [GSMA PRD-RCC.11].

If the client receives the CPM Group Chat session ending request, then it shall apply the UX procedures for a Group Chat where the user is no longer participating.

3.2.4.10 Restart of the Group Chat session

3.2.4.10.1 Normal Procedures

If a Group Chat session was closed due to inactivity and a client requires an active Group Chat session for the processing of user requests, then the client shall restart the Group

Chat session using the procedure defined in [GSMA PRD-RCC.11] for re-joining a CPM Group Chat session. The client shall use the stored values of the Group Chat Session Identity, Contribution-ID and Conversation-ID to re-join the Group Chat session.

The Group Chat is restarted if the client receives a 200 OK response in accordance with the procedures defined in [GSMA PRD-RCC.11].

If the SIP INVITE for the restart of the Group Chat session fails with a SIP 404 Not Found response in accordance with the procedures in [GSMA PRD-RCC.11], the Service Provider closed the Group Chat based on Service Provider policy in accordance with the definitions in section 3.2.4.9.2. The client shall apply the UX procedures for a Group Chat where the user is no longer participating.

If the SIP INVITE for the restart of the Group Chat session fails with a SIP 403 Declined response including a Reason header in accordance with the procedures in [GSMA PRD-RCC.11], then the user is not authorised to restart the Group Chat. The client shall apply the user interface procedures for a Group Chat where the user is no longer participating.

If a SIP response other than the ones defined in [GSMA PRD-RCC.11] is received by the client, then the client shall not alter the status of the Group Chat. The client may retry the Group Chat restart.

A client receiving an invitation to a Group Chat session with a known Conversation-ID shall accept the request automatically, unless there is a pending user request to leave the Group Chat.

3.2.4.10.2 Race Conditions

Since a Group Chat can be restarted by two participants simultaneously, race-conditions exist between rejoin requests coming from the client and SIP INVITE request originated by the Controlling Function. As the middle element, most of these situations will be detected by the Messaging Server Participating Function that shall handle these situations as follows:

1. For the case where the Messaging Server Participating Function receives an incoming SIP INVITE request from the client for a Group Chat for which a SIP INVITE request was already sent (matching shall be done based on the Group Chat Session Identity) (i.e. the INVITE requests have crossed between the client and the Participating Function):

    a) If no session is established yet with the Controlling Function (see also section 3.2.4.15), the Messaging Server Participating Function shall forward this INVITE request from the client to the conference focus and handle the SIP INVITE request from the client as a regular Back-to-Back User Agent (B2BUA) with this session setup to the Controlling Function.
    b) Otherwise, the Messaging Server Participating Function shall accept the SIP INVITE request from the client, establish the MSRP channel and forward any messages and notifications received from the client in the already established MSRP session with the Controlling Function.

Messages and notifications received from the Controlling Function shall only be forwarded in the MSRP channel to the client that was last to be established. Based on local policy, the Messaging Server Participating Function shall terminate the unused session by sending in the corresponding SIP dialog a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol_cause set to 480 (e.g. SIP;cause=480;text="bearer unavailable").

This means, that in all cases where such a race condition occurs temporarily, two sessions are established between the client and the Participating Function and only one between the Controlling Function and the Participating Function. Between the client and Participating Function, only the MSRP session that was last to be established shall be used.

2. For the case where the Messaging Server Participating Function receives an incoming SIP INVITE request from the Controlling Function for a Group Chat for which a SIP INVITE request was already sent (matching shall be done based on the Group Chat Session Identity) to the Controlling Function (i.e. the INVITE requests have crossed between the Controlling Function and the Participating Function), the Messaging Server Participating Function may either

   a) Forward this INVITE request to the client, or
   b) Accept both the session from the Controlling Function and the rejoin request from the client and link both dialogs as a B2BUA. In that case when the Controlling Function accepts the INVITE request, the Participating Function shall establish the MSRP session and only use the last MSRP channel to be established until either the Controlling Function closes one of the sessions, closes the entire chat or the user leaves the Chat. In the last case, the Messaging Server Participating Function shall send the corresponding SIP BYE request in both sessions.

Also, the Controlling Function shall accept a rejoin request received from a participant for which there was an outstanding INVITE request. It shall ensure that only one session is used and that messages from the participant are not returned in the other session. To achieve this, it is recommended to only send messages in the last MSRP channel to be established. Once it has received messages or notifications over that connection it may close the other session by sending in the corresponding SIP dialog a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol_cause set to 480 (e.g. SIP;cause=480;text="bearer unavailable").

3.2.4.11 Leaving a Group Chat

If the user requests the client to leave a Group Chat, and a Group Chat session exists, then the client shall send a SIP BYE request following the procedure defined in [GSMA PRD-RCC.11] for leaving a Group Chat session.

If the user requests the client to leave a Group Chat, and a Group Chat session does not exist, the client shall restart the Group Chat session first as described in section 3.2.4.10, and then follow the procedure defined in [GSMA PRD-RCC.11] for leaving a Group Chat session.

The client may also leave a Group Chat at the time Group Chat session restart by rejecting the Group Chat invitation with a 603 Decline response.

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] for leaving a Group Chat shall be followed.

3.2.4.12 Removal of a participant

The client and the Messaging Server shall support the procedures for a participant to remove another participant as defined [GSMA PRD-RCC.11]. The client shall offer the user removal of a participant only if it is entitled by the participant removal policy and the user's own role as indicated by the Messaging Server. If the participant removal policy is absent in the Group Chat, the client shall not offer the user the possibility to remove other participants.

If E2EE is enabled on the Group Chat, the additional procedures in [GSMA PRD-RCC.16] for removal of participants from a Group Chat shall be followed.

3.2.4.13 Procedures in the Messaging Server for the removal of a user's IMS identity and profile

If the Service Provider removes the IMS identity and the profile of a user, then the Messaging Server shall invoke the procedures defined in [GSMA PRD-RCC.11] for the termination of all CPM sessions due to administrative reasons.

In addition to the procedures defined in [GSMA PRD-RCC.11] for the creation of a CPM Group session with a participant the conference focus shall, if it receives a SIP 404 Not Found response to the SIP INVITE, remove the corresponding participant from the participant list of the Group Chat. The conference focus shall inform the other participants via an update of the participant information using the procedures described in section 3.2.4.8. The elements of the removed participant's user endpoint in the conference-info document shall be set as follows:

- Status of the user endpoint element is set to "*disconnected*",
- Disconnection-method of the user endpoint element is set to "*departed*".

3.2.4.14 Delivery and Display notifications

The client and the Messaging Server shall apply the procedures for Delivery and Display notifications as defined in [GSMA PRD-RCC.11] for the handling of disposition notifications for Group Chat.

The additional requirements for the composition of disposition notifications defined in section 3.2.4.17 shall apply also, if disposition notifications are sent via SIP MESSAGE.

3.2.4.15 Support of Store and Forward

The Messaging Server and the client shall support Store and Forward of Chat messages, and Group Chat participant and meta information via the procedures defined in [GSMA PRD-RCC.11] for deferred CPM Group session delivery.

3.2.4.16 Multidevice handling and the Common Message Store

If Multidevice and the Common Message Store is supported by the Service Provider the Messaging Server shall support forking of Group Chat sessions to multiple clients as defined in in [GSMA PRD-RCC.11] for CPM Group session establishment. The Messaging Server shall act as a Back-to-Back User Agent (B2BUA) in this case. Delivery of sessions and messages to non-CPM devices as defined in [GSMA PRD-RCC.11] is not applicable for Group Chat.

Multiple clients of a user will receive the Group Chat participant and meta-information during a group Chat session as described in section 3.2.4.8.

In addition, based on Service Provider Policy the Messaging Server may support direct delivery of messages in a CPM Group session as defined in in [GSMA PRD-RCC.11]. For the client the support of direct delivery of messages is mandatory.

If the Common Message Store is deployed by the Service Provider then the client shall retrieve the conversation history including participant and meta-information using the procedures described in section 4.1.

3.2.4.17 Media plane handling

In addition to the definitions in [GSMA PRD-RCC.11], client and servers shall apply the following procedures for media plane handling in a Group Chat session.

To prevent revealing the user identity when transmitted over unprotected links, the client should set the value of the CPIM To header for Chat messages in a Group Chat to *sip:anonymous@anonymous.invalid*. The client shall set the value of the CPIM From header to its public user identity and include the user's display name.

For Delivery and Display notifications, the client shall set the value of the CPIM To header to the identity of the original sender of the message, taken from the value of the CPIM From header of the chat message it relates to.

The originating Messaging Server shall always set the value of the CPIM DateTime header in the chat messages it receives. The originating Messaging Server shall also set the CPIM DateTime header and IMDN DateTime element in notifications. In both cases, the Messaging Server shall overwrite any DateTime information provided by the client. A client receiving these requests should therefore rely on these headers containing the correct time rather than on locally available time information.

The maximum size of a text Chat message in bytes that a user can enter in the Group Chat is controlled via the configuration parameter MAX SIZE IM as defined in section A.1.3.

3.2.4.17.1 Extended Messaging

When Extended Messaging features apply, an RCS client sender client shall follow the procedures described in section 3.2.8.

3.2.4.18 Group Chat Session Identity

The Group Chat Session Identity is conveyed during the establishment of SIP session as the value of the Contact header. Intermediate nodes in the path between the Messaging

Server and the client shall transparently forward the contact URI if the Contact header field contains the “isfocus” feature tag. For IMS entities, this behaviour is applied in accordance with the definitions in [3GPP TS 24.229]. For CPM entities this behaviour is applied in accordance with the definitions in [GSMA PRD-RCC.11].

3.2.4.19 Connection Model for Subscriptions from Participating Function to Controlling Function

The Messaging Server Participating Function acts as a subscriber to the conference event package of a Group Chat controlled by the Messaging Server Controlling Function. During an active subscription, the Messaging Server Controlling Function notifies the Participating function about changes of the Group State status and participant list.

Messaging Server Participating Functions and Controlling Function may reside in different Service Provider networks. The Connection Model applied on the interface between the two functions shall minimise the impact on the Controlling Function coming from the Participating Function Service Provider's network topology and service provisioning. This is achieved by limiting the number of active subscriptions per participant in a given Group Chat to "1" based on the Connection Model below.

If the Participating Function initiates a new subscription for a Group Chat Session Identity on behalf of a participant, the Controlling Function shall accept the request provided that the Group Chat Session Identity exists and the participant is authorised to subscribe to it, i.e. it is a participant in the Group Chat.

At the time of acceptance, if the Controlling Function has another subscription active for the same focus session identity and participant combination then the older subscription should be terminated as defined in [RFC6665].

The Service Provider of the Participating Function shall ensure that the connection model towards the Controlling Function does not restrict the service provided to its users. For Service Providers offering Group Chat for Multidevice the application of a Back-to-Back User Agent (B2BUA) for subscriptions in the Messaging Server Participating Function is mandatory.

3.2.5 File Transfer

3.2.5.1 Overview

This section describes the File Transfer mechanism that is based on

- the originating client uploading a file and optionally a thumbnail file to the HTTP Content Server,
- the HTTP Content Server returning a HTTP Content Server response body to the client containing the file meta data,
- the originating client processing the received HTTP Content Server response body to create the File Transfer message body,
- use of Standalone Messaging, 1-to-1 Chat and Group Chat procedures described in sections 3.2.2, 3.2.3 and 3.2.4 to send the File Transfer message body to the recipient(s),

- the terminating client extracting the file meta data from the File Transfer message body and downloading the file(s) from the HTTP Content Server, optionally via a Localisation Function.
- If E2EE is enabled for the 1-to-1 or Group Chat, the additional procedures in [GSMA PRD-RCC.16] shall be followed for File Transfers.

3.2.5.2 Configuration and Capability Discovery

The client is authorised for the use of the File Transfer if the value configuration parameter FT AUTH defined in section A.1.4 is set to "1".

If the client is authorised for File Transfer, then the client shall advertise the support of File Transfer via

- the IARI media feature tag for File Transfer added in the IMS registration in accordance with the definitions in section 2.4.4,
- the Capability Discovery of File Transfer as defined in section 2.6.1.3,
- the "+g.3gpp.iari-ref" media feature tag value "urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp" added in the Contact header of the SIP INVITE requests and the 200 OK responses for sessions related to Group Chat.
- the indication of the content type *application/vnd.gsma.rcs-ft-http+xml* in the SDP a=accept-wrapped-types attribute of the SIP INVITE requests or the 200 OK response for sessions related to 1-to-1 Chat, Large Message Mode Standalone Messaging and Group Chat.

Otherwise, if File Transfer is not authorised, then the client shall not advertise the support of File Transfer as described above.

The Messaging Server supporting File Transfer shall

- insert in the "+g.3gpp.iari-ref" media feature tag the value "urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp" in the Contact header of the SIP INVITE request or the 200 OK response for sessions related to Group Chat.
- indicate the content type *application/vnd.gsma.rcs-ft-http+xml* in the SDP a=accept-wrapped-types attribute of the SIP INVITE requests or the 200 OK response for sessions related to 1-to-1 Chat, Large Message Mode Standalone Messaging and Group Chat.

For the client File Transfer is available in a 1-to-1 Chat conversation, if:

- the *application/vnd.gsma.rcs-ft-http+xml* content type is indicated in the a=accept-wrapped-types attribute from the messaging server during the SDP negotiation and
- the recipient is known to support File Transfer based on Capability Discovery.

For the client File Transfer is available for 1-to-1 Standalone Messaging, if

- the recipients are known to support the File Transfer capability based on a Capability Discovery.

For the client File Transfer is available for a Group Chat conversation, if

- the *application/vnd.gsma.rcs-ft-http+xml* content type is indicated in the a=accept-wrapped-types attribute by the messaging server in SDP, and
- the "+g.3gpp.iari-ref" media feature tag value "urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp" in present the Contact header of the SIP INVITE or SIP 200 OK response received from the Messaging Server for set-up of a Group Chat session.

The Messaging Server shall not forward a message with a File Transfer message body to a recipient client which did not indicate the content type *application/vnd.gsma.rcs-ft-http+xml* in the SDP a=accept-wrapped-types attribute during session set-up.

3.2.5.3 File transfer procedure

3.2.5.3.1 Sender procedures

To initiate a File Transfer, the sending client shall first upload the file and, if applicable, the thumbnail to the HTTP Content Server via the procedures defined in section 4.2.2.

If the upload was successful as per definitions in section 4.2.2.1, the sending client shall process the HTTP Content Server response body and add the additional *file-disposition* attribute to the file-info element of the main file. This optional attribute provides functionality similar to the File-Disposition SDP attribute in file transfer via MSRP which is described in [RFC5547] and can take the same values (i.e. *render* and *attachment*). If the attribute is not included, the receiver will interpret the value as *attachment*. A non-normative File Transfer message body content is shown in Table 21.

```
<?xml version="1.0" encoding="UTF-8"?>
<file xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttp"
        xmlns:x="urn:gsma:params:xml:ns:rcs:rcs:up:fthttpext">
        <file-info type="thumbnail">
                <file-size>[thumbnail size in bytes]</file-size>
                <content-type>[MIME-type for thumbnail]</content-type>
                <data url = "[HTTP URL for the thumbnail]" until = "[validity of the thumbnail]"/>
        </file-info>
        <file-info type="file" file-disposition="[file-disposition]">
                <file-size>[file size in bytes]</file-size>
                <file-name>[original file name]</file-name>
                <content-type>[MIME-type for file]</content-type>
                <data url = "[HTTP URL for the file]" until = "[validity of the file]"/>
                <x:branded-url>[alternative branded HTTP URL of the file]</x:branded-url>
        </file-info>
</file>
```

Table 21: File Transfer message body content

The client shall send the File Transfer message body to the receiver(s) in the CPIM message body via the applicable transport service. The client shall use the content type defined for the File Transfer message body, i.e. *application/vnd.gsma.rcs-ft-http+xml*.

If sending to a single recipient, then there are the following possible scenarios:

- If there is a 1-to-1 chat session established with the user and File Transfer is supported in the session as described in section 3.2.5.2, the session shall be reused to convey the File Transfer message body content in a Chat message.
- If there is no 1-to-1 Chat session established, then the client shall apply the technology selection for 1-to-1 messaging as defined in section 3.2.1. If the client establishes a session to transmit the File Transfer message body via Standalone Messaging, then the client shall include a dedicated Accept-Contact header field that includes the File Transfer IARI tag defined in section 2.6.1.3 along with *require* and *explicit* parameters.

If sending to multiple recipients, there are the following possible scenarios:

- If the file is to be transferred in an existing Group Chat, a session exists and File Transfer is supported in the session as described in section 3.2.5.2, the session shall be reused to convey the File Transfer content body in a Chat message.
- If the file is to be transferred to an existing Group Chat and no session exists or no Group Chat exists and all recipients are capable to support Chat, then the client shall (re)started first a Group Chat session and transfer the file, if File Transfer is supported in the session.
- Otherwise, if the RCS client is enabled for Standalone Messaging then the File Transfer content body shall be sent using a Standalone Message to the list of recipients carrying a dedicated Accept-Contact header field that includes the File Transfer IARI tag defined in section 2.6.1.3 along with *require* and *explicit* parameters.
- Otherwise, if the client is not enabled for Standalone Messaging, then the client shall send the file via multiple 1-to1 messages.

IMDN delivery and display notifications apply as defined for the transport services applicable for File Transfer.

Figure 12 provides a summary overview of the File Transfer sender procedures.

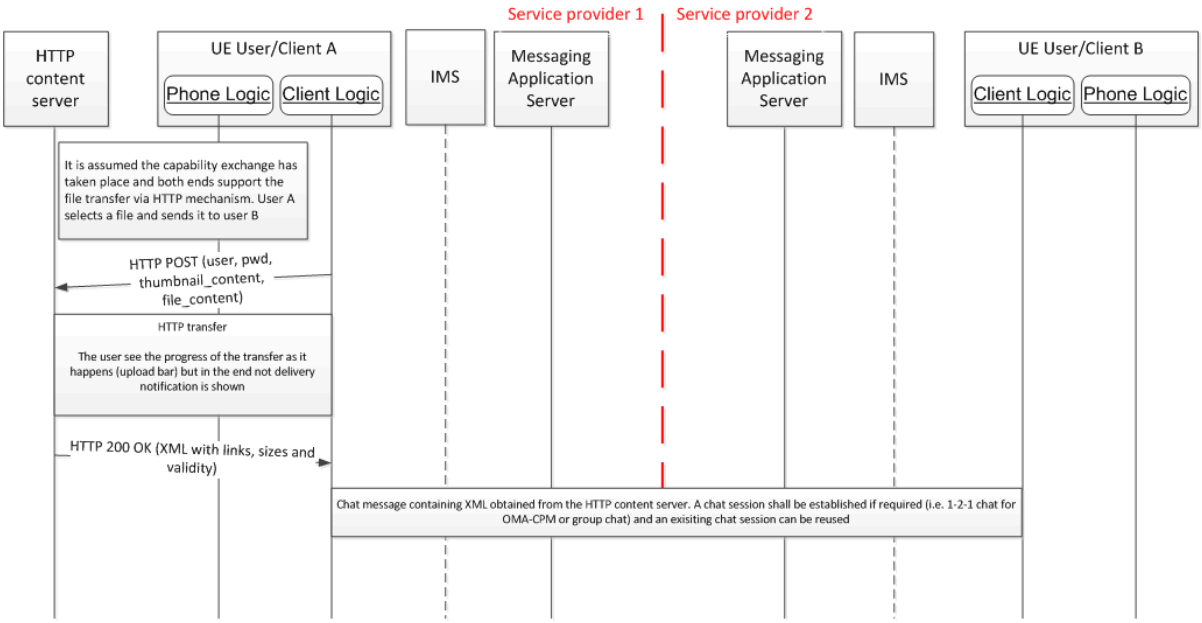


Figure 12: File Transfer: Sender procedures

3.2.5.3.2 Receiver procedures

If the client receives a Chat or Standalone Message with a File Transfer message body, then the client shall apply delivery disposition notifications as defined for the corresponding transport service.

The receiving client shall verify whether the domain part of the links provided for the file and, if applicable, the thumbnail in a File Transfer message body received in a person-to-person exchange complies with the format defined in section 4.2.5. If the domain part does not comply with the format defined in section 4.2.5, the client shall silently ignore the File Transfer.

NOTE: For the handling of the domain in the Chatbot service, see section 3.6.8.7.1.

If the thumbnail size is greater than 10 kB, the thumbnail should not be downloaded in below cases.

If auto acceptance applies, then the receiving client shall download automatically the thumbnail, if available, and also the file content.

If manual acceptance applies for the File Transfer, then the receiving client shall download automatically the thumbnail, if available. The file shall be downloaded only after user acceptance.

NOTE: For the determination of the mode of acceptance, refer to the corresponding service documentation, e.g. [GSMA PRD-RCC.71].

If validity of the file to be downloaded indicates that it may no longer be available on the server, the client shall inform the user of the circumstance when trying to download the file. The detailed UX is left intentionally outside the scope of this specification and it is up to the RCS client implementation.

To download a file or a thumbnail file, the receiving client shall invoke the procedure for file download via the HTTP Content Server as defined in section 4.2.3.

For the initiation of the file download procedure, the client shall provide the values of the download parameters defined in section 4.2.3 as follows:

Parameter	url	id	op	ci
Service	**Value**			
File Transfer in One-to-One/One-to-Many Message	Value of the "url" attribute of "data" element taken from the file-info element of the File Transfer message body with the type attribute set to "file" for the file itself	CPIM IMDN message-ID header of Chat or Standalone Message containing the File Transfer message body	Address of the originator of the Chat or Standalone Message in accordance with the definitions in section 2.5.2. If SIP URI is present, then the SIP URI shall be used, otherwise the tel URI.	n/a
	Value of the "url" attribute of "data" element taken from the file-info element of the File Transfer message body with the type attribute set to "thumbnail" for the thumbnail			
File Transfer in Group Chat	Value of the "url" attribute of "data" element taken from the corresponding file-info element of the File Transfer message body	CPIM IMDN message-ID header of Group Chat message containing the File Transfer message body	n/a	Conversation-ID header of the Group Chat session
	Value of the "url" attribute of "data" element taken from the file-info element of the File Transfer message body with the type attribute set to "thumbnail" for the thumbnail			

Table 22: File Transfer download parameters

If the file download from the HTTP Content Server is successful, then the client shall handle the file according to the content type received from the HTTP Content Server and the file-disposition attribute received in the File Transfer message body.

The client shall follow the service definitions of the File Transfer service for the handling of display disposition notifications.

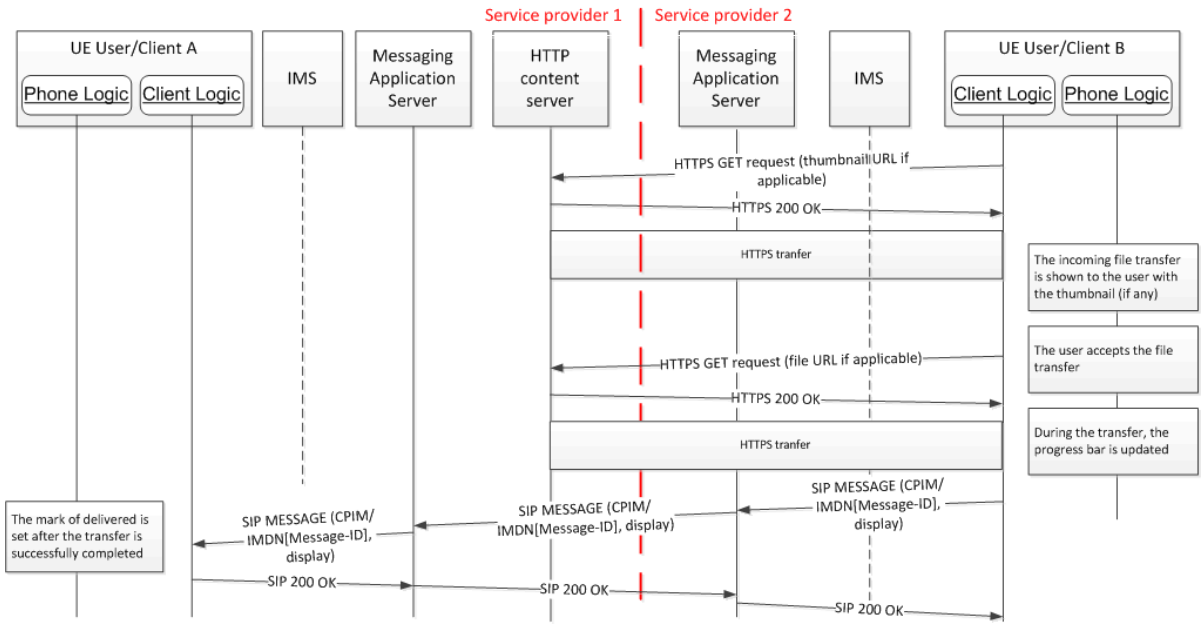


Figure 13: File Transfer: Receiver procedures

3.2.5.4 File Transfer Fallback

3.2.5.4.1 Introduction

The procedures for File Transfer fallback defined in this section enable RCS clients to provide a File Transfer experience if RCS is not available end-to-end.

The procedures for File Transfer fallback are applicable if

- a File Transfer is subject to delivery assurance via client fallback to SMS or
- a file is transmitted to a non-RCS capable recipient or a RCS capable recipient without File Transfer capability.

The procedures for File Transfer fallback rely on transport of File Transfer specific content via SMS messages.

3.2.5.4.2 File Transfer via SMS Capability

Section 2.6.1.3 defines the client capability for the support of the File Transfer via SMS service. A client supporting File Transfer and either SMS or Standalone Messaging or both shall advertise the capability

- via the media feature tag defined in Table 10 for File Transfer via SMS in SIP OPTIONS requests and responses for Capability Discovery or
- via the service description defined in Table 10 for File Transfer via SMS in presence documents for Capability Discovery

in accordance with the definitions for Capability Discovery defined in section 2.6.

3.2.5.4.3 Sender Procedures

The procedures defined in this section apply

- if the user requests to send a file to a recipient with no RCS capabilities or a recipient with RCS capabilities but without support of the File Transfer via SMS service as indicated by the capability defined in section 3.2.5.4.2. In this case, the user may prefer to send the message via MMS or as a "text message with a link". The operator is able to provide a default selection via the configuration parameter FT HTTP FALLBACK defined in section A.1.4 and A.2.4.
- if a chat message conveying a File Transfer message body is subject to client fallback as defined in section 3.2.3.8.

3.2.5.4.3.1 Sending of a File to a Recipient with no File Transfer Capability

If the user requests to send a file to a recipient with no RCS capabilities or a recipient with RCS capabilities but without support of the File Transfer via SMS service indicated by the capability defined in section 3.2.5.4.2, then

- if the user or the client has selected MMS to transfer the file and the file complies or can be converted to comply with the formats and codecs defined in [MMSCONF], then the client shall compose a MMS message and send it following the procedures defined in [3GPP TS 23.140], otherwise
- the client shall upload the file to the HTTP Content Server as defined in section 4.2.2.
- If successful, the client shall check whether the HTTP Content Server response body received from the HTTP Content Server contained an "branded-url" element in the file-info element with type "file", then
    - if the "branded-url" element is present it shall use the URL contained in its value, otherwise
    - it shall use the URL contained in the value of the "url" attribute of the "data" element of the "file-info" element with type "file",
- compose a new message containing the URL next to some explanatory text indicating the purpose of the message and send it to the recipient via
    - RCS Standalone Messaging, if enabled as defined in section 3.2.2, otherwise
    - via SMS.

3.2.5.4.3.2 File Transfer Client Fallback

If the originating client has uploaded a file to the HTTP Content Server for a File Transfer and the client applies monitoring of the delivery of chat messages within the 1-to-1 Chat session in accordance with the definitions in section 3.2.3.8, then the client shall keep the data of the HTTP Content Server response body at least until the delivery of the chat message is confirmed.

Precondition for the application of File Transfer via SMS is that the client has uploaded a file to the HTTP Content Server as defined in section 4.2.2 and has received and kept the data of the HTTP Content Server response body.

If the originating client decides to fall back to SMS for a File Transfer as result of the procedures for client fallback as defined in section 3.2.3.8.3 and the recipient supports File Transfer via SMS, as indicated by the capability defined in section 3.2.5.4.2, then the originating client

- shall inspect the URL contained in the "url" attribute of the "data" element of the "file-info" element with type "file" received in the HTTP Content Server response body to determine whether the "s", "t" and "e" URL parameters as defined in section 4.2.5 are present, then
- if there is none of these URL parameters present, then the client shall generate
    - a “s” parameter as defined in section 4.2.5 using the value extracted from the file-size element of the file-info element with type “file” included in the HTTP Content Server response body,
    - a “t” parameter as defined in section 4.2.5 using the value extracted from the content-type element of the file-info element with type “file” included in the HTTP Content Server response body,
    - an “e” parameter as defined in section 4.2.5 using the value extracted from the “until” attribute contained in the data element of the file-info element with type “file” included in the HTTP Content Server response body,
    - append it to the query part of the URL taken from "url" attribute of the "data" element of the "file-info" element with type "file" using the application/x-www-form-urlencoded format as defined in [HTML-4.0] respecting the definitions of [RFC3986]. If no query part exists, the client shall create one first,
- otherwise use the URL contained in the "url" attribute of the "data" element of the "file-info" element with type "file" unaltered
- if the file is a RCS Recorded Audio Message (RRAM) as defined in section 3.2.7, then the client shall generate a “d” parameter as defined in section 4.2.5 using the “playing-length” of the RRAM as defined in section 3.2.7.2.2 and append it to the URL resulting from previous processing using the application/x-www-form-urlencoded format as defined in [HTML-4.0] respecting the definitions of [RFC3986],
- add the resulting URL to the user data of a SMS message and
- send the SMS message to the recipient address.

If the originating client decides to fall back to SMS for a File Transfer as result of the procedures for client fallback as defined in section 3.2.3.8.2 and the recipient does not support File Transfer via SMS via the capability defined in section 3.2.5.4.2, then the client

- shall check whether the HTTP Content Server response body contained an "branded-url" element in the file-info element with type "file", then
    - if the "branded-url" element is present it shall use its value, otherwise
    - it shall use the URL contained in the "url" attribute of the "data" element of the "file-info" element with type "file",
- add the URL next to some explanatory text indicating the purpose of the message to the user data of a SMS message and

- send the SMS message to the recipient address.

3.2.5.4.4 Receiver Procedures

On reception of a SMS message, the client shall parse the user data of the message.

If the user data contains a HTTP(s) URL and the FQDN of the URL conforms to the definitions of the HTTP Content Server URI as defined in section 4.2.5, then the client shall apply the UX procedures defined for suppression and replacement of the HTTP Content Server URL. The client shall take the URL parameters for File Transfer fallback as defined in section 4.2.5 into account.

If the FQDN conforms to the definitions of the HTTP Content Server URL as defined in section 4.2.5, then the client shall apply the file download procedure defined in section 4.2.3 either automatically, if auto acceptance applies, or otherwise only after user acceptance.

NOTE: For the determination of the mode of acceptance refer to the corresponding service documentation, e.g. [GSMA PRD-RCC.71].

For the initiation of the file download procedure, the client shall provide the values of the download parameters defined in section 4.2.3 as follows:

Parameter	Value
url	URL in the format defined in section 4.2.5 from the user data of a SMS message
id	n/a
op	Value of the SMS originator address in the format defined below
ci	n/a

Table 23: File Transfer via SMS download parameters

The "op" parameter shall contain the address value of the originator address of the received SMS message prefixed by the string "sms:" If the address value contains an international E.164 number, then the address value shall be prefixed with "+"
Example: if the originator address of the SMS is an international E.164 number with an address value "491711234567", then the resulting "op" parameter is encoded as follows: op=sms%3A%2B491711234567.

3.2.5.4.5 Network Interworking

The procedures in the network for File Transfer fallback are network internal and therefore outside of the scope of this document. As defined in section 3.2.5.4.2, the client shall add the media feature tag defined in Table 3 for File Transfer via SMS in the Contact header field of the SIP REGISTER allowing the network to detect the client capability. The formats of SMS messages resulting from the network interworking procedures shall follow the formats defined for the client-based fallback defined in section 3.2.5.4.3.

3.2.5.5 Handling of specific content

3.2.5.5.1 Personal Card format

There are multiple formats for the transfer of Personal Cards. This section defines the transfer formats and procedures for Personal Cards in RCS.

An RCS compliant device shall support receiving following formats:

- the vCard 2.1 format as defined in [vCard21] and
- the vCard 3.0 format as defined in [RFC2425] and [RFC2426] and
- the vCard 4.0 format as defined in [RFC6350].

NOTE: vCard 3.0 [RFC2425] and [RFC2426] were obsoleted by [RFC6350]. These obsoleted references are used for vCard 3.0 formats that do not comply with the latest RFC.

In addition, an RCS may support receiving the following format

- the Personal Contact Card (PCC) format defined in [CAB_TS].

The vCard 3.0/4.0 format as defined above shall be used for sending.

Variations in the implementation of Personal Card formats may lead to data loss when Personal Cards are exchanged. To limit the effect, the following fields are considered key fields for RCS. No data of these fields should be lost when contact information is exchanged in RCS.

- **Name**: Composed names (such as “Jean-Baptiste”) shall be supported properly.
- **Personal Information**:
  - Nickname
  - Photo
  - Birthdate
  - Comment
- **Telephone numbers**: At least the following subtypes of telephone number shall be supported:
  - Land home
  - Land work
  - Land other
  - Mobile home
  - Mobile work
  - Mobile other
  - Fax work
  - Fax other
  - Beeper
  - Other
- **Email addresses**: The following subtypes shall be supported:

- Email work 1
- Email work 2
- Email home 1
- Email home 2
- Other

- **Address information:**
  - Address
  - Geographic Position
  - Time zone

Sending and receiving a Contact Card via File Transfer via HTTP or File Transfer fallback is technically the same as sending any other file.

If the format for transferring a Contact Card file is vCard 2.1, vCard 3.0 or vCard 4.0, then the MIME content type "*text/vcard*" shall be used for File Transfer.

If the format for transferring a Contact Card is the CAB (Converged Address Book) 1.0 PCC XML format, then the CAB PCC MIME content type "*application/vnd.oma.cab-pcc+xml*" shall be used for File Transfer.

On the receiving side, if the receiving RCS adds the Contact Card file delivered through File Transfer to the local address book, the receiving RCS client shall apply the mapping of the RCS supported fields between the received format and the used format of the local address book database. For conversion between PCC and vCard formats, refer to section 5.4.3 of [CAB_TS].

If the receiving client does not support the format of a Contact Card, then the client handling for unsupported content types applies.

3.2.5.5.2 Audio Message

The handling of audio messages is described in section 3.2.7.

3.2.5.6 Codec selection procedure

This section describes the codec selection procedures for Media Share which ensures interoperability and prevents the transfer of unsupported media files.

3.2.5.6.1 Capability Discovery

Media Profiles define a set of codecs that an RCS client supports for decoding media files. The Media Profile version advertised by an RCS client shall match one of the Media Profile versions defined herein. An RCS client that declares support for a Media Profile with a new minor version (e.g., "1.1") shall also support all codecs listed in all previous minor versions of the same major version (e.g., "1.0").

All RCS clients shall support the baseline Media Profile version "1.0" and should support the Media Profile version "1.1".

NOTE: Legacy clients are assumed to support Media Profile version "1.0".

Media Profile Version	Supported Media Capabilities
1.0	**Audio**: 26143_AUDIO_AMR2, 26143_AUDIO_MP3 **Video**: 26143_VIDEO_AVC-HD **Image**: 26143_IMG_JPEG, 26143_IMG_GIF, 26143_IMG_PNG.
1.1	**Audio**: 26143_AUDIO_XHE-AAC **Video**: 26143_VIDEO_AVC-FullHD, 26143_VIDEO_HEVC-HD, 26143_VIDEO_HEVC-FullHD **Image**: 26143_IMG_HEIC

Table 24: Media Profile Definitions

All media capabilities referenced in Table 24 shall follow the media capability type definition in Section 5 of [3GPP TS 26.143].

The RCS Client shall advertise the supported codecs for Media Share as follows:

- The RCS Client shall include the Media Share Codecs feature tag as defined in section 2.6.1.3.
- The RCS Client shall set the value of the Media Share Codecs feature tag to the list of supported Media Profiles and codecs for decoding.
- For Audio Messaging the list of codecs is defined in section 3.2.7.

The value of the Media Share Codecs feature tag parameter defined in section 2.6.1.3 shall be formatted as follows:

```
; MS Codecs
mscodecs-value = profile-version *( "," profile-version ) [ "," codecs-
parameter ]

; Profile Version bases on numeric as defined in [RFC3840] using "=" as
numeric-relation
profile-version = numeric

; Media Share Codecs feature tag parameter
codecs-parameter = codec-name *( "," codec-name )

; Codec name format
codec-name = media-type "_" codec-identifier

; Media type
media-type = "AUDIO" / "VIDEO" / "IMG"

; Codec identifier
codec-identifier = 1*(ALPHA / DIGIT / "_" / "-")

; ALPHA and DIGIT are defined in RFC 5234
ALPHA = %x41-5A / %x61-7A   ; A-Z / a-z
DIGIT = %x30-39             ; 0-9
```

- mscodecs-value: This represents the entire value of the feature tag. It shall start with one or more profile-version parameters and may be followed by an additional codecs-parameter.
- profile-version: This represents the highest Media Profile supported, as defined in section 3.2.5.6.2. If more than one Media Profile is indicated, they should be comma separated as per [RFC3840].
- codecs-parameter: This represents the entire parameter, which is a comma-separated list of codec names.
- codec-name: This represents a single codec name, which is composed of a media type followed by an underscore and a codec identifier. Codec-name is derived from the media capability name as defined in [3GPP TS 26.143] section 5 by removing “26143_” string.
- media-type: This can be "AUDIO", “VIDEO”, or “IMG”.
- codec-identifier: This is a sequence of one or more alphanumeric characters or underscores.
- ALPHA and DIGIT: These are standard definitions for alphabetic and numeric characters, respectively.

3.2.5.6.2 Sender add-on procedure

Before initiating a File Transfer for a media file, the sending client shall follow the following procedure:

- If the recipients’ media decoding capabilities have been discovered via the +g.gsma.rcs.mscodecs feature tag, the sending RCS client
    - shall identify the recipients’ advertised major Media Profile versions:
        - If there are matching major Media Profile versions between the recipients, the sending RCS client shall find the highest common minor Media Profile version between the recipients
        - Else the sending RCS client shall consider Media Profile version "1.0” as common Media Profile
    - shall add the commonly supported additional codecs to the list of considered codecs
    - shall select a mutually supported codec from this list (for Audio Messaging, it shall select highest priority codec, see section 3.2.7).
    - shall format the Media File using this codec
    - shall add the media type for this codec as defined in [3GPP TS 26.143] section 5 in the <content-type> element of the File Transfer message body
    - shall initiate the File Transfer as defined in section 3.2.5.3
- Else, the sending RCS client shall initiate the File Transfer as defined in section 3.2.5.3, using codecs from Media Profile version "1.0”.

3.2.6 Geolocation Push services

For Geolocation Push services, gRPC-configured RCS clients shall follow the procedures defined in section 5.2.3 of [GSMA PRD-RCC.30].

SIP-configured RCS clients shall follow the procedures in this section.

3.2.6.1 Overview

The geolocation information shall be sent directly as a message in a Chat session provided the intended recipient (for a 1-to-1 Chat) or the Controlling Function (for a Group Chat) supports Geolocation Push. The format that shall be used is described in section 3.2.6.5.

3.2.6.2 Geolocation Push Procedure

The Geolocation PUSH service shall send the geolocation information directly in a chat message. That allows potentially reusing an already established 1-to-1 or Group Chat session for Geolocation PUSH.

In an active Chat session the sending of the geolocation information shall be possible if

- the Geolocation PUSH content type was included in the a=accept-wrapped-types attribute of the SDP received during the setup of the Chat session and
    - In case of a 1-to-1 session, the contact supports Geolocation PUSH (i.e. the corresponding capability was discovered or was cached)
    - In case of a Group Chat, the Contact header received during the setup of the Group Chat included the Geolocation PUSH IARI tag defined in section 2.6.1.3.

When these conditions are fulfilled, a client can transfer the geolocation information in a CPIM wrapper that is transferred using an MSRP SEND request with the encapsulated Content-Type header of the CPIM message set to *application/vnd.gsma.rcspushlocation+xml*.

3.2.6.2.1 1-to-1 Exchange of Geolocation PUSH

In case a new 1-to-1 session needs to be established when the user wants to transfer geolocation information to a contact that has the Geolocation PUSH capability, the sending client shall generate a SIP INVITE request for a 1-to-1 Chat session as specified in section 3.2.3.1. The Geolocation PUSH XML message body itself (i.e. geolocation information in a CPIM wrapper with the encapsulated Content-Type header set to *application/vnd.gsma.rcspushlocation+xml*) shall then be sent as first message in the Chat.

If there is an active 1-to-1 Chat session with a Geolocation PUSH capable contact, but the *a=accept-wrapped-types* SDP attribute received during the setup of that Chat session did not include the *application/vnd.gsma.rcspushlocation+xml* MIME content type, Geolocation PUSH to that contact will not be available.

3.2.6.2.2 Multiparty Exchange of Geolocation PUSH

During Group Chats, the capability to use Geolocation PUSH depends on the Controlling Function. A Geolocation PUSH capable Controlling Function shall enable Geolocation PUSH by including the *application/vnd.gsma.rcspushlocation+xml* MIME content type in the *a=accept-wrapped-types* SDP attribute that it provides during the setup of the Group Chat session and include the Geolocation PUSH IARI tag defined in section 2.6.1.3 in the Contact headers that it includes in the SIP INVITE requests and 200 OK responses for the setup of the Group Chat. A Geolocation PUSH capable Controlling Function shall not

distribute Geolocation PUSH information to the participants in the chat that are not Geolocation PUSH capable. A client on which Geolocation PUSH was enabled shall during the setup of the Group Chat indicate to the Controlling Function that it supports Geolocation PUSH by including the *application/vnd.gsma.rcspushlocation+xml* MIME content type in the a=accept-wrapped-types SDP attribute and the Geolocation PUSH IARI tag defined in section 2.6.1.3 in the Contact headers of the SIP INVITE requests and 200 OK responses that it generates. When during a Group Chat the *a=accept-wrapped-types* SDP attribute received by a client or conference focus did not include the *application/vnd.gsma.rcspushlocation+xml* MIME content type or the Geolocation PUSH IARI tag was not provided in the received Contact header, Geolocation PUSH shall not be available for the Group Chat in which the client participates and for a specific client in the Group Chat respectively.

When the user wants to send the Geolocation information to the participants of an existing Group Chat that is idle, a client that is configured to support Geolocation PUSH shall first restart the chat and then send the file in the chat.

When the user wants to send geolocation information to multiple contacts outside of the context of an existing Group Chat, a client that is configured to support Geolocation PUSH shall first start a new Group Chat with the selected contacts and send the Geolocation XML body as first message in the chat.

3.2.6.3 Geolocation Push Fallback

A client supporting Geolocation PUSH and either SMS or Standalone Messaging shall support Geolocation PUSH fallback to SMS, and thus support the rendering of the user data of a short message for Geolocation Push as defined in section 3.2.6.3.2 and the procedures for "geo" URI defined in section 3.2.6.5.4.

3.2.6.3.1 Sender procedures

If the originating client decides to fallback to SMS for a Geolocation Push message and the recipient supports Geolocation Push via SMS as indicated by the capability defined in section 2.6.1.3, then the sender client shall use the position and the label of the RCS Location information data sent in the Geolocation Push message and generate a "geo" URI as defined in section 3.2.6.5.4.

3.2.6.3.2 Receiver procedures

On reception of a SMS message, the client shall parse the user data of the message.

If the user data contains a "geo" URI as defined in [RFC5870], then the client shall apply the UX procedures defined for suppression and replacement of the "geo" URI string. The client shall apply the rules for the presence and absence of the "label" via the "geo" URI extension defined in section 3.2.6.5.4 in accordance with the definitions of section 3.2.6.5.2.

3.2.6.3.3 Network Procedures for Fall Back for Geolocation Push

The procedures in the network for fallback for Geolocation Push are network internal and therefore outside of the scope of this document. However, to facilitate a solution at a later time, as specified in section 2.4.4.1 the User Equipment (UE) shall include the IMS Application Reference Identifier (IARI) defined in section 2.6.1.3 in the Contact header field

of the SIP REGISTER request along with the rest of the feature tags the UE is required to include. The format of the SMS messages sent shall follow the format defined for the client-based fallback described in section 3.2.6.5.4.

3.2.6.3.4 Geolocation Push service Technology Selection

If the RCS Geolocation Push service is enabled (i.e. PROVIDE GEOLOC PUSH is set to 1) and the Chat service is enabled (i.e. CHAT AUTH is set to 1), the client shall follow the procedures defined in this section.

If a message for RCS Geolocation Push is subject to delivery assurance as defined in this document, the client shall apply the procedures defined in this section. If a message for RCS Geolocation Push is subject to delivery assurance as defined in this document and the receiver does not support the procedures defined in this section, the client shall send the location based on the rules defined in section 3.2.1:

- Selecting an RCS Standalone message if RCS Standalone Messaging is enabled, otherwise
- Selecting an SMS.

If the RCS Geolocation Push service is enabled (i.e. PROVIDE GEOLOC PUSH is set to 1), the RCS 1-to-1 Chat service is disabled (i.e. CHAT AUTH is set to "0") and the RCS Standalone messaging service is enabled, the regular Geolocation Push service defined in earlier in this section cannot be used due to its dependency on Chat. Therefore, the location information shall be sent as a text message based on the technology selection rules described in section 3.2.1. For the format of the message, the procedures defined in this section shall be applied (i.e. it shall depend on whether the recipient indicated the Geolocation Push via SMS capability defined in section 2.6.1.3).

For sending location information towards non-RCS users, the 1-to-1 Messaging technology selection rules towards non-RCS users defined in section 3.2.1 shall apply.

3.2.6.4 Void

3.2.6.5 Location Information format

3.2.6.5.1 General

The following XML schema is defined:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:geolocation"
            xmlns:xs="http://www.w3.org/2001/XMLSchema"
            xmlns="urn:gsma:params:xml:ns:rcs:rcs:geolocation"
            elementFormDefault="qualified"
            attributeFormDefault="unqualified">
      <xs:element name="rcsenvelope">
            <xs:complexType>
                  <xs:sequence>
                        <xs:element name="rcspushlocation">
                              <xs:complexType>
                                    <xs:sequence>
                                          <xs:any namespace="##other" processContents="lax"
                                          minOccurs="0" maxOccurs="unbounded"/>
                                          <xs:element name="timestamp">
                                                <xs:simpleType>
                                                      <xs:restriction base="xs:dateTime"/>
                                                </xs:simpleType>
                                          </xs:element>
                                    </xs:sequence>
                                    <xs:attribute name="id" type="xs:ID" use="required"/>
                                    <xs:attribute name="label" type="xs:string" use="optional"/>
                              </xs:complexType>
                        </xs:element>
                        <xs:any namespace="##other" processContents="lax" minOccurs="0"
                        maxOccurs="unbounded"/>
                  </xs:sequence>
                  <xs:attribute name="entity" type="xs:anyURI" use="required"/>
            </xs:complexType>
      </xs:element>
</xs:schema>
```

Table 25: Geolocation PUSH Envelope XML schema

3.2.6.5.2 RCSPushLocation data model

Attribute	Specification	Comment
Person: <rcsenvelope> -> <rcspushlocation>	Table 25	Each client only publishes one <rcsenvelope> and one <rcspushlocation> element. The rcspushlocation element may have a label that can be used to tag the nature of the location (e.g. indicate that it's the home or provide an address, name of restaurant, etc.). If no label is provided, the location that is shared is assumed to be the sharing user's own position.
Time Zone <rcspushlocation> -> <time-offset>	Table 25, [RFC4480] and [Presence2.0_DDS]	The geolocation application may use this element to provide information on the current time zone See following chapter section for more information on the handling of the expiry of this information

Attribute	Specification	Comment
Geographical Information <rcspushlocation> -> <geopriv> -> <location-info> -> <usage-rules>	Table 25, [RFC5491] and [Presence2.0_DDS]	This element can be used to provide geographical location information. The accuracy of which can be controlled by the user. See following section for more details on its encoding and on the handling of the expiry of this information
Timestamp: <rcspushlocation> -> <timestamp>	Table 25, [RFC4479]	Timestamp when the location information was pushed

Table 26: RCSPushLocation data model attributes

3.2.6.5.3 RCS Location information

RCS clients shall not include a "*from*" attribute in the *<time-offset>* element. RCS clients shall ignore it when received.

RCS clients can provide (if authorised by the Service Provider) an "*until*" attribute in that element with a value provided by the user.

RCS clients shall not include the optional description attribute in the *<time-offset>* element as this overlaps with the Location Type. RCS clients shall ignore it when received.

The geographical information will be provided as geographic coordinates. As specified for the "Geographical Location" building block in [Presence2.0_DDS], encoding will use the *<geopriv>→<location-info>* and *<geopriv>→<usage-rules>* elements.

The optional *<usage-rules>* element shall contain, if present, only a "*retention-expiry*" element. The RCS client shall set the "*retention-expiry*" to the same value as the "*until*" attribute mentioned above.

The *<location-info>* published by an RCS Geolocation client will contain geographical information using the GML 3.1.1 Feature Schema (see [GML3.1.1]) which is the mandatory format to be used in the *<location-info>* element. The civic location format shall not be used by RCS and location information encoded in that way will be ignored by RCS clients when received.

RCS client will within the *<location-info>* element represent an exact position by providing a GML <point> element and an inaccurate position as a *<circle>* element, both referring to the European Petrol Survey Group EPSG::4326 spatial reference schema as described in [RFC5491].

The coordinates of either the centre of this circle or the exact position will be represented with a single GML *<pos>* element with the actual coordinates as value.

The radius of the circle will be represented in meters, which will be indicated by setting the unit of measure attribute of the radius element to the value of EPSG::9001 as described in [RFC5491].

The text value (that is, the *label* attribute) shall not exceed 200 characters. The text is entered by the user.

An RCS client shall ignore any other type of data provided in the *<location-info>* element.

The EPSG format requires that the coordinate representation be defined by the coordinate supplier. RCS client will always provide the coordinates in WGS 84 (latitude, longitude) decimal notion as described in [RFC5491], providing the latitude and longitude as “double”-encoded decimal numbers (as specified in [GML3.1.1]) representing the degrees, separated by a space starting with the latitude. Negative values represent Southern and Western hemisphere respectively.

The following gives an example of RCS Location information data:

```
<?xml version="1.0" encoding="UTF-8"?>
<rcsenvelope xmlns="urn:gsma:params:xml:ns:rcs:rcs:geolocation"
        xmlns:rpid="urn:ietf:params:xml:ns:pidf:rpid"
        xmlns:gp="urn:ietf:params:xml:ns:pidf:geopriv10"
        xmlns:gml="http://www.opengis.net/gml"
        xmlns:gs="http://www.opengis.net/pidflo/1.0"
        entity="tel:+1234578901">
        <rcspushlocation id="a1233" label="meeting location">
              <rpid:time-offset rpid:until="2012-03-15T21:00:00-05:00">-300</rpid:time-offset>
              <gp:geopriv>
                    <gp:location-info>
                          <gs:Circle srsName="urn:ogc:def:crs:EPSG::4326">
                                <gml:pos>26.1181289 -80.1283921</gml:pos>
                                <gs:radius uom="urn:ogc:def:uom:EPSG::9001">10</gs:radius>
                          </gs:Circle>
                    </gp:location-info>
                    <gp:usage-rules>
                          <gp:retention-expiry>2012-03-15T21:00:00-05:00</gp:retention-expiry>
                    </gp:usage-rules>
              </gp:geopriv>
              <timestamp>2012-03-15T16:09:44-05:00</timestamp>
        </rcspushlocation>
</rcsenvelope>
```

Table 27: Example of location information data

3.2.6.5.4 Geolocation Push URI for fallback

A client supporting Geolocation Push fallback shall be able to

- generate, using RCS Location information data, and
- resolve and render

a “geo” URI according to [RFC5870].

For the purpose of Geolocation Push fallback, the "geo" URI format of [RFC5870] is extended by a new parameter to carry a "label". The usage of the "label" parameter shall follow the definitions for the "label" in Geolocation Push defined in section 3.2.6.5.2.

"Geo" URI parameters extending [RFC5870] are defined in Table 28.

Parameter	Value Restriction	Value
rcs-l	Constrained	Contains an UTF-8 character encoded label text that can be used to tag the nature of the location (e.g. indicate that it is the home or provide an address, name of restaurant, etc.) in the context of Geolocation Push. If the label parameter is absent, the location that is shared is assumed to be the sharing users own position. NOTE: non-ASCII and reserved characters have to be represented using percent encoding in accordance with [RFC5870].

Table 28: "geo" URI Parameter Extensions

NOTE: It is recommended that implementations ensure that the maximum length of the URLs does not exceed the length of the user data of one short message.

Example "geo" URI with parameter extension:

```
geo:50.7311865,7.0914591;u=10;rcs-l=The%20Quiet%20Man%20%F0%9F%8D%BB
```

3.2.7 Audio Messaging

3.2.7.1 Overview

For Audio Messaging, the sender RCS client shall follow the codec selection procedure described in section 3.2.5.6 to identify a list of common supported codecs. For encoding the audio message, the RCS client

1. should support the xHE-AAC codec.
2. may support the HE-AAC v2 codec.
3. shall support the Adaptive Multi-Rate (AMR) codec.

NOTE: xHE-AAC® is a registered trademark of Fraunhofer in Germany and other countries and is used with Fraunhofer's permission.

The RCS client shall select the codec from the list above and according to the priority order defined in Table 29 ((1) is highest priority).

Priority	Codec name	mscodecs feature tag parameter name	Reference to specification	Content type
1	xHE-AAC	AUDIO_XHE-AAC	26143_AUDIO_XHE-AAC section 5.5.2 of [3GPP TS 26.143]	audio/mp4, codecs="mp4a.40.42"
2	HE-AAC v2	AUDIO_EAAC+	26143_AUDIO_EAAC + section 5.5.2 of [3GPP TS 26.143]	audio/mp4, codecs="mp4a.40.29"
3	AMR	AUDIO_AMR2	26143_AUDIO_AMR2 section 5.5.2 of [3GPP TS 26.143]	audio/amr

Table 29: Description of codecs used for Audio Messaging

The RCS Recorded Audio Message (RRAM) shall be formatted in the file format defined and signalled with the content type defined in Table 29.

An RCS client shall support decoding of audio messages encoded with any of the codecs listed in Table 29.

NOTE: Recommended bitrates for encoding of RRAMs are provided here as implementation guidelines:

- Speech/Audio, mono: xHE-AAC 12kbps, HE-AAC v2 24kbps, AMR 12kbps
- Standard Audio, stereo: xHE-AAC 24kbps, HE-AAC v2 48kbps
- High Quality Audio, stereo: xHE-AAC 64 kbps, HE-AAC v2 64kbps

The transport of RRAM uses the File Transfer as defined in section 3.2.5. The following features are applicable for Audio Messaging:

- disposition notifications of the File Transfer transport services
- store and forward of the File Transfer transport services
- auto-acceptance rules for File Transfer
- codec selection

3.2.7.2 Sender procedures

3.2.7.2.1 Recording

When the audio message is selected via the User Interface, the Client shall record an audio file via the device's microphone.

The duration of the RRAM shall be limited to a maximum duration of 10 minutes. The Client shall automatically stop the recording when this limit is reached.

Once recorded, the content should automatically be packaged into the file format described in section 3.2.7.1.

3.2.7.2.2 Sending

When sending a RRAM to a contact, the RRAM is transported via the File Transfer service (see section 3.2.5). The File Disposition shall be set to '*render*'.

NOTE: '*render*' means that the content of the file can be played directly from the Chat application upon user action.

In complement to the procedures of section 3.2.5.3.1, the Client shall create a playing-length element using the File Transfer message body schema extension defined in Table 30. The value of the playing-length element shall contain the duration of the RRAM in seconds.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:rram"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="urn:gsma:params:xml:ns:rcs:rcs:rram"
```

```
            elementFormDefault="qualified"
            attributeFormDefault="unqualified">
        <xs:element name="playing-length">
            <xs:simpleType>
                <xs:restriction base="xs:integer"/>
            </xs:simpleType>
        </xs:element>
</xs:schema>
```

Table 30: Extension to File Transfer via HTTP message body schema for Audio Message

The client shall add the playing-length element to the file-info element of the File Transfer message body.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<file xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttp"
    xmlns:am=" urn:gsma:params:xml:ns:rcs:rcs:rram">
        <file-info type="file" file-disposition="[file-disposition]">
            <file-size>[file size in bytes]</file-size>
            <file-name>[original file name]</file-name>
            <content-type>[MIME-type for file e.g. audio/amr]</content-type>
            <am:playing-length>[duration of the rram]</am:playing-length>
            <data url ="[HTTP URL for the file]" until="[validity of the file]"/>
        </file-info>
</file>
```

Table 31: Example of Audio Message Transfer using File Transfer via HTTP

3.2.7.3 Receiver procedures

On the receiving side, when a message with a File Transfer content body is received with the file-disposition set to "render" and the content is recognised as corresponding to the file format described in section 3.2.7.1, rather than announcing the reception of a File Transfer, the UI shall announce that an audio message is received. If accepted or auto-accepted, the received content shall be displayed in the corresponding 1-to-1 or Group Chat thread as an audio message with the option to play it. The RRAM shall not be played automatically. The Display Notification (if requested) shall be sent when the playing of the file is started.

3.2.8 Extended Messaging

3.2.8.1 Overview

This section describes the Extended Messaging features, which includes the Reply Message, Reaction, Custom Reaction, Edit Message, Message Recall and Delete Message features.

3.2.8.2 void

3.2.8.3 void

3.2.8.4 Reply Message

3.2.8.4.1 Overview

An RCS client can reply to an RCS message sent or received. If the Extended Messaging Reply Message feature is supported, the message can be plain text, emoticon, File Transfer, audio message, or Geolocation.

A Reply Message message can be sent using Standalone Messaging, 1-to-1 Chat and Group Chat procedures.

3.2.8.4.2 Reply Message Feature Availability

For Extended Messaging Reply Message to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Reply Message capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Reply Message feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging Message IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Reply Messaging capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.4.3 Sender add-on procedure

If an RCS client wants to send a message as a Reply Message message, the client shall:

1. follow section 2.14 for Extended RCS CPIM headers support.
2. add the additional headers referred to in section 3.2.8.10 for the Reply Message feature.

3.2.8.4.4 Receiver procedure

When an RCS client receives a plain text, emoticon, File Transfer, Audio, or Geolocation message, it shall check for the presence of the “rcs” namespace defined in section C.3.1. If present, the client:

1. shall find the type of the message in the rcs.Reference-Type header. If set to “Reply”, the message shall be treated as a Reply Message message.
2. shall lookup the message referenced in the rcs.Reference-ID header defined in section C.3.8. If the message is not found in the conversation, the Reply Message message shall be displayed with an indication of the associated message, which is left for client implementation.
3. If the message is found, shall show the new message in the User Interface indicating it’s a reply to the previously present message.

3.2.8.4.5 Backward compatibility

A client supporting the Reply Message feature should support older clients if the conversation does not support the Reply Message feature, refer to section 3.2.8.4.2.

For text-based replies, the Reply Message content should have an indication of reply to the associated message, which is left to the client implementation. The maximum length of the message shall be provided by the RCS provider through the MAX SIZE IM configuration parameter defined in Table 167 in section A.1.3. The client may choose the language and the text direction to represent the message. See section 3.2.8.4.6 for an example.

3.2.8.4.5.1 SMS Fallback

The Reply Message feature is not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send a Reply Message message.

3.2.8.4.6 Examples of Reply Message messages

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: K28f67bTydi45uf2GGd23

Content-Type: text/plain;charset=UTF-8
Content-Length: …

How are you doing?
```

Table 32: Original message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: K28f67bTydi45uf2GGd23
rcs.Reference-Type: Reply

Content-Type: text/plain;charset=UTF-8
Content-Length: …

I am doing great, how about you?
```

Table 33: Reply Message message

The following is an example of a backward compatible message. The client may choose the content, language, and text direction to represent the message.

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj

Content-Type: text/plain;charset=UTF-8
Content-Length: …

How are you doing?
RE:
↳ I am doing great, how about you?
```

Table 34: Backward compatible Reply Message message

3.2.8.5 Reaction

3.2.8.5.1 Overview

An RCS client can send an emoji/emoticon reaction to an existing message that has been exchanged in the conversation, if the Extended Messaging Reaction feature is supported. Multiple Reactions from the same sender shall only be associated with a message, if the recipient supports the Extended Messaging Multiple Reaction feature based on capability exchange procedures in section 3.2.8.4.3.

Sending the body of a Reaction message to the recipient(s) is based on Standalone Messaging, 1-to-1 Chat and Group Chat procedures described in sections 3.2.2, 3.2.3 and 3.2.4.

3.2.8.5.2 Reaction Feature Availability

For the Extended Messaging Reaction to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Reaction capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Reaction feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Reaction Messaging capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.5.3 Multiple Reaction Feature Availability

For the Extended Messaging Multiple Reaction feature to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Multiple Reaction capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Multiple Reaction feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging Multiple Reaction IARI tag defined in section 2.4.1.1
- The recipients are known to support the Extended Messaging Multiple Reaction Messaging capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.5.4 Sender procedures

The RCS client shall provide an option to send a Reaction message associated with an RCS message. When the user adds or removes a Reaction via the User Interface, the client shall send the Reaction and Multiple Reaction message body to the receiver(s). The client shall follow the procedures described in section 3.2.8.5.6 to set the CPIM message content.

Message Revocation (see section 3.2.3.8.2) is not applied for a Reaction message.

If the receiver only supports (single) Reaction; and the sender replaces an existing Reaction; then the RCS client shall send rcs.Reference-Type CPIM header defined in section C.3.9 along with “-Reaction” for the existing Reaction and “+Reaction” for the new Reaction.

3.2.8.5.5 Receiver procedures

When the client receives a plain/text message, the client:

1. shall check for the presence of the rcs.Reference-Type CPIM header defined in section C.3.9. If present and if its value is “+Reaction”, the message is handled as a Reaction Add message. If present and its value is “-Reaction”, the message is handled as a Reaction Remove message.
2. shall lookup the original message based on the message-ID provided in the rcs.Reference-ID header defined in section C.3.8 for adding or removing the Reaction sent in Reaction Message. If the original message does not exist, or the sender is not part of the conversation, the Reaction message shall be silently ignored.
3. shall parse the emoji sequences in the message body to obtain the emojis.
4. If the emojis being added already exist, the addition shall be silently ignored.
5. If the emojis being removed do not exist, the removal shall be silently ignored.
6. shall indicate in the User Interface that this Reaction was added to/removed from the original message by the sending user.

3.2.8.5.6 CPIM content

The RCS client:

1. shall use the text/plain content type for the Reaction message;
2. shall add the additional headers referred to in section 3.2.8.10 for a Reaction message, and
3. shall set the content of the CPIM body to emoji sequences encoded in UTF-8. Refer to [Emoji-Seq] for encoding rules.

NOTE: Any vendor-specific reaction carried inside a proprietary CPIM header is left to the RCS client implementation and is outside the scope of this document.

3.2.8.5.7 Backward compatibility

3.2.8.5.7.1 Capability Discovery

A client supporting the Reaction feature should support older clients if the conversation does not support the Reaction feature, refer to section 3.2.8.5.2.

3.2.8.5.7.2 Backward compatibility format

A backward compatible Reaction message content should have an indication of a Reaction to the associated message, which is left to the client implementation. The maximum length of the message shall be provided by the RCS provider through the MAX SIZE IM configuration parameter defined in Table 167 in section A.1.3. The client may choose the language and the text direction to represent the message.

3.2.8.5.7.3 SMS Fallback

Reaction messages are not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send a Reaction message.

3.2.8.5.8 Examples of Reaction messages

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: K28f67bTydi45uf2GGd23
rcs.Reference-Type: +Reaction

Content-Type: text/plain;charset=UTF-8
Content-Length: [content-length]

❤️‍🩹
```

Table 35: Add Reaction message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: j2324244cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: 0dRpW254cUYz9Oh23E624vXj
rcs.Reference-Type: -Reaction

Content-Type: text/plain;charset=UTF-8
Content-Length: [content-length]

❤️‍🩹
```

Table 36: Remove Reaction message

The following are examples of backward compatible messages. The client may choose the content, language, and text direction to represent the message.

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh243423423

Content-Type: text/plain;charset=UTF-8
Content-Length: 22

Added ❤️‍🩹 to “I am ok” message
```

Table 37: Emoji Reaction backward compatible message example

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz35434324vXj

Content-Type: text/plain;charset=UTF-8
Content-Length: ...

Added ❤️‍🩹 to an image
```

Table 38: Emoji Reaction backward compatible message example for an image

3.2.8.6 Custom Reaction

3.2.8.6.1 Overview

An RCS client can send a Custom Reaction message based on an existing or generated image, if the Extended Messaging Custom Reaction feature is supported. A Custom Reaction message is a File Transfer message with extended headers. An RCS client shall follow the File Transfer procedures in section 3.2.5.3 for Standalone messaging, 1-to-1 Chat and Group Chat.

3.2.8.6.2 Custom Reaction Feature Availability

For the Extended Messaging Custom Reaction feature to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Custom Reaction capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Custom Reaction feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Custom Reaction capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.6.3 Sender procedures

3.2.8.6.3.1 Adding procedures

The RCS client shall provide an option to send a Custom Reaction associated with an RCS message. When the user adds a Custom Reaction via the User Interface, the client shall follow the File Transfer procedures in section 3.2.5.3.1 without uploading a thumbnail.

In addition, the client shall add the additional CPIM headers described in section 3.2.8.6.5

The RCS client shall store the imdn.Message-ID of the Custom Reaction message, which will be used later if the user wishes to remove the reaction.

Message Revocation (see section 3.2.3.8.2) is not applied for the Custom Reaction feature.

3.2.8.6.3.2 Removing procedures

The RCS client shall provide an option to remove an already sent Custom Reaction. When the user removes a Custom Reaction via the User Interface, the client shall follow the procedures described in section 3.2.8.6.5 to set the CPIM message content.

Message Revocation (see section 3.2.3.8.2) is not applied for the Custom Reaction feature.

3.2.8.6.4 Receiver procedures

When the client receives a File Transfer message, the client:

1. shall check for the presence of the rcs.Reference-Type value header defined in section C.3.9. If present and if its value is “+Custom-Reaction”, the message is handled as a Custom Reaction Add message.
2. shall lookup the original message based on the message-ID provided in the rcs.Reference-ID header defined in section C.3.8. If the original message does not exist, or the sender is not part of the conversation, the Reaction message shall be silently ignored.
3. shall follow the File Transfer procedures in section 3.2.5.3.2
4. shall store the message-ID of the Custom Reaction message parsed from Message-ID header.
5. shall add the Custom Reaction to the referenced message in the User Interface.

When the client receives a text/plain message, the client:

1. shall check for presence of the rcs.Reference-Type value header defined in section C.3.9. If present and if its value is “-Custom-Reaction”, the message is handled as a Custom Reaction Remove message.
2. shall lookup the Custom Reaction based on the message-ID provided in the rcs.Reference-ID header defined in section C.3.8.
3. If the sender of the removal message is not the same sender of the File Transfer message carrying the Custom Reaction, the removal message shall be silently ignored.
4. shall remove the Custom Reaction from the referenced message in the User Interface

3.2.8.6.5 CPIM content

The RCS client shall add the additional headers defined in section 3.2.8.10 for a Custom Reaction message.

For adding a Custom Reaction, the RCS client shall follow the procedures in section 3.2.5.3.1 to set the content type and the body of the CPIM message.

For removing a Custom Reaction, the RCS client shall use the text/plain content type and shall set the content of the CPIM body to be empty.

3.2.8.6.6 Backward compatibility

If the Custom Reaction feature is not available in the conversation (refer to section 3.2.8.6.2), the client shall not the allow the user to send a Custom Reaction.

3.2.8.6.6.1 SMS Fallback

The Custom Reaction feature is not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send a Custom Reaction.

3.2.8.6.7 Examples of Custom Reaction messages

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: K28f67bTydi45uf2GGd23
rcs.Reference-Type: +Custom-Reaction

Content-Type: application/vnd.gsma.rcs-ft-http+xml
Content-Length: [content length]

<?xml version="1.0" encoding="UTF-8"?>
<file>
  <file-info type="file">
   <file-size>183524</file-size>
   <file-name>DSC_379395051.JPG</file-name>
   <content-type>image/jpeg</content-type>
   <data
url="https://ftcontentserver.rcs.mnc123.mcc456.pub.3gppnetwork.org/ftsf-
0d5ea6d1-a94c-2-9634-2d90244d3e8e" until="2024-04-25T12:17:07Z"/>
 </file-info>
</file>
```

Table 39: Custom Reaction Add message

```
imdn.Message-ID: 23499fuq34fuG9eRh98y3
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: 0dRpW254cUYz9Oh23E624vXj
rcs.Reference-Type: -Custom-Reaction

Content-Type: text/plain;charset=UTF-8
Content-Length: 0
```

Table 40: Custom Reaction Remove message

3.2.8.7 Edit Message

3.2.8.7.1 Overview

An RCS client can edit a previously sent message. If the Extended Messaging Edit Message feature is supported, the sender can edit their messages. Only plain text, emoticon, File Transfer, Audio Messaging and geolocation messages can be edited.

The Edit Message feature is based on Standalone messaging, 1-to-1 Chat and Group Chat procedures.

3.2.8.7.2 Edit Message Feature Availability

For the Extended Messaging Edit Message feature to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Edit Message capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Edit Message feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Edit Message capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.7.3 Sender add-on procedure

If an RCS client wants to send an Edit Message message, it shall:

1. follow section 2.14 for Extended RCS CPIM headers support, and
2. add the additional headers defined in section 3.2.8.10 for an Edit Message message.

3.2.8.7.4 Receiver procedure

When an RCS client receives a message of the supported types for an edit, it shall check for the presence of the “rcs” namespace defined in section C.3.1. If present, the client:

1. shall check for the presence of the rcs.Reference-Type CPIM header defined in section C.3.9. If present and if set to “Edit”, the message shall be treated as an Edit Message message.
2. lookup the message referenced in the rcs.Reference-ID header defined in section C.3.8.
3. If the referenced message was not sent by the same sender of the Edit Message message, the Edit Message message shall be silently ignored.
4. If the referenced message was not found the Edit Message message shall be displayed as new message to the user with an Edit Message Indication.
5. shall show the new message in the User Interface indicating that it is an edit of the previously sent message.

3.2.8.7.5 Backward compatibility

If the Edit Message feature is not available in the conversation (refer to section 3.2.8.7.2), the client implementation shall handle this scenario.

3.2.8.7.6 SMS Fallback

The Edit Message feature is not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send an Edit Message message.

3.2.8.7.7 Example of Edit Message

3.2.8.7.7.1 Original message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj

Content-Type: text/plain;charset=UTF-8
Content-Length: …

Let’s meet boday at 7:30PM!
```

Table 41: Original message example

3.2.8.7.7.2 Edit Message message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: K28f67bTydi45uf2GGd23
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: 0dRpW254cUYz9Oh23E624vXj
rcs.Reference-Type: Edit

Content-Type: text/plain; charset=UTF-8
Content-Length: …

Let’s meet today at 8:00PM!
```

Table 42: Edit Message message example

3.2.8.7.7.3 File transfer Edit Message message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: 0dRp34334cc33h23E624vXj
rcs.Reference-Type: Edit

Content-Type: application/vnd.gsma.rcs-ft-http+xml
Content-Length: [content length]

<?xml version="1.0" encoding="UTF-8"?>
<file>
  <file-info type="file">
   <file-size>183524</file-size>
   <file-name>DSC_379395051.JPG</file-name>
   <content-type>image/jpeg</content-type>
   <data
url="https://ftcontentserver.rcs.mnc123.mcc456.pub.3gppnetwork.org/ftsf-
0d5ea6d1-a94c-2-9634-2d90244d3e8e" until="2024-04-25T12:17:07Z"/>
 </file-info>
</file>
```

Table 43: File Transfer Edit Message message example

3.2.8.8 Message Recall

3.2.8.8.1 Overview

An RCS client can recall a previously sent message. If the Extended Messaging Message Recall feature is supported, the sender can recall their sent messages.

The Message Recall feature is based on Standalone messaging, 1-to-1 Chat and Group Chat procedures.

3.2.8.8.2 Message Recall Feature Availability

For the Extended Messaging Message Recall feature to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Message Recall capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Message Recall feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Message Recall capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.8.3 Sender Procedures

An RCS client shall provide an option to recall an RCS message shortly after they sent that message. When the user recalls a message via the User Interface, the client shall send the Recall Message body to the receiver(s). The client shall follow the procedures described in section 3.2.8.8.5 to set the CPIM content.

Message Revocation (see section 3.2.3.8.2) is not applied for the Message Recall feature.

3.2.8.8.4 Receiver procedure

When an RCS client receives a text/plain message, it shall check for the presence of the “rcs” namespace defined in section C.3.1. If present, the client:

1. shall check for the presence of the rcs.Reference-Type CPIM header defined in section C.3.9. If present and set to “Recall”, the message shall be treated as a Recall Message message.
2. shall lookup the message referenced in the rcs.Reference-ID CPIM header defined in section C.3.8.
3. If the referenced message was not sent by the same sender of the Recall Message message, the Recall Message message shall be silently ignored.
4. If the referenced message was not part of the Group Chat conversation, in which the Recall Message message was sent, the Recall Message message shall be silently ignored.
5. If the Recall Message message was sent as a private message in a Group Vhat conversation, the Recall Message message shall be silently ignored.
6. shall recall/remove the recalled message in the User Interface.
7. shall remove any Incoming Event Notification of recalled message.

3.2.8.8.5 CPIM content

The RCS client shall add the additional headers defined in section 3.2.8.10 for the Message Recall feature and shall use the text/plain content type and set the content of the CPIM body to be empty.

3.2.8.8.6 Backward compatibility

If the Message Recall feature is not available in the conversation (refer to section 3.2.8.8.2), the client shall not the allow the user to send a Recall Message message.

3.2.8.8.6.1 SMS Fallback

The Message Recall feature is not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send a Recall Message message.

3.2.8.8.7 Example of a Message recall

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: K28f67bTydi45uf2GGd23
rcs.Reference-Type: Recall

Content-Type: text/plain;charset=UTF-8
Content-Length: 0
```

Table 44: Recall Message message example

3.2.8.9 Delete Message

3.2.8.9.1 Overview

An RCS client can delete a previously sent message. If the Extended Messaging Delete Message feature is supported, the sender can delete their sent messages.

The Delete Message feature is based on Standalone messaging, 1-to-1 Chat and Group Chat procedures.

3.2.8.9.2 Delete Message Feature Availability

For the Extended Messaging Delete Message feature to be available for a 1-to-1 Chat conversation or Standalone Messaging, the following is required:

- The recipients are known to support the Extended Messaging Delete Message capability defined in section 2.6.1.3 based on Capability Discovery.

For the Extended Messaging Delete Message feature to be available in a Group Chat conversation, the following is required:

- The Contact header received during the setup of the Group Chat included the Extended Messaging IARI tag defined in section 2.4.4.1
- The recipients are known to support the Extended Messaging Delete Message capability defined in section 2.6.1.3 based on Capability Discovery.

3.2.8.9.3 Sender Procedures

An RCS client shall provide an option to delete an RCS message. When the user deletes a message via the User Interface, the client shall send the Delete Message body to the receiver(s). The client shall follow the procedures described in section 3.2.8.9.5 to set the CPIM content.

Message Revocation (see section 3.2.3.8.2) is not applied for the Delete Message feature.

3.2.8.9.4 Receiver procedure

When an RCS client receives a text/plain message, it shall check for the presence of the “rcs” namespace defined in section C.3.1. If present, the client:

1. shall check for the presence of the rcs.Reference-Type CPIM header defined in section C.3.9. If present and set to “Delete”, the message shall be treated as a Delete Message message.
2. shall lookup the message referenced in the rcs.Reference-ID CPIM header defined in section C.3.8.
3. If the referenced message was not sent by the same sender of the Delete Message message, the Delete Message message shall be silently ignored.
4. If the referenced message was not part of the Group Chat conversation, in which the Delete Message message was sent, the Delete Message message shall be silently ignored.
5. If the Delete Message message was sent as a private message in a Group Chat conversation, the Delete Message message shall be silently ignored.
6. shall delete the referenced message in the User Interface.

3.2.8.9.5 CPIM content

The RCS client shall add the additional headers defined in section 3.2.8.10 for the Delete Message feature and shall use the text/plain content type and set the content of the CPIM body to be empty.

3.2.8.9.6 Backward compatibility

If the Delete Message feature is not available in the conversation (refer to section 3.2.8.9.2), the client shall not the allow the user to send a Delete Message message.

3.2.8.9.6.1 SMS Fallback

The Delete Message feature is not supported if RCS is not available. Network Fallback to SMS and Client Fallback to SMS shall not be used to send a Delete Message message.

3.2.8.9.7 Example of a Delete Message message

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: 0dRpW254cUYz9Oh23E624vXj
NS: rcs <http://www.gsma.com>
rcs.Reference-ID: K28f67bTydi45uf2GGd23
rcs.Reference-Type: Delete

Content-Type: text/plain;charset=UTF-8
Content-Length: 0
```

Table 45: Delete Message message example

3.2.8.10 CPIM headers

Header	Specification	Message Type	Value
rcs.Reference-ID	Section C.3.8	Reply Message, Edit Message, Recall Message and Delete Message	The RCS Message-ID of the associated message
		Reaction, Custom Reaction	The RCS Message-ID of the original message for adding a reaction or the RCS Message-ID of the message or the File Transfer message that carried the Reaction or Custom Reaction for removing a previously added reaction.
rcs.Reference-Type	Section C.3.9	Reply Message	“Reply”
		Reaction	"+Reaction” for adding a reaction, and “-Reaction” for removing a reaction
		Custom Reaction	"+Custom-Reaction” for adding a reaction and “-Custom-Reaction" for removing a reaction
		Edit Message	“Edit”
		Recall Message	“Recall”
		Delete Message	“Delete”

Table 46: CPIM headers usage for Extended Messaging

3.2.9 Spam and Other Inappropriate User Behaviour Handling

gRPC-configured RCS clients shall follow the procedures defined in section 4.7 of [GSMA PRD-RCC.30]. CPIM Header procedures are defined in section 5.2.3 of [GSMA PRD-RCC.30].

SIP-configured RCS clients shall follow the procedures in this section.

Protection against spam, fraud, inappropriate content or other inappropriate behaviour originating from any RCS user can be realised by preventing access from and to such users:

- in the Spam Block function in the Service Provider network or
- in the client.

Where this protection is performed depends mostly on which party considers the user as behaving inappropriately. The exception would be Service Providers that can make use of their capabilities for managing the client to inform a client of the user that they consider problematic. This could be used either in addition to preventing the access in the Service Provider network or as an alternative.

If access to a user is prevented in a Spam Block function in the Service Provider network, this shall lead to the following behaviour:

- A SIP 403 Forbidden response including a Warning header field including the warning text set to "206 Spam Sender" shall be returned to a SIP request initiated by the client.

How this behaviour is realised is network internal and is therefore out of scope of this document.

A user shall be able to block the reception of incoming user communication sessions and messages through the client local blacklist as defined in section 18 of [GSMA PRD-RCC.71].

A Service Provider shall be able to prevent access to users for both incoming and outgoing communication based on its internal network infrastructure.

3.2.9.1 Spam Report Message

3.2.9.1.1 Overview

The Spam Report Message allows a client to report an RCS user as a spammer or as an identifier involved in fraud, sending inappropriate content, or other inappropriate behaviour. This functionality shall be supported if the configuration parameter IM-SPAM-REPORTING-URI defined in section A.1.3 is present.

The Spam Report Messages are system messages and as such:

- they are not sent with CPIM headers, and a delivery and/or displayed notification shall not be requested,
- there is no store and forward.

The Spam Report Messages are conveyed in a spam report system message (SIP MESSAGE) to the application server. They are targeted to the configuration parameter IM-SPAM-REPORTING-URI and shall be sent only by RCS clients. The Service Provider application server shall process the Spam Report Messages and take unilateral blocking action.

To categorize them among other system messages, Spam Report Messages shall use the following IARI:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.imspam"

The Service Provider may trigger (based on this IARI) an element in its network to process the Spam Report Message. As with other SIP requests, the Spam Report Message shall be routed towards the AF (if deployed in the Service Provider network).

The Spam Report XML schema is defined as shown in Table 47.

The associated MIME content type is:

application/vnd.gsma.rcsspam-report+xml

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:spamreport"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="urn:gsma:params:xml:ns:rcs:rcs:spamreport "
        elementFormDefault="qualified"
        attributeFormDefault="unqualified">
   <xs:element name="SR">
      <xs:complexType>
        <xs:element name="ReportedIdentity">
          <xs:simpleType>
            <xs:restriction base="xs:anyURI"/>
          </xs:simpleType>
        </xs:element>
        <xs:element name="Message-ID" minOccurs="0" maxOccurs="10">
          <xs:simpleType>
            <xs:restriction base="xs:token"/>
          </xs:simpleType>
        </xs:element>
        <xs:element name="Content" type="xs:string"/>
        <xs:any namespace="##other" processContents="lax" minOccurs="0"
        maxOccurs="unbounded"/>
      </xs:complexType>
   </xs:element>
</xs:schema>
```

Table 47: RCS Spam Report Message body schema

The schema defined in Table 47 is extended to enable the client to include a spam type and free text where the user can explain the reason for classifying the reported RCS user (ReportedIdentity) as inappropriate in some way:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:spamreportext"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="urn:gsma:params:xml:ns:rcs:rcs:spamreportext"
        elementFormDefault="qualified"
        attributeFormDefault="unqualified">
   <xs:element name="spam-type" type="xs:string"/>
   <xs:element name="free-text" type="xs:string"/>
</xs:schema>
```

Table 48: RCS Spam Report Message body schema Extension

3.2.9.1.2 Client Procedures

3.2.9.1.2.1 Sending a Spam Report Message

When a Spam Report Message is sent, the client shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the client:

1. shall set the Request-URI and To header field to the configuration parameter IM-SPAM-REPORTING-URI defined in section A.1.3.
2. shall include the address of the originating RCS client that has been authenticated as per [GSMA PRD-RCC.11];
3. shall set the P-Preferred-Service header field with the value set to

‘urn:urn-7:3gppservice.ims.icsi.oma.cpm.systemmsg’;

4. shall include an Accept-Contact header field with the CPM Feature Tag ‘urn:urn7:3gpp-service.ims.icsi.oma.cpm.systemmsg’ and carrying the Spam Report Message IARI tag defined in section 3.2.9.1.1;
5. shall include a User-Agent header field as specified in Annex C.4.1;
6. shall include the Content-Type header field with the value set to the Spam Report Message content type application/vnd.gsma.rcsspam-report+xml, as described in section 3.2.9.1.1;
7. shall create a Spam Report Message as described in section 3.2.9.1.1 and set the body of the Spam Report Message, as follows:

    a) The <ReportedIdentity> element set to the address of the reported RCS user or Chatbot;
    b) Zero or more, up to 10, <Message-ID> elements set to the value of the imdn.message-ID(s) of the original message(s) that is(are) requested to be reported as spam if any;
    c) Optionally the <Content> element with the text of up to 2000 octets with the content of the messages reported as spam. If multiple message IDs are reported and the <Content> element is included, all reported messages shall have the same content. For non-text messages, the <Content> element shall not be included;
    d) When sending a Spam Report Message related to a File Transfer and if the IM SPAM REPORTING FILE CONTENT configuration parameter defined in section A.1.3 is set to 1, optionally a <file> element as defined in Table 157 in section 4.2.4

        i. Reflecting the content of the corresponding element in the received File Transfer message body of the File Transfer that is being reported if the file was not encrypted and the validity of the file (see section 3.2.5.3.2) indicates that it is still available in the network.
        ii. Reflecting the content of the corresponding element obtained from uploading the file and if applicable the thumbnail unencrypted to the HTTP Content Server using the procedure defined in section 4.2.2 if the file was encrypted or the validity of the file (see section 3.2.5.3.2) indicates that it is no longer available in the network. In this upload the RCS client shall include the purpose indication setting it to “Spam Report”.

NOTE: as indicated in section 4.2.2.1, the policies associated to the purpose element are up to the RCS Service Provider. When the purpose is set to “Spam Report”, the links might be accessible only by the RCS Service Provider’s application servers and might be invalidated when the spam report has been processed. RCS clients can thus not assume that the received links can be used for other purposes.

NOTE: Given that it is not part of default namespace for the Spam Report as defined in section 3.2.9.1.1, this <file> element will have to refer to the

corresponding namespace (i.e. as defined in Table 157 in section 4.2.4) and thus the file element and its sub-elements will have to be prefixed when including them in body of the Spam Report Message rather than reusing the XML as received in the File Transfer message body or obtained from the unencrypted upload as is.

e) Optionally the <spam-type> element set to “spam”, “fraud”, “inappropriate-content”, or “other”. If this element is not included it shall be handled as if it had been included with the value “spam” and
f) Optionally the <free-text> element with text of up to 500 octets

If the client receives in result of processing the request for the Spam Report SIP MESSAGE request:

- any error, then the client shall notify the user that their request has not been processed.

3.2.9.2 Suspicion Indicator

This section defines the use of the CPIM headers to convey the Suspicion Indicator.

When server-side abuse protections are available in the terminating Messaging Server, a Suspicion Indicator may be supported by the Service Provider through the cpim extensions defined in section C.3.10 and C.3.11 (with an example in section C.3.12). In such case, the Messaging Server shall insert a Suspicion Indicator using these headers to any message that is considered suspicious but is not blocked from delivery.

NOTE: how the Service Provider determines whether a message is suspicious is out of scope of this specification.

3.2.9.2.1 Recommended Client Behaviour

It is recommended that the client takes the following actions on receiving an RCS message containing the CPIM headers conveying the Suspicion Indicator:

- **`rcs.advised-action: warn`** - a warning message should be displayed to the user. The user should be offered the options to move the message to the spam message folder or to proceed with caution. If the user agrees that the message is spam, a spam notification shall be sent to provide positive-reinforcement feedback to the protection system as described in section 3.2.9.1.
- **`rcs.advised-action: hide`** - the message should be automatically moved to the Spam Folder.

NOTE: **`rcs.source`** provides additional information on the system providing the Suspicion Indicator and may be of use when debugging message flows.

3.2.10 Rich Text Message

3.2.10.1 Overview

The Rich Text Message feature enables the application of text styling and the sending and receiving of those messages. The specific visual styles and elements supported (e.g., bold, italic) are defined in US5-46 of [GSMA PRD-RCC.71] including their version.

3.2.10.2 Capability

For the Rich Text Message feature to be available for a 1-to-1 Chat conversation, the following conditions shall be met:

- The application/vnd.gsma.richtextmessage+proto content type is indicated in the a=accept-wrapped-types attribute during the SDP negotiation of the 1-to-1 Chat session.
- The recipient is known to support the Rich Text Message capability defined in section 2.6.1.3, based on Capability Discovery.

For the Rich Text Message feature to be available in a Group Chat conversation, the following conditions shall be met:

- The application/vnd.gsma.richtextmessage+proto content type is indicated in the a=accept-wrapped-types attribute by the Messaging Server in the SDP for the Group Chat session.
- All recipients in the Group Chat are known to support the Rich Text Message capability defined in section 2.6.1.3, based on Capability Discovery.

The Rich Text Message capability uses a cumulative versioning scheme. A client that advertises support for Rich Text Message version vN using the feature tag +g.gsma.rcs.msg.richtext="vN" (see sections 2.4.4.1 and 2.6.1.3) shall also support all Rich Text Formatting styles defined for all lower versions in US5-46 of [GSMA PRD-RCC.71].

When a conversation includes participants that support different Rich Text Message versions, an RCS client shall offer a user composing a Rich Text Message only styles that are included in the highest commonly supported version.

3.2.10.3 Payload

3.2.10.3.1 Content type

The Rich Text Message payload shall be encoded using the MIME content type:

```
application/vnd.gsma.richtextmessage+proto
```

The Content-Type header field shall include the rt-ver media type parameter. The value of rt-ver represents the Rich Text Formatting version and shall use the same version value as the *version* field of RichTextMessage defined in section 3.2.10.3.3.

Example:

```
Content-Type: application/vnd.gsma.richtextmessage+proto; rt-ver=1
```

This content type is carried in the body of a CPIM message according to the procedures for 1-to-1 Chat (section 3.2.3) and Group Chat (section 3.2.4).

3.2.10.3.2 Payload structure

The Rich Text Message payload body shall be encoded using Protocol Buffers version 3 (proto3). The schema is defined as follows:

```
syntax = "proto3";

enum RichTextEntityType {
  UNSPECIFIED= 0;
  BOLD = 1;
  ITALIC = 2;
  UNDERLINE = 3;
  STRIKETHROUGH = 4;
}

message RichTextEntity {
  RichTextEntityType type = 1;
  uint32 offset = 2;
  uint32 length = 3;
}

message RichTextMessage {
  string version = 1;
  string text = 2;
  repeated RichTextEntity entities = 3;
}
```

Table 49: Rich Text Message Protobuf schema

The *RichTextEntityType* enumeration represents the Rich Text Formatting style elements defined in in US5-45 of [GSMA PRD-RCC.71]. For each style element defined in that table, the corresponding *RichTextEntityType* name shall be the upper-case form of the style name specified there, and its numeric value shall be assigned starting from 1 in the same order in which the style elements appear in the table.

3.2.10.3.3 Payload semantics

When using the data structure defined in section 3.2.10.3.2, the field *version* shall contain the lowest Rich Text Formatting version that includes all styles used in the payload (for example, "1" for version 1).

The field text shall contain the plain text content of the message. Each entity in *entities* applies the style indicated by its type to a portion of this text.

offset and *length* shall be measured in UTF-16 code units. For each entity, styling shall apply to the part of the text that starts at position *offset* and has *length* UTF-16 code units. The following must be true:

- offset >= 0
- length > 0

- offset + length <= the UTF-16 length of text.

Nested and overlapping entities are permitted. Multiple entities may apply to the same substring or character (e.g. text that is both bold and italic). The value UNSPECIFIED (0) of the *RichTextEntityType* enumeration is reserved. Rich Text Message senders shall not set the type field of any *RichTextEntity* to UNSPECIFIED.

3.2.10.3.4 Encoding and decoding

The payload shall consist of the raw binary data resulting from the Protocol Buffers serialization of the message defined in section 3.2.10.3.2. The client shall decode the binary payload using the same schema.

3.2.10.3.5 CPIM example

The following is an informative example of a CPIM message carrying a Rich Text Message payload.

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: ab889Xy697

Content-Type: application/vnd.gsma.richtextmessage+proto; rt-ver=1
Content-Length: 18

<18 bytes of binary Protobuf data>
```

Table 50: Rich Text Message CPIM example

3.2.10.4 Sender procedures

When the Rich Text Message capability is available for the destination (as defined in section 3.2.10.2), the client shall send a message that includes Rich Text Formatting as a Rich Text Message with the payload defined in section 3.2.10.3.

The client shall not perform automatic Client Fallback to SMS for any Rich Text Message.

When the client receives a "negative-delivery" IMDN with status "failed" containing the <unsupported-content> element for a sent Rich Text Message, the client should initiate a Capability Discovery check to refresh the Rich Text Message capability defined in section 3.2.10.2.

- If the refreshed capabilities indicate that the Rich Text Message capability or version used is not supported by the recipient, the client shall update the stored capabilities, inform the user, and offer the user a way to resend the message accordingly.
- Otherwise, the client shall treat this as an abnormal condition and inform the user that a recipient-side problem occurred.

3.2.10.5 Receiver procedures

Upon receiving a Rich Text Message, the client shall decode the payload defined in section 3.2.10.3.

- If decoding of the payload defined in section 3.2.10.3.2 fails, the client shall send a "negative-delivery" IMDN with status "failed" as defined in [GSMA PRD-RCC.11] and shall not render the message content.
- If the decoded payload does not follow the rules defined in section 3.2.10.3.3, the client shall send a “negative-delivery” IMDN with status “failed” and shall not render the message content.
- If the decoded payload contains a Rich Text Message version that is not supported by the client, the client shall send a "negative-delivery" IMDN with status "failed" as defined in [GSMA PRD-RCC.11], whose <failed> element contains a <not-supported> element with an <unsupported-content> child element, and shall not render the message content.
- Otherwise, the client shall generate the "delivery" IMDN according to the procedures defined in [GSMA PRD-RCC.11].
- When the message is displayed to the user in the conversation, the client shall render the message content based on the indicated styles and shall generate the "display" IMDN according to the procedures defined in [GSMA PRD-RCC.11].

3.2.11 User Profile sharing

3.2.11.1 Overview

This feature allows a user to share their User Profile (name and image) to designated contacts in RCS 1-to-1 and Group Chats.

3.2.11.2 Capability Discovery

The client shall advertise the support for User Profile sharing if:

- The User Profile sharing capability defined in section 2.6.1.3 was provided based on Capability Discovery.
- The Contact header received during the setup of the Group Chat included the User Profile IARI tag defined in section 2.4.4.1

3.2.11.3 User Profile Information Exchange Format

The RCS client shall send the profile information in the ProfileConfig protocol buffer as defined in Table 51. The ProfileConfig protocol buffer shall be Base64 encoded. The RCS client includes the resulting Base64 string in rcs.User-Profile-Config-Data CPIM header as per section C.3.15

The ProfileConfig protocol buffer shall include:

- The `profile_id` which the User Profile Server uses to locate the shared User Profile.
- The `token` which is the authentication token necessary to retrieve the shared User profile.
- The `profile_provider_id` which allows the Home User Profile Server to locate the Foreign User Profile Server where the shared User Profile is stored.

For Group Chat messages, the RCS client may also include the `share_to_id` along with `share_acl` to indicate to the Controlling Function which participant in a Group Chat can receive the ProfileConfig as illustrated in Table 52.

```
message ProfileConfig {
   enum ShareAcl {
     UNKNOWN = 0;
     ALL = 1;
     LIST = 2;
   }

   ShareAcl share_acl = 1;

   // if share_acl is LIST, list numbers
   repeated string share_to_id = 2;
   String profile_id = 3;
   bytes token = 4;
   int profile_provider_id = 5;
}
```

Table 51: ProfileConfig protocol buffer definition

```
{
  "share_acl": "LIST",
  "share_to_id": [
    "+15550109988",
    "+15550107766",
    "+15550105544"
  ],
  "profile_id": "user_8829",
  "token": "EjRWeJq83vA=",
  "profile_provider_id": 101
}
```

Table 52: ProfileConfig Group Chat message protocol buffer example

3.2.11.4 User Profile sharing procedures

3.2.11.4.1 Controlling the sharing of the User Profile

After the creation or update of the User Profile as per section 4.3.3 has completed, the RCS client shall store in the user's setting the categories or lists of recipients that are allowed to receive the shared User Profile information.

After the creation or update of the User Profile as per section 4.3.3 has completed, the RCS client shall share the User Profile information with the allowed recipients when the user sends a message in a 1-to1 or Group Chat. Once the sharing is completed, the RCS client may share the User Profile periodically or upon Group Chat membership changes, the RCS client should not send the User Profile information on every message.

3.2.11.4.2 1-to-1 Chat User Profile sharing

After the creation of a User Profile as per section 4.3.3 has completed, when the user chooses to send a message in the 1-to-1 Chat, to share the User Profile the RCS client shall:

1. Create the ProfileConfig protocol buffer as per section 3.2.11.3. The RCS client shall then Base64 encode the ProfileConfig protocol buffer. It includes the resulting Base64 string in an rcs.User-Profile-Config-Data CPIM header as per section C.3.15 and sends the RCS message.

The steps of this procedure are illustrated in Figure 14

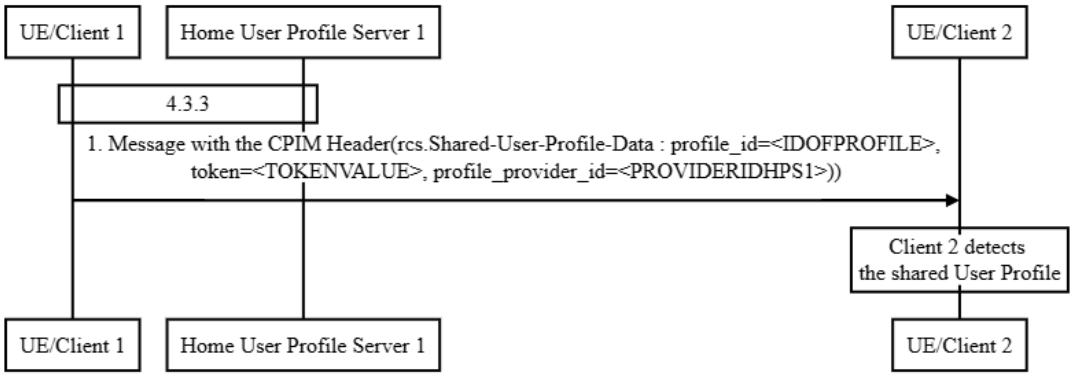


Figure 14: 1-to-1 Chat User profile sharing

3.2.11.4.3 Group chat User Profile sharing

The RCS client may share the User Profile using direct User Profile sharing as per section 3.2.11.4.3.1 or using batched User Profile sharing as per section 3.2.11.4.3.2.

3.2.11.4.3.1 Direct User Profile sharing

Direct User Profile sharing allows the RCS client to share the User Profile to individual Group Chat participants using a 1-to-1 Chat message.

After the creation of a User Profile as per section 4.3.3 has completed, when the user chooses to send a message in the Group Chat, to share the User Profile the RCS client shall:

1. Create a CPIM container with an empty body with Content-Length of 0 bytes as illustrated in Table 53.
2. Follow the sharing procedure in section 3.2.11.4.2 to share the User Profile individually with each allowed Group Chat participant in the corresponding 1-to-1 chat with that participant.

Upon receiving the 1-to1 Chat message, the recipient's RCS client shall discard the empty body message silently.

```
NS: imdn <urn:ietf:params:imdn>
imdn.Message-ID: K28f67bTydi45uf2GGd23
rcs.User-Profile-Config-Data:
ZGF0YTEsZGF0YTIsZGF0YTMsZGF0TQsZGF0YTUsZGF0YTYsZGF0YTcsZGF0YTgsZGF0YTksZ
GF0YY

Content-Type: text/plain;charset=UTF-8
Content-Length: 0
```

Table 53: Empty body message example

3.2.11.4.3.2 Batched User Profile sharing

Batched User Profile sharing allows the RCS client to configure, using a single RCS Group Chat message, the Controlling Function of the Group Chat with an allow list of what participants can receive the User Profile information.

After the creation of a User Profile as per section 4.3.3 has completed, when the user chooses to send an RCS message in the Group Chat, to share the User Profile the RCS client shall:

1. Format the User Profile configuration information into a ProfileConfig protocol buffer as per section 3.2.11.3.
2. Include the `share_to_id` containing the list of participants allowed to receive the User Profile along with `share_acl` as per section 3.2.11.3 into the ProfileConfig protocol buffer.
3. Base64 encode the ProfileConfig protocol buffer.
4. Include the resulting Base64 string in an rcs.User-Profile-Config-Data CPIM header as per section C.3.15.
5. Send the RCS message.

Upon receiving a message with the rcs.User-Profile-Config-Data CPIM header, the Controlling Function shall:

6. Check the contents of `share_acl` and `share_to_id` and determine which participants of the Group Chat shall receive the rcs.User-Profile-Config-Data CPIM header.

For the participants allowed to receive the rcs.User-Profile-Config-Data CPIM header, the Controlling Function shall:

7. Format the profile configuration information into a ProfileConfig protocol buffer as per section 3.2.11.3.
8. Base64 encode the ProfileConfig protocol buffer.
9. Include the resulting Base64 string as an rcs.User-Profile-Config-Data CPIM header in the message as per section C.3.15.
10. Send the RCS message.

The steps of this procedure are illustrated in Figure 15.

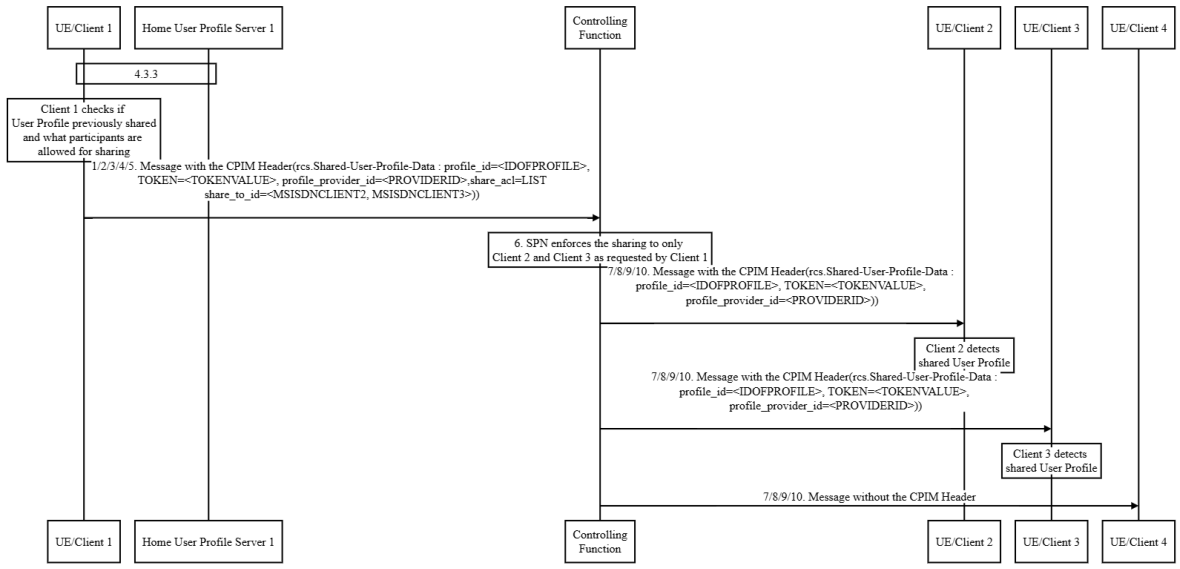


Figure 15: Group Chat User profile sharing

3.3 Void

3.4 Void

3.5 Void

3.6 Chatbots

This section describes the architecture and technical enablers supporting Chatbots in 1-to-1 conversations to realise the functionality described in section 15.2 of [GSMA PRD-RCC.71].

3.6.1 Architecture

Chatbots require support functionality in both the RCS Service Provider network and the Chatbot Platform. Figure 16 provides an overview of how that functional split is done whereby optional functions in Chatbot Platform or Service Provider network are indicated through dashed lines shaded boxes. Interfaces that are out of scope of this specification are indicated in a similar way.

NOTE: In this section, it has been assumed that the Chatbot Platform has been integrated in the overall architecture for RCS as described in section 2.1 and [GSMA PRD-IR.90]. The section therefore focuses on the mentioned functional split between the Chatbot Platform and the Service Provider network assuming that the connection between these is in place.

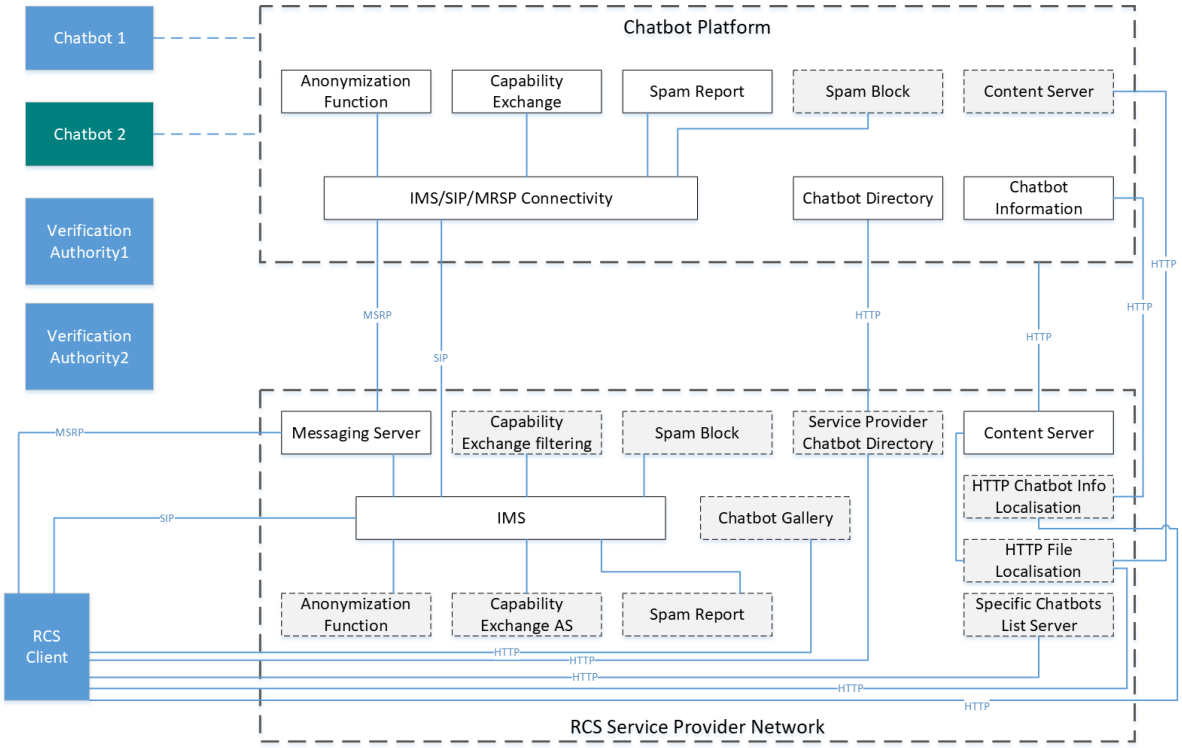


Figure 16: Functional Split between Service Provider Network and Chatbot Platform

NOTE: Figure 16 shows the case where HTTP Localisation functions are present. If not configured to use HTTP localisation for a function, the client shall contact the corresponding function directly.

NOTE: If the Chatbot Platform is provided by the Service Provider themselves, the Chatbot Platform is network internal and therefore it is up to the Service Provider to define the functional split between the different functions in their network.

NOTE: Figure 16 is a functional split, which is also reflected in the naming of the functions (e.g. Service Provider Chatbot Directory). Depending on commercial agreements and chosen deployment model, functions in the RCS Service Provider Network could be provided also by the Chatbot Platform or by a 3rd party provider. Similarly, functions in the Chatbot Platform could also be provided by an RCS Service Provider or by a 3rd party provider.

NOTE: Figure 16 is an illustration of a SIP-configured RCS client connected to an IMS-based RCS service. gRPC-configured clients can connect to an RCS service to connect to a Chatbot Platform without changing the functional split between the RCS Service Provider Network and the Chatbot Platform.

In the Service Provider network, the following functionality shall be required to enable the Chatbot use cases:

- SIP/IMS/gRPC routing and authentication functionality to receive the corresponding requests and responses from the client and the Chatbot Platform and route them to the required functions in the network
- A Messaging Server supporting 1-to-1 conversations with deferred messaging functionality
- Functionality to provide interworking to xMS is not required
- A function to support Capability Discovery that is mandatory when the interconnection to the Chatbot Platform relies on SIMPLE Presence for the Capability Discovery (see section 2.6.1.2) and optional if either SIP OPTIONS (see section 2.6.1.1.1) or RCS gRPC Capability Discovery (see section 4.4 of [GSMA PRD-RCC.30]) are used (see section 2.6.1.1.1).
- An optional Spam Report Function which is required only if the RCS Service Provider wants to process the Spam Reports sent by their subscribers (see section 3.6.6.2)
- An optional Spam Block Function preventing bots considered as Spam senders from contacting and being contacted by RCS users (see section 3.6.6).
- An optional Specific Chatbots List Server that may be providing the client with lists of Chatbot Service IDs (see section 2.5.4.1) of Chatbots requiring specific management (see section 3.6.3.3).
- An optional Anonymization Function as described in section 3.6.5.1 which is required if subject to the interworking agreements for the connection with the Chatbot Platform anonymization of the user towards the Chatbot is done by the Service Provider
- An optional Capability Exchange Filter Function for the filtering of capabilities exchanged with the Chatbot Platform if the RCS Service Provider wants to limit the RCS Services available to the Chatbot Platform as described in section 15.2.1 of [GSMA PRD-RCC.71]
- A Content Server to host the file content sent by the user to the Chatbot Platform
- An optional Chatbot Gallery to host the Chatbot Gallery logic displayed on the RCS client.
- An optional Service Provider Chatbot Directory Function to allow the user to discover Chatbots as described in section 3.6.3.1.

NOTE: The interfaces and processes to populate the directory are out of scope of this specification.

- Optional HTTP Localisation Functions to ensure that the HTTP traffic relating to Chatbots is routed through the Service Provider Network (as described in section 4.1.15.3). If this is not provided the client shall interact directly with the Chatbot functions that rely on HTTP (i.e. Content Servers and Chatbot Information Function).

In the Chatbot Platform, the following functionality shall be required to enable the Chatbot use cases:

- Connectivity for SIP/MSRP to send and receive requests from the RCS Service Providers relating to the Chatbots that it hosts

NOTE: In future, other protocols may be defined to provide this functionality.

- An Anonymization Function which is required if subject to the interworking agreements for the connection with the Chatbot Platform anonymising the user identity towards the Chatbot is done by the Chatbot Platform
- Capability Exchange Functionality to answer Capability Discovery requests for a Chatbot's capabilities
- Functionality to receive Spam Reports
- An optional Spam Block Function preventing bots considered as Spam senders from contacting RCS users
- An optional Content Server to host content sent by the Chatbots to the user. If this is not provided, the content exchanged must be hosted on webservers available on the internet and would be accessible only by users of RCS Service Providers that allow access to such content.
- A Chatbot Information Function providing contact details and other information on the Chatbots hosted by the Chatbot Platform as described in section 3.6.4.
- A Chatbot Directory Function to allow the RCS Service Provider to retrieve the Chatbot Information of the hosted Chatbots.

Next to those, there are Verification Authorities that act as a 3rd party verifying the data provided by the Chatbot in the Chatbot Information.

The following is not in scope of this specification:

- The interfaces that are internal to a Service Provider Network or Chatbot Platform. This includes the interfaces between the Service Provider Network and the Chatbot Platform for the case where both are provided by the same Service Provider.
- The interfaces between Chatbots and Verification Authorities and the interfaces between Verification Authorities and RCS Service Provider networks. Those interfaces are based on business processes rather than technical procedures.
- The interface between Chatbot and Chatbot Platform
- The Chatbots' logic
- The interaction of Chatbots with xMS messaging
- The Service Provider Chatbot Directory Function logic i.e. which entries will be returned based on the provided parameters and the information obtained from the Chatbot Platform Directory Function and in which order those entries will be returned

3.6.2 Chatbot Feature tags

3.6.2.1 Chatbot IARI value

The Chatbot IARI value is defined so that an entity can declare its support for Chatbots in general.

If the Chatbot communication is based on Chatbot Chat Sessions, the Chatbot IARI value is set to *urn:urn-7:3gpp-application.ims.iari.rcs.chatbot*, defined according to [3GPP TS 24.229]. An example of the IARI value carried in the IARI feature tag is shown below:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot"

If the Chatbot communication is based on Chatbot Standalone Messages, the Chatbot IARI value is set to *urn:urn-7:3gpp-application.ims.iari.rcs.chatbot.sa*, defined according to [3GPP TS 24.229]. An example of the IARI value carried in the IARI feature tag is shown below:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot.sa"

3.6.2.2 Chatbot application version

The Chatbot application version is represented by a decimal number. It indicates support by the entity for the content types and procedures defined in this specification for Chatbot communication. The Chatbot application version shall be increased with any new release by "1". The initial version of the Chatbot application version is "1". It indicates to an entity the Chatbot application versions supported by the entity to which it has connected.

The Chatbot application version defined in this document is "3". Because the functionality provided in this version is a superset of the version "2", clients and Chatbot Platforms shall indicate support for previous versions as well.

The indication of the Chatbot application version supported by an entity uses a feature parameter as defined in [RFC3840]. The feature parameter is encoded as an "other-tags" feature tag in accordance with the definitions in section 9 of [RFC3840].

The feature tag name shall be set to "g.gsma.rcs.botversion". The tag value shall be encoded as "numeric" in accordance with the definitions in section 9 of [RFC3840]. Multiple tag values can be included in the Chatbot application version media feature tag.

Security considerations for this media feature tag are discussed in section 11.1 of [RFC3840].

Non-normative examples:

- A Chatbot application version media feature tag indicating the support of all Chatbot application versions up to "4" could be represented as follows:
  - +g.gsma.rcs.botversion="#<=4", or
  - +g.gsma.rcs.botversion="#=1,#=2,#=3,#=4"
- A Chatbot application version media feature tag indicating the support of Chatbot application versions "2", "4", "5" and "6" could be represented as follows:
  - +g.gsma.rcs.botversion="#=2,#=4,#=5,#=6", or
  - +g.gsma.rcs.botversion="#=2,#4:6"

3.6.2.3 Chatbot role

The indication of the Chatbot role is a boolean expression with a value of "true". It indicates to a client that the server to which it has connected is representing a Chatbot.

The indication of the Chatbot role is provided by a feature parameter as defined in section 9 of [RFC3840]. The feature parameter is encoded as an "other-tags" feature tag in accordance with the definitions of section 9 of [RFC3840].

The feature tag name shall be set to "g.gsma.rcs.isbot". The media feature tag shall have no value.

Security considerations for this media feature tag are discussed in section 11.1 of [RFC3840].

The indication of the Chatbot role is represented as follows:

+g.gsma.rcs.isbot

3.6.2.4 Chatbot capability

A client shall consider a contact being a Chatbot if the SIP URI conforms to the definition of a Chatbot service identifier as per section 2.5.4.1, and is discoverable via the procedures in section 3.6.4.1.

3.6.3 Discovery and specific management of Chatbots

3.6.3.1 Client to Service Provider Chatbot Directory interface

A Service Provider Chatbot Directory is a logical entity in the RCS Service Provider network where a list of available Chatbots, aggregated from the interconnected Chatbot Platforms, is maintained based on which Chatbots the Service Provider wishes to make discoverable by its RCS users. This list can be retrieved, parsed and displayed with their metadata.

If the RCS service provider wishes to provide client access to the Service Provider Chatbot Directory, they shall configure a Chatbot Directory URL from where the client can retrieve a list of Chatbots through the CHATBOT DIRECTORY client configuration parameter defined in section A.1.3. If the client configuration parameter is not provided, the client shall assume that the Service Provider Chatbot Directory functionality is disabled. If the URL derived from the CHATBOT DIRECTORY client configuration parameter cannot be resolved through DNS, the client shall

- assume that the Service Provider Chatbot Directory functionality from the service provider is not available and
- verify availability of the configured URL after every restart of the device or every restart of the RCS client.

If the Chatbot Directory URL is available and accessible, the client shall

- establish a TLS connection to the Chatbot Directory using the Chatbot Directory URL (as indicated in section A.1.3 the protocol scheme for the URL shall be https) and
- verify that a certificate is provided by the Chatbot Directory and that this certificate is valid and
- If the certificate is valid,
    - send a HTTP GET request to retrieve the referred resource including the query parameters defined in Table 54 when Chatbot discovery is invoked through UX and
    - If the provisioned Chatbot Directory URL includes query parameters already, include the parameters defined in Table 54 in the existing query component and

- Otherwise, add a query component to the URL and include the parameters defined in Table 54 in that query component and

- If no certificate is provided or the certificate is invalid (e.g. it has expired or there is no chain of trust to a known Certification Authority),
  - Not continue with the query to the Service Provider Chatbot Directory and
  - assume that the Service Provider Chatbot Directory functionality from the service provider is not currently available and
  - Verify availability of the configured URL after every restart of the device or every restart of the RCS client.

NOTE: Given that for security reasons this client handling does not ensure a graceful fall-back to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

Query Parameter	Description	Mandatory	Format
q	Query terms that can specify words or phrases to filter the query results that the Service Provider Chatbot Directory returns. The value must be URL-escaped.	N	String
start	The start parameter indicates the first matching result that should be included in the query results. It uses a zero-based index, meaning the first result is 0; the second result is 1 and so forth. The start parameter works in conjunction with the num parameter to determine which query results to return.	N	Positive integer
num	The num parameter identifies the number of query objects to return. If not provided, the num value is determined by the server.	N	Positive integer
pl	It indicates the preferred language(s) for Chatbot search. Its default value is the selected device language setting. The default value may be edited by the user or additional values may be added based on user preferences. Its format shall follow the [ISO 639-1] definitions. It is provided based on user setting to enrich the search. Example: If the selected device setting is English and the user additional preferred languages for search is Spanish and German then the “pl” parameter is as follows in the request: pl=en&pl=es&pl=de	N	String Multi-valued parameter
lat	It indicates the latitude of the device/client based on signed degrees’ format. It is provided based on user setting to enrich the search.	N	Double

Query Parameter	Description	Mandatory	Format
lon	It indicates the longitude of the device/client based on signed degrees' format. It is provided based on user setting to enrich the search.	N	Double
ho	It indicates the home operator of the device and this shall be represented as <MCCMNC> combination. Whereby MCC and MNC shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning.	Y	String
i	It indicates the MSISDN of the user. The client shall provide the MSISDN value with the plus sign encoded as per [RFC3986].It is provided based on user setting to enrich the search and the value of the IDENTITY IN ENRICHED SEARCH parameter defined in section A.1.3 and A.2.4.	N	E.164 international format
chatbot_version	It contains the Chatbot application version the client supports defined in 3.6.2.2. If not provided, then it means only version 1 is supported. The version parameter shall be composed as follows: version = version_number * ["_" version_number] version_number = 1*DIGIT Example: chatbot_version=1_2_3	N	String
client_vendor	It contains the query client_vendor HTTP GET parameter included in the configuration requests (see 2.3.2.2).	Y	String
client_version	It contains the query client_version HTTP GET parameter included in the configuration requests (see 2.3.2.2).	Y	String

Table 54: Query parameters for the Client to Directory query

How these query parameters included in the URL are handled and the query results in the response are prepared is up to the Service Provider Chatbot Directory function implementation and is out of the scope of this specification. If query parameters are provided that are not known by the Service Provider Chatbot Directory (e.g. because the querying client complies with a future version of this specification), it is up to the Service Provider Chatbot Directory function implementation how to handle them.

The Service Provider Chatbot Directory shall return the query result in the JSON format with a HTTP 200 OK response. The schema in Table 55 defines all JSON payloads exchanged between the Service Provider Chatbot Directory and the query client.

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
    "properties": {
      "bots": {
        "type": "array",
```

```
      "items": {
        "type": "object",
        "properties": {
          "id": {
            "type": "string",
            "format": "uri"
          },
          "name": {
              "type": "string"
          },
          "icon": {
              "type": "string",
              "format": "uri"
          },
          "verified": {
              "type": "boolean"
          }
        },
        "required": ["id","name"]
      }
    },
    "itemsReturned": {
        "type": "integer"
    },
    "startIndex": {
        "type": "integer"
    },
    "totalItems": {
        "type": "integer"
    }
  },
  "required": ["bots"]
}
```

Table 55: JSON Schema for Service Provider Chatbot Directory payloads

A detailed description of each JSON object is specified in Table 56. The client shall gracefully ignore any JSON object that it does not recognize.

JSON	Description
id	Chatbot service identifier (see NOTE below)
name	Chatbot name
icon	Link to the Chatbot icon image. If not provided, the client shall assume that the icon is not available.
verified	Indicates whether the Chatbot is to be presented to the user as a Chatbot whose identity has been verified. If absent, the Chatbot shall not be presented as verified.
itemsReturned	The number of query objects returned
startIndex	The index of the first query result in the current set of query results
totalItems	The number of query objects available

Table 56: Description of the Service Provider Chatbot Directory query result

NOTE: An id value can occur only once in the query results. If there are more than one entry available with the same Chatbot service identifier, it is up to the Service Provider Chatbot Directory Function to select which one to return to the client.

The client implementation shall display the Chatbots in the same order that the Chatbot entries are ordered in this JSON object that is provided in response to the HTTP GET request. Caching of this object may be implemented on the client for efficiency e.g. when the same query is triggered by the UX within some seconds. It is left to the client implementation to set the validity time of this cached response and to apply further optimisations.

3.6.3.2 Directory to Directory interface

To allow the Service Provider Chatbot Directory function to aggregate the list of hosted Chatbots of the Chatbot Platforms that the RCS Service provider is interconnected with, the Service Provider Chatbot Directory function can send a HTTP GET request to retrieve the Chatbot Information details towards the Chatbot Directory of the interconnected Chatbot Platforms. The Chatbot Directory shall respond based on the interconnection agreement between the RCS service provider and the Chatbot Platform. The frequency with which this HTTP GET request is sent is based on the RCS Service Provider policies and it may be subject to the interconnection agreement between the RCS Service Provider and the Chatbot Platform. That agreement is outside of the scope of this specification. The Service Provider Chatbot Directory shall send a HTTP GET request to retrieve the searchable Chatbot Information for the different Chatbots with the query parameters defined in Table 57.

Query Parameter	Description	Mandatory	Format
qo	It indicates the querying operator and it shall be represented as <MCCMNC> combination. Whereby MCC and MNC shall be replaced by the respective values of the querying operator in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning. It is provided based on Service Provider Chatbot function policies.	N	String Multi-valued parameter
chatbot_version	It indicates the Chatbot application version of the clients that the Service Provider Chatbot Directory server supports. It is defined in section 3.6.2.2. If not provided, then it means only version 1 client are served. The version parameter shall be composed as follows: version = version_number * ["_" version_number] version_number = 1*DIGIT Example: chatbot_version=1_2_3	N	String

Table 57: Query parameters for Directory to Directory query

The Chatbot Directory of the Chatbot Platform shall return the query result in the JSON format with a HTTP 200 OK response. The schema in Table 58 defines the JSON payload exchanged between the Chatbot Directory of the Chatbot Platform and the Service Provider Chatbot Directory. It is possible to get more than one entry with the same Chatbot service identifier e.g. different values for some properties (name, email, SMS, tel, website, description) per country that correspond to the same Chatbot service identifier based on Chatbot or Chatbot Platform logic.

The description of each JSON object is specified in Table 59.

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "botsprofile": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "id": {
            "type": "string",
            "format": "uri"
          },
          "name": {
            "type": "string"
          },
          "provider": {
            "type": "string"
          },
          "email": {
            "type": "string"
          },
          "sms": {
            "type": "string"
          },
          "tel": {
            "type": "string"
          },
          "website": {
            "type": "string"
          },
          "icon": {
            "type": "string",
            "format": "uri"
          },
          "iconfingerprint": {
            "type": "string"
          },
          "description": {
            "type": "string"
          },
          "businessLocation": {
            "type": "string"
          },
          "category": {
            "type": "array",
```

```
            "items": {
              "type": "string"
            }
          },
          "co": {
            "type": "array",
            "items": {
              "type": "string"
            }
          },
          "cl": {
            "type": "array",
            "items": {
              "type": "string"
            }
          },
          "version": {
            "type": "string"
          },
          "verification-signatures": {
            "description":"RFC7515 (JWS) general JWS JSON serialisation",
            "type": "object"
          }
        },
        "required": [
          "id",
          "name",
          "version"
        ]
      }
    },
    "itemsReturned": {
      "type": "integer"
    },
    "startIndex": {
      "type": "integer"
    },
    "totalItems": {
      "type": "integer"
    }
  }
  "required": [
    "botsprofile"
  ]
}
```


Table 58: JSON Schema for Chatbot Directory payloads

JSON	Description
id	Chatbot Service ID
name	Chatbot Service Name
provider	Chatbot Provider Name
email	Chatbot email address
sms	Chatbot SMS address. It can be a short or long code.

JSON	Description
tel	Chatbot Call-back phone number
website	Chatbot website
icon	Chatbot Service Icon. Link to the Chatbot icon image
iconfingerprint	Fingerprint of the file providing the Chatbot icon image
description	Chatbot Service Description
businessLocation	Chatbot business location
category	Chatbot Category
co	Chatbot supported country or network where: • Country representation is using Alpha-2 country code as defined in ISO3166 used when the Chatbot is supported in all the operators of that country • Network representation is using <MCCMNC> combination. Whereby MCC and MNC shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning.
cl	Chatbot supported language encoded based on [ISO 639-1] definitions.
version	Chatbot version supported by the Chatbot. The version parameter shall be composed as follows: version = version_number * ["_" version_number] version_number = 1*DIGIT Example: chatbot_version=1_2_3
verification-signatures	Provides a JSON Web Signature (JWS) in general JWS JSON serialisation format according to [RFC7515] with non-detached payload that can be used to verify the Chatbot Directory entry
itemsReturned	The number of query objects returned
startIndex	The index of the first query result in the current set of query results
totalItems	The number of query objects available

Table 59: Description of the Chatbot Directory query result

3.6.3.2.1 Verification of Chatbot Directory entries

The Service Provider should verify that the entries in the Chatbot Directory query result comply with the information that has been verified by a Chatbot Verification Authority that is recognised by the MNO to perform verifications of Chatbots as follows:

1. If the entry for the Chatbot does not include a *verification-signatures* property, the Chatbot shall be considered as an unverified Chatbot and no further processing to verify whether the Chatbot has been verified by a recognised Chatbot Verification Authority shall be done.
2. If the *verification-signatures* property of the entry for the Chatbot does not correspond to a JWS in the General JWS JSON Serialization defined in section 7.2.1 of [RFC7515], the Chatbot shall be considered as an unverified Chatbot and no further processing to verify whether the Chatbot has been verified by a recognised Chatbot Verification Authority shall be done.

3. If the *verification-signatures* property of the entry for the Chatbot does not include a "*payload*" property (i.e. it is using a detached payload according to Annex F of [RFC7515]), the Chatbot shall be considered as an unverified Chatbot and no further processing to verify whether the Chatbot has been verified by a recognised Chatbot Verification Authority shall be done.
4. If (after Base64url-decoding) the "*payload*" property of the "*verification-signatures*" property of the entry for the Chatbot does not correspond to a JSON object with at least the properties "*id*" and "*name*", the Chatbot shall be considered as an unverified Chatbot and no further processing to verify whether the Chatbot has been verified by a recognised Chatbot Verification Authority shall be done.
5. Otherwise, the Service Provider shall verify the different signature objects in the "*signatures*" property as specified in section 5.2 of [RFC7515] taking into account the following restrictions:

    a) If the "*alg*" header parameter provided in the JWS Protected Header is not set to one of the algorithms defined as "Recommended" or "Recommended+" in section 3.1 of [RFC7518], the digital signature shall not be considered.
    b) If the signature object does not provide in the JWS Protected Header a "*crit*" Header parameter defined in section 4.1.11 of [RFC7515] including the value "*botvfexpires*" among the values, the digital signature shall not be considered.
    c) If the signature object does not contain a "*botvfexpires*" property in the JWS Protected Header set to a date in the future, the digital signature shall not be considered.
    d) If the signature object does contain the optional “*botvfdate*” property in the JWS Protected Header and it is malformed or when it has a value older than the local policy for freshness permits, or the date is set into the future, the digital signature shall not be considered.

NOTE: It is desirable to implement a consistent policy for the freshness rule across all Service Providers in a given market.

    e) If the *botvfdate* property is older than the last known change of the information used to verify the Chatbot, then the digital signature shall not be considered.

NOTE: Information on the timing of changes to the information might be available when the party verifying the signature is also the party operating the Chatbot Platform.

    f) If the signature object does not provide a "*kid*" property in the unprotected header as reference to the key, the digital signature shall not be considered.
    g) If the value of the "*kid*" property in the unprotected header of the signature object does not correspond to a key of a recognised Verification Authority, the digital signature shall not be considered.

NOTE: The process through which the Service Provider is made aware of the "kid" value of the Verification Authorities that it recognizes is out of scope of this specification. One such approach is to sign the public key with an X.509 certificate and apply the best practice of using certificate path validation as described in [RFC5280], whereby any subsequent Chatbot verification

signature using such a certificate can be checked to ensure the certificate was signed by a root certificate that was provided by one of the recognised Verification Authorities.

6. If at least one of the considered digital signatures was successfully verified, the Service Provider shall compare the data in the JSON object provided in the "*payload*" property to the data in the corresponding entry for the Chatbot in the Chatbot Directory query result as follows:

   a) If the value of the "*id*" property in the JSON object provided in the "*payload*" property does not match the value of the "*id*" property of the entry in the Chatbot Directory query result, the verification shall be considered to have failed
   b) If the value of the "*name*" property in the "*payload*" property does not match the value of the "*name*" property of the entry in the Chatbot Directory query result, the verification shall be considered to have failed
   c) If the entry in the Chatbot Directory query result had an "*iconfingerprint*" property and the JSON object provided in the "*payload*" property does not have an "*iconfingerprint*" property in the entry in the Chatbot Directory query result, the verification shall be considered to have failed.
   d) If the JSON object provided in the "*payload*" property provides an "*iconfingerprint*" property with a value and the entry in the Chatbot Directory query result does not provide an "*iconfingerprint*" property, the verification shall be considered to have failed.
   e) If the entry in the Chatbot Directory query result has an "*iconfingerprint*" property and the JSON object provided in the "*payload*" property has an "*iconfingerprint*" property, but their values do not match, the verification shall be considered to have failed.
   f) Otherwise, the verification of the "*payload*" property shall be considered to have succeeded

7. If in step 6 the verification of the "*payload*" property was successful, the verification of the entry for the Chatbot in the Chatbot Directory query result shall be considered to be successful if either of the following conditions is fulfilled:

   - No "*icon*" and "*iconfingerprint*" properties were included for the Chatbot in the Chatbot Directory query result, or
   - Both "*icon*" and "*iconfingerprint*" properties were included and the value of the "*iconfingerprint*" property matches a SHA-256 hash of the file referred to by the URL in the "*icon*" property in the Chatbot Directory result whereby the result of the hash is represented as a hexadecimal string in lowercase without a "0x" prefix.

   Otherwise, if those conditions are not fulfilled, the verification of the entry for the Chatbot in the Chatbot Directory query result shall be considered to have failed.

3.6.3.3 Service Provider Client Configuration for Chatbots requiring specific management

A Service Provider may want to indicate that some Chatbots shall require specific management on the client side (e.g. blacklisted as Spam senders, providing critical communication).

If the Service Provider wishes to have specific management for Chatbots, the Service Provider shall provide the clients with a URL from which they can retrieve a list of Chatbots requiring specific management. This URL shall be configured through the SPECIFIC CHATBOTS LIST client configuration parameter defined in section A.1.3. If the URL is not configured, the client shall assume that there are no Chatbots that require specific management.

If the configured URL does not resolve through DNS, the client shall

- assume that there are no Chatbots that require specific management and
- verify the availability of the configured URL after every restart of the device or client.

If the URL is available, the client shall

- If the URL's protocol scheme is https,

NOTE: Even if it is not required for the URL's protocol scheme to be https and the information exchanged is not confidential, it is highly recommended that the URL's protocol scheme be https to avoid misconfiguration of the client when the connection to the server is hijacked (e.g. through intercepting a DNS request).

  - establish a TLS connection using the URL from the SPECIFIC CHATBOTS LIST client configuration parameter and
  - verify whether a certificate is provided from the service provider side and that this the certificate received is valid (that it has not expired, that there is chain of trust to a known Certification Authority, that it corresponds to the domain part of the URL configured through the SPECIFIC CHATBOTS LIST client configuration parameter, etc.)
  - If the certificate is invalid,
    - Continue using the existing cached lists of Chatbots requiring specific management if such lists are cached and
    - assume that there are no Chatbots that require specific management if no cached lists exist and
    - verify the availability of the configured URL after every restart of the device or client.

NOTE: Given that for security reasons this client handling does not ensure a graceful fall-back to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

  - Otherwise (i.e. if the certificate is valid), continue with the rest of the steps

- send a HTTP GET request to retrieve the file using the URL from the SPECIFIC CHATBOTS LIST client configuration parameter.

If receiving a 200 OK response to the request, the client shall

8. Store the Etag and Cache-Control directive values associated with the resource according to the procedures of [RFC7232];
9. Store the lists of URIs of Chatbots requiring specific management which will be formatted as described in this section;
10. Whenever the validity of the lists of URIs of Chatbots requiring specific management expires (based on the Cache-Control directive received in the last retrieval stored in steps 8, 10.b)i and 10.c)i, send an HTTP GET request to the configured URL including the stored Etag in a if-none-match header according to the procedures of [RFC7232].

    a) If the configured URL no longer resolves through DNS, the client shall

        i. verify availability of the configured URL after every restart of the device or client and
        ii. delete the lists of URIs of Chatbots requiring specific management if the configured URL has failed to resolve through DNS for 10 consecutive attempts.

    b) If receiving a HTTP 200 OK response, the client shall

        i. Store the new Etag and Cache-Control directive values according to the procedures of [RFC7232];
        ii. Store the provided lists of URIs and use them according to the procedures defined for each list.

    c) If receiving a HTTP 304 response, the client shall

        i. Store the new Etag and Cache-Control directive values according to the procedures of [RFC7232];
        ii. Continue using the existing lists of URIs and use them according to the procedures defined for each list.

    d) If receiving another response, the client shall

        i. Repeat the procedure in step 10 at the next client or device restart
        ii. Continue using the existing lists of URIs and use them according to the procedures defined for each list.

For the Expires header, similarly to section 3.6.4.1.2, the Service Provider shall ensure that the entity that maintains the lists of Chatbots requiring specific management returns responses with Expires header value set to a date in the past. This will ensure that legacy proxies do not attempt to cache the content.

When data is switched off (see section 2.8.1.5), this interface shall remain available if device management is configured as cellular switched data off exempted service via the

client configuration parameters Device Management over PS data off exemption or Device Management over PS data off roaming exemption defined in section A.1.14.

If provisioning is not available, the client shall perform step 10 of the above procedure at the earliest opportunity (e.g. when data is switched on again or when connecting over Wi-Fi).

The lists of URIs of Chatbots requiring specific management shall be provided as a document of the Mime-type text/plain providing sub lists of URI entries. Each sub list shall begin with a specific marker:

```
LIST:<name of the list><CRLF>
<CRLF>
```

where <name of the list> is a distinct identifier, identifying the sub list. This identifier is defined in the section describing the specific management that is required.

Each sub list of URIs is constituted of URI entries separated by new lines (i.e. CRLF, resulting in one URI per line).

- The URI entries themselves shall be any of the following:
  - A SIP URI or
    In this case the entry will start with sip: or sips:
  - a tel URI or
    In this case the entry will start with tel:
  - a regular expression matching either of those URI types.
    In this case the entry will start with an exclamation mark character (i.e. '!'). The subsequent string shall be interpreted by the client to match a URI string using Portable Operating System Interface (POSIX) extended regular expression (see [POSIX]) e.g. !^(sip:.*-publisher\@botplatform\.example\.com)
- Entries that do not conform to these formats shall be ignored.

A sub list may be empty which shall be handled in the same way as when the sub list was not provided at all.

A client shall ignore any sub list for which the identifier (i.e. <name of the list>) is unknown.

3.6.3.4 Deep Linking

Deep links are URLs that can take RCS users directly to the Chatbots conversations with specific contents. Deep links can be embedded in a webpage, a mobile application, or a QR code. The usage of deep links is out of the scope of this document.

Two types of deep link are supported: rcs.me (https) and "sms" URI scheme with RCS specific parameters. Both schemas support Chatbot Platform provided contextual welcome messages and rcs.me enablesfallback to a webpage.

3.6.3.4.1 SMS Deep Links

The "sms" URI scheme defined in [RFC5724] shall be extended to construct the deep links to initiate a Chatbot conversation from an RCS user. In addition to the "body" query parameter defined in [RFC5724] an optional "service_id" parameter, which is the Chatbot

service identifier defined in section 2.5.4.1 without the “sip:” URI scheme, may be added as part of the deep link.

If the Chatbot does not have an SMS number, the Chatbot service identifier defined in section 2.5.4.1 shall be used without the "sip:" URI scheme as value for the “sms-recipient” part of the URI that is defined in [RFC5724]. However, in such a case, a non-RCS client may not be able to handle this deep link.

The following syntax extends the formal definition of the "sms" URI scheme defined in [RFC5724], with the RCS specific parameters defined in Table 60.

```
sms-field-name =/ "service_id" / "wid" / "ctx"
sms-recipient =/ service-id ; this increment is not compatible
                            ; with [RFC5724]
service-id = escaped-value
wid = escaped-value
ctx = escaped-value
```

NOTE: The “suggestions” parameter that was present in version 9 through 14 of this document, is no longer supported and shall henceforth be gracefully ignored.

- Based on the extensions defined above, the following additions apply to the processing of the "sms" URI as defined in section 2.3 of [RFC5724]:In step 1, if the client is registered for RCS services, then
    - if a “service_id” parameter is included in the "sms" URI, the value of the “service_id” parameter shall be extracted and converted to a SIP URI. The definitions for the handling of Vanity-Service-IDs as defined in section 2.5.4.1.3 shall be taken into account. The "sms-hier-part" of the "sms" URI as defined in [RFC5724] shall be ignored in this case.
    - Otherwise, if the "service_id" parameter is not included in the "sms" URI" and the first "sms-recipient" contains a Chatbot service identifier as per "sms" URI extension above, then the client shall extract the value and convert it to a SIP URI. The definitions for the handling of Vanity-Service-IDs as defined in section 2.5.4.1.3 shall be taken into account.
    - Otherwise, the client shall extract the phone number of the first "sms-recipient" as defined in [RFC5724].
    - Trigger handling for Chatbot Capability Discovery applies for the extracted contact address based on the procedures defined in section 3.6.4.1 and the corresponding service description.
    - In processing the Chatbot information response, the definitions for discovery of an alternative Chatbot service identifier as defined in section 3.6.4.1.3 shall be taken into account.

    Otherwise, if the client is not registered for RCS services, then the "sms" URI extension parameters defined above and a "sms-recipient" containing a Chatbot service identifier shall be ignored. The processing of [RFC5724] shall apply.

- In step 4, the 'sms-hier-part' of the 'sms' URI as defined in [RFC5724] shall be temporarily stored by the client. It shall only be ignored if the subsequent Chatbot Information Retrieval in Section 3.6.4.1 for the 'service_id' successfully confirms a valid Chatbot address. If the Chatbot address is invalid, the client shall revert to processing this 'sms-hier-part' as the message recipient.
- In step 5, if the processing of the "sms" URI was successfully completed based on a valid SIP URI extracted from the "service_id" parameter or the "sms-recipient" (confirmed via Chatbot Information Retrieval), then the processing of the comma separated list in the "sms-hier-part" of the "sms" URI is omitted. However, if the SIP URI validation failed and the client fell back to using the 'sms-hier-part', the processing of the comma separated list shall proceed as defined in [RFC5724].

Some non-normative examples are given in the following:

- if the Chatbot is assigned an SMS number (+15012011657) and
- the Chatbot service identifier is “bot@botplatform.example.com”, and
- the Chatbot wants the user to send a message of “tell me about checking accounts” to start the conversation

  then the deep link URL can be:

  `sms:+15012011657?service_id=bot%40botplatform.example.com&body=tell%20me%20about%20checking%20accounts.`
  When this deep link is activated by a RCS client, the message with the Chatbot, identified by the service ID, will be prepared for the user to send via RCS; when this deep link is activated by a non-RCS client, the message with the SMS-message-capable phone number will be prepared for the user to send via SMS.

- if the Chatbot does not have the SMS-message-capable phone number, then the deep link URL can be:

  `sms:bot%40botplatform.example.com?body=tell%20me%20about%20checking%20accounts.`
- If the Chatbot has either a SIP URI or SMS service identifier, then the deep link URL may include contextual "wid" and "ctx"parameters to enable a Chatbot Platform-originated Chatbot Welcome Message via procedures in Section 3.6.4.1. In this case the deep link can be:

  `sms:+15551234567?service_id=bot@example.com&wid=campaign123&ctx=store99`

3.6.3.4.2 rcs.me Deep Links

All rcs.me deep links shall use the format defined in Table 60.

`https://rcs.me?<set_of_query_parameters>`

Table 60: Composition of a rcs.me URL

To support rcs.me deep links, Clients shall accept and process incoming URLs with the https scheme, adhering to operating system-specific procedures for the deep link RCS client

association, such as Android Verified App Links [Android-AppLinks] or Apple Universal Links [Apple-UnivLinks].

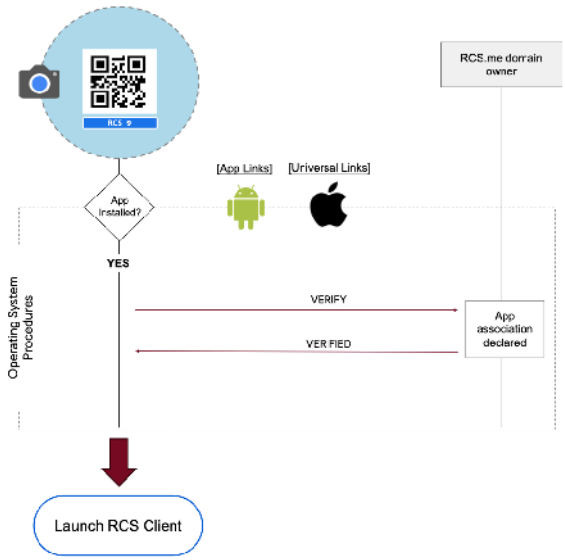


Figure 17: rcs.me link validation

Table 61 defines what rcs.me parameters are supported based on [RFC3986] definitions:

Query Parameter	Description	Mandatory	Format
id	Chatbot service identifier SIP URI. It shall be encoded respecting the definitions of [RFC3986].	Y	String
wid	Chatbot Welcome Message identifier to be activated as a result of opening the conversation (see 3.6.4.1). It shall be encoded respecting the definitions of [RFC3986].	N	String
ctx	Context to be shared with the Chatbot Platform as a result of opening the conversation (see 3.6.4.1). It shall be encoded respecting the definitions of [RFC3986].	N	String

Table 61: Query parameters for rcs.me

A non-normative example follows:

- if the Chatbot service identifier is “bot@botplatform.example.com”, and
- the Chatbot has registered the Chatbot Welcome Message identifier “1234” with the Chatbot Platform to start the conversation
- the Chatbot has assigned the context “source=website.com;campaign=chk-act-cmpgn-14” for this deeplink

then the deep link URL can be:

`https://rcs.me?id=bot%40botplatform.example.com&wid=1234&ctx=source%3Dwebsite.com%3Bcampaign%3Dchk-act-cmpgn-14.`

If a device is not RCS capable, or if there is no verified association between the RCS client and rcs.me domain, Operating System procedures will direct the user to open a browser, where the rcs.me domain may provide further redirection to a fallback URL defined by the Chatbot.

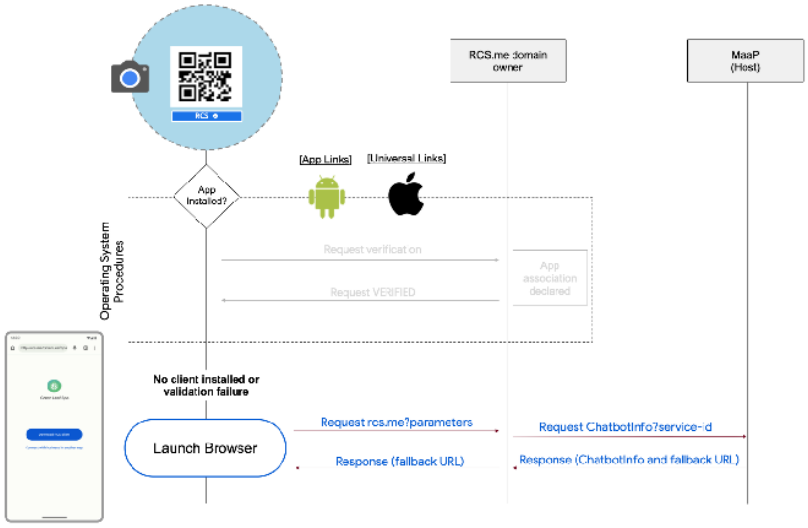


Figure 18: rcs.me validation failure

3.6.3.4.2.1 rcs.me Domain Owner Procedures

The rcs.me domain owner is responsible for maintaining the rcs.me domain and facilitating the initial connection between the RCS client and the appropriate Chatbot Platform.

The rcs.me domain owner shall configure DNS to ensure proper resolution of the rcs.me domain. Clients resolving this domain will follow the standard DNS resolution process.

The rcs.me domain shall have a valid TLS certificate issued by a trusted Certificate Authority. RCS clients shall validate this certificate during initial connection setup.

NOTE: Given that for security reasons client handling does not ensure a graceful fall-back to a connection without TLS if the certificate is invalid, rcs.me should ensure that the certificate is replaced before expiry.

The rcs.me web server shall host the list of valid Chatbot domain names at the path `.well-known/maap-domains`.

The list of domain names shall be provided as a document of the Mime-type text/plain.

The list is constituted of domain names separated by new lines (i.e. CRLF, resulting in one domain name per line). The URI entries themselves shall be any of the following:

- a domain name (e.g. botplatform.example.com)

- a regular expression.

  In this case the entry will start with an exclamation mark character (i.e. ‘!’). The subsequent string shall be interpreted by the client to match a domain name string using Portable Operating System Interface (POSIX) extended regular expression (see [POSIX]) e.g., !^(.*\.botplatform\.example\.com).

Entries that do not conform to these formats shall be ignored.

For client requests (section 3.6.3.4.2.3.2) and similarly to section 3.6.4.1.2, rcs.me shall return responses with an Expires header value set to a date in the past. This will ensure that legacy proxies do not attempt to cache the content.

For non rcs.me capable devices as per section 3.6.3.4.2 (Figure 18), the rcs.me webserver shall:

1. Check the domain of the “id” parameter against the `.well-known/maap-domains`;
2. If there is no matching domain name, provide an error webpage
3. If a match is found, trigger the Chatbot Information retrieval procedures as defined in section 3.6.4.1;

NOTE: It is expected that the rcs.me web server will not have access to home operator information (e.g. MCC or MNC) to include in the Chatbot Information retrieval request.

4. If valid and verified Chatbot Information can be retrieved including a fallback experience, redirect the user to the fallback URL including the deep link’s ”id”, ”wid”, and “ctx” parameters as query parameters.
5. Otherwise, provide an error webpage.

3.6.3.4.2.2 Chatbot Platform Procedures

In the context of rcs.me deep links, Chatbot Platforms identified by the deep link "id" parameter is responsible for:

1. Handling Chatbot information requests without the home operator parameter (Table 63) for non rcs.me capable devices and validation failures
2. Allowing Chatbots to define an alternative Chatbot service identifier and returning this identifier in the Chatbot Information when it determines that it is not the Chatbot Platform serving the user.

    a) When an alternative Chatbot identifier is provided, no other Chatbot Information shall be provided in the response.
    b) If no alternate Chatbot identifier is available, the fallback URL shall be provided in the response.
    c) An alternative Chatbot identifier or fallback URL shall only be returned when the Chatbot has been verified.
    d) If neither an alternate Chatbot identifier or fallback URL is available, an error shall be returned.

Additionally, all Chatbot Platforms are responsible for:

1. allowing Chatbots to define a contextual welcome messaging experience associated with the “id” of Chatbot service identifier, “wid” (Welcome ID) and “ctx” (Context) in a deep link URL
2. Providing Chatbot Welcome Messages as part of the Chatbot Information data in Section 3.6.4.1.6.

3.6.3.4.2.3 rcs.me Client Procedures

On successful launch of a RCS client supporting rcs.me deep links, the RCS client shall follow these steps to process the deep link parameters:

1. Establish the validity of the deep link “id” parameter (see 3.6.3.4.2.3.1) and trust in the associated Chatbot Information response (e.g. whether it is supported by an RCS Service Provider or created by a bad actor).
2. Once validated, the RCS client can safely process the validated Chatbot Information response.
3. If the Chatbot is not discoverable, e.g. unlaunched on the user’s Service Provider or unavailable, the RCS client shall invoke a fallback URL if defined by the validated Chatbot Information.

3.6.3.4.2.3.1 rcs.me “id” parameter validation procedures

This section describes how the RCS client validates the “id” parameter in an rcs.me URI to ensure it's a valid Chatbot service identifier. This process involves validation steps and a fallback procedure to handle invalid or unreachable identifiers.

1. If the “id” parameter is included in the URI, the value of the parameter shall be extracted and converted to a SIP URI. The definitions for the handling of Vanity-Service-IDs as defined in section 2.5.4.1.3 shall be taken into account.
2. Using the resulting contact address, the RCS client shall first check that the domain of the URI in the “id” parameter matches the value of the BOTINFO FQDN ROOT client configuration parameter provisioned by the service provider. If the domains match the client shall establish Chatbot capabilities using the procedures defined in section 3.6.4.1.
3. If the domains do not match, the RCS client shall validate the domain obtained from the “id” parameter using procedures defined in section 3.6.3.4.2.3.1. If the domain is valid, the client shall process the Deep Link using procedures defined in Section 3.6.4.1.
4. If the domain of the URI in the “id” parameter is not listed as a Chatbot well-known domain, the link shall be considered invalid.

Figure 14 depicts an example of successful fallback via an alternative Chatbot service identifier:

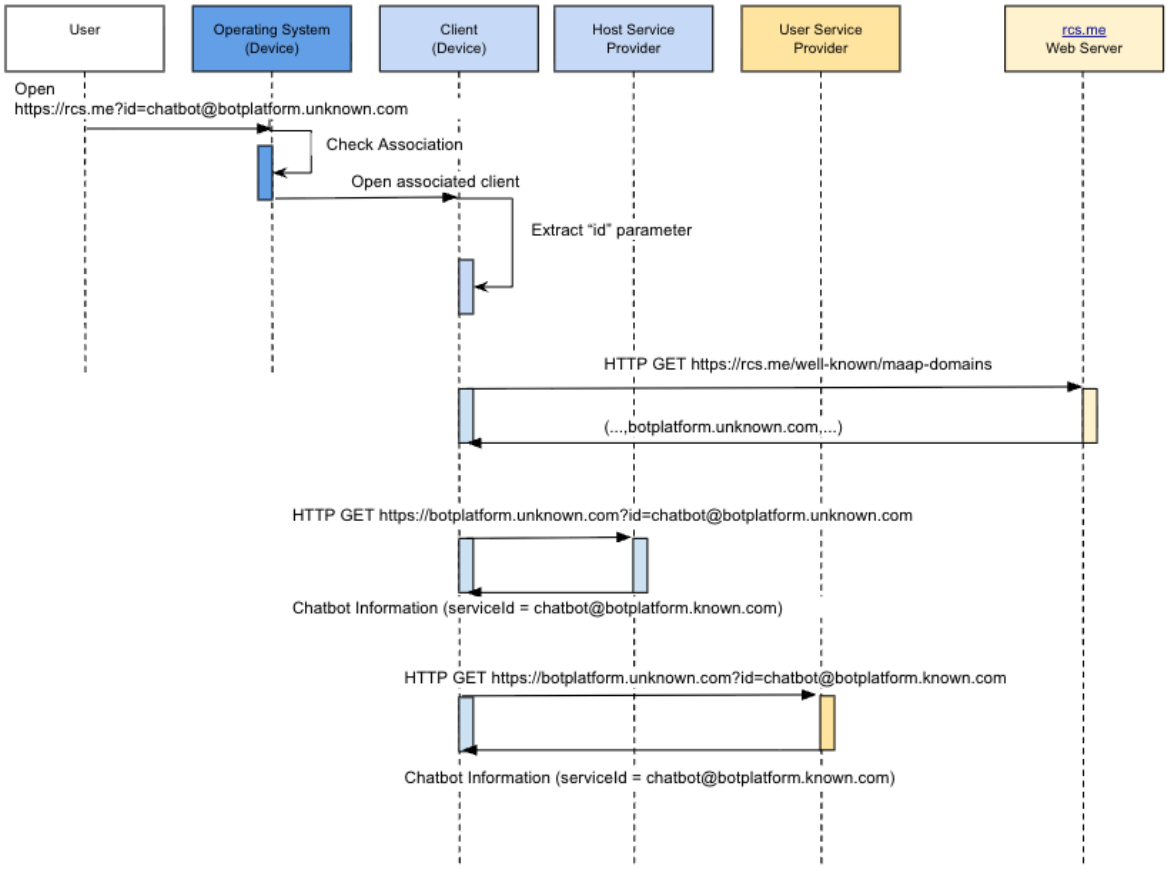


Figure 19: rcs.me fallback via alternative Chatbot service identifier

3.6.3.4.2.3.2 rcs.me Chatbot domain name verification

The rcs.me list of well-known Chatbot domain names is used to establish the validity of rcs.me Deep Link service id domains that do not match the BOTINFO FQDN. Before the client requests the Chatbot Information, it shall first validate the link against the rcs.me well-known domains list..

In this situation, the client shall determine whether it has a valid cached list of domains names based on the Cache-Control directive received in the last retrieval.

If there is a valid cached list, the client shall determine whether the domain is verified by searching the list for an entry matching the Chatbot service identifier's domain part

If there is no valid cached list, the client shall:

- Establish a TLS connection with rcs.me and
- verify whether a certificate is provided and that this certificate is valid (that it has not expired, that there is chain of trust to a known Certification Authority, that it corresponds to the domain part of rcs.me, etc.)
    - If the certificate is invalid, it shall consider the Chatbot domain verification failed
- Otherwise (i.e. if the certificate is valid), continue with the rest of the steps

- send a HTTP GET request to retrieve `.well-known/maap-domains` including the stored ETag (if any) in a `if-none-match` header according to the procedures of [RFC7232].

  - If receiving a 200 OK response to the request, the client shall

    - store the Etag and Cache-Control directive values associated with the resource according to the procedures of [RFC7232];
    - store the list of domains and
    - determine whether the domain is verified by searching the list for an entry matching the Chatbot service identifier's domain part

  - If receiving a HTTP 304 response, the client shall

    - store the new Etag and Cache-Control directive values according to the procedures of [RFC7232];
    - determine whether the domain is verified by searching the existing list for an entry matching the Chatbot service identifier's domain part

  - If receiving another response, the client shall

    - delete the stored list of Chatbot domains and
    - consider the Chatbot domain verification failed

3.6.3.4.2.4 Void

3.6.4 Chatbot Information

This section defines the procedures for the retrieval of Chatbot Information accessible from within the messaging implementation.

3.6.4.1 Chatbot Information Retrieval

3.6.4.1.1 Chatbot Information request

Chatbot Information is maintained in and provided by the Chatbot Information Function. To retrieve Chatbot Information of a Chatbot, the client shall send a HTTP GET request to the Chatbot Information Function using the botinfo URL formatted as defined in Table 62.

`https://<root_domain>/bot?<set_of_query_parameters>`

Table 62: Composition of the botinfo URL

The client shall use the URL scheme “https”, i.e. it shall establish a secured connection to the botinfo URL host.

The configuration parameter BOTINFO FQDN ROOT as defined in section A.1.3 provides the value of “root_domain” of the botinfo URL that the RCS client shall use.

Otherwise, if the configuration parameter is not present, the value of “root_domain” shall take the value of the domain part of the Chatbot service identifier SIP URI defined in section 2.5.4.1.

The client shall use the HTTPS standard port. During the establishment of the TLS connection, the client shall verify that the certificate provided by the botinfo URL host is valid (e.g. it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the "root_domain", etc.). If the certificate is invalid or no certificate is provided, the client shall not continue with the request and handle the situation as described for a HTTP 5XX response in section 3.6.4.1.2 or section 3.6.4.1.5 for an initial request or a refresh of Chatbot Information respectively.

NOTE: Given that for security reasons this client handling does not ensure a graceful fallback to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

The client shall include in the botinfo URL a path with the value “bot”

The client shall include in the botinfo URL the "`set_of_query_parameters`" using the parameters defined in Table 63 using the *application/x-www-form-urlencoded* encoding defined in [HTML-4.0].

Query Parameter	Description	Mandatory	Format
id	Chatbot SIP or tel URI service identifier. A SIP URI shall not contain URL parameters and shall be encoded respecting the definitions of [RFC3986].	Y	String
hl	It indicates the current device language setting. Its format shall follow the [ISO 639-1] definitions.	Y	String
ho	Home operator of the device and this shall be represented as <MCCMNC> combination. Whereby MCC and MNC shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning.	Y. except for rcs.me web server requests	String
gid1	If available, the value of the Group Identifier 1 (GID1) SIM file content which are used for Service Provider personalisation.	N. if the SIM includes it, it shall be included	String
gid2	If available, the value of the Group Identifier 2 (GID2) SIM file content which are used for Service Provider personalisation.	N. if the SIM includes it, it shall be included	String
v	The version indicates the JSON format version of the Chatbot Information data that the client is able to process. It shall be set to 5 for clients complying with this version of this specification.	Y	String
wid	A Chatbot Welcome Message identifier provided as part of the deep link to a Chatbot to provide a user conversation starter, consistent with the encoding in [RFC3986]	Y, if provided by the client	String

Query Parameter	Description	Mandatory	Format
ctx	Contextual information provided by a deep link to enable a Chatbot Welcome Message that is specific call to action surface e.g. a specific web-site or business listing, consistent with the encoding in [RFC3986].	Y, if available to the client	String

Table 63: Query parameters of the botinfo URL

The client shall connect to the Chatbot Information Function in accordance with the values of the scheme and authority of the botinfo URL and send a HTTP GET request using the botinfo URL resulting from the procedures above.

The client shall specify the botinfo JSON format version that it is able to process.

The Chatbot Information Function shall handle this version as follows:

- If the version is omitted or v=1, the Chatbot Information Function shall return Chatbot Information data in the JSON format that complies with version 8.0 of this document i.e. [GSMA PRD-RCC.07v8.0].
- If the version is v=2, the Chatbot Information Function shall return Chatbot Information data in the JSON format that complies with versions 9.0 and version 10.0 of this document.
- If the version is v=3, the Chatbot Information Function shall return Chatbot Information data in the JSON format that complies with versions 11.0 to 14.0 of this document.
- If the version is v=4, the Chatbot Information Function shall return Chatbot Information data in the JSON format that complies with versions 15.0 and 16.0 of this document.
- If the version is v=5, the Chatbot Information Function shall return Chatbot Information data in the JSON format defined in section 3.6.4.1.6.

To provide backward compatibility, an RCS client shall be able to process Chatbot Information data in the JSON format that complies with version 8.0 of this document i.e. [GSMA PRD-RCC.07v8.0].

3.6.4.1.2 Initial Chatbot Information response handling

If the client receives in result of processing of the request for the initial Chatbot Information retrieval:

- any HTTP 5XX error response other than HTTP 503 Internal Server error with a Retry-After header or HTTP 4XX error response other than 408:
    - If the request is initiated as the result of a Chatbot attempt to contact the user for the first time:
        - The client shall not attempt any retry request towards the Chatbot Information Function and it shall not apply any UX procedures for Chatbot initiated conversations.

- If the request is initiated as a result of a user attempt to contact a Chatbot for the first time:
  - For a tel URI, a 404 error indicates that the Chatbot service identifier in the Chatbot Information request is an invalid Chatbot address, and the client shall proceed accordingly using Capability Discovery in Section 2.6.
  - For other responses or where the Chatbot service identifier is a SIP URI, the client shall not attempt any retry request towards the Chatbot Information Function, shall consider the address used to retrieve Chatbot Information not to be linked to a Chatbot and shall notify the user.

- an HTTP 503 Internal Server error response with a Retry-After header:
  - The client shall retry the request towards the Chatbot Information Function. The recommended value to retry shall be specified in the “Retry-After” header. The client shall retry for a maximum of three times.
  - If a request is processed successfully after retry, the procedures described for the HTTP 200 OK response shall apply.
  - If the last retry of the request fails, the client shall consider the transaction failed and the procedures described for any HTTP 5XX error response other than HTTP 503 Internal Server error with a Retry-After header or HTTP 4XX error response other than 408 shall apply.

- an HTTP 408 error response or no response (Chatbot Information retrieval request timeout):
  - The client shall manage the procedures locally on the device. If for the failure of a request a retry is applicable, the client shall retry by sending the same request again. The client shall retry for a maximum of three times.
  - If a request is processed successfully, the procedures described for the HTTP 200 OK response shall apply.
  - If the last retry of the request fails, the client shall consider the transaction failed and the procedures described for any HTTP 5XX error response other than HTTP 503 Internal Server error with a Retry-After header or HTTP 4XX error response other than 408 shall apply

- an HTTP 200 OK response:
  - If the request was initiated as a result of a user attempt to contact a Chatbot for the first time, then the client
    - shall check whether the PCC part of the Chatbot Information data contains Service ID information as per section 3.6.4.1.6.
    - If Service ID information is contained, then the client shall compare the SIP URI contained in the Service ID information of the Chatbot Information data with the SIP URI for which the Chatbot Information retrieval was triggered. Prior to comparison, URI parameters and headers shall be removed from both SIP URIs. The comparison of the resulting SIP URIs shall follow the URI

comparison rules defined in [RFC3261]. If the SIP URIs do not match, then the client shall continue processing as defined in section 3.6.4.1.3.
- Otherwise, if Service ID information is contained and the SIP URIs match, then the client shall continue processing as defined in section 3.6.4.1.4.
- If the Service ID is not present, the client shall:
  - invoke the fallback URL provided in the Chatbot Information and include the "wid" and "ctx" as parameters.
  - ignore all other Chatbot Information data

- If the request was initiated as a result of a Chatbot attempt to contact a user the first time, then the client shall continue processing as defined in section 3.6.4.1.4.

3.6.4.1.3 Discovery of an alternative Chatbot Service Identifier

If the processing of the initial Chatbot Information returned a new SIP URI in the Service ID information of the PCC part of the Chatbot Information as per section 3.6.4.1.2, then the client

- shall use the SIP URI contained in the Service ID information to initiate a second Chatbot Information retrieval using the procedures defined in section 3.6.4.1.1.
- If the second Chatbot Information response results in
  - an HTTP 200 OK response, then the client shall
    - use the Chatbot Information data received in the second Chatbot Information response for the further processing, and
    - use the SIP URI received in the Service ID information of the PCC part of the Chatbot Information data received in first Chatbot Information response as the contact address of the Chatbot. The client shall discard the SIP URI which triggered the first Chatbot Information and the rest of the Chatbot Information itself.

    The client shall continue processing as defined in section 3.6.4.1.4.
  - any HTTP 5XX error response other than HTTP 503 Internal Server error with a Retry-After header or HTTP 4XX error response other than 408, then the client shall continue with the SIP URI which triggered the first Chatbot Information request and the Chatbot Information data received in the first Chatbot Information response.
  - an HTTP 408 error response or no response (Chatbot Information retrieval request timeout):
    - The client shall manage the procedures locally on the device. If for the failure of a request a retry is applicable, the client shall retry by sending the second request for Chatbot Information again. The client shall retry for a maximum of three times.
    - If a request is processed successfully, the procedures described for the HTTP 200 OK response shall apply.
    - If the last retry of the request fails, the client shall consider the transaction failed and the procedures described for any HTTP 5XX error response other

than HTTP 503 Internal Server error with a Retry-After header or HTTP 4XX error response other than 408 shall apply

3.6.4.1.4 Successful Chatbot Information Response Handling

If the Chatbot Information retrieval was successful, then the client shall

- store the Etag and Cache-Control mandatory header values according to the procedures of [RFC7232] (see Table 64),
- store the Chatbot Information data, returned by the Chatbot Information Function, and
- consider the address used to retrieve Chatbot Information to be linked to a Chatbot.
- For a request initiated as the result of a Chatbot attempting to contact a user for first time, apply any UX procedures for Chatbot initiated conversations.

The following definitions apply for the use of the cache control header values:

Header	Value
Date	Server current date.
Expires	A date in the past to ensure that legacy HTTP/1.0 proxies do not try to cache the content, e.g. Mon, 01 Jan 1990 00:00:00 GMT
Cache-Control	private, max-age = <days_in_seconds>. HTTP/1.1 proxies shall not cache the content either due to the private directive. Max-age value is determined by the bilateral contract between Chatbot and Chatbot Platform that is out of the scope of this specification.
ETag	<entity_tag_value> assigned by the server.

Table 64: Cache control headers for Chatbot Information

3.6.4.1.5 Chatbot Information refresh handling

Whenever the Chatbot Information needs to be refreshed, i.e. when it is required (for example, when the user opens the conversation with the Chatbot) and found to be out of date based on the *Cache-Control* directive received during the last retrieval, the client shall send an HTTP request including the stored ETag in a *if-none-match* header according to the procedures of [RFC7232].

If the client receives in result of processing the Chatbot Information refresh request:

- Any HTTP 5XX or HTTP 4XX error response or no response the client shall
  - continue using the existing Chatbot Information until the next client trigger and
  - if the Chatbot was considered to be verified (see section 3.6.4.2), remove the verified status of the Chatbot if the Chatbot Information refresh request fails 3 times (i.e. 3 client triggers) in succession.
- an HTTP 304 NOT_MODIFIED response, the client shall:
  - store the Etag and Cache-Control directive value according to the procedures of [RFC7232]
  - continue using the existing Chatbot Information

- an HTTP 200 OK response, the client shall:
    - store the Etag and Cache-Control directive value according to the procedures of [RFC7232]
    - store the provided Chatbot Information data

3.6.4.1.6 Chatbot Information data

The Chatbot Information comprises:

- Business information data and included verification information
- Chatbot Welcome Message and Chatbot capability data

3.6.4.1.6.1 Chatbot Business Information data

The Chatbot Information data is provided in the JSON format. Part of the whole payload is represented using the schema defined in OMA CAB Personal Contact Card (PCC) document [CAB_TS], and an “organization” type PCC shall be used. When saving the Chatbot Information to the Contacts, only the information that is part of the PCC would be considered. The mapping between the Chatbot Information elements defined in section 15.2 of [GSMA PRD-RCC.71] and the PCC is specified in Table 65. The client shall gracefully ignore any JSON object which cannot be recognised.

Information	PCC Element	Identifier
Service name	org-name.display-name NOTE: recommended to be a maximum of 100 characters	org-name with org-name-type equal to “OfficialName”
Service ID	comm-addr.uri-entry.addr-uri	uri-entry with label equal to “ServiceID”
SMS	comm-addr.uri-entry.addr-uri	uri-entry with label equal to “SMS”
Fallback	comm-addr.uri-entry.addr-uri	uri-entry with label equal to “Fallback”
Call-back phone number	comm-addr.tel.tel-nb.tel-str	tel-type is set to “Work”
Service icon	media-list.media-entry.media.media-url	media-entry with label equal to “Icon”
Service description	a customised string type element named “org-description” in org-details. NOTE: up to 500 characters.	
Category	category-list.category-entry NOTE: recommended to be a maximum of 15 categories in the category-list, with a maximum of 50 characters per category-entry.	

Table 65: Mapping between Chatbot Information and PCC

For the Service icon as indicated in the schema in Table 67 a URL will be provided and a fingerprint could be included that can be used to verify the integrity of the Service icon file after download. The fingerprint shall be a SHA-256 hash of the icon file that the link refers to, provided in lowercase hexadecimal representation without "0x" prefix.

This part with PCC information is complemented with further information on the Chatbot defined in section 15.2 of [GSMA PRD-RCC.71]:

- Version Number of the Chatbot Information version to which the provided information conforms. It is provided in the “version” property of the “botinfo” object in the JSON object providing the Chatbot Information. If the "version" property or "botinfo" object is absent, it shall be assumed that the Chatbot Information complies with version 8.0 of this document (i.e. [GSMA PRD-RCC.07v8.0]).
- Provider Name provided as a string recommended to be a maximum of 100 characters in the "provider" property of the "botinfo" object in the JSON object providing the Chatbot Information
- An email-address provided in the "email" property of the "botinfo" object in the JSON object providing the Chatbot Information
- One or more “web-entry” elements within the “web-resources” properties of the “botinfo” object in the JSON object providing the Chatbot Information
- A colour related to the Chatbot provided as a base64 url encoded hexadecimal RGB colour representation in the "colour" property of the "botinfo" object in the JSON object providing the Chatbot Information
- A link to an image recommended to be a maximum of 200 KB in jpg, png or GIF format that can be used as background provided as a URL in the "backgroundImage" property of the "botinfo" object in the JSON object providing the Chatbot Information
- A Chatbot business location provided as a string recommended to be a maximum of 200 characters in the "address" property of the "botinfo" object in the JSON object providing the Chatbot Information
- Disable User Feedback indicates the method for user interaction with a Chatbot. It is provided in the "Disable_User_feedback" property of the "botinfo" object in the JSON object providing the Chatbot Information. If the value of "Disable_User_feedback" is "true", the user shall not be able to send user-generated content to the Chatbot. If the value of "Disable_User_feedback" is "false", the user shall be able to send user-generated content to the Chatbot (default value).
- A persistent menu with a nested collection of suggested replies and suggested actions (see section 3.6.10) of up to 3 levels in the “persistent-menu” property in the JSON object providing the Chatbot Information

NOTE: in the examples in Table 68 and Table 69 contain only a basic persistent menu for illustration purposes. A more elaborate example of the menu structure is provided in section 3.6.12.4.

- A reference to a generic Cascading Style Sheet (CSS) applicable to all Rich Cards send by this Chatbot, provided in the "generic-CSS-template" property that refers to an external CSS file. When provided the value of this property shall be handled as defined in section 3.6.10.5.4. The client shall keep it inaccessible to the user.

- Verification information (see also section 3.6.4.2) which takes the format of an object providing signatures in JWS format that can be used to verify the information as specified in section 3.6.4.2.2

NOTE: If Verification information is provided, the verification procedures defined in sections 3.6.4.2.1 and 3.6.4.2.2 mean that a fingerprint must be provided to ensure a successful validation of this signature if an icon is provided.

3.6.4.1.6.2 Chatbot capabilities and Chatbot Welcome Message data

Chatbot capabilities and Chatbot Welcome Message data are defined in Table 66:

Field Name	Type	Requirement	Definition
chatbot_capabilities	object	Required	Container for feature tags and Chatbot version.
supported_features	array (of objects)	Required	List of objects. Each object contains a specific RCS feature tags or IARI for a Chatbot. The list of valid feature tag values is defined in section 2.6.1.3 and 3.6.2.1
chatbot_version_id	string	Required	The Chatbot's application version (e.g., "3") detailing the features supported.
welcome_message	object	Optional	Container for the introductory message. This may be one of either prefilled_text or message_payload
prefilled_text	object/string	Optional	The prefilled text content if message_payload is not present
message_payload	object/string	Optional	The full, canonicalised content (plain text body, Rich Card JSON, etc.) if a Chatbot Welcome Message is present and prefilled_text is not present
fallback_url	object/string	Optional	The Chatbot Information Fallback URL in Table 65 must be provided in the event that a service id or alternate service id is not provided by the Chatbot Platform.

Table 66: Chatbot Capability and Chatbot Welcome Message Data

3.6.4.1.6.2.1 Chatbot Welcome Message Response Handling

If the Chatbot Information request includes wid or ctx parameters, the welcome_message container may comprise either a message_payload object or pre-filled text.

- If the welcome_message indicates prefilled_text, the client shall display the payload as editable text. When the user submits this message (or an edited version of it), the client shall:

- Retrieve the locally stored wid and ctx values when available (associated with the resolved Chatbot Service Identifier).
- Include these values in the user message through the following Deep Link CPIM headers:

  - Welcome-ID: the wid parameter value from the deep link.
  - Deep-Link-Context: the ctx parameter value from the deep link.

  These headers are defined in Annex C.3.13 and C.3.14.

NOTE: If a user witholds the "ctx" parameter, it is expected that the Chatbot will process the "wid" parameter to provide a non-context aware Chatbot Welcome Message.

NOTE: in cases where no "wid" or "ctx" parameters are available, or the Chatbot Platform fails to deliver a Chatbot Welcome Message in response to these parameters, the client can choose to provide a default pre-filled conversation starter.

3.6.4.1.6.3 Chatbot Information Schema

The JSON schema for Chatbot Information comprising business information, Chatbot Welcome Message and capability data is defined in Table 67.

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "id": "http://www.gsma.com/rcs/chatbot_info_schema.json",

  "definitions": {
    "tel-nb": {
       "type": "object",
       "properties": {
          "tel-str": {
             "type": "string"
          }
       },
       "required": ["tel-str"]
    },
    "tel": {
       "type": "object",
       "properties": {
          "label": {
            "type": "string"
          },
          "tel-nb": {
            "$ref": "#/definitions/tel-nb"
          },
          "tel-type": {
            "type": "string"
          }
       },
       "required": ["label", "tel-nb", "tel-type"]
    },
    "uri-entry": {
      "type": "array",
```

```
    "items": {
      "type": "object",
      "properties": {
        "addr-uri": {
          "type": "string",
          "format": "uri"
        },
        "addr-uri-type": {
          "type": "string",
          "enum": ["SIP-URI", "Other"]
        },
        "label": {
          "type": "string",
          "enum": ["ServiceID", "SMS", "Fallback"]
        }
      },
      "required": ["addr-uri", "addr-uri-type", "label"]
    }
  },
  "web-entry": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "url": {
          "type": "string",
          "format": "uri"
        },
        "label": {
          "type": "string"
        }
      },
      "required": ["url", "label"]
    }
  },
  "comm-addr": {
    "type": "object",
    "properties": {
      "tel": {
        "$ref": "#/definitions/tel"
      },
      "uri-entry": {
        "$ref": "#/definitions/uri-entry"
      }
    },
    "required": ["uri-entry"]
  },
  "media": {
    "type": "object",
    "properties": {
      "media-url": {
        "type": "string",
        "format": "uri"
      },
      "fingerprint": {
        "type": "string"
      }
```

```
    },
    "required": ["media-url"]
  },
  "media-entry": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "label": {
          "type": "string",
          "enum": ["Icon"]
        },
        "media": {
          "$ref": "#/definitions/media"
        },
        "media-content": {
          "type": "string",
          "enum": ["Logo", "Other"]
        }
      },
      "required": ["label", "media", "media-content"]
    }
  },
  "org-name": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "display-name": {
          "type": "string"
        },
        "org-name-type": {
          "type": "string",
          "enum": ["OfficialName"]
        }
      },
      "required": ["display-name", "org-name-type"]
    }
  },
  "address-entry": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "addr-string": {
          "type": "string"
        },
       "addr-type": {
          "type": "string"
       },
        "label": {
          "type": "string"
        }
      },
      "required": ["addr-string", "label", "addr-type"]
    }
  },
```

```
    "category-entry": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "verification-info": {
      "type": "object",
      "properties": {
        "verifier-logo": {
          "type": "object",
          "properties": {
            "media-entry": {
              "$ref": "#/definitions/media-entry"
            }
          }
        },
        "verification-signatures": {
          "description": "RFC7515 (JWS) general JWS JSON serialisation",
          "type": "object"
        }
      },
      "required": ["verification-signatures"]
    },
    "chatbot_capabilities": {
      "type": "object",
      "properties": {
        "supported_features": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "feature_tag": {
                "type": "string",
                "description": "Supported chatbot feature tags as defined in Table 66"
              }
            },
            "required": ["feature_tag"]
          }
        },
        "chatbot_version_id": {
          "type": "string",
          "description": "The Chatbot's application version (e.g., '3') detailing the features supported."
        }
      },
      "required": ["supported_features", "chatbot_version_id"]
    },
    "welcome_message": {
      "type": "object",
      "description": "Defines the Chatbot Welcome Message. Must be EITHER a simple text draft OR a full Rich Card payload.",
      "oneOf": [
        {
          "title": "Option 1: Simple Draft Text",
          "description": "A simple string that populates the compose bar. The text acts as both the button label and the message content.",
          "properties": {
```

```
          "pre_filled_text": {
            "type": "string",
            "description": "The text to be inserted into the user's compose box."
          }
        },
        "required": ["pre_filled_text"]
      },
      {
        "title": "Option 2: Rich Card Payload",
        "description": "A complete RCC.07 Rich Card definition (JSON) or encoded payload. All UI/Text is
defined inside this payload.",
        "properties": {
          "content_type": {
            "type": "string",
            "description": "The content_type of the payload, e.g. application/vnd.gsma.botmessage.v1.0+json for
a Rich Card."
          },
          "message_payload": {
            "type": "object",
            "description": "The JSON string representing the Rich Card content."
          }
        },
        "required": ["content_type","message_payload"]
      }
    ]
  }
 },

 "type": "object",

 "properties": {
  "chatbot_capabilities": {
    "$ref": "#/definitions/chatbot_capabilities"
  },
  "welcome_message": {
    "$ref": "#/definitions/welcome_message"
  },
  "botinfo": {
    "type":"object",
    "properties": {
      "pcc": {
        "type": "object",
        "properties": {
          "org-details": {
            "type": "object",
            "properties": {
              "comm-addr": {
                "$ref": "#/definitions/comm-addr"
              },
              "media-list": {
                "type": "object",
                "properties": {
                  "media-entry": {
                    "$ref": "#/definitions/media-entry"
                  }
                }
              },
```

```
            "org-name": {
              "$ref": "#/definitions/org-name"
            },
            "org-description": {
              "type": "string",
              "maxLength": 500
            },
            "category-list": {
              "type": "object",
              "properties": {
                "category-entry": {
                  "$ref": "#/definitions/category-entry"
                }
              }
            }
          }
        },
        "pcc-type": {
          "type": "string"
        }
      }
    },
    "version":{
      "type":"string"
    },
    "provider":{
      "type":"string"
    },
    "email":{
      "type":"string",
      "format":"email"
    },
    "colour":{
      "type":"string",
      "description": "base64 url encoded colour representation"
    },
    "backgroundImage":{
      "type":"string",
      "format":"uri"
    },
    "web-resources": {
      "type": "object",
      "properties": {
        "web-entry": {
          "$ref": "#/definitions/web-entry"
        }
      }
    },
    "address": {
      "type": "object",
      "properties": {
        "address-entry": {
          "$ref": "#/definitions/address-entry"
        }
      }
    },
    "Disable_User_feedback":{
```

```
        "type":"boolean",
       }
     },
     "required": ["pcc"]
   },
   "persistent-menu":{
     "$ref": "http://www.gsma.com/rcs/chatbot_menu_schema.json#/properties/menu"
   },
   "generic-CSS-template": {
     "type":"string",
     "format":"uri"
   },
   "bot-verification": {
     "type": "object",
     "properties":{
       "verification-info":{
         "$ref": "#/definitions/verification-info"
       }
     },
     "required": ["verification-info"]
   }
 },
 "required": ["chatbot_capabilities","botinfo"]
}
```

Table 67: JSON schema for Chatbot Information

A non-normative example Chatbot Information request and response between Chatbot Platform and Service Provider Network can be found in Table 68

```
GET
/bot?id=sip%3Afoo.bar%40botplatform.examplebot.com&&hl=en&ho=370270&v=4
HTTP/1.1
Host: botinfo.botplatform.com
Accept: application/json
If-None-Match: 1

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: nnnn
Date: Mon, 17 Apr 2017 20:00:00 GMT
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Cache-Control: private, max-age=86400
ETag: 2

{
 "botinfo":{
  "pcc": {
   "pcc-type": "organization",
   "org-details": {
    "org-name": [{
     "display-name": "Service Name",
     "org-name-type": "OfficialName"
     }
    ],
   "comm-addr": {
```

```
        "uri-entry": [{
          "addr-uri-type": "SIP-URI",
          "addr-uri": "sip:serviceid@example.com",
          "label": "ServiceID"
        },
        {
          "addr-uri-type": "Other",
          "addr-uri": "sms:+15105550101",
          "label": "SMS"
        },
        {
          "addr-uri-type": "Other",
          "addr-uri": " http://example.com/fallback",
          "label": "Fallback"
        }],
        "tel": {
          "tel-type": "Work",
          "tel-nb": {
            "tel-str": "1-800-555-1213"
          },
          "label": "Callback"
        }
      },
      "media-list": {
        "media-entry": [{
          "media-content": "Logo",
          "media": {
            "media-url": "http://example.com/myLogo.png",
            "fingerprint": "73a982e7930460d3c047c2429f35cdeeebba6e86eed259d50c40ef27b8a87711"
          },
          "label": "Icon"
        }]
      },
      "category-list": {
        "category-entry": [
          "News", "Finance"
        ]
      },
      "org-description": "Example service description"
    }
  },
  "version": "5",
  "provider": "Brand X",
  "colour": "IzAwZmYwMA==",
  "backgroundImage":"http://example.com/myBackground.png",
  "web-resources": {
     "web-entry": [
        {
          "url": "http://example.com/index.html",
          "label": "Website",
        },
        {
          "url": "http://example.com/tc.html",
          "label": "Terms and Conditions",
        },
        {
          "url": "http://example.com/privacy.html",
```

```
          "label": "Privacy Policy",
        }
       ]
    },
   "address": {
    "address-entry": [{
     "addr-string": "XYZ Corporation,111 Park Avenue,Huntsville AL 11111, USA",
     "addr-type": "Business",
     "label": "BusinessLocation"
    }]
   },
   "Disable_User_feedback": true
  },
  "persistent-menu": {
   "entries": [{
    "reply": {
     "displayText": "reply1",
     "postback": {
      "data": "set_by_chatbot_reply1"
     }
    }
   }
   ]
  },
  "chatbot_capabilities": {
   "chatbot_version_id": "3",
   "supported_features": [
    {
     "feature_tag": "+g.3gpp.iari-ref=\"urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot\"",
     "description": "Chatbot Session"
    },
    {
     "feature_tag": "+g.3gpp.iari-ref=\"urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp\"",
     "description": "File Transfer"
    }
   ]
  },
  "generic-CSS-template": "http://example.com/default.css",
  "bot-verification": {
   "verification-info": {
    "verification-signatures": {
     "payload":
"eyJpY29uZmluZ2VycHJpbnQiOiI3M2E5ODJlNzkzMDQ2MGQzYzA0N2MyNDI5ZjM1Y2RlZWViYmE2ZTg2ZWVkMjU5ZDUwYzQwZWYyN2I4YTg3NzExIiwiaWQiOiJzaXA6c2VydmljZWlkQGV4YW1wbGUuY29tIiwibmFtZSI6IlNlcnZpY2UgTmFtZSJ9",
     "signatures": [{
      "protected":
"eyJhbGciOiJFUzI1NiIsImNyaXQiOlsiYm90dmZleHBpcmVzIl0sImJvdHZmZXhwaXJlcyI6IjIwMTgtMDItMjJUMTQ6MjI6NTJaIn0=",
      "header":{
       "kid":"ec45e387-b742-436a-7312-e37e9a41148d"
      },
      "signature":
       "DpNiU3ljbEg87O8PLifUAqOyKAM6-Xx-
E7GawxeppmNFCgfTiDoi7djxLa8ILOSApmWNgfKTUJqPP3-Kg6NY2R"
     },
     {
```

```
        "protected":
"eyJhbGciOiJFUzI1NiIsImNyaXQiOlsiYm90dmZleHBpcmVzIl0sImJvdHZmZXhwaXJlcyI6IjIwMjAtMDQtMDR
UMjM6NTk6MDBaIiwiYm90dmZkYXRlIjoiMjAxOS0wNC0wNFQyMzo1OTowMFoifQ==",
        "header": {
          "kid":"ec4c03a7-ca54-4719-8327-b1c4e874db0d"
        },
        "signature":
          "DiNiUpilbEg8e88OLifUZqOyKsD6-Xx-
E7GaIUenfcNFCgfTiRop7dj8La78LOSApmSNmgKTUJUPP3-Kp6RS2t"
      }
      ]
    }
   }
  }
}
```


Table 68: Example Chatbot Information request and response between Chatbot Platform and Service Provider Network

A non-normative example Chatbot Information request and response between Service Provider Network and Client can be found in Table 69

```
GET
/bot?id=sip%3Afoo.bar%40botplatform.examplebot.com&&hl=en&ho=370270&v=4
HTTP/1.1
Host: botinfo.operator.com
Accept: application/json
If-None-Match: 1

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: nnnn
Date: Mon, 17 Apr 2017 20:00:00 GMT
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Cache-Control: private, max-age=86400
ETag: 2

{
 "botinfo":{
  "pcc": {
   "pcc-type": "organization",
   "org-details": {
    "org-name": [{
     "display-name": "Service Name",
     "org-name-type": "OfficialName"
     }
    ],
    "comm-addr": {
     "uri-entry": [{
      "addr-uri-type": "SIP-URI",
      "addr-uri": "sip:serviceid@example.com",
      "label": "ServiceID"
     },
     {
      "addr-uri-type": "Other",
      "addr-uri": "sms:+15105550101",
```

```
        "label": "SMS"
       },
       {
        "addr-uri-type": "Other",
        "addr-uri": " http://example.com/fallback",
        "label": "Fallback"
       }],
       "tel": {
        "tel-type": "Work",
        "tel-nb": {
         "tel-str": "1-800-555-1213"
        },
        "label": "Callback"
       }
      },
      "media-list": {
       "media-entry": [{
        "media-content": "Logo",
        "media": {
         "media-url": "http://example.com/myLogo.png",
         "fingerprint": "73a982e7930460d3c047c2429f35cdeeebba6e86eed259d50c40ef27b8a87711"
        },
        "label": "Icon"
       }]
      },
      "category-list": {
       "category-entry": [
        "News", "Finance"
       ]
      },
      "org-description": "Example service description"
     }
    },
    "version": "5",
    "provider": "Brand X",
    "colour": "IzAwZmYwMA==",
    "backgroundImage":"http://example.com/myBackground.png",
    "web-resources": {
      "web-entry": [
       {
        "url": "http://example.com/index.html",
        "label": "Website",
       },
       {
        "url": "http://example.com/tc.html",
        "label": "Terms and Conditions",
       },
       {
        "url": "http://example.com/privacy.html",
        "label": "Privacy Policy",
       }
      ]
     },
    "address": {
     "address-entry": [{
      "addr-string": "XYZ Corporation,111 Park Avenue,Huntsville AL 11111, USA",
      "addr-type": "Business",
```

```
      "label": "BusinessLocation"
    }]
  },
  "Disable_User_feedback": true
},
"persistent-menu": {
  "entries": [{
    "reply": {
      "displayText": "reply1",
      "postback": {
        "data": "set_by_chatbot_reply1"
      }
    }
  }
  ]
},
"chatbot_capabilities": {
  "chatbot_version_id": "3",
  "supported_features": [
    {
      "feature_tag": "+g.3gpp.iari-ref=\"urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot\"",
      "description": "Chatbot Session"
    },
    {
      "feature_tag": "+g.3gpp.iari-ref=\"urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.fthttp\"",
      "description": "File Transfer"
    }
  ]
},
"welcome_message": {
  "content_type": "application/vnd.gsma.botmessage.v1.0+json",
  "message_payload": {
    "generalPurposeCard": {
      "layout": {
        "cardOrientation": "VERTICAL",
        "titleFontStyle": ["bold"]
      },
      "content": {
        "title": "Welcome to Service Name",
        "description": "We see you are interested in the offer at Store 99. Tap below to redeem.",
        "media": {
          "mediaUrl": "https://example.com/offer_image.jpg",
          "mediaContentType": "image/jpeg",
          "mediaFileSize": 50000,
          "height": "MEDIUM_HEIGHT"
        },
        "suggestions": [
          {
            "reply": {
              "displayText": "View Offer",
              "postback": {
                "data": "action_view_offer",
                "wid": "campaign123",
                "ctx": "store99"
              }
            }
          }
```

```
        ]
       }
      }
     }
    },
    "generic-CSS-template": "http://example.com/default.css",
    "bot-verification": {
     "verification-info": {
      "verifier-logo": {
       "media-entry": [{
        "media-content": "Logo",
        "media": {
         "media-url": "http://example.com/myLogo.png",
         "fingerprint": "73a982e7930460d3c047c2429f35cdeeebba6e86eed259d50c40ef27b8a87711"
        },
        "label": "Icon"
       }]
      },
      "verification-signatures": {
       "payload":
"eyJpY29uZmluZ2VycHJpbnQiOiI3M2E5ODJlNzkzMDQ2MGQzYzA0N2MyNDI5ZjM1Y2RlZWViYmE2ZTg2ZWVkMjU5ZDUwYzQwZWYyN2I4YTg3NzExIiwiaWQiOiJzaXA6c2VydmljZWlkQGV4YW1wbGUuY29tIiwibmFtZSI6IlNlcnZpY2UgTmFtZSJ9",
       "signatures": [{
        "protected":
"eyJhbGciOiJFUzI1NiIsImNyaXQiOlsiYm90dmZleHBpcmVzIl0sImJvdHZmZXhwaXJlcyI6IjIwMTgtMDItMjJUMTQ6MjI6NTJaIn0=",
        "header":{
         "kid":"ec45e387-b742-436a-7312-e37e9a41148d"
        },
        "signature":
         "DpNiU3ljbEg87O8PLifUAqOyKAM6-Xx-
E7GawxeppmNFCgfTiDoi7djxLa8ILOSApmWNgfKTUJqPP3-Kg6NY2R"
        },
        {
         "protected":
"eyJhbGciOiJFUzI1NiIsImNyaXQiOlsiYm90dmZleHBpcmVzIl0sImJvdHZmZXhwaXJlcyI6IjIwMjAtMDQtMDRUMjM6NTk6MDBaIiwiYm90dmZkYXRlIjoiMjAxOS0wNC0wNFQyMzo1OTowMFoifQ==",
         "header": {
          "kid":"ec4c03a7-ca54-4719-8327-b1c4e874db0d"
         },
         "signature":
          "DiNiUpilbEg8e88OLifUZqOyKsD6-Xx-
E7GaIUenfcNFCgfTiRop7dj8La78LOSApmSNmgKTUJUPP3-Kp6RS2t"
        }
       ]
      }
     }
    }
}
```


Table 69: Example Chatbot Information request and response between Service Provider Network and client

3.6.4.2 Verification of Chatbot Information

3.6.4.2.1 Verification Authority Procedures

When a verification authority verifies a Chatbot and considers it as trustworthy, it shall generate a signature using its private key over a JSON object complying with the following schema:

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "id": {
      "type": "string",
      "format": "uri"
    },
    "name": {
      "type": "string"
    },
    "iconfingerprint": {
      "type": "string"
    },
  },
  "required": [
    "id", "name"
  ]
}
```

Table 70: JSON Schema for Chatbot Verification payloads

In this JSON object, the different properties shall be set as follows:

JSON	Description
id	Chatbot service identifier of the verified Chatbot without URI parameters
name	Chatbot name of the verified Chatbot
iconfingerprint	Fingerprint of the file providing the Chatbot icon image. The fingerprint shall be a SHA-256 hash of the icon file that was verified, provided in lowercase hexadecimal representation without "0x" prefix. If no icon is available, the iconfingerprint property shall not be provided.

Table 71: Mapping of Chatbot data to JSON Schema for Chatbot verification payloads

This JSON shall be used as payload for a JWS signature. That JWS shall be created as follows:

- Unless the actual formatting of the JSON object used for a particular Chatbot is fixed by a central entity (e.g. Chatbot Platform or Chatbot itself) guaranteeing that object is provided in the exact same format to all verification authorities verifying the Chatbot, the verification authority shall use the following rules for the formatting of the object in the payload:
  - no whitespace or line breaks shall be provided before or after any syntactic elements and
  - the included members shall be ordered lexicographically by the Unicode code points of the member names.

Example:

{"iconfingerprint":"73a982e7930460d3c047c2429f35cdeeebba6e86eed259d50c40ef27b8a87711","id":"sip:serviceid@example.com","name":"Service Name"}

Table 72: Example of Chatbot verification payloads

The JSON object representation shall then be base64url encoded as defined in [RFC7515] to be used as payload in the JWS

- The Verification Authorities public key shall be identified through the key's fingerprint which will be determined as specified in [RFC7638]. It shall be provided in the JWS in a *kid* property in the unprotected header.
- the JWS Protected header shall include
  - a *crit* header as defined in [RFC7515] with including the value "*botvfexpires*" and
  - a *botvfexpires* property set to the chosen expiry date of the signature, encoded as a JSON string of the date-time format.
  - An optional botvfdate property set to the date and time of the signature, encoded as a JSON string of the date-time format.
- The JWS shall be generated as specified in section 5.1 of [RFC7515] using one of the algorithms specified as "Recommended+" or "Recommended" in section 3.1 of [RFC7518].

3.6.4.2.2 Chatbot Platform Procedure

The Chatbot Platform shall provide the signatures that are generated by the Verification Authorities following the process defined in section 3.6.4.2.1 together with the Chatbot Information as specified in section 3.6.4.1.6.

3.6.4.2.3 Service Provider procedures

To perform the verification of Chatbot Information, the Service Provider shall ensure that it is included in the retrieval path of such information by configuring the clients with a BOTINFO FQDN ROOT configuration parameter defined in section A.1.3 that routes the request to where the verification shall be performed as defined in section 3.6.4.1.1.

The procedure through which a Service Provider verifies Chatbot Information should rely on the procedure defined in section 3.6.4.2.3.1 or on the verification of the Chatbot that was done when receiving the information from the Chatbot Directory defined in section 3.6.3.2.1. To verify the signatures the Service Provider shall support the algorithms defined as "Recommended" and "Recommended+" in section 3.1 of [RFC7518]. The verification of the Chatbot Information based on a recognised verification authority’s signature is recommended. Also, to ensure consistency in the verification status of a given Chatbot across Chatbot Directory query and Chatbot Information retrieval, the same type of verification procedure, i.e., signature-based or otherwise, should be followed in both scenarios. The Service Provider shall

- Construct the Chatbot Information that will be provided to the client by

- Including the "*botinfo*" property from the Chatbot Information that was received from the Chatbot Platform and
- when the Chatbot Information has been verified
    - add a *bot-verification* property with a *verification-info* property according to the schema defined in section 3.6.4.1.6

3.6.4.2.3.1 Signature Verification Procedure

The procedure to verify whether the Chatbot Information provided by the Chatbot Platform matches the information that has been verified by a Chatbot Verification Authority that is recognised by the Service Provider to perform verifications of Chatbots shall be as follows:

1. If the Chatbot Information does not include a *bot-verification* property that has a *verification-info* property with a *verification-signatures* property, the Chatbot Information shall be considered as an unverified and no further processing to verify whether the Chatbot Information has been verified by a recognised Chatbot Verification Authority shall be done.
2. If the *verification-signatures* property does not correspond to a JWS in the General JWS JSON Serialization defined in section 7.2.1 of [RFC7515], the Chatbot Information shall be considered as an unverified and no further processing to verify whether the Chatbot Information has been verified by a recognised Chatbot Verification Authority shall be done.
3. If the *verification-signatures* property does not include a *payload* property (i.e. it is using a detached payload according to Annex F of [RFC7515]), the Chatbot Information shall be considered as an unverified and no further processing to verify whether the Chatbot Information has been verified by a recognised Chatbot Verification Authority shall be done.
4. If (after Base64url-decoding) the *payload* property of the *verification-signatures* property does not correspond to a JSON object with at least the properties *id* and *name*, the Chatbot Information shall be considered as an unverified and no further processing to verify whether the Chatbot Information has been verified by a recognised Chatbot Verification Authority shall be done.
5. Otherwise, the different signature objects in the *signatures* property shall be verified as specified in section 5.2 of [RFC7515] taking into account the following restrictions:

    a) If the "*alg*" header parameter provided in the JWS Protected Header is not set to one of the algorithms defined as "Recommended" or "Recommended+" in section 3.1 of [RFC7518], the digital signature shall not be considered.
    b) If the signature object does not provide in the JWS Protected Header a "*crit*" Header parameter defined in section 4.1.11 of [RFC7515] including the value "*botvfexpires*" among the values, the digital signature shall not be considered.
    c) If the signature object does not contain a *botvfexpires* property in the JWS Protected Header set to a date in the future, the digital signature shall not be considered.
    d) If the optional *botvfdate* property in the JWS Protected Header is malformed or when it has a value older than the local policy for freshness permits, or the date is set into the future, the digital signature shall not be considered.

NOTE: It is desirable to implement a consistent policy for the freshness rule across all Service Providers in a given market.

e) If the *botvfdate* property is older than the last known change of the information used to verify the Chatbot, then the digital signature shall not be considered.

NOTE: Information on the timing of changes to the information might be available when the party verifying the signature is also the party operating the Chatbot Platform.

f) If the signature object does not provide a *kid* property in the unprotected header as reference to the key, the digital signature shall not be considered
g) If the value of the *kid* property in the unprotected header of the signature object does not correspond to a key of a Verification Authority recognised by the service provider, the digital signature shall not be considered

6. If at least one of the considered digital signatures was successfully verified, the data in the JSON object provided in the *payload* property shall be compared to the data in the corresponding entry for the Chatbot in the Chatbot Information as follows:

a) If the value of the *id* property in the JSON object provided in the *payload* property does not match the value of the *id* query parameter used to retrieve the Chatbot Information (see Table 63), the verification shall be considered to have failed
b) If the value of the *name* property in the *payload* property does not match the Service name information in the Chatbot Information (mapped as defined in Table 65), the verification shall be considered to have failed
c) If the Chatbot Information has a *fingerprint* property in the *media* property carrying the Service icon information (mapped as defined in Table 65) and the JSON object provided in the *payload* property does not have an *iconfingerprint* property, the verification shall be considered to have failed.
d) If the JSON object provided in the *payload* property provides an *iconfingerprint* property with a value and the Chatbot Information does not have a *fingerprint* property in the *media* property carrying the Service icon information (mapped as defined in Table 65), the verification shall be considered to have failed.
e) If the JSON object provided in the *payload* property provides an *iconfingerprint* property and the Chatbot Information has a *fingerprint* property in the *media* property carrying the Service icon information (mapped as defined in Table 65), but their values do not match, the verification shall be considered to have failed.
f) Otherwise, the verification of the "*payload*" property shall have succeeded.

7. If in step 6 the verification of the "*payload*" property was successful, it shall be verified whether a property was provided for an icon in the Chatbot Information,

a) If not, the verification of the Chatbot Information shall be considered to have succeeded
b) Otherwise, the verification of the Chatbot Information shall be considered to have succeeded when the value of the "fingerprint" property matches a SHA-256 hash of the file referred to by the URL of the icon in the Chatbot Information whereby

the result of the hash is represented as a hexadecimal string in lowercase without a "0x" prefix.

3.6.4.2.4 Client Procedures

The client shall determine the verification status of a Chatbot as per the steps defined in section 3.6.4.2.3.1. To verify the digital signature as mentioned in section 3.6.4.2.3.1 step 5.g), if the client doesn't have a cached certificate for the Verification Authority, corresponding to the "kid" in the JWS header in the verification-info property, it shall retrieve the certificate by sending a HTTP GET request to the Chatbot Information Function as specified in section 3.6.4.1.1. The URL format shall include "/certificates" in the path as defined in Table 73. The client shall use the URL scheme "https" for establishing a secure connection.

```
https://<root_domain>/certificates?<set_of_query_parameters>
```

Table 73: Composition of botinfo URL to retrieve the Verification Authority certificate

The client shall include the "set_of_query_parameters" using the parameters defined in Table 74 using the application/x-www-form-urlencoded encoding defined in [HTML-4.0].

Query Parameter	Description	Mandatory	Format
v	The version indicates the JSON format version of the Chatbot Information data that the client can process. It shall be set to 5 for clients complying with this version of this specification.	Y	String
kid	"kid" value as included in the JWS header in the verification-info property	Y	String

Table 74: Query parameters of botinfo URL to retrieve the certificate

A non-normative example for HTTP GET request and response is captured in the Table 75 below:

```
GET/certificates?v=5&kid=<kid_value_from_JWS_header_in_verification-
info>
HTTP/1.1
Host: botinfo.operator.com
Accept: application/x-pem-file
If-None-Match: 1


HTTP/1.1 200 OK
Content-Type: application/x-pem-file
Content-Length: nnnn
Date: Mon, 17 Apr 2017 20:00:00 GMT
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Cache-Control: private, max-age=86400
ETag: 2

-----BEGIN CERTIFICATE-----
```

```
MIIFNjCCBB6gAwIBAgIQKVMIIDdTCCAl2gAwIBAgILBAAAAAABFU3PczEwDQYJKoZIhvcNAQ
ELBQAwVzELMAkGA1UEBhMCVVMxHDAaBgNVBAoTE0dvb2dsZSBUcnVzdCBTZXJ2aWNlczEfm
... (This is the Leaf Cert for Verification Authority)
… (Base64 Encoded Data) ...
... (Contains the Public Key for Verification Authority) ...
... (Issuer: Intermediate CA) ... 9s/2+s12345==
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIIEsTCCA5mgAwIBAgIQBOHMIIDdTCCAl2gAwIBAgILBAAAAAABFU3PczEwDQYJKoZIhvcNA
QELBQAwVzELMAkGA1UEBhMCVVMxHDAaBgNVBAoTE0dvb2dsZSBUcnVzdCBTZXJ2aWNlczEfm
... (This is the Intermediate CA Cert)
... (Base64 Encoded Data) ...
... (Contains Public Key for Intermediate CA) ...
... (Issuer: Root CA) ... v8N7yy5Y0b2==
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIIDrzCCApegAwIBAgIQCD5MIIDdTCCAl2gAwIBAgILBAAAAAABFU3PczEwDQYJKoZIhvcNA
QELBQAwVzELMAkGA1UEBhMCVVMxHDAaBgNVBAoTE0dvb2dsZSBUcnVzdCBTZXJ2aWNlczEfm
... (This is the Root CA Cert) …
... (Base64 Encoded Data) ...
... (Contains Public Key for Root CA) ...
... (Issuer: Root CA ) ... P+a1b2c3d4e==
-----END CERTIFICATE-----
```

Table 75: Example Certificate request and response

The 200 OK response includes the Base64 PEM encoded string containing the public certificate chain of the Verification Authority, with leaf certificate at the top, followed by intermediate certificate and root certificate at the end. The client shall verify that the certificate chain received is valid (i.e. it is not expired, that there is a chain of trust to a known Root CA e.g. using a pre-installed “Trust Store”). If certificate is valid, the client shall use the public key to verify the JWS digital signature received in the verification-info property. The client shall continue with steps 6 onwards of section 3.6.4.2.3.1 to complete the verification process.

NOTE: How the Root CA certificate, and any associated policy, is defined or made available to the client is outside the scope of this CR document.

If the client is unable to parse the public certificate of the Verification Authority or the certificate is not valid for any reason e.g. expired or revoked, the client shall handle the situation as described for a HTTP 5XX response in section 3.6.4.1.2 or section 3.6.4.1.5 for an initial request or a refresh of Chatbot Information respectively. In case any certificates in the certificate chain are revoked, it is expected that the Service Provider may block the Chatbot and apply the Service Provider policy e.g. rejecting the traffic or removing the Chatbot from the Chatbot directory.

If the JWS signature verification procedure succeeds, the client shall consider the Chatbot Information as verified. Else, the client shall consider the Chatbot Information as unverified

NOTE: It is up to the client implementation on how user is made aware of the verification status. The client may show a verification pending state while it is in the process of retrieving and validating Verification Authority certificate.

The client shall cache the received public certificate of the Verification Authority based on steps defined in section 3.6.4.1.4. The client shall refresh the certificate based on steps defined in section 3.6.4.1.5.

In case the client has passed all checks for the verification process, it shall use the Verification Authority public certificate field called Subject Distinguished Name to derive the necessary information displayed to the user about the verifier of the Chatbot Information. Where this information is displayed is based on client implementation.

NOTE: If SPN and client both perform the verification, the results are expected to be the same.

3.6.5 Privacy Protection

3.6.5.1 Anonymization

3.6.5.1.1 Overview

The procedures for Anonymization defined in this section enable an RCS client to communicate with Chatbots while hiding the identity of the user.

It is a Service Provider's decision to enable the Anonymization feature on its clients via the PRIVACY DISABLE client configuration parameter defined in annex A.1.3.

The Anonymization Function (AF) is the functional element in charge of hiding the identity of the user.

The AF is responsible for the association between a user's URI, a Chatbot URI and a token. The AF manages the repository where those associations are stored. How long the token is valid is up to the Service Provider's policy and the user decision on when to reset the token.

The location of the AF is a deployment decision. The AF can be deployed

- in the Service Provider's network (i.e. MNO AF) or
- as part of the Chatbot Platform (i.e. Chatbot Platform AF).

For Chatbot Service traffic, it is up to the client to request privacy (i.e. Anonymization) or not (see section 3.6.5.1.2.1) when contacting a Chatbot. When privacy is requested, an AF shall be triggered.

NOTE: A Service Provider's policy may as well decide to trigger an AF for each SIP request and corresponding SIP response exchanged between a client and a Chatbot Platform.

For Capability Discovery between a client and a Chatbot, it is up to the Service Provider's policy to decide if the AF shall be triggered. If yes, all requests may be processed as requesting Anonymization.

A client can request to delete a token previously used to communicate with a Chatbot via an extension of the CPM system message. To indicate this, the following IARI is defined and shall be used (see section 3.6.5.1.2.3):

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcsmaap.tkdel"

3.6.5.1.2 Client procedures

3.6.5.1.2.1 Requesting Privacy for a Chatbot conversation

A client can request privacy for a Chatbot conversation (see step 5 in section 3.6.8.3).

3.6.5.1.2.2 Receiving a SIP request from a Chatbot

A client can know if privacy for a Chatbot conversation is enabled or not (see section 3.6.8.4).

3.6.5.1.2.3 Deleting a token

Before deleting a token, the client shall ensure that it has sent all disposition notifications associated with the received messages of the conversation using the token.

When a token associated with a Chatbot needs to be deleted, the client shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the client:

1. shall set the Request-URI and To header fields to the address of the Chatbot;
2. shall include the address of the originating RCS client that has been authenticated as per [GSMA PRD-RCC.11];
3. shall set the P-Preferred-Service header field with the value set to ‘urn:urn-7:3gpp-service.ims.icsi.oma.cpm.systemmsg’;
4. shall include an Accept-Contact header field with the CPM Feature Tag ‘urn:urn-7:3gpp-service.ims.icsi.oma.cpm.systemmsg’;
5. shall add another Accept-Contact header field carrying the IARI tag defined in section 3.6.5.1.1;
6. shall include a User-Agent header field as specified in Annex C.4.1;

The client shall consider the token as deleted only after receiving the corresponding SIP 200 OK to the SIP MESSAGE request as sent above.

When client receives 200 OK to the SIP MESSAGE request and there is an active Chatbot session between client and the Chatbot that is anonymised (i.e. that included the 'tk=on' parameter in the P-Asserted-Identity header of the received SIP INVITE request or SIP 200 OK response), then the client shall close the session by sending in the corresponding SIP dialog(s) a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200 (e.g. SIP;cause=200;text="Call completed").

Upon receipt of a SIP 500 Server Internal Error response, the client shall consider that the token is not deleted and the client may retry to delete the token.

Upon receipt of any other error response, the client shall consider that the token is not deleted and inform the user accordingly.

3.6.5.1.2.4 Linking a token with the User's MSISDN

A client requesting to link or share the MSISDN of the user with a Chatbot shall establish a Chatbot session with the Chatbot, by sending a SIP INVITE following the procedures of section 3.6.8.3.

The client shall consider that the MSISDN is linked only after receiving the corresponding SIP 200 OK to the SIP INVITE as sent above.

Upon receipt of a SIP 500 Server Internal Error response, the client shall consider that the token is not linked and the client may retry to link the token.

Upon receipt of any other error response, the client shall consider that the token is not linked and inform the user accordingly.

NOTE: If the SIP URI of the user is not based on the user's MSISDN then the MSISDN will not be shared, but the real user's identity is shared with the Chatbot.

3.6.5.1.3 MNO AF procedures

3.6.5.1.3.1 Processing SIP requests from clients and SIP responses to clients

Upon receiving a SIP request carrying the Chatbot IARI feature tag defined in section 3.6.2.1 included in the Accept-Contact header field and without the Chatbot role as defined in section 3.6.2.3 included in the Contact header, the MNO AF:

1. if the request is a SIP INVITE or a SIP MESSAGE, shall check if a Privacy header with a value "tk" or "tklink" is part of the received request.

    a) If not, the MNO AF shall forward the request unaltered to the Chatbot Platform and not continue with the following steps;
    b) Otherwise, continue with the following steps:

2. shall retrieve the URIs of the user and the Chatbot respectively from the P-Asserted-Identity and Request-URI headers;
3. shall check if a token is already associated to the (user URI, Chatbot URI) pair in its repository;

    a) If for any reason the AF cannot access its repository, the MNO AF shall respond to the SIP request with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value and shall not proceed with the rest of the steps.
    b) If no token is associated to the (user URI, Chatbot URI) pair,

        i. if a Privacy header with a value "tklink" is part of the received request, the MNO AF shall respond to the SIP INVITE with a SIP 403 Forbidden response with a Warning header with the warning text set to "204 Token Not Found" and the MNO AF shall not proceed with the rest of the steps.

ii. Otherwise, the MNO AF shall create a token as per section 2.5.4.2 and associate it with the (user URI, Chatbot URI) pair; If for any reason, the MNO AF cannot create a token, the MNO AF shall respond to the SIP request with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value and shall not proceed with the rest of the steps;

c) Otherwise the MNO AF shall use the token associated with the pair;

4. shall act as a SIP B2BUA with the following precisions. The MNO AF:

a) if the request is a SIP INVITE or a SIP MESSAGE with a Privacy header set to “tk”

i. shall remove the “tk” value from the Privacy header. If there is no other value in the Privacy header, the AF shall remove the Privacy header.
ii. shall replace the URIs representing the Public User Identity in the From; P-Asserted-Identity and in any other SIP headers carrying the Public User Identity with a URI formatted as defined in section 2.5.4.2 using the token and the domain of the MNO AF, leaving all URI parameters untouched, in the received SIP request.

b) if the request is a SIP INVITE with a Privacy header set to “tklink”

i. shall remove the “tklink” value from the Privacy header. If there is no other value in the Privacy header in the SIP INVITE, the AF shall remove the Privacy header.
ii. shall set in the SIP INVITE another P-Asserted-Identity header field to a URI formatted as defined in section 2.5.4.2 using the token and the domain of the MNO AF.

c) if the request is related to Capability Discovery, i.e. a SIP OPTIONS or SIP SUBSCRIBE request, may according to Service Provider’s policy replace the URIs representing the Public User Identity in the From, P-Asserted-Identity and in any other SIP headers carrying the Public User Identity with a URI formatted as defined in section 2.5.4.2 using the token and the domain of the MNO AF, leaving all URI parameters untouched, in the received SIP request.
d) shall send the SIP request.
e) if the request is a SIP INVITE or a SIP MESSAGE with a Privacy header set to “tk”, shall add an ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘on’ to each of the URIs in the P-Asserted-Identity of the related SIP response(s) sent back on the originating leg;
f) Otherwise (i.e. the Privacy header is set to “tklink”), shall add a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘off’ to each of the URIs in the P-Asserted-Identity of the related SIP response(s) sent back on the originating leg;

Upon receiving a SIP 200 OK to a SIP INVITE with a Privacy header set to “tklink”, the MNO AF:

1. shall invalidate the corresponding token in the repository.

2. If there is an established 1-to-1 Chatbot session between the user and the Chatbot where the MNO AF acts as a B2B UA, the MNO AF shall close the session by sending in the corresponding SIP dialog(s) a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200 (e.g. SIP;cause=200;text="Call completed").
3. shall forward the SIP 200 OK to the requesting client with "tk" parameter set to "off" in the P-Asserted-Identity header.

Upon receiving a SIP request that is part of a SIP B2BUA dialog (e.g. SIP ACK, SIP BYE, SIP NOTIFY), or a SIP CANCEL, the MNO AF shall:

1. replace the URIs representing the Public User Identity in the From, P-Asserted-Identity (if present) and in any other SIP headers carrying the Public User Identity with a URI formatted as defined in section 2.5.4.2 using the token and the domain of the MNO AF, leaving all URI parameters untouched, in the received SIP request;
2. send the SIP request;
3. for all SIP provisional or final responses to be sent back to the client on the originating leg, if the P-Asserted-Identity header is present, the MNO AF shall add a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘on’, to the URI in the P-Asserted-Identity if not already present;

If the SIP transaction is not anonymised, for all SIP provisional or final responses to be sent back to the client on the originating leg, the MNO AF shall add a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘off’, to each of the URIs in the P-Asserted-Identity if not already present.

3.6.5.1.3.2 Processing SIP requests from Chatbots and SIP responses to Chatbots

Upon receiving a SIP request carrying the Chatbot IARI feature tag as defined in section 3.6.2.1 included in the Accept-Contact header field and the Chatbot role as defined in section 3.6.2.3 included in the Contact header, the MNO AF shall act as a SIP B2BUA with the following precisions: the MNO AF

1. shall retrieve the URIs of the user and the Chatbot respectively from the Request-URI and P-Asserted-Identity headers;
2. if the URI of the user is in the format defined in section 2.5.4.2, shall check in its repository if the token that is used in the URI is associated to the Chatbot;

    a) If there is no association, the MNO AF shall return a SIP 404 Not Found error response to the Chatbot Platform;
    b) Otherwise, if there is an association, the AF shall use its repository information associated with the token and the Chatbot URI to retrieve the user’s URI. The MNO AF shall forward the SIP request, replacing in all relevant SIP headers the URI of the user in the format defined in section 2.5.4.2 with the user’s URI and adding a ‘tk’ URI parameter set to ‘on’ to the Chatbot URI(s) provided in the P-Asserted-Identity header if not already present. The MNO AF shall use the URI received in the Request URI as the user’s identifier used in the SIP response returned on the originating leg.

3. if the URI of the user is not in the format defined in section 2.5.4.2, then the MNO AF shall

    a) add a ‘tk’ URI parameter set to ‘off’ to the Chatbot URI(s) contained in the P-Asserted-Identity header if a ‘tk’ URI parameter is not already present;
    b) send the SIP request

Upon receiving a SIP request that is part of a SIP dialog (e.g. SIP ACK, SIP BYE, SIP NOTIFY), or a SIP CANCEL, the MNO AF shall

1. use its repository information associated with the token and the Chatbot URI to retrieve the user’s URI. The MNO AF shall forward the SIP request, replacing in all relevant SIP headers the URI of the user in the format defined in section 2.5.4.2 with the user’s URI and adding a ‘tk’ URI parameter set to ‘on’ to the Chatbot URI(s) provided in the P-Asserted-Identity header if not already present;
2. send the SIP request.
3. remove the P-Asserted-Identity header in the response towards the Chatbot.

3.6.5.1.3.3 Processing a token deletion

Upon receiving a SIP MESSAGE as defined in section 3.6.5.1.2.3, the MNO AF:

1. shall retrieve the URIs of the user and the Chatbot respectively from the P-Asserted-Identity and Request-URI headers;
2. shall check in its repository if a token is associated with the retrieved user and Chatbot URIs;

    a) If there is no association, the MNO AF shall return a SIP 200 OK to the client to the received SIP MESSAGE request and shall not proceed with the rest of the steps
    b) otherwise, if there is such association, the MNO AF shall invalidate the token in the repository and shall return a SIP 200 OK to the client to the received SIP MESSAGE request.
    c) if for any reason, the MNO AF cannot access its repository, the MNO AF shall respond to the SIP MESSAGE with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value.

3. If there is an established 1-to-1 Chatbot session between the user and the Chatbot where the MNO AF acts as a B2B UA, the MNO AF shall close the session by sending in the corresponding SIP dialog(s) a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200 (e.g. SIP;cause=200;text="Call completed").

3.6.5.1.4 Chatbot Platform AF procedures

3.6.5.1.4.1 Processing SIP requests from clients

Upon receiving a SIP request carrying the Chatbot IARI feature tag defined in section 3.6.2.1 included in the Accept-Contact header field and without the Chatbot role as defined in section 3.6.2.3 included in the Contact header, the AF:

1. shall retrieve the URIs of the user and the Chatbot respectively from the P-Asserted-Identity and Request-URI headers;
2. if the request is a SIP INVITE or a SIP MESSAGE with a Privacy header set to “tk”,

    a) if for any reason (e.g. the AF cannot access its repository, or a token cannot be generated) anonymity cannot be performed, the Chatbot Platform AF shall respond to the SIP request with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value and shall not proceed with the rest of the steps.
    b) the Chatbot Platform AF shall check if the (user URI, Chatbot URI) pair is known in its repository;

        i. If the pair is not known, the AF shall create a token and associate it with the (user URI, Chatbot URI) pair;
        ii. otherwise, the AF shall use the token associated with the pair.

    c) shall provide the token to the Chatbot as the user’s identity.
    d) shall send a SIP response according to section 3.6.8.2 with the following clarification:

        i. a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘on’ shall be added to each of the URIs in the P-Asserted-Identity header field.

3. if the request is a SIP INVITE with a Privacy header set to “tklink”,

    a) if for any reason the AF cannot access its repository, the Chatbot Platform AF shall respond to the SIP request with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value and shall not proceed with the rest of the steps.
    b) shall check if the (user URI, Chatbot URI) pair is known in its repository;

        i. If the pair is not known, the Chatbot Platform AF shall respond to the SIP INVITE with a SIP 403 Forbidden with a Warning header with the warning text set to “204 Token Not Found” and shall not proceed with the rest of the steps;
        ii. otherwise, the Chatbot Platform shall

            - indicate to the Chatbot that the user’s URI and the token associated to the pair represent the same user.
              How the Chatbot Platform interacts with the Chatbot for such indication is outside the scope of this specification and
            - invalidate the corresponding token in the repository and
            - If there is an established 1-to-1 Chat session between the user and the Chatbot, close the session by sending to the client a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200 (e.g. SIP;cause=200;text="Call completed").

c) shall send a SIP response according to section 3.6.8.2 with the following clarification:

i. a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘off’ shall be added to each of the URIs in the P-Asserted-Identity header field.

4. if the request is a SIP OPTIONS or SIP SUBSCRIBE, then the Chatbot Platform shall either respond itself, or interact with the Chatbot without providing the user’s URI.
5. Otherwise (i.e. the conditions in steps 2, 3 and 4 do not apply), the AF

a) shall provide the user’s URI(s) to the Chatbot as the user’s identity(ies) and
b) shall send a SIP response according to section 3.6.8.2 with the following clarification:

i. a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘off’ shall be added to each of the URIs in the P-Asserted-Identity header field.

3.6.5.1.4.2 Processing requests from Chatbots

How the Chatbot Platform interacts with the Chatbots and the internal Anonymization Function is up to the implementation.

The AF shall reflect in the P-Asserted-Identity of outgoing SIP request, the identity that was used to interact with the Chatbot:

- If the Chatbot was using a user’s URI, the Chatbot Platform AF shall set the P-Asserted-Identity to the Chatbot URI with a ‘tk’ URI parameter set to ‘off’
- If the Chatbot was using a token, the Chatbot Platform AF shall set the P-Asserted-Identity to the Chatbot URI with a ‘tk’ URI parameter set to ‘on’.

3.6.5.1.4.3 Processing a token deletion

Upon receiving a SIP MESSAGE as defined in section 3.6.5.1.2.3, the Chatbot Platform AF:

1. shall retrieve the URIs of the user and the Chatbot respectively from the P-Asserted-Identity and Request-URI headers;
2. shall check in its repository if a token is associated with the retrieved user and Chatbot URIs;

a) If there is no association, the Chatbot Platform AF shall return a SIP 200 OK to the client to the received SIP MESSAGE request and shall not proceed with the rest of the steps
b) otherwise, if there is such association the AF shall invalidate the token and shall return a SIP 200 OK to the client to the received SIP MESSAGE request.
c) if for any reason, the Chatbot Platform AF cannot access its repository, the Chatbot Platform AF shall respond to the SIP MESSAGE with a SIP 500 Server Internal Error response with optionally a Retry-After header set to a locally configured value and shall not proceed with the rest of the steps.

3. If there is an established 1-to-1 Chat session between the user and the Chatbot where privacy was requested, the AF shall close the session by sending to the client a SIP

BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200 (e.g. SIP;cause=200;text="Call completed").

3.6.6 Spam and other Inappropriate Chatbot Behaviour Handling

Protection against spam, fraud, inappropriate content or other inappropriate behaviour originating from Chatbots can be realised by preventing access from and to such Chatbots

- in the Spam Block function in the Chatbot Platform or
- in the Spam Block function in the Service Provider network or
- in the client.

Where this protection is performed depends mostly on which party considers the Chatbot as behaving inappropriately. The exception would be Service Providers that can make use of their capabilities for managing the client to inform a client of the Chatbots that they consider problematic. This could be used either in addition to preventing the access in the Service Provider network or as an alternative.

Preventing access to a Chatbot in either the Chatbot Platform or the Service Provider network should affect both the regular messaging services and the retrieval of Chatbot Information (see section 3.6.4). In addition, if the Service Provider blocks the traffic, the Service Provider shall ensure that the corresponding Chatbot cannot be discovered through the Service Provider Chatbot Directory function (see section 3.6.3.1).

If access to a Chatbot is prevented in a Spam Block function, either in the Service Provider network or in the Chatbot Platform, this shall lead to the following behaviour:

- A SIP 403 Forbidden response including a Warning header field including the warning text set to "206 Spam Sender" shall be returned to a SIP request initiated by the client.
- A SIP 404 Not Found response shall be returned to a SIP request initiated by the Chatbot Platform (if blocked in the MNO network).

How this behaviour is realised is network internal and is therefore out of scope of this document. However, RCS Clients receiving such error responses shall inform the user accordingly.

A user shall be able to block the reception of incoming Chatbot communication sessions and messages through the client local blacklist as defined in section 18 of [GSMA PRD-RCC.71].

A Service Provider shall be able to prevent access to Chatbots on the client for both incoming and outgoing communication based on the management of blocked Chatbot service identifiers (see section 2.5.4.1) via the Service Provider managed client based spam prevention as described in section 3.6.6.1.

3.6.6.1 Service Provider Managed Client-based Spam Prevention and Prevention of Other Inappropriate Chatbot Behaviour

If the Service Provider wishes to prevent spam, fraud, reception of inappropriate content and other reported inappropriate Chatbot behaviour through the client, the Service Provider

shall do so by defining a list of Blacklisted Chatbots via the procedures defined in section 3.6.3.3. The list of Blacklisted Chatbots shall be identified as BLACKLISTED (see <name of the list> in section 3.6.3.3). The specific marker to identify the list shall therefore be:

```
LIST:BLACKLISTED<CRLF>
<CRLF>
```

For every incoming SIP INVITE or SIP MESSAGE request where the P-Asserted-Identity carries an address that corresponds to a Chatbot Service ID, the client shall return a SIP 486 Busy Here response if that Service ID matches an entry in the Blacklisted Chatbots list.

When performing a Service Provider Chatbot Directory lookup as described in section 3.6.3.1, the client shall not show any entries in the result to the user for which the Service ID matches an entry in the Blacklisted Chatbots list.

When the user tries to access a Chatbot (e.g. entered directly by the user, discovered on a website, continuing a conversation started before the Chatbot's Service ID was included on the list) for which the Service ID matches an entry in the Blacklisted Chatbots list, the client shall show an error message to the user without retrieving the Chatbot Information as described in section 3.6.4.

3.6.6.2 Spam Report Message

3.6.6.2.1 Overview

- The Spam Report Message allows a client to report a Chatbot as a spammer or as a Chatbot involved in fraud, sending inappropriate content, or other inappropriate behaviour.The Spam Report Messages are system messages and as such:they are not sent with CPIM headers, and a delivery and/or displayed notification shall not be requested,
- there is no store and forward.

The Spam Report Messages are conveyed in a SIP MESSAGE request. They are targeted to the Chatbot service identifier that the user is reporting on and shall only be sent by RCS clients.

To categorize them among other system messages, Spam Report Messages shall use the following IARI:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.botspam"

The Service Provider may trigger (based on this IARI) an element in its network to process the Spam Report Message, but in any case, the Chatbot Platform hosting the targeted Chatbot shall receive the Spam Report Message.

As for other SIP requests, the Spam Report Message shall be routed towards the AF (if deployed in the Service Provider network).

The Spam Report XML schema is defined as shown in Table 47 in section 3.2.9.1.1. If the IM-SPAM-REPORTING-URI defined in section A.1.3 is not present the client should fallback to schema defined in Table 44 of [GSMA PRD-RCC.07v13.0].

The schema defined in Table 47 is extended to enable the client to include a spam type and free text where the user can explain the reason for classifying the Chatbot message as inappropriate in some way. This Spam Report Message body schema Extension is defined as shown in Table 48 in section 3.2.9.1.1.

3.6.6.2.2 Client Procedures

3.6.6.2.2.1 Sending a Spam Report Message

When a Spam Report Message directed to a specific Chatbot is sent, the client shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the client:

1. shall set the Request-URI and To header field to the address of the Chatbot;
2. shall include the address of the originating RCS client that has been authenticated as per [GSMA PRD-RCC.11];
3. shall set the P-Preferred-Service header field with the value set to 'urn:urn-7:3gpp-service.ims.icsi.oma.cpm.systemmsg';
4. shall include an Accept-Contact header field with the CPM Feature Tag 'urn:urn-7:3gpp-service.ims.icsi.oma.cpm.systemmsg' and carrying the Spam Report Message IARI tag defined in section 3.6.6.2.1;
5. shall include a User-Agent header field as specified in Annex C.4.1;
6. shall include the Content-Type header field with the value set to the Spam Report Message content type *application/vnd.gsma.rcsspam-report+xml*, as described in section 3.6.6.2.1;
7. shall create a Spam Report Message as described in section 3.6.6.2.1 and set the body of the Spam Report Message, as follows:

    a) The <ReportedIdentity> element set to the address of the Chatbot;
    b) Zero or more, up to 10, <Message-ID> elements set to the value of the imdn.message-ID(s) of the original message(s) that is(are) requested to be reported as spam if any;
    c) Optionally the <spam-type> element set to "spam", "fraud", "inappropriate-content", or "other". If this element is not included it shall be handled as if it had been included with the value "spam";
    d) Optionally the <free-text> element with text of up to 500 octets.

If the client receives in result of processing the request for the Spam Report SIP MESSAGE request:

- any error, then the client shall notify the user that their request has not been processed.

3.6.6.3 Subscription Messages

3.6.6.3.1 Overview

The Subscription Request Message allows a client to subscribe to receive messages from a Chatbot or to unsubscribe from a Chatbot to stop receiving messages. The Chatbot Platform processes Subscription Request Messages and responds with a Subscription

Response Message with the result. Both messages are system messages and as such: they shall not be sent with CPIM headers, and a delivery and/or displayed notification shall not be requested. There is no store and forward for Subscription Request and Response Messages.

Subscription Request and Response Messages are conveyed in a SIP MESSAGE request. Subscription Request Messages are targeted to the Chatbot service identifier that the user is subscribing to or unsubscribing from and shall only be sent by RCS clients.

To categorize them among other system messages, Subscription Request and Response Messages shall use the following IARI:

+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot.subscription"

The Service Provider may trigger (based on this IARI) an element in its network to process the Subscription Request Message, but in any case, the Chatbot Platform hosting the targeted Chatbot shall receive the Subscription Request Message.

The Subscription Request Message and Subscription Response Message XML schema is defined as shown in Table 76.

The associated MIME content type for both messages is:

application/vnd.gsma.rcschatbot-subscription+xml

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:subscription"
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns="urn:gsma:params:xml:ns:rcs:rcs:subscription"
    elementFormDefault="qualified" attributeFormDefault="unqualified">

 <xs:simpleType name="unsubscribeReasonType">
  <xs:union memberTypes="xs:string unsubscribeReasonEnumType"/>
 </xs:simpleType>

 <xs:simpleType name="unsubscribeReasonEnumType">
  <xs:restriction base="xs:string">
   <xs:enumeration value="no_interest"/>
   <xs:enumeration value="not_signed_up"/>
   <xs:enumeration value="too_many_messages"/>
   <xs:enumeration value="other"/>
  </xs:restriction>
 </xs:simpleType>

 <xs:simpleType name="unsubscribeFreeTextType">
  <xs:restriction base="xs:string">
   <xs:maxLength value="500"/>
  </xs:restriction>
 </xs:simpleType>

 <xs:element name="SR">
  <xs:complexType>
   <xs:sequence>
    <xs:element name="Chatbot">
```

```
        <xs:simpleType>
          <xs:restriction base="xs:anyURI"/>
        </xs:simpleType>
      </xs:element>
      <xs:element name="request-type">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="subscribe"/>
            <xs:enumeration value="unsubscribe"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="result" minOccurs="0" maxOccurs="1">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="success"/>
            <xs:enumeration value="failure"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="unsubscribe-reason" type="unsubscribeReasonType" minOccurs="0"/>
      <xs:element name="unsubscribe-free-text" type="unsubscribeFreeTextType" minOccurs="0"/>
      <xs:any namespace="##other" processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
 </xs:element>
</xs:schema>
```

Table 76: Subscription Request Message and Subscription Response Message body schema

3.6.6.3.2 RCS Client: Sending a Subscription Request Message

When a Subscription Request Message is sent, the client shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the client:

1. shall set the Request-URI and To header field to the address of the Chatbot.
2. shall include the address of the originating RCS client that has been authenticated as per [GSMA PRD-RCC.11];
3. shall set the P-Preferred-Service header field with the value set to

   'urn:urn-7:3gppservice.ims.icsi.oma.cpm.systemmsg';

4. shall include an Accept-Contact header field with the CPM Feature Tag 'urn:urn7:3gpp-service.ims.icsi.oma.cpm.systemmsg' and carrying the Subscription Message IARI tag defined in section 3.6.6.3.1;
5. shall include a User-Agent header field as specified in Annex C.4.1;
6. shall include the Content-Type header field with the value set to the Subscription Request Message content type application/vnd.gsma.rcschatbot-subscription+xml, as described in section 3.6.6.3.1;
7. shall create a Subscription Request Message body as described in section 3.6.6.3.1 and set the fields of the Subscription Request Message, body as follows:

   a) The <Chatbot> element set to the Chatbot service identifier of the target Chatbot;

b) Optionally the <request-type> element set to “subscribe” or “unsubscribe”. If this element is not included, it shall be handled as if it had been included with the value “unsubscribe”;
c) If <request-type> is “unsubscribe”, optionally the <unsubscribe-reason> element set “no_interest”, “not_signed_up”, “too_many_messages”, “other” or custom reason text. If the <unsubscribe-reason> element is not included, it shall be handled as if had been included with the value “no_interest” and
d) If <request-type> is “unsubscribe”, optionally the <unsubscribe-free-text> element with text of up to 500 octets when <unsubcribe-reason> is set to “other”

If the client receives in result of processing the request for the Subscription Message SIP MESSAGE request:

- if a SIP 200 OK response is received and a Subscription Response Message (see section 3.6.6.3.3) is received referring to the Chatbot for which it was sent and including a <request-type> element set to the same value as in the Subscription Request Message, the client shall notify the user that their request has been processed.
- In case of an error result, the client shall notify the user that their request has not been processed and shall not expect a Subscription Response Message.

3.6.6.3.3 Chatbot Platform: Handling Subscription Request Messages

A Chatbot Platform receiving a Subscription Request Message as defined in section 3.6.6.3.2 shall:

1. Verify whether the Request URI, To header and the address conveyed in the <Chatbot> element of the Subscription Request Message body match each other.

   a) If they do not, return a SIP 403 Forbidden response and stop further processing of the request.

2. Lookup the Chatbot referred to in the <Chatbot> element of the Subscription Request Message body

   a) If the Chatbot is unknown return a SIP 404 Not Found response and stop further processing of the request

3. If the Chatbot is known, convey the user’s request to the Chatbot and based on its reply generate a Subscription Response Message indicating the result of the Subscription Request Message that can either have been successful or have failed
4. Similarly to the Subscription Request Message, the Chatbot Platform shall send a SIP MESSAGE request according to the rules and procedures of [GSMA PRD-RCC.11] with the clarifications listed here. In this SIP MESSAGE request, the Chatbot Platform handling the Subscription Request Message, shall respond to the RCS client with a response message as follows:

   a) set the Request-URI and To header field to the address of the originating RCS client;
   b) set the P-Preferred-Service header field with the value set to

'urn:urn-7:3gppservice.ims.icsi.oma.cpm.systemmsg';

c) include an Accept-Contact header field with the CPM Feature Tag 'urn:urn7:3gpp-service.ims.icsi.oma.cpm.systemmsg' and carrying the Subscription Message IARI tag defined in section 3.6.6.3.1;
d) include a User-Agent header field as specified in Annex C.4.1;
e) include the Content-Type header field with the value set to the Subscription Request Message content type application/vnd.gsma.rcschatbot-subscription+xml, as described in section 3.6.6.3.1;
f) create a Subscription Response Message as described in section 3.6.6.3.1 and set the fields of the Subscription Response Message, as follows:
g) The <Chatbot> element set to the address of the target Chatbot matching the Subscription Request Message;
h) The <request-type> element set to "subscribe" or "unsubscribe" matching the Subscription Request Message;
i) The <result> element set to the result which shall be "success" if the Subscription request was successful and "failure" if it has failed;

NOTE: A Subscription Response Message is sent irrespective of any message that the Chatbot may itself send in reaction to receiving the Subscription request.

3.6.7 Traffic identification

This section defines the traffic identification CPIM header. Section 3.6.8.7 defines when it is set by the Chatbot Platform and client.

3.6.7.1 Messaging as a Platform CPIM Namespace

A CPIM namespace is defined for Messaging as a Platform (MaaP) related CPIM headers.

As per CPIM [RFC3862], this specification defines a namespace for the CPIM extension header fields defined in the following sections.

The namespace is:

<http://www.gsma.com/rcs/maap>

NOTE: The namespace is considered as a placeholder for a final one to be defined by GSMA or other committees.

As per CPIM [RFC3862] requirements, the header fields defined in the following sections are prepended, in CPIM messages, by a prefix assigned to the URN through the NS header field of the CPIM message.

The remainder of this specification always assumes an NS header field like this one:

NS: maap <http://www.gsma.com/rcs/maap/>

As specified in [RFC5438], clients are free to use any namespace prefix while servers and intermediaries must accept any legal namespace prefix specification.

3.6.7.2 CPIM header Traffic-Type

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
Traffic-Type	0	1

Table 77: Traffic-Type header

The field itself is defined in Augmented Backus–Naur form (ABNF) as follows. By including the Token, the list of traffic-type values may be extended in a later version of this specification:

```
traffic-type = "Traffic-Type:" traffic-type-value CRLF
traffic-type-value = "acknowledgement" | "advertisement" |
"authentication" | "payment" | "premium" | "servicerequest" |
"subscription" | Token
```

An example CPIM header is `maap.Traffic-Type: advertisement`.

3.6.8 Chatbot Service

The Chatbot Service is the service used for the communication between clients and Chatbots. It is based on the RCS Messaging Service.

NOTE: There can be 2 active sessions at a time between a client and a Chatbot: an anonymous session and a non-anonymous session.

When a Chatbot is willing to establish a messaging communication with a client, the Chatbot Platform performs Capability Discovery towards clients to be contacted. If Capability Discovery is disabled in the network which the client registered, the network shall follow the requirements described in section 2.6.1.4.2.

3.6.8.1 Chatbot Platform: Initiating a 1-to-1 Chatbot conversation to a user

When initiating communication to a client, a Chatbot Platform shall select either the procedure defined in section 3.6.8.1.1 or the procedure defined in section 3.6.8.1.2 depending on the Chatbot Communication Services that it supports and the services that the client has indicated support for in the Capability Discovery. If both the client and the Chatbot Platform support both the Chatbot Chat Session and the Chatbot Standalone Messaging technologies, the Chatbot Platform shall initiate a Chatbot conversation to a user based on the Chatbot's use cases. Chatbot Chat Sessions should for example be used if

- The Chatbot “knows” that a conversational exchange is expected
- towards anonymised contacts

3.6.8.1.1 Chatbot Platform: Initiating a 1-to-1 Chatbot Chat Session request to a user

When a request from a Chatbot to initiate a 1-to-1 Chatbot conversation towards a user is to be sent and the Chatbot Chat Session service is selected, the Chatbot Platform shall send a

SIP INVITE request according to section 3.2.3.1 of this specification and to the rules and procedures of section 7.3.1.1 of [GSMA PRD-RCC.11] with the clarifications listed here.

In this SIP INVITE request, the Chatbot Platform:

1. shall include the Chatbot IARI, the Chatbot application version feature tag and the *isbot* feature tag as defined in section 3.6.2, in the Contact header in addition to values already included following section 3.2.3.1;
2. shall in addition to the Accept-Contact header already added, add another Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1 and the Chatbot application version feature tag defined in section 3.6.2.2 and shall include the *require* and *explicit* parameters;
3. shall set the P-Asserted-Service header field with the value of the CPM Feature Tag ‘urn:urn-7:3gpp-service.ims.icsi.oma.cpm.session’;
4. shall set the P-Asserted-Identity header to the service-ids of the Chatbot defined as per section 2.5.4.1;
5. shall, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types the list of the Chatbot-related content types defined in section 3.6.10.2 that the Chatbot Platform is willing to receive, i.e. *application/vnd.gsma.botsuggestion.response.v1.0+json* and *application/vnd.gsma.botsharedclientdata.v1.0+json*.

Upon receipt of a SIP response, section 3.2.3.1 and the rules and procedures of section 7.3.2 of [GSMA PRD-RCC.11] apply. If a 1-to-1 Chatbot Chat session is set up and the terminating network indicates support for revocation as per section 3.2.3.8.2, the Chatbot Platform may inform the Chatbot that it may revoke Chatbot messages if required.

Even though the Chatbot application version feature tag is returned in the Contact header of the SIP INVITE response, the Chatbot Platform shall use the value received via Capability Discovery. If the served user is offline, the Messaging Server may not know the actual version supported by the user’s device.

Non-normative example:

- Contact header of a SIP INVITE request sent from the Chatbot Platform:

```
Contact:<sip:foo.bar@botplatform.domain>;+g.3gpp.icsi-ref="urn%3Aurn-7%3A3gpp-service.ims.icsi.oma.cpm.session";+g.3gpp.iari-ref="urn%3Aurn-7%3A3gpp-application.ims.iari.rcs.chatbot";+g.gsma.rcs.botversion="#=1,#=2";+g.gsma.rcs.isbot
```

3.6.8.1.2 Chatbot Platform: Initiating a 1-to-1 Chatbot Standalone Message request to a user

When a request from a Chatbot to initiate a 1-to-1 Chatbot Conversion towards a user is to be sent, and the Chatbot Standalone Message is selected, the Chatbot Platform shall send a SIP MESSAGE request according to section 3.2.2.1 of this specification and to the rules and procedures of section 7.2.1.1 of [GSMA PRD-RCC.11] with the clarifications listed here.

In this SIP MESSAGE request, the Chatbot Platform:

1. shall include the isbot and Chatbot application version feature tags as defined in section 3.6.2, in the CPIM From header in addition to values already included following section 3.2.2.6;
2. shall in addition to the Accept-Contact header already added, add another Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1 and the Chatbot application version feature tag defined in section 3.6.2.2 and shall include the *require* and *explicit* parameters;
3. shall set the P-Asserted-Service header field with the value of the CPM Feature Tag 'urn:urn-7:3gpp-service.ims.icsi.oma.cpm.msg'
4. shall set the P-Asserted-Identity header to the service-ids of the Chatbot defined as per section 2.5.4.1;

Non-normative example:

- CPIM.From of a SIP MESSAGE request sent from the Chatbot Platform:

```
From:
<sip:anonymous@anonymous.invalid?Contact=+g.gsma.rcs.isbot%3B+g.gsma.
rcs.botversion%3D%22%23%3D1%2C%23%3D2%22>
```

Upon receipt of a SIP response, section 3.2.2.1 and the rules and procedures of section 7.2.2 of [GSMA PRD-RCC.11] apply.

3.6.8.2 Chatbot Platform: Receiving a 1-to-1 Chatbot conversation request from a user

3.6.8.2.1 Chatbot Platform: Receiving a 1-to-1 Chatbot Chat Session request from a user

When handling a request from a user to set up a 1-to-1 Chatbot Chat Session with a Chatbot, section 3.2.3.1 and the rules and procedures of section 7.3.2 of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

In this SIP INVITE response, the Chatbot Platform:

1. shall include the Chatbot IARI, the Chatbot application version feature tag and the isbot feature tag as defined in section 3.6.2, in the Contact header in addition to values already included following section 3.2.3.1.
2. shall set the P-Asserted-Identity header to the service-ids of the Chatbot defined as per section 2.5.4.1
3. shall, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types the list of the Chatbot-related content types defined in section 3.6.10.2 that the Chatbot Platform is willing to receive, i.e. *application/vnd.gsma.botsuggestion.response.v1.0+json* and *application/vnd.gsma.botsharedclientdata.v1.0+json*.

When handling a request from a user to set up a 1-to-1 Chat session with a Chatbot, and a Chatbot IARI is not included in the SIP INVITE request, the Chatbot Platform shall return a SIP 403 Forbidden response code including a warning header set to "488 Chatbot Conversation Needed" to the user.

Since there is no requirement for the Chatbot Platform to provide delivery assurance for messages from a user to the Chatbot, section 3.2.3.8 does not apply and the Chatbot Platform shall not indicate NFS and shall not indicate revocation support in its 200 OK response to the incoming SIP INVITE request.

3.6.8.2.2 Chatbot Platform: Receiving a 1-to-1 Chatbot Standalone Message request from a user

When handling a SIP MESSAGE request with the OMA CPM ICSI "3gpp-service.ims.icsi.oma.cpm.msg" from a user, section 3.2.2.1 and the rules and procedures of section 7.2.2 of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

In this SIP MESSAGE response, the Chatbot Platform:

1. shall set the P-Asserted-Identity header to the service-ids of the Chatbot defined as per section 2.5.4.1

When handling a request from a user to set up a 1-to-1 Standalone Message with a Chatbot without Chatbot IARI in the SIP MESSAGE request:

- If follow-up conversation from the user is expected by the Chatbot, the Chatbot Platform Shall return a SIP 403 Forbidden response code including a warning header set to "488 Chatbot Conversation Needed" to the user.
- If follow-up conversation from the user is not expected by the Chatbot, the Chatbot Platform Shall return a SIP 200 OK response.

NOTE: the client can know the contact is a Chatbot from the P-Asserted-Identity header of SIP 200 OK response.

3.6.8.2.3 Handling incoming requests for a temporarily unavailable Chatbot

A Chatbot Platform may support handling of Chatbots to be temporarily unavailable; the Chatbots can be suspended or temporarily out of service.

When receiving a Capability Discovery from a user to a temporarily unavailable Chatbot, the Chatbot Platform shall return the correct capabilities to the user as per section 3.6.2.4.

When receiving a Chatbot Chat Session initiation request as defined in section 3.6.8.3 from a user to a temporarily unavailable Chatbot, the Chatbot Platform:

1. shall accept the session as per the procedures defined in section 3.6.8.2;
2. shall send a Chatbot specific "out of service" message in the MSRP session;
3. shall close the session by sending a SIP BYE request.

When receiving a Chatbot Standalone Message request as defined in section 3.6.8.3 from a user to a temporarily unavailable Chatbot, the Chatbot Platform:

1. shall send a Chatbot specific "out of service" message in a Pager Mode Standalone Message toward the user using the procedure defined in section 3.6.8.1.2;

3.6.8.2.4 Handling incoming requests for a restricted Chatbot

When a user attempts to contact a Chatbot, and the Chatbot access is restricted for the user (for example, a Chatbot providing services only to users of a specific MNO) the Chatbot Platform shall respond to the SIP request or Capability Discovery request with a SIP 403 Forbidden response with a Warning header field including the warning text set to “205 Chatbot has declined”.

3.6.8.3 Client: Initiating a 1-to-1 Chatbot conversation request to a Chatbot

When initiating communication to a Chatbot, the client shall select the procedure defined in section 3.6.8.3.1, the procedure defined in section 3.6.8.3.2 or SMS as defined in section 3.2.1.2.

3.6.8.3.1 Client: Initiating a 1-to-1 Chatbot Chat Session request to a Chatbot

When a request from a user’s client to initiate a 1-to-1 Chatbot Chat Session towards a Chatbot is to be sent, the client shall send a SIP INVITE request according to section 3.2.3.1 and to the rules and procedures of section 7.3.1.1 of [GSMA PRD-RCC.11] with the clarifications listed here.

In this SIP INVITE request, the client:

1. shall in addition to values already included in the Contact header, include the Chatbot IARI and the Chatbot application version feature tag as defined in section 3.6.2, in the Contact header;
2. shall in addition to the Accept-Contact header already added, add another Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1 and the Chatbot application version feature tag defined in section 3.6.2.2, and shall include the *require* and *explicit* parameters;
3. shall set the Request-URI of the Chat session request to the service-id of the Chatbot defined as per section 2.5.4.1. The Chatbot SIP URI should be used if available instead of the tel URI;
4. shall, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types the list of the Chatbot-related content types defined in 3.6.10.2 that the client is willing to receive, i.e. multipart/mixed, application/vnd.gsma.botsuggestion.v1.0+json, and application/vnd.gsma.botmessage.v1.0+json.
5. shall, when privacy is requested for the conversation, add a privacy header as defined in [RFC3323] if not already included, and add a value “tk” to it.
6. shall, when initiating a session to link the user's token to their actual identity, add a privacy header as defined in [RFC3323] if not included already and add a value “tklink” to it.

Upon receipt of a SIP 200 OK response,

1. the client shall verify whether the Contact header of the SIP 200 OK response contains the Chatbot role feature tag as defined in section 3.6.2.3.
2. If the Chatbot role feature tag is not provided, the client shall not establish the media plane and terminate the SIP session by sending a SIP BYE request to the Chatbot

according to the rules and procedures of section 7.3.4.1 of [GSMA PRD-RCC.11] and the client shall consider that request is not successful and inform the user accordingly.

3. the client shall check the value of the ‘tk’ parameter in the Chatbot URI provided in the P-Asserted-Identity in 200 OK response:

   a) When the Privacy header was included with a value set to tk and value of the ‘tk’ parameter is ‘off’,
   b) When the Privacy header was not included with a value set to tk and value of the ‘tk’ parameter is ‘on’,
   c) When the Privacy header was included with a value set to tklink and value of the ‘tk’ parameter is ‘on’

   the client shall not establish the media plane and terminate the SIP session by sending a SIP BYE request to the Chatbot according to the rules and procedures of section 7.3.4.1 of [GSMA PRD-RCC.11] and the client shall consider that request is not successful and inform the user accordingly.

4. When the request was to link the user's token to their actual identity and there is an anonymous active Chatbot session between client and the linked Chatbot, then the client shall close the active anonymous session by sending in the corresponding SIP dialog a SIP BYE request carrying a Reason header field with the protocol set to SIP and the protocol cause code set to 200.
5. Otherwise, section 3.2.3.1 and the rules and procedures of section 7.3.2 of [GSMA PRD-RCC.11] apply. There shall be no indication of support for delivery assurance by the Chatbot Platform in the SIP response.

If receiving a SIP final response different from SIP 200 OK, the rules and procedures of section 7.3.1.1 of [GSMA PRD-RCC.11] apply

Upon receipt of a SIP 403 Forbidden error response with a Warning header with the warning text set to “204 Token Not Found” to a Chatbot Chat Session initiation requesting to link the user's token to their actual identity, the client shall consider that it is not possible to link an existing token for the user with their MSISDN, and the client shall not automatically retry the token link request. In this case the client shall check if there is an active session with that particular Chatbot; if there is one and it is anonymised, the client shall close that session.

Non-normative example:

- Contact header of a SIP INVITE request sent from the client:

```
Contact:<sip:foo.bar@domain>;+g.3gpp.icsi-ref="urn%3Aurn-7%3A3gpp-
service.ims.icsi.oma.cpm.session";+g.3gpp.iari-ref="urn%3Aurn-
7%3A3gpp-
application.ims.iari.rcs.chatbot";+g.gsma.rcs.botversion="#=1,#=2"
```

3.6.8.3.2 Client: Initiating a 1-to-1 Chatbot Standalone Message request to a Chatbot

When a client needs to send a 1-to-1 Chatbot Standalone Message towards a Chatbot, the client shall send a SIP MESSAGE request according to section 3.2.2.1 of this specification

and to the rules and procedures of section 7.2.1.1 of [GSMA PRD-RCC.11] with the clarifications listed here.

In the SIP MESSAGE request, the client:

1. shall in addition to the Accept-Contact header already added, add another Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1 and the Chatbot application version feature tag defined in section 3.6.2.2, and shall include the require and explicit parameters;
2. shall set the Request-URI of the SIP MESSAGE request to the service-id of the Chatbot defined as per section 2.5.4.1. The Chatbot SIP URI should be used if available instead of the tel URI;

Upon receipt of a SIP 200 OK response,

- Section 3.2.2.1 and the rules and procedures of section 7.2.2 of [GSMA PRD-RCC.11] apply.

If receiving a SIP final response different from SIP 200 OK, the rules and procedures of section 7.2.1.1 of [GSMA PRD-RCC.11] apply.

3.6.8.4 Client: Receiving a 1-to-1 Chatbot conversation request from a Chatbot

3.6.8.4.1 Client: Receiving a 1-to-1 Chatbot Chat Session request from a Chatbot

When receiving a SIP INVITE for a Chat session with an Accept-Contact header field containing the Chatbot IARI feature tag defined in section 3.6.2.1, the client

- shall reject the SIP INVITE request with a SIP 488 Not Acceptable Here response if the client is not authorised to use Chatbot Communication using sessions (see section 3.6.8.9), otherwise
- shall reject the SIP INVITE request with a SIP 606 Not Acceptable response, if the Contact header of the SIP INVITE request does not contain the Chatbot role feature tag as defined in section 3.6.2.3, otherwise
- shall reject the SIP INVITE request with a SIP 606 Not Acceptable response, if privacy is enabled using the PRIVACY DISABLE client configuration parameter defined in section A.1.3 and the P-Asserted-Identity in the SIP INVITE request doesn't contain the 'tk' parameter with a value "on" or "off"; otherwise
- shall apply the rules and procedures of section 3.2.3.1 of this document and of section 7.3.2 of [GSMA PRD-RCC.11] with the clarifications listed below:
    - shall if privacy is enabled using the PRIVACY DISABLE client configuration parameter defined in section A.1.3, check the value of the 'tk' parameter in the Chatbot URI provided in the P-Asserted-Identity in the SIP INVITE request:
        - If the value of the 'tk' parameter is 'off', the client shall assume that the Chatbot is using the user's identity.
        - Otherwise, the client shall assume that a token is used between the Chatbot and the AF.

- When the client returns a SIP 200 OK response, the client

  - shall include in the Contact header of the SIP 200 OK response the Chatbot IARI as defined in section 3.6.2.1 and the Chatbot application version feature tags as defined in section 3.6.2.2
  - shall, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types the list of the Chatbot-related content types defined in 3.6.10.2 that the client is willing to receive, i.e. *multipart/mixed*, *application/vnd.gsma.botsuggestion.v1.0+json*, and *application/vnd.gsma.botmessage.v1.0+json*.
  - shall check if the Chatbot Information needs to be refreshed according to section 3.6.4.

3.6.8.4.2 Client: Receiving a 1-to-1 Chatbot Standalone Message request from a Chatbot

When receiving a SIP MESSAGE with an Accept-Contact header field containing the Chatbot IARI feature tag defined in section 3.6.2.1 and another Accept-Contact header with the CPM Standalone Messaging ICSI, the client

- shall reject the SIP MESSAGE request with a SIP 488 Not Acceptable Here response if the client is not authorised to use Chatbot Communication using Standalone Messaging (see section 3.6.8.9), otherwise
- shall reject the SIP MESSAGE request with a SIP 606 Not Acceptable response, if the CPIM From header of the SIP MESSAGE does not contain the Chatbot role feature tag as defined in section 3.6.2.3, otherwise
- shall reject the SIP MESSAGE request with a SIP 606 Not Acceptable response, if privacy is enabled using the PRIVACY DISABLE client configuration parameter defined in section A.1.3 and the P-Asserted-Identity in the SIP MESSAGE request doesn't contain the 'tk' parameter with a value "on" or "off", otherwise
- shall apply the rules and procedures of section 3.2.2.1 of this document and of section 7.2.2 of [GSMA PRD-RCC.11] with the clarifications listed below:

  - When the client returns a SIP 200 OK response, the client shall check if the Chatbot Information needs to be refreshed according to section 3.6.4.

3.6.8.5 Messaging Server serving a user: handling incoming 1-to-1 conversation requests from a Chatbot

3.6.8.5.1 Messaging Server serving a user: handling incoming 1-to-1 session requests from a Chatbot

When handling an incoming 1-to-1 chat session request with the Chatbot IARI from a Chatbot the rules and procedures of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

The user's Messaging Server

1. shall indicate in the response as per section 3.2.3.8.1 "Network Fallback Support Capability" whether it supports revocation for chat messages received from a Chatbot.

2. shall in addition to values already to be included in the Contact header in the response, include the Chatbot IARI and the Chatbot application version feature tag as defined in section 3.6.2, in the Contact header. The Chatbot application version shall be set to the version supported by the served user if available, otherwise, shall be set to the version supported by the Messaging Server;
3. shall, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types of the response the list of the Chatbot-related content types defined in 3.6.10.2 that the Messaging Server is willing to receive on behalf of the client. This will match the content types associated with the supported Chatbot application version of the served user if available, otherwise, shall be set to the content types supported by the Messaging Server;
4. shall not indicate in the response support for Network Fallback to SMS (NFS) for this session, and shall not perform NFS, even if NFS is supported in general, since the incoming request includes the require and explicit parameters on a dedicated Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1.
5. shall, if none of the P-Asserted-Identities in the incoming SIP INVITE request contains a ‘tk’ URI parameter (as defined in section 2.5.4.3), add a ‘tk’ URI parameter set to ‘off’ to each URI in the P-Asserted-Identity header before propagating the request towards the served user.

3.6.8.5.2 Messaging Server serving a user: handling incoming 1-to-1 Standalone Message requests from a Chatbot

When handling an incoming SIP MESSAGE request with the Chatbot IARI from a Chatbot, the rules and procedures of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

The user’s Messaging Server

1. shall not perform interworking to xMS even if interworking is supported in general, since the incoming request includes the require and explicit parameters on a dedicated Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1.

When it receives MessageRevoke requests from a Chatbot Platform, the Messaging Server shall handle the request and shall send a MessageRevokeResponse request as described in section 3.2.2.5.2;

3.6.8.6 Messaging Server serving a user: handling outgoing 1-to-1 conversation requests to a Chatbot

3.6.8.6.1 Messaging Server serving a user: handling outgoing 1-to-1 session requests to a Chatbot

When handling an outgoing 1-to-1 chat session request with the Chatbot IARI from a user, the rules and procedures of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

The user’s Messaging Server

1. shall indicate in the request as per section 3.2.3.8.1 “Network Fallback Support Capability” whether it supports revocation for chat messages received from a Chatbot.
2. shall not indicate support for Network Fallback to SMS (NFS) for this session, and shall not perform NFS, even if NFS is supported in general, since the outgoing request includes the *require* and *explicit* parameters in a dedicated Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1.
3. if a Privacy header is present and contains the value "tk" then for any received responses that contain one or more P-Asserted-Identity headers add a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘on’ to each URI in the P-Asserted-Identity header, if not already present; Otherwise,
4. if no Privacy header is present or if a Privacy header is present but it does not contain the value "tk" then for any received responses that contain a P-Asserted-Identity header, add a ‘tk’ URI parameter (as defined in section 2.5.4.3) set to ‘off’ to each URI in the P-Asserted-Identity header, if not already present.

There shall be no indication of support for delivery assurance by the Chatbot Platform in the incoming SIP INVITE response.

3.6.8.6.2 Messaging Server serving a user: handling outgoing 1-to-1 Standalone Message requests to a Chatbot

When handling an outgoing 1-to-1 SIP MESSAGE request with the Chatbot IARI from a user, the rules and procedures of [GSMA PRD-RCC.11] apply with no additional requirement.

3.6.8.7 Chat messages exchanged within a 1-to-1 Chat session with a Chatbot

Chat messages within the 1-to-1 chat session with a Chatbot are exchanged according to the rules and procedures of section 8.6 of [GSMA PRD-RCC.11] apply as well as the clarifications listed here.

If the user’s network indicated support for revocation, and if requested by the Chatbot, the Chatbot Platform shall initiate the MessageRevoke procedures as described in section 3.2.3.8.2. The Chatbot Platform shall inform the Chatbot of the result of the message revocation request.

When a Chatbot sends a message of one of the traffic types defined in section 3.6.7.2 the Chatbot Platform shall add the CPIM traffic type header with the corresponding value defined in section 3.6.7.2.

When a client receives a message with any traffic type header value defined in section 3.6.7.2 that includes one or more suggested actions or replies, the client shall add the same CPIM traffic type header and value in the message payload that it sends back to the Chatbot Platform related to these suggested actions and replies (see section 3.6.10.6.2).

The following additional rule applies for the user's Messaging Server when processing CPIM header extensions according to the media plane handling defined in section 2.14.3.2.

If for a 1-to-1 Chatbot session, CPIM header extensions are not supported according to the definitions in section 2.14.3.2, and

1. if the Messaging Server needs to send a message with a "NS" header containing the URI value defined in section 3.6.7.1, then the Messaging Server

   - shall not remove the "NS" header, and
   - shall not remove the CPIM headers containing the associated name prefix,

2. if the Messaging Server needs to send a message with a "NS" header containing the URI value defined in section C.1.13 of [GSMA PRD-RCC.11], then the Messaging Server

   - shall not remove the "NS" header, and
   - shall not remove the CPIM headers containing the associated name prefix.

For payload that is sent from the client to the Chatbot Platform, there are two possible scenarios:

- If there is a 1-to-1 chat session established between the client and the Chatbot Platform, it shall be re-used to convey the payload as defined in section 3.6.10.6.2.
- If there is no 1-to-1 chat session established, the client shall initiate a session based on the procedures described in section 3.6.8.3. Once the session is established, the client shall use it to send the payload as defined in section 3.6.10.6.2.

3.6.8.7.1 File Transfer

When sending a message to relay a File Transfer as defined in section 3.2.5, a Chatbot Platform may use a different domain as the one defined in section 4.2.5 for the URLs related to the file and, if applicable, the thumbnail.

When receiving a message to relay a File Transfer as part of a Chatbot messaging communication, the client shall not verify whether the domain for the file and, if provided, the thumbnail complies with the URL format defined in section 4.2.5.

3.6.8.8 Delivery and Display Notifications

The rules for sending delivery and display notifications shall follow those of section 3.2.2.2 and 3.2.3.1.

In addition, for any disposition notification sent outside of an established MSRP session (i.e. sent by SIP MESSAGE) the following requirements and clarifications apply:

- When sending disposition notifications using a SIP MESSAGE method for a message that was received from or sent to a Chatbot, the client and the Chatbot Platform shall in addition to the Accept-Contact header already added, add another Accept-Contact header field carrying the Chatbot IARI feature tag defined in section 3.6.2.1 and the Chatbot application version feature tag defined in section 3.6.2.2 and shall include the *require* and *explicit* parameters;
- If the disposition notification is related to a message that was received by the client in an anonymous Chatbot Chat Session (i.e. the 'tk' parameter of the Chatbot URI was set to ‘on’ in the P-Asserted-Identity received by the client), the client shall add to the SIP MESSAGE conveying it a Privacy header as defined in [RFC3323] if not already included, and add a value “tk” to it.

3.6.8.9 Client: Authorisation of Chatbot Communication Services

The client is authorised to offer the Chatbot Communication Services to the user if the configuration parameter CHATBOT MSG TECH as defined in section A.1.4 is set to a value different from "0". The client is authorised to use Chatbot Communication using sessions for communication with a Chatbot if the configuration parameter CHATBOT MSG TECH as defined in section A.1.4 is set to "1" or "2" and shall in this case include the capability for Chatbot Communication using sessions in the Capability Discovery. The client is authorised to use Chatbot Communication using Standalone Messaging if the configuration parameter CHATBOT MSG TECH as defined in section A.1.4 is set to "2" or "3" and shall in this case include the capability for Chatbot Communication using Standalone Messaging in the Capability Discovery.

3.6.9 Deferred Messaging

Chat messages to a user from a Chatbot that cannot be immediately delivered shall be deferred according to the rules and procedures of sections 8.3.1.6.1, 8.3.2.9 and 8.2.2.3 of [GSMA PRD-RCC.11] as well as the clarifications listed here. Upon deferral, the Messaging Server:

1. shall ensure that the Accept-Contact header including the Chatbot IARI and the Accept-Contact header including the Chatbot application version and the require and explicit parameters are kept per deferred message,
2. shall ensure that all CPIM headers including the new Chatbot related ones are stored with the Chatbot message; and
3. shall ensure that separate deferred messaging queues are kept for messages and disposition notifications from the Chatbot sent in a SIP MESSAGE request or a session that was anonymised, and for messages and disposition notifications sent in a SIP MESSAGE request or a session that was not anonymised. The 'tk' URI parameter with value other than “off” on the SIP URI in the P-Asserted-Identity header means the session or SIP MESSAGE request was anonymised, and 'tk=off' means the session or SIP MESSAGE request was not anonymised. The 'tk' URI parameter with its value shall be kept with the SIP URI identifying the deferred messaging queue. See also section 3.6.5.1.

Chat messages to a user from a Chatbot that have been deferred shall be delivered according to the rules and procedures of sections 8.3.1.6.2, 8.3.1.6.6, 8.3.2.9 and 8.3.2.9.1.1 of [GSMA PRD-RCC.11] as well as the clarifications listed here. Upon deferred delivery, the Messaging Server:

1. for messages and disposition notifications from one or more Chatbot sessions or SIP MESSAGE requests that were anonymised (i.e. deferred P-Asserted-Identity has a SIP URI with the 'tk' URI parameter set to a value other than “off”)

    a) shall for a session include in the Contact header the Chatbot IARI, the Chatbot application version feature tag and the isbot feature tag as defined in section 3.6.2 set to the value that was deferred with the chat messages;
    b) shall for a message sent in a SIP MESSAGE request include the isbot feature tag in the CPIM From header as defined in section 3.2.2.1;

c) shall add to the Accept-Contact header already added the Chatbot IARI feature tag defined in section 3.6.2.1 and add another Accept-Contact header with the Chatbot application version feature tag defined in section 3.6.2.2 set to the value that was deferred with the chat messages and shall include the *require* and *explicit* parameters;
d) shall set the Referred-By header for a session and the P-Asserted-Identity header for a SIP MESSAGE request to the SIP URI service-id of the Chatbot defined as per section 2.5.4.1 that was deferred in the P-Asserted-Identity header with the chat messages, including the 'tk' URI parameter and its value, and may include the tel URI service-id of the Chatbot if it was also deferred with the chat messages;
e) Shall for a session, in addition to values to be included as per section 3.2.3.1, include in the a=accept-wrapped-types the list of the Chatbot-related content types defined in section 3.6.10.2 that the Chatbot Platform is willing to receive.

And

2. for messages from one or more Chatbot sessions or SIP MESSAGE requests that were not anonymised (i.e. deferred P-Asserted-Identity has a SIP URI with the 'tk=off' URI parameter)

   a) repeat steps 1.a) to 1.e) above in this section.

NOTE: Because this procedure is the same as in earlier versions, a Chatbot Platform will not have to support the special session for the delivery of stored IMDNs described in section 3.2.3.3.

Upon receipt of a SIP response, the rules and procedures of section 8.3.2.9.1.1 of [GSMA PRD-RCC.11] apply for session setup and for delivering the deferred Chat messages.

Example when two values are carried in the Referred-By header:

```
Referred-By:<sip:boty@botplatform.botplatformz.com;tk=on>;add-
refs="tel:+123456789;tk=on"
```

When a deferred session request is received, client shall follow the rules and procedures of sections 7.3.11 of [GSMA PRD-RCC.11] as well as the clarifications listed here. Upon receiving deferred request, the Client:

- shall reject the SIP INVITE request with a SIP 606 Not Acceptable response, if privacy is enabled using the PRIVACY DISABLE client configuration parameter defined in section A.1.3 and the Referred-By header field in the SIP INVITE request doesn't contain the 'tk' parameter with a value "on" or "off"

3.6.10 Rich Cards and Suggested Chip Lists

3.6.10.1 Payloads

Chatbots can send two different types of messages to RCS clients:

- Regular RCS messages using existing content types (e. g. plain text, file transfer, or geo location push), or

- Chatbot messages using the content type as defined in section 3.6.10.2.1, currently only used for sending Rich Cards

NOTE: Due to the inclusion of multiple HTTP links, postback data, etc. in the schema of Rich Cards and Chip lists, the size of the JSON payload may be much larger than for conventional person-to-person Chat. It is suggested that developers, Chatbot Platforms and MNOs assume a maximum size of 250KB for these JSON payloads.

Both message types can either be

- combined with a Suggested Chip List or
- sent on their own without a Suggested Chip List.

Whenever messages are combined with a Suggested Chip List, they shall be sent as a multipart CPIM message where the message shall be sent as the first part and the Suggested Chip List as the second. Only combining messages and Suggested Chip List into a multipart CPIM message is allowed. Chatbot Platforms shall not combine more than one message (e.g. a Rich Card and a plain text message) in a multipart CPIM message. A Suggested Chip List shall not be sent without a message to which it is to be associated.

Whenever messages are sent on their own without a Suggested Chip List they shall not be sent as a multipart CPIM message but as a regular, single CPIM message.

The benefit of sending message and Suggested Chip List in a single, multipart CPIM message is the atomicity. This prevents clients from entering an undefined state where only one of the CPIM messages reaches the client. It also provides a user experience where both message and suggestions are always shown at the same time.

For multipart CPIM messages a Content-Disposition header shall be included for each part of the Multipart message with a value set to "inline" and a filename parameter to allow for correct storage as per [CPM-MSGSTOR-REST]. Each part shall have a unique filename and these shall be added by the Chatbot Platform.

The following table provides an overview of when to use multipart CPIM:

	with Suggested Chip List	without Suggested Chip List
Regular RCS message	multipart CPIM	single CPIM
Chatbot message	multipart CPIM	single CPIM

Table 78: use of multipart bodies

When receiving a non-conformant message (e.g. non-compliant to the JSON or XML schema or a multipart message with inappropriate body types), the client shall acknowledge the reception and send a delivery notification, but shall not display the message to the user.

NOTE: Chatbot Platforms are expected to verify the content that is sent and as such, this case should not occur in practice. This silent discarding is therefore not considered a problem.

NOTE: This behaviour might be used also when receiving non-conformant messages in a regular person-to-person communication.

3.6.10.2 Content Types

3.6.10.2.1 Chatbot Message for Rich Cards

A Chatbot Message containing a Single Rich Card or a Carousel of Rich Cards must be wrapped as a CPIM message in a Chat message with the following content type:

`application/vnd.gsma.botmessage.v1.0+json`

NOTE: Chatbots can also send regular RCS messages as stated in section 3.6.10.1 above.

3.6.10.2.2 Suggested Chip List

A suggestion (suggested replies and suggested actions) must be wrapped as a part of a multipart CPIM message with the following content type:

`application/vnd.gsma.botsuggestion.v1.0+json`

3.6.10.2.3 Client Response to Suggestion

A response from an RCS client based on the suggestions provided must be wrapped as a CPIM message in a chat message with the following content type:

`application/vnd.gsma.botsuggestion.response.v1.0+json`

3.6.10.2.4 Data shared by the client to the Chatbot

As a result of the user interacting with certain suggested actions, clients can share data with Chatbots. This data must be wrapped as a CPIM message in a chat message with the following content type:

`application/vnd.gsma.botsharedclientdata.v1.0+json`

3.6.10.2.5 SDP Content

The `a=accept-wrapped-types` attribute must be included by Chatbot Platforms and clients for the content types defined above for Chatbot Messages, Suggested Chip Lists and responses. The content types to be included by Chatbot Platforms are listed in section 3.6.8.1, and the content types to be included by clients are listed in section 3.6.8.3. These content types are always sent as part of a CPIM message and therefore should not be included in the `a=accept-types` attribute.

Example of the SDP offer/answer for a 1-to-1 Chatbot Chat Session sent from client to Chatbot:

```
Content-Type: application/sdp

v=0
o=- 3688370227 3688370227 IN IP4 47.73.238.12
s=
c=IN IP4 10.100.1.2
```

```
t=0 0
m=message 13500 TCP/MSRP *
a=accept-types:message/cpim application/im-iscomposing+xml
a=path:msrp://10.100.1.2:13500/a7f2d26995f6c206;tcp
a=sendrecv
a=accept-wrapped-types:message/imdn+xml
application/vnd.gsma.rcspushlocation+xml text/plain
application/vnd.gsma.rcs-ft-http+xml multipart/mixed
application/vnd.gsma.botsuggestion.v1.0+json
application/vnd.gsma.botmessage.v1.0+json
a=setup:active
```

Table 79: Chatbot Communication SDP example

3.6.10.2.6 Example of a multipart CPIM message

The following example contains a File Transfer via HTTP XML message, combined with a Chatbot Suggested Chip List:

```
Content-Type: message/cpim

From: <sip:anonymous@anonymous.invalid>
To: <sip:anonymous@anonymous.invalid>
DateTime: 2016-11-17T11:17:08.589Z
NS: imdn <urn:ietf:params:imdn>
NS: cpm <http://www.openmobilealliance.org/cpm/>
cpm.Payload-Type:application/vnd.gsma.rcs-ft-
http+xml;application/vnd.gsma.botsuggestion.v1.0+json
imdn.Message-ID: JOBlDiOnVaK9FpQTeOasX6wGptTjkvD6
imdn.Disposition-Notification: positive-delivery, display

Content-Type: multipart/mixed; boundary="next"

--next
Content-Type: application/vnd.gsma.rcs-ft-http+xml
Content-Disposition: inline; filename=”DSC_379395051.JPG”
Content-Length: [content length]

<?xml version="1.0" encoding="UTF-8"?>
<file>
 <file-info type="thumbnail">
   <file-size>7427</file-size>
   <content-type>image/jpeg</content-type>
   <data
url="https://ftcontentserver.rcs.mnc123.mcc456.pub.3gppnetwork.org/ftsf-
58cdb29d1-a3d3-427c-a8a4-a496759fbf6b" until="2017-04-25T12:17:07Z"/>
 </file-info>
  <file-info type="file">
   <file-size>183524</file-size>
   <file-name>DSC_379395051.JPG</file-name>
   <content-type>image/jpeg</content-type>
```

```
    <data
url="https://ftcontentserver.rcs.mnc123.mcc456.pub.3gppnetwork.org/ftsf-
0d5ea6d1-a94c-2-9634-2d90244d3e8e" until="2017-04-25T12:17:07Z"/>
 </file-info>
</file>
--next
Content-Type: application/vnd.gsma.botsuggestion.v1.0+json
Content-Disposition: inline; filename="Chiplist.lst"
Content-Length: [content length]

{
  "suggestions": [
    {
      "reply": {
        "displayText": "Yes",
        "postback": {
          "data": "set_by_chatbot_reply_yes"
        }
      }
    },
    {
      "reply": {
        "displayText": "No",
        "postback": {
          "data": "set_by_chatbot_reply_no"
        }
      }
    },
    {
      "action": {
        "urlAction": {
          "openUrl": {
            "url": "https://www.google.com",
            "application": "browser"
          }
        },
        "displayText": "Browser or app deep link",
        "postback": {
          "data": "set_by_chatbot_open_url_browser"
        }
      }
    },
    {
      "action": {
        "urlAction": {
          "openUrl": {
            "url": "https://www.dotgo.com",
            "application": "webview"
          }
        },
        "displayText": "Webview or app deep link",
        "postback": {
```

```
            "data": "set_by_chatbot_open_url_webview"
          }
        }
      },
      {
        "action": {
          "urlAction": {
            "openUrl": {
              "url": "https://botstore.info",
              "application": "webview",
              "ignore_app_deep_link": "true"
            }
          },
          "displayText": "Strictly Webview",
          "postback": {
            "data": "set_by_chatbot_open_url_strictly_webview"
          }
        }
      },
      {
        "action": {
          "dialerAction": {
            "dialPhoneNumber": {
              "phoneNumber": "+1650253000"
            }
          },
          "displayText": "Call a phone number",
          "postback": {
            "data": "set_by_chatbot_open_dialer"
          }
        }
      }
    ]
}
--next--
```

Table 80: Chatbot Communication multipart CPIM message

3.6.10.2.7 Text Encoding in JSON payload

All fields within JSON payloads containing text should be encoded in UTF-8.

3.6.10.2.8 Date/Time Format in JSON payload

All date/time fields within JSON payloads should be specified as defined in ISO 8601, including all three fields, date, time, and time zone offset.

3.6.10.3 Data structure overview

Figure 20 shows a high-level overview of the data structure for all JSON payloads, each containing one of:

- message, which is a Chatbot Message sent from Chatbot Platform to client,
- suggestions, which is a Suggested Chip List sent from Chatbot Platform to client, or

- response, which is a response sent from client to Chatbot Platform,
- sharedData, which is Shared Client Data sent from client to Chatbot Platform.

NOTE: This overview does not contain all fields and possible sub-objects. See the JSON schema in section 3.6.10.4 for more details.

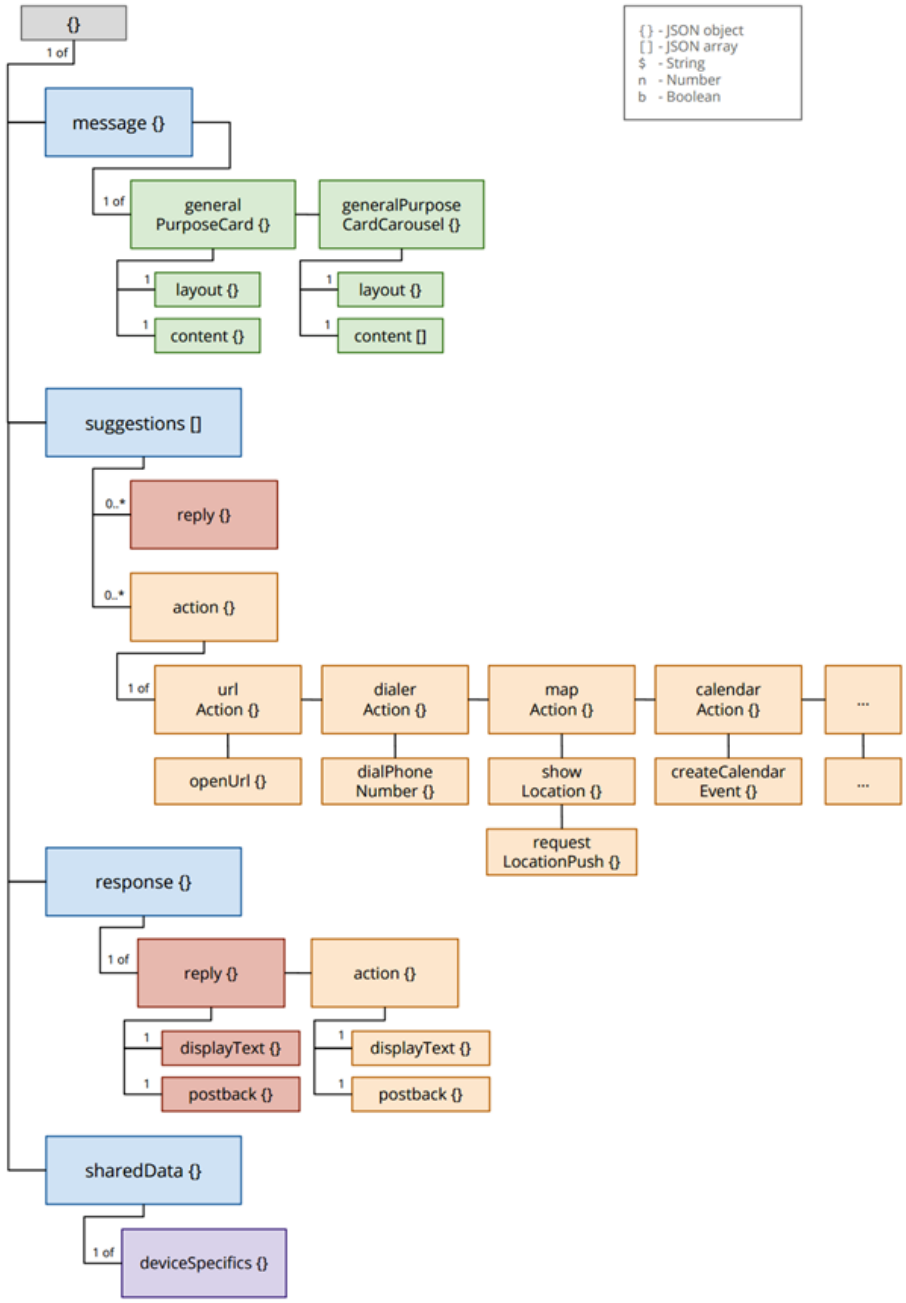


Figure 20: JSON message payloads data structure overview

3.6.10.4 JSON schema for validation

The following schema defines all JSON payloads exchanged between Chatbot Platform and clients:

```
{
 "$schema": "http://json-schema.org/draft-04/schema#",
  "id": "http://www.gsma.com/rcs/chatbot_message_schema.json",

 "title": "Root Object",
 "type": "object",

 "properties": {

  "message": {
    "title": "A Chatbot Message, sent from Chatbot Platform to client.",
    "type": "object",
    "oneOf": [{
      "$ref": "#/definitions/messages/generalPurposeCardMessage"
    }, {
      "$ref": "#/definitions/messages/generalPurposeCardCarouselMessage"
    }]
  },

  "suggestions": {
    "title": "Suggested replies and/or suggested actions for a Chatbot Message, send from Chatbot Platform to client.",
    "type": "array",
    "items": {
      "oneOf": [{
        "$ref": "#/definitions/suggestions/replies/reply"
      }, {
        "$ref": "#/definitions/suggestions/actions/action"
      }]
    },
    "minItems": 1,
    "maxItems": 11,
    "additionalItems": false
  },

  "response": {
    "title": "Response to a suggested reply or suggested action, sent from RCS client to Chatbot Platform.",
    "type": "object",
    "oneOf": [{
       "properties": {
        "reply": {
          "$ref": "#/definitions/suggestions/suggestion"
        }
       },
       "required": ["reply"]
      },
      {
       "properties": {
        "action": {
          "$ref": "#/definitions/suggestions/suggestion"
        }
       },
       "required": ["action"]
      }
    ]
  },
```

```
    "sharedData": {
      "title": "Data shared by the client with the Chatbot Platform (e. g. device specifics).",
      "type": "object",
      "oneOf": [{
        "properties": {
          "deviceSpecifics": {
            "$ref": "#/definitions/sharedData/deviceSpecifics"
          }
        }
      }]
    }

  },
  "oneOf": [{
      "required": ["message"]
    },
    {
      "required": ["suggestions"]
    },
    {
      "required": ["response"]
    },
    {
      "required": ["sharedData"]
    }
  ],

  "definitions": {

    "messageFragments": {
      "cardMedia": {
        "type": "object",
        "properties": {
          "mediaUrl": {
            "type": "string",
            "format": "uri"
          },
          "mediaContentType": {
            "type": "string"
          },
          "mediaFileSize": {
            "title": "Media file size in bytes",
            "type": "integer",
            "minimum": 0
          },
          "thumbnailUrl": {
            "type": "string",
            "format": "uri"
          },
          "thumbnailContentType": {
            "type": "string"
          },
          "thumbnailFileSize": {
            "title": "Thumbnail file size in bytes",
            "type": "integer",
            "minimum": 0
          },
```

```
        "height": {
          "type": "string",
          "enum": ["SHORT_HEIGHT", "MEDIUM_HEIGHT", "TALL_HEIGHT"]
        },
        "contentDescription": {
          "title": "Optional textual description of media content",
          "description": "Accessiblity text for use with screen readers. Will not be shown on screen.",
          "type": "string",
          "minLength": 1,
          "maxLength": 200
        },
        "postback": {
          "$ref": "#/definitions/suggestionFragments/postback",
          "description": "Postback to be sent when media is played. Not relevant for static media e.g. images",
        }
      },
      "required": ["mediaUrl", "mediaContentType", "mediaFileSize", "height"],
      "dependencies": {
        "thumbnailUrl": ["thumbnailContentType", "thumbnailFileSize"]
      }
    },
    "cardTitle": {
      "type": "string",
      "minLength": 1,
      "maxLength": 200
    },
    "cardDescription": {
      "type": "string",
      "minLength": 1,
      "maxLength": 2000
    },
    "fontStyle": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": ["italics", "bold", "underline"]
      },
      "minItems": 1,
      "maxItems": 3,
      "additionalItems": false
    },
    "cardStyle": {
     "title": "A reference to a CSS for the rendering of the Rich Card(s). To be used as defined in 3.6.10.5.4",
     "type": "string",
    "format": "uri"
    }
  },

  "messages": {
    "generalPurposeCardMessage": {
      "title": "This defines a general-purpose, standalone Rich Card message.",
      "type": "object",
      "properties": {
        "generalPurposeCard": {
          "type": "object",
          "properties": {
            "layout": {
```

```
        "type": "object",
        "oneOf": [{
          "properties": {
            "cardOrientation": {
              "type": "string",
              "enum": ["VERTICAL"]
            },
            "titleFontStyle": {
              "$ref": "#/definitions/messageFragments/fontStyle"
            },
            "descriptionFontStyle": {
              "$ref": "#/definitions/messageFragments/fontStyle"
            },
            "style": {
              "$ref": "#/definitions/messageFragments/cardStyle"
            }
          },
          "required": ["cardOrientation"]
        },
        {
          "properties": {
            "cardOrientation": {
              "type": "string",
              "enum": ["HORIZONTAL"]
            },
            "imageAlignment": {
              "type": "string",
              "enum": ["LEFT", "RIGHT"]
            },
            "titleFontStyle": {
              "$ref": "#/definitions/messageFragments/fontStyle"
            },
            "descriptionFontStyle": {
              "$ref": "#/definitions/messageFragments/fontStyle"
            },
            "style": {
              "$ref": "#/definitions/messageFragments/cardStyle"
            }
          },
          "required": ["cardOrientation", "imageAlignment"]
        }
        ]
      },
      "content": {
        "type": "object",
        "properties": {
          "media": {
            "$ref": "#/definitions/messageFragments/cardMedia"
          },
          "title": {
            "$ref": "#/definitions/messageFragments/cardTitle"
          },
          "description": {
            "$ref": "#/definitions/messageFragments/cardDescription"
          },
          "suggestions": {
            "type": "array",
```

```
            "items": {
              "oneOf": [{
                "$ref": "#/definitions/suggestions/replies/reply"
              }, {
                "$ref": "#/definitions/suggestions/actions/action"
              }]
            },
            "minItems": 1,
            "maxItems": 4,
            "additionalItems": false
          }
        },
        "anyOf": [{
          "required": ["media"]
        },
        {
          "required": ["title"]
        },
        {
          "required": ["description"]
        }
        ]
      }
    },
    "required": ["layout", "content"]
  }
 },
 "required": ["generalPurposeCard"]
},
"generalPurposeCardCarouselMessage": {
 "title": "This defines a message containing a carousel of general-purpose Rich Cards.",
 "type": "object",
 "properties": {
  "generalPurposeCardCarousel": {
   "type": "object",
   "properties": {
    "layout": {
     "type": "object",
     "properties": {
      "cardWidth": {
       "type": "string",
       "enum": ["SMALL_WIDTH", "MEDIUM_WIDTH"],
       "default": "SMALL_WIDTH"
      },
      "titleFontStyle": {
       "$ref": "#/definitions/messageFragments/fontStyle"
      },
      "descriptionFontStyle": {
       "$ref": "#/definitions/messageFragments/fontStyle"
      },
      "style": {
       "$ref": "#/definitions/messageFragments/cardStyle"
      }
     },
     "required": ["cardWidth"]
    },
    "content": {
```

```
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "media": {
                  "$ref": "#/definitions/messageFragments/cardMedia"
                },
                "title": {
                  "$ref": "#/definitions/messageFragments/cardTitle"
                },
                "description": {
                  "$ref": "#/definitions/messageFragments/cardDescription"
                },
                "suggestions": {
                  "type": "array",
                  "items": {
                    "oneOf": [{
                      "$ref": "#/definitions/suggestions/replies/reply"
                    }, {
                      "$ref": "#/definitions/suggestions/actions/action"
                    }]
                  },
                  "minItems": 1,
                  "maxItems": 4,
                  "additionalItems": false
                }
              },
              "anyOf": [{
                  "required": ["media"]
                },
                {
                  "required": ["title"]
                },
                {
                  "required": ["description"]
                }
              ]
            },
            "minItems": 2,
            "maxItems": 12,
            "additionalItems": false
          }
        },
        "required": ["layout", "content"]
      }
    },
    "required": ["generalPurposeCardCarousel"]
  }
},

"suggestionFragments": {
  "postback": {
    "title": "Definition of data to be posted back from UE to Chatbot.",
    "type": "object",
    "properties": {
      "data": {
        "type": "string",
```

```
        "maxLength": 2048
       }
      },
      "required": ["data"]
    }
  },

  "suggestions": {
    "suggestion": {
      "title": "Common base definition for suggested replies and suggested actions.",
      "type": "object",
      "properties": {
        "displayText": {
          "type": "string",
          "minLength": 1,
          "maxLength": 25
        },
        "postback": {
          "$ref": "#/definitions/suggestionFragments/postback"
        }
      },
      "required": ["displayText"]
    },
    "replies": {
      "reply": {
        "title": "Definition of a suggested reply.",
        "type": "object",
        "properties": {
          "reply": {
            "allOf": [{
              "$ref": "#/definitions/suggestions/suggestion"
            }]
          }
        },
        "required": ["reply"]
      }
    },
    "actions": {
      "action": {
        "title": "Common base definition of a suggested action.",
        "type": "object",
        "properties": {
          "action": {
            "type": "object",
            "allOf": [{
              "$ref": "#/definitions/suggestions/suggestion"
            }, {
              "oneOf": [{
                "$ref": "#/definitions/suggestions/actions/urlAction"
              }, {
                "$ref": "#/definitions/suggestions/actions/dialerAction"
              }, {
                "$ref": "#/definitions/suggestions/actions/mapAction"
              }, {
                "$ref": "#/definitions/suggestions/actions/calendarAction"
              }, {
                "$ref": "#/definitions/suggestions/actions/composeAction"
```

```
            }, {
              "$ref": "#/definitions/suggestions/actions/deviceAction"
            }, {
              "$ref": "#/definitions/suggestions/actions/settingsAction"
            },{
              "$ref": "#/definitions/suggestions/actions/copyAction"
            }]
          }]
        }
      },
      "required": ["action"]
    },
    "urlAction": {
      "title": "Suggested actions to interact a website or deep app link.",
      "properties": {
        "urlAction": {
          "type": "object",
          "oneOf": [{
            "properties": {
              "openUrl": {
                "type": "object",
                "oneOf": [{
                  "properties": {
                    "url": {
                      "type": "string",
                      "format": "uri"
                    },
                    "application": {
                      "type": "string",
                      "enum": ["browser"]
                    },
                    "ignore_app_deep_link": {
                     "type": "boolean"
                     }
                  },
                  "required": ["url","application"]
                },
                {
                  "properties": {
                    "url": {
                      "type": "string",
                      "format": "uri"
                    },
                    "application": {
                      "type": "string",
                      "enum": ["webview"]
                    },
                    "viewMode": {
                      "type": "string",
                      "enum": ["full", "half", "tall"]
                    },
                    "parameters": {
                      "type": "string",
                      "minLength": 1,
                      "maxLength": 200
                    },
                    "ignore_app_deep_link": {
```

```
                    "type": "boolean"
                   }
                  },
                  "required": ["url","application"]
                 }
                ]
               }
              },
              "required": ["openUrl"]
             }]
            }
           },
           "required": ["urlAction"]
          },
          "dialerAction": {
           "title": "Suggested actions for interacting with a phone number.",
           "properties": {
            "dialerAction": {
             "type": "object",
             "oneOf": [{
              "properties": {
               "dialPhoneNumber": {
                "type": "object",
                "properties": {
                 "phoneNumber": {
                  "type": "string"
                 },
                 "fallbackUrl": {
                  "type": "string",
                  "format": "uri"
                 }
                },
                "required": ["phoneNumber"]
               }
              },
              "required": ["dialPhoneNumber"]
             }, {
              "properties": {
               "dialEnrichedCall": {
                "type": "object",
                "properties": {
                 "phoneNumber": {
                  "type": "string"
                 },
                 "subject": {
                  "type": "string",
                  "maxLength": 60
                 },
                 "fallbackUrl": {
                  "type": "string",
                  "format": "uri"
                 }
                },
                "required": ["phoneNumber"]
               }
              },
              "required": ["dialEnrichedCall"]
```

```
        }, {
          "properties": {
           "dialVideoCall": {
            "type": "object",
            "properties": {
             "phoneNumber": {
              "type": "string"
             },
             "fallbackUrl": {
              "type": "string",
              "format": "uri"
             }
            },
            "required": ["phoneNumber"]
           }
          },
          "required": ["dialVideoCall"]
         }]
        }
       },
       "required": ["dialerAction"]
      },
      "mapAction": {
       "title": "Suggested actions for interacting with a location on a map.",
       "properties": {
        "mapAction": {
         "type": "object",
         "oneOf": [{
           "properties": {
            "showLocation": {
             "title": "Shows a given location on a map.",
             "type": "object",
             "properties": {
              "location": {
               "type": "object",
               "properties": {
                "latitude": {
                 "type": "number"
                },
                "longitude": {
                 "type": "number"
                },
                "crs":{
                 "type": "string"
                },
                "label": {
                 "type": "string",
                 "minLength": 1,
                 "maxLength": 100
                },
                "query": {
                 "title": "Search for location(s) by query",
                  "description": "Search is based on user's current location",
                  "examples": [
                   "restaurants",
                   "GSMA Head Office, 25 Walbrook, London, UK"
                  ],
```

```
                    "type": "string",
                    "minLength": 1,
                    "maxLength": 200
                   }
                  },
                  "oneOf": [{
                    "required": ["latitude", "longitude"]
                   },
                   {
                    "required": ["query"]
                   }
                  ]
                 },
                 "fallbackUrl": {
                  "type": "string",
                  "format": "uri"
                 }
                },
                "required": ["location"]
               }
              },
              "required": ["showLocation"]
             },
             {
              "properties": {
               "requestLocationPush": {
                "title": "One-time request to send a geo location push from UE to Chatbot",
                "type": "object"
               }
              },
              "required": ["requestLocationPush"]
             }
            ]
           }
          },
          "required": ["mapAction"]
         },
         "calendarAction": {
          "title": "Suggested actions for interacting with a calendar event.",
          "properties": {
           "calendarAction": {
            "type": "object",
            "oneOf": [{
             "properties": {
              "createCalendarEvent": {
               "type": "object",
               "properties": {
                "startTime": {
                 "type": "string",
                 "format": "date-time"
                },
                "endTime": {
                 "type": "string",
                 "format": "date-time"
                },
                "title": {
                 "type": "string",
```

```
                "minLength": 1,
                "maxLength": 100
              },
              "description": {
                "type": "string",
                "minLength": 1,
                "maxLength": 500
              },
              "fallbackUrl": {
                "type": "string",
                "format": "uri"
              }
            },
            "required": ["startTime", "endTime", "title"]
          }
        },
        "required": ["createCalendarEvent"]
      }]
    }
  },
  "required": ["calendarAction"]
},
"composeAction": {
  "title": "Suggested actions for composing draft messages.",
  "properties": {
    "composeAction": {
      "type": "object",
      "oneOf": [{
        "properties": {
          "composeTextMessage": {
            "title": "Compose a draft text message.",
            "type": "object",
            "properties": {
              "phoneNumber": {
                "type": "string"
              },
              "text": {
                "type": "string",
                "maxLength": 100
              }
            },
            "required": ["phoneNumber", "text"]
          }
        },
        "required": ["composeTextMessage"]
      },
      {
        "properties": {
          "composeRecordingMessage": {
            "title": "Compose a draft message with a media recording.",
            "type": "object",
            "properties": {
              "phoneNumber": {
                "type": "string"
              },
              "type": {
                "type": "string",
```

```
                    "enum": ["AUDIO", "VIDEO"]
                   }
                  },
                   "required": ["phoneNumber", "type"]
                 }
                },
                "required": ["composeRecordingMessage"]
               }
              ]
             }
            },
            "required": ["composeAction"]
           },
           "deviceAction": {
            "title": "Suggested actions for interacting with the user's device.",
            "properties": {
             "deviceAction": {
              "type": "object",
              "oneOf": [{
                 "properties": {
                  "requestDeviceSpecifics": {
                   "title": "Request specifics about the user's device.",
                   "type": "object"
                  }
                 },
                 "required": ["requestDeviceSpecifics"]
                }
               ]
             }
            },
            "required": ["deviceAction"]
           },
           "settingsAction": {
            "title": "Suggested actions for interacting with app settings",
            "properties": {
             "settingsAction": {
              "type": "object",
              "oneOf": [{
                 "properties": {
                  "disableAnonymization": {
                   "title": "Ask the user to disable the anonymization setting.",
                   "type": "object"
                  }
                 },
                 "required": ["disableAnonymization"]
                },
                {
                 "properties": {
                  "enableDisplayedNotifications": {
                   "title": "Ask the user to enable sending displayed notifications.",
                   "type": "object"
                  }
                 },
                 "required": ["enableDisplayedNotifications"]
                }
               ]
             }
```

```
      },
      "required": ["settingsAction"]
    },
    "copyAction": {
      "title": "Suggested actions for copying to the clipboard.",
      "properties": {
        "copyAction": {
          "type": "object",
          "properties": {
            "copyText": {
              "title": "Copy text to the clipboard.",
              "type": "object",
              "properties": {
                "text": {
                  "type": "string",
                  "maxLength": 512
                }
              },
              "required": ["text"]
            }
          },
          "required": ["copyText"]
        }
      },
      "required": ["copyAction"]
    }
  }
},

"sharedData": {
  "deviceSpecifics": {
    "title": "Device specifics shared by the client with the Chatbot Platform.",
    "type": "object",
    "properties": {
      "deviceModel": {
        "title": "Short description of the device model.",
        "type": "string",
        "minLength": 1,
        "maxLength": 10
      },
      "platformVersion": {
        "title": "Version information about the operating system on the device.",
        "type": "string",
        "minLength": 1,
        "maxLength": 20
      },
      "clientVendor": {
        "title": "Short code for client vendor, same as used during RCS Auto-Configuration.",
        "type": "string",
        "minLength": 1,
        "maxLength": 4
      },
      "clientVersion": {
        "title": "Version information about the client, same as used during RCS Auto-Configuration ",
        "type": "string",
        "minLength": 1,
        "maxLength": 15
```

```
        },
        "batteryRemainingMinutes": {
          "title": "Remaining battery use of device in minutes",
          "type": "integer",
          "minimum": 0
        }
      }
    }
  }
 }
}
```

Table 81: JSON message payloads schema

3.6.10.5 Types of Rich Cards

This specification supports two different kinds of Chatbot Messages which are sent from Chatbot Platforms to clients:

- Single Rich Card
- Carousel of Rich Cards

NOTE: Chatbots can also send regular RCS messages as stated in section 3.6.10.1.

The following sections describe the two different kinds of Chatbot Messages.

3.6.10.5.1 Single Rich Card

3.6.10.5.1.1 JSON data format diagram

NOTE: This overview doesn't contain all fields and possible sub-objects. See the JSON schema in section 3.6.10.4 for more details.

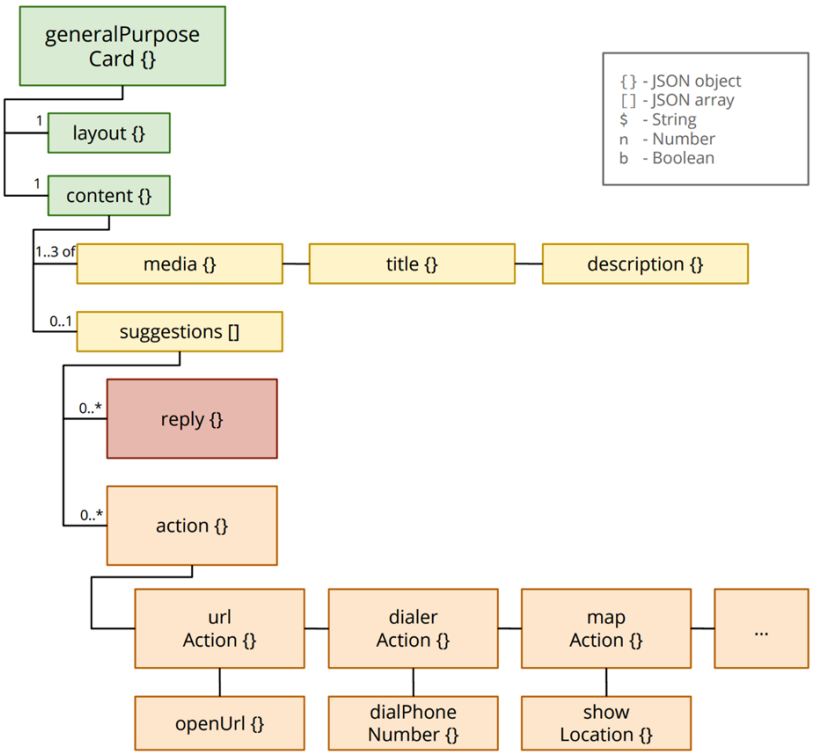


Figure 21: JSON Single Rich Card data structure overview

3.6.10.5.1.2 Example JSON payload

The following payload is an example for a Single Rich Card including suggested replies and suggested actions:

```
{
 "message": {
  "generalPurposeCard": {
   "layout": {
    "cardOrientation": "HORIZONTAL",
    "imageAlignment": "LEFT",
    "titleFontStyle": ["underline","bold"],
    "descriptionFontStyle": ["italics"],
    "style": "http://example.com/default.css"
   },
   "content": {
    "media": {
     "mediaUrl": "https://cdn.server/path/media.mp4",
     "mediaContentType": "video/mp4",
     "mediaFileSize": 2718288,
     "thumbnailUrl": "https://cdn.server/path/media.png",
     "thumbnailContentType": "image/png",
     "thumbnailFileSize": 314159,
     "height": "MEDIUM_HEIGHT",
     "contentDescription": "Textual description of media content, e. g. for use with screen readers.",
      "postback": {
       "data": "set_by_chatbot_video_played"
      }
    },
    "title": "This is a single rich card.",
```

```
      "description": "This is the description of the rich card. It's the first field that will be truncated if it exceeds the maximum width or height of a card.",
      "suggestions": [{
        "reply": {
         "displayText": "No",
         "postback": {
          "data": "set_by_chatbot_reply_no"
         }
        }
       },
       {
        "action": {
         "urlAction": {
          "openUrl": {
           "url": "https://www.google.com",
           "application": "webview",
           "viewMode": "half",
           "ignore_app_deep_link": "false”
          }
         },
         "displayText": "Website or app deep link",
         "postback": {
          "data": "set_by_chatbot_open_url_webview"
         }
        }
       },
       {
        "action": {
         "dialerAction": {
          "dialPhoneNumber": {
           "phoneNumber": "+1650253000"
          }
         },
         "displayText": "Call a phone number",
         "postback": {
          "data": "set_by_chatbot_open_dialer"
         }
        }
       }
      ]
    }
   }
  }
}
```


Table 82: Chatbot communication Single Rich Card example

3.6.10.5.2 Carousel of Rich Cards

3.6.10.5.2.1 JSON data format diagram

NOTE: This overview doesn't contain all fields and possible sub-objects. See the JSON schema in section 3.6.10.4 above for more details.

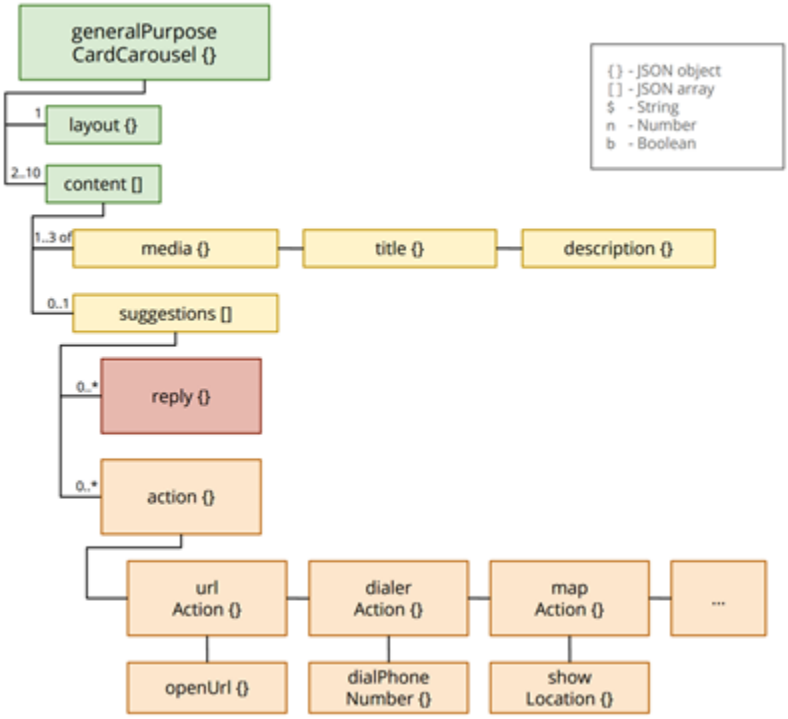


Figure 22: JSON Carousel of Rich Cards data structure overview

3.6.10.5.2.2 Example JSON payload

Two or more (up to twelve) cards can be combined into a carousel. The following payload is an example for a carousel of cards including suggested replies and suggested actions:

```
{
 "message": {
  "generalPurposeCardCarousel": {
   "layout": {
    "cardWidth": "MEDIUM_WIDTH",
    "titleFontStyle": ["underline"],
    "style": "http://example.com/default.css"
   },
   "content": [{
     "media": {
      "mediaUrl": "https://cdn.server/path/media.mp4",
      "mediaContentType": "video/mp4",
      "mediaFileSize": 2718288,
      "thumbnailUrl": "https://cdn.server/path/media.png",
      "thumbnailContentType": "image/png",
      "thumbnailFileSize": 314159,
      "height": "SHORT_HEIGHT",
      "contentDescription": "Textual description of media content, e. g. for use with screen readers.",
      "postback": {
       "data": "set_by_chatbot_video_played"
      }
     },
     "title": "This is the first rich card in a carousel.",
     "description": "This is the description of the rich card. It's the first field that will be truncated if it exceeds the maximum width or height of a card.",
     "suggestions": [{
```

```
          "action": {
           "mapAction": {
            "showLocation": {
             "location": {
              "latitude": 37.4220041,
              "longitude": -122.0862515,
              "crs": "wgs84",
              "label": "Googleplex"
             },
             "fallbackUrl": "https://www.google.com/maps/@37.4219162,-122.078063,15z"
            }
           },
           "displayText": "Show location on a map",
           "postback": {
            "data": "set_by_chatbot_open_map"
           }
          }
         },
         {
          "action": {
           "calendarAction": {
            "createCalendarEvent": {
             "startTime": "2017-03-14T00:00:00Z",
             "endTime": "2017-03-14T23:59:59Z",
             "title": "Meeting",
             "description": "GSG review meeting"
            }
           },
           "displayText": "Schedule Meeting",
           "postback": {
            "data": "set_by_chatbot_create_calendar_event"
           }
          }
         }
        ]
       },
       {
        "title": "This is the second rich card in the carousel.",
        "description": "Carousel cards need to specify a card width in the 'layout' section. For small width cards, only short and medium height media are supported.",
        "[...]": "[...]"
       }
      ]
     }
    }
}
```


Table 83: Chatbot Communication Carousel Rich card Example

3.6.10.5.3 Client Processing of Card Media of Rich Cards

If the RCS client receives a Chatbot message containing Media objects, the RCS client shall determine if the media is a downloadable file or an embedded stream based on the mediaContentType and the presence of mediaFileSize.

3.6.10.5.3.1 Downloading Rich Card Media

If mediaFileSize is present and the mediaContentType represents a downloadable format (e.g. video/mp4), the RCS client shall download the file and thumbnail included in a media object depending on the auto acceptance conditions.

If auto acceptance applies for the media object, then the receiving RCS client shall download automatically the thumbnail, if available, and also the file content.

If manual acceptance applies for the media object, then the receiving RCS client shall download automatically the thumbnail, if available. The file shall be downloaded only after user acceptance.

NOTE: For the determination of the mode of acceptance refer to the corresponding service documentation, e.g. [GSMA PRD-RCC.71].

The RCS client shall download the files and thumbnails using the values of the mediaUrl and thumbnailUrl properties of a Media object via the file download procedure defined in section 4.2.3. The client shall not verify whether the domain for the file and, if provided, the thumbnail complies with the URL format defined in section 4.2.5.

For the initiation of the file download procedure, the RCS client shall provide the values of the download parameters defined in section 4.2.3 as follows:

Parameter	Value
url	URL contained in the mediaUrl or thumbnailUrl property
id	CPIM IMDN Message ID header of a Chatbot message containing the Rich Card message body
op	SIP URI of the Chatbot in accordance with the definitions in section 2.5.4 including the "tk" parameter
ci	n/a

Table 84: Rich Card Media download parameters

If the media is played and the Media object included a postback property, the receiving RCS client shall generate and sent a response to the Chatbot using the procedure defined in section 3.6.10.6.2.2 where the `display text` property shall be set to a fixed value of “Card Media Played” and the `postback` property to the full postback object that was provided in the Media object.

3.6.10.5.3.2 Streaming Rich Card Media

If the mediaContentType indicates a supported streaming format (e.g. text/html, video/vnd.youtube, video/vnd.vimeo) or mediaFileSize is absent:

1. The RCS client shall not invoke the file download procedure defined in section 4.2.3 for the mediaUrl.
2. The RCS client shall download the content referred to by the thumbnailUrl (if present) to display a preview.
3. Upon user interaction (e.g., tapping the thumbnail), the RCS client shall render the mediaUrl within the Rich Card media frame using an appropriate embedded player or

restricted WebView. If the Media object included a postback property, the receiving RCS client shall generate and sent a response to the Chatbot using the procedure defined in section 3.6.10.6.2.2 where the `display text` property shall be set to a fixed value of “Card Media Played” and the `postback` property to the full postback object that was provided in the Media object.

NOTE: To embed services such as YouTube or Vimeo, Chatbots are expected to provide the embeddable version of the URL (e.g., https://www.youtube.com/embed/VIDEO_ID) in the mediaUrl field and set the mediaContentType to text/html or a vendor-specific MIME type agreed upon with the client (e.g. video/vnd.youtube).

3.6.10.5.4 CSS for the rendering of Rich Cards

The CSS can either be sent as generic CSS template (to be used for all Rich Cards sent by that Chatbot) or as attached to a single Rich Card (which does not influence the appearance of any other Rich Card sent by that Chatbot) as specified in [GSMA PRD-RCC.71]. The CSS elements shall not be included in the Rich Cards.

3.6.10.5.4.1 Retrieval of CSS

If a CSS Style link is attached in the Rich Card or a CSS Style link is included in the Chatbot Information, a client supporting CSS for the rendering of Rich Cards shall retrieve the applicable CSS before rendering the Rich Cards as defined in section 3.6.10.5.4.5. If available from the local cache defined in section 3.6.10.5.4.2, the client shall retrieve the CSS from the cache. Otherwise, the client shall retrieve the CSS from the URL provided as value of the `generic-CSS-template` property in the Chatbot Information defined in section 3.6.4.1.6 or from the `style` property of the Rich Card defined in the JSON schema in section 3.6.10.4, depending on which is applicable.

When Chatbot Information is retrieved as specified in section 3.6.4.1.2, a client supporting CSS for the rendering of Rich Cards shall check the `generic-CSS-template` property and download the new external CSS file to the local storage defined in section 3.6.10.5.4.2 if the URL in the `generic-CSS-template` property is changed compared to the earlier available Chatbot Information.

3.6.10.5.4.2 Caching of CSS

A client supporting CSS for the rendering of Rich Cards shall support caching of CSS locally. After retrieval of the CSS as defined in section 3.6.10.5.4.1, the client shall store the CSS locally and use the CSS from this local storage. The client shall respect directives for cache control as defined in [RFC7234]. If no cache control directive has been provided, then the client shall apply a local refresh policy, i.e. refresh every 24 hours.

NOTE1: When CSS provider (Chatbot, Chatbot Platform or MNO) releases a new CSS style, it is recommended that the file name of the CSS is changed to ensure that clients will use the new version at the earliest opportunity.

NOTE2: A URI parameter, can be added to the CSS URI by the CSS provider to force the client to retrieve the CSS from the applicable URI within 24 hours if the file name of CSS is not changed.

The client shall render Rich Cards received after a new CSS was retrieved based on the triggers defined in section 3.6.10.5.4.1 using this new CSS. Rich Cards received before the new CSS was retrieved shall be rendered based on the old CSS. If an old CSS has expired or is no longer available, the Rich Card rendered based on that CSS style shall be shown to the user with the default view in the client.

The CSS attached to a single Rich Card shall not be changed after the Rich Card is sent to the client.

3.6.10.5.4.3 Properties of CSS

A client supporting CSS for the rendering of Rich Cards shall support the following elements in Rich Card to be controlled by CSS:

1. The whole Rich Card, referred to with the message selector in the CSS
2. Rich Card Title, referred to with the `message.content.title` selector in the CSS
3. Rich Card description, referred to with the `message.content.description` selector in the CSS
4. Rich Card suggestions, referred to with the `message.content.suggestions` selector in the CSS

A client supporting CSS for the rendering of Rich Cards, shall support the CSS properties described in Table 85 for the rendering of these Rich Card elements based on the definitions of those properties in [CSS2.1]:

CSS Selector	Supported CSS Properties for selector	Description of CSS properties and supported values for CSS properties
message	color	The foreground colour of all elements' text content in the Rich Card. The value references [CSS2.1]
	font	The font of all elements' text content in the Rich Card. The value references [CSS2.1]
	text-align	The property describes how to align the inline-level content of all elements' text content in the Rich Cards. The value references [CSS2.1]
	background-color	The background colour of the Rich Card. The value references [CSS2.1] and the default value is the keyword 'transparent'
	background-image	The background image of the Rich Card.

CSS Selector	Supported CSS Properties for selector	Description of CSS properties and supported values for CSS properties
		Values for this property are either <uri> to specify the image, or 'none' (default), when no image is used. When background image is specified, a background colour shall also be specified. The background colour will be used when the image is unavailable. When the image is available, it is rendered on top of the background colour.
message.content.title	color	The foreground colour of the title element's text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	font	The shorthand property for setting all font-related properties (specified in [CSS2.1]) of title element' text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	text-align	The property describes how to align the inline-level content of the title element's text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
message.content.description	color	The foreground colour of the description element's text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	font	The shorthand property for setting all font-related properties (specified in [CSS2.1]) of description element' text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	text-align	The property describes how to align the inline-level content of the description element's text content.

CSS Selector	Supported CSS Properties for selector	Description of CSS properties and supported values for CSS properties
		The value references [CSS2.1]. It can be inherited from the message element (if not specified).
message.content.suggestions	color	The foreground colour of the suggestions element's text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	font	The shorthand property for setting all font-related properties (specified in [CSS2.1]) of suggestions element' text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).
	text-align	The property describes how to align the inline-level content of the suggestions element's text content. The value references [CSS2.1]. It can be inherited from the message element (if not specified).

Table 85: CSS properties applicable to Rich Card elements

3.6.10.5.4.4 Example CSS for Rich Card

A non-normative example CSS of Rich Card can be found in Table 67:

```
@charset "utf-8";
/* CSS Document */

Message {
    text-align:center;
  font-size:12px;
  color:#555;
  font-family:Verdana, Arial, Helvetica, sans-serif;
  background-color: green;
}
message.content.title {
  font-size:14px;
  font-weight:bold;
  color:#666699
}
message.content.description {
  color:#666699
}
message.content.suggestions {
  color:#669999
}
```

Table 86: Example CSS of Rich Card

3.6.10.5.4.5 Use of CSS for rendering

A client supporting CSS for the rendering of Rich Cards shall render the general purpose cards as follows.

1. If a Rich Card specified style is attached in the Rich Card using the style property defined in section 3.6.10.4, the client shall retrieve the CSS based on the value of the style property as specified in section 3.6.10.5.4.1 and render the general purpose cards based on the properties and selectors defined in section 3.6.10.5.4.3 that are provided in the CSS that was retrieved.
2. If no `style` property is provided in the Rich Card or the CSS that is referred to in the URL that is provided as value of the `style` property cannot be retrieved, the client shall check whether the Chatbot Information includes a `generic-CSS-template` property as defined in section 3.6.4.1.6. If such a property is provided, the client shall retrieve the CSS based on the value of the `generic-CSS-template` property as specified in section 3.6.10.5.4.1 and render the general purpose cards based on the properties and selectors defined in section 3.6.10.5.4.3 that are provided in the CSS that was retrieved.
3. If both a `generic-CSS-template` property is provided in the Chatbot Information and a `style` property is provided in the general purpose card, the client shall render the general purpose card based on the CSS referred to from the `style` property is provided in the general purpose card unless as specified in step 2 the CSS that it refers to cannot be retrieved. Multiple external CSS style sheets for the same Rich Card shall not be supported, if some properties of Rich Card element are not specified in the Rich Card specified style, the client shall not use the value of same properties provided in the Chatbot specified style.
4. If some properties of Rich Card element are absent and cannot be inherited from the parent element (e.g. the color property of title element is not specified and the color

property of message element is also not specified), the client shall use the default value of these properties.

5. If no style was provided or the CSS referred to from the `generic-CSS-template` property provided in the Chatbot Information and the `style` property is provided in the general purpose card (where applicable) cannot be retrieved, the client shall use the default style that it provides for the rendering of the general purpose card.

3.6.10.6 Suggested Chip List

As specified in section 3.6.10.1, a Chatbot Platform shall provide a Suggested Chip List always in combination with the message it is associated to.

3.6.10.6.1 Payload from Chatbot Platform to Clients

3.6.10.6.1.1 Suggested Replies

Suggested Replies consist of a display text and a set of postback data.

3.6.10.6.1.2 Suggested Actions

Suggested Actions are grouped into seven different categories supporting a total of thirteen different suggested actions:

- urlAction
  - openUrl — opens a web site (in the default browser or in an integrated Webview in the messaging application) or app via deep linking
- dialerAction
  - dialPhoneNumber — calls a phone number via the user's dialler app
  - dialEnrichedCall — start an Enriched Call via the user’s dialler app
  - dialVideoCall — start a video call via the user’s dialler app
- mapAction
  - showLocation — show location(s) on a map for given coordinates or search query
  - requestLocationPush — request for a one-time geo location push
- calendarAction
  - createCalendarEvent — creates a new event on the user's calendar
- composeAction
  - composeTextMessage — compose a draft text message
  - composeRecordingMessage — compose a draft message and start recording audio or video
- deviceAction

  - requestDeviceSpecifics — request for a one-time share of device specifics (device model, operating system version, messaging client identifier and version, and remaining battery charge in minutes)

- settingsAction
  - disableAnonymization — ask the user to disable the anonymization setting
  - enableDisplayedNotifications — ask the user to enable sending displayed notifications

- copyAction
  - copyText — copy text to the clipboard

This design allows for easily extending action categories and concrete actions in the future.

Most actions allow fallback URLs in case a user does not have any app of the required type installed. Chatbot Platforms can use the fallback URL to suggest an appropriate app to the user.

For the mapAction, if a showLocation request includes a crs (coordinate reference system) parameter, the RCS client shall use the coordinate reference system that is referred by the value of this parameter when representing the position on the map. Following values shall be supported for the crs parameter:

- "wgs84" or "urn:ogc:def:crs:EPSG::4326": referring to the WGS 84 coordinate reference system, see [WGS84].

NOTE: MNOs and RCS client providers may agree on the support of further reference systems and their corresponding values, e.g. to support systems that are of importance only within a certain region.

If the crs parameter is not included in a showLocation request or the parameter includes a value referring to an unknown or not supported coordinate reference system, the RCS client shall use the WGS 84 coordinate reference system [WGS84] when representing the position on a map.

3.6.10.6.1.3 Example JSON payload

The following payload defines a Suggested Chip List with two suggested replies and all currently supported actions:

```
{
 "suggestions": [{
    "reply": {
      "displayText": "Yes",
      "postback": {
        "data": "set_by_chatbot_reply_yes"
      }
    }
  },
  {
    "reply": {
      "displayText": "No",
      "postback": {
        "data": "set_by_chatbot_reply_no"
      }
```

```
    }
   },
   {
    "action": {
     "urlAction": {
      "openUrl": {
       "url": "https://www.google.com",
       "application": "webview"
      }
     },
     "displayText": "Webview or app deep link",
     "postback": {
      "data": "set_by_chatbot_open_url_webview"
     }
    }
   },
   {
    "action": {
     "urlAction": {
      "openUrl": {
       "url": "https://botstore.info/",
       "application": "webview",
       "ignore_app_deep_link": "true”
      }
     },
     "displayText": "Strictly Webview",
     "postback": {
      "data": "set_by_chatbot_open_url_strictly_webview"
     }
    }
   },
   {
    "action": {
     "dialerAction": {
      "dialPhoneNumber": {
       "phoneNumber": "+1650253000"
      }
     },
     "displayText": "Call a phone number",
     "postback": {
      "data": "set_by_chatbot_dial_phone_number"
     }
    }
   },
   {
    "action": {
     "dialerAction": {
      "dialEnrichedCall": {
       "phoneNumber": "+1650253000",
       "subject": "The optional subject for the enriched call"
      }
     },
     "displayText": "Start enriched call",
     "postback": {
      "data": "set_by_chatbot_dial_enriched_call"
     }
    }
```

```
  },
  {
   "action": {
    "dialerAction": {
     "dialVideoCall": {
      "phoneNumber": "+1650253000"
     }
    },
    "displayText": "Start video call",
    "postback": {
     "data": "set_by_chatbot_dial_video_call"
    }
   }
  },
  {
   "action": {
    "mapAction": {
     "showLocation": {
      "location": {
       "latitude": 37.4220041,
       "longitude": -122.0862515,
       "crs": "wgs84",
       "label": "Googleplex"
      },
      "fallbackUrl": "https://www.google.com/maps/@37.4219162,-122.078063,15z"
     }
    },
    "displayText": "Show location on a map",
    "postback": {
     "data": "set_by_chatbot_show_location"
    }
   }
  },
  {
   "action": {
    "mapAction": {
     "showLocation": {
      "location": {
       "query": "restaurants"
      },
      "fallbackUrl": "https://www.google.com/maps/search/restaurants"
     }
    },
    "displayText": "Search location(s) on map",
    "postback": {
     "data": "set_by_chatbot_search_locations"
    }
   }
  },
  {
   "action": {
    "mapAction": {
     "requestLocationPush": {}
    },
    "displayText": "Request a geo location",
    "postback": {
     "data": "set_by_chatbot_request_location_push"
```

```
      }
     }
    },
    {
     "action": {
      "calendarAction": {
       "createCalendarEvent": {
        "startTime": "2017-03-14T00:00:00Z",
        "endTime": "2017-03-14T23:59:59Z",
        "title": "Meeting",
        "description": "GSG review meeting"
       }
      },
      "displayText": "Schedule Meeting",
      "postback": {
       "data": "set_by_chatbot_create_calendar_event"
      }
     }
    },
    {
     "action": {
      "composeAction": {
       "composeTextMessage": {
        "phoneNumber": "+1650253000",
        "text": "Draft to go into the send message text field."
       }
      },
      "displayText": "Draft a text message",
      "postback": {
       "data": "set_by_chatbot_compose_text_message"
      }
     }
    },
    {
     "action": {
      "composeAction": {
       "composeRecordingMessage": {
        "phoneNumber": "+1650253000",
        "type": "VIDEO"
       }
      },
      "displayText": "Record audio or video",
      "postback": {
       "data": "set_by_chatbot_compose_recording_message"
      }
     }
    },
    {
     "action": {
      "deviceAction": {
       "requestDeviceSpecifics": {}
      },
      "displayText": "Request device specifics",
      "postback": {
       "data": "set_by_chatbot_request_device_specifics"
      }
     }
```

```
    },
    {
     "action": {
      "settingsAction": {
       "disableAnonymization": {}
      },
      "displayText": "Share your phone number",
      "postback": {
       "data": "set_by_chatbot_disable_anonymization"
      }
     }
    },
    {
     "action": {
      "settingsAction": {
       "enableDisplayedNotifications": {}
      },
      "displayText": "Send read receipts",
      "postback": {
       "data": "set_by_chatbot_enable_displayed_notifications"
      }
     }
    },
    {
     "action": {
      "copyAction": {
       "copyText": {
        "text": "PROMOCODE1234",
       }
      },
      "displayText": "Copy Coupon",
      "postback": {
       "data": "set_by_chatbot_copy_text"
      }
     }
    }
   ]
}
```

Table 87: Chatbot Communication Suggested Chip List example

3.6.10.6.2 Payload from Client to Chatbot Platform

Whenever a user selects a suggested reply or suggested action, the RCS client shall send a special response message back to the Chatbot with the following precisions:

1. The message shall be a CPIM message.
2. The content type of the CPIM body of this message shall be set to the value defined in section 3.6.10.2.3.
3. The CPIM body of the response message shall be set as

    a) defined in section 3.6.10.6.2.1 for suggested replies responses
    b) as defined in section 3.6.10.6.2.2 for suggested actions responses.

For the `requestDeviceSpecifics` suggested action, RCS clients share data with Chatbot Platforms in a separate payload. The behaviour is defined as follows:

1. The Chatbot sends a `requestDeviceSpecifics` suggested action to the RCS client.
2. The user selects the suggested action.
3. The RCS client sends a response to the suggested action including the postback data (see section 3.6.10.6.2.2 below).
4. The user optionally interacts with a client-defined user interface to confirm sharing data with the Chatbot.
5. The RCS client sends the shared data in a separate payload (see section 3.6.10.6.2.3 below).

3.6.10.6.2.1 Response for suggested replies

When sending a response message pertaining to a user interaction with a suggested reply, an RCS client shall generate the body of the response as follows:

1. Generate a `response` object
2. Include in this `response` object, a `reply` object including all properties that were provided in the suggested reply (such as the `postback` object).

The following payload is an example of a response for a suggested reply from a client to a Chatbot Platform:

```
{
"response": {
  "reply": {
   "displayText": "No",
   "postback": {
    "data": "set_by_chatbot_reply_no"
   }
  }
 }
}
```

Table 88: Chatbot Communication Suggested Reply example

The RCS client shall present to the user the value of the `displayText` property of the suggested reply as a message that was sent.

3.6.10.6.2.2 Response for suggested actions

Following a user interaction with a suggested action, an RCS client shall send a response to the Chatbot Platform if the suggested action that was used included a postback data object. In this case, the RCS client shall generate the body of the response as follows:

1. Generate a response object
2. Include in this `response` object, an `action` object including the `displayText` property and the full `postback` object that were provided for this suggested action.

The following payload is an example of a response for a suggested action from a client to a Chatbot Platform:

```
{
 "response": {
  "action": {
   "displayText": "Open website or deep link",
   "postback": {
    "data": "set_by_chatbot_open_url"
   }
  }
 }
}
```

Table 89: Chatbot Communication Suggested Action example

The RCS client shall not present this message to the user as a message that was sent.

3.6.10.6.2.3 Shared client data

When the user agrees to share client data following a `requestDeviceSpecifics` suggested action, the client shall send a `sharedData` payload with a `deviceSpecifics` property as a CPIM Message to the Chatbot Platform. In this payload, the client shall set

- The value of the `deviceModel` property to the same value as provided for the `terminal_model` Configuration request parameter defined in section 2.4 of [GSMA PRD-RCC.14]
- The value of the `platformVersion` property to the same value as provided for the `terminal_sw_version` defined in section 2.4 of [GSMA PRD-RCC.14]
- The value of the `clientVendor` property to the same value as provided for the `client_vendor` Configuration request parameter defined in section 2.3.2.2
- The value of the `clientVersion` property to the same value as provided for the `client_version` Configuration request parameter defined in section 2.3.2.2
- The value of the `batteryRemainingMinutes` property to the estimated number of remaining minutes of battery life

The Content-Type header of the CPIM body shall be set as specified in section 3.6.10.2.4.

The following payload is an example of data that clients share with a Chatbot Platform as a result of the user interacting with the `requestDeviceSpecifics` suggested action:

```
{
 "sharedData": {
  "deviceSpecifics": {
   "deviceModel": "SmartPhone",
   "platformVersion": "Android-7.1.2-N481G3",
   "clientVendor": "VNDR",
   "clientVersion": "RCSAndrd-1.0",
   "batteryRemainingMinutes": 517
  }
 }
}
```

Table 90: Chatbot Communication Shared Client data example

3.6.11 Critical Chatbots

The list of critical Chatbots shall be defined by the Service Provider and stored in the list of Chatbots requiring specific management in the format defined in section 3.6.3.3.

The list of critical Chatbots shall be identified as CRITICAL (see <name of the list> in section 3.6.3.3). The specific marker to identify the list shall therefore be:

```
LIST:CRITICAL<CRLF>
<CRLF>
```

When the list of Chatbots requiring specific management includes a list of critical Chatbots via the procedures defined in section 3.6.3.3, the client shall store the list of critical Chatbots locally.

When receiving a 1-to-1 Chatbot session request as per section 3.6.8.4, from a Chatbot whose URI is in the critical Chatbots List, the client shall accept the session.

When initiating a 1-to-1 Chatbot session request as per section 3.6.8.3, with a Chatbot whose URI is in the critical Chatbots List, the client shall never request privacy as per section 3.6.5.1.2.1.

The URI comparisons shall be done in the client without considering the 'tk' parameter.

3.6.12 Menu

A Chatbot can provide a multi-level menu with suggested actions and replies to the client.

NOTE: Currently this is only used possible, as a persistent menu that is part of the Chatbot Information as specified in section 3.6.4.1.

3.6.12.1 Menu Structure

The menu would allow to include following items:

- Suggested replies (as described in section 3.6.10.6.1.1)
- Suggested actions (as described in section 3.6.10.6.1.2)
- A level 1 sub menu that on its turn can include
    - Suggested replies (as described in section 3.6.10.6.1.1)
    - Suggested actions (as described in section 3.6.10.6.1.2)
    - A level 2 sub menu that on its turn can include
        - Suggested replies (as described in section 3.6.10.6.1.1)
        - Suggested actions (as described in section 3.6.10.6.1.2)

Whenever a user selects a suggested reply or suggested action in the menu, the RCS client shall send a special response message back as specified in section 3.6.10.6.2.

When the user selects a sub menu, the client shall show the suggested replies, suggested actions and sub menus included in the sub menu.

3.6.12.2 Data structure overview

Figure 23 shows a high-level overview of the menu payload:

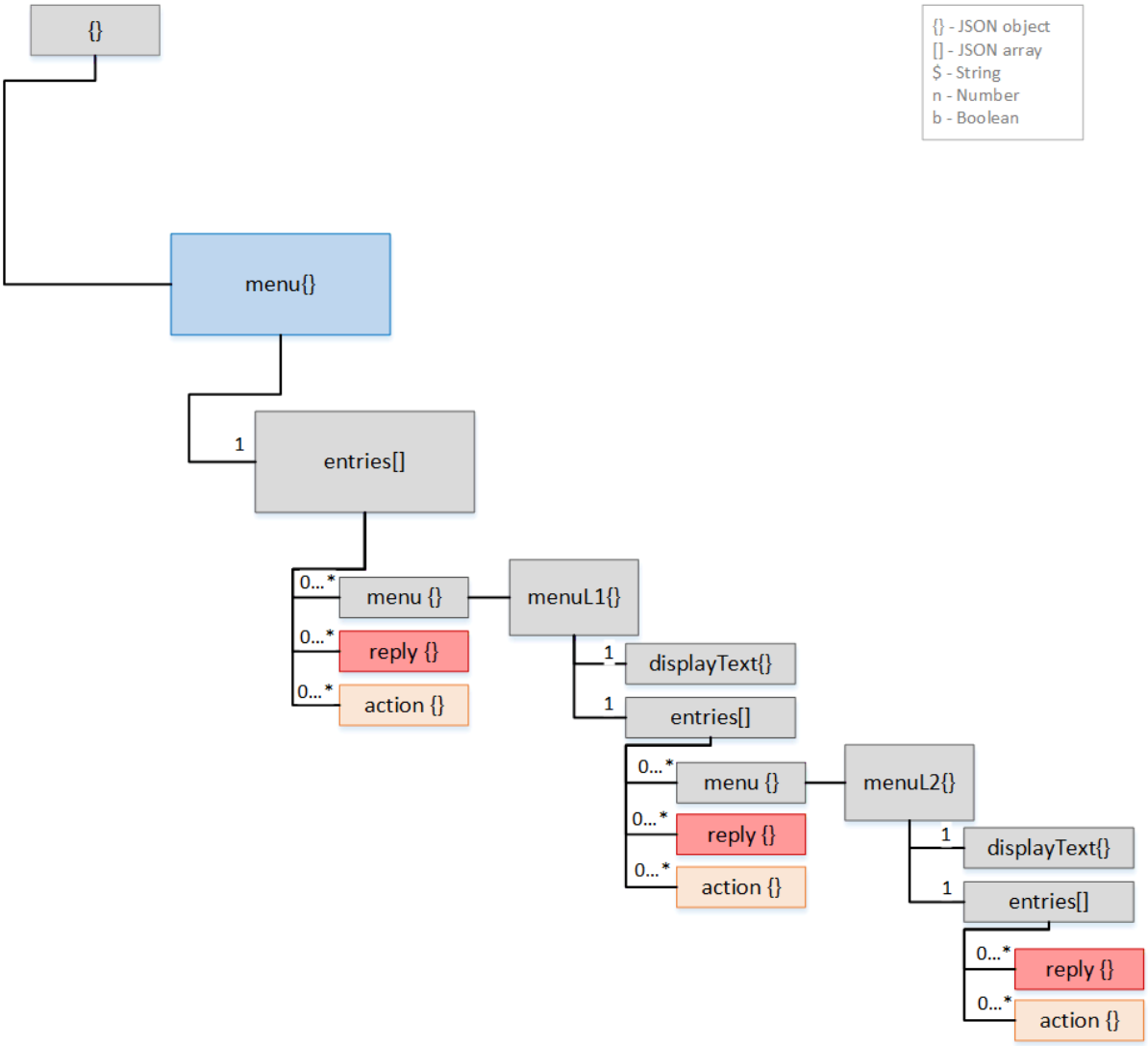


Figure 23: JSON Chatbot Menu data structure overview

3.6.12.3 JSON schema for validation

The following schema defines all JSON menus exchanged between Chatbot Platform and clients:

```
{
 "$schema": "http://json-schema.org/draft-04/schema#",
 "id": "http://www.gsma.com/rcs/chatbot_menu_schema.json",

 "title": "Root Object",
 "type": "object",

 "properties": {
  "menu": {
   "title": "Data used for a menu provided by Chatbot to the client",
   "type": "object",
   "properties": {
    "entries": {
     "$ref": "#/definitions/menus/menuL0"
    }
   },
```

```
    "required":["entries"]
   }
  },
  "required": ["menu"],

  "definitions": {
   "menus": {
    "menuL0": {
     "title": "Top Level menu",
     "type": "array",
     "items": {
      "oneOf": [{
       "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/replies/reply"
      }, {
       "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/actions/action"
      }, {
       "$ref": "#/definitions/menus/menuL1"
      }]
     },
     "minItems": 1,
     "maxItems": 5,
     "additionalItems": false
    },
    "menuL1": {
     "title": "First level sub menu.",
     "type": "object",
     "properties":{
      "menu" : {
       "type": "object",
       "properties":{
        "displayText":{
         "title": "Label to show for menu in Top Level menu.",
         "type": "string",
         "minLength": 1,
         "maxLength": 25
        },
        "entries":{
         "type":"array",
         "items": {
          "oneOf": [{
           "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/replies/reply"
          }, {
           "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/actions/action"
          }, {
           "$ref": "#/definitions/menus/menuL2"
          }]
         },
         "minItems": 1,
         "maxItems": 5,
         "additionalItems": false
        }
       },
       "required":["displayText","entries"]
```

```
        }
      },
      "required":["menu"]
    },
    "menuL2": {
      "title": "Second level sub menu.",
      "type": "object",
      "properties":{
        "menu" : {
          "type": "object",
          "properties":{
            "displayText":{
              "title": "Label to show for menu in Level 1 menu.",
              "type": "string",
              "minLength": 1,
              "maxLength": 25
            },
            "entries":{
              "type":"array",
              "items": {
                "oneOf": [{
                  "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/replies/reply"
                }, {
                  "$ref":
"http://www.gsma.com/rcs/chatbot_message_schema.json#/definitions/suggestions/actions/action"
                }]
              },
              "minItems": 1,
              "maxItems": 5,
              "additionalItems": false
            }
          },
          "required":["displayText","entries"]
        }
      },
      "required":["menu"]
    }
  }
 }
}
```

Table 91: JSON menu schema

3.6.12.4 Example JSON payload

The following payload defines a menu with at top level a suggested reply and one submenu including another suggested reply, a dialer action and a level 2 submenu containing a suggested reply and a Map action:

```
{
 "menu": {
  "entries": [{
   "reply": {
    "displayText": "reply1",
    "postback": {
     "data": "set_by_chatbot_reply1"
    }
   }
  },
  {
   "menu": {
    "displayText": "SubmenuL1",
    "entries": [{
     "reply": {
      "displayText": "reply2",
      "postback": {
       "data": "set_by_chatbot_reply2"
      }
     }
    },
    {
     "menu" : {
      "displayText": "SubmenuL2",
      "entries": [{
       "reply": {
        "displayText": "reply3",
        "postback": {
         "data": "set_by_chatbot_reply3"
        }
       }
      },
      {
       "action": {
        "mapAction": {
         "showLocation": {
          "location": {
           "latitude": 51.512194,
           "longitude": -0.090458,
           "label": "GSMA"
          },
          "fallbackUrl": "https://www.google.com/maps/@51.512194,-0.090458,15z"
         }
        },
        "displayText": "Find our Offices",
        "postback": {
         "data": "set_by_chatbot_show_location"
        }
       }
      }
      ]
     },
    },
    {
     "action": {
      "dialerAction": {
       "dialPhoneNumber": {
```

```
            "phoneNumber": "+442073456789"
           }
          },
          "displayText": "Call a phone number",
          "postback": {
           "data": "set_by_chatbot_dial_menu_phone_number"
          }
         }
        }
       ]
      }
     }
    ]
  }
}
```

Table 92: Chatbot Communication Suggested Chip List example

3.7 MIVCs

3.7.1 Introduction

This section describes the architecture and procedures for MIVCs as defined in sections 5, 6 and 11 of [GSMA PRD-RCC.71].

As described in section 3.7.2, the RCS Service Provider configures MIVC technologies weights used by the selection algorithm to select the final communication protocol used as defined in section 3.7.3: ViLTE and/or RCS Video Chat. If selecting ViLTE, MIVC shall be established as defined in section 3.7.4. If selecting RCS Video Chats, MIVCs shall be established as defined in section 3.7.5.

3.7.2 Provisioning

During provisioning, as defined in section 2.3.2, the RCS Service Provider shall configure the client configuration parameters related to MIVCs defined in section A.1.3.

3.7.3 Technology Selection

The RCS client procedures for technology selection is based on the MIVC TECHNOLOGY client configuration parameter defined in section A.1.3.

3.7.3.1 MIVC Technology: None

If the RCS Service Provider sets the value of the MIVC TECHNOLOGY client configuration parameter defined in section A.1.3 to None, the RCS client shall not advertise the ViLTE or the RCS Video Chat feature tags as per sections 3.7.3.5 and 3.7.3.6. The RCS client shall consider MIVCs as not available.

3.7.3.2 MIVC Technology: ViLTE

If the RCS Service Provider sets the value of the MIVC TECHNOLOGY client configuration parameter defined in section A.1.3 to ViLTE, the RCS client shall advertise the ViLTE capability as per section 3.7.3.5.

The RCS client shall initiate the Capability Discovery as defined in section 3.7.3.5. If any participant in the 1-to-1 or Group Chat is not capable of ViLTE, the RCS client shall consider MIVCs as not available. However, if all participants are capable of ViLTE, then any MIVCs shall be initiated using procedures defined in section 3.7.4.

3.7.3.3 MIVC Technology: ViLTE preferred, RCS Video Chat fallback

If the RCS Service Provider sets the value of the MIVC TECHNOLOGY client configuration parameter defined in section A.1.3 to “ViLTE preferred, RCS Video Chat fallback”, the RCS client shall advertise the ViLTE capability as per section 3.7.3.5. The RCS client shall also advertise the RCS Video Chat feature tag as per section 3.7.3.6.

The RCS client shall initiate the Capability Discovery as defined in sections 3.7.3.5 and 3.7.3.6. If there is no common capability (ViLTE or RCS Video Chat) among all participants, the RCS client shall consider MIVCs as not available.

If all participants in a 1-to-1 or Group Chat are capable of ViLTE, then any MIVCs initiated shall be initiated using the procedures defined in section 3.7.4, Otherwise, if all participants are capable of RCS Video Chat, then any MIVCs initiated shall be initiated via the procedures defined in section 3.7.5.

3.7.3.4 MIVC Technology: RCS Video Chat

If the RCS Service Provider sets the value of the MIVC TECHNOLOGY Parameter as per A.1.3 to RCS Video Chat, the RCS client shall advertise the RCS Video Chat feature tag as per section 3.7.3.6.

The RCS client shall initiate the Capability Discovery as defined in section 3.7.3.6. If all participants are capable of RCS Video Chat, then any MIVCs shall be initiated via the procedures defined in section 3.7.5.

3.7.3.5 ViLTE Capability Discovery

If the RCS Service Provider enables ViLTE for MIVCs as per section 3.7.2 and the RCS client is capable of ViLTE, the RCS client shall advertise the ViLTE capability as per section 5.2 in [GSMA PRD-IR.94] or section 2.2.10 in [GSMA PRD-NG.114].

The RCS client shall initiate the Capability Discovery for ViLTE as per section 5.2 in [GSMA PRD-IR.94] or section 2.2.10 in [GSMA PRD-NG.114].

3.7.3.6 RCS Video Chat Capability Discovery

If the RCS Service Provider enables RCS Video Chat for MIVCs as per section 3.7.2 and the RCS client is capable of RCS Video Chat, the RCS client shall advertise the RCS Video Chat feature tag as defined in section 2.6.1.3 via the procedures defined in section 2.6.

3.7.4 ViLTE

If a ViLTE call is initiated from a 1-to-1 messaging conversation by the RCS client, the RCS client shall follow the procedures defined in [GSMA PRD-IR.94] and [GSMA PRD-NG.114] when setting up an MIVC with the contact in the Messaging Conversation.

If a ViLTE group call is initiated from a Group Chat by the RCS client, the RCS client shall follow the procedures defined in in [GSMA PRD-IR.94], [GSMA PRD-NG.114] and [3GPP TS 24.605] for setting up an MIVC with the contacts in the Group Chat that the client is aware of as defined in section 3.2.4.8.

NOTE: Some aspects of the ViLTE implementation of MIVC are for further study.

3.7.5 RCS Video Chat

3.7.5.1 Architecture

The RCS Service Provider Network (both over UNI and NNI) enables signalling for RCS Video Chat. The RCS client shall use SDP for WebRTC sessions as per [RFC8445] and [RFC8841] which are necessary for the RCS client to negotiate media sessions. The RCS client shall send the SDP via the RCS Video Chat signalling protocol as described in section 3.7.5.2. This is indicated via the solid lines in Figure 24.

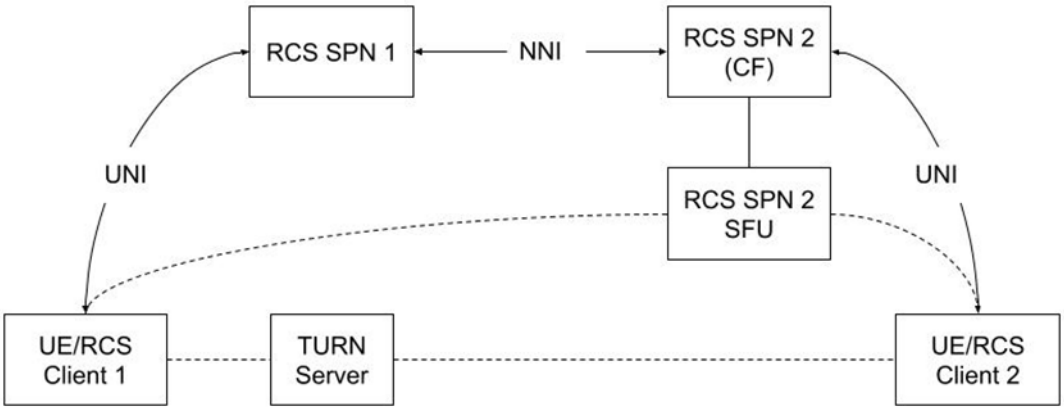


Figure 24: Architecture for RCS Video Chat

Once the RCS client receives the SDP for the WebRTC sessions, it shall setup WebRTC connections as per [RFC5764] and [RFC8832]. This is indicated through dotted lines in Figure 24. For 1-to-1 Chats, that may be a direct Peer-to-Peer connection, or indirectly via a TURN server as per [RFC8155]. For Group Chats, the WebRTC connections shall be managed by a Selective Forwarding Unit (SFU) (as defined in [RFC9605]) provided by the Service Provider Network (SPN) of the Controlling Function (CF) for the Group Chat.

NOTE: The use of TURN is for NAT traversal or hiding the IP address of the UE.

Once the RCS Video Chat session is established, if there are any updates in media, the RCS client shall create a new SDP Offer/Answer as per [RFC8445] and [RFC8841]. The RCS client shall use the new SDP via the RCS Video Chat signalling protocol over the RCS Service Provider Network (as per section 3.7.5.2).

If the 1-to-1 and Group Chat are protected by E2EE, then the RCS Video Chats shall be subject to E2EE. The RCS Video Chat signalling messages for 1-to-1 Chats shall be subject to E2EE, and Group Chat signalling messages shall be signed as per [GSMA PRD-RCC.16]. For both 1-to-1 and Group Chats, all media for RCS Video Chats shall be subject to E2EE.

3.7.5.2 Signalling

gRPC-configured RCS clients shall follow the procedures defined in section 3.1.4 of [GSMA PRD-RCC.30].

3.7.5.2.1 Creating RCS Video Chat SDP

To create an SDP for RCS Video Chat, the RCS client shall first gather ICE candidates. The RCS client shall use Full ICE as per [RFC8445]; the RCS client shall not use ICE LITE. The RCS client shall only use UDP for all ICE candidates.

If the RCS Service Provider has set the RCS VIDEO CHAT TURN parameter (as specified in section A.1.3) to 1, then the RCS client shall gather an ICE candidate for each of the TURN server URIs as per section 3.7.5.2.1.1 and shall only send those ICE candidates to the other RCS clients. If none of the TURN servers specified in RCS VIDEO CHAT TURN SERVER URI client configuration parameters are unavailable, the RCS client shall consider the RCS Video Chat as failed.

If the RCS VIDEO CHAT TURN parameter is set to 0, then the RCS client shall follow procedures in section 5 of [RFC8445]. The RCS client is free to select its own STUN and TURN servers or to follow the procedures in section 3.7.5.2.1.1 to use the RCS SPN's TURN server if it is available. The ICE candidates shall be prioritised as per [RFC8445].

The RCS client shall include the codecs that it supports as per 3.7.5.3.2, include DTLS keys as per [RFC5764] and generate an SDP as per [RFC8841].

3.7.5.2.1.1 ICE Candidate for TURN Server

To gather ICE candidates, the RCS client shall gather ICE candidates for all Turn Credentials specified in section A.1.3. Each Turn Credential specifies a number of TURN server URIs defined in the parameter RCS VIDEO CHAT TURN SERVER URI as specified in section A.1.3. All the URIs under the same Turn Credential share the RCS VIDEO CHAT TURN SERVER USERNAME and RCS VIDEO CHAT TURN SERVER PASSWORD parameters defined in section A.1.3. Using the URI, USERNAME, and PASSWORD the RCS client shall connect to the TURN servers as per [RFC5766]. The RCS client shall then gather ICE candidates for the TURN servers as per section 5.1.1.2 of [RFC8445].

3.7.5.2.2 Establishing an RCS Video Chat Session

Before the RCS client can establish an RCS Video Chat session, it shall first establish (or reuse) an MSRP session as per sections 3.2.3 and 3.2.4. The MSRP session shall be the same session that the RCS client uses for messaging.

The offerer RCS client shall generate the SDP offer as per section 3.7.5.2.1. It shall include the SDP offer in a CallInvitation protocol buffer and include the CallInvitation in an MSRP message as per section 3.7.5.2.3. The offerer RCS client shall send the MSRP message via the MSRP session (as per section 3.2.3.1) to the recipients' RCS clients.

Upon acceptance of the call, the answerer's RCS client shall generate its own SDP answer as specified in section 3.7.5.2.1. The answerer's RCS client shall include the SDP answer in an AcceptInvitation protocol buffer. The answerer's RCS client shall include the resulting protocol buffer in an MSRP message as per section 3.7.5.2.3. The answerer's RCS client

shall send the MSRP message via the MSRP session (as per section 3.2.3.1) to the offerer's RCS client.

3.7.5.2.3 Wireformat

3.7.5.2.3.1 CPIM representation

3.7.5.2.3.1.1 RCS Video Chat content type

The CPIM message content type for an RCS Video Chat message is defined in Table 93

Content Type	CPIM Body Content
rcs/video-chat+proto	VideoChatMessage, ClientGroupVideoMessage, ServerGroupVideoMessage

Table 93: RCS Video Chat Content Type

3.7.5.2.3.1.2 Message Format

The client shall compose a CPIM message with the content type defined in section 3.7.5.2.3.1.1. For 1-to-1 RCS Video Chat, the RCS client shall include a VideoChatMessage as defined in section 3.7.5.2.3.2 in the CPIM body as illustrated in Figure 25. For Group RCS Video Chat, the RCS client shall include a ClientGroupVideoMessage as defined in section 3.7.5.2.3.3.1 in the CPIM body as illustrated in Figure 26. The server shall include a ServerGroupVideoMessage as defined in section 3.7.5.2.3.3.2 in the CPIM body as illustrated in Figure 27.

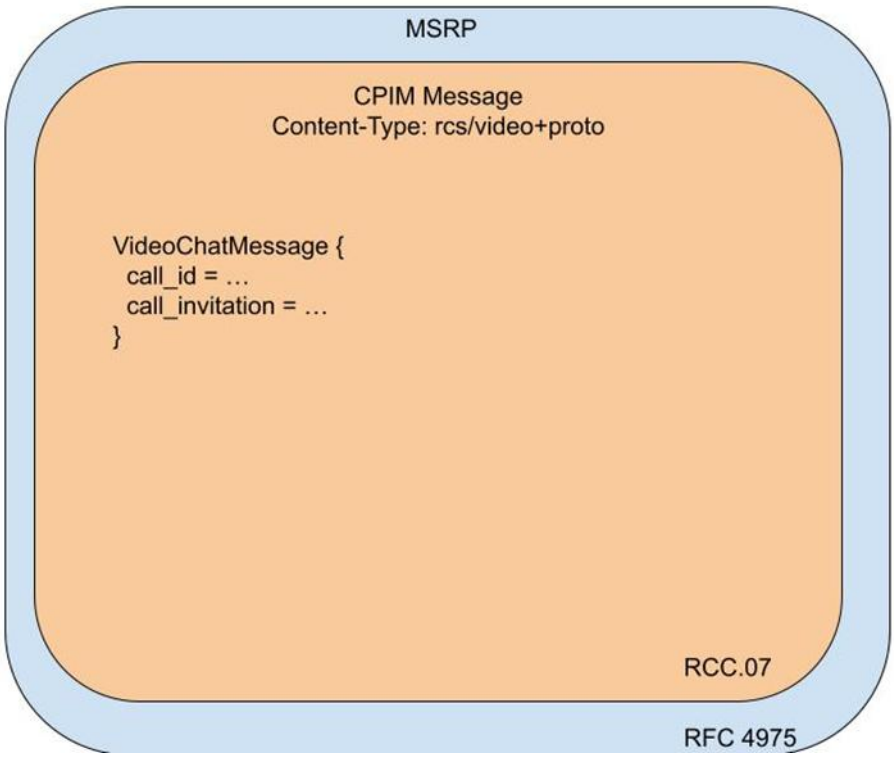


Figure 25: Format for 1-to-1 RCS Video Chat Signalling Message

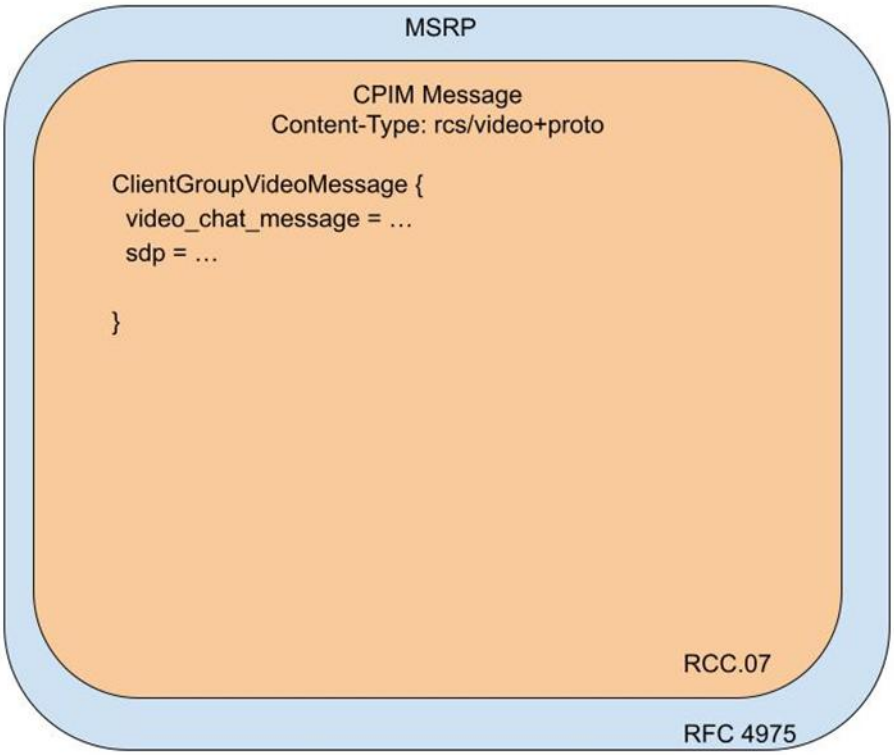


Figure 26: Format for a Group RCS Video Chat RCS Client Signalling Message

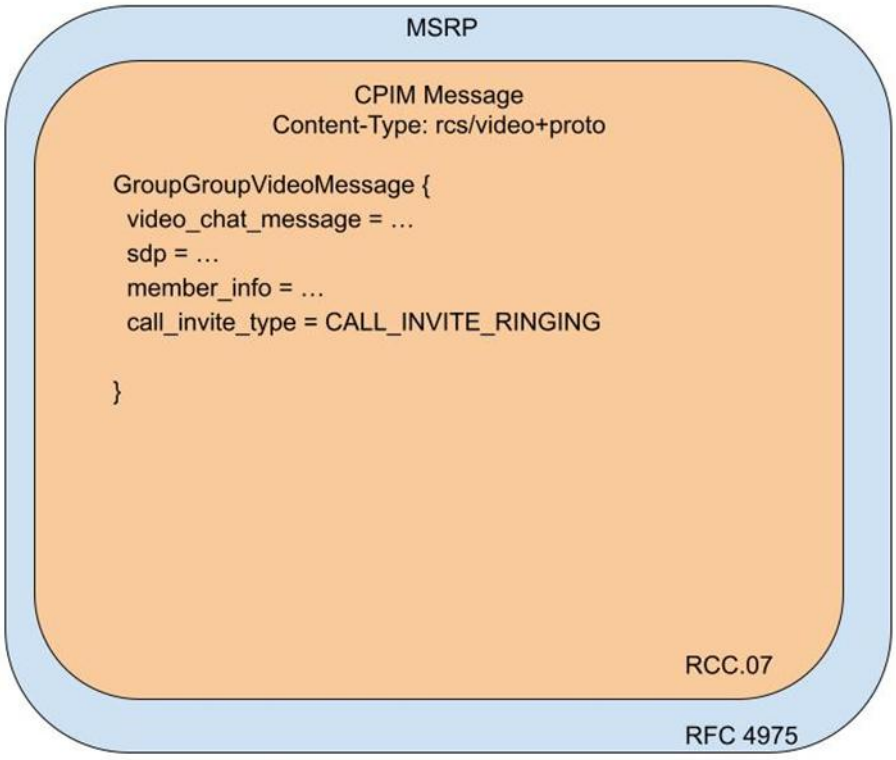


Figure 27: Format for a Group RCS Video Chat RCS Server Signalling Message

3.7.5.2.3.2 VideoChatMessage

This section defines the format for RCS Video Chat signalling.

The `VideoChatMessage` is the container message for all the events of this section. It includes the field `call_id` that is a unique identifier of the call of the RCS Video Chat and the (one) event that the RCS client or SPN shall transmit.

```
message VideoChatMessage{
  string call_id = 1;

  oneof event {
    CallInvitation call_invitation = 101;
    AckInvitation ack_invitation = 102;
    AcceptInvitation accept_invitation = 103;
    RejectInvitation reject_invitation = 104;
    Hangup hangup = 105;
    CancelInvitation cancel_invitation = 106;
    UpdateMedia update_media = 107;
    IceCandidatesUpdate ice_candidates_update = 108;
  }
}
```

Table 94: VideoChatMessage protocol buffer definition

3.7.5.2.3.2.1 CallInvitation Event

The CallInvitation message shall include an `sdp_offer` string field defined as per [RFC8445] and [RFC8841]. The `sdp_offer` field shall be populated only for 1-to-1 RCS Video Chats.

```
message CallInvitation {
  string sdp_offer = 1;  // for 1-to-1 RCS Video Chat only
}
```

Table 95: CallInvitation protocol buffer definition

```
{
  "call_id": "a93b-4c2d-8e6f-7a1h",
  "call_invitation": {
    "sdp_offer": "..."
  }
}
```

Table 96: CallInvitation 1-to-1 RCS Video Chat protocol buffer example

3.7.5.2.3.2.2 AckInvitation Event

The `AckInvitation` message does not contain any fields.

```
message AckInvitation {
}
```

Table 97: AckInvitation protocol buffer definition

3.7.5.2.3.2.3 AcceptInvitation Event

The `AcceptInvitation` message shall include an `sdp_answer` string field defined as per [RFC8445] and [RFC8841]. The `sdp_answer` field shall be populated only for 1-to-1 RCS Video Chats.

```
message AcceptInvitation {
  string sdp_answer = 1; // for 1-to-1 RCS Video Chat only
}
```

Table 98: AcceptInvitation protocol buffer definition

3.7.5.2.3.2.4 RejectInivitation Event

The `RejectInvitation` message sent by the callee's RCS client shall include one `reject_reason` field that is defined in section 3.7.5.2.3.2.4.1.

```
message RejectInvitation {
  RejectReason reject_reason = 1;
}
```

Table 99: RejectInvitation protocol buffer definition

```
{
  "call_id": "a93b-4c2d-8e6f-7a1h",
  "reject_invitation": {
    "reject_reason": "REJECT_REASON_USER_REJECT"
  }
}
```

Table 100: RejectInvitation protocol buffer example

3.7.5.2.3.2.4.1 *RejectReason*

The `RejectReason` is an enum of reasons that the callee's RCS client shall provide when rejecting an RCS Video Chat.

Rejection Reason	Description
`REJECT_REASON_UNSPECIFIED`	Should not be used.
`REJECT_REASON_USER_REJECT`	The callee rejected the call.
`REJECT_REASON_ALREADY_IN_PROGRESS`	The RCS client rejected call because an RCS Video Chat was already in progress

Table 101: Rejection reason mapping

```
enum RejectReason {
  REJECT_REASON_UNSPECIFIED = 0;
  REJECT_REASON_USER_REJECT = 1;.
  REJECT_REASON_ALREADY_IN_PROGRESS = 2;
}
```

Table 102: RejectReason Enum definition

3.7.5.2.3.2.5 Hangup Event

The `Hangup` message shall contain one `hangup_reason` enum that is defined in section 3.7.5.2.3.2.5.1.

```
message Hangup {
  HangupReason hangup_reason = 1;
}
```

Table 103: The Hangup protocol buffer definition

```
{
  "call_id": "a93b-4c2d-8e6f-7a1h",
  "hangup": {
    "hangup_reason": "HANGUP_REASON_USER_INITIATED"
  }
}
```

Table 104: Hangup 1-to-1 RCS Video Chat protocol buffer example

3.7.5.2.3.2.5.1 *HangupReason enum*

The `HangupReason` is an enum of reasons that the participant's RCS client or the Group RCS Video Chat Controlling Function shall provide for hanging up a Group RCS Video Chat.

Hangup Reason	Description
`HANGUP_REASON_UNSPECIFIED`	Should not be used.
`HANGUP_REASON_USER_INITIATED`	The participant explicitly hung up the call.
`HANGUP_REASON_CLIENT_TIMEOUT`	The RCS client detected a timeout and hung up the call.
`HANGUP_REASON_SERVER_DETECTED_DISCONNECT`	For Group RCS Video Chat only. The SFU detected a disconnection of a participant in the Group Video Call and hangs up the call.

Table 105: HangupReason mapping

```
enum HangupReason {
  HANGUP_REASON_UNSPECIFIED = 0;
  HANGUP_REASON_USER_INITIATED = 1;
  HANGUP_REASON_CLIENT_TIMEOUT = 2;
  HANGUP_REASON_SERVER_DETECTED_DISCONNECT =3;
}
```

Table 106: HangupReason Enum definition

3.7.5.2.3.2.6 CancelInvitation Event

The `CancelInvitation` message sent by the caller's RCS client shall include one `cancel_reason` enum that is defined in section 3.7.5.2.3.2.6.1.

```
message CancelInvitation {
  CancelReason cancel_reason = 1;
}
```

Table 107: CancelInvitation protocol buffer definition

```
{
  "call_id": "a93b-4c2d-8e6f-7a1h",
  "cancel_invitation": {
    "cancel_reason": "CANCEL_REASON_USER_CANCEL"
  }
}
```

Table 108: CancelInvitation protocol buffer example

3.7.5.2.3.2.6.1 *CancelReason enum*

The `CancelReason` enum includes reasons that the caller's RCS client shall provide for cancelling a Group RCS Video Chat.

Cancel Reason	Description
`CANCEL_REASON_UNSPECIFIED`	Should not be used.
`CANCEL_REASON_USER_CANCEL`	The caller cancelled the call.
`CANCEL_REASON_TIMEOUT`	The RCS client cancelled the call because the call initiation timed out.

Table 109: CancelInvitation reason mapping

```
enum CancelReason {
  CANCEL_REASON_UNSPECIFIED = 0;
  CANCEL_REASON_USER_CANCEL = 1;
  CANCEL_REASON_TIMEOUT = 2;
}
```

Table 110: CancelReason Enum definition

3.7.5.2.3.2.7 IceCandidatesUpdate Event

The `IceCandidatesUpdate` message includes two fields: `add_ice_candidates` and `remove_ice_candidates` which contain the lists of ICE candidates that need to be added and/or removed by the callee's RCS client.

```
message IceCandidatesUpdate {
  repeated string add_ice_candidates = 1;
  repeated string remove_ice_candidates = 2;
}
```

Table 111: IceCandidatesUpdate protocol buffer definition

```
{
  "add_ice_candidates": "...",
  "remove_ice_candidates": "..."
}
```

Table 112: IceCandidatesUpdate protocol buffer example

3.7.5.2.3.2.8 UpdateMedia Event

The `UpdateMedia` message includes two fields, one or many `view_request` messages and the `bandwidth_constraints` message defined in this section. This message is sent by the RCS client to indicate its current set of preferred video capabilities or to update them as network conditions or client configuration change.

```
message UpdateMedia {
  repeated ViewRequest view_request = 1;
  BandwidthConstraints bandwidth_constraints = 2;
}
```

Table 113: UpdateMedia protocol buffer definition

```
{
  "call_id": "a93b-4c2d-8e6f-7a1h",
  "update_media": {
    "view_request": [
      {
        "stream_id": {
          "media_stream_id": "user-456-video-main",
          "media_type": "MEDIA_TYPE_VIDEO"
        },
        "resolution": {
          "height": 720,
          "width": 1280,
          "frame_rate": 30
        }
      }
    ],
    "bandwidth_constraints": {
      "max_bandwidth_kbps": 2500,
      "min_bandwidth_kbps": 500,
      "start_bandwidth_kbps": 1000
    }
  }
}
```

Table 114: UpdateMedia protocol buffer example

3.7.5.2.3.2.8.1 *ViewRequest Message*

The `ViewRequest` message which is included in the `UpdateMedia` message shall include two fields: the `stream_id` message and the `resolution` message defined in this section.

The `VideoResolution` message shall be only present in this message if the `MediaType` of the `MediaStreamId` is `MEDIA_TYPE_VIDEO`.

```
message ViewRequest {
  MediaStreamId stream_id = 1;
  VideoResolution resolution = 2;
}
```

Table 115: ViewRequest protocol buffer definition

3.7.5.2.3.2.8.1.1 *MediaStreamId Message*

The `MediaStreamId` message included in the `ViewRequest` message shall include two fields: the `media_stream_id` string that represents the media ID of the media and the `media_type` enum as defined in section 3.7.5.2.3.2.8.1.2.

```
message MediaStreamId {
  string media_stream_id = 1;
  MediaType media_type = 2;
}
```

Table 116: MediaStreamId protocol buffer definition

3.7.5.2.3.2.8.1.2 *MediaType*

The `MediaType` enum indicates the type of media being used: audio or video.

Media Type	Description
`MEDIA_TYPE_UNSPECIFIED`	Should not be used.
`MEDIA_TYPE_AUDIO`	The call is audio only.
`MEDIA_TYPE_VIDEO`	The call is a video call with audio.

Table 117: MediaType mapping

```
enum MediaType {
  MEDIA_TYPE_UNSPECIFIED = 0;
  MEDIA_TYPE_AUDIO = 1;
  MEDIA_TYPE_VIDEO = 2;
}
```

Table 118: MediaType enum definition

3.7.5.2.3.2.8.1.3 *VideoResolution Message*

The `VideoResolution` message which is included in the `ViewRequest` message shall include three fields: the resolution `height` in pixels, the resolution `width` in pixels and the `frame_rate` in frames per second preferred by the RCS client.

Video resolution parameters	Type	Description
`height`	int32	Resolution height in pixels.
`width`	int32	Resolutions width in pixels.
`frame_rate`	int32	Video frame rate in frames per second.

Table 119: VideoResolution parameter definition

```
message VideoResolution {
  int32 height = 1;
  int32 width = 2;
  int32 frame_rate = 3;
}
```

Table 120: VideoResolution protocol buffer definition

3.7.5.2.3.2.8.2 *BandwidthConstraints Message*

The `BandwidthConstraints` message which is included in the Update message, shall include three fields: the `max_bandwidth_kbps` in kilobits per second, the `min_bandwidth_kbps` in kilobits per second and the `start_bandwidth_kbps` in kilobits per second preferred by the RCS client.

Video resolution parameters	Type	Description
`max_bandwidth_kbps`	int32	Maximum bandwidth to transmit the media.
`min_bandwidth_kbps`	int32	Minimum bandwidth to transmit the media.
`start_bandwidth_kbps`	int32	The preferred data rate to start the media transmission.

Table 121: BandwidthConstraints parameter definition

```
message BandwidthConstraints {
  int32 max_bandwidth_kbps = 1;
  int32 min_bandwidth_kbps = 2;
  int32 start_bandwidth_kbps = 3;
}
```

Table 122: BandwidthConstraints protocol buffer definition

3.7.5.2.3.3 Group messages

3.7.5.2.3.3.1 ClientGroupVideoMessage

The `ClientGroupVideoMessage` includes the raw bytes of the `video_chat_message` that is defined in section 3.7.5.2.3.2, which must be sent end to end as is, as well as the `sdp` offer/answer defined as per [RFC8445] and [RFC8841].

The client may include an `UpdateMedia` field or an `IceCandidatesUpdate` field in the `ClientGroupVideoMessage`.

```
message ClientGroupVideoMessage {
  // raw bytes of the video chat message sent by the RCS client.
  bytes video_chat_message = 1;

  string sdp = 2;

  // Some events are not propagated E2E and are sent directly as
  // protocol buffers to the server.
  oneof event {
    UpdateMedia update_media = 101;
    IceCandidatesUpdate ice_candidates_update = 102;
  }
}
```

Table 123: ClientGroupVideoMessage protocol buffer Definition

3.7.5.2.3.3.2 ServerGroupVideoMessage

The `ServerGroupVideoMessage` includes the raw bytes of the `video_chat_message` defined in 3.7.5.2.3.2, that the client sent. The SPN shall send the `video_chat_message` end to end as is. It shall include also the `sdp` offer/answer defined as per [RFC8445] and [RFC8841], the `member_info` field and, if the `VideoChatMessage` contains a `CallInvitation` message, the `call_invite_type` field.

The SPN may include an `UpdateMedia` field or `IceCandidatesUpdate` field in the `ServerGroupVideoMessage`.

```
message ServerGroupVideoMessage {
  // raw bytes of the video chat message sent by RCS client in a
  // ClientGroupVideoMessage.
  // Server shall not modify it and send it E2E.
  bytes video_chat_message = 1;

  string sdp = 2;

  // Set by the RCS service provider network (SPN) only for Group video
calls.
  repeated MemberInfo member_info = 3;

  // Indicates the type of call invite.
  // Only used if the VideoChatMessage contains a CallInvitation.
  CallInviteType call_invite_type = 4;

  // Some events are not propagated E2E and are sent directly as
  // protocol buffers from the server.
  oneof event {
    UpdateMedia update_media = 101;
    IceCandidatesUpdate ice_candidates_update = 102;
  }
}
```

Table 124: ServerGroupVideoMessage protocol buffer Definition

```
{
  "video_chat_message": "...",
  "sdp": "..."

  "member_info": [
    {
      "user_id": {
        "msisdn": "+14155551212"
      },
      "sfu_user_id": {
        "sfu_user_id": "sfu-alice-1a2b"
      },
      "stream_identifiers": {
        "media_stream_ids": [
          {
            "media_stream_id": "alice-audio-track",
            "media_type": "MEDIA_TYPE_AUDIO"
          },
          {
            "media_stream_id": "alice-video-track",
            "media_type": "MEDIA_TYPE_VIDEO"
          }
        ]
      },
      "user_call_state": "USER_CALL_STATE_JOINED"
    },
    {
      "user_id": {
        "msisdn": "+14155551313"
      },
      "sfu_user_id": {
        "sfu_user_id": "sfu-bob-3c4d"
      },
      "stream_identifiers": {
        "media_stream_ids": []
      },
      "user_call_state": "USER_CALL_STATE_NOT_JOINED"
    }
  ],
  "call_invite_type": "CALL_INVITE_TYPE_RINGING"
}
```

Table 125: ServerGroupVideoMessage protocol buffer example

3.7.5.2.3.3.2.1 *MemberInfo Message*

The `MemberInfo` message is included in the `ServerGroupVideoMessage` messages and used only during Group RCS Video Chats by the SPN. It includes four fields: the `user_id` message, `sfu_user_id` message, the `stream_identifiers` message and the `user_call_state` enum used only when a new participant joins a Group RCS Video Chat. These four fields are defined in this section.

```
message MemberInfo {
  RcsId user_id = 1;
  SfuUserId sfu_user_id = 2;
  StreamIdentifiers stream_identifiers = 3;
  UserCallState user_call_state = 4;
}
```

Table 126: MemberInfo protocol buffer definition

3.7.5.2.3.3.2.1.1 *RcsId message*

The `RcsId` message is included in the `MemberInfo` message and it defines one E.164 formatted `msisdn` field. This message represents the identity used for RCS.

```
message RcsId {
  string msisdn = 1;
}
```

Table 127: RcsId protocol buffer definition

3.7.5.2.3.3.2.1.2 *UserCallState enum*

The `UserCallState` enum is part of the `MemberInfo` message and indicates state of the user the current Group RCS Video Chat.

Call state	Description
`USER_CALL_STATE_UNSPECIFIED`	Should not be used.
`USER_CALL_STATE_JOINED`	The participant has joined the call.
`USER_CALL_STATE_NOT_JOINED`	The participant has not joined the call.

Table 128: UserCallState mapping

```
enum UserCallState {
  USER_CALL_STATE_UNSPECIFIED = 0;
  USER_CALL_STATE_JOINED = 1;
  USER_CALL_STATE_NOT_JOINED = 2;
}
```

Table 129: UserCallState enum definition

3.7.5.2.3.3.2.1.3 *SfuUserId message*

The `SfuUserId` message is included in the `MemberInfo` message and it shall include one temporary sfu_user_id string field that is generated by the SPN for this participant for the duration of the call and that will be used to identify streams owners in the SFU.

```
message SfuUserId {
  string sfu_user_id = 1;
}
```

Table 130: SfuUserId protocol buffer definition

3.7.5.2.3.3.2.1.4 *StreamIdentifiers message*

The `StreamIdentifiers` message is included in the `ServerGroupVideoMessage`. It includes one repeated field: the `media_stream_ids` message that is defined in section 3.7.5.2.3.2.8.1.1.

```
message StreamIdentifiers {
  repeated MediaStreamId media_stream_ids = 1;
}
```

Table 131: StreamIdentifiers protocol buffer definition

3.7.5.2.3.3.2.2 *CallInviteType enum*

The `CallInviteType` enum is part of the `ServerGroupVideoMessage` message and indicates the type of call for the current Group RCS Video Chat.

Call Invite Type	Description
`CALL_INVITE_TYPE_UNSPECIFIED`	Should not be used.
`CALL_INVITE_TYPE_RING`	The RCS client should show ringing to the participants as this is a new call.
`CALL_INVITE_TYPE_ONGOING`	The RCS client should not show ringing to the participants as the call is ongoing.

Table 132: CallIniviteType mapping

```
enum CallInviteType CallInviteType {
  CALL_INVITE_TYPE_UNSPECIFIED = 0;
  CALL_INVITE_TYPE_RING = 1;
  CALL_INVITE_TYPE_ONGOING = 2;
}
```

Table 133: CallInviteType enum definition

3.7.5.2.3.4 RCS Video Chat Event Notification

The `VideoChatEventNotification` message shall be included in the body of the SIP message to indicate a JOIN or a LEAVE sent by the RCS client to the RCS SPN contains the `one_to_one_participant_uri`, the URI of the callee of the 1-to-1 RCS Video Chat (if applicable), the `group_conversation_id`, the ID of a Group RCS Video Chat (if applicable) and the `video_chat_event` which indicates if this a JOIN or LEAVE event.

```
message VideoChatEventNotification{
  string one_to_one_participant_uri = 1;  // 1-to-1 RCS Video Chat only
  string group_conversation_id = 2; // Group RCS Video Chat only
  VideoChatEvent video_chat event = 3;
}
```

Table 134 VideoChatEventNotification protocol buffer definition

3.7.5.2.3.4.1 VideoChatEvent enum

The `VideoChatEvent` is an enum part of the `VideoChatEventNotification` message and indicates whether the participant has joined or left a video chat.

Video Chat Event	Description
`EVENT_UNSPECIFIED`	Should not be used.
`EVENT_JOIN`	This indicates that the participant has joined the Video Chat.

Video Chat Event	Description
EVENT_LEAVE	This indicates that the participant has left the Video Chat.

Table 135: VideoChatEvent mapping

```
enum VideoChatEvent {
  EVENT_UNSPECIFIED = 0;
  EVENT_JOIN = 1;
  EVENT_LEAVE = 2;
}
```

Table 136: VideoChatEvent enum definition

3.7.5.2.4 Timers

RCS Video Chat defines a number of timers that are defined in Table 137. All timers are multiples of Timer_T1, which is defined in Annex A.1.6.1.

Name	Timer	Value
Timer_T1	As defined in Annex A.1.6.1	Timer_T1
Timer_Alpha	CallInivitation sent until AckInvitation received	Max value: 64*T1
Timer_Beta	CallInvitation sent until final response (AcceptInvitation or RejectInvitation)	Max value: 64*T1
Timer_Gamma	CallInvitation received is too old to ring recipient	Expiry value: 64*T1
Timer_Delta	Timer to establish peer-to-peer WebRTC connection before declaring call failed	Max value: 20*T1
Timer_Epsilon	After UE disconnects, timer on other UE waiting on a reconnect	Max value: 20*T1
Timer_Zeta	After UE disconnects, timer on SFU waiting on a reconnect	Max value: 20*T1

Table 137: RCS Video Chat Timers

3.7.5.2.4.1 Timer_Alpha

The caller's RCS client shall start Timer_Alpha when it sends the CallInvitation protocol buffer to the callee. The RCS client shall stop the timer when the caller's RCS client receives an AckInvitation from the callee. If the timer expires before receiving an AckInvitation, the caller's RCS client shall send a CancelInvitation and consider the call not answered.

3.7.5.2.4.2 Timer_Beta

The caller's RCS client shall start Timer_Beta when it sends the CallInvitation protocol buffer to the callee. The caller's RCS client shall stop the timer when the caller's RCS client receives either an AcceptInvitation or RejectInvitation from a callee. If the timer expires before receiving either an AcceptInvitation or RejectInvitation, the caller's RCS client shall send a CancelInvitation and consider the call not answered.

3.7.5.2.4.3 Timer_Gamma

The calllee RCS client shall calculate Timer_Gamma from the time of the CallInvitation to the current time. If the value of the timer is less than 64*Timer_T1, then the callee's RCS client shall ring the caller's device until (CallInvitation timestamp + 64 * Timer_T1). If the value of the timer is greater than 64*Timer_T1, then the callee's RCS client shall not ring the callee's device and consider the call missed.

3.7.5.2.4.4 Timer_Delta

The caller's RCS client shall start Timer_Delta from the time the caller's RCS client receives an AcceptInvitation protocol buffer. The caller's RCS client shall stop Timer_Delta when the media channel connects. 1. If the timer expires before the media channel connects, then the RCS client shall consider the call to have failed.

Similarly, the callee's RCS client shall start Timer_Delta at the time of sending the AcceptInvitation. The calle's RCS client shall stop the Timer_Delta when media channel connects. If the timer expires before the media channel connects, then the callee's RCS client shall consider the call to have failed.

3.7.5.2.4.5 Timer_Epsilon

The RCS client shall start Timer_Epsilon when either RCS client in a 1-to-1 RCS Video Chat disconnects from a call. The RCS client shall stop the timer when the disconnected RCS client reconnects. If the disconnected RCS client does not reconnect when the timer expires, the other RCS client shall consider the call as failed.

3.7.5.2.4.6 Timer_Zeta

The SFU shall start Timer_Zeta when it discovers that an RCS client in a Group RCS Video Chat has disconnected. The SFU shall stop Timer_Zeta when the disconnected RCS client reconnects. If the disconnected RCS client does not reconnect to the SFU when the timer expires, the SFU shall consider the RCS client to be disconnected from the Group RCS Video Chat.

3.7.5.2.5 RCS Video Chat Event Notifications

When an RCS client connects via the media channel to an RCS Video Chat as per section 3.7.5.2.2, the RCS client shall send a SIP MESSAGE request to the URI provided in the MIVC REPORTING URI client configuration parameter (if provided) as defined in Annex A.1.3. The body of the SIP MESSAGE shall conform to the format defined in section 3.7.5.2.3.4 with the EVENT field set to JOIN.

When an RCS client disconnects via the media channel from an RCS Video Chat as per section 3.7.5.2.3, the RCS client shall send a SIP Message to the URI provided in the MIVC REPORTING URI client configuration parameter (if provided) as defined in Annex A.1.3. The body of the SIP MESSAGE shall conform to the format defined in section 3.7.5.2.3.4 with the EVENT field set to LEAVE.

3.7.5.3 Media

3.7.5.3.1 Media Connection

Once the RCS Video Chat session (described in section 3.7.5.2.2) is established and the nominated ICE pair (described in section 2.3 of [RFC8445]) is selected, the RCS client shall set up WebRTC connections as per [RFC5764] and [RFC8832]. The RCS client shall apply all security considerations described in [RFC8827].

For media transport, the RCS client shall implement [RFC8834] and [RFC8835].

For Group RCS Video Chats, the RCS clients shall connect to the SFU (as defined in [RFC9605]) provided by the SPN of the Controlling Function (CF) for the Group Chat. If the RCS Group Chat is subject to E2EE, the RCS client shall follow procedures in [GSMA PRD-RCC.16] to encrypt the media transport.

3.7.5.3.2 Codec Negotiation

The codec negotiation for RCS Video Chat shall be conducted over SDP using the Offer / Answer model, as defined in [RFC8866]. This allows for the flexibility to extend and add new codecs without requiring changes to this specification.

When generating the session offer or answer, the RCS client shall include the 3GPP codecs as specified in [GSMA PRD-NG.114] section 3.2 for audio and [GSMA PRD-NG.114] section 3.3 for video and the WebRTC codecs for both audio [RFC7874] and video [RFC7742] streams.

The RCS client shall prioritise the 3GPP codecs as specified in [GSMA PRD-NG.114] and prioritise them over the WebRTC codecs.

3.7.5.4 1-to-1 Procedures

3.7.5.4.1 Initiating a Call

To initiate a 1-to-1 RCS Video Chat, the caller’s RCS client shall:

1. Set up the media peer connection and gather ICE candidates as per section 3.7.5.2.1.
2. Format the CallInvitation message as per section 3.7.5.2.3.2.1.
3. Wrap the CallInvitation message into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM to the callee’s RCS client in an MSRP message as per section 3.2.3.
5. Start Timer_Alpha and Timer_Beta as described as per section 3.7.5.2.4.1 and 3.7.5.2.4.2.
6. Show the caller that the call is connecting.
7. Wait for the AckInvitation message.

This is illustrated in Figure 28.

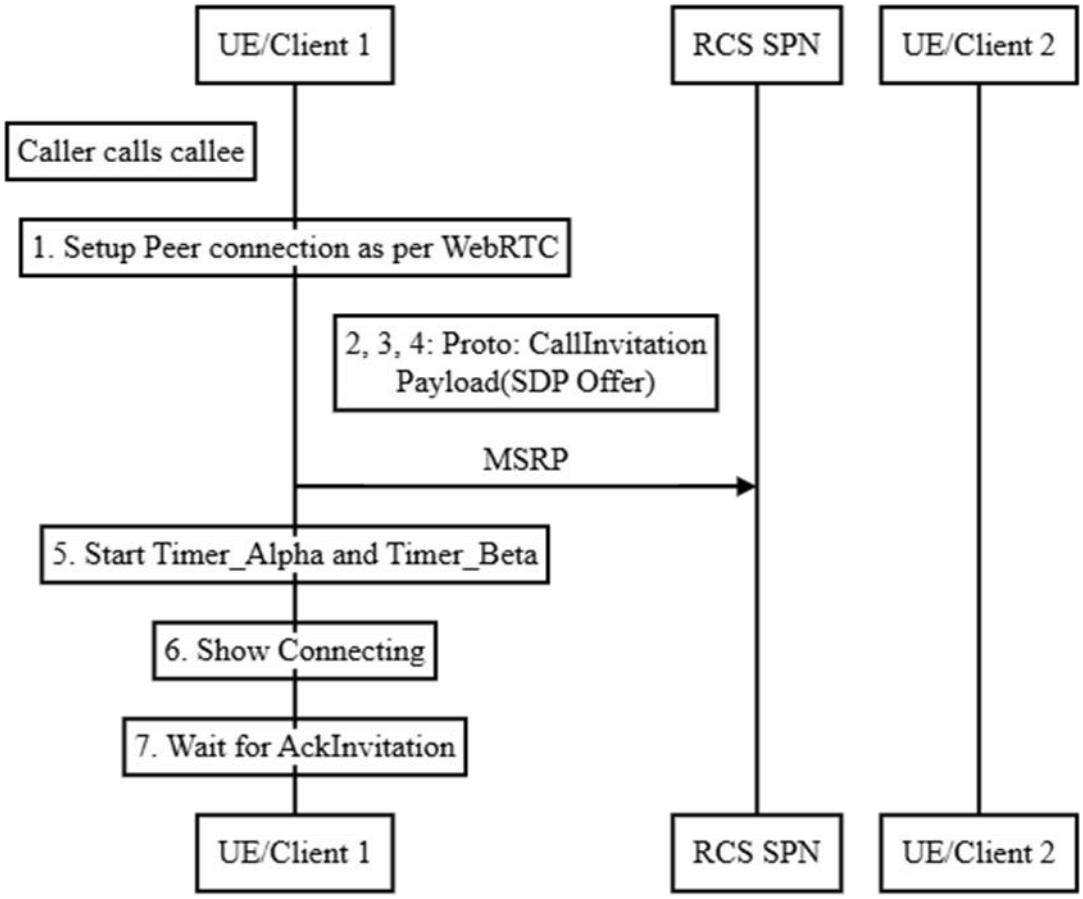


Figure 28: Caller initiating a 1-to-1 RCS Video Chat

3.7.5.4.2 Callee online during Call initiation

After the 1-to-1 RCS Video Chat call initiation procedure in section 3.7.5.4.1 has completed, upon receiving the CallInvitation message containing the SDP offer the callee RCS client shall:

1. Verify that CallInvitation is not expired by comparing CallInvitation sending timestamp with the current time and if it's less than or equal to Timer_Gamma as described in 3.7.5.2.4.3, continue the procedure.
2. Format the AckInvitation message as per section 3.7.5.2.3.2.2.
3. Wrap the AckInvitation message into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message as per section 3.2.3.
5. Alert the callee of the incoming call.
6. Wait for the callee to answer or reject the 1-to-1 RCS Video Chat.

Upon receiving the AckInvitation message the caller's RCS client shall:

7. Show the caller that the call is alerting on the callee's RCS client.
8. Stop Timer_Alpha as described in 3.7.5.2.4.1.

This is illustrated in Figure 29.

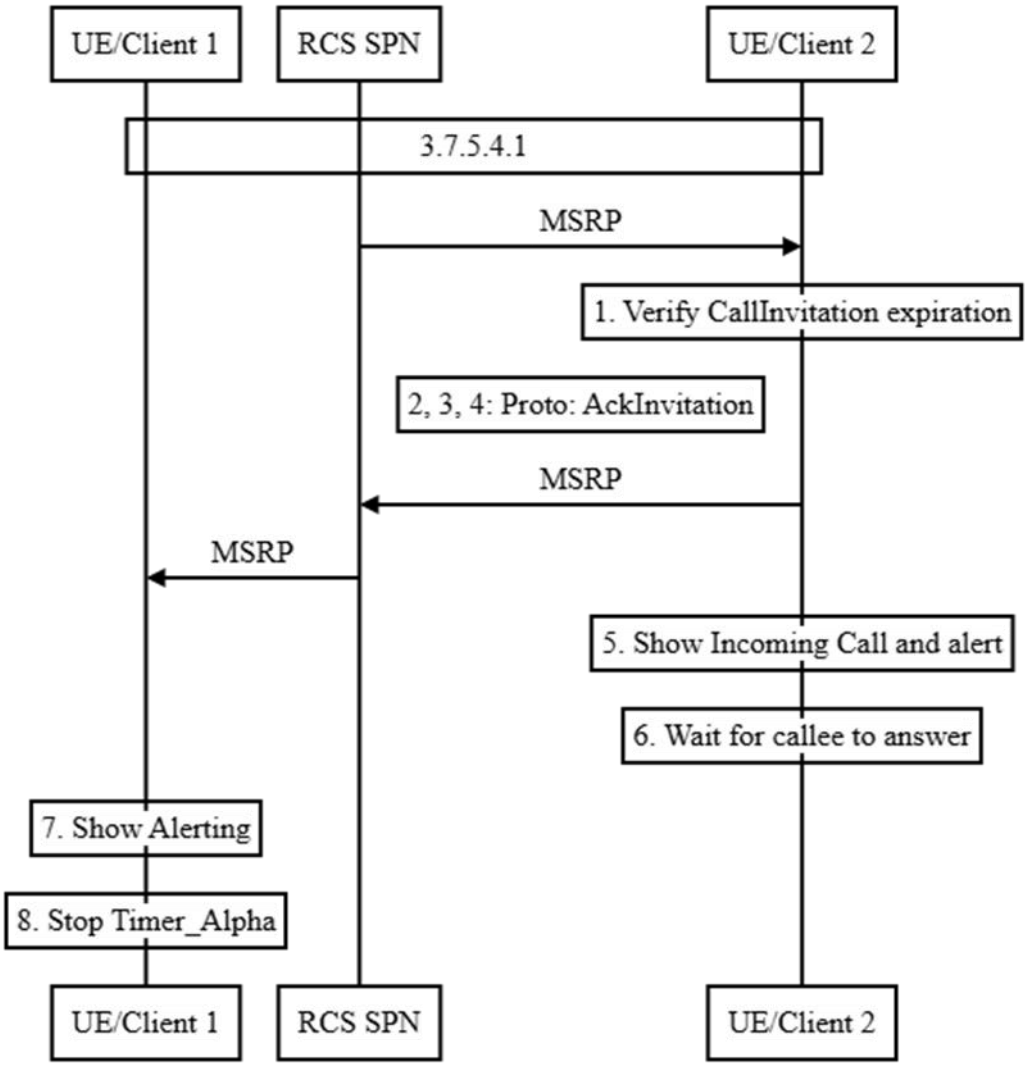


Figure 29: Callee is online during in 1-to-1 RCS Video Chat initiation

3.7.5.4.3 Call Answered

After the AckInvitation response procedure in section 3.7.5.4.2 has completed, and after the callee chooses to answer the 1-to-1 RCS Video Chat, the callee's RCS client shall:

1. Stop the alerting
2. Set up the media peer connection and gather ICE candidates as per section 3.7.5.2.2.
3. Format the AcceptInvitation message as per section 3.7.5.2.3.2.3.
4. Wrap the AcceptInvitation message into a CPIM container as per section 3.7.5.2.3.1.
5. Send CPIM in an MSRP message as per section 3.2.3.
6. Start Timer_Delta as described in 3.7.5.2.4.4.
7. Attempt to connect the media to the caller's RCS client as per section 3.7.5.3.1.
8. If the media session is successfully connected and Timer_Delta has not expired, continue with the procedure.
9. Stop Timer_Delta.

10. Send the JOIN RCS Video Chat notification event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

Upon receiving the AcceptInvitation message containing the SDP answer the caller's RCS client shall:

11. Stop Timer_Beta.
12. Show that the callee has answered the call
13. Start Timer_Delta as described in 3.7.5.2.4.4.
14. Attempt to connect the media to the callee's RCS client as per section 3.7.5.3.1.
15. If the media session is successfully connected and Timer_Delta not expired continue with the procedure.
16. Stop Timer_Delta.
17. Send the JOIN RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

This is illustrated in Figure 30.

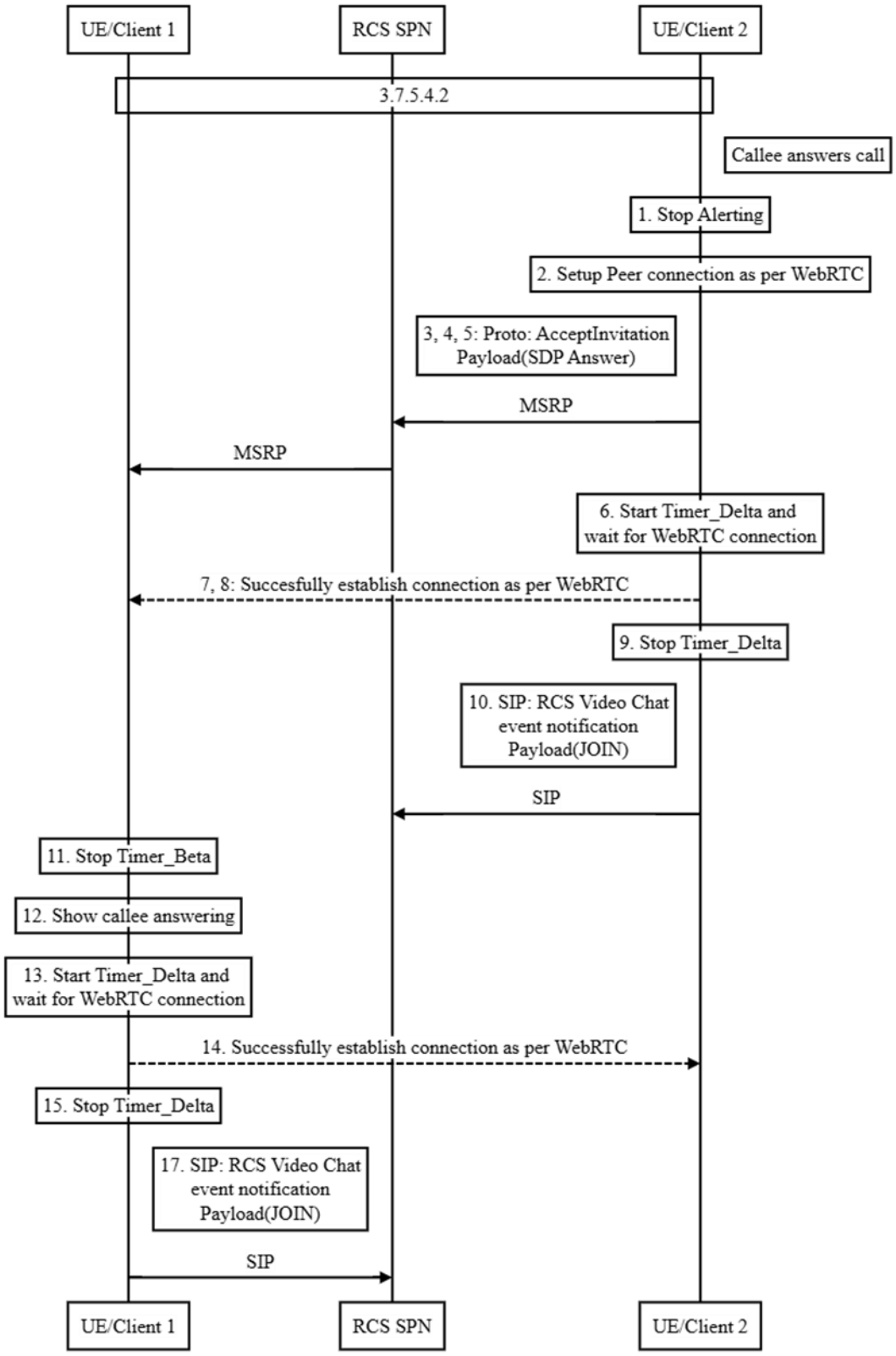


Figure 30: Callee answering a 1-to-1 RCS Video Chat

3.7.5.4.4 Call Hung up

After the answering procedure in section 3.7.5.4.3 has completed, to hangup a 1-to-1 RCS Video Chat, the RCS client of the user who chose to hang up shall:

1. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_USER_INITIATED as defined in section 3.7.5.2.3.2.5.1.
2. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.3.
4. Close the media connection.
5. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
6. Show the caller that the call has ended

Upon receiving the Hangup message the other RCS client shall:

7. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_USER_INITIATED as defined in section 3.7.5.2.3.2.5.1.
8. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
9. Send CPIM in an MSRP message as per section 3.2.3.
10. Close the media connection.
11. Show the callee that the call has ended.
12. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

This is illustrated in Figure 31.

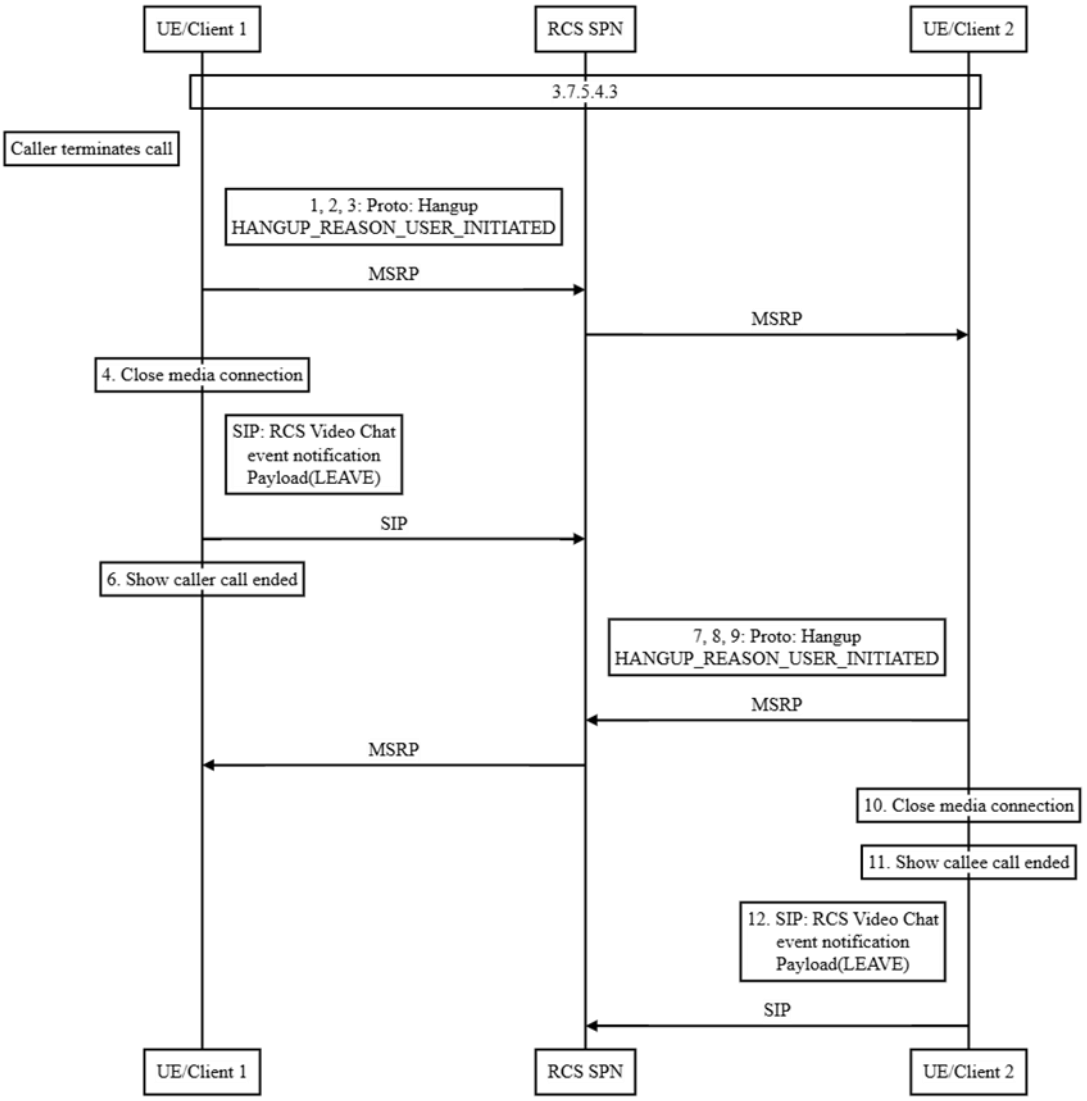


Figure 31: Caller hanging up a 1-to-1 RCS Video Chat

3.7.5.4.5 Call Rejected

After the AckInvitation response procedure in section 3.7.5.4.2 has completed, to reject a 1-to-1 RCS Video Chat, the callee's RCS client shall:

1. Format the RejectInvitation message as per section 3.7.5.2.3.2.4 and include a RejectReason REJECT_REASON_USER_REJECT as defined in section 3.7.5.2.3.2.4.1.
2. Wrap the RejectInvitation message into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.3.
4. Stop the alerting.
5. Show the callee that the call is rejected.

Upon receiving the RejectInvitation message the caller's RCS client shall:

6. Stop Timer_Alpha and Timer_Beta.
7. Show the caller that the call was rejected and stop the alerting indication.

This is illustrated in Figure 32.

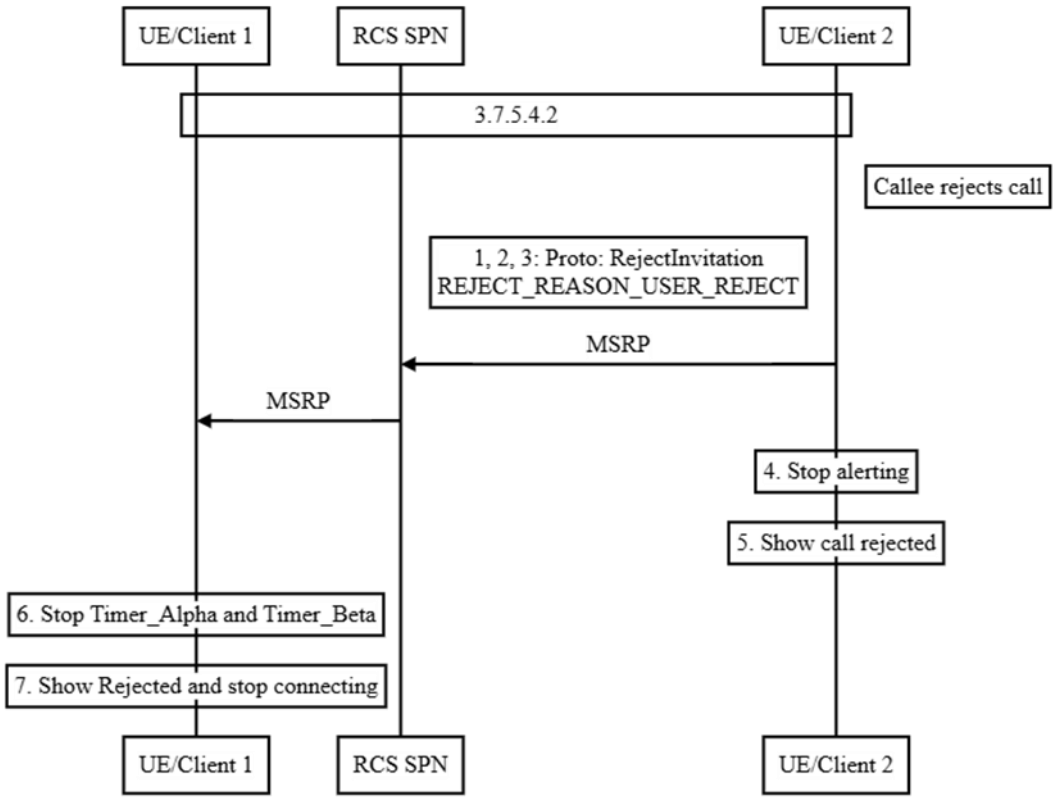


Figure 32: Callee rejecting a 1-to-1 RCS Video Chat

3.7.5.4.6 Call Cancelled

After the AckInvitation response procedure in section 3.7.5.4.2 has completed, to cancel a 1-to-1 RCS Video Chat prior to receiving an AcceptInvitation, the caller's RCS client shall:

1. Format the CancelInvitation message as per section 3.7.5.2.3.2.6 and include a CancelReason CANCEL_REASON_USER_CANCEL as defined in section 3.7.5.2.3.2.6.1.
2. Wrap the CancelInvitation message into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.3.
4. Stop the connecting indication.
5. Stop Timer_Beta.

Upon receiving the CancelInvitation message the recepient's RCS client shall:

6. Stop alerting.
7. Show the callee that the call was missed

This is illustrated in Figure 33.

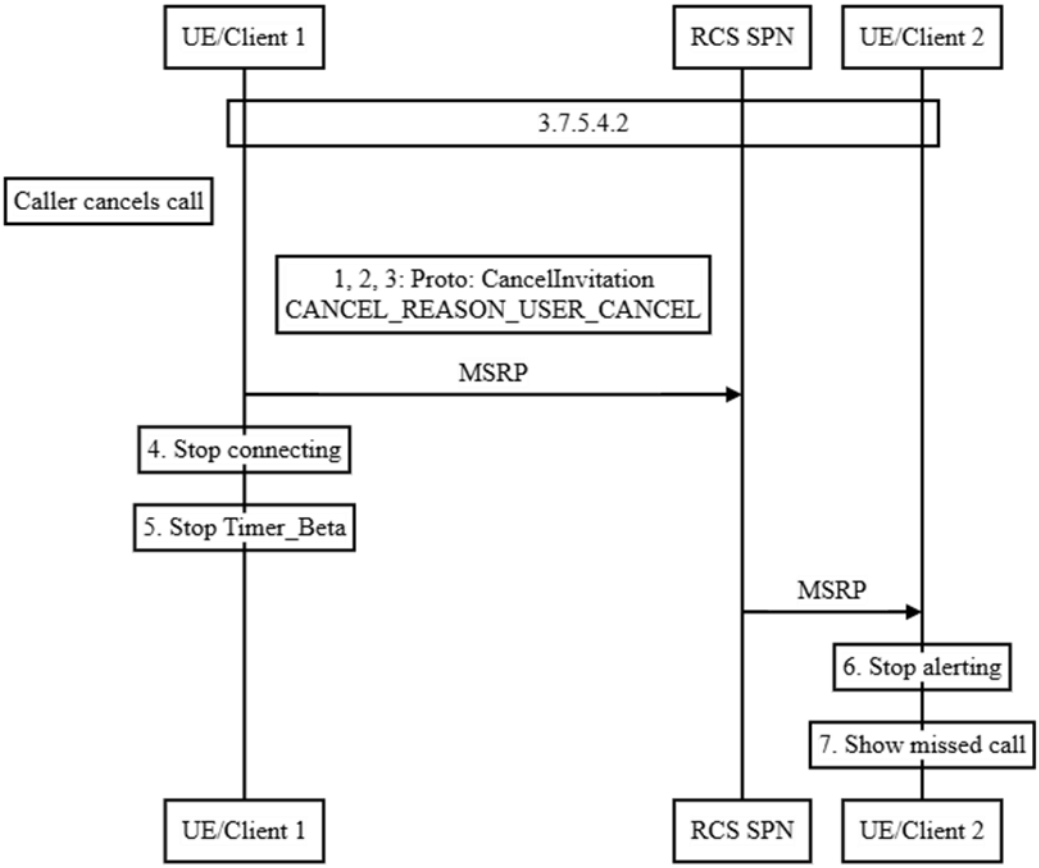


Figure 33: Caller cancels RCS 1-to-1 RCS Video Chat

3.7.5.4.7 Call Ignored

During the AckInvitation response procedure described in 3.7.5.4.2, when Timer_Beta expires, the caller's RCS client shall:

1. Format the CancelInvitation message as per section 3.7.5.2.3.2.6 and include a CancelReason CANCEL_REASON_TIMEOUT as defined in section 3.7.5.2.3.2.6.1.
2. Wrap the CancelInvitation message into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.3.
4. Show the caller that the call is missed.

Upon receiving the CancelInvitation message the callee's RCS client shall:

5. Show the callee that the call was missed
6. Stop alerting.

This is illustrated in Figure 34.

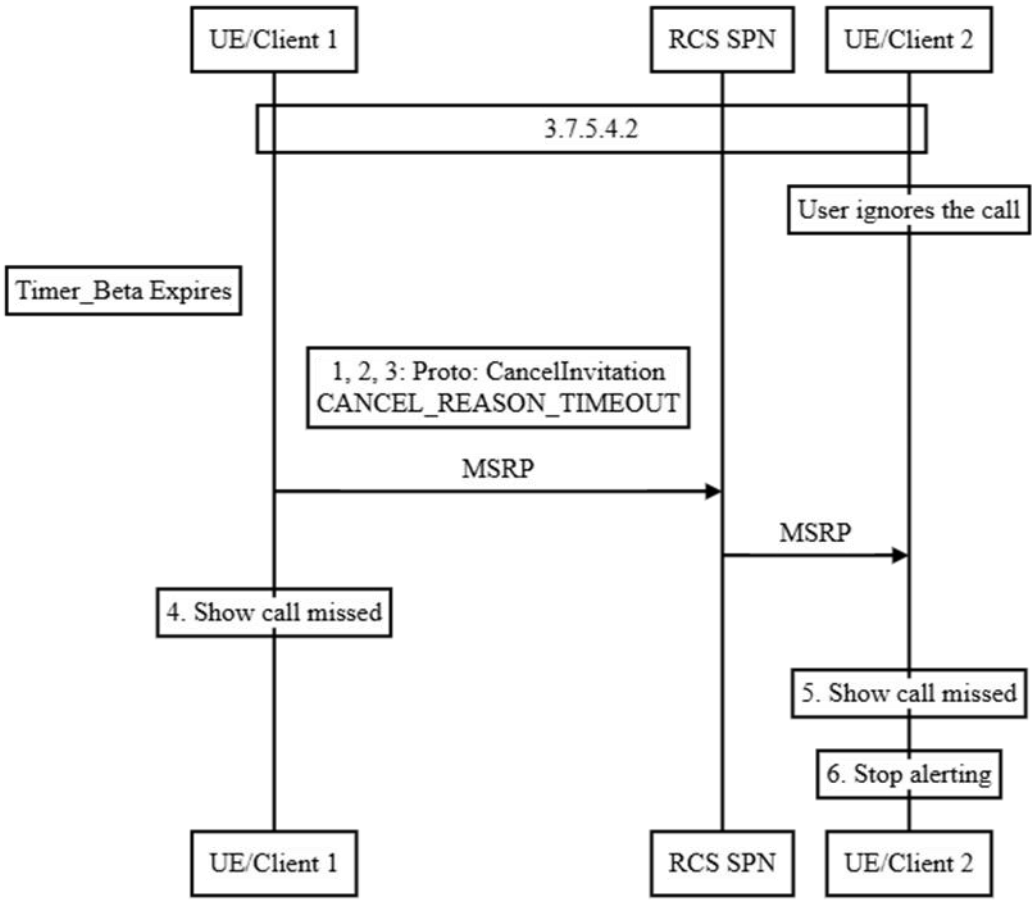


Figure 34: Callee ignores the 1-to-1 RCS Video Chat

3.7.5.4.8 Error Handling

3.7.5.4.8.1 Callee Offline

3.7.5.4.8.1.1 Callee Offline during original MSRP session establishement

During the 1-to-1 RCS Video Chat call initiation procedure described in 3.7.5.4.1, when Timer_Alpha expires, the caller's RCS client shall:

1. Stop Timer_Beta.
2. Format the CancelInvitation message as per section 3.7.5.2.3.2.6 and include a CancelReason CANCEL_REASON_TIMEOUT as defined in section 3.7.5.2.3.2.6.1.
3. Wrap the CancelInvitation message into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message as per section 3.2.3.
5. Show the caller the call is missed.

This is illustrated in Figure 35.

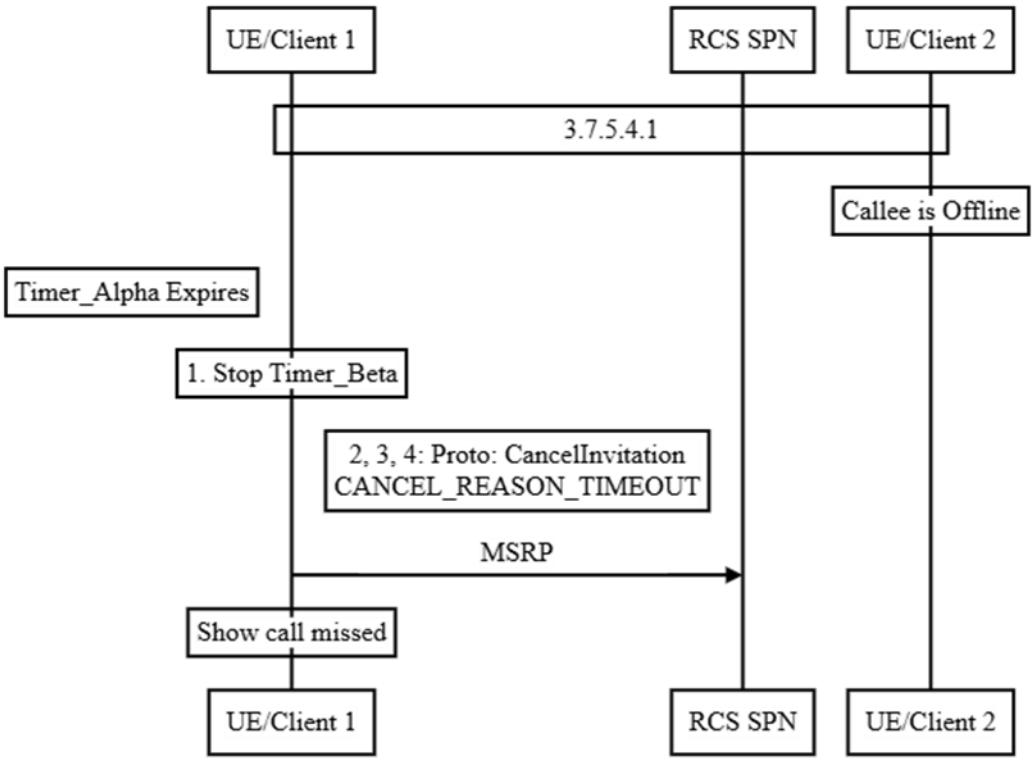


Figure 35: Caller discovers through that callee is offline during call initiation

3.7.5.4.8.1.2 User offline during media establishment

During the answering procedure described in 3.7.5.4.3, when Timer_Delta expires for either RCS client, the RCS client shall:

1. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_CLIENT_TIMEOUT as defined in section 3.7.5.2.3.2.5.1.
2. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.3.
4. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
5. Show the caller that the call has failed.

Upon receiving the Hangup message the other RCS client shall:

6. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_CLIENT_TIMEOUT as defined in section 3.7.5.2.3.2.5.1.
7. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
8. Send CPIM in an MSRP message as per section 3.2.3.
9. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
10. Show the callee that the call has failed.

This is illustrated in Figure 36.

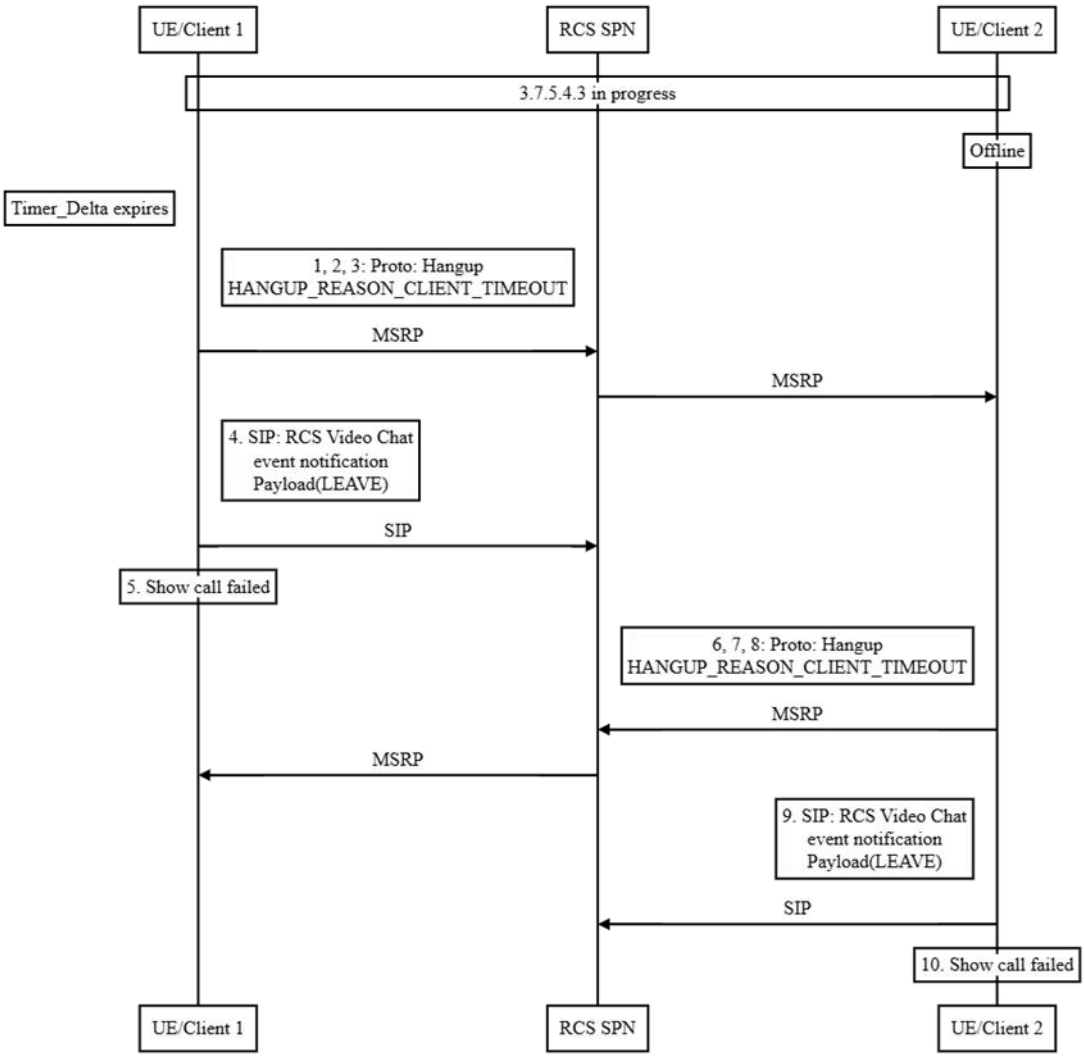


Figure 36: Caller detected that the media establishment failed due to callee being offline and hangs up the call.

3.7.5.4.8.1.3 Callee back online receiving a CallInvitation and CancelInvitation from store and forward

Upon receiving the CallInvitation and CancelInvitation from store and forward described in 3.7.5.4.6, the callee's RCS client shall:

1. Compare CallInvitation sending timestamp with the current time and if it's less than the Timer_Gamma as per section 3.7.5.2.4.3, continue the procedure.
2. Show the callee that the call is missed.

This is illustrated in Figure 37.

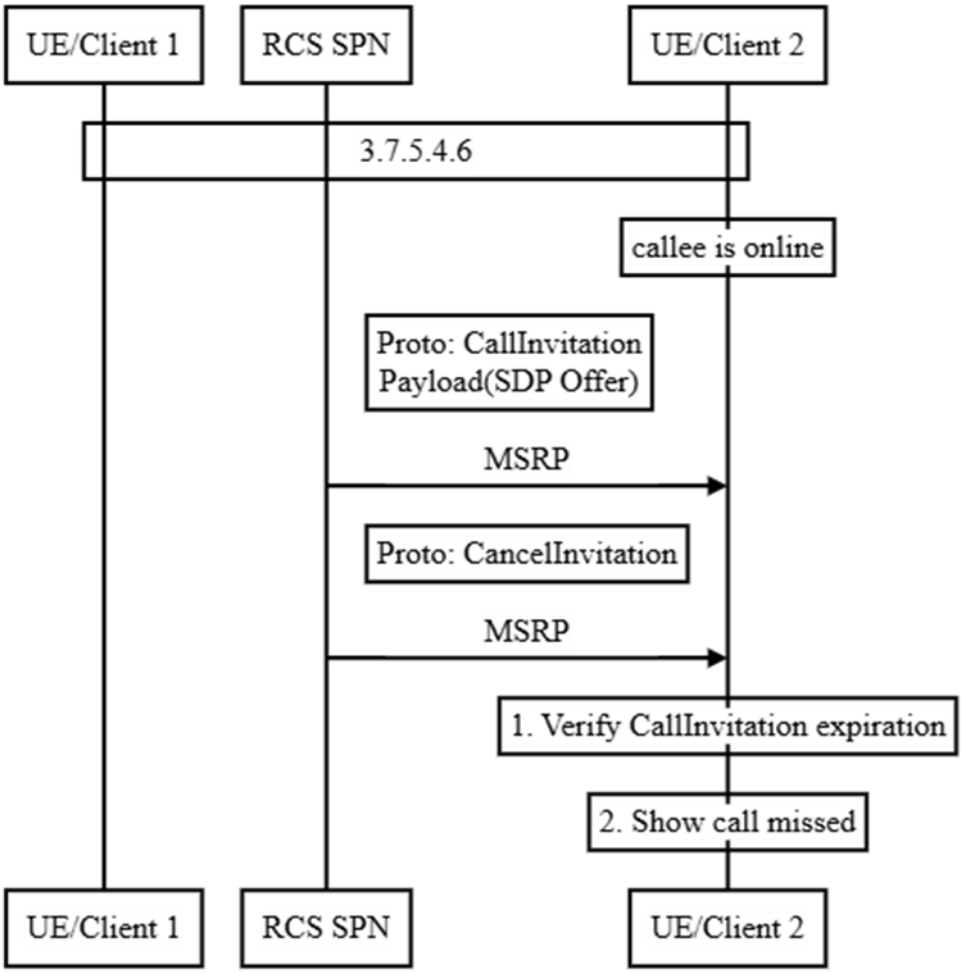


Figure 37: callee receiving a CallInviation and CancelInvation from store and forward

3.7.5.4.8.1.4 Callee back online Receiving a CallInvitation from store and forward Timer_Gamma expired

Upon receiving the CallInvitation from store and forward, the callee's RCS client shall:

1. Compare CallInvitation sending timestamp with the current time and if it's greater than Timer_Gamma as described in 3.7.5.2.4.3, continue the procedure.
2. Show the callee that the call is missed.

This is illustrated in Figure 38.

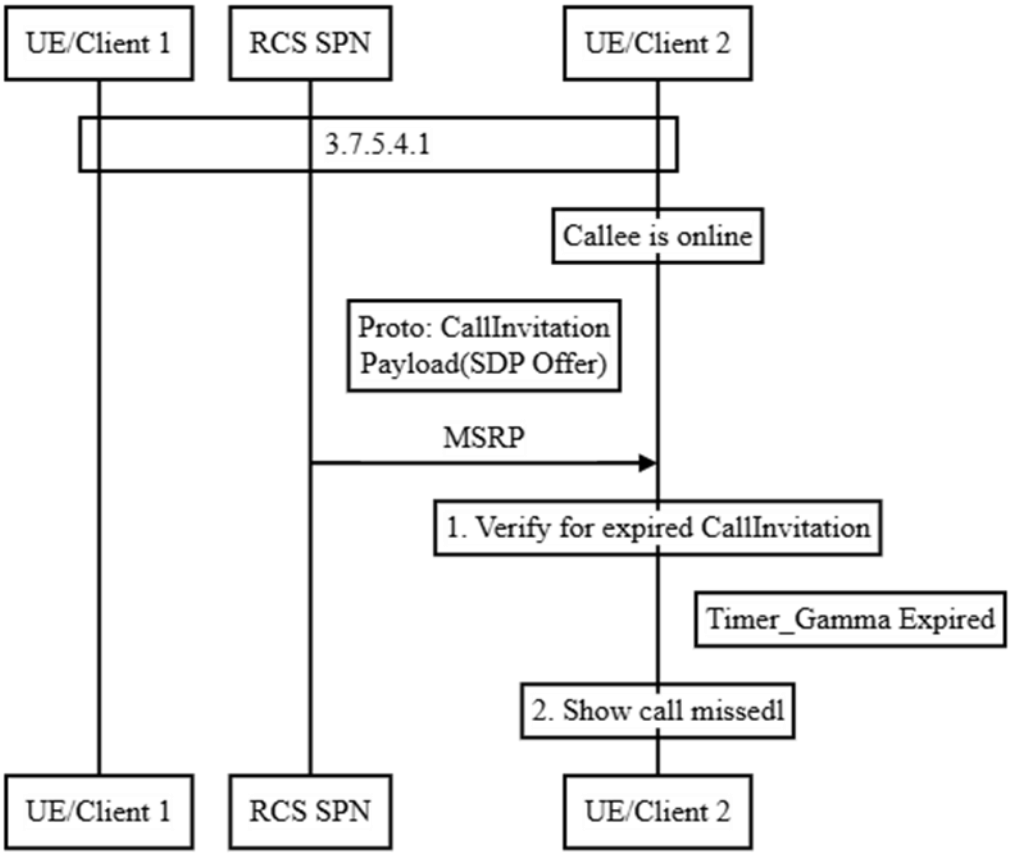


Figure 38: callee receives expired CallInvitation

3.7.5.4.8.1.5 Callee back online receiving a CallInvitation from store and forward Timer_Gamma valid

Follow the callee's procedure as per section 3.7.5.4.2.

3.7.5.4.8.1.6 RCS client detects that the other party's media is disconnected

After the AckInvitation response procedure in section 3.7.5.4.2 has completed, if either RCS client detects a media disconnection from the other RCS client, the RCS client shall:

1. Start Timer_Epsilon as described in 3.7.5.2.4.5.
2. Wait for the Timer_Epsilon.

Upon Timer_Epsilon expiration, the RCS client shall:

3. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_CLIENT_TIMEOUT as defined in section 3.7.5.2.3.2.5.1.
4. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
5. Send CPIM in an MSRP message as per section 3.2.3.
6. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
7. Show the caller that the call has failed.

Upon receiving the Hangup message the other RCS client shall:

8. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_CLIENT_TIMEOUT as defined in section 3.7.5.2.3.2.5.1.
9. Wrap the Hangup message into a CPIM container as per section 3.7.5.2.3.1.
10. Send CPIM in an MSRP message as per section 3.2.3.
11. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
12. Show the callee that the call has failed.

This is illustrated in Figure 39.

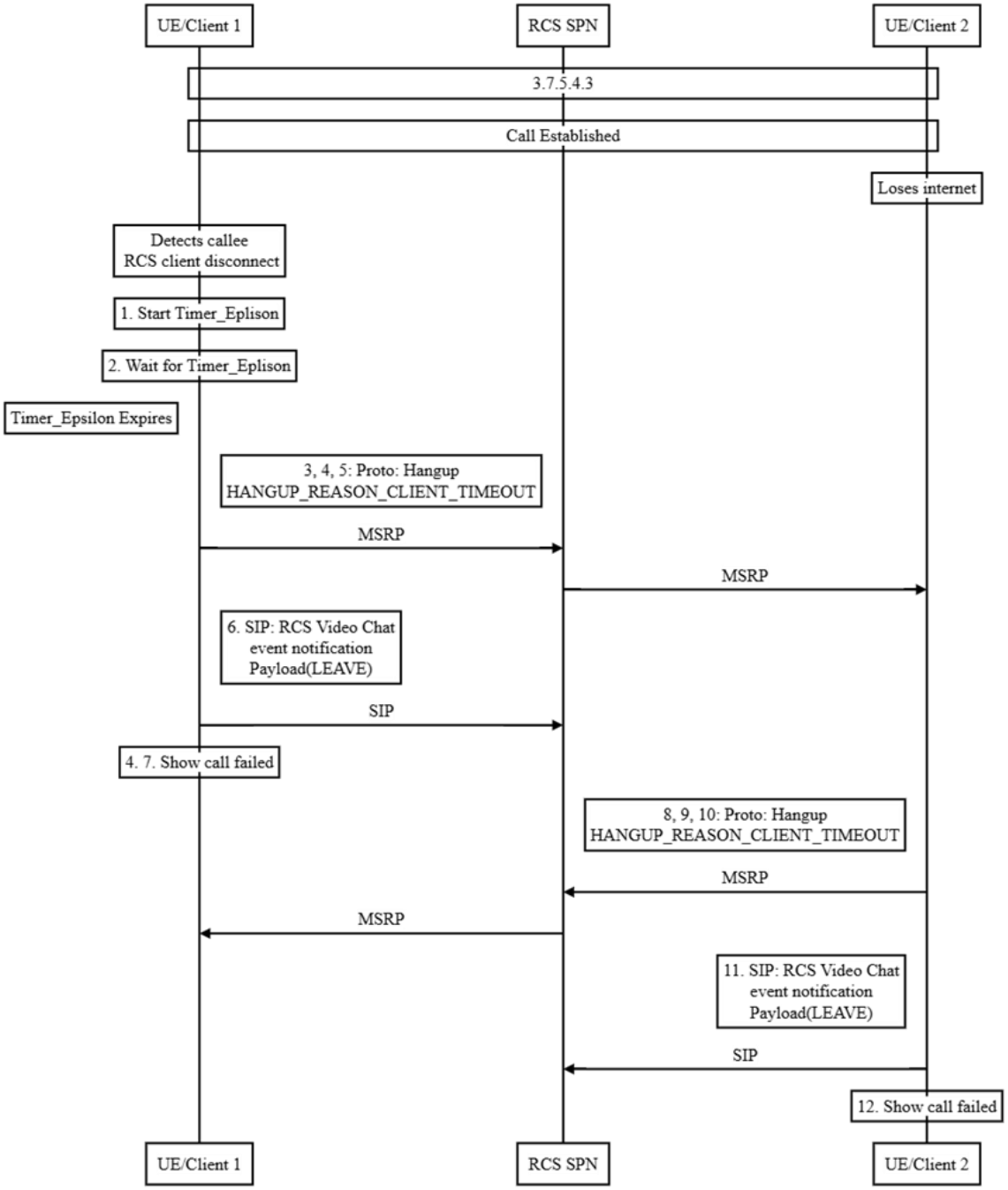


Figure 39: Media disconnected during 1-to-1 RCS Video Chat

3.7.5.4.8.2 IP Switching /Handover

After the answering procedure in section 3.7.5.4.3 has completed, when either RCS client detects an IP address switch or change during an active 1-to-1 RCS Video Chat, the RCS client shall:

1. Show the caller that the call is reconnecting.
2. Set up the media peer connection
3. Gather ICE candidates as per section 3.7.5.2.2.
4. Format the IceCandidatesUpdate message as per section 3.7.5.2.3.2.7.

5. Wrap the IceCandidatesUpdate message into a CPIM container as per section 3.7.5.2.3.1.
6. Send CPIM in an MSRP message as per section 3.2.3.
7. Start Timer_Delta as described in 3.7.5.2.4.4.
8. Attempt to connect the media to the callee's RCS client as per section 3.7.5.3.1.
9. If the media session is successfully connected and Timer_Delta has not expired continue with the procedure.
10. Stop Timer_Delta

Upon receiving the IceCandidatesUpdate the RCS Client of the callee shall:

11. Show the callee that the call is reconnecting.
12. Start Timer_Delta as described in 3.7.5.2.4.4.
13. Attempt to connect the media to the caller's RCS client as per section 3.7.5.3.1.
14. If the media session is successfully connected and Timer_Delta has not expired continue with the procedure.
15. Stop Timer_Delta

This is illustrated in Figure 40.

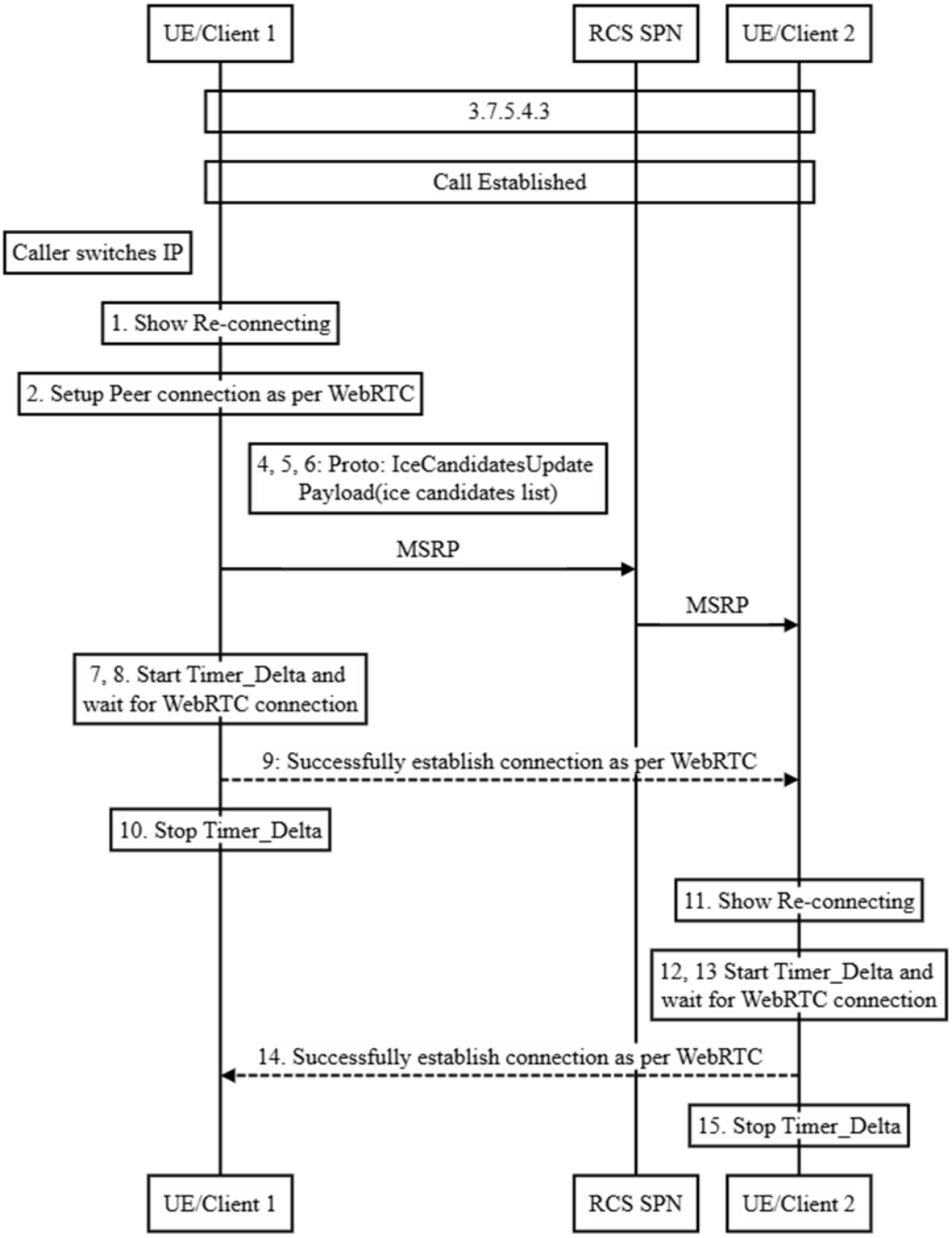


Figure 40: Caller sending an IceCandidatesUpdate after switching IP

3.7.5.4.8.3 Media capabilities update

After the answering procedure in section 3.7.5.4.3 has completed, when either RCS client wishes to indicate a change to its media capabilities during an active 1-to-1 RCS Video Chat, the RCS client shall:

1. Format the UpdateMedia message with its new media parameters as per section 3.7.5.2.3.2.8.
2. Wrap the UpdateMedia message into a CPIM container per 3.7.5.2.3.1.

3. Send CPIM in an MSRP message as per section 3.2.3.

This is illustrated in Figure 41.

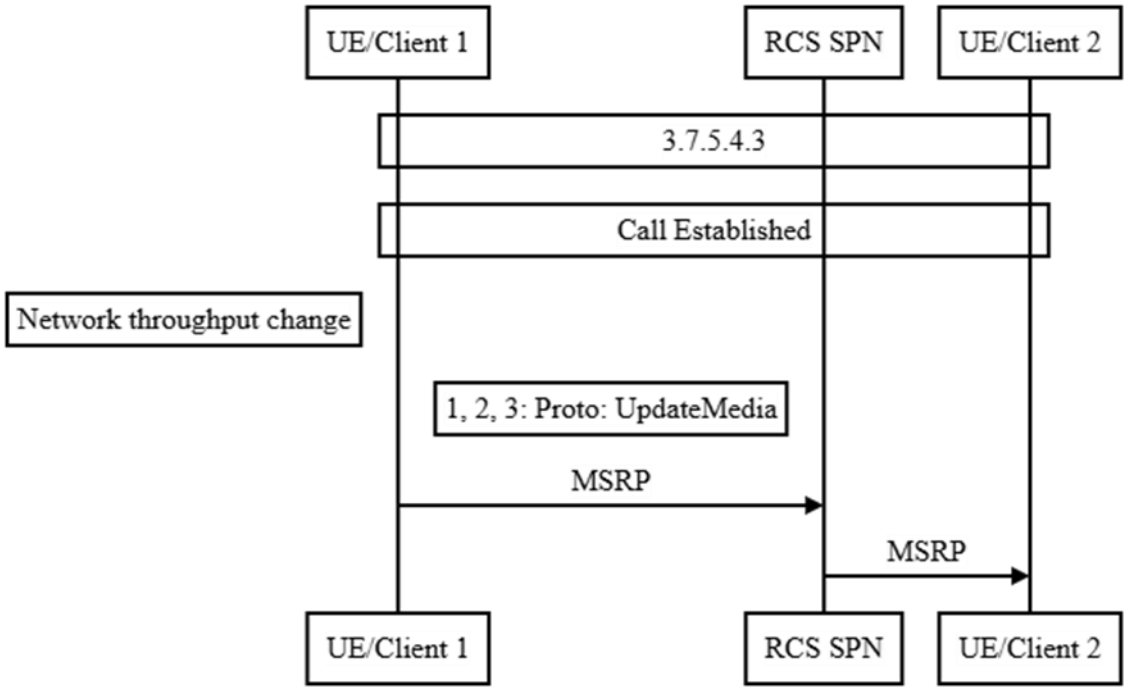


Figure 41: Caller updating media capabilities after a network throughput change

3.7.5.5 Group Procedures

3.7.5.5.1 Initiating a Call

To initiate a Group RCS Video Chat, the caller's RCS client shall:

1. Set up the media peer connection and gather ICE candidates as per section 3.7.5.2.2.
2. Format the CallInvitation message as per section 3.7.5.2.3.2.1.
3. Wrap the CallInvitation message into a ClientGroupVideoMessage and include the SDP offer as per section 3.7.5.2.3.3.1.
4. Wrap the ClientGroupVideoMessage into CPIM container as per section 3.7.5.2.3.1.
5. Send CPIM in an MSRP message to the SPN as per section 3.2.4.
6. Start Timer_Alpha and Timer_Beta as described as per section 3.7.5.2.4.1 and 3.7.5.2.4.2.
7. Show caller that the call is connecting.
8. Wait for the ServerGroupVideoMessage message and the first AckInvitation message

Upon receiving the ClientGroupVideoMessage message containing the SDP offer the Controlling Function shall:

9. Verify that CallInvitation is not expired by comparing the CallInvitation sending timestamp with the current time and if it is less than or equal to Timer_Gamma as described in 3.7.5.2.4.3, continue the procedure.

10. Verify that this message contains a unique call_id.
11. Build the member_info list and define the sfu_user_id for the Group RCS Video Chat as per section 3.7.5.2.3.3.2.1.
12. Gather ICE candidates and compose the SDP offer for each Group Chat participant as per section 3.7.5.2.2.
13. Format the ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
14. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
15. Send CPIM in an MSRP message as per section 3.2.4 to the caller's RCS client.
16. Include the CallInvitation message sent from the caller and the member_info list into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
17. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
18. Send CPIM in an MSRP message to each participant as per section 3.2.4.
19. Start Timer_Delta
20. Connect the media to the caller as per section 3.7.5.3.1. If the media session is successfully connected and Timer_Delta has not expired, continue with the procedure.
21. Stop Timer_Delta

Upon receiving the ServerGroupVideoMessage, the caller's RCS client shall:

22. Start Timer_Delta
23. Connect the media to the SFU as per section 3.7.5.3.1. If the media session is successfully connected and Timer_Delta has not expired, continue with the procedure.
24. Stop Timer_Delta

Upon receiving the ServerGroupVideoMessage, the Group Chat participants' RCS clients shall:

25. Alert the callees' of the incoming call.

This is illustrated in Figure 42.

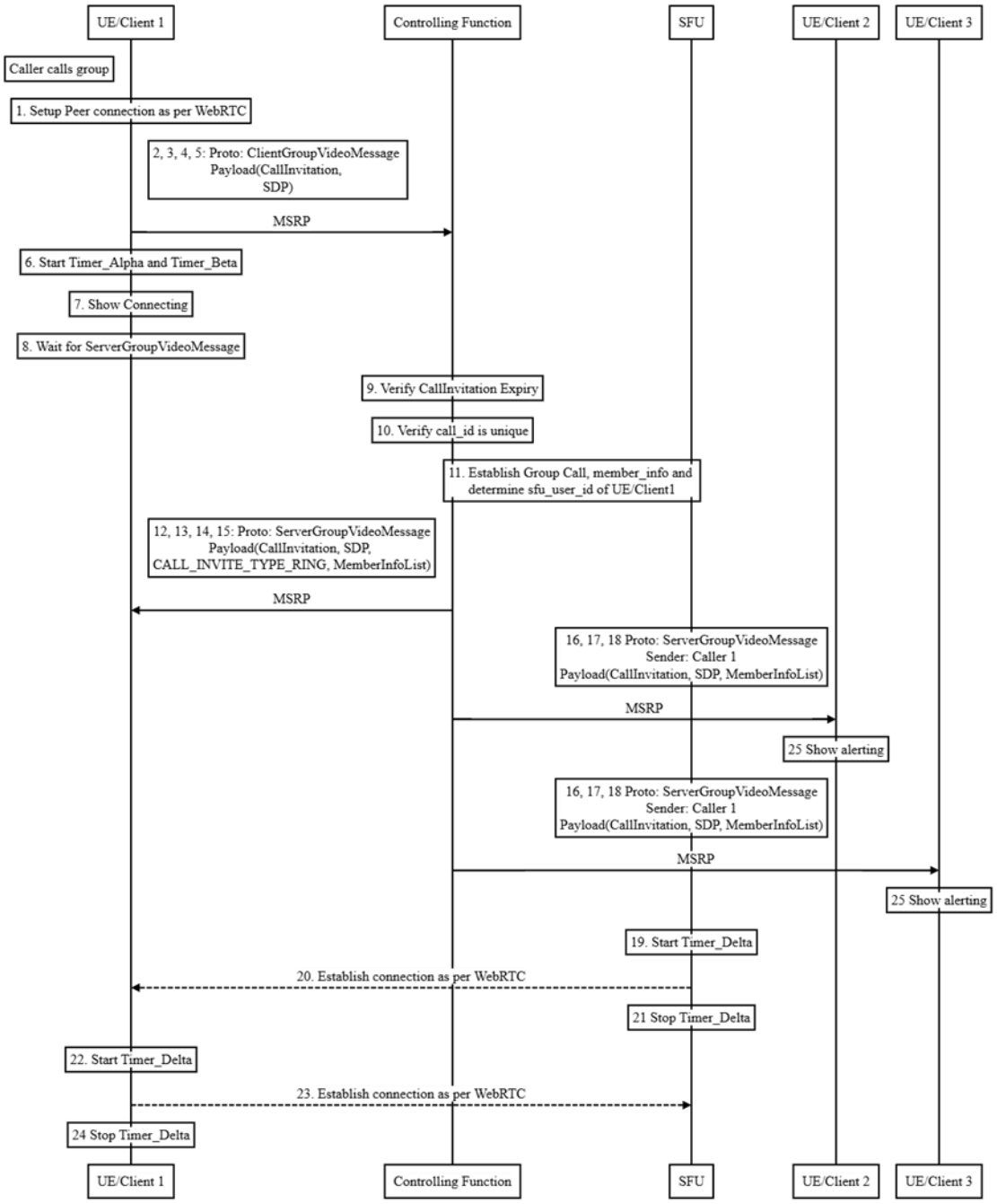


Figure 42: Caller initiating a Group RCS Video Chat

3.7.5.5.2 Call Acknowledged

After the Group RCS Video Chat call initiation procedure in section 3.7.5.5.1 has completed, upon receiving the CallInvitation, RCS clients for participants in the Group Chat shall:

1. Format the AckInvitation message as per section 3.7.5.2.3.2.2.
2. Wrap the AckInvitation message into a ClientGroupVideoMessage as per section 3.7.5.2.3.3.1.
3. Wrap the ClientGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message to the SPN as per section 3.2.4.

Upon receiving the ClientGroupVideoMessage containing the AckInvitation, the Controlling Function shall:

5. Wrap the AckInvitation into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
6. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
7. Send CPIM in an MSRP message to each participant as per section 3.2.4.

Upon receiving a ServerGroupVideoMessage containing an AckInvitation RCS clients for Group Chat participants' shall:

8. Show the participant as ringing to their respective users.
9. For the first AckInvitation received, the RCS client of the caller shall stop Timer_Alpha

This is illustrated in Figure 43.

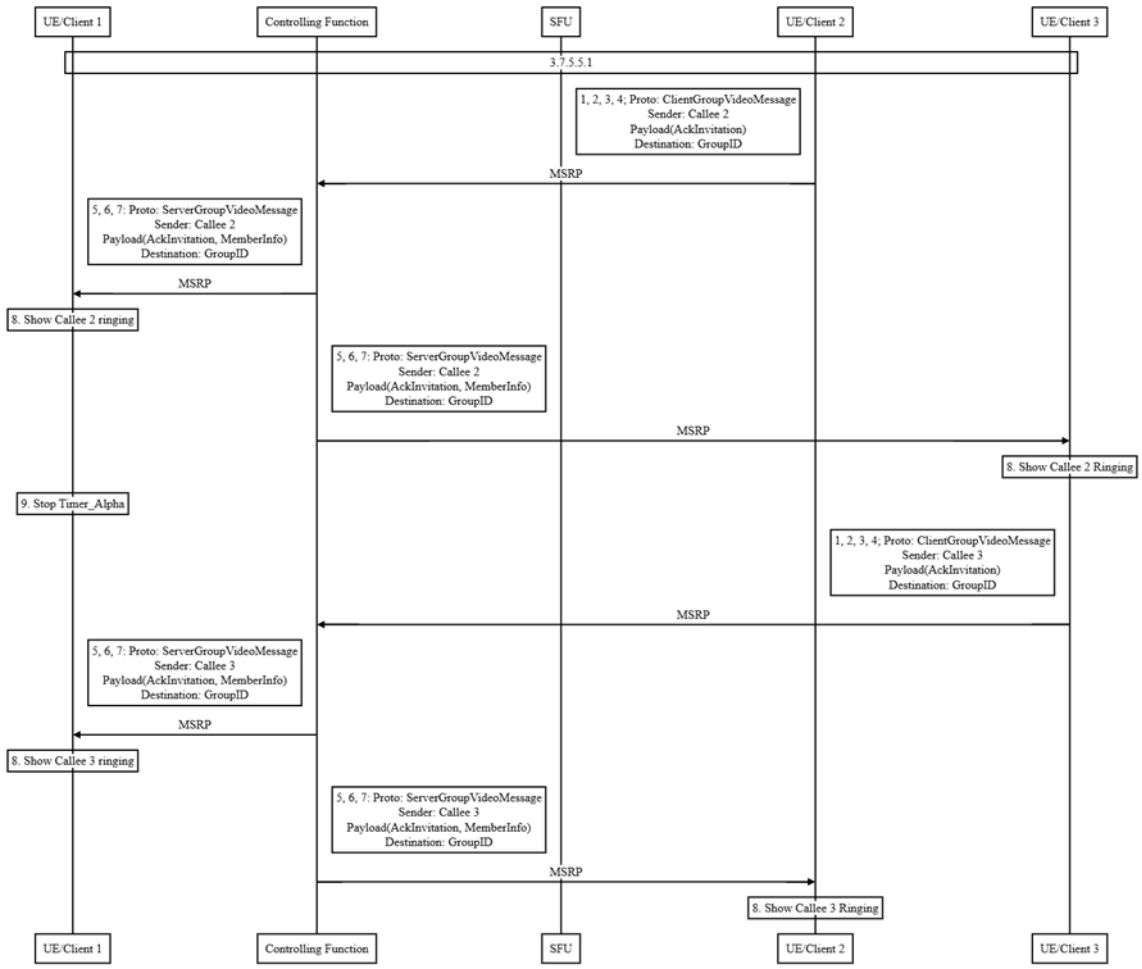


Figure 43: SPN relaying AckInvitation to all remaining Group Chat participants

3.7.5.5.3 Call Answered

After the AckInvitation procedure in section 3.7.5.5.2 has completed and after the Group Chat participant chooses to answer the Group RCS Video Chat, the participant's RCS client shall:

1. Stop the alerting.
2. Set up the media peer connection and gather ICE candidates as per section 3.7.5.2.2.
3. Start Timer_Delta as described in 3.7.5.2.4.4.
4. Connect the media to the SFU as per section 3.7.5.3.1.
5. If the media session is successfully connected and Timer_Delta has not expired, continue with the procedure.
6. Stop Timer_Delta.
7. Format the AcceptInvitation message into a as per section 3.7.5.2.3.2.3.
8. Wrap the AcceptInvitation message into a ClientGroupVideoMessage and include the SDP as per section 3.7.5.2.3.3.1.
9. Wrap the ClientGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
10. Send CPIM in an MSRP message as per section 3.2.4.
11. Send the JOIN RCS Video Chat notification event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

Upon receiving the ClientGroupVideoMessage containing an AcceptInvitation and the SDP answer, the Controlling Function shall:

12. Wrap the AcceptInvitation message and the SDP answer into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
13. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
14. Send CPIM in an MSRP message to each participant as per section 3.2.4.
15. Start Timer_Delta as described in 3.7.5.2.4.4.
16. If the media session is successfully connected and Timer_Delta has not expired, continue with the procedure.
17. Stop Timer_Delta

Upon receiving the ServerGroupVideoMessage containing the AcceptInvitation message from the first Group Chat participant joining the Group RCS Video Chat, the caller's RCS client shall:

18. Stop Timer_Beta.
19. Show the caller that the RCS Video Chat is answered and stop the ringing.
20. Send the JOIN RCS Video Chat notification event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

For every ServerGroupVideoMessage containing an AcceptInvitation message received by Group Chat participants, their RCS client shall:

21. Show the user that the new participant has joined the Group RCS Video Chat.

This is illustrated in Figure 44.

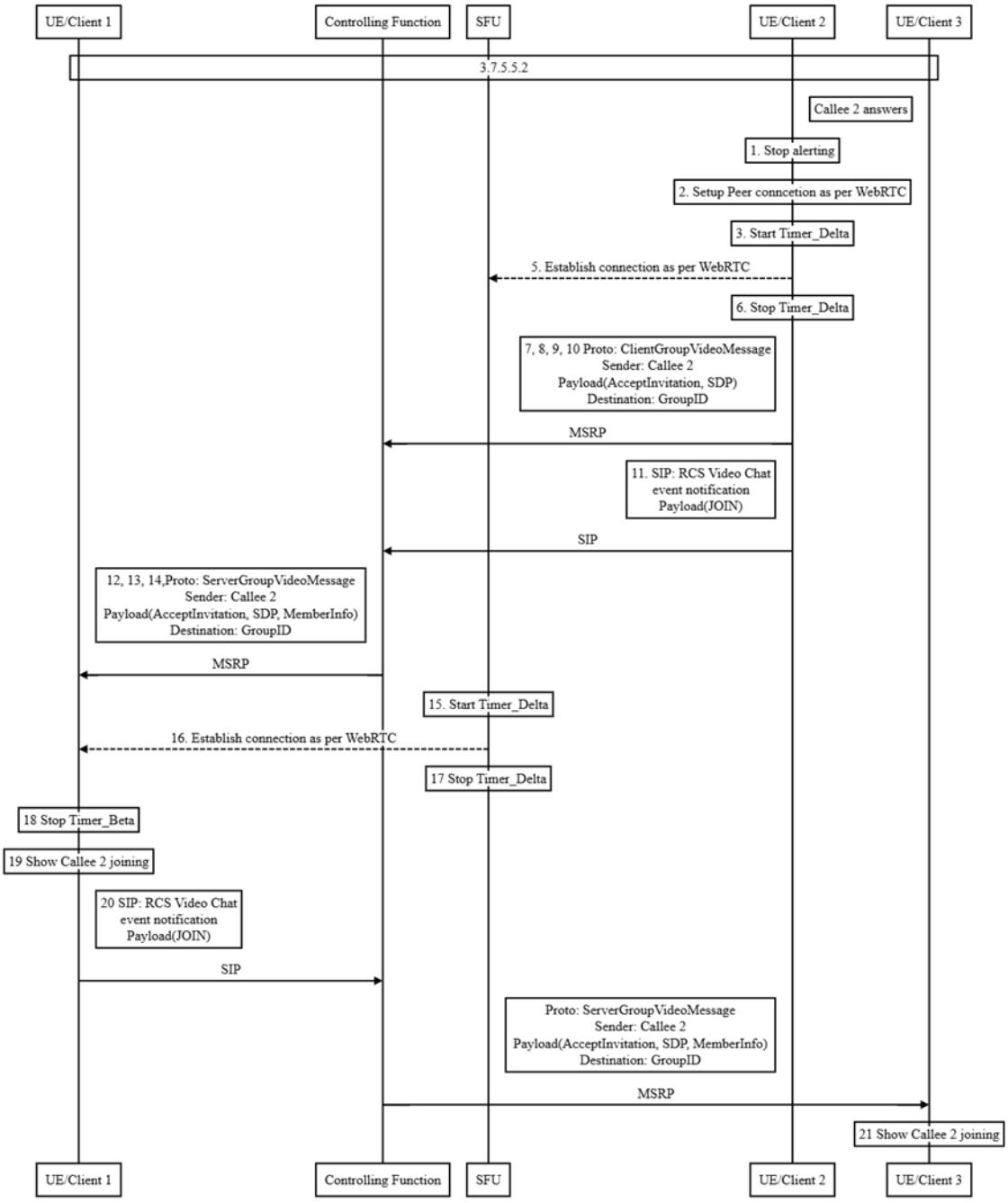


Figure 44: User answering a Group RCS Video Chat

3.7.5.5.4 Call Rejected

After the AckInvitation procedure in section 3.7.5.5.2 has completed, and if or when the Group Chat participant chooses to reject the Group RCS Video Chat, the participant's RCS client shall:

1. Stop the alerting.
2. Format the RejectInvitation message as per section 3.7.5.2.3.2.4 and include a RejectReason REJECT_REASON_USER_REJECT as defined in 3.7.5.2.3.2.4.1.

3. Wrap the RejectInvitation message into a ClientGroupVideoMessage as per section 3.7.5.2.3.3.1
4. Wrap the ClientGroupVideoMessage a CPIM container as per section 3.7.5.2.3.1.
5. Send CPIM in an MSRP message as per section 3.2.4.
6. Show the participant that the call is rejected.

Upon receiving the ClientGroupVideoMessage containing a RejectInvitation, the Controlling Function shall:

7. Wrap the RejectInvitation message into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
8. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
9. Send CPIM in an MSRP message to each participant as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing the RejectInvitation message from the first Group Chat participant joining the Group RCS Video Chat, the caller's RCS client shall:

10. Stop Timer_Beta.
11. Show the caller that the call was rejected by the participant.

Upon receiving the ServerGroupVideoMessage RejectInvitation message, Group Chat participant's RCS client shall:

12. Show the caller that the call was rejected by the participant.

This is illustrated in Figure 45.

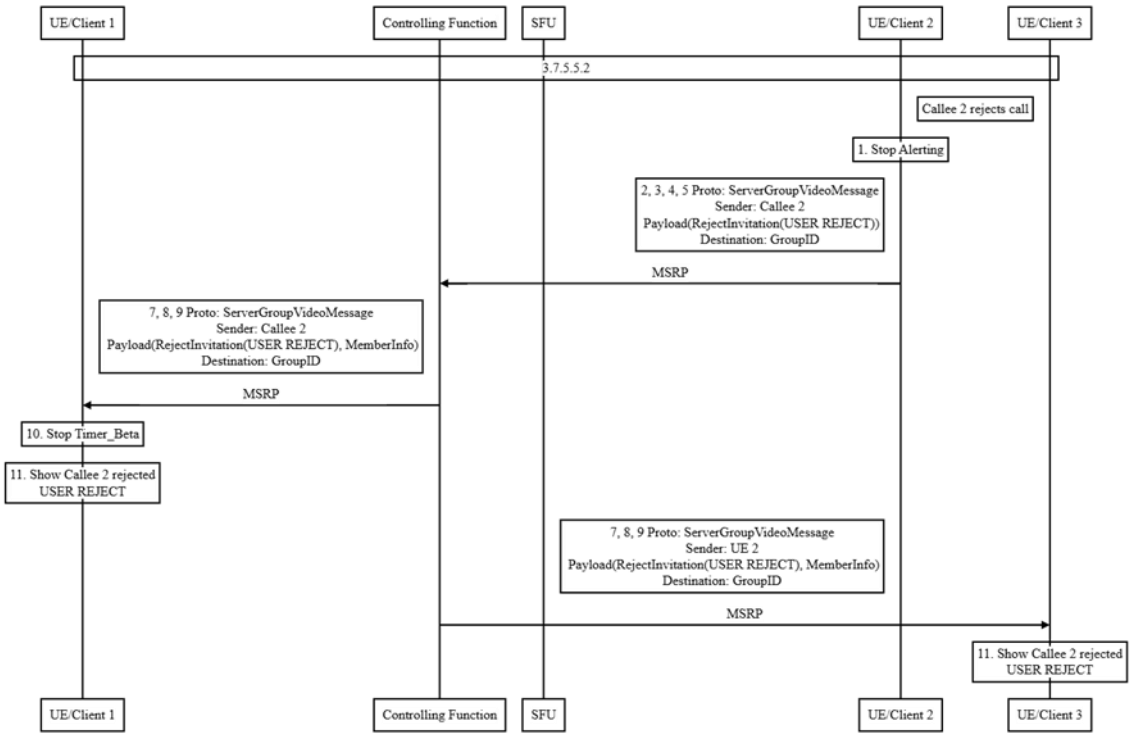


Figure 45: User rejects a Group RCS Video Chat

3.7.5.5.5 Call Cancelled

After the Group RCS Video Chat call initiation procedure in section 3.7.5.5.1 has completed, if the caller chooses to cancel a Group RCS Video chat call prior to receiving any AcceptInvitation, the caller's RCS client shall:

1. Format the CancelInvitation message into a as per section 3.7.5.2.3.2.6 and include a CancelReason CANCEL_REASON_USER_CANCEL as defined in section 3.7.5.2.3.2.6.1.
2. Wrap the CancelInvitation message into a ClientGroupVideoMessage and as per section 3.7.5.2.3.3.1
3. Wrap the ClientGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message as per section 3.2.4.
5. Stop the ringing.
6. Stop Timer_Beta.

Upon receiving the ClientGroupVideoMessage containing the CancelInvitation, the Controlling Function shall:

7. Wrap the CancelInvitation message into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
8. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
9. Send CPIM in an MSRP message to each participant as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing the CancelInvitation message the Group Chat participant's RCS client shall:

10. Stop the alerting.
11. Show the participant that the call was missed.

This is illustrated in Figure 46.

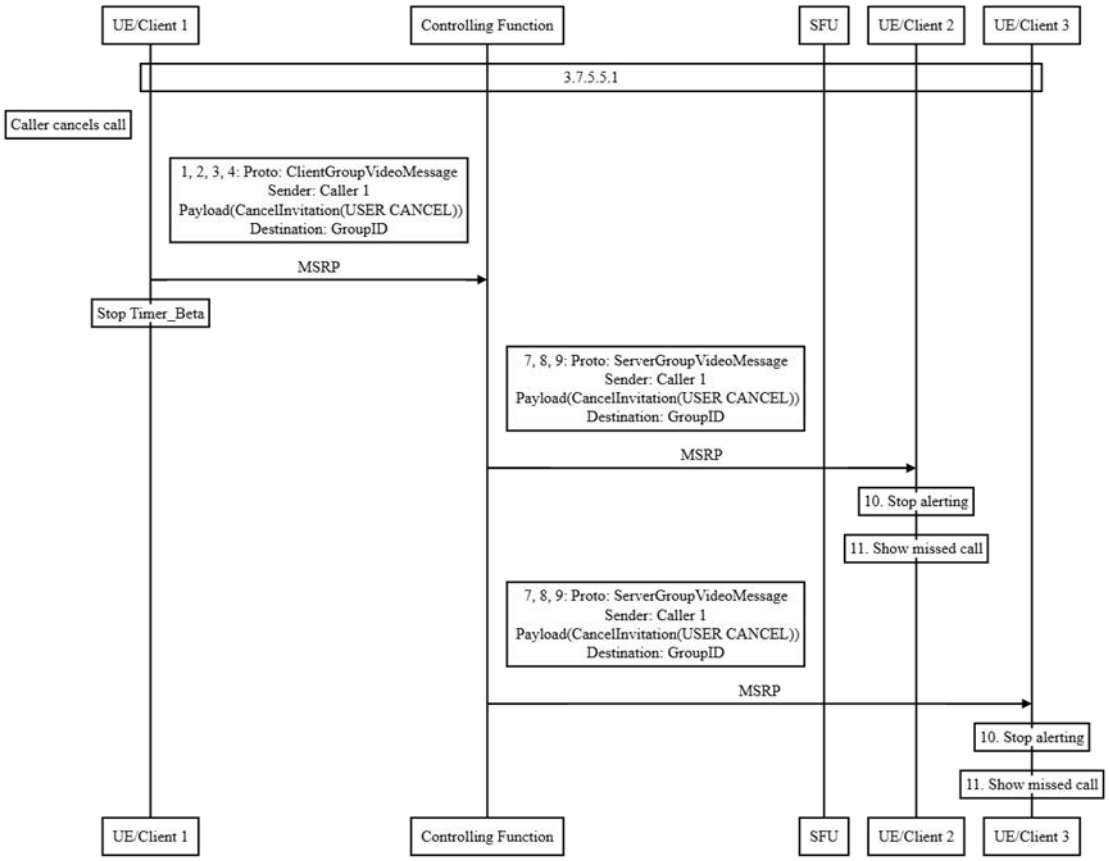


Figure 46: Caller cancels the Group RCS Video Chat

3.7.5.5.6 Call Hung up

After the answering procedure in section 3.7.5.5.3 has completed, to hangup a Group RCS Video Chat, the RCS client of the user who chooses to hang up shall:

1. Format the Hangup message into a as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_USER_INITIATED as defined in 3.7.5.2.3.2.5.1.
2. Wrap the Hangup message into a ClientGroupVideoMessage and as per section 3.7.5.2.3.3.1.
3. Wrap the ClientGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message as per section 3.2.4.
5. Close the media connection.
6. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.
7. Show the user that the call has ended

Upon receiving the ClientGroupVideoMessage containing the Hangup, the Controlling Function shall:

8. Wrap the Hangup message into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
9. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
10. Send CPIM in an MSRP message to each participant as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing the Hangup message other RCS clients shall:

11. Show the user that the participant has left the call.

This is illustrated in Figure 47.

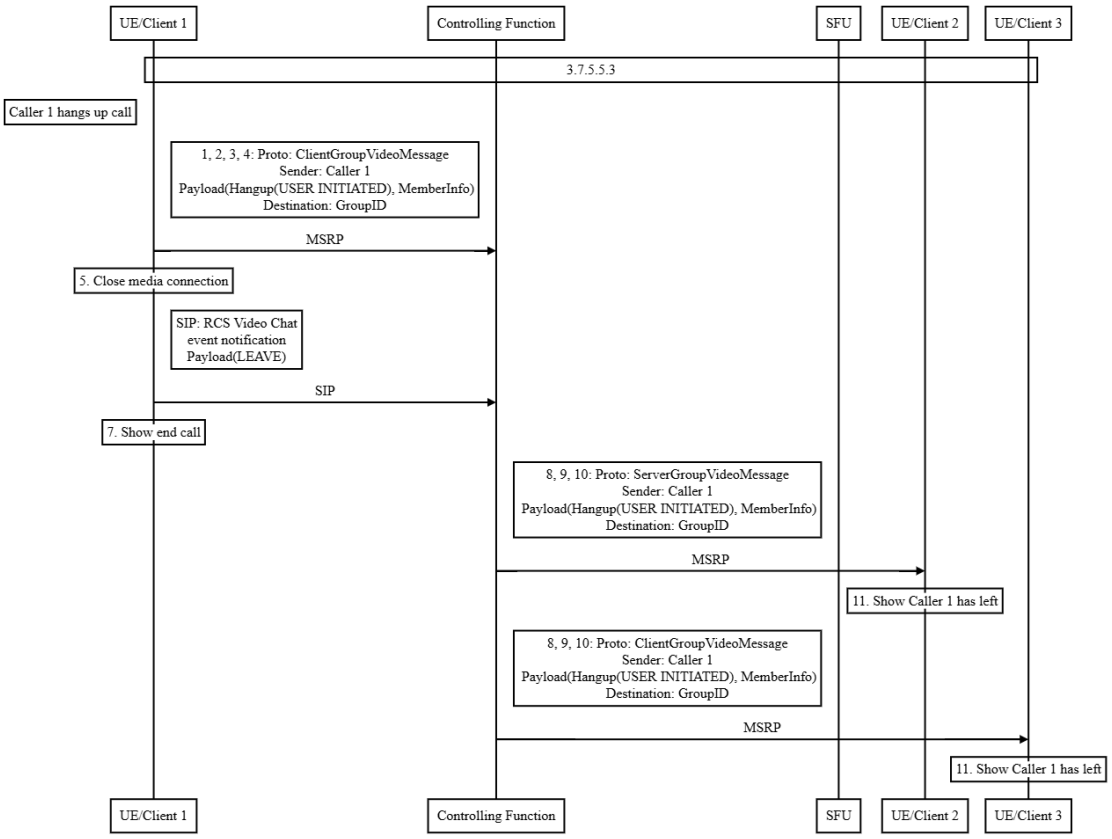


Figure 47: User 1 hangs up the Group RCS Video Chat

3.7.5.5.7 Adding User During Call

After the answering procedure in section 3.7.5.5.3 has completed, a Group Chat participant added a user to a Group Chat during an ongoing Group RCS Video Chat, the Controlling Function shall:

1. Format the CallInvitation message into a as per section 3.7.5.2.3.2.1 and all current MemberInfo. The CallInviteType shall be set to CALL_INVITE_TYPE_ONGOING as per section 3.7.5.2.3.3.2.2.
2. Wrap the CallInvitation message into a ServerGroupVideoMessage and as per section 3.7.5.2.3.3.2.
3. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
4. Send CPIM in an MSRP message to the SPN as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing the CallInvitation, the added user's RCS client shall:

5. Show alerting

6. Follow procedures defined in 3.7.5.5.2 and 3.7.5.5.3.

This is illustrated in Figure 48.

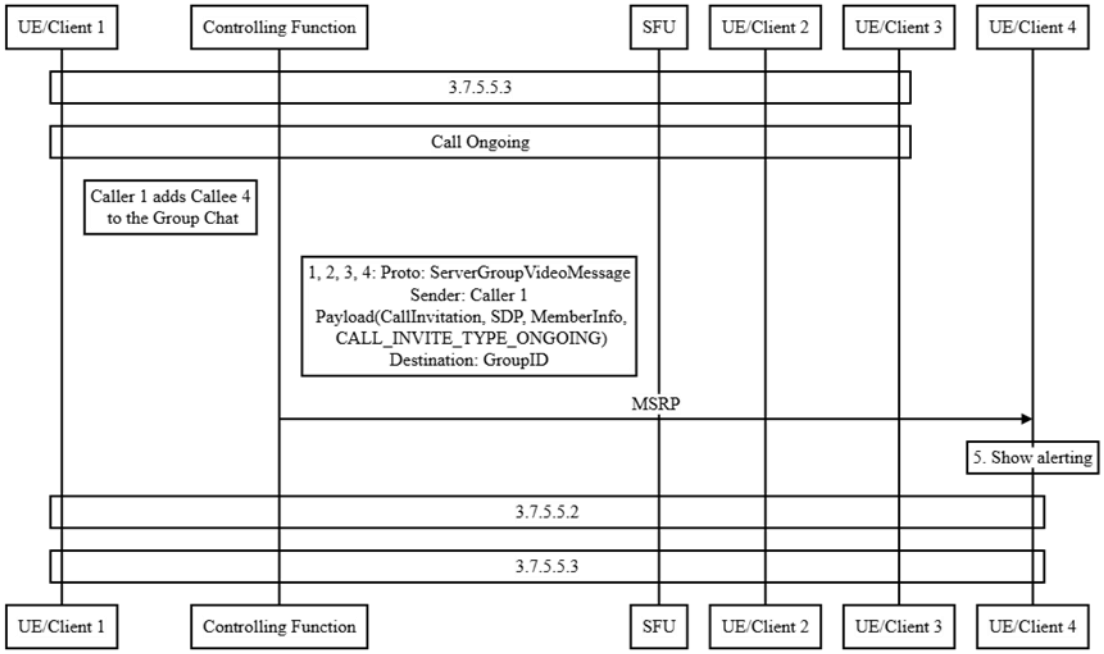


Figure 48: Caller adds new Group Chat participant to a RCS Group Chat with an ongoing Group RCS Video Chat

3.7.5.5.8 User Leaving During Call

After the answering procedure in section 3.7.5.5.3 has completed, a Group Chat participant leaves the Group Chat, the RCS client shall follow the procedure in 3.7.5.5.6.

3.7.5.5.9 Error Handling

3.7.5.5.9.1 Callee Offline

3.7.5.5.9.1.1 Callee receives ServerGroupVideoMessage containing the CallInvitation before Timer_Gamma expires

If the Callee receives a ServerGroupVideoMessage containing the CallInvitation from store and forward before Timer_Gamma expires, the client shall follow the procedure in 3.7.5.5.2

3.7.5.5.9.1.2 Callee receives ServerGroupVideoMessage containing the CallInvitation After Timer_Gamma expires

Upon receiving the ServerGroupVideoMessage containing the CallInvitation from store and forward, the RCS client shall:

1. Compare CallInvitation sending timestamp with the current time and if it's greater than Timer_Gamma as described in 3.7.5.2.4.3, continue the procedure.
2. Show the call as missed.

3.7.5.5.9.1.3 Callee back online receiving a ServerGroupVideoMessage containing the CallInvitation and CancelInvitation from store and forward

Upon receiving the CallInvitation and CancelInvitation from store and forward, the RCS client shall:

1. Compare CallInvitation sending timestamp with the current time and if it's less than the Timer_Gamma as per section 3.7.5.2.4.3, continue the procedure.
2. Receive CancelInvitation message
3. Show the call as missed.

3.7.5.5.9.2 All Participants Ignore the Group RCS Video Chat

During the answering procedure described in 3.7.5.5.3, if Timer_Beta expires, the caller's RCS client shall:

1. Format the CancelInvitation message into a ClientGroupVideoMessage as per section 3.7.5.2.3.3.1
2. Wrap the ClientGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
3. Send CPIM in an MSRP message as per section 3.2.4.
4. Show the call as missed.

Upon receiving the ClientGroupVideoMessage containing the CancelInvitation, the Controlling Function shall:

5. Wrap the CancelInvitation into a ServerGroupVideoMessage as per section 3.7.5.2.3.3.2.
6. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
7. Send CPIM in an MSRP message to each participant as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing CancelInvitation message the Group Chat participants RCS client shall:

8. Stop the alerting
9. Show the call as missed.

This is illustrated in Figure 49.

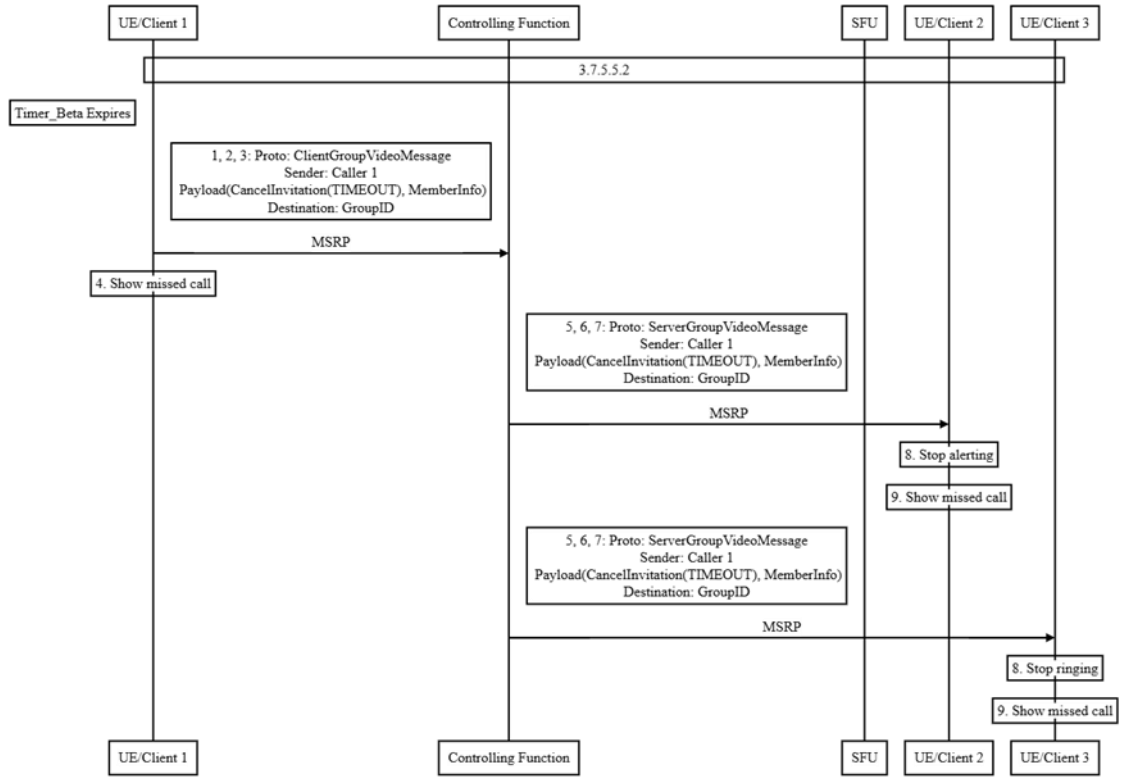


Figure 49: All callees ignore the incoming RCS Group Video Chat

3.7.5.5.9.3 Disconnect During Call

3.7.5.5.9.3.1 RCS client reconnects

After following the procedure in section 3.7.5.5.3, if the SFU detects a media disconnection from an RCS client, the Controlling Function shall:

1. Start Timer_Zeta as described in 3.7.5.2.4.6.

If the RCS client detects a disconnect, it shall:

2. Start Timer_Delta as described in 3.7.5.2.4.4.
3. Connect the media to the SFU as per section 3.7.5.3.1.
4. If the media session is successfully connected and Timer_Delta has not expired continue with the procedure.
5. Stop Timer_Delta.

Upon re-establishing the media connection, the SFU shall:

6. Stop Timer_Zeta.

This is illustrated in Figure 50.

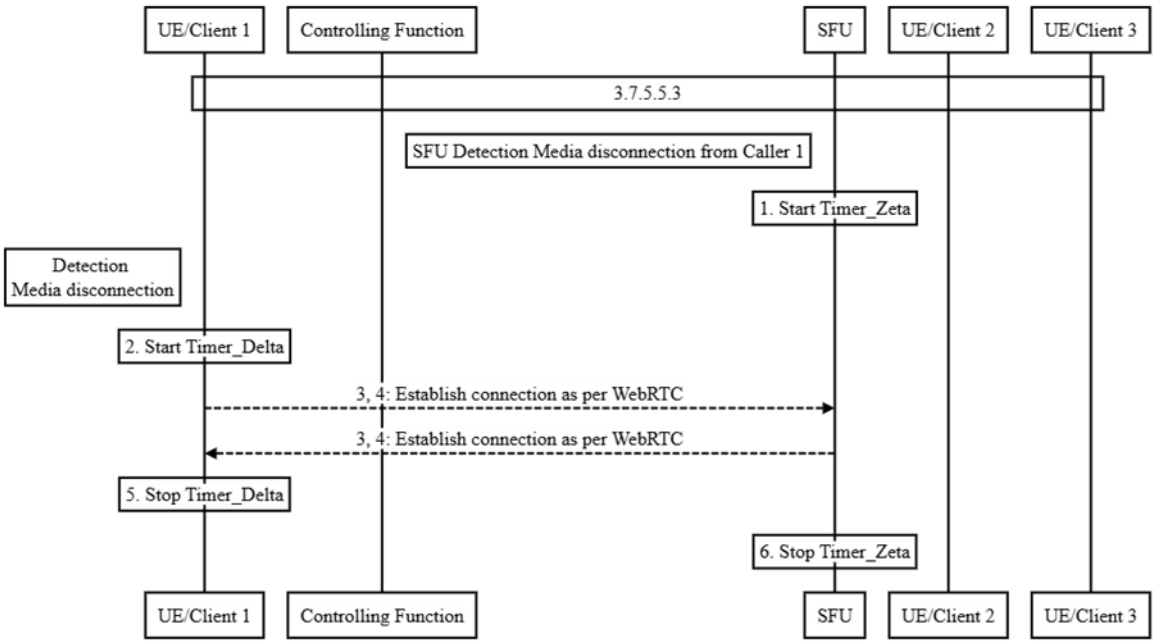


Figure 50: Media successfully reconnected after connection loss

3.7.5.5.9.3.2 RCS client does not reconnect

After the answering procedure in section 3.7.5.5.3 has completed, if the SFU detects a media disconnection from an RCS client, the Controlling Function shall:

1. Start Timer_Zeta as described in 3.7.5.2.4.6 and wait for the Timer_Zeta to expire.
2. Format the Hangup message as per section 3.7.5.2.3.2.5 and include a HangupReason HANGUP_REASON_ SERVER_DETECTED_DISCONNECT as defined in 3.7.5.2.3.2.5.1.
3. Wrap the Hangup message into a ServerGroupVideoMessage protocol buffer as per section 3.7.5.2.3.3.2.
4. Wrap the ServerGroupVideoMessage into a CPIM container as per section 3.7.5.2.3.1.
5. Send CPIM in an MSRP message to all participants as per section 3.2.4.

Upon receiving the ServerGroupVideoMessage containing the Hangup message the disconnected RCS client shall:

6. Send the LEAVE RCS Video Chat Event event SIP MESSAGE as per sections 3.7.5.2.3.4 and 3.7.5.2.5.

Upon receiving the ServerGroupVideoMessage containing the Hangup message other RCS clients shall:

7. Show the user that the participant has left the call.

This is illustrated in Figure 51.

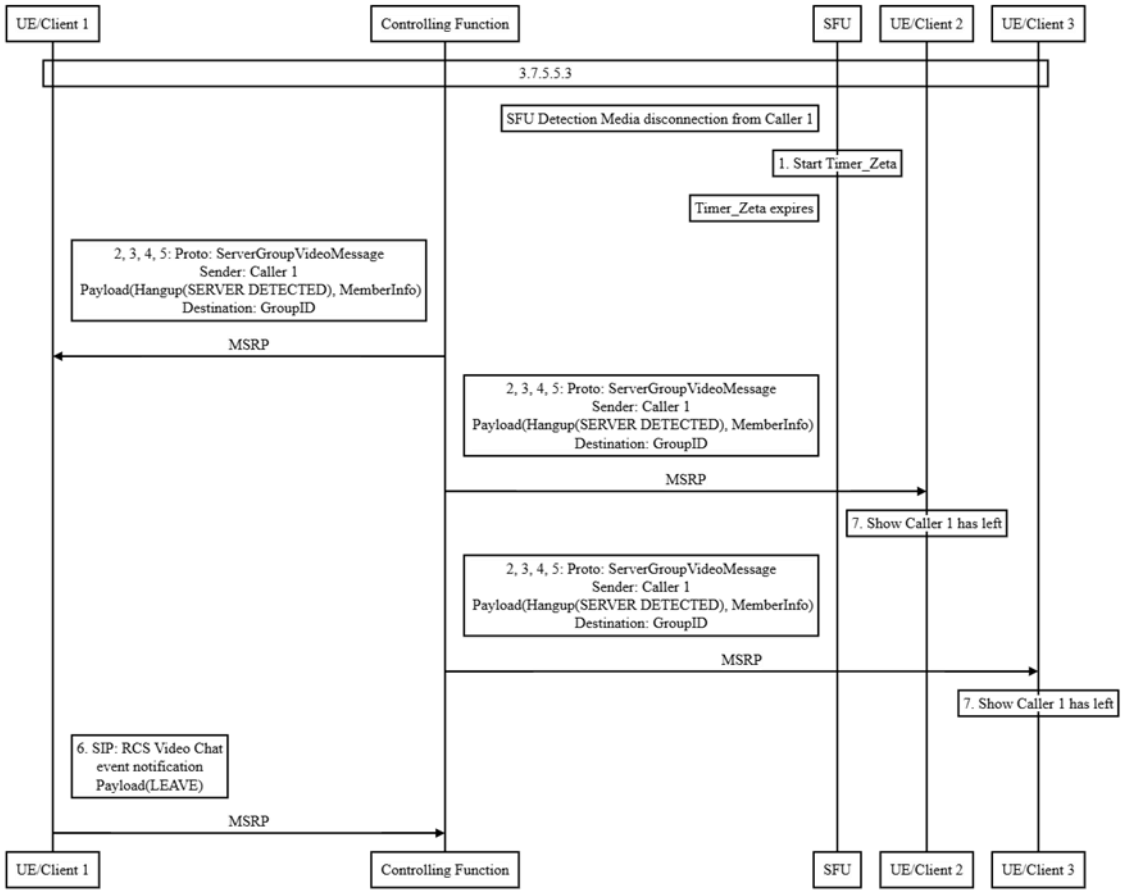


Figure 51: Media unsuccessfully reconnected after connection loss

3.7.5.5.9.4 IP Switching

After the answering procedure in section 3.7.5.5.3 has completed, when an RCS client detects an IP address switch or change during an active Group RCS Video chat, the RCS client shall:

1. Show the user that the call is reconnecting.
2. Set up the media peer connection and gather ICE candidates as per section 3.7.5.2.2.
3. Format the IceCandidatesUpdate message as per section 3.7.5.2.3.2.7
4. Wrap the IceCandidatesUpdate message. into a ClientGroupVideoMessage as per section 3.7.5.2.3.3.1
5. Wrap the ClientGroupVideoMessage message into a CPIM container as per section 3.7.5.2.3.1.
6. Send CPIM in an MSRP message as per section 3.2.4.
7. Start Timer_Delta as described in 3.7.5.2.4.4.
8. Connect the media to the recipient's RCS client as per section 3.7.5.3.1.
9. Stop Timer_Delta

Upon receiving the IceCandidatesUpdate the SFU shall:

10. Start Timer_Delta as described in 3.7.5.2.4.4.
11. Connect the media to the SFU as per section 3.7.5.3.1.

12. Stop Timer_Delta

This is illustrated in Figure 52.

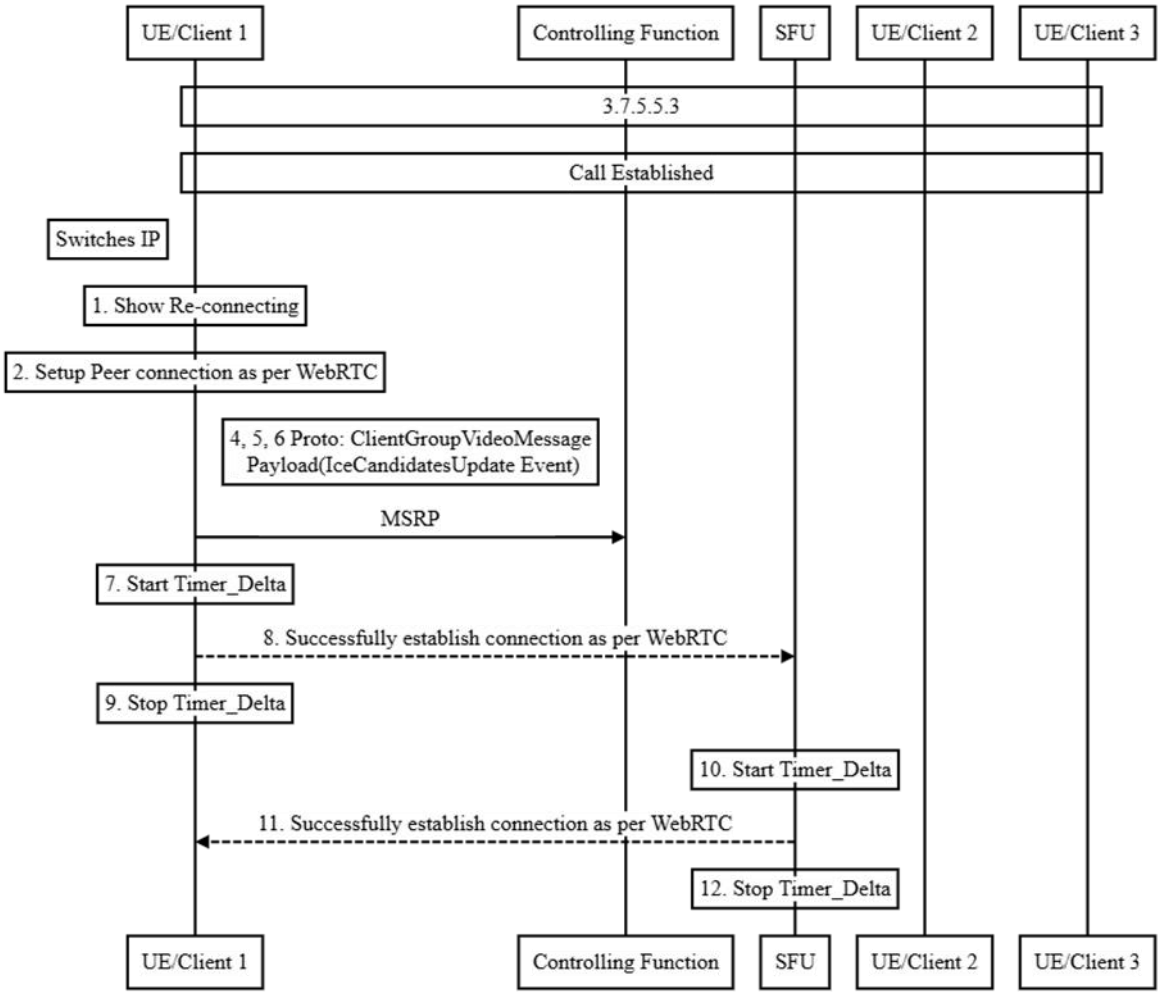


Figure 52: Media reconnected after IP change by UE/Client 1

3.7.5.6 Quality of Service

The RCS SPN should provide QoS for RCS Video Chat. The exact mechanism for enablement is deployment specific.

4 Cross-service functionality

4.1 Common Message Store

4.1.1 Overview

RCS supports a “Common Message Store” as described in Section 5.5 of [CPM-SYS_DESC] and specified in [CPM-MSGSTOR-REST]. Using an HTTP RESTful interface, an RCS client can access and manage stored objects, as described in [CPM-MSGSTOR-REST] regardless of their RCS service registration.

The Restful resources from Client to Message Store Server are executed either during an “initial sync” i.e. First synchronization between client and store, or following a notification to the client of store content changes. The Restful design uses a notification channel as its main method of understanding when to synchronize, and this greatly helps with battery life on the client (there are few connection requests to the Message Store Server and less need for periodic synchronising).

Regarding the client synchronisation mechanism that applies, client synchronization guidelines are described in section 4.1.16.7.

A primary device sending or receiving messages via the SMS or MMS (e.g. in case of no data connection) may, subject to Service Provider policy regarding automatic SMS or MMS storage in the Common Message Store (CMS), also receive these messages via the synchronisation from the Common Message Store. Since legacy messages do not contain Conversation-ID and Contribution-ID, a different mechanism is required to link together the two representations of the same message.

Sections 4.1.8, 4.1.9, 4.1.10 and 4.1.11 describe the mechanism used for a device to correlate legacy SMS/MMS messages with the same messages already stored in the Common Message Store.

The Common Message Store shall support storage and retrieval of RCS and xMS message objects and call log objects as defined in [CPM-MSGSTOR-REST].

4.1.2 Support of GBA in the Common Message Store

The General Bootstrapping Architecture (GBA) defined in [3GPP TS 33.220] provides mechanisms for AKA based user authentication using the 3GPP Authentication Centre (AuC) and the USIM or ISIM. The Common Message Store supports the authentication of primary devices via GBA with the extension defined in this section.

The use of the GBA is based on an existing bootstrapped security association managed between the client and the Bootstrapped Security Function (BSF) operated by the Service Provider. The bootstrapped security association provides the client with a Bootstrapping Transaction Identifier (B-TID) and key material which can be used to authenticate the user with the Service Provider's network.

The procedures for the client and the Common Message Store shall follow the definitions of [3GPP TS 24.109].

4.1.3 Support of OpenID Connect in the Common Message Store

If the client receives in result of an API request to the Common Message Store a HTTP 302 FOUND response, the client shall change the type of the request to HTTP GET and follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the Common Message Store commencing in the processing of the API request.

If the client receives a 403 FORBIDDEN response in the result of the processing, then the client is not authorised to use the Common Message Store.

4.1.4 Support for Digest Authentication

If the value of the MESSAGE STORE AUTH client configuration parameter is set to 2 and the client receives in result of an HTTP request for the RESTful interface of the Common Message Store a HTTP 401 AUTHENTICATION REQUIRED error response carrying a WWW Authenticate header indicating the Digest scheme without the 3GPP-bootstrapping indication as defined in [3GPP TS 24.109], then the client shall invoke the Digest authentication procedures as per [RFC2617] sending a subsequent request with the Authorization header. An RCS client shall include the qop directive if provided by the server.

The client shall compose the Authorization header using the value derived from the MESSAGE STORE USERNAME client configuration parameter defined in section A.1.3 for the username and the value derived from the MESSAGE STORE PASSWORD client configuration parameter defined in section A.1.3 for the password.

4.1.5 Support for Basic Authentication

If the value of the MESSAGE STORE AUTH parameter is set to 1, then the client shall, in accordance with the definition of [RFC2617], pre-emptively send a HTTP Authorization header in all HTTP requests using a secure connection and targeting the protection space identified by the URL derived from configuration parameter MESSAGE STORE URL defined in section A.1.3.

If the value of the MESSAGE STORE AUTH parameter is set to 2 and the client receives in result of an HTTP request for the RESTful interface of the Common Message Store a HTTP 401 AUTHENTICATION REQUIRED error response carrying a WWW Authenticate header field indicating the Basic scheme, then the client shall invoke the procedures for Basic Authentication as defined in [RFC2617]. If successfully authenticated, the client shall from then on pre-emptively send a HTTP Authorization header in all HTTP requests using a secure connection and targeting the protection space identified by the "realm" provided by the Common Message Store.

The client shall compose the Authorization header using the value derived from MESSAGE STORE USERNAME parameter defined in section A.1.3 for user-id and the value derived from the MESSAGE STORE PASSWORD parameter defined in section A.1.3 for the password.

4.1.6 RESTful Web Service Calls

The web service endpoints for the Message Store Server are members of two APIs.

- The Network Message Store (NMS) API defined in [CPM-MSGSTOR-REST] defines endpoints for managing messages, folders and for creating and managing subscriptions.
- The Notification Channel (NC) API defined in [CPM-MSGSTOR-REST] defines endpoints for creating and managing notification channels suitable for use with NMS polling. Using the NC API, you can create long polling channels.

As shown in the prototype API calls in section 4.1.6.1 and 4.1.6.4, the protocol scheme for accessing these APIs shall be https. Therefore, when accessing the web service endpoints for the Message Store Server, the client shall establish a TLS connection and verify that a

certificate is provided and that this certificate is valid (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the expected domain for the API, etc.). If no certificate is provided or the certificate is invalid, the client shall consider the web service endpoint to be unavailable and not continue using it to access the message store. The client shall then verify at every restart of the client or the device whether a valid certificate is provided.

NOTE: Given that for security reasons this client handling does not ensure a graceful fallback to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

4.1.6.1 Example NMS API Call

Prototype (base API call)

```
https://{nmsHost}/nms/v1/base/{boxId}/{endpoint}
```

Example

```
https://nms-mc4.enc.MNO.net/nms/v1/base/tel+12021001000/objects/6E
```

All elements are case sensitive and are as below:

Part	Type	Value/Example
{nmsHost}	Supplied from the RCS configuration document Parameter NMS_URL	https://nmsHost
API	Constant	nms
API Version	Variable	v1
Store	Constant	base
{boxId}	Variable (user identifier)	tel:+12021001000
{endpoint}	Desired endpoint	/objects/{objectId} (endpoint prototype) objects/6E (example)

4.1.6.2 API Version

Many calls require a version ID to be included in the call's URL path. The prescribed version of the interface as per [CPM-MSGSTOR-REST] is "v1". The examples provided will include this version ID as a static portion of the call's URL. Any attempts to request an unsupported version will result in the server responding with the supported versions available.

4.1.6.3 Box ID

The subscriber number with the country prefix i.e. "tel: E164".

4.1.6.4 A Notification Channel (NC) API Call

Prototype

```
https://{ncHost}/notificationchannel/v1/{userid}/{endpoint}
```

Example

```
https://nms-
mc4.enc.mno.net/notificationchannel/v1/tel:+12021001000/channels/f7d8c62c-
969b-4090-896f-67708f615a89
```

All elements are case sensitive and are as below:

Part	Type	Value/Example
{nmsHost}	Supplied from the RCS configuration document parameter NMS_NC_URL	https://ncHost
API	Constant	notificationchannel
API Version	Variable	v1
Store	Constant	base
{userId}	Variable (user identifier)	tel:+12021002003
{endpoint}	Desired endpoint and IDs	channels/f7d8c62c-969b-4090-896f-67708f615a89

Version and UserId are as defined for the NMS API.

4.1.7 Folder Structure

The message stores messages in folders as follows:

- RCS/CPM and legacy one-to-one messages are stored in folders based on
  - the identity of the contact (for 1-to-1 messages)
  - the Conversation-ID (for group messages, i.e. 1-to-N messages).

Clients should facilitate folder selection by using the Is-CPM-Group parameter (set to “yes”) when depositing RCS group messages. Clients should use “no” for all other messages.

To discover the folder structure, clients should follow the process described in [CPM-MSGSTOR-REST] section 6.1.3 – i.e. descend recursively from root. The /folders/operations/search resource should be used to find the root folder.

4.1.7.1 Folder identifies

As per [CPM-MSGSTOR-REST] section 6.1.3 the root folder of the user’s mailbox is the top level of the folder tree and shall be uniquely identified i.e. it shall have an attribute called “root” with the value “yes”. There is only one root folder, and the name of the root folder is an empty string unless specifically assigned to be some other name by the Message Storage Server.

The folder structure is shown below in Figure 53. The format of the conversation folder names shall be as follows:

- For RCS objects relating to a 1-to-1 conversation (i.e. file transfer objects, session info objects, message objects related to Standalone Messages, 1-to-1 Chat messages, and legacy messages (i.e. SMS/MMS)) shall be stored in a folder identified by the identity of the contact in the conversation. This identity shall be determined from the P-Asserted-identity of the contact, received by the Messaging

Server via the incoming SIP request towards the served user, or via the incoming SIP response towards the served user.

- If a global E.164 phone number is available and there is no 'tk' parameter present, that shall be used in the global-number representation of a tel URI as defined in [RFC3966]. Parameters are not used in the folder name.
  Example: tel:+4917123456789

NOTE: RCS user clients storing messages in the Common Message Store may not have sufficient information to normalize all destination addresses to global E.164 phone number format. In this case the folder name may be formatted as defined below for non E.164 numbers.

- If a non E.164 phone number is available and there is no 'tk' parameter present, it shall be used in the local-number representation of a tel URI as defined in [RFC3966]. These numbers are typically short codes used for addressing of value added services outside the public numbering plan. Parameters and context are not used in the folder name.
  Example: tel:22632677
- Phone numbers are typically derived from
  - tel URI in SIP Signalling
  - SIP URI in SIP signalling if appended by user=phone parameter
  - SMS Originator Address
  - SMS Destination Address
  - MMS Originator Address
  - MMS Recipient Address
- If both a SIP URI and tel URI are present and there is a 'tk' URI parameter, the SIP URI shall be used as defined in [RFC3261] after converting it to lower case first. The 'tk' URI parameter (see section 2.5.4.3) shall be included in the folder name.
  Examples: sip:chatboty@botplatform.botplatformz.com;tk=off, or sip:chatboty@botplatform.botplatformz.com;tk=on
- If only a SIP URI is available the SIP URI shall be used as defined in [RFC3261] after converting it to lower case first. If present the 'tk' URI parameter (see section 2.5.4.3) shall be included in the folder name.
  Examples: sip:joe@example.com, or sip:chatboty@botplatform.botplatformz.com;tk=off, or sip:chatboty@botplatform.botplatformz.com;tk=on
- If an e-mail address is available it shall be used as mailto URI [RFC6068] after converting characters to lower case first.
  Example: mailto:joe@example.com
  e-mail addresses are typically derived from
  - MMS Originator Address
  - MMS Recipient Address

- If an alphanumeric address is available it shall be used with no conversion. These addresses are typically display names from SMS value added services without a routing address.
  Example: ACME Corporation
  Alphanumerical addresses are typically derived from
  - SMS Originator Address

Entities managing folder names in the Common Message Store shall comply with the mailbox international naming conventions defined in [RFC3501].

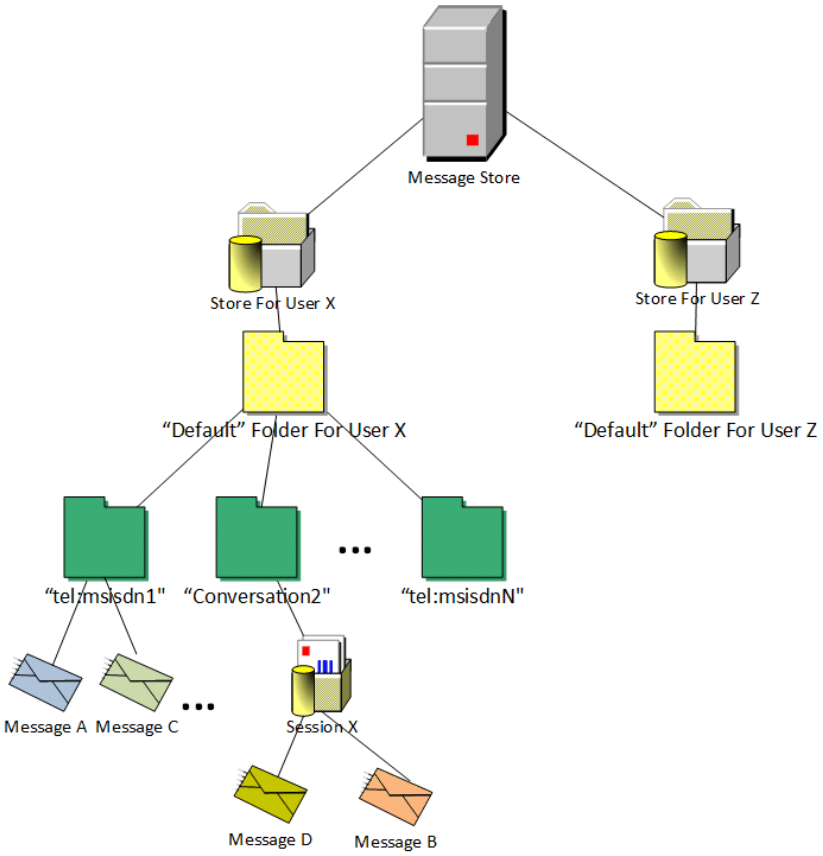


Figure 53: Example Folder Structure

4.1.7.2 Storage of IMDN Objects,

IMDNs are deposited to the message storage server with a Message Context of "imdn-message" and synchronised to the clients as individual message objects. The IMDN message object contains an attribute that links it to the original message (DispositionOriginalMessageID), see section 5.4.10 of [CPM-MSGSTOR-REST].

4.1.8 Common Message Store and pager/multimedia-messages

To identify the messages in the Common Message Store that will match legacy SMS/MMS messages sent or received by the device via legacy means, it shall be possible to keep information about the submission or delivery path (SMS or MMS) for converted messages in the Common Message Store.

The information shall be stored for messages by means of the message context for internet mail (see [RFC3458]).

In RCS the following values of the "message-context" are applicable:

- For received messages:
  - pager-message: the message is delivered to a primary device via SMS or stored as SMS
  - multimedia-message: the message is delivered to a primary device via MMS

NOTE: In RCS the message context is only used in the relation between the terminating CPM Participating Function, the Common Message Store and the recipient user's device. It does not provide information of the message context on the originating side nor on the NNI.

- For sent messages:
  - pager-message: the message was sent via SMS or stored as SMS
  - multimedia-message: the message was sent via MMS

4.1.8.1 Client initiated storage of SMS/MMS

The RCS Client stores messages in the Default folder.

NOTE: This procedure is applied when synchronising as described in section 4.1.16.7.

When storing a SMS or MMS message and their delivery and read reports in the Common Message Store the client shall follow the definitions for recording of messages in sections 4.1.12 and 4.1.13 respectively.

If the SMS MESSAGE STORE or MMS MESSAGE STORE configuration parameter defined in Table 167 is set to “always store in the Common Message Store" (i.e. 2) then if storing them locally the client shall upload new sent or received messages and for sent messages, their delivery and read reports (SMS or MMS respectively) to the Common Message Store and link them with the local message. When storing a short message in the Common Message Store the client should not set the correlation information defined in section 4.1.9 for SMS. When storing a multimedia message, a delivery or read report in the Common Message Store the client shall set the correlation information defined in section 4.1.9 for MMS.

If the SMS MESSAGE STORE or MMS MESSAGE STORE configuration parameter is set to “store if not found in the Common Message Store” (i.e. 1) then the client shall apply the message correlation for new sent or received messages and their delivery and read reports if storing them locally as described in section 4.1.9 for SMS and MMS respectively. If the client determines that a locally stored message and its delivery and read report (for sent SMS or MMS respectively) is not already stored in the Common Message Store, the client shall

- store this message to the Common Message Store with the correlation information defined in section 4.1.9 (for SMS and MMS) respectively, and
- Link it with the local message.

The client shall store SMS and MMS messages and their delivery and read reports in the Common Message Store in a folder identified by the identity of the Contact in the Conversation under the Default folder. If this folder is not yet created in the Default folder, the client shall first create it. For definition of the folder names refer to section 4.1.7.

When storing a message sent or received as an SMS/MMS in the Common Message Store, the client shall set

- a Message-Context header and attribute to pager-message/multimedia-message as described in section 4.1.8 and
- The Message-Direction header and Direction attribute as described in section 4.1.16.8.

4.1.9 Correlating SMS/MMS messages with messages stored in the Common Message Store

The following mechanisms describe how to correlate messages received via legacy means with messages stored in the Common Message Store.

For SMS messages:

- The entity in the network storing the message shall store the prefix of the SMS text body (as defined in section 4.1.10) in the Message-Correlator header of the message (see [CPM-MSGSTOR-REST]).
- The device shall use this Message-Correlator header value, along with To/From headers to find the corresponding locally stored SMS message
- The algorithm is as described in section 4.1.10.

For MMS messages:

- Each MMS message and the corresponding delivery reports are defined by a unique MMS Message ID
- The entity in the network storing the message and corresponding delivery reports shall store the MMS Message ID in the Message-Correlator header defined in [CPM-MSGSTOR-REST]. The header value shall contain the Message-ID-value defined in [MMSENC] as "ascii-value".
- The device uses the MMS Message ID from MMS messages and delivery reports to find the Unique Identifier (UID) of corresponding MMS messages or delivery notifications in the Common Message Store by matching it with the Message-ID contained in the Message-Correlator of each stored messages.

Since the Common Message Store remains the master storage for these legacy messages, it is up to the client implementation whether or not to discard matched messages received via legacy means.

While correlation collisions will generally be infrequent, there are particular circumstances where they are quite likely to occur. Therefore, in addition to this basic process, additional logic is required to handle correlation collisions for SMSs, see section 4.1.11.

4.1.10 Correlation Algorithm for SMS

In order to ensure the message correlation algorithm succeeds on both the client and Message Store Server, RCS clients or the entity in the network sending SMS messages with characters from the GSM 7 bit national single shift and locking shift tables, shall instead use UCS2 (2-byte Universal Character Set) encoding.

NOTE: The RCS client may still receive SMS messages with characters from the GSM 7 bit national single shift and locking shift tables since they may come from non-RCS compliant clients or networks.

When other alphabets (e.g. Latin-1, HP Roman-8) are used as consequence of the SMS bearer technology in the network such as SMPP, the entity creating the Message-Correlator converts the message from the received alphabet into UTF-8 format. The entity which converts from UCS2 or GSM 7 bit alphabets to other alphabets (e.g. Latin-1, HP Roman-8), or vice versa, shall ensure a one-to-one character mapping. This entity can be the entity that stores the message and creates the Message-Correlator, or it can be the entity that sends the message towards the recipient.

NOTE: The one-to-one character mapping is vendor dependent when anything other than GSM 7 bit default alphabet or UCS2 is received by the SMS-C.

The correlation is based on the following field values:

- To: It should be the format as taken from the address field defined in [3GPP TS 23.040]. If TON (Type Of Number) indicates “international”, then a “+” is inserted before the number string. If TON indicates “unknown” only the number string is used. If the address is “alphanumeric”, then the address shall be encoded to UTF-8 format
- From: It should be the format as taken from the address field defined in [3GPP TS 23.040]. If TON indicates “international”, then a “+” is inserted before the number string. If TON indicates “unknown” only the number string is used. If the address is “alphanumeric”, then the address shall be encoded to UTF-8 format.
- The Message-Correlator header value which is generated from the Text Payload contained in the user data of the short message with up to 160 characters as defined below. Characters or data contained in SMS user data information elements (i.e. SMS and EMS control data as well as EMS content data) are not considered for the correlation algorithm.
  Entities storing the message and clients correlating messages shall compose the Message-Correlator header value as follows:
  - For messages with no text payload in the SMS user data a Message-Correlator header with no value shall be generated.
  - The entity creating the Message-Correlator converts from its original encoding (GSM 7 bit default alphabet or UCS2, see [3GPP TS 23.038], or any other GSM 7 bit national single shift and locking shift tables) into UTF-8 format. The same

applies when alphabets other than GSM 7 bit or UCS2 are used (e.g. Latin-1, HP Roman-8).

- Any UTF-8 "Null" character is removed.
- Any UTF-8 characters "CR" and "LF", and the sequence "CR LF", are all removed
- In the case of concatenated SMS messages once the message is reassembled and the above rules have been applied, only the first 160 characters shall be used to generate the Message-Correlator header value in accordance with the procedures defined above.
- If the resulting string contains only US-ASCII characters (0x20 – 0x7e) it will be taken as the value of the Message-Correlator header.
- If the resulting string contains at least one non US-ASCII character, the Message-Correlator header value shall be encoded as defined in [RFC2047]. The value shall be encoded by the use of the UTF-8 character set (charset = utf-8) and base64 encoding (encoding = b). In this case the client should use for correlation of messages the "encoded-text" part of the header value. For details of the Message-Correlator header encoding refer to [CPM-MSGSTOR-REST].
- Examples of Message-Correlator header values:
  the Message-Correlator header value of a short message with the text payload:
  `To your health, my friend`
  will encoded as follows
  `Message-Correlator: To your health, my friend`
  the Message-Correlator header of a short message with the text payload
  `На здоровье, мой друг`
  will be encoded as follows
  `Message-Correlator: =?utf-8?b?`
  `0J3QsCDQt9C00L7RgNC+0LLRjNC1LCDQvNC+0Lkg0LTRgNGD0LM=?=`

Table 138 `illustrates the required coding conversions for the Message-Correlator algorithm to succeed.`

ID	Messaging Technology	Original Encoding type	Client Sender		Client Receiver	Messaging Server is Receiver from SMS-C via SMPP or MAP
			Encoding for Sending Message	Calculation of Message-Correlator	Calculation of Message-Correlator	Calculation of Message-Correlator
1	Legacy SMS	GSM 7 bit default alphabet	GSM 7 bit default alphabet	Convert original message text payload into UTF-8	Convert received message text payload into UTF-8	Convert received message text payload into UTF-8

ID	Messaging Technology	Original Encoding type	Client Sender		Client Receiver	Messaging Server is Receiver from SMS-C via SMPP or MAP
			Encoding for Sending Message	Calculation of Message-Correlator	Calculation of Message-Correlator	Calculation of Message-Correlator
2		UCS2	UCS2	Convert original message text payload into UTF-8	Convert received message text payload into UTF-8	Convert received message text payload into UTF-8
3		Other alphabets using an 8 bit encoding (e.g. Latin-1, HP Roman-8)	Not applicable [1]	Not applicable[1]	Not applicable[1]	Convert received message text payload into UTF-8. The entity which converts from UCS2 or GSM 7 bit alphabets to other alphabets (e.g. Latin-1, HP Roman-8) shall ensure a one-to-one character mapping
4		Message with GSM 7 bit national alphabet (single shift and locking shift tables)	UCS2	Convert original message text payload into UTF-8	Convert received message text payload into UTF-8	Convert received message text payload into UTF-8
5		Message with GSM 7 bit national alphabet (single shift and locking shift tables)	GSM 7 bit (including shift tables[2]) (used by legacy SMS clients that are not RCS compliant)	Original message to UTF-8 using the GSM 7 bit national single shift and locking shift tables[2]	Convert received message text payload into UTF-8 using the GSM 7 bit national single shift and locking shift tables[2]	Convert received message text payload into UTF-8. Support of GSM 7 bit national single shift and locking shift table is required by the entity generating the Message-Correlator. NOTE: if a network entity is involved which converts this to another alphabet (e.g. Latin-1, HP Roman-8), a one-to-one mapping may not be possible and thus the Message-Correlator algorithm would not succeed.

ID	Messaging Technology	Original Encoding type	Client Sender		Client Receiver	Messaging Server is Receiver from SMS-C via SMPP or MAP
			Encoding for Sending Message	Calculation of Message-Correlator	Calculation of Message-Correlator	Calculation of Message-Correlator
6	Standalone Message (when stored as SMS)	UTF-8	UTF-8	When stored as SMS, use original message text payload which is already in UTF-8	When stored as SMS, use original message text payload which is already in UTF-8	Convert received message text payload into UTF-8. NOTE: For standalone messages with Pager Mode, it is possible based on Service Provider policies, for these messages to be automatically stored in the CMS as legacy SMS messages. In this case the client will not be able to correlate these messages with their copy in the Common Message Store based on the procedures for matching of messages defined in [GSMA PRD-RCC.11]. Since the client is not able to correlate the standalone messages with Pager Mode based on the procedures defined in [GSMA PRD-RCC.11], it shall then attempt to correlate them as per this row, i.e. using Message-Correlator. See section 4.1.16.7.

Table 138: Encoding conversions for Message-Correlator algorithm

NOTES to Table 138:

1. A client encoding SMS messages using 8 bit encoding alphabets (e.g. Latin-1, HP Roman-8) instead of GSM 7 bit or UCS2 or a client sending encrypted messages over SMS are not supported by the current algorithm.
2. Correlation of SMS messages using national single shift and locking shift tables indicated in the user data header Info Element is not fully supported, especially if either the client or the entity in the network has no access to the SMS PDU or does not support the used national single shift or locking shift tables. Therefore an RCS Client shall encode SMS messages as UCS2.

Additional considerations:

- For the correlation of outgoing messages the From field is not used

- For the correlation of incoming messages the To field is not use
- The correlation is achieved by Message-Correlator header value, using a case-sensitive comparison.
  The matching algorithm should take into account differences in the presentation of the address string according to different types of numbers.
  The creation of a Message-Correlator header value used for the correlation via a full string match requires in some scenarios access to the native SMS Transfer Protocol Data Units (TPDU, i.e. the TP-UD, TP-DCS data units). Client implementations that do not have access to the TPDU but only to the "interpreted" payload of the short message or if the message contained characters encoded via single or locking shift tables may compensate for this by using alternative matching algorithms which are out of scope for this specification.

4.1.11 Dealing with Collisions

The correlation field values are used to correlate between SMS messages on the Common Message Store and on the device. Specifically, when the device synchronises with the Message Store Server it will obtain UIDs and the correlation field values for those SMS messages that are new or have changed since the last synchronisation. The device will then attempt to correlate the UIDs and correlation field values with any messages it has received or subsequently receives from the network. Therefore, if any of the messages have the same correlation field values (this is considered a correlation “collision”) then the device cannot distinguish between them when matching to its local messages.

The device should compare the direction (originating or terminating) in addition to comparing the correlation field values, meaning that correlation collisions can only occur on messages with the same direction.

Correlation collisions can occur in these two cases:

1. Messages in the same thread with the same content, typically when they are chronologically close (so returned on the same synchronization) SMS messages in the same thread with the same content, such as successive replies both saying “OK”.
2. Messages in the same thread with content that is different only after the first 140 bytes. This is more likely when higher numbers of messages are being compared, for example, a likely worst case example would be when a phone has been switched off for a long period (e.g. a vacation, a repair). This rare scenario is not addressed here further.

If there are collisions, the device should identify the chronologically first received message on the device with the lower UID on Message Store Server.

For example, suppose Message Store Server returns two new messages both with the same value C for the Message-Correlator header but with UIDs x and y, x < y, and the device has received two messages with the same value C for the Message-Correlator header at times t1 and t2, t1 < t2. Then the device should identify t1 = x and t2 = y.

The same principle applies when the number of correlation collisions on the device is different from the number on the Message Store Server; those are usually cases of temporary lack of synchronisation between the device and the Common Message Store.

As an example, suppose as above the Common Message Store has the same two new messages but the device has only received one message with value C for the Message-Correlator header. It should identify that with UID x, in the presumption that the network will shortly deliver a second message with value C for the Message-Correlator header which it will then identify with UID y. Similarly, if the Common Message Store only has UID x producing value C for the Message-Correlator header but the device has both t1 and t2, the device should identify t1 with message x and expect a subsequent synchronisation to return message y which it will then identify with t2.

Note that some legacy messages might not have been stored in Common Message Store by the network. Therefore the length of time between the messages should be considered by the client when determining whether the messages are duplicates. Note also that the device would have to take into account messages the device might have that it received before the Common Message Store was in place.

The impact of correlation collisions in this method may result in a wrong correlation; in the case above, to identify t1 = x and t2 = y when the correct mapping was in fact t1 = y and t2 = x. In this case, the view from one device and another will be out of sync: a user making a state change to t1 on one device will see it applied to t2 on the other device, when they would expect it to apply to message y.

For example, take the case of successive identical messages. If the user marks on one device the earlier of these messages as a favourite, then the device view might be as follows:

> “are you still on for tonight?”
> “yes” <- FAVOURITE
> “do you have the tickets?”
> “yes”

whereas on another device the view would be:

> “are you still on for tonight?”
> “yes”
> “do you have the tickets?”
> “yes” <- FAVOURITE

No messages are lost, so there is no need to define any more advanced methods.

4.1.12 Recording of SMS messages

Apart from text transfer SMS provides a number of enhanced messaging capabilities as well as device and service control functions. For the synchronisation of SMS messages across devices it is essential that the client and the Common Message Store assume a common rule for SMS message recording.

Short messages are recorded in the Common Message Store either by the network or by the device based on the definitions in section 4.1.8.1. Entities recording short messages need to follow these guidelines.

NOTE: If the client has no access to the SMS PDUs, it may assume that all locally received messages that it has access to fulfil the rules for recording.

Clients matching received messages with a message in the Common Message Store need to consider these guidelines to apply matching only for messages that are subject to recording.

For clients fetching messages from the Common Message Store to update the local storage no special considerations are required.

The following PDU types need to be recorded for mobile originated SMS:

- SMS-SUBMIT
- SMS-STATUS-REPORT

The following PDU types need to be recorded for mobile terminated SMS:

- SMS-DELIVER

For a definition of the SMS PDU types refer to [3GPP TS 23.040].

4.1.12.1 Recording of SMS-SUBMIT and SMS-DELIVER

A short message may consist of multiple parts, the content transferred in the actual user data and the content of the user data headers inserted in the user data. The following sections define the recording rules for the two parts of the message.

4.1.12.1.1 User Data

The User Data is the field of a Short Message PDU that carries the user content (see [3GPP TS 23.040]). This section provides the rules for recording of messages based on the values of the SMS Data Coding Scheme and Protocol Identifier fields [3GPP TS 23.040].

4.1.12.1.2 SMS Data Coding Scheme

The SMS Data Coding Scheme field of the short message indicates the encoding used for the user data.

The indication of text compression and character set are used by the recording entity to decode the message content. The message is not stored in the Common Message Store if the Character Set indicates "8 bit data". In all other cases the message is converted in text using the UTF-8 character set for storage. If the User Data is the only content of an SMS message it is stored in the Common Message Store the body of a CPIM message.

The SMS Class determines the routing of a mobile terminated short message on the device. Messages without a Message Class shall be recorded based on the principles defined in section 4.1.8.

A mobile terminated message indicating Class "0" shall not be recorded in the network. It is sent to the mobile device via SMS. The device will display the message immediately without storing. If the user decides to store the message locally on the device after display the client shall upload the message to the Common Message Store without applying the Correlation Mechanism defined in section 4.1.8. The client shall store the message under the Default folder identified by the identity of the Contact in the Conversation. If this folder is not yet created in the Default folder, the client shall first create it. The client shall store the message with a Message-Context header set to pager-message as described in section 4.1.8.

Mobile terminated messages with Message Class "1", "2" and "3" shall be recorded by the network or the client unless content encoding (e.g. "8 bit data") or the high layer protocol indication prohibit recording.

If the Data Coding Scheme indicates a message for automatic deletion after reading it shall not be recorded.

Messages with a Message Waiting Indication shall not be recorded.

4.1.12.1.3 Protocol Identifier

The Protocol Identifier indicates whether a higher layer protocol is used to encode the content of the User Data. Only messages with a Protocol Identifier set to the default 0x00 and to values from the range 0x81-0x87 (Replace Short Message Type) shall be recorded.

If a Replace Short Message Type is set in the Protocol Identifier of a mobile terminated short message (value in the range of 0x81-0x87), then the client shall apply the handling as defined in [3GPP TS 23.040], i.e. it replaces the content of a stored message that matches the Replace Type of the new message. If the client configuration parameter SMS MESSAGE STORE is set to value "1" or "2" the client shall update the Common Message Store according to the result of the local message processing. If the client stores a message in the Common Message the Replace-Short-Message-Type header value shall be set to the value received in the short message.

The entity in the network storing the message in the Common Message Store shall apply the following handling. If a message with a Replace Short Message Type is received, it shall select the folder where the conversation is stored, as identified by the originator address. It shall search in the folder the messages with the same Replace Short Message Type as the new message.

If a message with the same Replace Short Message Type header is found, the \Deleted flag shall be set for this message. The new message is stored in the selected folder with a Replace Short Message Type header set.

If no related message or folder exists the message is stored in the regular fashion. The Message is always stored with the Replace-Short-Message-Type header value set to the value received in the short message.

4.1.12.1.4 Short Message Object

A short message shall be recorded in the Common Message Store with the headers and attributes set as defined below.

Attribute	Status	Content
From	Mandatory	For mobile originated short messages it contains the public user identity of the sender. It is either encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261]. For mobile terminated short messages the value is derived from the SMS originator address. If the originator is identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If the originator is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an alphanumeric string, the From field contains the string only.
To	Mandatory	For mobile originated short messages the value is derived from the SMS destination address. If the destination is identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If the destination is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. For mobile terminated short messages it contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261].
Date	Mandatory	Indicates the time the message was recorded. Date format is according to [CPM-MSGSTOR-REST]
Conversation-ID	Mandatory	It shall be assigned by the entity that stores the message
Contribution-ID	Mandatory	It shall be assigned by the entity that stores the message.
Correlation-ID	Mandatory	IMDN Message ID will be the Object ID of the IMDN within the Message Store and assigned by the store. The IMDN is correlated to the original message using the correlation ID of that CPM message.
Correlation-Tag	Optional	If present it shall contain the SMS message Correlator for message correlation as defined in section 4.1.9. See also section 4.1.8.
Message-Context	Mandatory	Message-Context shall be set to "pager-message"
Direction	Optional	Direction attribute value shall be set as defined in section 4.1.16.8.
Message-ID	Optional	It is assigned by the entity that stores the message. If stored, the header value shall conform to the definitions of [RFC5322].
Replace-Short-Message-Type	Optional	Indicates the replacement type of the short message. It can have the values 1 – 7. The value shall be taken from the value of SMS Protocol Identifier (see [3GPP TS 23.040])

Attribute	Status	Content
Content-Type	Mandatory	Message/CPIM
CPIM	Mandatory	The attribute “CPIM” contains the entire CPIM message headers as a single string. The other fields and attributes are as defined in Table 140 below. In particular, the Content-Type attribute is the content type of the encapsulated MIME message body. The payload(s) of the message object are the body part(s) of the encapsulated MIME message body of the CPIM message.

Table 139: Object Attributes of the Short Message Object

The CPIM attribute of the Short Message Object shall contain the CPIM headers and their values defined in Table 140. The message body shall the contain the content only.

CPIM Header	Status	Content
From	Mandatory	For mobile originated short messages it contains the public user identity of the sender. It is either encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261]. For mobile terminated short messages the value is derived from the SMS originator address. If the originator is identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If the originator is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an alphanumeric string, the From field contains the string only.
To	Mandatory	For mobile originated short messages the value is derived from the SMS destination address. If the destination is identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If the destination is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. For mobile terminated short messages it contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or a SIP URI as defined in [RFC3261].

CPIM Header	Status	Content
DateTime	Mandatory	For Mobile Terminated Messages it should indicate the time the message was received in the Originator Service Centre. Derived from the SMS Service Centre Time Stamp (see [3GPP TS 23.040]). If the SMS Service Centre Timestamp is not available it should contain the time of message recording. For Mobile Originated Messages it should indicate the time the message was sent by the client that recorded the message or the time the message was received in the Originator Service Centre.
rcs.Message-Correlator	Optional	If present it shall contain the SMS message Correlator for message correlation as defined in section 4.1.9. See also section 4.1.8.
rcs.Message-Context	Optional	The value of rcs.Message-Context shall be set to "pager-message". It shall be present if the rcs.Message-Correlator header is present. It indicates that the rcs.Message-Correlator contains the SMS message correlation as defined in section 4.1.9.
rcs.Service-Centre-Address	Optional	Indicates the address of the Short Message Service Centre used for the transfer of the short message. For mobile originated messages it is derived from the transport destination address. For mobile terminated messages it is derived from the transport originating address. See [3GPP TS 23.040]
rcs.Reply-Path	Optional	Indicates whether a reply-path exists for the message. If the value is set to "1" the reply-path exists, if set to "0" or the header is not present the reply-path does not exist. For a description of the reply-path refer to [3GPP TS 23.040].
rcs.Replace-Short-Message-Type	Optional	Indicates the replacement type of the short message. It can have the values 1 – 7. The value shall be taken from the value of SMS Protocol Identifier (see [3GPP TS 23.040])
Content-Type	Mandatory	For messages with text part only the Message body should be encoded using content type text/plain in UTF-8 encoding.
Content-Transfer-Encoding	Optional	Typical content transfer encoding shall be used, e.g. quoted-printable or base64

Table 140: CPIM Headers of the Short Message Object

Example of a recorded short message:

```
POST http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel:+19717774171/objects HTTP/1.1
Accept-Encoding: gzip,deflate
MIME-Version: 1.0
Accept: application/json
Authorization: Bearer
PAT_bceNdO3GSKwkHBkL1rnEceW2TTLx3ijGzbonv3qFvVuYHnocDopEZ2eyMfzea/2POmQrLn
YJch2VDeEA2ooRh4o5RoiNxpUVvp+e4P+GDYfkL8+5SwGYjVIBhO+AaDS8wWxYA4lA/qk9lUO+
DEhCcp+JHP2QDqvhvschz0ZUesIOe/cLgAQpMhD2k2mOhtL3BO6N5zPm1rBLeM8QjaMHKfdoDg
BnWBFKTf+NGlHwbutkX2ZAfApQ4VCL7wryWkuBRNpWVHacFfjC+kxJy+IYdno5VC2yxEj3l9Mp
gJgpEpbSAf5JtpMhBX+Il+It9vG8
```

```
Content-Type: multipart/form-data; boundary=ZQ87HPOZX.OBADLDBLD
Content-Length: 951
Host: nms-sib01.si.enclab.MNO.net
Proxy-Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.1.1 (java 1.5)

--ZQ87HPOZX.OBADLDBLD
Content-Disposition: form-data; name=root-fields
Content-Type: application/json; charset=UTF-8

{
  "object": {
    "attributes": {
      "attribute": [{
        "name": "Subject",
        "value": ["Test Object"]
      }, {
        "name": "From",
        "value": ["+19995551212"]
      }, {
        "name": "Date",
        "value": ["2015-04-01T14:30:30Z"]
      }, {
        "name": "Message-Context",
        "value": ["pager-message"]
      }, {
        "name": "To",
        "value": ["+19717774171"]
      }, {
        "name": "Direction",
        "value": ["In"]
      }, {
        "name": "CPIM",
        "value": ["From:+19995551212\r\nTo:+19717774171\r\nDate:2015-04-
               01T14:30:30Z\r\nrContent-Type:text/plain\r\nNS:rcs
               <http://www.gsma.com>\r\nrcs.Message-Context:pager-
               message"]
      }]
    },
    "flags": { "flag": []}
  }
}
--ZQ87HPOZX.OBADLDBLD
Content-Disposition: form-data; name=message
Content-Type: multipart/mixed; boundary=EFO94Y8QA.T8UCA0REW

--EFO94Y8QA.T8UCA0REW
Content-Disposition: attachment; filename=sms; name=sms
Content-Type: text/plain

this is a test SMS message
--EFO94Y8QA.T8UCA0REW--
```

--ZQ87HPOZX.OBADLDBLD—

4.1.12.1.5 User Data Header

User Data Headers can be used for encoding for SMS control functions as well as special SMS content (see [3GPP TS 23.040]. The type of content of a User Data Header is identified by the Information Element Identifier (IEI) within a User Data header. The following Information Identifiers need to be considered for the processing of short messages.

Concatenated short messages reference numbers IEI need to be considered when processing a concatenated message. Only the re-assembled message shall be recorded in the Common Message Store.

NOTE: The recording requirements for other values of Information Element Identifiers are for further study.

4.1.12.2 Recording of SMS-STATUS-REPORT

The SMS-STATUS-REPORT informs the message sender about the status of a previously sent message if it has been requested by the sender. The SMS-STATUS-REPORT shall be used by the entity storing SMS messages in the Common Message Store to record a delivery notification as defined in [CPM-MSGSTOR-REST].

The SMS-STATUS-REPORT is matched on the originating side to the original sent message by use of SMS Message Reference assigned by the originating device. The SMS Message Reference is not globally unique, thus the matching of SMS-STATUS-REPORT to the original short message across multiple devices has some limitations.

A delivery notification shall be recorded if the Status field in the SMS-STATUS-REPORT indicates that the short message has been "received by the SME" (see [3GPP TS 23.040]). In all other cases a delivery notification shall not be recorded.

A SMS delivery notification shall be recorded in the Common Message Store with Object Attributes and CPIM headers set as follows.

Attribute	Status	Content
From	Mandatory	It contains the address of the recipient of the original message as derived from the recipient address field of the SMS-STATUS-REPORT see [3GPP TS 23.040]. If identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context.
To	Mandatory	It contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261].
Date	Mandatory	Indicates the time the message was recorded
Conversation-ID	Mandatory	It shall have the same value as the original sent message.

Attribute	Status	Content
Contribution-ID	Mandatory	It shall have the same value as the original sent message.
Message-ID	Optional	It is assigned by the entity that stores the message. If stored, the header value shall conform to the definitions of [RFC5322].
Content-Type	Mandatory	Message/CPIM
CPIM	Mandatory	The attribute “CPIM” contains the entire CPIM message headers as a single string. The other fields and attributes are as defined in Table 142 below. In particular, the Content-Type attribute is the content type of the encapsulated MIME message body. The payload(s) of the message object are the body part(s) of the encapsulated MIME message body of the CPIM message

Table 141: Object Attributes of the SMS Delivery Notification

The CPIM attributes of the SMS Delivery Notification shall contain the CPIM headers and their values defined in Table 142:

CPIM Header	Status	Content
From	Mandatory	It contains the address of the recipient of the original message as derived from the recipient address field of the SMS-STATUS-REPORT see [3GPP TS 23.040]. If identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context.
To	Mandatory	It contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966]
DateTime	Mandatory	It should indicate the time the message was delivered by the Service Centre. The value is derived from the Discharge Time of the SMS-STATUS-REPORT (see [3GPP TS 23.040]). If it is not available then it should contain the time of message recording.
Content-Type	Optional	message/imdn+xml

Table 142: CPIM Headers of the SMS Delivery Notification

The message body of the CPIM message shall contain an IMDN with status set to "delivered" and the imdn.Message-ID assigned to the original sent message.

4.1.13 Recording of MMS messages

MMS provides a rich messaging service for multimedia content to be sent to a single recipient or to a list of recipients.

Multimedia messages are recorded in the Common Message Store either by the network or by the device based on the definitions in section 4.1.8.1. Entities recording multimedia messages need to follow these guidelines.

The following PDU types need to be recorded for mobile originated MMS:

- MM1 Submission
- MM1 Delivery Report
- MM1 Read-Reply Report

The following PDU types need to be recorded for mobile terminated MMS:

- MM1 Delivery
- MM1 Read-Reply Report

4.1.13.1 Recording of MM1 Submission and MM1 Delivery

The entity storing a MMS message in the Common Message Store shall use the Multimedia Message object defined in this section.

A multimedia message shall be recorded in the Common Message Store with the headers set as defined below.

NOTE: The encoding of addresses in Object Attributes headers of a recorded MMS message follows the SIP encoding principles as defined in [CPM-MSGSTOR-REST] whereas the address encoding of an MMS is based on the MMS Addressing Model of [MMSENC]. The entity storing a MMS message needs to re-format address headers.

For an MMS message to multiple recipients the entity storing the message shall store the participant list in a recipient-list-history content part with *copyControl* set in accordance with the recipient address type, To:, Cc: or Bcc:.

Attribute	Status	Content
From	Mandatory	For mobile originated MMS messages it contains the public user identify of the sender. It is either encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261]. For mobile terminated MMS messages the value is derived from the MMS originator address. If the originator is identified by a E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the originator is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address it is encoded as mailbox as defined in [RFC5322].

Attribute	Status	Content
To	Optional	For mobile originated MMS messages the value is derived from the MMS recipient address "To" field. If the recipient is identified by an E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the recipient is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address it is encoded as mailbox as defined in [RFC5322]. For a MMS message to multiple recipients the To header value shall be encoded as [RFC5322] address-list with each address value following the encoding rules above. In addition the multiple recipient addresses shall be represented in a body part of the message containing a recipient-list-history as defined in [RFC5365]. For mobile terminated MMS messages it contains the public user identity of the recipient. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or as SIP URI as defined in [RFC3261]. At least one of To, Cc or Bcc header shall be present in the message.
Cc	Optional	For mobile originated MMS messages the value is derived from the MMS recipient address "Cc" field. If the recipient is identified by an E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the recipient is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address it is encoded as mailbox as defined in [RFC5322]. For a MMS message to multiple recipients the Cc header value shall be encoded as [RFC5322] address-list with each address value following the encoding rules above. Multiple recipient addresses shall be represented in a body part of the message containing a recipient-list-history as defined in [RFC5365]. For mobile terminated MMS messages it contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or a SIP URI as defined in [RFC3261]. At least one of To, Cc or Bcc header shall be present in the message.

Attribute	Status	Content
Bcc	Optional	For mobile originated MMS messages the value is derived from the MMS recipient address "Bcc" field. If the recipient is identified by an E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the recipient is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address it is encoded as mailbox as defined in [RFC5322]. For a MMS message to multiple recipients the Bcc header value shall be encoded as [RFC5322] address-list with each address value following the encoding rules above. Multiple recipient addresses shall be represented in a body part of the message containing a recipient-list-history as defined in [RFC5365]. For mobile terminated MMS messages it contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or a SIP URI as defined in [RFC3261]. At least one of To, Cc or Bcc header shall be present in the message.
Date	Mandatory	Indicates the time the message was recorded
Conversation-ID	Mandatory	It shall be assigned by the entity that stores the message
Contribution-ID	Mandatory	It shall be assigned by the entity that stores the message.
Message-ID	Optional	It is assigned by the entity that stores the message. If stored, the header value should conform to the definitions of [RFC5322]. In a transition period Service Providers may store in the Message-ID header the value of the MMS Message-ID for compatibility with earlier versions of this specification.
Message-Correlator	Mandatory	It shall contain the MMS Message-ID for message correlation as defined in section 4.1.9. See also section 4.1.8
Message-Context	Mandatory	Message-Context shall be set to "multimedia-message"
Direction	Optional	Direction attribute value shall be set as defined in section 4.1.16.8.
Content-Type	Mandatory	Message/CPIM
CPIM	Mandatory	The attribute “CPIM” contains the entire CPIM message headers as a single string. The other fields and attributes are as defined in Table 144 below. In particular, the Content-Type attribute is the content type of the encapsulated MIME message body. The payload(s) of the message object are the body part(s) of the encapsulated MIME message body of the CPIM message.

Table 143: Object Attributes of the Multimedia Message Object

The CPIM attribute of the Multimedia Message Object shall contain the CPIM headers and their values defined in Table 144. The body of the Multimedia Message Object shall contain the content only.

CPIM Header	Status	Content
From	Mandatory	For mobile originated MMS messages it contains the public user identity of the sender. It is either encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261]. For mobile terminated MMS messages the value is derived from the MMS originator address. If the originator is identified by a E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the originator is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address it is encoded as mailbox as defined in [RFC5322].
To	Optional	For mobile originated MMS messages the value is derived from the MMS recipient address "To" field. If the recipient is identified by an E.164 number it is encoded in the global-number representation of an tel URI as defined in [RFC3966] If the recipient is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address, it is encoded as mailbox as defined in [RFC5322]. For a MMS message to multiple recipients there shall be To and Cc header fields per recipient address. Bcc recipients addresses shall be contained in a To header field. If a client needs to represent the Bcc destination address classification it shall use the header from the mail headers instead. Multiple recipient addresses shall be represented in a body part of the message containing a recipient-list-history as defined in [RFC5365] with a mapping of the address header field type to copyControl parameter. For mobile terminated short messages it contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or a SIP URI as defined in [RFC3261]. At least one of To or Cc header shall be present in the message.

CPIM Header	Status	Content
Cc	Optional	For mobile originated MMS messages the value is derived from the MMS recipient address "Cc" field. If the recipient is identified by an E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966]. If the recipient is identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If the originator is identified by an e-mail address, it is encoded as mailbox as defined in [RFC5322]. For a MMS message to multiple Cc recipients there shall be a Cc header field per recipient address. At least one of To or Cc header shall be present in the message.
DateTime	Mandatory	For Mobile Terminated Messages it should indicate the time the message was received in the Originator Service Centre. Derived from the MMS Date header (see [MMSENC]). If the Date header is not available, it should contain the time of message recording. For Mobile Originated Messages it should indicate the time the message was sent by the client that recorded the message or the time the message was received in the Originator Service Centre.
Subject	Optional	Indicates the Subject of the MMS message. It shall be taken from the Subject header of the MMS message.
imdn.Disposition-Notification	Optional	This header indicates whether the sender has requested a MMS read-reply report. The entity recording the message shall derive the value from the MMS X-Mms-Read-Reply header defined in [MMSENC]. If the X-Mms-Read-Reply header is present and set to "yes", the imdn.Disposition-Notification header shall take the value "display". Otherwise, the header shall be absent.
rcs.Message-Correlator	Mandatory	It shall contain the MMS Message-ID.
rcs.Message-Context	Mandatory	The value of rcs.Message-Context shall be set to "multimedia-message". It indicates that the rcs.Message-Correlator contains the MMS message correlation (MMS Message-ID) as defined in section 4.1.9.
rcs.Mms-Message-Class	Optional	The MMS Message Class indicates the class of the multimedia message. It may take the values "Personal", “Advertisement", "Informational" and "Auto". Multimedia messages with value "Auto" typically contain MMS read report. If rcs.Mms-Class is absent the value "Personal" shall be assumed.

CPIM Header	Status	Content
Content-Type	Mandatory	The Content-Type defines the type of content of the body part of the CPIM message which is likely to be a multipart. For the user content the Content-Type shall be taken from the original multimedia message. All types of content allowed for MMS are relevant for storage in the Common Message Store, e.g. presentation, text, audio and video. For definitions of the MMS Message Body structure refer to [MMSENC]. For a resource list body the content type as defined in [RFC4826] shall be used.

Table 144: CPIM Headers of the Multimedia Message Object

Example of a recorded two part multimedia message:

```
POST http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel:+19717774171/objects HTTP/1.1
Accept-Encoding: gzip,deflate
MIME-Version: 1.0
Accept: application/json
Authorization: Bearer
PAT_bceNdO3GSKwkHBkL1rnEceW2TTLx3ijGzbonv3qFvVuYHnocDopEZ2eyMfzea/2POmQrLn
YJch2VDeEA2ooRh4o5RoiNxpUVvp+e4P+GDYfkL8+5SwGYjVIBhO+AaDS8wWxYA4lA/qk9lUO+
DEhCcp+JHP2QDqvhvschz0ZUesIOe/cLgAQpMhD2k2mOhtL3BO6N5zPm1rBLeM8QjaMHKfdoDg
BnWBFKTf+NGlHwbutkX2ZAfApQ4VCL7wryWkuBRNpWVHacFfjC+kxJy+IYdno5VC2yxEj3l9Mp
gJgpEpbSAf5JtpMhBX+Il+It9vG8
Content-Type: multipart/form-data; boundary=I937053OH.VXS8GY5VX
Content-Length: 1022
Host: nms-sib01.si.enclab.MNO.net
Proxy-Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.1.1 (java 1.5)

--I937053OH.VXS8GY5VX
Content-Disposition: form-data; name=root-fields
Content-Type: application/json; charset=UTF-8

{
  "object": {
    "attributes": {
      "attribute": [{
        "name": "Subject",
        "value": ["Test Object"]
      }, {
        "name": "From",
        "value": ["+19995551212"]
      }, {
        "name": "Date",
        "value": ["2015-04-01T14:30:30Z"]
      }, {
        "name": "Message-Context",
        "value": ["multimedia-message"]
```

```
      }, {
        "name": "To",
        "value": ["+19717774171"]
      }, {
        "name": "Direction",
        "value": ["In"]
            }, {
        "name": "CPIM",
        "value": ["From:+19995551212\r\nTo:+19717774171\r\nDate:2015-04-
        01T14:30:30Z\r\nrContent-Type:multipart/mixed\r\nrcs.Message-
        Correlator:6HUU0UN9C.C4F0GD8SR\r\nrcs.Message-Context :multimedia-
        message\r\nNS:rcs <http://www.gsma.com>"]
        }]
    },
    "flags": { "flag": []},
    "correlationId": "6HUU0UN9C.C4F0GD8SR"
  }
}
--I937053OH.VXS8GY5VX
Content-Disposition: form-data; name=message
Content-Type: multipart/mixed; boundary=PFLOG1DC7.NVJS9F8UL

--PFLOG1DC7.NVJS9F8UL
Content-Disposition: attachment; filename=body; name=body
Content-Type: text/plain

this is the text body of a multimedia message
--PFLOG1DC7.NVJS9F8UL--
--I937053OH.VXS8GY5VX--
```

4.1.13.2 Recording of MM1 Delivery Report

The entity storing a mobile originated MMS messages in the Common Message Store shall also store MM1 Delivery Reports.

A MM1 Delivery Report is matched on the originating side to the original sent message by use of MMS message-id value. In case of multiple recipients the address of the recipient from which the delivery report has been received needs to be taken into account.

The MM1 Delivery Indication shall be used by the entity storing MMS messages in the Common Message Store to record a Delivery Notification as defined in [CPM-MSGSTOR-REST].

A delivery notification shall be recorded if the MMS Status field in the MMS delivery report indicates that the short message has been "retrieved" (see [MMSENC]). In all other cases a delivery notification shall not be recorded.

A MMS delivery notification shall be recorded in the Common Message Store with Object Attributes and CPIM headers set as follows.

Attribute	Status	Content
From	Mandatory	It contains the address of the recipient of the original message as derived from the MMS deliver indication, see [MMSENC]. If identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If identified by an e-mail address it is encoded as mailbox as defined in [RFC5322].
To	Mandatory	It contains the public user identity of the sender of the original message. It is encoded in the global-number representation of a tel URI as defined in [RFC3966] or as a SIP URI as defined in [RFC3261].
Date	Mandatory	Indicates the time the message was recorded
Conversation-ID	Mandatory	It shall have the same value as the original sent message.
Contribution-ID	Mandatory	It shall have the same value as the original sent message.
Message-ID	Optional	It is assigned by the entity that stores the message. If stored, the header value should conform to the definitions of [RFC5322]. In a transition period Service Providers may store in the Message-ID header the value of the MMS Message-ID for compatibility with earlier versions of this specification.
Message-Correlator	Mandatory	It shall contain the MMS Message-ID for message correlation as defined in section 4.1.9. See also section 4.1.8. It has the same value as the Message-Correlator header of the original sent message.
Content-Type	Mandatory	Message/CPIM
CPIM	Mandatory	The attribute “CPIM” contains the entire CPIM message headers as a single string. The other fields and attributes are as defined in Table 146 below. In particular, the Content-Type attribute is the content type of the encapsulated MIME message body. The payload(s) of the message object are the body part(s) of the encapsulated MIME message body of the CPIM message.

Table 145: Object Attributes of the MMS Delivery Notification

The CPIM attribute of the MMS Delivery Notification shall contain the CPIM headers and their values defined in Table 146:

CPIM Header	Status	Content
From	Mandatory	It contains the address of the recipient of the original message as derived from the MMS deliver indication, see [MMSENC]. If identified by a E.164 number it is encoded in the global-number representation of a tel URI as defined in [RFC3966] If identified by a non E.164 number then it is encoded in the local-number representation of a tel URI as defined in [RFC3966] without parameters and context. If identified by an e-mail address it is encoded as mailbox as defined in [RFC5322].
To	Mandatory	It contains the public user identity of the sender. It is encoded in the global-number representation of a tel URI as defined in [RFC3966]
DateTime	Mandatory	It should indicate the time the message was delivered by the Service Centre. The value is derived from the Date field of the MM1 Delivery report, see [MMSENC]. If it is not available then it should contain the time of message recording.
rcs.Message-Correlator	Mandatory	It shall contain the MMS Message-ID for message correlation as defined in section 4.1.9. See also section 4.1.8. It has the same value as the rcs.Message-Correlator header of the original sent message.
Content-Type	Optional	message/imdn+xml

Table 146: CPIM Headers of the MMS Delivery Notification

The message body of the CPIM message shall contain an IMDN with status set to "delivered" and the imdn.Message-ID assigned to the original sent message.

4.1.13.3 Recording of MMS Read Reports

According to [MMSCTR] there are two methods in MMS to transfer a Read Report from the recipient to the sender.

1. Multimedia Message Read Report:
   After display of a MMS message for which the originator has requested a read reply and the sending of a read report is authorised by the recipient user the client generates an "automatic" MMS message and sends it back to the sender. The entity storing MMS messages in the Common Message Store shall store the multimedia message read report message as a normal MMS message as defined in section 4.1.13.1. It is essential that the storing entity sets the value of the rcs.Mms-Message-Class header with the value received in the message, i.e. "auto".
   Clients fetching a MMS message and the related multimedia message read report from the Common Message Store should present it in the message history in the same way as if the message and the read report would have been received via MMS.
2. PDU Read Report:
   The implementation of recording of MMS PDU Read Reports is for further study.

A client sending a MMS Read Report to the sender shall not regard it as a display IMDN notification which is used by the messaging server participating functions as a trigger to set the \Seen flag in the Common Message Store. Therefore the client shall follow the

procedures for the setting of the \Seen flag in Common Message store defined in section 4.1.14 as if an IMDN display notification was not requested.

4.1.14 Optimisations for UNI operations to Common Message Store

If the MESSAGE STORE EVENT REPORTING configuration parameter is enabled (see section A.1.3), the RCS client shall use the CPM event reporting framework procedures (see section 6.7 of [GSMA PRD-RCC.11]) in an established 1-to-1 Chat session.

NOTE: Section 6.7 of [GSMA PRD-RCC.11] specifies the functionality as CPM IMAP events with a content type "*application/vnd.oma.cpm-event-imap+xml*". The functionality can be used in combination with the RESTful based message store access used in this section though.

The RCS client shall use this for the following cases between RCS client and Participating Function, in order to report:

1. when one or more messages have been read by the RCS user, so that the Participating Function can set the “\Seen” flag for the stored message object(s) in CMS on behalf of the RCS client; and,
2. When a message was deleted by the RCS user, so that the Participating Function can set the “\Deleted” flag for the stored message object(s) in CMS.

To set the “\Seen” flag (case 1):

When the RCS user has read/displayed a received message, the RCS client shall inform the Common Message Store so that the message will be shown as “read” on other user devices after synchronizing with the Common Message Store.

This can be realised in the following ways:

1. If an IMDN display notification was requested for the received message and one was generated by the RCS client, the Participating Function (B2BUA) receiving the IMDN display notification from the client (via MSRP or SIP MESSAGE request) shall set the “\Seen” flag in CMS for the messages reported;
2. If an IMDN display notification was not requested, or if the RCS user settings on the client disabled sending them for read messages, then the CPM event reporting framework is used to report to the Participating Function that a message was read by the RCS client.

To set the “\Deleted” flag (case 2):

When the RCS user has deleted a received message, the RCS client shall inform the Common Message Store so that the message will be shown as “deleted” on other user devices after synchronizing with the Common Message Store. This is realised by the CPM event reporting framework being used to report to the Participating Function that a message was deleted by the RCS client.

4.1.15 A Common File Store for File Transfer via HTTP

4.1.15.1 Overview

For File Transfer as defined in section 3.2.5, the storage for files/content transferred via File Transfer may be kept separate from the smaller, text based content and metadata. It also may be desirable to keep received content for longer than the original validity period. For these purposes, a Common File Store may be used together with a Common Message Store. The Common Message Store stores the chat messages carrying the File Transfer content body containing the meta-information and links for the thumbnail (optional) and the file(content), while the Common File Store stores the actual (i.e. local) copy of the thumbnail and/or content referred to by those URL links.

Similar to the Common Message Store, the Common File Store may be used to synchronise files between devices in the recipient's network in addition to keeping a back-up of files in the local network.

When the Common File Store is deployed, it shall behave towards the client as a HTTP Content Server supporting both sender procedures for File Upload (see section 4.2.2) and procedures for File Download (see section 4.2.3).

For the Common File Store to work with the Common Message Store a Service Provider deploying these shall support functions for File Download localisation in the terminating network as defined in section 4.1.15.3.

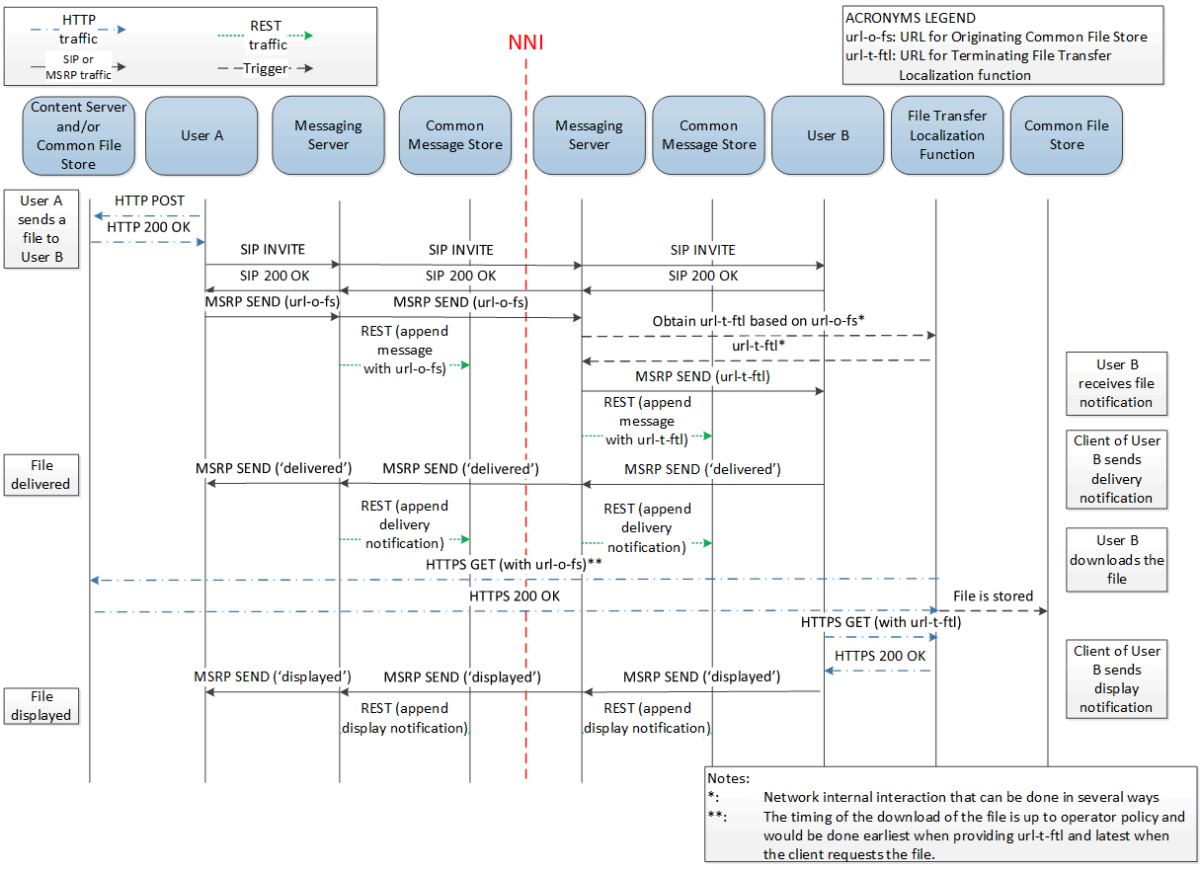


Figure 54: File Transfer Localisation Function: global flow

4.1.15.2 File Upload Procedures

The file upload procedures defined in section 4.2.2 apply. The Service Provider shall direct the files uploaded by the sender towards a permanent storage with or without triggering the File Transfer Localisation Function. The Service Provider can assign a specific HTTP Content Server URI for different types of clients (e.g. legacy) via the configuration parameter FT HTTP CS URI defined in section A.1.4.

4.1.15.3 Localisation in the terminating Service Provider's network

Localisation is the process to ensure that the terminating Service Provider is involved in the download of files referred to in the URLs conveyed using the messaging services. Localisation applies to the URLs

- for the file and thumbnail contained in a File Transfer body defined in section 4.2.4,
- for the media and thumbnail contained in a Rich Card's body as described in section 3.6.10.5.3,
- conforming to the definitions of the HTTP Content Server URL as defined in section 4.2.5 in the text body of a Standalone Message.

Once downloaded, the terminating Service Provider shall store files in the Common File Store if localisation is done for the purposes of offering a Common File Store. Otherwise, it is considered implementation-specific whether the file is stored in the Terminating Service Provider’s local storage.

Localisation requires

- a function to localise the download URL received via the messaging service by replacing the remote download URL by a local download URL. A Service Provider performing localisation shall apply localisation of the download URL via

  - the client-based local download of files as defined in section 4.2.3, and/or
  - procedures in the terminating messaging server as defined in section 4.1.15.3.1.

  The local download URL shall be resolvable to the remote download URL. It may be used to convey additional meta information.

  NOTE: If the terminating Service Provider relies on CFS as defined in section 3.2.3.8.1 and if the terminating Service Provider has no control over the content of SMS messages that are sent on File Transfer fall-back as defined in section 3.2.5.4.3.2, then the client-based local download of files approach can be used only.

- a function to process the local download of files in the network that

  - processes the client's request to download the file via the local download URL,
  - resolves the remote download URL,
  - downloads the file from the originating network and,
  - manages the local storage of the file in the Common File Store.

  This function is provided via the Localisation Function defined in section 4.1.15.3.3.

4.1.15.3.1 Terminating Messaging Server Localisation Procedures

If the Service Provider applies localisation of the URL in the terminating Messaging Server, the terminating Messaging Server shall, when receiving a Chat, Group Chat, Chatbot or Standalone message, screen the message content for bodies containing download URLs.

If a relevant message body with URL(s) is found, then the Messaging Server shall

- either
  - not deliver the message in which the URL was conveyed if the remote download URL does not comply to the format defined in section 4.2.5 and the URL was received in a File Transfer body in a conversation other than a Chatbot Conversation (as described in section 3.6.8) and
  - create the local download URL(s) complying to the URL format defined in section 4.2.5 otherwise,
- or
  - as a deployment option, invoke the Localisation Function to convey the remote download URL(s) to the Localisation Function, to obtain the local download URL(s) and optionally to update the validity of the file. The technical procedures for the trigger processing of the Messaging Server and the Localisation Function are out of the scope of this document.

The terminating Messaging Server shall update the message to contain the local download URL(s) and optionally the new validity of the file(s). The terminating Messaging Server shall record the message resulting from the processing in the Common Message Store.

4.1.15.3.2 File Download Procedures

The file download procedures defined in section 4.2.3 apply. The procedures apply for cases where the client retrieves the file

- in result of reception of a Chat or Standalone Message with a File Transfer content body or any other content URL the client received via RCS messaging received from another user or sent by the own user as indicated via the CPIM "Message-Direction" header defined in section C.1.9 of [GSMA PRD-RCC.11],
- in result of retrieval of a message object with a File Transfer content body from the Common Message Store which is received from another user or sent by the own user as indicated via the "Direction" attribute defined in [CPM-MSGSTOR-REST].
- in result of the retrieval of a message object with a Rich Card object from the Common Message Store.

Since the validity attribute of a file in the File Transfer message body indicates the validity for the first time download request, the client shall respect the value of the validity parameter only for messages received via Chat or Standalone Message which are not synchronised with the Common Message Store as defined in section 4.1.16.7. Once the client sends the file download request, the file is localised and made available for future retrievals. In the case where the Common File Store is deployed with the Common Message Store, for the XML in File Transfer messages retrieved from the Message Store Server, the validity in the

XML is not relevant and should be ignored since the validity period configured for localised files in the Common File Store applies.

NOTE: When a Common File Store is deployed, it is up to service provider policy whether the received files are assumed to be archived if the Archived flag is set for the link to the file.

4.1.15.3.3 Localisation Function Procedures

If the Localisation Function is invoked by the Messaging Server via the deployment option defined in section 4.1.15.3.1, then the Localisation Function shall interact accordingly, i.e. the Localisation Function

- shall if the domain of a remote download URL received in a File Transfer body received in a conversation other than a Chatbot Conversation (as described in section 3.6.8), does not comply with section 4.2.5, return an error response to the Messaging Server, and
- otherwise, shall for a given remote download URL provide the local download URL complying to the URL format defined in section 4.2.5, and
- may provide a new validity for the file.

The Localisation Function may download the file from the originating network using the remote download URL the earliest when invoked by the Messaging Server, but latest when it receives a download request from the client, as per Service Provider policy. Once downloaded and if localisation was done to support a Common File Store, the Localisation Function shall store the file in the Common File Store in the Service Provider network, in relation to the remote download URL.

If the Localisation Function receives a download request from the client, then the Localisation Function

- shall support the file download procedure defined in section 4.1.15.3.2,
- shall determine based on the provided URL whether the file is stored in the Common File Store in the Service Provider network
- if stored already, the Localisation Function shall provide the file from the Common File Store, otherwise
- the Localisation Function shall attempt to download the file from the originating network as defined in section 4.2.3. The Localisation Function shall relay responses received from the originating network to the client. If the file is downloaded successfully and if localisation was done to support a Common File Store, then the Localisation Function shall store it in the Common File Store in the Service Provider network.

4.1.15.4 Mutual Authentication for HTTP File Transfer Localization

NOTE: The solution options in this section are optional for implementation and only if agreed by the service agreements between the interconnected MNOs.

For UNI traffic, it is not feasible for an HTTP Content Server to maintain all of an HTTP Content Server’s Clients’ Certificates, where the Clients are the UEs. Due to the number of

devices within an MNO, only 1-way Authentication is applied to UNI HTTP File Transfer traffic (where UE authenticates the HTTP Content Server, and not vice versa). However, with HTTP File Transfer Localization for NNI traffic, on a per Interop-Partnership basis, there exists only one HTTP Content Server Client. While the HTTP Content Server Client can authenticate the HTTP Content Server, in order for the HTTP Content Server to verify the identity of HTTP Content Server Client, the HTTP Content Server must request the SSL Certificate from the Client. In order for an HTTP Content Server to employ 1-way authentication for UNI traffic and employ 2-way authentication for NNI traffic, the HTTP Content Server must have a means to distinguish UNI traffic and NNI traffic.

This solution addresses a lack of Mutual Authentication between an HTTP Content Server Client and an HTTP Content Server, by providing a means for the HTTP Content Server to discern UNI and NNI traffic, and subsequently enforces 2-way authentication appropriately.

The HTTP Content Server can use “2 Ports” or “2 IPs” for such distinction. Should the originating MNO chooses to use the “2 Ports” for UNI/NNI traffic distinction, the only means to deliver the ports information to the terminating MNO HTTP Content Server, is through using SRV records. This is referred to as Option 1. Should the originating MNO choose to use “2 IPs” for the distinction, the originating MNO can deliver the IPs using Host (A/AAAA) records. NNI-prefix prepending to the HTTP File Transfer Content location URL will be needed. May it be the originating or terminating MNO choosing to prepend an NNI-prefix, the originating MNO must supply an additional set of Host records for NNI. When the terminating HTTP Content Server queries DNS to learn the IP of the originating HTTP Content Server NNI FQDN, the originating DNS will return Host records with IPs that are different than the UNI set of Host records. This is referred to as Option 2.

For Option 2, there are three possible architectures based on agreement between originating MNO and terminating MNO, one of which must be chosen.

- The originating MNO may perform “NNI-prefix prepending” prior to sending the URL across the NNI.
- For a particular Interoperability relationship between the originating MNO and the terminating MNO, if both parties agreed to have the terminating MNO to perform “NNI-prefix prepending”, the terminating MNO may perform “NNI-prefix prepending” prior to sending the URL across the NNI.
- If a particular Interoperability relationship between the originating MNO and the terminating MNO fails to reach agreement (either the originating MNO network or the terminating MNO network), Option 1 may be pursued.

	MT MNO choosing Option 1 (SRV based)	MT MNO choosing Option 2 (NNI-prefix based)
MO MNO choosing Option 1 (SRV based)	MO MNO can choose a unique port (e.g. 8443). An SRV record must be provided by MO MNO, and it will contain the port number. Port Suffix must be absent inside the HTTP URL or the Port Suffix must contain a port that's the same as the port listed in the SRV record. MO MNO can choose a unique IP. An SRV record must be provided by MO MNO, and it will contain a unique FQDN. The FQDN will point to a unique Host record (A or AAAA).	MO MNO can choose a unique port (e.g. 8443). An SRV record must be provided by MO MNO, and it will contain the port number. Port Suffix must be absent inside the HTTP URL or the Port Suffix must contain a port that's the same as the port listed in the SRV record. MO MNO can choose a unique IP. An SRV record must be provided by MO MNO, and it will contain a unique FQDN. The FQDN will point to a unique Host record (A or AAAA).
MO MNO choosing Option 2 (NNI-prefix based)	Either originating MNO or terminating MNO must perform "NNI-prefix prepending". If the originating or terminating MNO chooses to prepend an NNI-prefix, the originating MNO must supply an additional set of Host records for NNI. When the terminating HTTP Content Server queries DNS learns the IP of the originating HTTP Content Server NNI FQDN, the originating DNS will return Host records with IPs that are different than the UNI set of Host records. MO MNO must choose a unique IP for NNI traffic, that's different than the IP for UNI. And this MO MNO must supply a Host record for the NNI FQDN.	Either originating MNO or terminating MNO must perform "NNI-prefix prepending". If the originating or terminating MNO chooses to prepend an NNI-prefix, the originating MNO must supply an additional set of Host records for NNI. When the terminating HTTP Content Server queries DNS learns the IP of the originating HTTP Content Server NNI FQDN, the originating DNS will return Host records with IPs that are different than the UNI set of Host records. MO MNO must choose a unique IP for NNI traffic, that's different than the IP for UNI. And this MO MNO must supply a Host record for the NNI FQDN.

Terminating HTTP Content Server discerns NNI/UNI FQDN, and uses port from SRV or IP from SRV record to establish connection for NNI traffic while using port 443 to establish connection for UNI traffic.

Either MO MNO or MT MNO in this section can be a Hub.

- 2-way authentication is required
- Option 1: SRV based
    - The SRV record can contain either
        - Ports, or
        - IP (an IP that's different than the IP for UNI traffic)
    - Port Suffix must be absent inside the HTTP URL or the Port Suffix must contain a port that's the same as the port listed in the SRV record.

- HTTP Content Server as a Client must
  - Make an explicit SRV query for localization.
  - If SRV record was not returned or an error was returned to the HTTP Content Server Client, then the Client must make a Hostname (A/AAAA) query.
- Recommendation for port selection is 8443.
- Recommendation for SRV record: _nni._tcp.ftcontentserver.rcs.<mnc>.<mcc>.pub.3gppnetwork.org.

- Option 2: NNI-prefix based (non-SRV based)
  - 2 sets of Host records – one for UNI and one for NNI
  - For Option 2, there are three possible architectures based on agreement between originating MNO and terminating MNO, one of which must be chosen.
    - The originating MNO may perform “NNI-prefix prepending” prior to sending the URL across the NNI.
    - For a particular Interoperability relationship between the originating MNO and the terminating MNO, should both parties agreed to have the terminating MNO to perform “NNI-prefix prepending”, the terminating MNO may perform “NNI-prefix prepending” prior to sending the URL across the NNI.
    - Should a particular Interoperability relationship between the originating MNO and the terminating MNO fails to reach agreement (either the originating MNO network or the terminating MNO network), Option 1 may be pursued.
  - The terminating network must make pre-fix NNI FQDN A/AAAA record query.

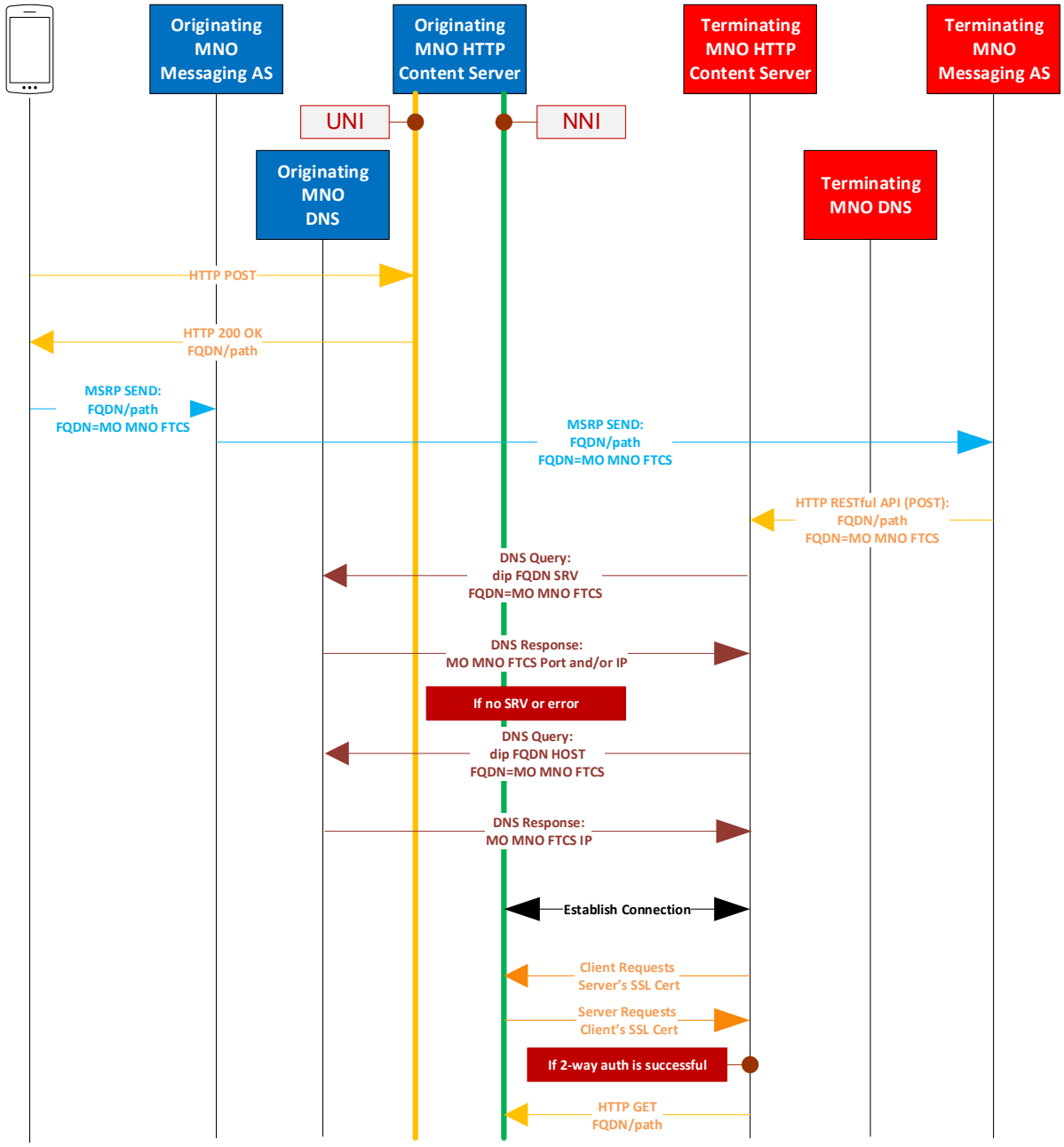


Figure 55: Mutual Authentication for HTTP File Transfer Localization Option 1 (SRV based)

For Option 2 (NNI-prefix based), Originating MNO may perform “nni-prefix prepending” or Terminating MNO may perform “nni-prefix prepending”, as outlined in solution above.

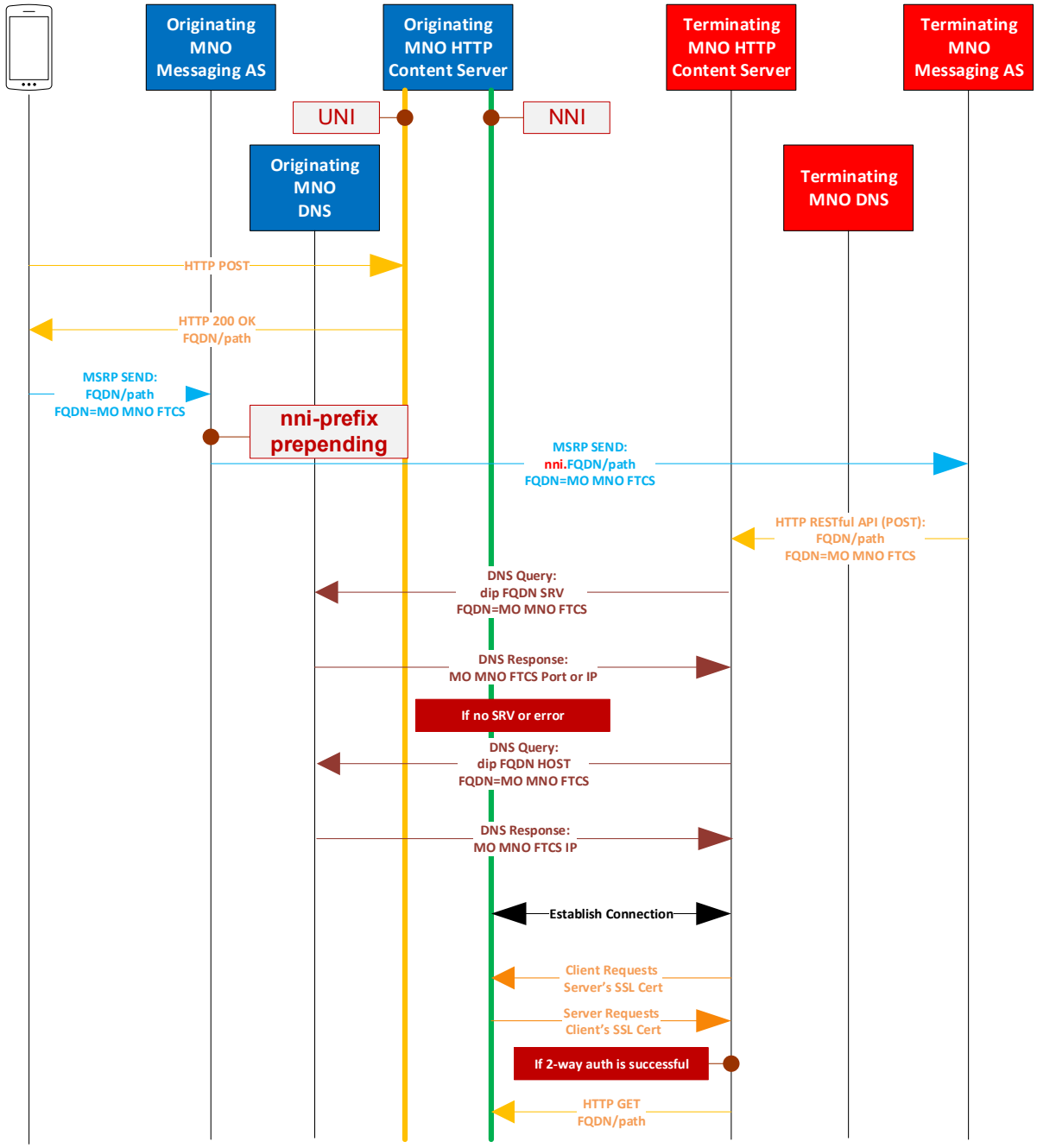


Figure 56: Mutual Authentication for HTTP File Transfer Localization Option 2 with Originating MNO performing "nni-prefix prepending"

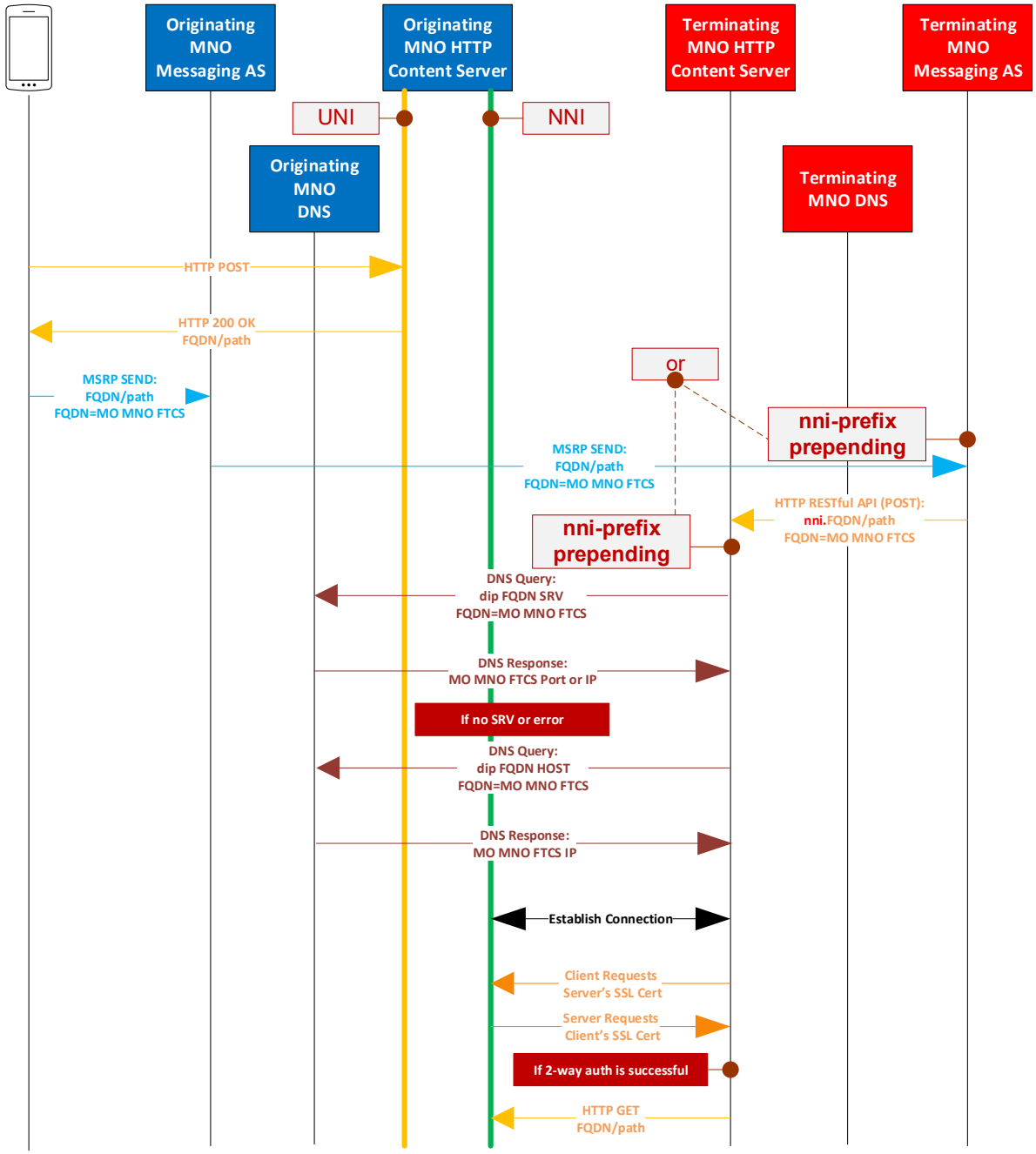


Figure 57: Mutual Authentication for HTTP File Transfer Localization Option 2 with Terminating MNO performing "nni-prefix prepending"

The RCS file transfers between MNOs can happen as direct peering partners or via a third-party carrier acting as RCS hub provider. There is another option using NAPTR records to transfer files which will cover both those direct file transfers and file transfers through hub provider(s). There will be an agreement between carriers and hub providers to use the option they choose and implement the necessary artifacts. When the end parties transfer these files, the URL link provided in MSRP message by the originating MNO indicates the location of the file. Without additional procedures, this presents a problem that may require the originating MNO HTTP Content Server provided URL to distinguish the URL location to be either the originating MNO HTTP Content Server location or the RCS hub provider's HTTP Content Server. Then, the Originating MNO has to change this URL depending on

the condition that the peering is a direct connection or via RCS hub provider(s). If the RCS hub transfers the file via localisation from the originating MNO and modifies the URL link in the MSRP message to point to its HTTP Content Server, then this is not required. However, if the RCS hub acts as a proxy, meaning it only passes the MSRP message without modifying the URL in the MSRP message, but does localisation of the file then the URL in the MSRP message, the link needs to be handled differently by the terminating network. In this scenario the burden is put on the originating MNO to determine if the file is going to be retrieved from the RCS hub provider's HTTP Content Server or its own and then change the URL accordingly. There is a need to remove this burden from the Originating MNO.

The following solution solves the problem by using NAPTR records to determine the location of the file.

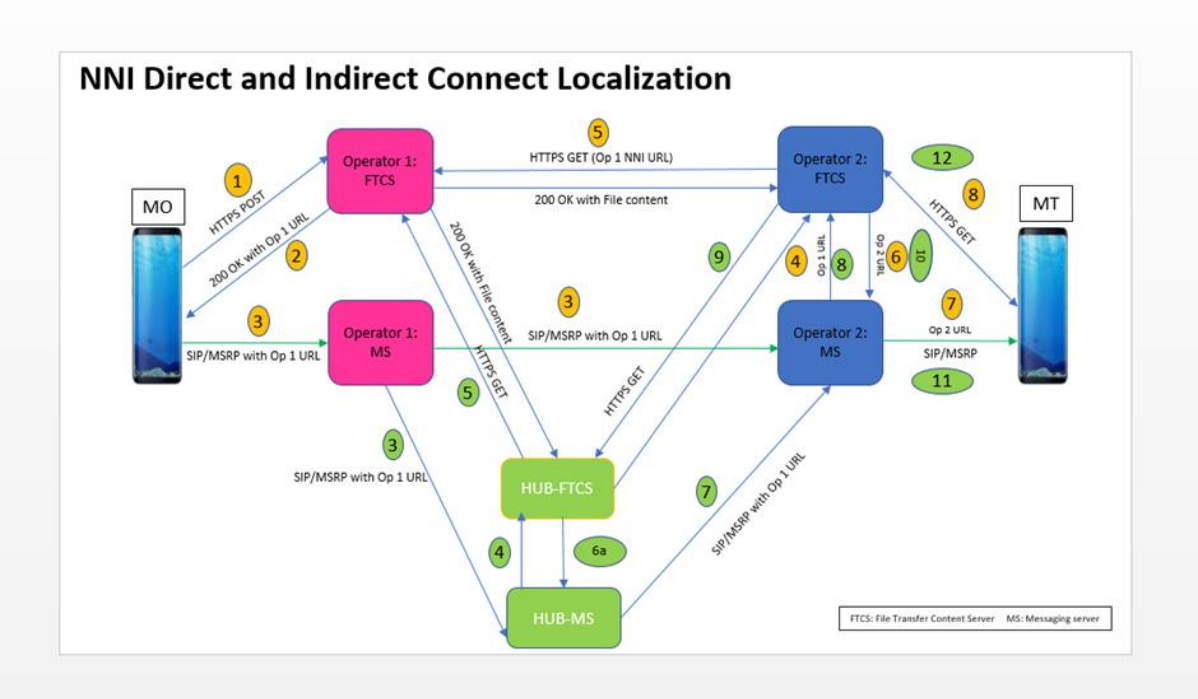


Figure 58: NNI Localisation with direct and indirect connections

NOTE: For historic reasons Figure 58 and subsequent figures in this section refer to the HTTP Content Server as File Transfer Content Server (FTCS).

The approach presented in Figure 58 addresses this issue and removes the burden on the Originating MNO to provide different URLs depending on the peering arrangement. This approach solves the problem by using NAPTR records to determine the location of the file. The Terminating MNO requests the NAPTR record for the domain in the URL and determines whether they need to use the RCS hub for the File Transfer or use the direct peering service This removes the burden of changing the URL based on the peering arrangement. It only requires the terminating MNO to determine dynamically what the peering arrangement is applicable. This enables the terminating MNO to determine the location of the file that is to be transferred. This solution is based on [RFC3403].

```
ftcontentserver.rcs.gtm.mnc482.mcc311.pub.3gppnetwork.org
   ;;       order pref flags
   IN NAPTR 100   10   "s"   "RCS_NNIFT:https"         ( ; service
                             ""                    ; regexp

_nni._tcp.ftcontentserver.rcs.mnNNN.mccXXX.pub.3gppnetwork.org.  ;
replacement
                                                   )
 IN NAPTR 200   30   "a"   "RCS_NNIFT:https"        ( ; service
                             ""                    ; regexp
 ftcontentserver.rcs.mnNNN.mccXXX.pub.3gppnetwork.org.
     ; replacement
                )
   IN NAPTR 200   20   ""    "RCS_NNIHUB:https"        ( ; service
                             ""                    ; regexp
                             RCS_HUB.com    ; replacement
                                                   )
```

Table 147: NAPTR record example

The following is the process for determining what FT service to use

- Two services are defined:
  - Direction connection: RCS_NNIFT:https
  - Connection through hub: RCS_NNIHUB:https
- At the receiving MNO, the FTCS retrieves all the NAPTR records associated with the target domain name

  `(ftcontentserver.rcs.gtm.mnc482.mcc311.pub.3gppnetwork.org).` These are to be sorted in terms of increasing ORDER and increasing PREF within each ORDER.
- Starting with the first sorted NAPTR record, the HTTP Content server examines the SERVICE field to find a match. In the case of the S-NAPTR DDDS application, this means a SERVICE field that includes the tags for the desired application service and a supported application protocol. If more than one NAPTR record matches, they are processed in increasing sort order.
- The operator determines if they want to perform the NNI transfer through a hub for this domain
  - For a hub, the operator looks for the service RCS_NNIHUB:https. This service as seen in Table 147 shows the domain of the hub provider.
  - The operator requests NAPTR records for that domain. And proceeds
- The operator determines if they want to perform the NNI transfer directly from the provided domain
  - The MNO looks for the service RCS_NNI:https, this service as seen in Table 147 shows the terminal S record for the originating MNO. The MNO requests S records and proceeds to get the AAA record.

- In case the S record cannot be obtained the MNO back tracks the NAPTR record for this service and discovers the next terminal record to be an A record, the operator makes the attempt with this FQDN.

The following call flow diagram depicts Hub to Hub file transfer in details.

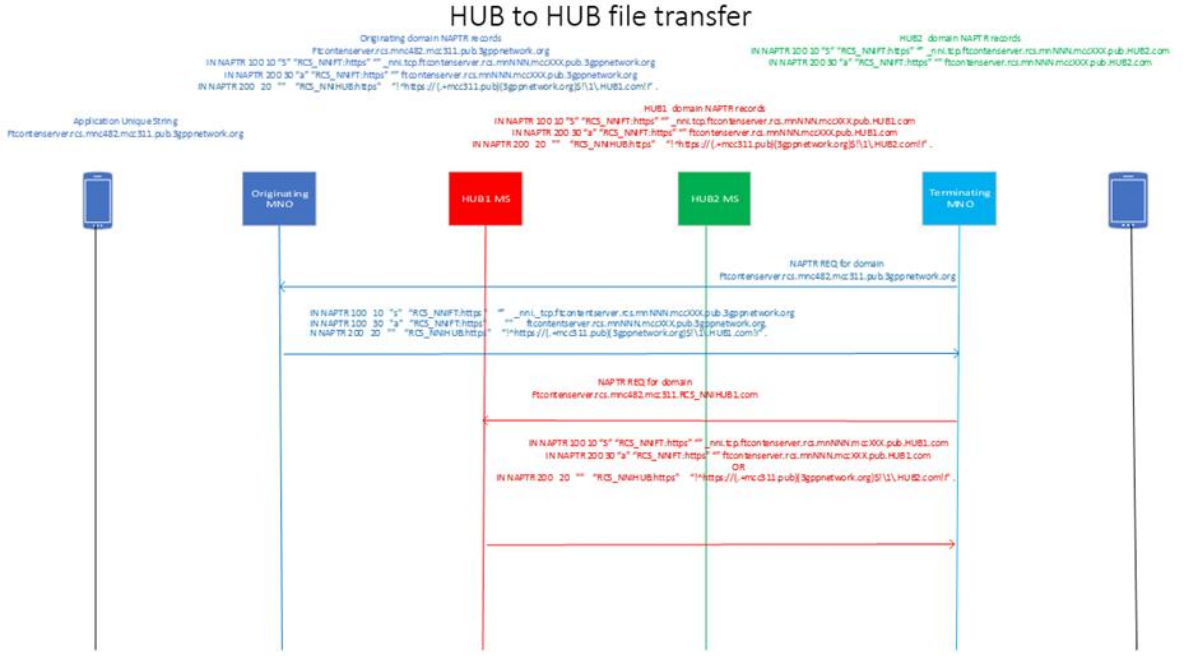


Figure 59: Mutual Authentication for HTTP File Transfer Localization Hub to Hub transfer (Part 1)

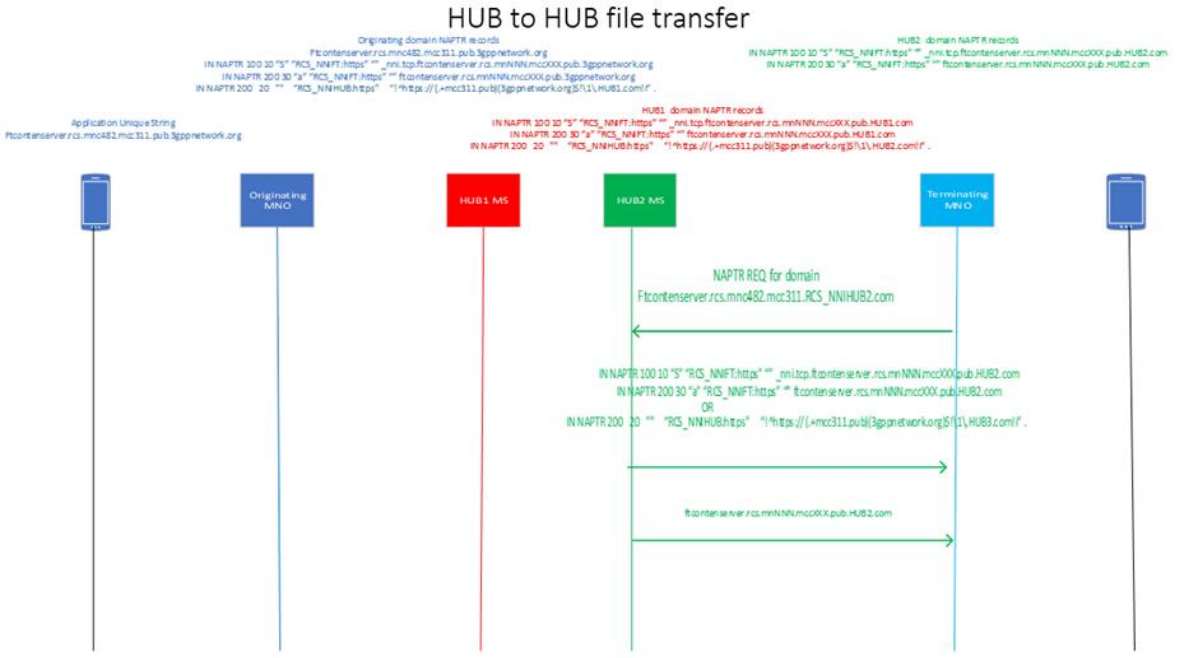


Figure 60: Mutual Authentication for HTTP File Transfer Localization Hub to Hub transfer (Part 2)

4.1.16 Client behaviour

Clients shall comply with the operations and procedures described in sections 5.5.2 of [CPM-SYS_DESC] and 6 of [CPM-MSGSTOR-REST]. Some further clarifications on the client expected behaviour when it interacts with the Message Store Server are presented in the following sections.

4.1.16.1 Storing new messages (Object Store Operation)

For the cases that the client stores new messages and there is no existing folder where these messages can be stored (e.g. a brand new conversation with user B offline and interworking procedures in place), the client needs to allocate the name to the new folder and follow the naming procedures as described in 6.4.1.3 of [CPM-MSGSTOR-REST].

4.1.16.2 Message Archive

When a user wants to archive a message, the client shall set the Archived flag on that message.

When a user wants to unarchive a message the client shall remove the Archived flag from that message.

For the case that permanent message storage is required due to Service Provider policy, the client is configured with the MESSAGE STORE ARCHIVE AUTH set to disabled and the client does not need to set the Archived flag.

4.1.16.3 Search Operation

Search operation shall first be performed locally when local storage exists. A search may be expanded to a network based search.

4.1.16.4 Message Displayed on a Client (Object Store Operation to set a flag)

When a client displays a previously unseen message for the user then it shall trigger the change of the message flag in the Common Message Store as follows:

If a message

- was originally received at a time when there the Common Message Store was permanently disabled for the client and it has not been uploaded (e.g. by user activity) or
- the message is kept only locally stored, e.g. after it was deleted from the Default folder

then the client shall manage the state only locally.

In all other cases the client shall attempt to update the message status in the Common Message Store with the procedures defined below.

If the message status is changed to seen and the client does not send a display notification then the client shall act depending on the value of the MESSAGE STORE EVENT REPORTING configuration parameter (see section A.1.3) as follows:

1. If the MESSAGE STORE EVENT REPORTING configuration parameter is disabled, the client shall set the \Seen flag for that message right after it displays the message as follows.

    - If the UID of the message for which the flag is to be set is not known because it was previously sent or received but not synchronised with the Common Message Store yet, the client shall first attempt to match it with the message in the Common Message Store as defined for synchronisation in section 4.1.16.7. If no match is found the message is handled as defined in section 4.1.16.7, otherwise it shall follow the steps below.
    - If there is no connection with the Message Store server, the client shall establish a connection to it.
    - If there is a connection to the server then the client shall use it to set the flag. If the session with the server is currently used for synchronisation as defined in section 4.1.16.7 the client shall

        - stop the synchronisation procedure at the next reasonable point (e.g. after completion of the ongoing API command or synchronisation action),
        - perform the procedure to change the flag,
        - continue the synchronisation from the point where it was stopped.

    - Then the client shall select the conversation folder where the message is stored on the server.
    - When setting the flag, the client shall follow procedures as described in section 6.6 of [CPM-MSGSTOR-REST].
    - If the Message Store server returns a success response the client shall inspect the updated flags in the response. If the result in the response is different than the local message status, then the client shall synchronise messages in the conversation folder in accordance with the definitions in section 4.1.16.7.
    - If the Message Store server returns an error response the client shall synchronise messages in the conversation folder in accordance with the definitions in section 4.1.16.7.

    Optimisations for setting the \Seen flags when many messages are displayed or deleted by the client in a short period of time are left to the device implementation, e.g. by keeping the Message Store connection open as long the user has the messaging application opened or until a client local inactivity timer expires.
    If the connection with the server has been established by the client specifically to manage message status but not by a synchronisation trigger defined in section 4.1.16.7 the client may use this session to run a full synchronisation as defined in section 4.1.16.7.

2. If the MESSAGE STORE EVENT REPORTING configuration parameter is enabled, the client shall inform the Messaging Server as per section 4.1.14 so that the Messaging Server will set the \Seen flag.

For client fetching flags, procedures as described in section 6.6 of [CPM-MSGSTOR-REST] apply.

4.1.16.5 Message Removal via Clients

When the client deletes a message from the Default folder from one device, all other clients belonging to the user shall also show that message as deleted.

When the client deletes a message which

- was originally received at a time when there the Common Message Store was permanently disabled for the client and it has not been uploaded (e.g. by user activity) or
- is kept only locally stored, e.g. after it was deleted from the Default system folder as defined in section 6.7 of [CPM-MSGSTOR-REST]

then the deletion is managed only locally.

In all other cases the client shall delete the message in the Common Message Store with the procedures defined below.

A client deletes a message from the Default folder in one of two ways, depending on the value of the MESSAGE STORE EVENT REPORTING configuration parameter (See section A.1.3):

1. If the MESSAGE STORE EVENT REPORTING configuration parameter is disabled, the client shall delete that message on the message store right after the local deletion. If the RESTful Call to the message store fails, or is not possible, the client is expected to keep a record of deleted items until the message store connection is completed and the items are successfully deleted. If there is a connection with the Message Store Server the client shall use it to delete the message.

   Optimisations for when many messages are being displayed or deleted by the client in a short period of time are to use the Bulk Delete API (section 6.4.16 of [CPM-MSGSTOR-REST]) and left to the device implementation.

2. If the MESSAGE STORE EVENT REPORTING configuration parameter is enabled, the client shall inform the Messaging Server as per section 4.1.14 so that the Messaging Server will delete the Object.

4.1.16.6 Client impact of Message Removal due to Service Provider Policy

When the Message Store Server deletes (expunges) messages in the Default folder because of e.g. message expiry, the client Shall Not remove these same messages from their own device.

Clients will not receive notifications for system deleted messages over the Notification Channel, as the Message Storage server shall not create notifications for the event type of “expired object” (see Notification event types in section B.2.2).

4.1.16.7 Synchronization

Client synchronization procedures are defined in section 6.3 of [CPM-MSGSTOR-REST].

NOTE: Client synchronization guidelines are built under the assumption that roaming scenarios are out of scope.

Clients using the RESTful interface shall use the notification channel to determine when to perform synchronization and this is described in section 6.2 of [CPM-MSGSTOR-REST].

When a RCS client is triggered for synchronisation it shall follow the procedures defined in section 6.7 of [CPM-MSGSTOR-REST]. Specifically it shall consider the folder structure as defined in section 4.1.7.

During synchronisation the client shall match all new sent or received messages and disposition notifications with messages of the Common Message Store as follows:

If

- a new message has been sent via SMS or MMS or
- a new message has been received via SMS or MMS or
- a delivery report has been received via SMS or MMS or
- a read report has been received or MMS or
- a flag has been changed on a message object

the client shall follow the procedures for matching of messages defined in sections 4.1.8.1 and 4.1.9.

If

- a message has been sent or
- a message has been received or
- a delivery notification has been received or
- a display notification has been received or
- a flag has been changed on a message object

via standalone messaging, chat, group chat and file transfer, the client shall follow the procedures for matching of messages as defined for these services in [CPM-MSGSTOR-REST].

For standalone messages with Pager Mode, it is possible based on Service Provider policies, to be automatically stored in the CMS as legacy SMS messages. In this case the client will not be able to correlate these messages with their copy in the Common Message Store based on the procedures for matching of messages defined in [GSMA PRD-RCC.11]. If the client is not able to correlate the standalone messages with Pager mode based on the procedures defined in [CPM-MSGSTOR-REST] it shall then attempt to correlate them based on the procedures defined for SMS messages in sections 4.1.8, 4.1.9, 4.1.10 and 4.1.11.

When a client attempts to match a new sent or received message with the Common Message Store and no match is found, then

- If the message was received via SMS or MMS and the corresponding client configuration parameter SMS MESSAGE STORE or MMS MESSAGE STORE

defined in Table 167 is set to value "1", then the client shall apply the procedure defined in section 4.1.8.1.

- In all other cases the client shall keep the message locally stored. The client shall try to match the message again at least in the next synchronisation. The client shall perform message management for this message only locally.

For a new sent or received message, if a match is found, then the client shall align the local message status with the status in the Common Message Store.

The procedures for the client for setting a message seen in the Message Store Server are described in section 4.1.16.4.

The procedures for the client to delete messages in the Message Store Server are described in section 4.1.16.5.

The client shall consider the size of messages to be retrieved by examining the message meta-data in the Notification list in order to avoid delaying the fetch of the body of basic messages (chat messages, standalone messages and SMS/MMS messages). The fetch of the body of larger messages may come later in the synchronisation procedure.

If a message was originally received at a time when the Common Message Store was permanently disabled or a message is kept only locally stored, e.g. after it was deleted from the "Default" system folder as defined in section 6.7 of [CPM-MSGSTOR-REST], then the client should not consider it for synchronisation. However the user or client may consider to upload such messages to the Default folder.

Upon synchronisation, clients shall find all messages that might have been deleted on the Common Message Store by other devices.

If a message in a conversation folder of the default folder is expunged, the client shall continue to keep the message locally (as it will not receive details of expunged objects).

4.1.16.8 Identification of Message Direction of Messages in a conversation

To simplify the client procedure to display messages as sent or received within a conversation an entity storing the message in the Common Message Store shall add the message transfer direction via a Direction attribute.

The Direction attribute should be stored for all types of user messages in the Common Message Store.

If the Direction attribute is present and the value is set to "Out", then the client shall display the message as sent from the own user. The client should disregard the address value in the From: header of the message. If the Direction attribute is present and the value is set to "In", then the client shall display the message as received by the own user. The client should disregard the address value in the To: header of the message.

If the Direction attribute is not present then the client should display the message direction based on a local device policy.

The Direction attribute is defined in [CPM-MSGSTOR-REST].

4.1.16.9 Identification of anonymised and non-anonymised messages in a Chatbot conversation

If the 'tk' URI parameter as defined in section 2.5.4.3 is set to a value different from ‘off’ in the SIP URI associated with the folder name, then the client shall display the message as being sent in an anonymised session with that Chatbot contact.

If the 'tk' URI parameter as defined in section 2.5.4.3 is set to ‘off’ in the SIP URI associated with the folder name, then the client shall display the message as being sent in a non-anonymised session with that Chatbot contact.

The client shall handle the messages from two folders with the same Chatbot service identifier but with different values of the 'tk' URI parameter as coming from the same Chatbot.

The client shall handle the messages from a folder with an SMS number associated with the same Chatbot service identifier as coming from a non-anonymous conversation with the Chatbot.

4.1.16.10 Message Bookmarks

Users can bookmark messages for their special attention.

When a user changes the bookmark of a message, and if the message

- was originally sent or received at a time when the Common Message Store was permanently disabled for the client and it has not been uploaded (e.g. by user activity) or
- the message is kept only locally stored, e.g. after it was deleted from the Default folder

then the client shall manage the bookmark only locally.

In all other cases the client shall manage the message bookmark in the Common Message Store with the procedures defined below.

If a bookmark is set for a message, then the client shall add the \Flagged flag for the message in the Common Message Store.

If a bookmark is removed from a message, then the client shall remove the \Flagged flag for the message in the Common Message Store.

The client shall manage the \Flagged flag as follows:

- If the UID of the message for which the flag is to be set or removed is not known because it was previously sent or received but not synchronised with the Common Message Store yet, the client shall first attempt to match it with the message in the Common Message Store as defined for synchronisation in section 4.1.16.7. If no match is found the message is handled as defined in section 4.1.16.7, otherwise it shall follow the steps below.
- If there is no connection with the Message Store server, the client shall establish a connection to it.

- If there is a connection to the server then the client shall use it to add or remove the flag. If the session with the server is currently used for synchronisation as defined in section 4.1.16.7 the client shall

    - stop the synchronisation procedure at the next reasonable point (e.g. after completion of the ongoing API command or synchronisation action),
    - perform the procedure to change the flag,
    - continue the synchronisation from the point where it was stopped.

- Then the client shall select the conversation folder where the message is stored on the server.
- When setting or removing the flag, the client shall follow procedures as described in section 6.6 of [CPM-MSGSTOR-REST].
- If the Message Store server returns a success response the client shall inspect the updated flags in the response. If the result in the response is different than the local message status, then the client shall synchronise messages in the conversation folder in accordance with the definitions in section 4.1.16.7.
- If the Message Store server returns an error response the client shall synchronise messages in the conversation folder in accordance with the definitions in section 4.1.16.7.

The procedures described in section 6.6 of [CPM-MSGSTOR-REST] apply for the client to fetch flags. A message shall be considered as bookmarked, if the \Flagged flag is set for a message.

4.2 HTTP Content Server

4.2.1 Overview

This section defines the procedures between a client and an HTTP Content Server for file upload and download for the use in RCS services as per corresponding service description.

An originating client uploads a file and optionally a thumbnail picture to the HTTP Content Server via the file upload procedures. In result of the upload procedure, the client is provided by the HTTP Content Server with a file transfer message body containing file meta data including the file download URLs. The client shall use the file transfer message body as per corresponding service description.

A terminating client receiving a file download URL as defined in a corresponding service description shall download the file via the receiver procedures.

4.2.2 File Upload Procedures

4.2.2.1 File Upload

The procedures of the originating client and the HTTP Content Server to upload a file and optionally a thumbnail file are defined as follows:

1. To initiate upload a file, the client shall

a) Given that the protocol scheme of the URL provided as value of the FT HTTP CS URI configuration parameter defined in section A.1.4 shall be "https" establish a TLS connection to the HTTP Content Server using the value of this configuration parameter and verify that a certificate is received from the HTTP Content Server and that this certificate is valid (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the expected domain for the API, etc.), see section 4.2.7. If no certificate is provided or the certificate is invalid, the client shall consider the HTTP Content Server to be unavailable. The client shall then consider the upload to have failed and shall not retry the upload using the resume upload procedure defined in section 4.2.2.2.

NOTE1: Given that for security reasons this client handling does not ensure a graceful fallback to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry

b) send a HTTP POST (see NOTE2) request without a body to the HTTP Content Server using the value of the FT HTTP CS URI configuration parameter defined in section A.1.4.
c) If the client supports user authentication via HTTP Embedded EAP-AKA as defined in Annex C of [GSMA PRD-RCC.14], then the client shall add to the query component of the URL the EAP_ID URI query parameter defined in section C.2 of [GSMA PRD-RCC.14]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986]. If the client supports authentication with an GBA bootstrapped security association as defined in [3GPP TS 33.220] it shall indicate this by addition of a GBA product token in the User-Agent header as defined in [3GPP TS 24.109].

NOTE2: This specification uses the term “HTTP POST” and “HTTP GET” as a generic reference to the action of using the POST or GET methods of HTTP. However, the use of a secure connection via HTTPS is required. This is enforced by the service provider having to configure a FT HTTP CS URI with the "https" schema as defined in section A.1.4.

2. The client shall continue processing depending on the received response from the HTTP Content Server:

a) HTTP 401 AUTHENTICATION REQUIRED error response carrying a WWW-Authenticate header field as defined in [RFC7235] if authentication is required. If the client and the Service Provider's HTTP Content Server supports GBA based authentication then the HTTP Content Server returns an HTTP 401 AUTHENTICATION REQUIRED response with an WWW Authenticate header instructing the client to use HTTP digest Authentication with a bootstrapped security association. In this case, the client shall authenticate with the bootstrapped security association as defined in [3GPP TS 24.109]. If the client has no bootstrapped security association in place, the client shall invoke the bootstrapping procedure defined in [3GPP TS 24.109].
Otherwise, the client shall authenticate using the values of FT HTTP CS USER

and FT HTTP CS PWD from the device configuration as defined in Table 168 in section A.1.4.

b) HTTP 204 NO CONTENT response, the client shall continue processing without authentication.
c) HTTP 503 INTERNAL ERROR with Retry-After header if the server is busy and cannot handle the request, the client shall retry to upload after the time specified in the Retry-After header.
d) A HTTP 302 FOUND response, the client shall change the type of the request to HTTP GET and follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the HTTP Content Server commencing in step 2 of this procedure.
e) A HTTP 403 FORBIDDEN response, the client is not authorised to upload a file to the HTTP Content Server.
f) Any other response, the client shall retry the request.

3. The client shall generate a HTTP POST request to upload the file to the HTTP Content Server as follows:

   The client shall create the following elements for the transfer of the file to the HTTP Content Server.

   - A File Transfer Transaction ID (TID): this TID value shall be a unique ID generated by the client according to [RFC4122] section 4.2;
   - An optional indication of the purpose of the upload: The values will be defined where they are used, and the indication shall not be provided for use cases where no value has been defined. If not provided, the HTTP Content Server shall assume that the upload is to support File Transfer as defined in section 3.2.5.

NOTE: Even if not included in the response defined in step 4 explicitly, the HTTP Content Server might apply specific policies for the expiry and download of the file based on the indicated purpose. Such policies are up to the RCS Service Provider to define.

   - The thumbnail content: The thumbnail is optional as it is only required for images and videos. The size of this thumbnail shall be smaller than 10 kByte;

     - For a picture, the raw binary result shall be a thumbnail of the picture itself. For a video clip, the raw binary result shall be a thumbnail either of the first I-Frame at 20% of the total length of the video clip or of another relevant frame. The procedure describing how to create the thumbnail itself, in its raw binary form, is out of scope of this specification.
     - The size of a thumbnail should be restricted to the minimum number of octets that provide significance.

   - The file content.

   The client shall use the elements defined above to create a HTTP POST content body using the *multipart/form-data* content type to encapsulate the following parts in the listed order:

- A mandatory form element with the name set to “tid” containing the transaction ID:

```
Content-Disposition: form-data; name="tid”
Content-Type: text/plain

<Transaction-ID generated by the client>
```

Table 148: First form of the HTTP POST method request to upload the file to the HTTP Content Server (Transaction ID)

- An optional form element with the name set to “purpose” indicating the purpose of the upload:

```
Content-Disposition: form-data; name="purpose”
Content-Type: text/plain

<purpose of the transfer>
```

Table 149: Second form of the HTTP POST method request to upload the file to the HTTP Content Server (purpose of upload)

- An optional form element with the name set to “Thumbnail” containing the thumbnail file content:

```
Content-Disposition: form-data; name="Thumbnail"; filename="<local_filename>"
Content-Type: [mime type depending on the thumbnail; e.g. image/jpeg]

<Thumbnail content>
```

Table 150: Third form of the HTTP POST method request to upload the file to the HTTP Content Server (Thumbnail contents)

- A mandatory form element with the name set to “File” containing the file content:

```
Content-Disposition: form-data; name="File"; filename="<local_filename>"
Content-Type: [mime type depending on the file; e.g. image/jpeg]

<file content>
```

Table 151: Third form of the HTTP POST method request to upload the file to the HTTP Content Server (file contents)

The client should include the Content-Length header to indicate the size of the HTTP request body, as described in [RFC7230]. If present, the Content-Length shall indicate the size of HTTP POST body part, i.e. the multipart/form-data entity body.

If the client was requested to authenticate the user in the previous step, then the client shall include an Authorization header as follows:

- If the client was requested to authenticate the user via the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD, then the

client shall add an Authorization header to the HTTP POST request in accordance with the requested authentication scheme as per [RFC2617] using the *FT HTTP CS USER* and *FT HTTP CS PWD* configuration parameters as credentials. The client shall include the qop directive if provided by the HTTP Content Server.

- If the client was requested to authenticate the user via a bootstrapped security association, then the client shall use the stored key material and the B-TID to generate keys specific to the HTTP Content Server as defined in [3GPP TS 33.220]. The client shall add an Authorization header to the HTTP POST request generated from the key material and the B-TID.

The client shall send the HTTP POST request using the HTTP Content Server URL derived from the client configuration parameter FT HTTP CS URI defined in section A.1.4.

4. The following cases apply for the result returned by the HTTP Content Server:

a) If the upload is successful, the client shall get a HTTP 200 OK response containing a HTTP Content Server response body complying with the schema defined in section 4.2.4 specifying:

i. The URL, size, content type and validity for the thumbnail, if applicable.
ii. The URLs, size, filename, content type and validity for the file.

```
<?xml version="1.0" encoding="UTF-8"?>
<file xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttp"
      xmlns:x="urn:gsma:params:xml:ns:rcs:rcs:up:fthttpext">
      <file-info type="thumbnail">
            <file-size>[thumbnail size in bytes]</file-size>
            <content-type>[MIME-type for thumbnail]</content-type>
            <data url = "[HTTP URL for the thumbnail]" until = "[validity of the thumbnail]"/>
      </file-info>
      <file-info type="file">
            <file-size>[file size in bytes]</file-size>
            <file-name>[original file name]</file-name>
            <content-type>[MIME-type for file]</content-type>
            <data url = "[HTTP URL for the file]" until = "[validity of the file]"/>
            <x:branded-url>[alternative branded HTTP URL of the file]</x:branded-url>
      </file-info>
</file>
```

Table 152: HTTP Content Server response: XML contained in the body

Please note that referring to the XML body in Table 152:

- The thumbnail part is only included if the sender uploaded a thumbnail to the server.
- The validity of the files shall be specified by providing the date the files shall be removed on the server using the [ISO8601] format including the date and time in UTC (Coordinated Universal Time) time zone (e.g. 2007-04-05T14:30:00Z). The validity depends on the configuration the originating Service Provider has set on the HTTP Content Server.

The client usage of the elements of the HTTP Content Server response body is defined in the corresponding service definitions.

b) If the upload is not successful, then:

i. if the HTTP Content Server returns a HTTP 401 AUTHENTICATION REQUIRED and if the client has requested authentication via the values of client configuration parameters FT HTTP CS USER and FT HTTP CS PWD, then the client shall trigger a configuration request to the Auto-Configuration Server via the procedures defined in section 2.3.2.
ii. if the HTTP Content Server returns a HTTP 401 AUTHENTICATION REQUIRED and if the client has requested authentication via the bootstrapped security association then the client shall invoke the bootstrapping procedure defined in [3GPP TS 24.109].
iii. if the HTTP Content Server returns a HTTP 403 FORBIDDEN response, then the client is not authorised to upload a file to the HTTP Content Server.
iv. if the server is busy and cannot handle the request, a HTTP 503 INTERNAL ERROR with Retry-After header. The client shall retry to upload after the time specified in the Retry-After header for a maximum of three (3) times.
v. if any other error is returned, then the client shall automatically retry the upload as described in section 4.2.2.2.

An example flow of the file upload procedure including the mandatory form elements is shown in Figure 61.

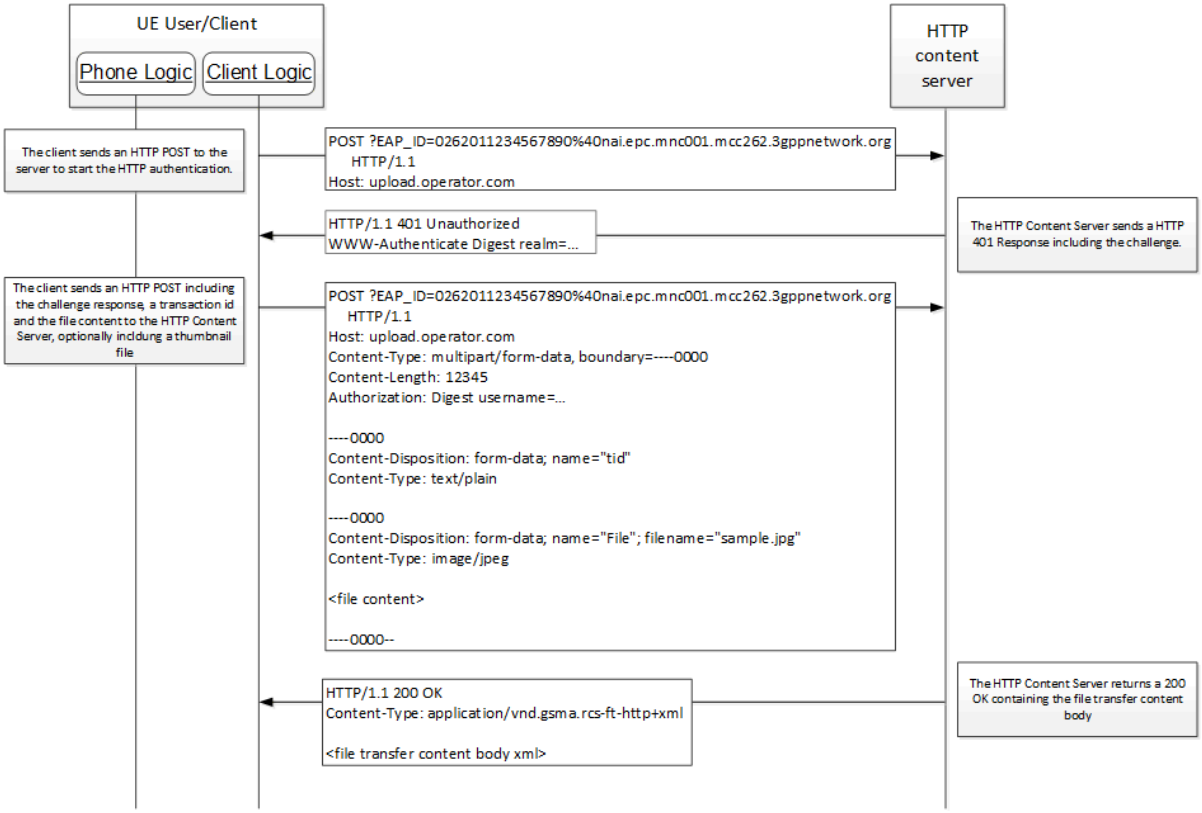


Figure 61: File Upload

4.2.2.2 File Upload Resume

In case a file upload cannot be completed, e.g. because the file sender loses network coverage, the client shall resume the file upload by using the procedure described in this section. It is intended to resume the upload of the file itself but not of an optional thumbnail which has small size. The HTTP Content Server shall store partial uploads and make them accessible via the related TID defined in section 4.2.2.1. A Service Provider policy to remove partially uploaded files after some time may apply, thus the resume upload may be possible only for a limited time after the last upload or upload resume attempt.

Overall the client shall retry per file upload up to a maximum of three (3) times after which the upload cannot be resumed and the complete file needs to be uploaded again following the procedure in section 4.2.2.1. The following procedure shall be used to resume a failed upload request:

1. A client that intends to resume the upload of an interrupted file upload shall first fetch the upload information of the file by

   a) establishing a TLS connection to the HTTP Content Server using the value of the FT HTTP CS URI configuration parameter defined in section A.1.4 (because the protocol scheme of the URL provided as value of this configuration parameter shall be "https"), and verifying whether a valid certificate is received from the HTTP Content Server (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the expected domain for the API, etc., see section 4.2.7). If no certificate is provided or the certificate is invalid, the

client shall consider the HTTP Content Server to be unavailable. The client shall then consider the upload to have failed and do no further attempts to resume the upload.

NOTE1: Normally this failure case should not occur given that the certificate had been verified as part of the initial upload procedure defined in section 4.2.2.1, but the certificate verification is required to close any opportunity for misuse.

NOTE 2: Given that for security reasons this client handling does not ensure a graceful fallback to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

b) sending a HTTP GET request using the HTTP Content Server URI derived from the configuration parameter FT HTTP CS URI defined in section A.1.4 by appending to the query component of the URI the parameters defined below using the application/x-www-form-urlencoded format as defined in [HTML-4.0]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986]. The client shall add

- a "tid" parameter. The "tid" parameter value shall contain the value of the TID related assigned by the client in the initial upload request (see section 4.2.2.1)
- a "get_upload_info" parameter with no value, and
- if the client supports user authentication via HTTP Embedded EAP-AKA as defined in Annex C of [GSMA PRD-RCC.14], then the client shall add to the query component of the URL the EAP_ID URI query parameter defined in section C.2 of [GSMA PRD-RCC.14].

A non-normative example is given in the following:

- If the value of the configuration parameter FT HTTP CS URI is set to the value `https://upload.operator.com/upload?parm1=foo&parm2=bar`
- and the value of the TID assigned by the client in the initial upload request is "0815"
- and the client supports HTTP embedded EAP-AKA
- and the client's IMSI is "262011234567890

then the client shall use the following URI to construct the HTTP GET request:
`https://upload.operator.com/upload?parm1=foo&parm2=bar&tid=0815&get_upload_info&EAP_ID=0262011234567890%40nai.epc.mnc001.mcc262.3gppnetwork.org`

2. The HTTP Content Server sends back the upload information in the following XML structure describing the file content without optional thumbnail including the stored byte range within a file-range tag and the direct upload URI that shall use the “https” protocol scheme.

```
<?xml version="1.0" encoding="UTF-8"?>
<file-resume-info xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttpresume">
        <file-range start="[start-offset in bytes]" end="[end-offset in bytes]" / >
        <data url="[HTTP upload URL for the file]"/>
</file-resume-info>
```

Table 153: File Transfer upload information content

Complying with following schema:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:fthttpresume"
            xmlns:xs="http://www.w3.org/2001/XMLSchema"
            xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttpresume"
            elementFormDefault="qualified"
            attributeFormDefault="unqualified">
        <xs:element name="file-resume-info">
            <xs:complexType>
                <xs:sequence>
                        <xs:element name="file-range">
                                <xs:complexType>
                                        <xs:attribute name="start" type="xs:integer"
                                        use="required" />
                                        <xs:attribute name="end" type="xs:integer"
                                        use="required" />
                                        <xs:anyAttribute namespace="##other"
                                        processContents="lax"/>
                                </xs:complexType>
                        </xs:element>
                        <xs:element name="data">
                                <xs:complexType>
                                        <xs:attribute name="url" type="xs:anyURI"
                                        use="required"/>
                                        <xs:anyAttribute namespace="##other"
                                        processContents="lax"/>
                                </xs:complexType>
                        </xs:element>
                        <xs:any namespace="##other" processContents="lax"
                        minOccurs="0"
                        maxOccurs="unbounded"/>
                </xs:sequence>
            </xs:complexType>
        </xs:element>
</xs:schema>
```

Table 154: File Transfer upload information schema

If the client receives a HTTP 200 OK response, including an XML description of the file, the following procedure applies depending on the content of the XML description:

- If it includes file-resume-info for the uploaded file content with file range which matches the original file size, the file has been uploaded successfully.
- If it includes file-resume-info of the uploaded file content but with file range below the file size, the remaining file content needs to be uploaded using step 3.

- If it does not include the file-resume-info of the file content, the full upload needs to be started from beginning using the HTTP POST request as described section 4.2.2.1.

NOTE: The file-range refers to the part of the file that has been uploaded prior to the resume upload.

If the client receives a HTTP 404 NOT FOUND or 410 GONE response, then the resume upload cannot be performed (e.g. because the partial files are no longer available). The client shall start the upload procedure beginning using the HTTP POST request as described section 4.2.2.1.

If the client receives a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication with a bootstrapped required indication in accordance with [3GPP TS 24.109], then the client shall perform the procedures as defined for authentication with a bootstrapped security association.

If the client receives a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication without a bootstrapped required indication, or instruction the client to use HTTP Basic authentication, then the client shall perform the procedures as defined for HTTP digest or HTTP basic authentication using the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD from the device configuration as defined in in section A.1.4.

If the client receives a HTTP 302 FOUND response, the client shall follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the HTTP Content Server commencing in step 2 of this procedure.

If the client receives a HTTP 403 FORBIDDEN response, then the client is not authorised to upload the file.

If the “**Get upload info**" request fails with any other error response, then the client shall retry the “**Get upload info**" request.

An HTTP response that does not contain an XML description of the file, an XML structure with a direct upload URI that does not use the “https” protocol scheme or an XML structure that does not include a range field shall indicate to the client that a resume of the upload of the file is not possible. Then the resume upload cannot be performed (e.g. because the partial files are no longer available). The client shall start the upload procedure beginning using the HTTP POST request as described section 4.2.2.1.

3. **Resume upload**: In case the client wants to resume the upload of the file content it generates an HTTP PUT request to the direct upload URL that was included in the

XML description provided by the HTTP Content Server in step 2. For this, the client shall establish a TLS connection to the domain of this direct upload URL and verify whether a valid certificate (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the expected domain for the API, etc., see section 4.2.7) is provided. If no certificate is provided or the certificate is invalid, the resume upload cannot be performed, and the client shall start the upload procedure beginning using the HTTP POST request as described section 4.2.2.1.

If the client supports user authentication via HTTP Embedded EAP-AKA as defined in Annex C of [GSMA PRD-RCC.14], then the client shall add to the query component of the upload URI the EAP_ID URI query parameter defined in section C.2 of [GSMA PRD-RCC.14]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986].

In this request, it shall provide the remaining bytes started from the already uploaded byte position that was included in the received XML description. To indicate the byte range that is included in the HTTP PUT request a HTTP *Content-Range* header as defined in [RFC7233] is added to the request:

```
PUT <file_upload_uri> HTTP/1.1
Content-Type: [mime type depending on the file; e.g. image/jpeg]
Content-Length: <remaining_upload_size>
Content-Range: bytes <first-byte-pos> - <last-byte-pos> / <file_size>
Authorization: Digest ...

<file content>
```

Table 155: File Transfer upload information content

The client shall ensure that the file content related to the TID has not been changed between the initial HTTP POST request and the resume upload operation. When the server receives the partial file, it shall append the data according to the Content-Range header. If the upload is successful, a HTTP 200 OK response without body is returned.

If the client receives a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication with a bootstrapped required indication, then the client shall perform the procedures as defined for authentication with a bootstrapped security association.

If the client receives a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication without a bootstrapped required indication, or instruction the client to use HTTP Basic authentication, then the client shall perform the procedures as defined for HTTP digest or HTTP basic authentication using the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD from the device configuration as defined in in section A.1.4

If the client receives a HTTP 302 FOUND response, the client shall change the type of the request to HTTP GET and shall follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the HTTP Content Server commencing in step 3 of this procedure.

If the client receives a HTTP 403 FORBIDDEN response, then the client is not authorised to upload the file.

If the "**Resume upload**" request fails with any other response, then the client shall retry by restarting the file upload resume procedure defined in this section.

The **"Resume upload"** request can fail due to loss of network coverage. In that case, steps 1 and 2 shall be repeated with the same TID. In that case, the file-range tag returned from the HTTP Content Server indicates the sum of all the data uploaded in the uploaded resumes that have taken place so far.

4. **Get download info**: To get the HTTP Content Server response body for the complete uploaded file complying with the schema defined in section 4.2.4, the client shall send a HTTP GET request using the HTTP Content Server URI derived from the configuration parameter FT HTTP CS URI defined in section A.1.4 by appending to the query component of the URI the parameters defined below using the application/x-www-form-urlencoded format as defined in [HTML-4.0]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986]. The client shall add:

   - a "tid" parameter. The "tid" parameter value shall contain the value of the TID assigned by the client in the initial upload request (see section 4.2.2.1).
   - a "get_download_info" parameter with no value
   - if the client supports user authentication via HTTP Embedded EAP-AKA as defined in Annex C of [GSMA PRD-RCC.14], then the client shall add to the query component of the URL the EAP_ID URI query parameter defined in section C.2 of [GSMA PRD-RCC.14].

   A non-normative example is shown below:

```
https://upload.operator.com/upload?
      parm1=foo&parm2=bar&tid=0815&get_download_info&
      EAP_ID=0262011234567890%40nai.epc.mnc001.mcc262.3gppnetwork.org
```

   Given that the protocol scheme of the URL provided as value of the FT HTTP CS URI configuration parameter defined in section A.1.4 is "https", the client shall establish a TLS connection to the HTTP Content Server and verify whether a valid certificate is received from the HTTP Content Server (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the expected domain for the API, etc., see section 4.2.7). If no certificate is provided or the certificate is invalid, then the "Get download info" request failed. The client shall retry by restarting the file upload resume procedure defined in this section.

The server sends back a successful HTTP response including the HTTP Content Server response body complying with the schema defined in section 4.2.4 if the file has been uploaded successfully. The HTTP Content Server response body contains the file info for the fully uploaded file, and if provided earlier, the file info for the thumbnail.

If the server sends back a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication with a bootstrapped required indication, then the client shall perform the procedures as defined for authentication with a bootstrapped security association.

If the server sends back a HTTP 401 AUTHENTICATION REQUIRED with an WWW Authenticate header instructing the client to use HTTP digest authentication without a bootstrapped required indication, or instruction the client to use HTTP Basic authentication, then the client shall perform the procedures as defined for HTTP digest or HTTP basic authentication using the values of the configuration parameters FT HTTP CS USER and FT HTTP CS PWD from the device configuration as defined in in section A.1.4.

If the server sends back a HTTP 302 FOUND response, the client shall follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the HTTP Content Server commencing in step 4 of this procedure.

If the server sends back a HTTP 403 FORBIDDEN response, then the client is not authorised to apply the File Transfer procedures and shall disable File Transfer in accordance with the definition in section 3.2.5.2.

If the server sends back any other HTTP error response, then the "**Get download info**" request failed. The client shall retry by restarting the file upload resume procedure defined in this section.

An example flow of the file upload resume procedure is shown in Figure 62.

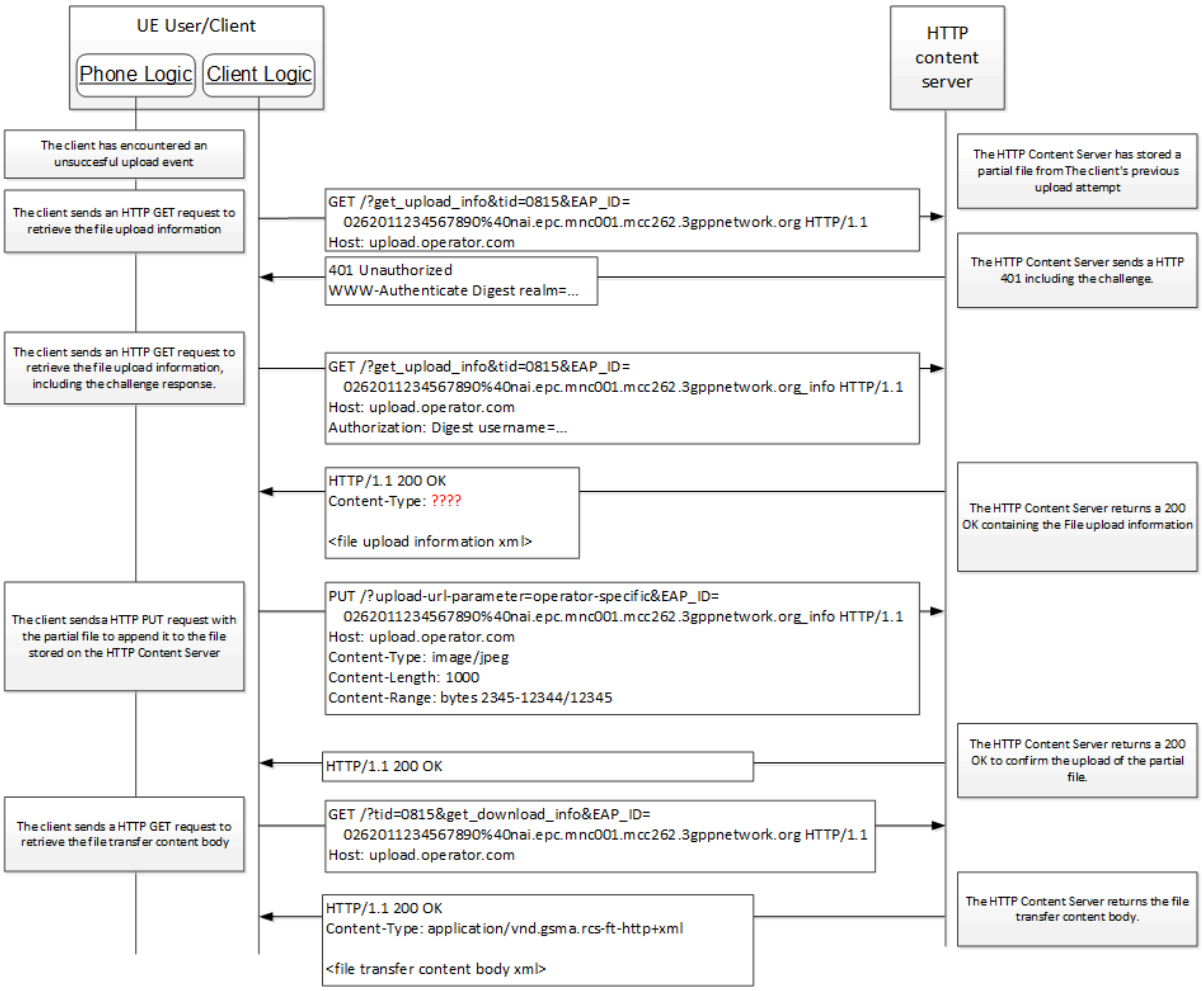


Figure 62: File Upload Resume

4.2.3 File Download Procedure

To download a file referred to through a HTTP URL being qualified by the corresponding service documentation for the download via the HTTP Content Server, the client shall

1. If the value of the configuration parameter FT HTTP DL URI defined in section A.1.4 is present, create the download URI by appending to the query component of the URL contained in the configuration parameter the request parameters defined below using the application/x-www-form-urlencoded format as defined in [HTML-4.0]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986]. The client shall add the following parameters as per corresponding service definition:

Parameter	Description	Use
url	The parameter shall contain the file download URL for which the client requests the download.	Mandatory
id	The parameter shall contain the value of the IMDN message-ID taken from the message that conveyed the download URL to the client.	Optional, shall be present if the IMDN message-ID

Parameter	Description	Use
		is available to the client
op	The parameter shall contain the address of the other party of the message or event conveying the file URL.	Optional, shall be present if the other party address is available to the client
ci	The parameter shall contain the Conversation-ID of the Group Chat in which the message has been conveyed.	Optional, shall be present if the Conversation-ID is available to the client

Table 156: HTTP Content Server download parameters

Example: If

- the value of the configuration parameter FT HTTP DL URI is set to *https://dl.operator.com/path?parm=foo*
- and if the download URL referring to the file that was received is: *https://ftcontentserver.rcs.mnc001.mcc262.pub.3gppnetwork.org/dl?uid=1234*
- and the message ID of the chat message is *123456789*
- and if the authenticated originator address of the 1-to-1 Chat message is *sip:+491711234567@ims.mnc001.mcc262.3gppnetwork.org;user=phone*

then the client's download URI results in
https://dl.operator.com/path?parm=foo&url=https%3A%2F%2Fftcontentserver.rcs.mnc001.mcc262.pub.3gppnetwork.org%2Fdl%3Fuid%3D1234&id=123456789&op=sip%3A%2B491711234567%40ims.mnc001.mcc262.3gppnetwork.org%3Buser%3Dphone

2. otherwise (i.e. if no value of the configuration parameter FT HTTP DL URI defined in section A.1.4 is present), use the URL that was received without changes as the download URI in the following steps
3. Create an HTTP GET request using the download URI
4. if the client supports authentication with a GBA bootstrapped security association as defined in [3GPP TS 33.220], indicate this by the addition of a GBA product token in the User-Agent header of the HTTP GET request as defined in [3GPP TS 24.109]
5. if the client supports user authentication via HTTP Embedded EAP-AKA as defined in Annex C of [GSMA PRD-RCC.14], then the client shall add to the query component of the URL the EAP_ID URI query parameter defined in section C.2 of [GSMA PRD-RCC.14]. If no query component exists, the client shall add one first, in accordance with the definitions in [RFC3986].
6. Given that the protocol scheme of the derived file URI is "https", establish a TLS connection to the HTTP Content Server using the value of the derived file URI and verify whether a valid certificate is received (it has not expired, there is chain of trust to a known Certification Authority, it corresponds to the domain part of the derived file

URI, etc., see section 4.2.7). If no certificate is provided or the certificate is invalid, the client shall consider the HTTP Content Server to be unavailable. The client shall then stop the file download procedure and inform the user that the file is no longer available.

NOTE: Given that for security reasons this client handling does not ensure a graceful fallback to a connection without TLS if the certificate is invalid, Service Providers should ensure that certificates are replaced before expiry.

7. send the HTTP GET request using the derived file URI.
8. If the client receives in result of the processing of the request from the HTTP Content Server

   - a HTTP 401 AUTHENTICATION REQUIRED response with an WWW Authenticate header instructing the client to use HTTP digest Authentication with a bootstrapped security association as defined in [3GPP TS 24.109].

     If the client has no bootstrapped security association in place it shall invoke the bootstrapping procedure defined in [3GPP TS 24.109].

     The client shall generate another HTTP GET request to the download URI and use the stored key material and the B-TID to generate keys specific to the HTTP Content Server as defined in [3GPP TS 33.220]. The client shall add an Authorization header generated from the key material and the B-TID to the HTTP GET request and send the HTTP GET request for authentication.

     If a HTTP 401 AUTHENTICATION REQUIRED response is received for this HTTP GET request, then the client shall invoke the bootstrapping procedure defined in [3GPP TS 24.109] and repeat the procedure based on the new key material.

   - a HTTP 401 AUTHENTICATION REQUIRED response with an WWW Authenticate header without a "bootstrapping required" indication, then the client shall generate another HTTP GET request to the download URI and create an Authorization header from the values of FT HTTP CS USER and FT HTTP CS PWD from the device configuration as defined in in section A.1.4. The client shall then send the HTTP GET request for authentication.

     If a HTTP 401 AUTHENTICATION REQUIRED is received in the result of the HTTP GET request then the client shall trigger a configuration request to the Auto-Configuration Server via the procedures defined in section 2.3.2 and retry the procedure if there were changes to the FT HTTP CS USER or FT HTTP CS PWD parameters. Otherwise, the client shall inform the user that the file is not available.

   - a HTTP 302 FOUND response, the client shall follow the procedures for OpenID Connect based authentication as defined in section 2.12.2. The procedure results in a reconnection back to the HTTP Content Server commencing in the processing of the HTTP Content Server response.

- a HTTP 503 SERVICE UNAVAILABLE with a Retry-After header, then the client shall retry the procedure, the recommended interval to retry will be specified in the “Retry-After” header.
- a HTTP 404 NOT FOUND or HTTP 410 GONE, then the client shall stop the file download procedure and inform the user that the file is no longer available.
- any other error, then the client shall retry the procedure up to a maximum of 3 times. In case the file was partially downloaded already, a partial HTTP GET request as defined in [RFC7233] may be used to obtain the remaining part of the file.
- a HTTP 200 OK response with a file object in the body then client shall handle the file file in accordance with the corresponding service documentation.

4.2.4 Schema Definition

The HTTP Content Server response body shall comply with the following XML Schema. The schema is extensible via the standard schema extension mechanism. Clients receiving unsupported elements or attributes shall ignore them.

The schema applies in addition for the File Transfer message body transferred between clients for the File Transfer service as defined in section 3.2.5 and the Call Composer service using Call Composer session as defined in [GSMA PRD-RCC.20].

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:fthttp"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="urn:gsma:params:xml:ns:rcs:rcs:fthttp"
        elementFormDefault="qualified"
        attributeFormDefault="unqualified">
    <xs:element name="file">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="file-info" minOccurs="1" maxOccurs="2">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="file-size">
                                <xs:simpleType>
                                    <xs:restriction base="xs:integer"/>
                                </xs:simpleType>
                            </xs:element>
                            <xs:element name="file-name" minOccurs="0"
                            maxOccurs="1">
                                <xs:simpleType>
                                    <xs:restriction base="xs:string"/>
                                </xs:simpleType>
                            </xs:element>
                            <xs:element name="content-type">
                                <xs:simpleType>
                                    <xs:restriction base="xs:string"/>
                                </xs:simpleType>
                            </xs:element>
                            <xs:element name="data">
                                <xs:complexType>
                                    <xs:attribute name="url"
                                    type="xs:anyURI" use="required"/>
```

```
                                                    <xs:attribute name="until"
                                                    type="xs:dateTime" use="required"/>
                                                    <xs:anyAttribute
                                                    namespace="##other"
                                                    processContents="lax"/>
                                                </xs:complexType>
                                            </xs:element>
                                            <xs:any namespace="##other" processContents="lax"
                                            minOccurs="0" maxOccurs="unbounded"/>
                                        </xs:sequence>
                                        <xs:attribute name="type" use="required">
                                            <xs:simpleType>
                                                <xs:restriction base="xs:string">
                                                    <xs:enumeration value="file"/>
                                                    <xs:enumeration value="thumbnail"/>
                                                </xs:restriction>
                                            </xs:simpleType>
                                        </xs:attribute>
                                        <xs:attribute name="file-disposition" use="optional">
                                            <xs:simpleType>
                                                <xs:restriction base="xs:string">
                                                    <xs:enumeration value="render"/>
                                                    <xs:enumeration
                                                    value="attachment"/>
                                                </xs:restriction>
                                            </xs:simpleType>
                                        </xs:attribute>
                                        <xs:anyAttribute namespace="##other" processContents="lax"/>
                                    </xs:complexType>
                                </xs:element>
                                <xs:any namespace="##other" processContents="lax" minOccurs="0"
                                maxOccurs="unbounded"/>
                            </xs:sequence>
                        </xs:complexType>
                    </xs:element>
</xs:schema>
```

Table 157: Schema Definition

The schema defined in Table 157 is extended to enable the HTTP Content Server to assign a user friendly URL to a file. The user friendly URL shall be conveyed by the HTTP Content Server to the client via the "branded-url" element defined in the schema in Table 158.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema targetNamespace="urn:gsma:params:xml:ns:rcs:rcs:up:fthttpext"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="urn:gsma:params:xml:ns:rcs:rcs:up:fthttpext"
        elementFormDefault="qualified"
        attributeFormDefault="unqualified">
        <xs:element name="branded-url" type="xs:anyURI"/>
</xs:schema>
```

Table 158: Branded URL Schema Extension

The "branded-url" element shall be added, if available on the HTTP Content Server, as an extension to the "data" element of the "file info" element of the HTTP Message Body schema defined in Table 157.

Clients receiving the "branded-url" element in an HTTP Content Server response body shall handle it in accordance with the relevant service documentation.

The nature and structure of the URL value in the "branded-url" element is left to the discretion of the service provider of the HTTP Content Server.

The content type assigned to the schema of the HTTP Content Server response body and the File Transfer body shall be *application/vnd.gsma.rcs-ft-http+xml*.

4.2.5 HTTP Content Server URL

For the composition of the URL to locate files on the HTTP Content Server, the requirements of this section apply.

To enable the traceability of the HTTP transactions between operators over the NNI, the URL to locate files on the HTTP Content Server shall include the FQDN as defined in Table 159.

<host>.ftcontentserver.rcs.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org

Table 159: HTTP Content Server FQDN

The presence of the <host> part of the HTTP Content Server FQDN is optional. Service Providers can use the <host> part of the HTTP Content Server FQDN for the addressing of multiple HTTP Servers in their network. For backward compatibility reasons, the <host> part of the HTTP Content Server FQDN shall be omitted for URLs used for the File Transfer Service defined in section 3.2.5.

The values of <MNC> and <MCC> are the MNC and MCC of the Service Provider represented in decimal form, with any 2 digit-MNC padded out to 3 digits by inserting a zero ("0") on the beginning.

Apart from the provider specific path and query elements of the HTTP Content Server URL to locate files on the HTTP Content Server, the URL may contain the following well-known URL parameters to provide meta-information describing the file. The URL parameters are appended to the query component of the HTTP Content Server URL using the application/x-www-form-urlencoded format as defined in [HTML-4.0].

Table 160 defines the parameters to provide additional meta-data describing the file for File Transfer fallback.

Parameter	Type	Value
s	Integer	Identifies the size of the file in bytes The presence of the parameter is mandatory, if the URL locating a file on a HTTP Content Server contains additional meta data describing the file.

Parameter	Type	Value
t	String	Value of the Multipurpose Internet Mail Extensions (MIME) Content-Type header of the file as defined in [RFC2045]. NOTE: reserved characters in the Content-Type header value have to be represented using percent encoding in accordance with [RFC3986]. The presence of the parameter is mandatory, if the URL locating a file on a HTTP Content Server contains additional meta data describing the file.
e	String	Combined date and time in UTC time zone in ISO8601 basic format, i.e. YYYYMMDDThhmmssZ. It indicates the date and time of expiry of the file, e.g. 20170419T135227Z The presence of the parameter is mandatory, if the URL locating a file on a HTTP Content Server contains additional meta data describing the file.
d	Integer	Playing length in seconds of the audio message. It indicates that the file located via the URL is a RCS Recorded Audio Message as defined in section 3.2.7 that can be played directly from the Chat application upon user action. The parameter is optional in the URL location a file on a HTTP Content Server. The parameter shall not be present in the HTTP Content Server response body for a URL locating a file.

Table 160: HTTP URL parameters for File Transfer fallback

Service providers should add the parameters in the HTTP Content Server response body defined in section 4.2.4 to the value of the "url" attribute of the "data" element of the "file-info" element with the "type" attribute set to "file". It is recommended that implementations ensure that the maximum length of the URL locating a file on a HTTP Content Server does not exceed the length of the user data of one SMS message.

4.2.6 HTTP State Management

The client shall support for the HTTP state management defined in [RFC6265] for the HTTP up- and download procedures. This includes all HTTP requests and responses between the client and HTTP Content Servers as part of the HTTP Content Server procedures for upload, download and the corresponding resume procedures. This allows a HTTP Content Server to return a Set-Cookie header in HTTP responses. The client shall apply the parsing and storage procedures of the Set-Cookie header as defined [RFC6265]. It shall send the cookie header in HTTP requests to HTTP Content Servers respecting the cookie attributes provided by the HTTP Content Server in the Set-Cookie header in accordance with [RFC6265].

With this, the HTTP Content Server is able to make use of all the functions of HTTP state management.

4.2.7 Security considerations

In order to guarantee the integrity and security of the transfer of files via the HTTP Content Server, the following three principles shall be taking into account:

1. The security of the solution relies on the security of the transport of file URL via RCS services. Therefore, encryption of the media associated to RCS services conveying file URIs (e.g. Chat) is recommended.
2. All HTTP transactions with the HTTP Content Server shall be secured using TLS.

    a) Clients setting up the TLS connection to the HTTP Content Server shall verify at least that

        i. An X.509 certificate is received,
        ii. The certificate is valid at the time when the connection is set up (e.g., it has not expired)
        iii. The certificate has chain of trust to a known Certification Authority (e.g., self-signed certificates shall not be accepted),
        iv. The Issuer's (i.e., the CA) signature in the certificate matches the certificate received,
        v. The certificate's Subject or Alternate Subject correspond to the domain of the HTTP Content Server to which the client is setting up the connection,
        vi. The key usage and/or extend key usage indicated in the certificate are valid for the offered TLS cypher suite, and
        vii. the certificate has not been revoked.

    b) If this verification fails, the client shall consider the TLS establishment and the procedure in which it was used to have failed (as indicated in for example section 4.2.2.1)

3. To secure interoperability between Service Providers and to reduce complexity on the RCS device/client, the HTTP Content Server shall make use of public root certificates issued by a recognised CA. That is, the root certificates are similar to those used by standard webservers which are widely recognised by browsers and web-runtime implementations in both PCs and devices.

4.3 User Profile servers

User Profile servers allow users to store and retrieve User Profile (name and image) through RCS clients.

4.3.1 Architecture

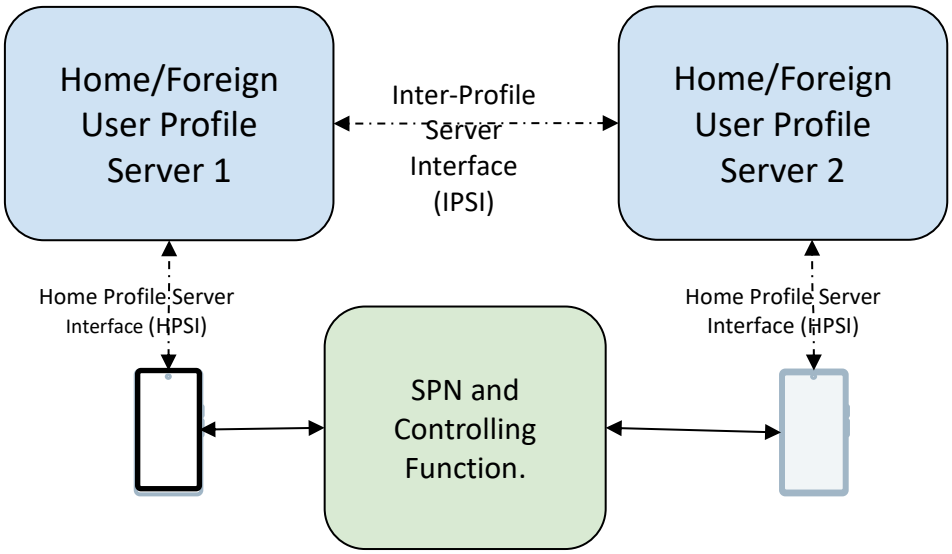


Figure 63: Architecture of User Profiles

The User Profile sharing architecture includes the RCS clients, the User Profile Servers that store and share the User Profiles. The RCS clients shall use the RCS SPNs for sharing User Profile identifiers and credentials.

In addition to storing the User Profile, the User Profile Server provides the RCS client with the identifiers and credentials associated with the stored User Profile. These identifiers and credentials are sent to recipient RCS clients. Those recipient RCS clients request, using the Home Profile Server Interface (HPSI), the shared User Profile using the identifiers and credentials.

To support multiple User Profile Servers from multiple providers, the User Profile Servers shall implement the Inter-Profile Server Interface (IPSI). This enables all Home User Profile Servers to forward an RCS client's User Profile request to the designated Foreign User Profile Server.

The Profile Server shall be responsible for:

- Storing the User Profiles.
- Processing the User Profile requests.
- Generating the necessary identifiers and credentials identifying the shared User Profiles.
- Forwarding the RCS client User Profile requests to the proper Foreign User Profile Server.
- Policy enforcement and security.

The Controlling Function shall be responsible for:

- Profile sharing access control list enforcement for Group Chats.

The RCS client shall be responsible for:

- Storing the User Profiles on their Home User Profile Server
- Fetching the User Profile using sharing identifiers and credentials.
- Profile sharing enforcement for 1-to-1 Chats and Group Chats.

4.3.2 Inter-Profile Server Interface wireframe

4.3.2.1 GetSharedUserProfileRequest

When a User Profile is associated with another `profile_provider_id`, the Home User Profile Server shall forward the shared User Profile request to the Foreign User Profile Server, the necessary credentials and identifiers shall be sent using the GetSharedUserProfileRequest protocol buffer over gRPC [gRPC]. This request includes a list of `profile_identifier` which contain the `profile_id` being shared along with the authentication `token`.

```
message ProfileIdentifier {
   String profile_id = 1;
   bytes token = 2;
}

message GetSharedUserProfileRequest {
   repeated ProfileIdentifier profile_identifier = 1;
}
```

Table 161: GetSharedUserProfileRequest protocol buffer definition

```
{
  "profile_id": "user_8829",
  "token": "EjRWeJq83vA="
}
```

Table 162: GetSharedUserProfileRequest protocol buffer example

4.3.2.2 GetSharedUserProfileResponse

When the Foreign User Profile Server needs to respond to a `GetSharedUserProfileRequest`, it shall send a `GetSharedUserProfileResponse` containing a list of `ProfileDetails` back to the requester.

Each `profile_details` shall include a `ResponseStatus` that indicates if the User Profile was found or not and, if found, the `user_profile` shall contain the binary of a vCard [RFC6350]. The vCard's PHOTO Identification Property [RFC6350] may be assigned with URI of the image of the User Profile and the FN Identification Property [RFC6350] may be assigned with the name of the User Profile.

```
message ProfileDetails {
   enum ResponseStatus {
      UNKNOWN_STATUS = 0;
      OK = 1;
      NOT_FOUND = 2;
   }
```

```
   String profile_id = 1;
   ResponseStatus status = 2;
   bytes user_profile = 3;
}

message GetSharedUserProfileResponse{
   repeated ProfileDetails profile_details = 1;
}
```

Table 163: GetSharedUserProfileResponse protocol buffer definition

```
{
  "profileDetails": [
    {
      "profileId": "usr_948201",
      "status": "OK",
      "userProfile": "CgZBbGljZRIbYWxpY2VAZXhhbXBsZS5jb20="
    },
    {
      "profileId": "usr_773042",
      "status": "NOT_FOUND",
      "userProfile": ""
    }
  ]
}
```

Table 164: GetSharedUserProfileResponse protocol buffer example

4.3.3 Creation of a User Profile

To create a sharable User Profile, the RCS client shall:

1. Store the User Profile (name and image) on the Home User Profile Server using the HPSI.

Upon storing the User Profile configuration information, the Home User Profile Server shall:

2. Generate a unique `profile_id`, `token` and send these along with its `profile_provider_id` to the RCS client using the HPSI.

The Profile Server shall update the `token` when the User Profile is updated. Additionally, if the User Profile is not updated, the `token` shall not be updated.

Upon receiving the `profile_id`, `token` and the `profile_provider_id` the RCS client shall store them on the local storage.

The HPSI used for creation of the User Profile sharing is up to the client implementation.

4.3.4 Recipient procedures

Upon receiving the User Profile information as per section 3.2.11.4, the RCS client shall extract the contents of the rcs.User-Profile-Config-Data header and contact the Home User Profile Server to fetch the User Profile via the HPSI.

Upon receiving the User Profile, the recipient's RCS client shall store the User Profile in its local storage.

If the server does not return the User Profile, the RCS client shall discard the rcs.User-Profile-Config-Data silently.

4.3.5 User Profile Server Procedures

4.3.5.1 User Profile found in the Home User Profile Server

Upon receiving the User Profile fetch request where the `profile_provider_id` maps to the Home User Profile Server, the server shall:

1. Validate the `token`.
2. Locate the `profile_id`.
3. And if the User Profile is found, respond with the User Profile to the RCS client over the HPSI.

The steps of this procedure are illustrated in Figure 64.

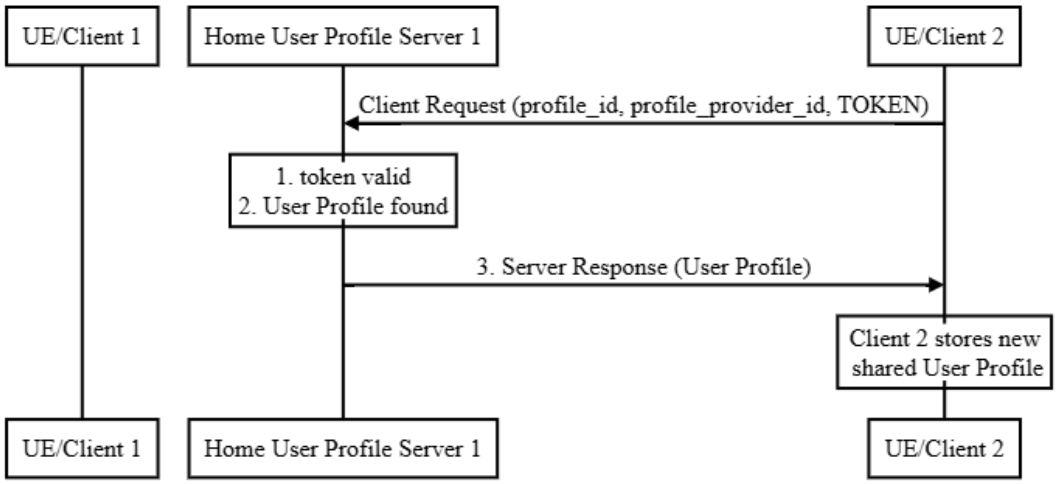


Figure 64: User profile found in Home User Profile Server

4.3.5.2 User Profile token invalid in the Home User Profile Server

Upon receiving the User Profile fetch request where the `profile_provider_id` maps to the Home User Profile Server, if the token validation fails, the server shall:

1. Respond with no User Profile to the RCS client over the HPSI.

4.3.5.3 User Profile not found in the Home User Profile Server

4.3.5.3.1 User Profile fetch request referring to Home User Profile Server

Upon receiving the User Profile fetch request where the `profile_provider_id` maps to the Home User Profile Server, if the server fails to locate the User Profile, the server shall

1. Respond with no User Profile to the RCS client over the HPSI.

4.3.5.3.2 User Profile found in a Foreign User Profile Server

Upon receiving the request where the profile_provider_id maps to a Foreign User Profile Server, the Home User Profile Server shall:

1. Forward the request using the GetSharedUserProfileRequest protocol buffer as per section 4.3.2.1 over the IPSI interface using gRPC [gRPC] to Foreign User Profile Server using the `token` and the `profile_id`.

Upon receiving the GetSharedUserProfileRequest, the Foreign User Profile Server shall:

2. Validate the `token`.
3. Locate the `profile_id`.
4. Package the `user_profile` containing a vCard [RFC6350] object as per section 4.3.2.2 with a `status` of `OK`.
5. Include the `user_profile` into the GetSharedUserProfileResponse protocol buffer.
6. Respond to the GetSharedUserProfileRequest gRPC [gRPC] with the GetSharedUserProfileResponse over the IPSI to the Home User Profile Server.

Upon receiving the GetSharedUserProfileResponse via the IPSI from the Foreign User Profile Server, the Home User Profile Server responds with the User Profile to the recipient's RCS client via the HPSI.

The steps of this procedure are illustrated in Figure 65.

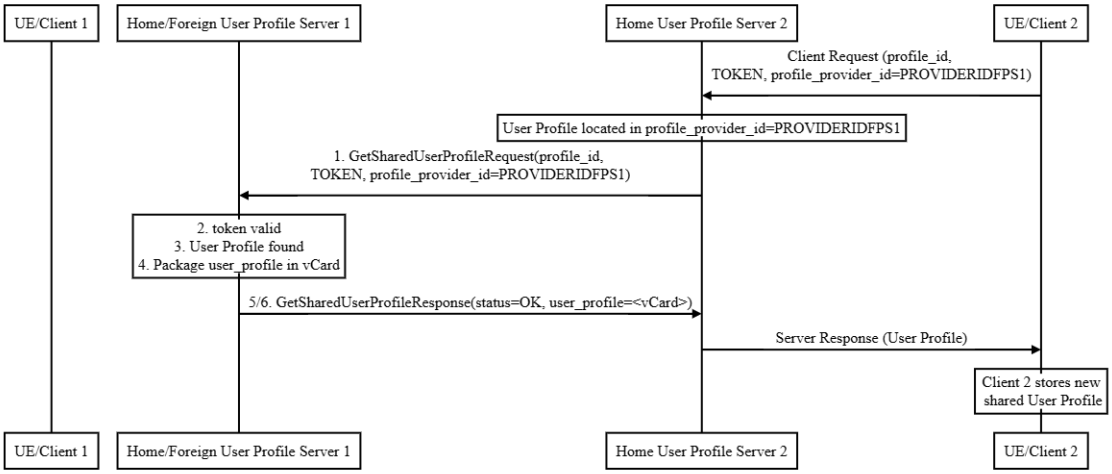


Figure 65: User profile found in Foreign User Profile Server

4.3.5.3.3 User Profile token invalid in a Foreign User Profile Server

Upon receiving the request where the `profile_provider_id` maps to a Foreign User Profile Server, the Home User Profile Server shall:

1. Forward the request using the GetSharedUserProfileRequest protocol buffer as per section 4.3.2.1 over the IPSI interface using gRPC [gRPC] to Foreign User Profile Server using the `token` and the `profile_id`.

Upon receiving the GetSharedUserProfileRequest, if the Foreign User Profile Server token validation fails, the server shall:

2. Package the `user_profile` containing no vCard [RFC6350] object as per section 4.3.2.2 with a `status` of `NOT_FOUND`.
3. Include the `user_profile` into the GetSharedUserProfileResponse protocol buffer.
4. Send the resulting protocol buffer over the IPSI using gRPC [gRPC] to the Home User Profile Server.

Upon receiving the GetSharedUserProfileResponse via the IPSI from the Foreign User Profile Server, the Home User Profile Server responds with no User Profile to the recipient's RCS client via the HPSI.

4.3.5.3.4 User Profile not found in a Foreign User Profile Server

Upon receiving the request where the profile_provider_id maps to a Foreign User Profile Server, the Home User Profile Server shall:

1. Forward the request using the GetSharedUserProfileRequest protocol buffer as per section 4.3.2.1 over the IPSI interface using gRPC [gRPC] to Foreign User Profile Server using the `token` and the `profile_id`.

Upon receiving the GetSharedUserProfileRequest, if the Foreign User Profile Server fails to locate the User Profile, the server shall:

2. Package the `user_profile` containing no vCard [RFC6350] object as per section 4.3.2.2 with a `status` of `NOT_FOUND`.
3. Include the `user_profile` into the GetSharedUserProfileResponse protocol buffer.
4. Send the resulting protocol buffer over the IPSI using gRPC [gRPC] to Home User Profile Server.

Upon receiving the GetSharedUserProfileResponse via the IPSI from the Foreign User Profile Server, the Home User Profile Server responds with no User Profile to the recipient's RCS client via the HPSI.

Annex A Managed objects and configuration parameters

This Annex provides the details on the RCS data model including an overview of all configuration parameters. These parameters will be set using the mechanisms described in section 2.3. Section A.1 provides a functional overview of all configuration parameters applicable to RCS (except those that influence only the UX). Section A.2 provides the realisation of RCS-specific parameters in an RCS Management Object (MO). Section A.3 covers the use for RCS of other MOs that are not RCS-specific and that would cover the remaining parameters. Section A.4, defines how those different MOs would be combined in a configuration document for RCS.

NOTE: To allow matching the functional definition to the parameter in the MO, section A.1 uses similar naming for the parameters as the one used in the MO, albeit with different case and spacing conventions to support readability, e.g. when the functional parameter is referred to in the body of this document.

The functional definitions also indicate whether a parameter is mandatory for a network to provide when configuring a client, providing a default behaviour for the case where it is not. If optional, for an RCS-specific parameter this would be reflected in the “Occurrence” field of the parameter definition or the branch where it would be located in the MO. That indicates how many instances of the parameter/branch can occur in the MO. Thus, for an optional parameter, that would have to allow for that to be zero instances.

The definitions also indicate whether a client receiving the parameter should be able to handle it. This is reflected in the “Status” field in the MO definition. This field doesn’t have an equivalent in the functional parameter definition in section A.1 because other than branches in which (future) extensions to the MO would be defined, most RCS-specific parameters are required to be supported by RCS clients.

The aim of this section is to provide a complete configuration data model for reference by both Service Providers and OEMs.

A.1 Management objects parameters overview

This section provides an overview of the configuration parameters used for RCS. These parameters can either come from an existing management object (like for instance the OMA defined objects for Presence, Messaging and so on) or may be RCS specific. In the latter case they will be formally defined in section A.2.

NOTE: This may not be the only document where parameters controlling an RCS device are defined (see e.g. [GSMA PRD-RCC.53] and [GSMA PRD-RCC.15]).

A.1.1 Configuration parameters for the management of RCS services

This RCS specification includes the following configuration parameters for the management of RCS services:

Configuration parameter	Description	RCS usage
RCS DISABLED STATE	Controls the way in which an RCS client is disabled on the device. It can have following values resulting in the behaviour described in section 2.3.2.5: • 0, the client is temporarily disabled • -1, the client is permanently disabled • -2, the client is permanently disabled, but user triggered events result in an attempt to re-enable the client • -3, the client is placed in dormant state If included a configuration document should not include other RCS related configuration parameters. NOTE: if not included, the RCS client should be enabled and the document should include a valid configuration for the other RCS configuration parameters.	Optional Parameter
SUPPORTED RCS PROFILE VERSIONS	Indicates the RCS profile versions supported by the Auto-Configuration Server. The value can be used by the client to adapt its own behaviour and its configuration requests.	Optional Parameter

Table 165: RCS status configuration parameters

A.1.2 Presence related configuration

A.1.2.1 OMA Presence Provisioning parameters

OMA Presence Client provisioning parameters are defined in [PRESENCE2MO]. Table 166 lists the OMA Presence parameters applicable to RCS.

Configuration parameter	Description	RCS usage
CLIENT-OBJ-DATA-LIMIT	maximum size of the MIME object in SIP PUBLISH request	Optional parameter It is mandatory and becomes relevant only if DEFAULT DISCOVERY MECHANISM is set to PRESENCE
CONTENT-SERVER-URI	HTTP URI of the Content Server to be used for content indirection	Not Used

Configuration parameter	Description	RCS usage
SOURCE-THROTTLE-PUBLISH	minimum time interval (in seconds) between two consecutive publications	Optional parameter It is mandatory and becomes relevant only if DEFAULT DISCOVERY MECHANISM is set to PRESENCE
MAX-NUMBER-OF-SUBSCRIPTIONS-IN-PRESENCE-LIST	Limits the number of back-end subscriptions allowed for a presence list.	Not used
SERVICE-URI-TEMPLATE	syntax of the service URI	Not Used
RLS-URI	SIP URI of the RLS to be used by the Watcher when subscribing to a Request-contained Presence List Default if not provided: *presence-rls@ims.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org* whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning	Optional parameter

Table 166: RCS usage of OMA presence configuration parameters

A.1.3 Messaging related configuration

As there are no OMA defined parameters for CPM Messaging, this RCS specification includes only RCS specific parameters. These are described in the following table:

Configuration parameter	Description	RCS usage
CHAT AUTH	This parameter controls the initiation of 1-to-1 Chat sessions. When set to **0**, initiation of 1-to-1 Chat sessions is disabled. When set to **1**, initiation of 1-to-1 Chat sessions is enabled. When this parameter is set to 1. GROUP CHAT AUTH parameter shall also be set to 1.	Mandatory Parameter

Configuration parameter	Description	RCS usage
GROUP CHAT AUTH	This parameter controls the Group Chat service and receiving 1-to-1 Chat session invitations. If set to **0** the Group Chat service and receiving 1-to-1 chat invitations is disabled. When set to **1** the Group Chat service and receiving 1-to-1 chat invitations is enabled. If CHAT AUTH is set to 0 (disabled), GROUP CHAT AUTH can be enabled or disabled. If CHAT AUTH is set to 1 (enabled), GROUP CHAT AUTH shall be set to 1.	Mandatory parameter
STANDALONE MSG AUTH	This parameter Enables/Disables the Standalone Messaging Service. If set to **0** the service is disabled. When set to **1** it is enabled. When set to **2** it is enabled for receiving Standalone Messages but not for sending.	Mandatory Parameter
MAX_AD-HOC_GROUP_SIZE	Maximum number of Participants allowed for the initiation of a Group Chat. The number includes the initiator of the Group Chat.	Optional parameter It is mandatory and becomes relevant only if GROUP CHAT AUTH is set to 1 or is not provided and CHAT AUTH is set to 1
CONF-FCTY-URI	SIP URI used as Request URI when initiating a Group Chat. Default value if not provided: chat@conf-factory.<home-network-domain-name> Where <home-network-domain-name> is replaced with the Home Network Domain Name used by the client (see [GSMA PRD-RCC.15] and [GSMA PRD-NG.102])	
EXPLODER-URI	SIP URI used as Request URI when sending 1-to-Many Standalone Messages Default value if not provided: exploder@conf-factory.<home-network-domain-name> Where <home-network-domain-name> is replaced with the Home Network Domain Name used by the client (see [GSMA PRD-RCC.15] and [GSMA PRD-NG.102])	
IM SESSION AUTO ACCEPT GROUP CHAT	This parameter controls whether the client automatically accepts incoming Group Chat session invitations (**1**, default) or whether acceptance depends on a user action (**0**) for explicitly accepting the session.	Optional parameter

Configuration parameter	Description	RCS usage
IM SESSION TIMER	This parameter controls the time during which a 1-to-1 Chat session is allowed to be idle before it's closed. When set to **0**, there shall be no timeout. The recommended value is 3 (three) minutes.	Optional parameter (It is mandatory if CHAT AUTH is set to 1)
MAX SIZE IM	This parameter controls the maximum size in bytes of a text message that a user can enter in a 1-to-1 Chat or Group Chat session.	Optional parameter (It is mandatory if CHAT AUTH is set to 1.)
MAX SIZE STANDALONE	This parameter controls the maximum authorised size in bytes of the content payload of a text or multimedia message without transfer encoding.	Optional parameter (It is mandatory if STANDALONE MSG AUTH is set to 1.)
STANDALONE SWITCHOVER SIZE	The value of the configuration parameter indicates the maximum size in bytes for a Pager Mode CPM Standalone Message to switch over from the Pager Mode to the Large Message Mode. If the configuration parameter is absent, then the default value 1300 shall be used.	Optional parameter
CONTENT COMPRESS SIZE	This parameter controls the size in bytes of the SIP request/response content to decide whether compressing the content or not. If the configuration parameter is absent, SIP content compression is not supported.	Optional parameter
MINIMUM CPIM BODY SIZE FOR COMPRESSION	This parameter controls the minimum CPIM size that should be compressed. A value of zero indicates that all CPIM bodies should be compressed. If the configuration parameter is absent, the RCS client may use its own default threshold	Optional parameter
SIGNED ENCRYPTION IDENTITY PROOF	This parameter returns a signed tuple of (MSISDN, Public Participant Key, Home KDS) as defined in [GSMA PRD-RCC.16] based on the uploaded Public Participant Key and Home KDS in the request. The client can use this signature to enrol with the Home KDS.	Optional parameter
MIVC TECHNOLOGY	This parameter returns the configured technology for MIVCs. **0**: None **1**: VILTE **2**: ViLTE preferred, RCS Video Chat fallback (default) **3**: RCS Video Chat	Optional parameter

Configuration parameter	Description	RCS usage
RCS VIDEO CHAT TURN	This parameter specifies if the use of a TURN server is mandatory for all RCS Video Chats. If the value is set to 1, then at least one RCS VIDEO CHAT TURN URI must be specified **0**: Not mandatory (default) **1**: mandatory	Optional parameter
RCS VIDEO CHAT TURN URI	URI of the TURN servers, as specified in [RFC7065], used by RCS clients for RCS Video Chats	Optional parameter
RCS VIDEO CHAT TURN USERNAME	The username for authentication with the TURN server.	Optional parameter
RCS VIDEO CHAT TURN PASSWORD	The password for authentication with the TURN server.	Optional parameter
MIVC REPORTING URI	The SIP URI where MIVC event notifications should be sent to.	Optional parameter
MESSAGE STORE URL	The URL used to access the Message Store Server Default value if not provided: https://msg-store.rcs. mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org Whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning (as defined in [GSMA PRD-IR.67]).	Optional parameter
MESSAGE STORE NOTIFICATION URL	The URL used to access the message store notification server, can be a different URL from the message store URL. If not provided, the value of MESSAGE STORE URL shall be used.	
MESSAGE STORE USERNAME	The username for authentication with the Message Store Server via HTTP Basic or Digest authentication. If the client is requested by the Message Store server for HTTP Basic or Digest authentication or the client is configured via the MESSAGE STORE AUTH parameter for initiating HTTP Basic authentication and this parameter is absent then the client shall use the value of "Realm User Name" defined in Table 2 of [GSMA PRD-RCC.15] for authentication instead.	Optional parameter

Configuration parameter	Description	RCS usage
MESSAGE STORE PASSWORD	The password for authentication with the Message Store Server via HTTP Basic or Digest authentication. If the client is requested by the Message Store server for HTTP Basic or Digest authentication or the client is configured via the MESSAGE STORE AUTH parameter for initiating HTTP basic authentication and this parameter is absent, then the client shall use the values of "Realm User Password" defined in Table 2 of [GSMA PRD-RCC.15] for authentication instead.	Optional parameter.
MESSAGE STORE AUTH	This parameter controls the authentication mechanism used to access the Message Store Server. **0**: Message Store server is not enabled (default) **1**: Message Store server is enabled and the client shall initiate HTTP Basic authentication with user name and password from MESSAGE STORE USERNAME and MESSAGE STORE PASSWORD as described in section 4.1.5. **2**: Message Store Server is enabled and the client shall perform the authentication mechanism requested by the Message Store server.	Optional parameter
MESSAGE STORE EVENT REPORTING	This parameter is used to inform the Message Store Client whether to directly set flags in the Message Store or whether to indicate to the Messaging Server that it should set flags in the Message Store on behalf of the client. When set to **0** the client shall set flags in the Message Store as needed via a Message Store connection (default). When set to **1**, Indicates that the client shall make use of the Event Reporting framework as described in section 4.1.14 when no Message Store connection exists so that the Messaging Server may set the flags in the Message Store on behalf of the client. If not provided, the Message Store Client SHALL assume the same method as if value 0 had been specified.	Optional parameter
MESSAGE STORE ARCHIVE AUTH	This parameter Enables/Disables the Archive service. If set to **0** the Archive service is disabled. When set to **1** it is enabled and the client may archive messages. Default value is **0**.	Optional Parameter
SMS MESSAGE STORE	This parameter indicates to the client whether it shall store in the RCS Default folder any sent or received SMS. If this parameter is set to **0**, client shall not store any sent or received SMS/MMS. If this parameter is set to **1**, client shall store every sent and received SMS that cannot be correlated with the Common Message Store. If this parameter is set to **2**, client shall store every sent and received SMS and shall not attempt to correlate with the Common Message Store.	Optional parameter (It is mandatory if MESSAGE STORE AUTH is configured)

Configuration parameter	Description	RCS usage
MMS MESSAGE STORE	This parameter indicates to the client whether it shall store in the RCS Default folder any sent or received MMS. If this parameter is set to **0**, client shall not store any sent or received MMS. If this parameter is set to **1**, client shall store every sent and received MMS that cannot be correlated with the Common Message Store. If this parameter is set to **2**, client shall store every sent and received MMS and shall not attempt to correlate with the Common Message Store.	Optional parameter (It is mandatory if MESSAGE STORE AUTH is configured)
CHAT REVOKE TIMER	This parameter determines the maximum time between the client sending a Chat message and receiving its delivery notification. Once this timer expires without the client having received the delivery notification, the client shall automatically send a MessageRevoke request. For the case of a successful result, the user may be informed and the client shall fallback to SMS. When set to **0** (Default Value), sending MessageRevoke requests by the client is disabled.	Optional Parameter
RECONNECT GUARD TIMER	This parameter is applicable when CHAT REVOKE TIMER is set to a value higher than 0. It provides the minimum time the client shall be registered in IMS prior to sending a message revocation request for a chat message when the revocation timer (i.e. CHAT REVOKE TIMER) expired. Default value: 120 seconds	Optional parameter
CFS TRIGGER	This parameter is applicable when CHAT REVOKE TIMER is set to a value higher than 0. It controls the trigger for the client to fallback to SMS when revocation procedures apply. 0 (default), the client shall fall back to SMS and send the Message Revoke request right after sending the SMS 1, the client shall fall back to SMS right after receiving the MessageRevokeResponse request with the value of the result equal to “success”	Optional parameter
IM SPAM REPORTING URI	The SIP URI where the spam reports should be sent to. If specified, the service provider is expected to support processing these reports.	Optional parameter
IM SPAM REPORTING FILE CONTENT	This parameter indicates whether file content can be referred to in spam reports and can be uploaded to the HTTP Content Server if not available in the network in unencrypted format. If set to **0** (default), file content cannot be referred to in spam reports. If set to **1**, file content can be referred to in spam reports	Optional parameter

Configuration parameter	Description	RCS usage
MAX 1 TO MANY RECIPIENTS	This parameter is applicable when CHAT AUTH or STANDALONE MSG AUTH is set to 1 and it provides the maximum contacts allowed to be included in the distribution list of the 1-to-Many messaging service. 0 (default): the 1-to-Many Messaging service is disabled 1: the client can add unlimited number of recipients >1: integer value that indicates the maximum total number of recipients that can be included in the distribution list	Optional parameter
1 TO MANY SELECTED TECHNOLOGY	This parameter is applicable when the RCS Standalone messaging service is disabled (i.e. STANDALONE MSG AUTH is set to 0) and it controls the selected messaging technology for the 1-to-Many messaging service. 0 (default): SMS is selected 1: RCS 1-to-1 Chat service is selected	Optional parameter
DISPLAY NOTIFICATION SWITCH	This parameter controls whether sending of Display Notifications is enabled/disabled on the recipient's client. 0 (default), Enable sending Display Notifications, The user may still disable it using the Display Notification setting 1, Disable sending Display Notifications. The Display Notification setting is not available to the user.	Optional parameter
CHATBOT GALLERY	This parameter provides the URL pointing to the Service Provider Chatbot Gallery. The URL shall contain the "https" scheme to enforce use of secure connections between the RCS client and the Service Provider Chatbot Gallery.	Optional parameter
CHATBOT DIRECTORY	This parameter provides the URL from where a list of Chatbots can be retrieved as described in section 3.6.3.1. The URL shall contain the "https" scheme to enforce use of secure connections for the client's Chatbot Directory retrieval requests.	Optional parameter
BOTINFO FQDN ROOT	This parameter provides the root part of the FQDN to be used by the client to compose the botinfo URL as defined in section 3.6.4.1.1.	Mandatory parameter
SPECIFIC CHATBOTS LIST	This parameter provides a URL from which a list of Chatbots requiring specific management can be retrieved as described in section 3.6.3.3. Default behaviour if not provided: the procedures related to the Chatbots requiring specific management are not applicable	Optional parameter

Configuration parameter	Description	RCS usage
IDENTITY IN ENRICHED SEARCH	This parameter determines whether the i query parameter defined in section 3.6.3.1 is included in the client to Service Provider Chatbot Directory requests when the user setting to enrich the search is enabled. 0 (default): the i query parameters is not included 1: the i query parameter is included	Optional parameter
PRIVACY DISABLE	This parameter determines whether a user is allowed to request anonymization for Chatbot sessions. When set to **0**, anonymization of Chatbot sessions is enabled (default). When set to **1**, anonymization of Chatbot sessions is disabled.	Optional parameter
CHATBOT MSG TECH	This parameter controls the messaging technology for Chatbot messaging. **0**: the Chatbot Services is disabled **1** (default): the Chatbot Service is enabled with support only for 1-to-1 Chatbot sessions **2**: the Chatbot Service is enabled with support for both 1-to-1 Chatbot sessions and 1-to-1 Chatbot Standalone Messages whereby for communication to a Chatbot the message technology selection described in section 3.2.1.2 applies **3**: the Chatbot Service is enabled with support only for 1-to-1 Chatbot Standalone Messaging.	Optional parameter

Table 167: RCS Messaging related configuration parameters

A.1.4 File Transfer related configuration

The following configuration parameters for File Transfer, are defined in RCS:

Configuration parameter	Description	RCS usage
FT AUTH	This parameter controls the Service Provider's authorisation of the File Transfer service. The following values are defined: 0: File Transfer is not enabled 1: File Transfer is enabled.	Mandatory parameter
FT MAX SIZE	This parameter provides a file transfer size limit in Kilobyte (KB). If a file to be transferred is bigger than FT MAX SIZE, then the client shall not initiate procedures to send the file via the File Transfer sender procedures. The configuration parameter is not applicable for the File Transfer receiver procedures. If it is set to **0**, then no limit shall apply.	Mandatory parameter

Configuration parameter	Description	RCS usage
FT WARN SIZE	This parameter provides a warning size limit in Kilobyte (KB) to control the client behaviour for the handling of File Transfer and Rich Card media objects. If the value is set to **0**, no warning size limit applies. For the use of the configuration parameter refer to the corresponding service documentation, e.g. [GSMA PRD-RCC.71].	Mandatory parameter
FT AUT ACCEPT	This parameter provides the default for the user's setting for auto-acceptance for File Transfer and Rich Card media objects. The default setting for auto-acceptance can be set to: **0**: auto-accept is off, **1**: auto-accept is on (default). For the use of the configuration parameter refer to the corresponding service documentation, e.g. [GSMA PRD-RCC.71]..	Optional parameter
FT HTTP CS URI	This parameter configures the URI of the HTTP Content Server where files will be uploaded by the originating side in case the destination cannot accept within the validity period. The parameter shall contain a full qualified URI. The URI shall contain the "https" schema to enforce use of secure connections for the client's HTTP Content Server transactions. Default value if not provided: https://ftcontentserver.rcs.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org Whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning (as defined in [GSMA PRD-IR.67]).	Optional parameter
FT HTTP DL URI	This parameter configures the URL of the Service Provider's local File Transfer download server. If present, the client shall apply the procedures for localisation of content URLs taken from File Transfer message bodies or other content URLs the client received via RCS messaging as defined in section 4.2.3. The URL shall contain the "https" schema to enforce use of secure connections for the client's HTTP Content Server transactions. If present, it is recommended to use the following FQDN value as domain in the URL: dl.rcs.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning (as defined in [GSMA PRD-IR.67]).	Optional parameter

Configuration parameter	Description	RCS usage
FT HTTP CS USER	This parameter is the name or identity that shall be used to authenticate the RCS client trying to either get a root URL (HTTP GET request) or upload a file (HTTP post request). If not provided, the client shall use GBA authentication as described in section 4.2.2 and 4.2.3.	Optional parameter
FT HTTP CS PWD	This parameter is the password that shall be used to authenticate the RCS client trying to either get a root URL (HTTP GET request) or upload a file (HTTP post request). If not provided, the client shall use GBA authentication as described in section 4.2.2 and 4.2.3.	Optional parameter
FT HTTP FALLBACK	This parameter provides the operator default of the technology to transfer files to contacts not supporting File Transfer via HTTP. The parameter can take the following values: 0: MMS (default value) 1: text message with a link.	Optional parameter
FT MAX 1 TO MANY RECIPIENTS	This parameter controls the number of recipients for the one to many sending of a File Transfer. 0 (default): no limitation in the number of recipients positive integer value: indicates the maximum total number of recipients of a File Transfer.	Optional parameter

Table 168: RCS additional File Transfer related configuration parameters

A.1.5 Push Notification Mechanism Parameter

The following configuration parameters for Push Notification, are defined in RCS:

Configuration parameter	Description	RCS usage
PUSH NOTIFICATION	This parameter allows enabling (1) or disabling (0) the Push Notification Mechanism. If not provided; the Push Notification Mechanism is not supported for this network	Optional
CAPABILITY REFRESH	This parameter specifies capability cache expiry timer for the RCS Service Provider. An RCS client shall trigger a SIP registration before this timer expires in accordance with section 2.15.1.4. This parameter shall only be present if PUSH NOTIFICATION is set to (1).	Optional

Table 169: RCS Push Notification related configuration parameters

A.1.6 IMS Core / SIP related configuration

If the SPN configures the RCS client for SIP, the configuration document shall include the parameters defined in sections A.1.6.1 and A.1.6.2 but not the parameters in A.1.1 of [GSMA PRD-RCC.30]. If instead the SPN is configures the RCS client for RCS gRPC, it shall include the parameters in section A.1.1 of [GSMA PRD-RCC.30] but not in sections A.1.6.1 and A.1.6.2.

A.1.6.1 IMS Provisioning Parameters

IMS parameters shall be configured as defined in [GSMA PRD-RCC.15]. Not all parameters in the IMS Management Object defined in [3GPP TS 24.167] that is referred from [GSMA PRD-RCC.15] are relevant within the context of RCS though. Following table lists the parameters and their RCS usage:

Configuration parameter	Description	RCS usage
ConRef	Represents a network access point object	Mandatory parameter Not used for RCS Provided with dummy value if RCS is only user of MO: *dummy.apn* Selection of the APN for RCS depends on other parameters as defined in section 2.8.1.4
PDP_ContextOperPref	Indicates an operator's preference to have a dedicated Packet Data Protocol (PDP) context for SIP signalling.	Mandatory parameter Not used for RCS, always set to 0 (no preference) if RCS is only user of MO.
P-CSCF_Address	an FQDN or an IPv4 address to an IPv4 P-CSCF	Optional parameter Not provided if RCS is only user of MO. The P-CSCF address for RCS is provided in LBO_P-CSCF_Address
Timer_T1	Defines the SIP timer T1 – the RTT estimate	Mandatory parameter
Timer_T2	Defines the SIP timer T2 – the maximum retransmit interval for non-INVITE requests and INVITE responses.	Mandatory parameter
Timer_T4	Defines the SIP timer T4 – the maximum duration a message will remain in the network.	Mandatory parameter

Configuration parameter	Description	RCS usage
Private_user_identity	Represents the private user identity NOTE: It is recommended to set it to the same value as for the configuration parameter Realm User Name defined in section 2.2.1.2 of [GSMA PRD-RCC.15].	Mandatory parameter Used as defined in [GSMA PRD-RCC.15]
Public_user_identity	Represents a public user identity.	Mandatory parameter Used as defined in [GSMA PRD-RCC.15] and if applicable [GSMA PRD-NG.102]
Home_network_domain_name	Indicates the operator's home network domain.	Mandatory parameter Used as defined in [GSMA PRD-RCC.15] and if applicable [GSMA PRD-NG.102] Recommended to use ims.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org whereby <MNC> and <MCC> shall be replaced by the respective values of the home network in decimal format and with a 2-digit MNC padded out to 3 digits by inserting a 0 at the beginning (as defined in [GSMA PRD-IR.67]).
ICSI_List	Used to allow a reference to a list of IMS communication service identifiers that are supported by a subscriber's network for that subscriber.	Mandatory Tree Tree is not used for RCS, but mandatory to be provided. No leafs shall be provided if RCS is only user of MO.

Configuration parameter	Description	RCS usage
LBO_P-CSCF_Address	Provides a reference to a list of P-CSCFs	Optional tree Used as defined in [GSMA PRD-RCC.15] and if applicable [GSMA PRD-NG.102]
Address (LBO_P-CSCF_Address)	Defines the FQDN, the IPv4 address or the IPv6 address of a P-CSCF	Mandatory parameter
AddressType (LBO_P-CSCF_Address)	Defines the type of address stored in the Address leaf node	Mandatory parameter
Resource_Allocation_Mode	Indicates whether UE initiates resource allocation for the media controlled by IM CN subsystem for all IMS sessions not covered by any "ICSI Resource Allocation Mode", when both UE and network can initiate resource allocation	Optional parameter Parameter not provided if RCS is only user of MO.
Voice_Domain_Preference_E_UTRAN	Indicates network operator's preference for selection of the domain to be used for voice communication services by the UE.	Optional parameter Parameter not provided if RCS is only user of MO.
SMS_Over_IP_Networks_Indication	Indicates network operator's preference for selection of the domain to be used for short message service (SMS) originated by the UE.	Optional parameter Parameter not provided if RCS is only user of MO.
Keep_Alive_Enabled	Indicates whether the UE sends keep alives	Mandatory parameter
Voice_Domain_Preference_UTRAN	Indicates network operator's preference for selection of the domain to be used for voice communication services by the UE.	Optional parameter Parameter not provided if RCS is only user of MO.
/<X>/Mobility_Management_IMS_Voice_Termination	Indicates whether the UE mobility management performs additional procedures to support terminating access domain selection by the network.	Optional parameter Parameter not provided if RCS is only user of MO.
RegRetryBaseTime	Represents the value of the base-time parameter	Optional parameter
RegRetryMaxTime	Represents the value of the max-time parameter	Optional parameter

Configuration parameter	Description	RCS usage
PhoneContext_List	Used to allow a reference to a list of phone-context parameter values for other local numbers, than geo-local or home-local numbers	Optional tree Tree is not provided for RCS because only home-local and geo-local numbers are used (see section 2.5)
SS_domain_setting	Indicates the network operator's preference for the selection of the domain used by the UE when performing supplementary services (SS) setting control for voice services.	Optional parameter Parameter not provided if RCS is only user of MO.
PS_domain_IMS_SS_control_ preference	Provides a means to define the method for which Supplementary Services are controlled by the UE when SS setting control is to be invoked over the PS domain.	Optional parameter Parameter not provided if RCS is only user of MO.

Table 170: Usage of [3GPP TS 24.167] MO parameters for RCS

A.1.6.2 RCS Specific Provisioning parameters

This RCS specification includes the following additional IMS Core/SIP related configuration parameters:

Configuration parameter	Description	RCS usage
RCS VOLTE SINGLE REGISTRATION	This parameter defines the behaviour regarding the registration for RCS services for devices supporting the IMS well-known APN. 0, the device *Shall* follow a dual registration approach (transition solution) 1 (default if not provided), the device *Shall* follow a single registration (target solution). 2, the device Shall follow a single registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP) when in the home network, and Shall follow a dual registration approach when roaming (transition solution). The parameter is also used to control the APN selection on devices that are not enabled for Multimedia Telephony as described in section 2.8.1.4. Devices not supporting the IMS well-known APN shall ignore the configuration parameter.	Optional Parameter

Table 171: RCS additional IMS Core/SIP related configuration parameters

A.1.7 Geolocation related configuration

This RCS specification includes the following geolocation related configuration parameters.

Configuration parameter	Description	RCS usage
PROVIDE GEOLOC PUSH	This parameter allows enabling (**1**) or disabling (**0**) the Geolocation PUSH service.	Mandatory Parameter

Table 172: RCS additional geolocation related configuration parameters

A.1.8 Configuration related with Address book Back-up/Restore

This RCS specification does not include any additional address book back-up/restore related configuration parameters.

A.1.9 Capability Discovery related configuration

This RCS specification includes the following RCS Specific configuration parameters related to the Capability Discovery:

Configuration parameter	Description	RCS usage
DISABLE INITIAL ADDRESS BOOK SCAN	This parameter controls whether the device/client performs a Capability Discovery for all contacts in the address book when it is first started. When set to **0** (Default value), the device/client shall perform the scan When set to **1**, the device shall skip the scan and only perform Capability Discovery requests based on the other triggers defined in section 2.6.	Optional parameter
CAPABILITY INFO EXPIRY	When using the Capability Discovery mechanism and with the aim of minimising the traffic, an expiry time is set in the capability information for real-time and non-real-time communication services fetched using SIP OPTIONS or Presence. When capability information was obtained more recently than the value configured for this parameter, it shall be considered as being still valid. When set to 0, the cached capabilities shall be considered to never expire and shall be invalidated only by a conclusive capability and service availability exchange indicating that they are no longer valid Default value:2592000 (30 days)	Optional parameter
SERVICE AVAILABILITY INFO EXPIRY	This parameter controls expiration of cached service availability information of contacts. Its value shall indicate the validity time of cached availability information in seconds. Default value: 60	Optional Parameter

Configuration parameter	Description	RCS usage
CAPABILITY DISCOVERY MECHANISM	This parameter allows selecting the default Capability and new user Discovery mechanism. If set to OPTIONS (**0**), the default mechanism employed for Capability Discovery and new users will be OPTIONS. If set to PRESENCE (**1**), the mechanism employed for Capability Discovery and new users will relay presence-based information. If not provided or set to OFF (**2**) the Capability Discovery mechanism is disabled.	Optional parameter
CAPABILITY DISCOVERY ALLOWED PREFIXES	The configuration parameter provides prefixes or rules to identify phone numbers contained in the address book or entered by the user which shall be considered for Capability and new user Discovery. If the configuration parameter is absent, all phone numbers shall be considered for Capability and new user Discovery. If a number shall be considered for Capability and new user Discovery, then the client shall invoke the procedures for Capability and new user Discovery in accordance with the definitions in section 2.6. If a number shall not be considered for Capability and new user Discovery, the client shall not invoke the procedures for Capability and new user Discovery defined in section 2.6 and shall consider the phone number as not RCS capable. If Capability and new user Discovery through SIP OPTIONS is applied, the client shall process received discovery request in accordance with the definitions in section 2.6.1.1. The service provider should take the subscriber's HPLMN numbering scheme into account when defining the value of the configuration parameter.	Optional parameter
NON RCS CAPABILITY INFO EXPIRY	This parameter allows to better control the amount of Capability Discovery requests sent to non RCS contacts. When updating a capability for a non RCS contact, a Capability Discovery takes place only if the time since the last capability update took place is greater than this parameter. Default value: 2592000 (30 days)	Optional parameter

Table 173: RCS additional Capability Discovery related configuration parameters

A.1.10 Void

A.1.11 Void

A.1.12 Service Provider specific extensions

A Service Provider may provide Service Provider specific extensions to the configuration parameters. This can be done both at the individual service level and add the global level (e.g. for the configuration of Service Provider specific services). All parameters are optional and if provided may be ignored by clients that are not Service Provider specific.

A.1.13 Void

A.1.14 Data Off

A.1.14.1 RCS Configuration Parameters

The RCS specification includes following configuration parameters controlling the behaviour of the respective services when connected over cellular networks using a primary device that is not using the internet APN for RCS (see section 2.8.1.4) and data is switched off by the user:

Configuration parameter	Description	RCS usage
RCS MESSAGING DATA OFF	This parameter indicates whether the • 1-to-1 and Group Chat (see section 3.2.3 and 3.2.4 respectively), • Standalone Messaging (see section 3.2.2), • Geolocation PUSH (see section 3.2.6) and • Chatbot Communication (see section 3.6.8, including Chatbot Directory Access defined in section 3.6.3.1, Chatbot Information Retrieval defined in section 3.6.4 and the procedures to control anonymization defined in section 3.6.5.1) services *Should* remain available in case the cellular data switch is switched off (either toggled manually by the user or automatically during roaming). When set to 0 the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are not cellular data off exempt services on cellular networks when cellular data is switched off. When set to 1 the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are cellular data off exempt services on cellular networks when cellular Data is switched off. When set to 2 the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are cellular data off exempt services on cellular networks when cellular Data is switched off and the device is attached to the HPLMN. NOTE: This configuration parameter when set to 0 or 2 will disable the File Transfer service. An MNO is thus not able to disable chat or standalone messaging but enable File Transfer.	Mandatory Parameter

Configuration parameter	Description	RCS usage
FILE TRANSFER DATA OFF	This parameter indicates whether the File Transfer service (defined in section 3.2.5, including Audio Messaging defined in section 3.2.7) *Should* remain available in case the cellular data switch is switched off (either toggled manually by the user or automatically during roaming). When set to 0 the File Transfer Service is not a cellular data off exempt service on cellular networks when cellular data is switched off. When set to 1 the File Transfer service is a cellular data off exempt service on cellular networks when cellular Data is switched off. When set to 2 the File Transfer service is a cellular data off exempt service on cellular networks when cellular Data is switched off and the device is attached to the HPLMN. When File Transfer is disabled, then the client shall behave as defined in section 3.2.5.2 for the case where File Transfer is not authorised via client configuration.	Optional Parameter It becomes Mandatory if RCS MESSAGING DATA OFF is set to 1 or 2.
MMS DATA OFF	This parameter indicates whether MMS should remain available in case the cellular data switch is switched off (either toggled manually by the user or automatically during roaming). When set to **0** MMS is not a cellular data off exempt service when cellular data is switched off. When set to **1** (default value) MMS a cellular data off exempt service when cellular Data is switched off. When set to **2** MMS is a cellular data off exempt service when cellular data is switched off and the device is attached to the HPLMN, otherwise MMS is disabled. NOTE: the device's settings to enable/disable automatic download of received MMS messages remain applicable.	Optional Parameter

Configuration parameter	Description	RCS usage
SYNC DATA OFF	This parameter indicates whether the synchronization with the Common Message Store (defined in section 4.1) should remain available in case the cellular data is switched off (either toggled manually by the user or automatically during roaming). When set to **0** the synchronisation with the Common Message Store is not a cellular data off exempt service when cellular data is switched off. When set to **1** synchronisation with the Common Message Store is a cellular data off exempt service when cellular data is switched off. When set to **2** the synchronisation with the Common Message Store is a cellular data off exempt service when cellular data is switched off and the device is attached to the HPLMN. If the synchronisation with the Common Message Store is not a cellular data off exempt service, the client shall not invoke the synchronisation with the Common Message Store, shall not set message flags via the Message Store and the event notification framework and shall not store SMS and MMS messages. Once synchronisation is enabled again, the client shall invoke the procedures for the synchronisation with the Common Message Store for the missed events.	Optional Parameter It becomes mandatory if MESSAGE STORE AUTH is present (see A.1.3)

Table 174: RCS Data Off Configuration Parameters

NOTE: These parameters only affect behaviour on cellular networks. Services that can be offered over non-cellular networks remain available over such networks irrespective of the setting of the cellular data switch. These parameters also affect services when RCS uses the HOS APN.

A.1.14.2 Non-Access Stratum Configuration Parameters

RCS uses the configuration parameters of the Non-Access Stratum Management Object defined in [3GPP TS 24.368] to control the behaviour of the respective services when connected over cellular networks using a primary device that is not using the internet APN for RCS (see section 2.8.1.4) and data is switched off by the user. The following parameters of the non-access stratum management object are used.

Configuration parameter	Description	RCS usage
Device Management over PS data off exemption	This parameter indicates whether client configuration for Chatbots according to section 3.6.3.3 is a cellular data off exempt service. The following values are defined: 0: the device management services are not defined as a cellular data off exempted services. 1: the device management services are defined as a cellular data off exempted services (default value).	Optional Parameter
Device Management over PS data off roaming exemption	This parameter indicates whether client configuration for Chatbots according to section 3.6.3.3 is a cellular data off exempt service when roaming. The following values are defined: 0: the device management services are not defined as a cellular data off exempted services when roaming. 1: the device management services are defined as a cellular data off exempted services when roaming (default value).	Optional Parameter

Table 175: Non-Access Stratum Configuration Parameters

A.2 Provisioning Document of the RCS Management tree

The RCS Management tree data is conveyed between the Auto-Configuration Server and the client by use of a configuration XML document as defined in section 4 of [GSMA PRD-RCC.14].

This section defines the application characteristic for the management of RCS configuration parameters.

A.2.1 Application characteristic type for the RCS Management tree

The following parameters and values are defined for the application characteristic type for the use in the RCS Management tree.

The AppId parameter is used to uniquely identify the RCS Management tree in a configuration XML document.

Status	Occurrence	Format	Min. Access Types
Required	One	chr	Get

Table 176: Application characteristic type parameter AppID

- Value: ap2002
- Post-reconfiguration actions: n/a
- Associated HTTP XML parameter: “AppID”

The To-Appref parameter provides the link between the RCS Management tree application characteristic with an instance of the IMS MO identified by the APPREF parameter defined in [GSMA PRD-RCC.15].

Status	Occurrence	Format	Min. Access Types
Required	One	chr	Get

Table 177: Application characteristic type parameter To-Appref

- Value: string containing the reference identity of the linked IMS MO instance
- Post-reconfiguration actions: If the value changes, the client shall re-register or de-register in IMS to disable RCS services and RCS services and register or re-register in IMS for RCS services using the new IMS core network configuration.
- Associated HTTP XML parameter: “To-Appref”

```
<characteristic type=”APPLICATION”>
        <parm name=”AppID” value=”ap2002”/>
        <parm name=”To-Appref” value=”X”/>
        <!-- other characteristics of the RCS configuration are embedded here -->
</characteristic>
```

Table 178: Application type XML structure

A.2.2 Services sub tree additions

This RCS specification includes the following additions as a new services sub tree, the Services MO sub tree. Please note this sub tree is not included in any other specifications. So no other nodes from those specifications need to be added:

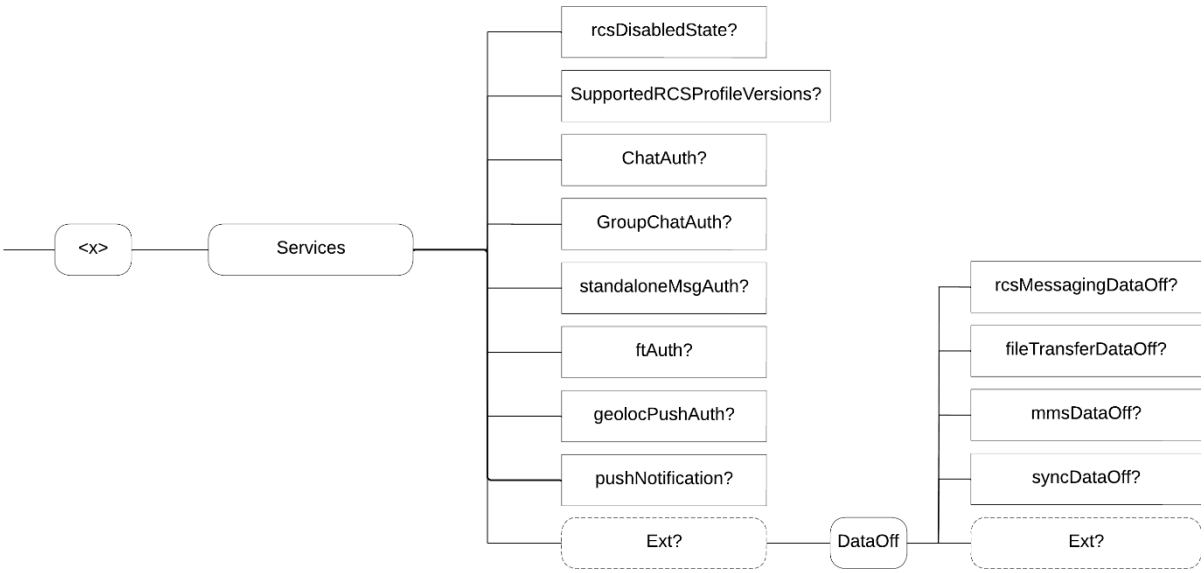


Figure 66: RCS additions, Services sub tree

The associated HTTP configuration XML structure is presented in the table below:

```
<characteristic type="SERVICES">
        <parm name="SupportedRCSProfileVersions" value="X"/>
        <parm name="ChatAuth" value="X"/>
        <parm name="GroupChatAuth" value="X"/>
        <parm name="ftAuth" value="X"/>
        <parm name="standaloneMsgAuth" value="X"/>
        <parm name="geolocPushAuth" value="X"/>
        <parm name=pushNotification" value="X"/>
        <characteristic type="Ext">
              <characteristic type="DataOff">
                    <parm name="rcsMessagingDataOff" value="X"/>
                    <parm name="fileTransferDataOff" value="X"/>
                    <parm name="mmsDataOff" value="X"/>
                    <parm name="syncDataOff" value="X"/>
                    <characteristic type="Ext"/>
              </characteristic>
        </characteristic>
</characteristic>
```

Table 179 : Services MO sub tree associated HTTP configuration XML structure

NOTE: rcsDisabledState is not included in Table 179 since the presence of that parameter would invalidate most of the other parameters included.

Node: /<x>/Services

Under this interior node the RCS parameters related to the enabling/disabling of services are placed

Status	Occurrence	Format	Min. Access Types
Required	One	node	Get

Table 180: Services MO sub tree addition services node

- Values: N/A
- Associated HTTP XML characteristic type: “SERVICES”

Node: /<x>/Services/rcsDisabledState

Leaf node that controls the state of the RCS services on the device

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 181: Services MO sub tree addition parameters (rcsDisabledState)

- Values:
  0, the client is temporarily disabled
  -1, the client is permanently disabled
  -2, the client is permanently disabled, but user triggered events result in an attempt to re-enable the client
  -3, the client is placed in dormant state
- Post-reconfiguration actions: The client shall change the state of the relevant RCS services as described in section 2.3.2.5.

- Associated HTTP XML parameter: “rcsDisabledState”

NOTE: Given that due to the definition in section A.1.1, this parameter and the other parameters in this management object are mutually exclusive it is not included in Table 179.

Node: /<x>/Services/SupportedRCSProfileVersions

Leaf node that indicates the RCS profile versions supported by the Auto-Configuration Server

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 182: Services MO sub tree addition parameters (SupportedRCSProfileVersions)

- Values:
  Comma separated list of RCS profile versions supported by the Auto-Configuration Server. The RCS profile versions are defined in the rcs_profile parameter for the client configuration request in [GSMA PRD-RCC.71].
  Example:
  foo,bar
- Post-reconfiguration actions: No action at the time of re-configuration.
- Associated HTTP XML parameter ID: “SupportedRCSProfileVersions”

Node: /<x>/Services/ChatAuth

Leaf node that represents the authorisation for the user to send chat messages

The node shall be instantiated if the rcsDisabledState node is not provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 183: Services MO sub tree addition parameters (ChatAuth)

- Values: 0, 1
  0- Indicates that initiating 1-to-1 chat session is disabled
  1- Indicates that initiating 1-to-1 chat sessions is enabled
- Post-reconfiguration actions: No action at the time of re-configuration.
- Associated HTTP XML parameter ID: “ChatAuth”

Node: /<x>/Services/GroupChatAuth

Leaf node that represents the authorisation for the user to receive chat messages and use the group chat service

The node shall be instantiated if the rcsDisabledState node is not provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 184: Services MO sub tree addition parameters (GroupChatAuth)

- Values: 0, 1
  0- Indicates that Group Chat service and receiving 1-to-1 chat session invitations is disabled
  1- Indicates that Group Chat and receiving 1-to-1 chat session invitations service is enabled
- Post-reconfiguration actions: If the value transits from "0" to "1" then the client shall register or re-register in IMS to add the feature tags for Chat defined in section 2.4.4, if conditions allow. If the value of transits from "1" to "0" and if the client is registered in IMS for Chat and Group Chat as defined in section 2.4.4, then the client shall initiate an IMS de-registration or re-registration to remove the feature tags for Chat defined in section 2.4.4.
- Associated HTTP XML parameter ID: “GroupChatAuth”

Node: /<x>/Services/ftAuth

Leaf node that represents the authorisation for user to use the File Transfer

The node shall be instantiated if the rcsDisabledState node is not provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 185: Services MO sub tree addition parameters (ftAuth)

- Values: 0, 1
  0- Indicates that File Transfer is disabled
  1- Indicates that File Transfer is enabled
- Post-reconfiguration actions: If the value of configuration parameter transits from 0 to 1 and the client is registered in IMS, then the client shall re-register to add the media feature tag defined for File Transfer. If the value of configuration parameter transits from 1 to 0 and the client is registered in IMS, then the client shall re-register to remove the media feature tag defined for File Transfer.
- Associated HTTP XML parameter ID: “ftAuth”

Node: /<x>/Services/standaloneMsgAuth

Leaf node that represents the authorisation for user to use the standalone messaging service

The node shall be instantiated if the rcsDisabledState node is not provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 186: Services MO sub tree addition parameters (standaloneMsgAuth)

- Values: 0, 1, 2
  0- The standalone messaging service is not provided. SMS and MMS is used instead
  1- Sending and receiving of messages via the standalone messaging service is provided and uses CPM as specified in [GSMA PRD-RCC.11].
  2- Receiving of messages via the standalone messaging service is provided and uses CPM as specified in [GSMA PRD-RCC.11]. SMS and MMS are still used for sending.
- Post-reconfiguration actions:
  - If the value of the configuration parameter transits from value 0 to value 1 or 2, the client may wait till the next scheduled refresh re-REGISTER request or may issue re-REGISTER request immediately.
  - If the value of the configuration parameter transits from value 1 or 2 to value 0, the client may wait till the next scheduled refresh re-REGISTER request or may issue re-REGISTER request immediately.
  - If the value of the configuration parameter transits from value 1 to value 2, the client shall stop sending messages via the standalone messaging service.
  - If the value of the configuration parameter transits from value 2 to value 1, the client shall be able to send messages via the standalone messaging service.
- Associated HTTP XML parameter ID: “standaloneMsgAuth”

Node: /<x>/Services/geolocPushAuth

Leaf node that represents the authorisation for the user to use the Geolocation PUSH service

The node shall be instantiated if the rcsDisabledState node is not provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 187: Services MO sub tree addition parameters (geolocPushAuth)

- Values: 0, 1
  0- Indicates that Geolocation PUSH service is disabled
  1- Indicates that Geolocation PUSH service is enabled
- Post-reconfiguration actions: The client should be reset and should perform the complete first-time registration procedure following a reconfiguration (e.g. OMA-DM/HTTP) as described in section 2.3.
- Associated HTTP XML parameter ID: “geolocPushAuth”

Node: /<x>/Services/pushNotification

Leaf node that represents the authorisation for the user to use the Push Notification Mechanism

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 188: Services MO sub tree addition parameters (pushNotification)

- Values: 0, 1
  0 - Indicates that the Push Notification Mechanism is disabled (Default if not provided)
  1- Indicates that the Push Notification Mechanism is enabled
- Post-reconfiguration actions: The client should be reset and should perform the complete first-time registration procedure following a reconfiguration (e.g. OMA-DM/HTTP) as described in section 2.3.
- Associated HTTP XML parameter ID: “pushNotification”

Node: /<x>/Services/Ext

An extension node for Service Provider specific parameters. Clients that are not aware of any extensions in this subtree (e.g. because they are not Service Provider specific) should not instantiate this tree.

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	Node	Get

Table 189: Services MO sub tree addition Service Provider Extension Node

- Values: N/A
- Post-reconfiguration actions: The client should be reset and should perform the complete first-time registration procedure following a reconfiguration as described in section 2.3.
- Associated HTTP XML characteristic type: “EXT”

Node: /<x>/Services/Ext/DataOff

Under this interior node where the specific RCS parameters are placed that relate to the services behaviour on cellular networks when the cellular data switch is switched off.

It shall be instantiated for primary devices where it is required to be supported.

NOTE: This tree is included as part of the ext tree rather than sitting directly under Services for historic reasons

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	Node	Get

Table 190: Data Off Services Extension MO sub tree addition node

- Values: N/A
- Associated HTTP XML characteristic type: “DataOff”

Node: /<x>/Services/Ext/DataOff/rcsMessagingDataOff

Controls the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication (including Chatbot Directory access, Chatbot Information Retrieval and control of anonymization) service behaviour when the cellular data switch is switched off.

The parameter is only applicable in case the Chat or Standalone services are supported. It will not be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Int	Get, Replace

Table 191: Data Off Services Extension MO sub tree addition parameters (rcsMessagingDataOff)

- Values:
  0: the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are not cellular data off exempt services
  1: the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are cellular data off exempt services
  2: the Chat, Standalone Messaging, Geolocation PUSH and Chatbot Communication services are cellular data off exempt services if the device is attached to the HPLMN.
- Post-reconfiguration actions: If the value of the configuration parameter transits from 0 to 1 or from 0 to 2 while the device is connected to the HPLMN and at least one RCS messaging service is authorised the client shall (re-)register in IMS to add the relevant media feature tags for Chat, File Transfer, Standalone Messaging, Geolocation PUSH and Chatbot Communication services according to the authorisation of these services.
  If the value of the configuration parameter transits from 1 to 0 or from 2 to 0 while the device is connected to a cellular access network other than the HPLMN and at least one of the RCS messaging services is registered in IMS, the client shall de- or re-register with the IMS to remove the media feature tags for Chat, File Transfer, Standalone Messaging, Geolocation PUSH or Chatbot Communication services if these have been registered.
- Associated HTTP XML characteristic type: “rcsMessagingDataOff”

Node: /<x>/Services/Ext/DataOff/fileTransferDataOff

Controls the File Transfer service behaviour (including Audio Messaging) when the cellular data switch is switched off.

The parameter is only applicable in case the File Transfer service is supported and rcsMessagingDataOff is set to 1 or 2. It will not be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Int	Get, Replace

Table 192: Data Off Services Extension MO sub tree addition parameters (fileTransferDataOff)

- Values:
  0: the File Transfer service is not a cellular data off exempt service
  1: the File Transfer service is a cellular data off exempt service
  2: the File Transfer service is a cellular data off exempt service if the device is attached to the HPLMN.
- Post-reconfiguration actions: If the value of the configuration parameter transits from 0 to 1 or from 0 to 2 while the device is connected to the HPLMN and the File Transfer service is authorised, the client shall (re-)register in IMS to add the media feature tag for File Transfer via HTTP.
  If the value of the configuration parameter transits from 1 to 0 or from 2 to 0 while the device is connected to a cellular access network other than the HPLMN and File Transfer via HTTP is registered in IMS, the client shall de- or re-register in IMS to remove the media feature tag for File Transfer via HTTP.
- Associated HTTP XML characteristic type: “fileTransferDataOff”

Node: /<x>/Services/Ext/DataOff/mmsDataOff

Controls the MMS behaviour when the cellular data switch is switched off.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Int	Get, Replace

Table 193: Data Off Services Extension MO sub tree addition parameters (mmsDataOff)

- Values:
  0: the MMS is not a cellular data off exempt service.
  1: the MMS is a cellular data off exempt service
  2: the MMS is a cellular data off exempt service if the client is attached to the HPLMN.
- Post-reconfiguration actions: no specific actions.
- Associated HTTP XML characteristic type: “mmsDataOff”

Node: /<x>/Services/Ext/DataOff/syncDataOff

Controls the behaviour for synchronisation with the Common Message when the cellular data is switched off.

The parameter is only applicable in case the Common Message Store is supported. It will not be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Int	Get, Replace

Table 194: Data Off Services Extension MO sub tree addition parameters (syncDataOff)

- Values:
  0: the synchronisation with the Common Message Store is not a cellular data off exempt service

1: the synchronisation with the Common Message Store is a cellular data off exempt service
2: the synchronisation with the Common Message Store is a cellular data off exempt service if the device is attached to the HPLMN.

- Post-reconfiguration actions: If the value of the configuration parameter transits from 1 to 0 or from 2 to 0 while the device is connected to a cellular access network other than the HPLMN the client should trigger a data connection triggered synchronization with the Common Message Store.
- Associated HTTP XML characteristic type: “syncDataOff”

A.2.3 Presence sub tree

The HTTP configuration XML structure associated with the Presence parameters from the Presence MO defined in [PRESENCE2MO] is presented in the table below

```
<characteristic type="PRESENCE">
        <parm name="client-obj-datalimit" value="X"/>
        <parm name="content-serveruri" value="X"/>
        <parm name="source-throttlepublish" value="X"/>
        <parm name="max-number-ofsubscriptions-inpresence-list" value="X"/>
        <parm name="service-uritemplate" value="X"/>
        <parm name="RLS-URI" value="X"/>
</characteristic>
```

Table 195: Presence sub tree associated HTTP configuration XML structure

A.2.4 Messaging sub tree additions

RCS includes the following additions as a new configuration sub tree, the Messaging MO subtree:

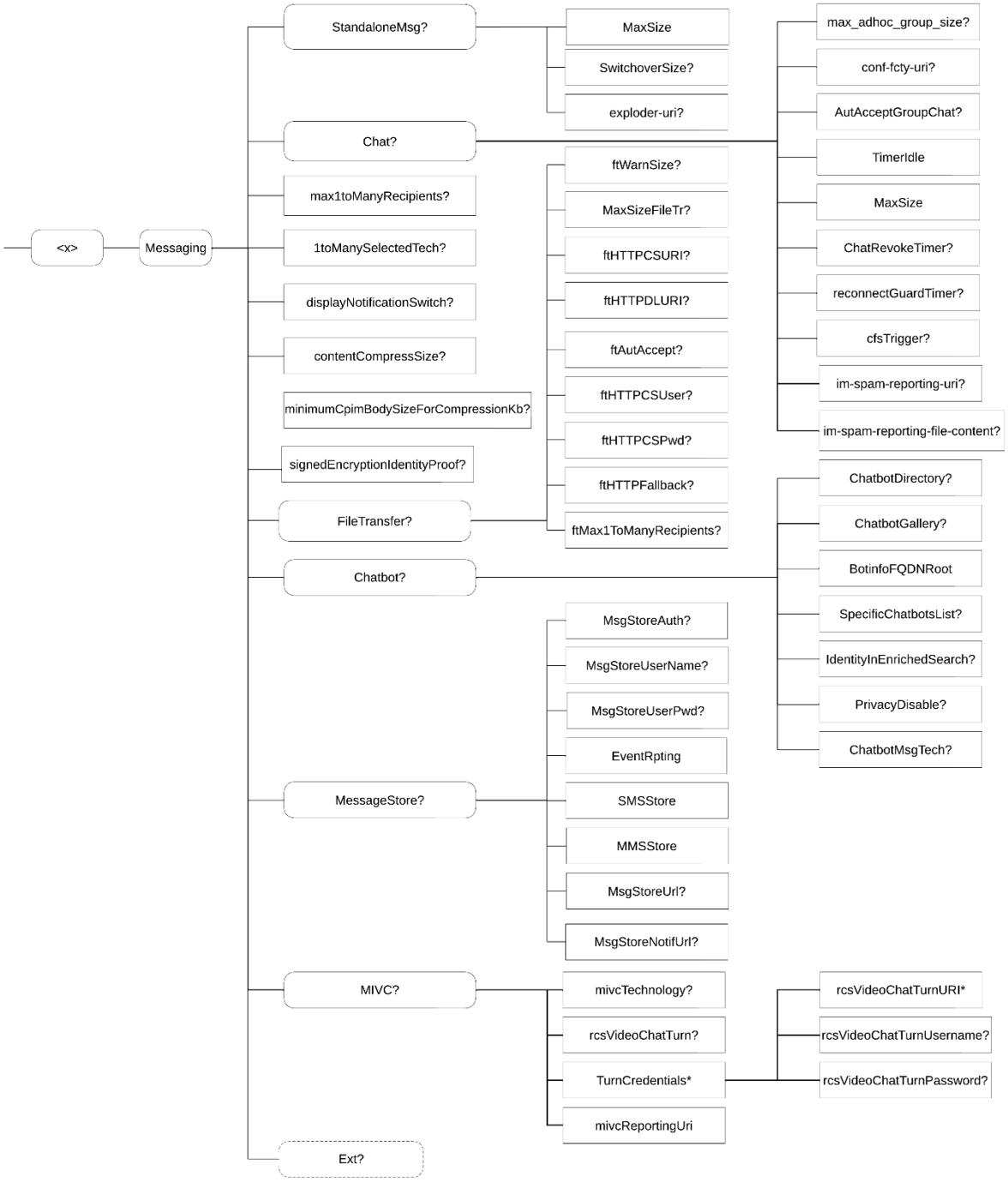


Figure 67: RCS additions to the Messaging MO sub tree

The associated HTTP configuration XML structure associated to the Messaging parameters is presented in the table below. Only RCS specific parameters (shown in blue) are included as OMA does not define a CPM MO.

```
<characteristic type="MESSAGING">
        <characteristic type="StandaloneMsg">
                <parm name="MaxSize" value="X"/>
                <parm name="SwitchoverSize" value="X"/>
                <parm name="exploder-uri" value="X"/>
        </characteristic>
        <characteristic type="Chat">
                <parm name="max_adhoc_group_size" value="X"/>
                <parm name="conf-fcty-uri" value="X"/>
                <parm name="AutAcceptGroupChat" value="X"/>
                <parm name="TimerIdle" value="X"/>
                <parm name="MaxSize" value="X"/>
                <parm name="ChatRevokeTimer" value="X"/>
                <parm name="reconnectGuardTimer" value="X"/>
                <parm name="cfsTrigger" value="X"/>
                <parm name="im-spam-reporting-uri" value="X"/>
                <parm name="im-spam-reporting-file-content" value="X"/>
        </characteristic>
        <parm name="max1ToManyRecipients" value="X"/>
        <parm name="1toManySelectedTech" value="X"/>
        <parm name="displayNotificationSwitch" value="X"/>
        <parm name="contentCompressSize" value="X"/>
        <parm name="minimumCpimBodySizeForCompressionKb" value="X"/>
        <parm name="signedEncryptionIdentityProof" value="X"/>
        <characteristic type="FileTransfer">
                <parm name="ftWarnSize" value="X"/>
                <parm name="MaxSizeFileTr" value="X"/>
                <parm name="ftAutAccept" value="X"/>
                <parm name="ftHTTPCSURI" value="X"/>
                <parm name="ftHTTPDLURI" value="X"/>
                <parm name="ftHTTPCSUser" value="X"/>
                <parm name="ftHTTPCSPwd" value="X"/>
                <parm name="ftHTTPFallback" value="X"/>
                <parm name=" ftMax1ToManyRecipients" value="X"/>
        </characteristic>
        <characteristic type="Chatbot">
                <parm name="ChatbotDirectory" value="X"/>
                <parm name="ChatbotGallery" value="X"/>
                <parm name="BotinfoFQDNRoot" value="X"/>
                <parm name="SpecificChatbotsList" value="X"/>
                <parm name=”IdentityInEnrichedSearch” value=”X”/>
                <parm name=”PrivacyDisable” value=”X”/>
                <parm name=”ChatbotMsgTech” value=”X”/>
        </characteristic>
        <characteristic type="MessageStore">
                <parm name="MsgStoreUrl" value="X"/>
                <parm name="MsgStoreNotifUrl" value="X"/>
                <parm name="MsgStoreAuth" value="X"/>
                <parm name="MsgStoreUserName" value="X"/>
                <parm name="MsgStoreUserPwd" value="X"/>
                <parm name="EventRpting" value="X"/>
                <parm name="AuthArchive" value="X"/>
                <parm name="SMSStore" value="X"/>
                <parm name="MMSStore" value="X"/>
        </characteristic>
        <characteristic type="MIVC">
                <parm name="mivcTechnology" value="X"/>
```

```
            <parm name="rcsVideoChatTurn" value="X"/>
            <characteristic type="TurnCredentials">
                <parm name="rcsVideoChatTurnURI" value="X"/>
                <parm name="rcsVideoChatTurnUsername" value="X"/>
                <parm name="rcsVideoChatTurnPassword" value="X"/>
            </characteristic>
            <parm name="mivcReportingUri" value="X"/>
        </characteristic>
        <characteristic type="Ext"/>
</characteristic>
```

Table 196 : Messaging sub tree associated HTTP configuration XML structure

Node: /<x>/Messaging

Under this interior node the RCS parameters related to the Messaging configuration are placed.

Status	Occurrence	Format	Min. Access Types
Required	One	node	Get

Table 197: Messaging MO sub tree addition Messaging node

- Values: N/A
- Associated HTTP XML characteristic type: “MESSAGING”

Node: /<x>/Messaging/StandaloneMsg

Interior node where parameters related to the RCS Text message and Multimedia message service based on CPM Standalone Messaging are provided

This node is not instantiated if the Service Provider does not enable Standalone Messaging.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	node	Get

Table 198: Messaging MO sub tree addition Standalone messaging node

- Values: N/A
- Associated HTTP XML characteristic type: “StandaloneMsg”

Node: /<x>/Messaging/StandaloneMsg/MaxSize

Leaf node that represents the maximum authorised size of the content payload of a text or multimedia message without transfer encoding

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get

Table 199: Messaging MO sub tree addition parameters (MaxSize)

- Values: <content maximum size in bytes>
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “MaxSize”

Node: /<x>/Messaging/StandaloneMsg/SwitchoverSize

Leaf node that determines the maximum message size of a Pager Mode CPM Standalone Message for the switch over from the Pager Mode to the Large Message Mode.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 200: Messaging MO sub tree addition parameters (SwitchoverSize)

- Values: <switchover size in bytes>
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “SwitchoverSize”

Node: /<x>/Messaging/StandaloneMsg/exploder-uri

Leaf node that represents the address to be used as Request URI when sending a 1-to-Many CPM Standalone Message

Status	Occurrence	Format	Min. Access Types
Required	One	chr	Get

Table 201: Messaging MO sub tree addition parameters (exploder-uri)

- Values: the URI to be used as exploder URI
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “exploder-uri”

Node: /<x>/Messaging/Chat

Interior node where parameters related to the RCS Chat messaging service are provided

This node is not instantiated if the Service Provider does not enable Chat or Group Chat and is required to be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	node	Get

Table 202: Messaging MO sub tree addition Chat messaging node

- Values: N/A
- Associated HTTP XML characteristic type: “Chat”

Node: /<x>/Messaging/Chat/max_adhoc_group_size

Leaf node that represents the maximum number of participants that are allowed to be invited for a new Group Chat

It is required to be instantiated if a service provider enables Group Chat.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 203: Messaging MO sub tree addition parameters (max_adhoc_group_size)

- Values: <the maximum number of invitees>
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “max_adhoc_group_size”

Node: /<x>/Messaging/Chat/conf-fcty-uri

Leaf node that represents the URI of the conference factory to be used as Request-URI when initiating a new Group Chat

It is required to be instantiated if a service provider enables Group Chat.

Status	Occurrence	Format	Min. Access Types
Required	One	chr	Get

Table 204: Messaging MO sub tree addition parameters (conf-fcty-uri)

- Values: <the conference factory URI>
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “conf-fcty-uri”

Node: /<x>/Messaging/Chat/AutAcceptGroupChat

Leaf node that represents the automatic/manual Group Chat session answer mode

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get

Table 205: Messaging MO sub tree addition parameters (AutAcceptGroupChat)

- Values: 0, 1
  0- Indicates manual answer mode
  1- Indicates automatic answer mode (default value)
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “AutAcceptGroupChat”

Node: /<x>/Messaging/Chat/TimerIdle

Leaf node that represents the timeout for a chat session in idle mode (when there is no chat user activity)

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get

Table 206: Messaging MO sub tree addition parameters (TimerIdle)

- Values: <Timer value in seconds>
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.

- Associated HTTP XML parameter ID: “TimerIdle”

Node: /<x>/Messaging/Chat/MaxSize

Leaf node that represents the maximum size in bytes of a text Chat message that a user can enter in a 1-to-1 Chat or Group Chat session.

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get

Table 207: Messaging MO sub tree addition parameters (MaxSize)

- Values: < maximum size in bytes of a text Chat message that a user can enter>
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MaxSize”

Node: /<x>/Messaging/Chat/ChatRevokeTimer

Leaf node that represents the time the service provider allows to elapse after the client has sent the message and before Revoke Message request is automatically triggered by the client when it has not received the delivery notification for that message.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 208: Messaging MO sub tree addition parameters (ChatRevokeTimer)

- Values: <Timer value in seconds>
  When set to 0, the client is not able to send MessageRevoke requests
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ChatRevokeTimer”

Node: /<x>/Messaging/Chat/reconnectGuardTimer

Leaf node that provides the minimum time the client shall be registered in IMS prior to sending a message revocation request for a chat message.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 209: Messaging MO sub tree addition parameters (reconnectGuardTimer)

- Values: integer value defining the timeout to be used in seconds
  When set to 0, the client is not able to send MessageRevoke requests
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “reconnectGuardTimer”

Node: /<x>/Messaging/Chat/cfsTrigger

Leaf node that controls the client trigger to fallback to SMS when revocation procedures apply.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get

Table 210: Messaging MO sub tree addition parameters (cfsTrigger)

- Values:
  0 (default): the client shall fall back to SMS and send the Message Revoke request right after sending the SMS
  1: the client shall fall back to SMS right after receiving the MessageRevokeResponse request with the value of the result equal to “success”
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “cfsTrigger”

Node: /<x>/Messaging/Chat/im-spam-reporting-uri

Leaf node that represents the URI where spam reports should be sent.

It is required to be instantiated if a Service Provider supports spam reporting.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 211: Messaging MO sub tree addition parameters (im-spam-reporting-uri)

- Values: <The IM Spam reporting URI>
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “im-spam-reporting-uri”

Node: /<x>/Messaging/Chat/im-spam-reporting-file-content

Leaf node that represents whether File Content can be referred to in spam reports.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 212: Messaging MO sub tree addition parameters (im-spam-reporting-file-content)

- Values:
  0 (default): file content cannot be referred to in spam reported
  1: file content can be referred to in spam reported and can be uploaded to the content server is not available in the network in unencrypted format.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “im-spam-reporting-file-content”

Node: /<x>/Messaging/max1toManyRecipients

Leaf node that provides the maximum number of contacts allowed to be included in the distribution list of the 1-to-Many messaging service.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 213: Messaging MO sub tree addition parameters (max1ToManyRecipients)

- Values:
  0 (default): the 1-to-Many Messaging service is disabled
  1: the client can add unlimited number of recipients
  >1: integer value that indicates the maximum total number of recipients that can be included in the distribution list
- Post-reconfiguration actions:
  If the configuration parameter value transits from 0 to a value higher than zero, or the parameter is added with value higher than zero to the client configuration, the client shall unhide the UX elements for the management of the distribution lists.
  If the configuration parameter value transits from a value higher than 0 to 0 or the configuration parameter is removed, then the client shall hide the UX elements for the management of distribution lists. Any stored distribution lists shall be deleted.
  The new value of the configuration parameter shall be stored and applied from this time on.
- Associated HTTP XML parameter ID: “max1ToManyRecipients”

Node: /<x>/Messaging/1toManySelectedTech

Leaf node that allows selecting the 1-to-1 messaging technology to be used for the 1-to-Many messaging service.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 214: Messaging MO sub tree addition parameters (1toManySelectedTech)

- Values:
  0 (default): SMS is selected
  1: RCS 1-to-1 Chat service is selected
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “1toManySelectedTech”

Node: /<x>/Messaging/displayNotificationSwitch

Leaf node that controls whether the sending of Display Notification is enabled/disabled on the recipient’s client.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 215: Messaging MO sub tree addition parameters (displayNotificationSwitch)

- Values: <Timer value in seconds>
  0 (default), Enable sending Display Notifications, The user may still disable it using the Display Notification setting

1: Disable sending Display Notifications, The Display Notification setting is not available to the user.

- Post-reconfiguration actions: Start using the provided value the next time when receiving a message that requires Display Notification.
- Associated HTTP XML parameter ID: “displayNotificationSwitch”

Node: /<x>/Messaging/contentCompressSize

Leaf node that controls the size in bytes of the SIP request/response content to decide whether compressing the content or not.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 216: Messaging MO sub tree addition parameters (contentCompressSize)

- Values: <content compress size in bytes> - the size in bytes of the SIP request/response content to decide whether compressing the content or not.
- Post-reconfiguration actions:
  - If the value of configuration parameter transitions from absent to any value, the client is registered in IMS, and SIP Content-Compression is supported by the client, the client shall (re-)register in IMS to add the Accept-Encoding parameter set to GZIP in the SIP REGISTER request.
  - If the value of configuration parameter transitions from any value to absent, the client is registered in IMS, and SIP Content-Compression is supported by the client, the client shall (re-)register in IMS to remove the Accept-Encoding parameter set to GZIP in the SIP REGISTER request
- Associated HTTP XML parameter ID: “contentCompressSize”

Node: /<x>/Messaging/minimumCpimBodySizeForCompressionKb

Leaf node that controls the minimum CPIM size that should be compressed.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 217: Messaging MO sub tree addition parameters (minimumCpimBodySizeForCompressionKb)

- Values: <CPIM content compress size in kilobytes> - 0 and above where a value of zero indicates that all CPIM bodies should be compressed. When not provided, the RCS client may use its own default threshold.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “minimumCpimBodySizeForCompressionKb”

Node: /<x>/Messaging/signedEncryptionIdentityProof

Leaf node that contains the signed tuple necessary for encryption.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	String	Get, Replace

Table 218: Messaging MO sub tree addition parameters (signedEncryptionIdentityProof)

- Values: Base 64 encoded expiring signature of (MSISDN, Participant Public Key, Home KDS) tuple defined in [GSMA PRD-RCC.16]. The Auto-Configuration Server shall return this parameter if the client uploaded the Participant Public Key and the vendor ID of the Home KDS.
- Post-reconfiguration actions:
    - The client shall use the signedEncryptionIdentityProof as per [GSMA PRD-RCC.16]
- Associated HTTP XML parameter ID: “signedEncryptionIdentityProof”

Node: /<x>/Messaging/FileTransfer

Interior node where parameters related to the RCS File Transfer service are provided.

This node is not instantiated if the Service Provider does not enable File Transfer and is required to be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Node	Get

Table 219: Messaging MO sub tree addition File Transfer node

- Values: N/A
- Associated HTTP XML characteristic type: “FileTransfer”

Node: /<x>/Messaging/FileTransfer/ftWarnSize

Leaf node that provides a warning size limit in kilobyte to control the client behaviour for the handling of File Transfer and Rich Card media objects.

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get, Replace

Table 220: Messaging MO sub tree addition parameters (ftWarnSize)

- Values: The warning size in KB, if set to 0 then no warning size limit applies
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftWarnSize”

Node: /<x>/Messaging/FileTransfer/MaxSizeFileTr

Leaf node that represents the maximum authorised size of a file that can be sent using the RCS File Transfer service

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get

Table 221: Messaging MO sub tree addition parameters (MaxSizeFileTr)

- Values: The maximum file size threshold (in KB) or 0 to disable the limit
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MaxSizeFileTr”

Node: /<x>/Messaging/FileTransfer/ftAutAccept

Leaf node that provides the default for the user's setting for auto-acceptance for File Transfer and Rich Card media objects.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 222: Messaging MO sub tree addition parameters (ftAutAccept)

- Values:
  0, auto-acceptance is off
  1 (default), auto-accept is on
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftAutAccept”

Node: /<x>/Messaging/FileTransfer/ftHTTPCSURI

This parameter configures the URI of the HTTP Content Server where files are going to be uploaded on the originating side.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get, Replace

Table 223: Messaging MO sub tree addition parameters (ftHTTPCSURI)

- Values: URL of the Service Provider's File Transfer upload server
  Example:
  https://ftcontentserver.rcs. mnc001.mcc262.pup.3gppnetwork.org/path/?parm=foo
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftHTTPCSURI”

Node: /<x>/Messaging/FileTransfer/ftHTTPDLURI

This parameter provides the URL of Service Provider's File Transfer download server. If present, it enables the local file download for content URLs received via File Transfer or any other RCS messaging service.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	Chr	Get, Replace

Table 224: Messaging MO sub tree addition parameters (ftHTTPDLURI)

- Values: URL of the Service Provider's File Transfer download server
  Example:
  https://dl.rcs.mnc001.mcc262.pup.3gppnetwork.org/path?parm=foo
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftHTTPDLURI”

Node: /<x>/Messaging/FileTransfer/ftHTTPCSUser

This parameter is the value of the user value that shall be used to authenticate the RCS client trying to either get a root URL (HTTP GET request) or upload a file (HTTP post request).

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	No Get, No Copy

Table 225: Messaging MO sub tree addition parameters (ftHTTPCSUser)

- Values: The string containing **user value**.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftHTTPCSUser”

Node: /<x>/Messaging/FileTransfer/ftHTTPCSPwd

This parameter is the value of the password value that shall be used to authenticate the RCS client trying to either get a root URL (HTTP GET request) or upload a file (HTTP post request).

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	No Get, No Copy

Table 226: Messaging MO sub tree addition parameters (ftHTTPCSPwd)

- Values: The string containing **password value**.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftHTTPCSPwd”

Node: /<x>/Messaging/FileTransfer/ftHTTPFallback

Leaf node that describes the operator's default setting client switch to control the user dialog for File Transfer fallback to SMS.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get, Replace

Table 227: Messaging MO sub tree addition parameters (ftHTTPFallback)

- Values: 0, 1
  - 0, MMS is used (default)
  - 1, Text message with a link is used
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.

- Associated HTTP XML parameter ID: “ftHTTPFallback”

Node: /<x>/Messaging/FileTransfer/ftMax1ToManyRecipients

Leaf node that provides the maximum number of recipients allowed for a File Transfer to multiple recipients.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 228: Messaging MO sub tree addition parameters (ftMax1ToManyRecipients)

- Values:
  0 (default): the client can send a File Transfer to an unlimited number of recipients
  Positive integer value: indicates the maximum total number of recipients a File Transfer can be sent to.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “ftMax1ToManyRecipients”

Node: /<x>/Messaging/Chatbot

Interior node where there are filled parameters related to RCS Chatbot Functionality

This node is not instantiated if the Service Provider does not enable Chat.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	node	Get

Table 229: Messaging MO sub tree addition Chatbot node

- Values: N/A
- Associated HTTP XML characteristic type: “Chatbot”

Node: /<x>/Messaging/Chatbot/ChatbotDirectory

Leaf node that represents the URL address from which the client should retrieve the list of Chatbots.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 230: Messaging MO sub tree addition parameters (ChatbotDirectory)

- Values: the URL for retrieving the list for Chatbots provided by the Service Provider
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “ChatbotDirectory”

Node: /<x>/Messaging/Chatbot/ChatbotGallery

Leaf node that represents the URL address from which the RCS client should access the Chatbot Gallery.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 231: Messaging MO sub tree addition parameters (ChatbotGallery)

- Values: the URL pointing to the Chatbot Gallery provided by the Service Provider
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “ChatbotGallery”

Node: /<x>/Messaging/Chatbot/BotinfoFQDNRoot

Leaf node that represents the root part of the FQDN to be used by the client to compose the botinfo URL as defined in section 3.6.4.1.1.

Status	Occurrence	Format	Min. Access Types
Required	One	chr	Get

Table 232: Messaging MO sub tree addition parameters (BotinfoFQDNRoot)

- Values: string indicating the root part of the FQDN to be used by the client to compose the botinfo URL
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “BotinfoFQDNRoot”

Node: /<x>/Messaging/Chatbot/SpecificChatbotsList

Leaf node that represents the URL address from which the client should retrieve the lists with URIs of Chatbots requiring specific management.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 233: Messaging MO sub tree addition parameters (SpecificChatbotsList)

- Values: the URL for retrieving the list of Chatbots requiring specific management
- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “SpecificChatbotsList”

Node: /<x>/Messaging/Chatbot/IdentityInEnrichedSearch

Leaf node that determines whether the i query parameter defined in section 3.6.3.1 is included in the client to Service Provider Chatbot Directory requests when the user setting to enrich the search is enabled.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 234: Messaging MO sub tree addition parameters (IdentityInEnrichedSearch)

- Values: 0,1

  0- the i query parameter is not included (default value)
  1- the i query parameter is included

- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “IdentityInEnrichedSearch”

Node: /<x>/Messaging/Chatbot/PrivacyDisable

Leaf node that represents the authorisation for the user to request anonymization for a Chatbot session.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get

Table 235: Messaging MO sub tree addition parameters (PrivacyDisable)

- Values: 0,1

  0- Indicates that Anonymization is enabled (default value)
  1- Indicates that Anonymization is disabled

- Post-reconfiguration actions: no additional actions at the time of re-configuration.
- Associated HTTP XML parameter ID: “PrivacyDisable”

Node: /<x>/Messaging/Chatbot/ChatbotMsgTech

Leaf node that controls the messaging technology for Chatbot conversation from the client.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get

Table 236: Messaging MO sub tree addition parameters (ChatbotMsgTech)

- Values: 0,1,2,3

  0: Chatbot Services are disabled
  1 (default): Chatbot Services are enabled only with 1-to-1 Chatbot Chat Sessions
  2: Chatbot Services are enabled with both 1-to-1 Chatbot Chat Sessions and 1-to-1 Chatbot Standalone Messaging
  3: Chatbot Services are enabled only with 1-to-1 Chatbot Standalone Messaging

- Post-reconfiguration actions:

  - If the value transits from "0" or "3" to "1" or "2", then the client shall register or re-register in IMS to add the feature tags for Chatbot Communication with sessions and Chat defined in section 2.4.4, if conditions allow.
  - If the value transits from "1" or "2" to "0" or "3" and if the client is registered in IMS for Chatbot Communication with sessions as defined in section 2.4.4, then the client shall initiate an IMS de-registration or re-registration to remove the feature tags for Chatbot Communication with sessions defined in section 2.4.4. If the GROUP CHAT AUTH client configuration parameter is not set to 1, the client shall also remove the feature tag for Chat defined in section 2.4.4.
  - If the value transits from "0" or "1" to "2" or "3", then the client shall register or re-register in IMS to add the feature tags for Standalone Messaging and for Chatbot

Communication with Standalone Messaging defined in section 2.4.4, if conditions allow.

- If the value transits from "2" or "3" to "0" or "1" and if the client is registered in IMS for Chatbot Communication with Standalone Messaging as defined in section 2.4.4, then the client shall initiate an IMS de-registration or re-registration to remove the feature tags for Chatbot Communication with Standalone Messaging defined in section 2.4.4. If the STANDALONE MSG AUTH client configuration parameter is not set to 1, the client shall also remove the feature tags for Standalone Messaging defined in section 2.4.4.

- Associated HTTP XML parameter ID: “ChatbotMsgTech”

Node: /<x>/Messaging/MessageStore

Interior node where there are filled parameters related to RCS CPM Common Message Store

This node is not instantiated if the Service Provider does not provide the Common Message Store.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	node	Get

Table 237: Messaging MO sub tree addition Message Store node

- Values: N/A
- Associated HTTP XML characteristic type: “MessageStore”

Node: /<x>/Messaging/MessageStore/MsgStoreUrl

Leaf node that represents the URL address of the Message Store Server

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 238: Messaging MO sub tree addition parameters (MsgStoreUrl)

- Values: the URL for accessing the Message Store Server
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MsgStoreUrl”

Node: /<x>/Messaging/MessageStore/MsgStoreNotifUrl

Leaf node that represents the URL address of the Message Store notification server

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	Get

Table 239: Messaging MO sub tree addition parameters (MsgStoreNotifUrl)

- Values: the URL for accessing the Message Store notification server
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.

- Associated HTTP XML parameter ID: “MsgStoreNotifUrl”

Node: /<x>/Messaging/MessageStore/MsgStoreAuth

Leaf node indicating whether a Message Store server is available and whether HTTP Basic or other authentication mechanism requested by the Message Store server shall be used.

Status	Occurrence	Format	Min. Access Types
Required	One	int	Get

Table 240: Messaging MO sub tree addition parameters (MsgStoreAuth)

- Values: 0, 1, 2
  0- Indicates that the Message Store server is not enabled (default)
  1- Indicates that Message Store server is enabled and the client shall initiate HTTP Basic authentication using user name and password.
  2- Indicates that the Message Store server is enabled and the client shall perform the authentication mechanism requested by the Message Store server.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MsgStoreAuth”

Node: /<x>/Messaging/MessageStore/MsgStoreUserName

Optional leaf node that represents the User Identity information used by the Message Store Client to access the subscriber Message Store account on the Message Store Server

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	No Get, No Copy

Table 241: Messaging MO sub tree addition parameters (MsgStoreUserName)

- Values: <username assigned to the user for access to the Message Store Server>
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MsgStoreUserName”

Node: /<x>/Messaging/MessageStore/MsgStoreUserPwd

Optional leaf node that represents the user password associated to his/her user name identity information used by the Message Store Client to access the subscriber Message Store account on the Message Store Server

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	chr	No Get, No Copy

Table 242: Messaging MO sub tree addition parameters (MsgStoreUserPwd)

- Values: <password assigned to the user for access to the Message Store Server>
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MsgStoreUserPwd”

Node: /<x>/Messaging/MessageStore/EventRpting

Optional leaf node that can be used to inform the Message Store Client whether to directly set flags in the Message Store or whether to indicate to the Messaging Server that it should set flags in the Message Store on behalf of the client. If not instantiated, the Message Store Client SHALL assume the same method as if value 0 had been specified.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get

Table 243: Messaging MO sub tree addition parameters (EventRpting)

- Values: 0, 1
  0- Indicates that the client shall set flags in the Message Store as needed via an Message Store connection (default)
  1- Indicates that the client shall make use of the Event Reporting framework as described in section 4.1.16.4 and 4.1.16.5 when no Message Store connection exists so that the Messaging Server may set the flags in the Message Store on behalf of the client
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “EventRpting”

Node: /<x>/Messaging/MessageStore/AuthArchive

Optional leaf node that can be used to enable the Message Store Client to archive messages.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	bool	Get

Table 244: Messaging MO sub tree addition parameters (AuthArchive)

- Values: 0, 1
  0- Indicates that the archive service is disabled (default)
  1- Indicates that archive service is enabled and thus the client may archive messages.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “AuthArchive”

Node: /<x>/Messaging/MessageStore/SMSStore

Leaf node that describes whether the client is expected to store to the Message Store Server sent or received SMS.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 245: Messaging MO sub tree addition parameters (SMSStore)

- Values: This parameter can have 3 possible values:
  0- The device shall not store any sent or received SMS to the Message Store Server

1- The device shall store to the Message Store every sent and received SMS that cannot be correlated with the Common Message Store in the RCS Default folder
2- The device shall store every sent and received SMS and shall not attempt to correlate with the Common Message Store in the RCS Default folder.

- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “SMSStore”

Node: /<x>/Messaging/MessageStore/MMSStore

Leaf node that describes whether the client is expected to store to the Message Store Server sent or received MMS.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 246: Messaging MO sub tree addition parameters (MMSStore)

- Values: This parameter can have 3 possible values:
  0- The device shall not store any sent or received MMS to the Message Store Server
  1- The device shall store to the Message Store every sent and received MMS that cannot be correlated with the Common Message Store
  2- The device shall store every sent and received MMS and shall not attempt to correlate with the Common Message Store.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “MMSStore”

Node: /<x>/ Messaging/MIVC

Interior node where parameters related to MIVCs are provided.

This node is not instantiated if the Service Provider does not enable MIVCs and is required to be instantiated otherwise.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	node	Get

Table 247: Messaging MO sub tree addition Message Store node

- Values: N/A
- Associated HTTP XML characteristic type: “MIVC”

Node: /<x>/Messaging/MIVC/mivcTechnology

Leaf node that contains technology used for MIVCs.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 248: Messaging MO sub tree addition parameters (mivcTechnology)

- Values:

0: None
1: VILTE
2: ViLTE preferred, RCS Video Chat fallback (default)
3: RCS Video Chat

- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “mivcTechnology”

Node: /<x>/Messaging/MIVC/rcsVideoChatTurn

Leaf node that contains whether it is mandatory to use the TURN servers provided by the RCS Service Provider (in the TurnCredentials).

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 249: Messaging MO sub tree addition parameters (rcsVideoChatTurn)

- Values:

0: Not mandatory (default)
1: mandatory

- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “rcsVideoChatTurn”

Node: /<x>/MIVC/TurnCredentials

Interior node where parameters related to Turn Credentials are provided.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrMore	node	Get

Table 250: Messaging MO sub tree addition Turn Credentials node

- Values: N/A
- Associated HTTP XML characteristic type: “TurnCredentials”

Node: /<x>/Messaging/MIVC/TurnCredentials/rcsVideoChatTurnUri

Leaf node that contains the URI, as per [RFC7065], of the TURN server to be used by the RCS clients for RCS Video Chats.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrMore	String	Get, Replace

Table 251: Messaging MO sub tree addition parameters (rcsVideoChatTurnUri)

- Values: Uri for the TURN server, as per [RFC7065]
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “rcsVideoChatTurnUri”

Node: /<x>/Messaging/MIVC/TurnCredentials/rcsVideoChatTurnUsername

Leaf node that contains the username to authenticate to the TURN server for RCS Video Chats.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	String	Get, Replace

Table 252: Messaging MO sub tree addition parameters (rcsVideoChatTurnUsername)

- Values: username for the TURN server
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “rcsVideoChatTurnUsername”

Node: /<x>/Messaging/MIVC/TurnCredentials/rcsVideoChatTurnPassword

Leaf node that contains the password to authenticate to the TURN server for RCS Video Chats.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	String	Get, Replace

Table 253: Messaging MO sub tree addition parameters (rcsVideoChatTurnPassword)

- Values: password for the TURN server
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “rcsVideoChatTurnPassword”

Node: /<x>/Messaging/MIVC/mivcReportingUri

Leaf node that contains the SIP URI where MIVC event notifications should be sent to.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	String	Get, Replace

Table 254: Messaging MO sub tree addition parameters (mivcReportingUri)

- Values: SIP URI to send MIVC Event Notifications to
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “mivcReportingUri”

Node: /<x>/Messaging/Ext

An extension node for Service Provider specific parameters. Clients that are not aware of any extensions in this subtree (e.g. because they are not Service Provider specific) should not instantiate this tree.

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	node	Get

Table 255: Messaging MO sub tree addition Service Provider Extension Node

- Values: N/A
- Associated HTTP XML characteristic type: “Ext”

A.2.5 Capability Discovery MO sub tree

This RCS specification includes the following additions as a new configuration sub tree, the Capability Discovery MO sub tree. Please note this sub tree is not included in any other specifications. So no other nodes from those specifications need to be added:

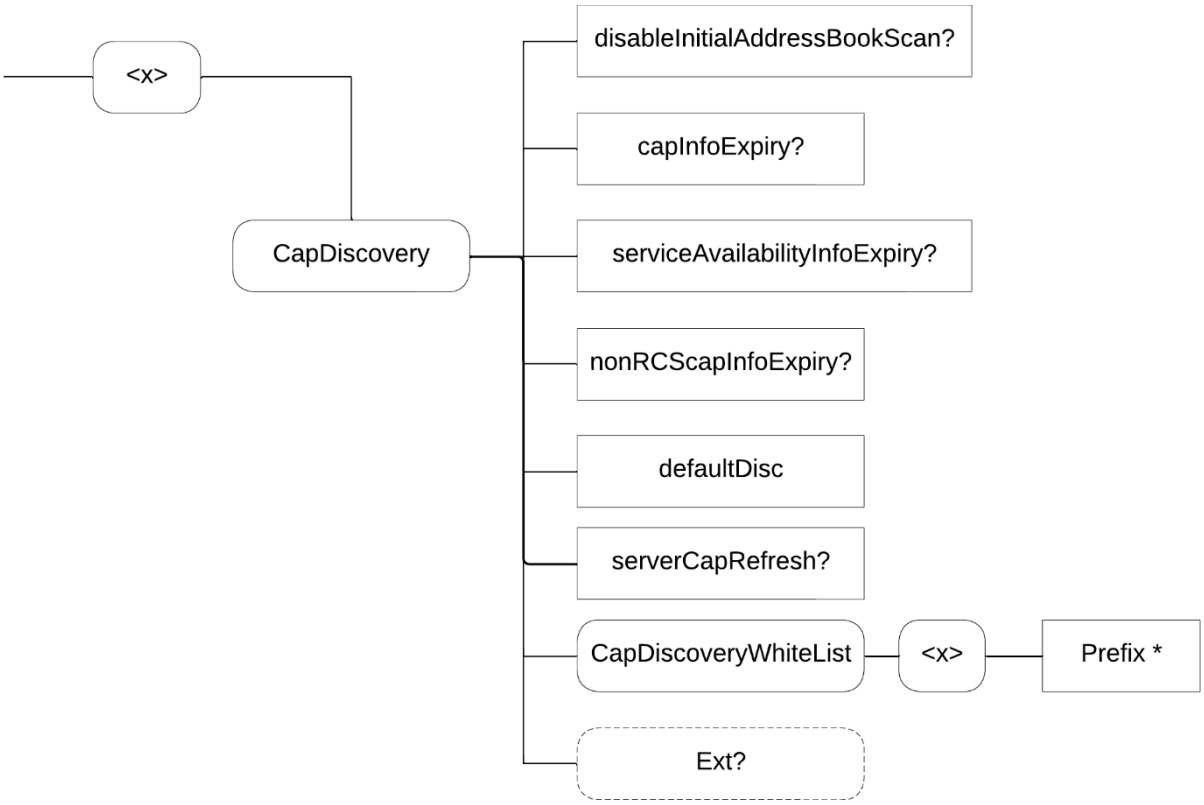


Figure 68: RCS additions, Capability Discovery sub tree

The associated HTTP configuration XML structure is presented in the table below:

```
<characteristic type="CAPDISCOVERY">
        <parm name="disableInitialAddressBookScan" value="X"/>
        <parm name="capInfoExpiry" value="X"/>
        <parm name="nonRCScapInfoExpiry" value="X"/>
        <parm name="serviceAvailabilityInfoExpiry" value="X"/>
        <parm name=”serverCapRefresh” value=”X”/>
        <characteristic type="CapDiscoveryWhitelist">
              <characteristic type="CapDiscoveryAllowedPrefixes">
                    <parm name="Prefix1" value="X"/>
                    <parm name="Prefix2" value="X"/>
                    <parm name="Prefix3" value="X"/>
                    …
              </characteristic>
        </characteristic>
        <characteristic type="Ext"/>
</characteristic>
```

Table 256 : Capability Discovery sub tree associated HTTP configuration XML structure

Node: /<x>/CapDiscovery

Under this interior node the RCS parameters related to Capability Discovery are placed

Status	Occurrence	Format	Min. Access Types
Required	One	node	Get

Table 257: Capability Discovery MO sub tree addition Capability Discovery node

- Values: N/A
- Associated HTTP XML characteristic type: “CAPDISCOVERY”

Node: /<x>/CapDiscovery/disableInitialAddressBookScan

Leaf node that describes whether the device/client should when it is first started, perform a Capability Discovery for all contacts in the address book

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 258: Capability Discovery MO sub tree addition parameters (disableInitialAddressBookScan)

- Values:
  0, The scan of the address book shall be done
  1, The scan of the address book shall not be done.
- Post-reconfiguration actions: if changed from 1 to 0 and no prior address book scan was done, the client shall initiate an address book scan.
- Associated HTTP XML parameter ID: “disableInitialAddressBookScan”

Node: /<x>/CapDiscovery/capInfoExpiry

Leaf node that describes the validity of the capability information cached in the terminal

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 259: Capability Discovery MO sub tree addition parameters (capInfoExpiry)

- Values: The validity time in seconds, 0 indicates that there is no expiry.
- Post-reconfiguration actions:
  - When changing from a positive value to 0, the already cached capability information shall be considered to never expire
  - When changing from 0 to a positive value, the client shall consider the expiry of the currently cached capabilities to be the newly configured value and start monitoring from the moment of reconfiguration
  - When increasing the value, the newly configured expiry value shall be applied to newly cached capabilities only

  - When decreasing the value, the client shall consider the expiry of the currently cached capabilities to be the newly configured value and start monitoring from the moment of reconfiguration

- Associated HTTP XML parameter ID: “capInfoExpiry”

Node: /<x>/CapDiscovery/serviceAvailabilityInfoExpiry

Leaf node that describes the validity of the availability information cached in the terminal in seconds.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 260: Capability Discovery MO sub tree addition parameters (serviceAvailabilityInfoExpiry)

- Values: The time in seconds
- Post-reconfiguration actions:
  - When increasing the value, the newly configured expiry value shall be applied to newly cached availability information only
  - When decreasing the value, the client shall consider the expiry of the currently cached capabilities to be the new value and start monitoring from the moment of reconfiguration
- Associated HTTP XML parameter ID: “serviceAvailabilityInfoExpiry”

Node: /<x>/CapDiscovery/nonRCScapInfoExpiry

Leaf node that describes how long a non RCS contact shall be prevented from being queried for its capabilities.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 261: Capability Discovery MO sub tree addition parameters (nonRCScapInfoExpiry)

- Values: The time in seconds.
- Post-reconfiguration actions: No additional actions at the time of reconfiguration.
- Associated HTTP XML parameter ID: “nonRCScapInfoExpiry”

Node: /<x>/CapDiscovery/defaultDisc

Leaf node that describes the default Capability and new user Discovery mechanism used by the terminal (Presence or Options).

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 262: Capability Discovery MO sub tree addition parameters (defaultDisc)

- Values:
  0, the default mechanism employed for Capability Discovery and new users will be OPTIONS.
  1, the default mechanism employed for Capability Discovery and new users will be Presence
  2, the mechanism employed for Capability Discovery will be disabled.
- Post-reconfiguration actions: When the value is changed from 0 or 2 to 1, the client shall also publish its capabilities as defined in section 2.6.1.2.2.
- Associated HTTP XML parameter ID: “defaultDisc”

Node: /<x>/CapDiscovery/serverCapRefresh

Leaf node that describes when a client relying on the Push Notification Mechanism should register to allow the network to refresh its cached capabilities.

The node shall be provided only if the Push Notification Mechanism is enabled.

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrOne	int	Get, Replace

Table 263: Capability Discovery MO sub tree addition parameters (serverCapRefresh)

- Values: The time in seconds
- Post-reconfiguration actions: When increasing the value, the newly configured expiry value shall be applied from the next (re-)registration onwards. When decreasing the value, an RCS client in the Push Control Idle state shall re-register with the network and follow the procedures in section 2.15.4 using the newly provided value.
- Associated HTTP XML parameter ID: “serverCapRefresh”

Node: /<x>/CapDiscovery/CapDiscoveryWhiteList

A Placeholder interior node for the Capability Discovery white list configuration

Status	Occurrence	Format	Min. Access Types
Required	One	node	Get

Table 264: Capability Discovery MO sub tree addition Capability Discovery White List node

- Values: N/A
- Associated HTTP XML characteristic type: “CapDiscoveryWhiteList”

Node: /<x>/CapDiscovery/CapDiscoveryWhiteList/<x>

A Placeholder interior node where to place 1 or more Prefix leaf nodes

Status	Occurrence	Format	Min. Access Types
Required	One	node	Get

Table 265: Capability Discovery MO sub tree addition CapDiscoveryAllowedPrefixes node

- Values: N/A
- Associated HTTP XML characteristic type: “CapDiscoveryAllowedPrefixes”

Node: /<x>/CapDiscovery/CapDiscoveryWhiteList/<x>/Prefix

Leaf node that represents a prefix configured by the Service Provider

Status	Occurrence	Format	Min. Access Types
Required	ZeroOrMore	chr	No Get, No Copy

Table 266: Capability Discovery MO sub tree addition parameters (Prefix)

- Values:
  The value can contain either a single prefix or a single regular expression for matching with phone numbers. If the phone number matches the prefix or the regular expression, then the phone number shall be considered for Capability and new user Discovery. If the phone number does not match the prefix or regular expression, then the client shall match the phone number with the value of the next configuration parameter in the Capability Discovery white list. If the phone number matches with none of the values of the "Prefix" configuration parameters in the Capability Discovery white list, then the phone number shall not be considered for Capability and new user Discovery.

  To match a phone number with prefixes and regular expressions, the client shall remove visual separators and white space from the input phone number string.

  - The configuration parameter contains a prefix if the value consists of a number string, optionally preceded by a "+" character. The client shall match the phone number and the prefix contained in the configuration parameter by string match. The phone number matches, if there is a full match of the prefix with the beginning of the phone number string.
    Examples:
    +446
    +4479
    00446
    004479
    06
    079
  - The configuration parameter contains a regular expression if the value starts with the "!" character. The subsequent string shall be interpreted by the client using Portable Operating System Interface (POSIX) extended regular expression (see [POSIX]). The phone number matches, if the application of the regular expression results in a non-empty string.
    Examples:
    !^(0044|0)(6|79)
    !\+44(6|79)\d*

- Post-reconfiguration actions: If a Prefix is no longer included in the list, the client shall remove capability and service availability information that is stored for contacts whose phone number matches the removed prefix from the cache.
- Associated HTTP XML parameter ID: “Prefix<X>” where <X> is a positive integer value

Node: /<x>/CapDiscovery/Ext

An extension node for Service Provider specific parameters. Clients that are not aware of any extensions in this subtree (e.g. because they are not Service Provider specific) should not instantiate this tree.

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	node	Get

Table 267: Capability Discovery MO sub tree addition Service Provider Extension Node

- Values: N/A
- Associated HTTP XML characteristic type: “Ext”

A.2.6 Service Provider Extensions MO sub tree

This RCS specification includes the following additions as a new and optional configuration sub tree, the Service Provider extensions MO sub tree. This tree should not instantiate by clients that are not aware of any extensions in this tree. Please note this sub tree is not included in any other specifications. So no other nodes from those specifications need to be added:

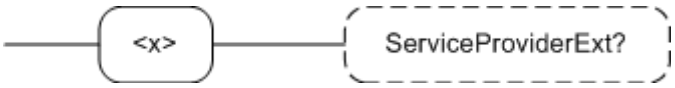


Figure 69: RCS additions, Service Provider Extensions sub tree

The associated HTTP configuration XML structure is presented in the table below:

```
<characteristic type="SERVICEPROVIDEREXT"/>
```

Table 268: Service Provider Extensions sub tree associated HTTP configuration XML structure

Node: /<X>/ServiceProviderExt

Under this interior node the RCS parameters related to Service Provider specific extensions are placed

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	node	Get

Table 269: Service Provider Extensions MO sub tree addition node

- Values: N/A
- Associated HTTP XML characteristic type: “SERVICEPROVIDEREXT”

A.3 Other Management Objects

A.3.1 Overview

The configuration XML document for RCS services includes configuration parameters from other than the Management Object defined in this document. The following table provides an overview of other Management Objects applicable for RCS services.

Management Object	Application Characteristic Reference	AppID Value	RCS Usage
OMA Management Object for Presence SIMPLE 2.0 [PRESENCE2MO]	Embedded in RCS Application characteristic as defined in section A.2.3	n/a	Section A.1.2.1
3GPP IMS Management Object [3GPP TS 24.167]	[GSMA PRD-RCC.15]	urn:oma:mo:ext-3gpp-ims:1.0	Section A.1.6.1
Non-Access Stratum (NAS) configuration Management Object [3GPP TS 24.368]	NAS configuration MO DDF as defined in [3GPP TS 24.368] via transformation as defined in Annex A of [GSMA PRD-RCC.14]	urn:oma:mo:ext-3gpp-nas-config:1.0	Section A.1.14.2
Enriched Calling	[GSMA PRD-RCC.20]	ap2007	See [GSMA PRD-RCC.20]
Capability Discovery, (as used for Enriched Calling and other services)	[GSMA PRD-RCC.20]	ap2008	See [GSMA PRD-RCC.20]

Table 270: Other Management Objects applicable for RCS

NOTE: For an overview of other Management Objects applicable for Multimedia Telephony and SMS over IP refer to [GSMA PRD-NG.114], [GSMA PRD-IR.92], [GSMA PRD-IR.51] and [GSMA PRD-IR.94].

A.3.2 IMS sub tree additions

The IMS Core network configuration for RCS shall be provided in a configuration XML document by use of the IMS MO defined in [3GPP TS 24.167] via the provisioning document defined in [GSMA PRD-RCC.15].

Additional parameters applicable for RCS have been defined for the IMS MO in [GSMA PRD-RCC.15].

This section extends the IMS MO defined in [3GPP TS 24.167] with additional RCS specific parameters that are added to the existing IMS MO extension tree defined in [GSMA PRD-RCC.15].

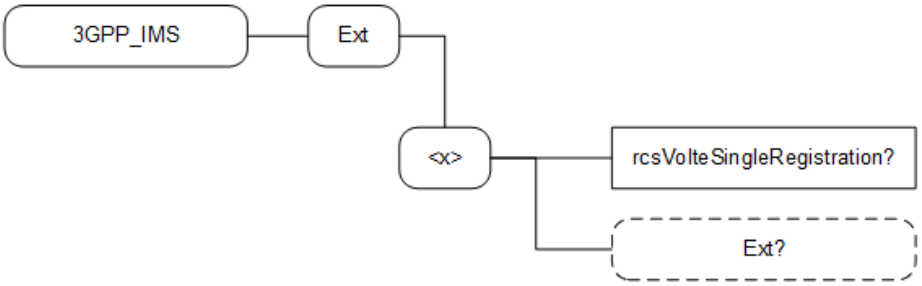


Figure 70: Additions to the IMS MO sub tree

Following parameters have been defined:

Node: <x>/rcsVolteSingleRegistration

Where <x> corresponds to the <x> node below the Ext node of the IMS sub tree defined in [GSMA PRD-RCC.15].

Leaf node that describes the behaviour regarding the instantiation of the IMS stack in devices supporting all RCS services (i.e. including Multimedia Telephony and SMSoIP). It is also used to control the APN selection on devices that are not enabled for Multimedia Telephony as described in section 2.8.1.4.

Status	Occurrence	Format	Min. Access Types
Optional	ZeroOrOne	int	Get, Replace

Table 271: IMS Configuration sub tree addition parameters (rcsVolteSingleRegistration)

- Values:
  - 0, the device shall follow a dual registration approach (transition solution) where RCS services other than Multimedia Telephony and SMSoIP use a separate registration from the Multimedia Telephony one.
  - 1 (default if not provided), the device shall follow a single registration (target solution) for all RCS services (i.e. including Multimedia Telephony and SMSoIP) services.
  - 2, the device shall follow a single registration for all RCS services (i.e. including Multimedia Telephony and SMSoIP) when in the home network or on EPC-integrated Wi-Fi, and shall follow a dual registration when roaming (transition solution).

- Post-reconfiguration actions: If the value of the configuration parameter changes, then the device shall de-register RCS services from IMS, establish bearers if required for the new value of the configuration parameter and shall register or re-register in IMS to update registration for RCS services in accordance with the new value of the configuration parameter and release bearers from the old configuration if no longer needed.
- Associated HTTP XML characteristic type: “rcsVolteSingleRegistration”

The non-normative provisioning document structure for the IMS MO extension is presented in the table below

```
<characteristic type="3GPP_IMS">
        …
        <characteristic type="Ext">
            <charcteristic type="GSMA">
                …
                <parm name="rcsVolteSingleRegistration" value="X"/>
                ...
                <characteristic type="Ext"/>
            </characteristic>
        </characteristic>
</characteristic>
```

Table 272: Provisioning document structure of the IMS MO extension

A.4 Configuration XML document structure

A.4.1 HTTP configuration XML structure

The provisioning documents applicable for RCS are embedded in a configuration XML document as depicted in the non-normative document structure shown in Table 273.

```
<?xml version="1.0"?>
<wap-provisioningdoc version="1.1">
        <characteristic type="VERS">
                <parm name="version" value="1"/>
                <parm name="validity" value="1728000"/>
        </characteristic>
        <characteristic type="TOKEN">                   <!-- This section is OPTIONAL -->
                <parm name="token" value="X"/>
        </characteristic>
        <!-- Potentially optional characteristics e.g. MSG, User and Access Control, see [GSMA PRD-
        RCC.14]-->

        <characteristic type="APPLICATION">
                <parm name=”AppID” value=”urn:oma:mo:ext-3gpp-ims:1.0”/>
                <characteristic type="3GPP_IMS">
                        <parm name="AppID" value="ap2001"/>
                        <parm name="Name" value="RCS IMS Settings"/> <!-- This section applies for SIP
                        transport only -->
                        …       <!-- see [GSMA PRD-RCC.15] -->
                         <characteristic type="Ext">
                                <characteristic type="GSMA">
                                        <parm name=”AppRef” value=”IMS-Setting”/>
                                        <parm name="rcsVolteSingleRegistration" value="X"/>
                                        …       <!-- see section [GSMA PRD-RCC.15] -->
                                </characteristic>
                        </characteristic>
                </characteristic>
        </characteristic>

        <characteristic type="APPLICATION">
                <parm name=”AppID” value=” urn:oma:mo:ext-3gpp-nas-config:1.0”/>
                <characteristic type="NODE">
                        … <!-- See [3GPP TS 24.368] -->
                </characteristic>
        </characteristic>

        <characteristic type="APPLICATION">
                <parm name="AppID" value="ap2002"/>
                <parm name="To-AppRef" value="IMS-Setting"/>
                <characteristic type="SERVICES">
                        …       -- See section A.2.2
                </characteristic>
                <characteristic type="PRESENCE">
                        …       -- See section A.2.3
                </characteristic>
                <characteristic type="MESSAGING">
                        …       -- See section A.2.4
                </characteristic>
                <characteristic type="CAPDISCOVERY">
                        …       -- See section A.2.5
```

```
            </characteristic>
            <characteristic type="SERVICEPROVIDEREXT">
                  …          -- See section A.2.6
            </characteristic>
            <characteristic type="gRPC">                  <!-- This section applies for gRPC transport only -->
              <!—see [GSMA PRD-RCC.30] -->
            </characteristic>
      </characteristic>
      <characteristic type="APPLICATION">
            <parm name="APPID" value="ap2007"/>
            <parm name="TO-APPREF" value="IMS-Setting"/>
            …  -- See [GSMA PRD-RCC.20]
      </characteristic>
      <characteristic type="APPLICATION">
            <parm name="APPID" value="ap2008"/>
            <parm name="TO-APPREF" value="IMS-Setting"/>
            …  -- See [GSMA PRD-RCC.20]
      </characteristic>
</wap-provisioningdoc>
```

Table 273: Complete configuration XML structure

A.4.2 Void

Annex B Additional diagrams

B.1 Chat and store and forward diagrams

B.1.1 Store and forward: Receiver offline

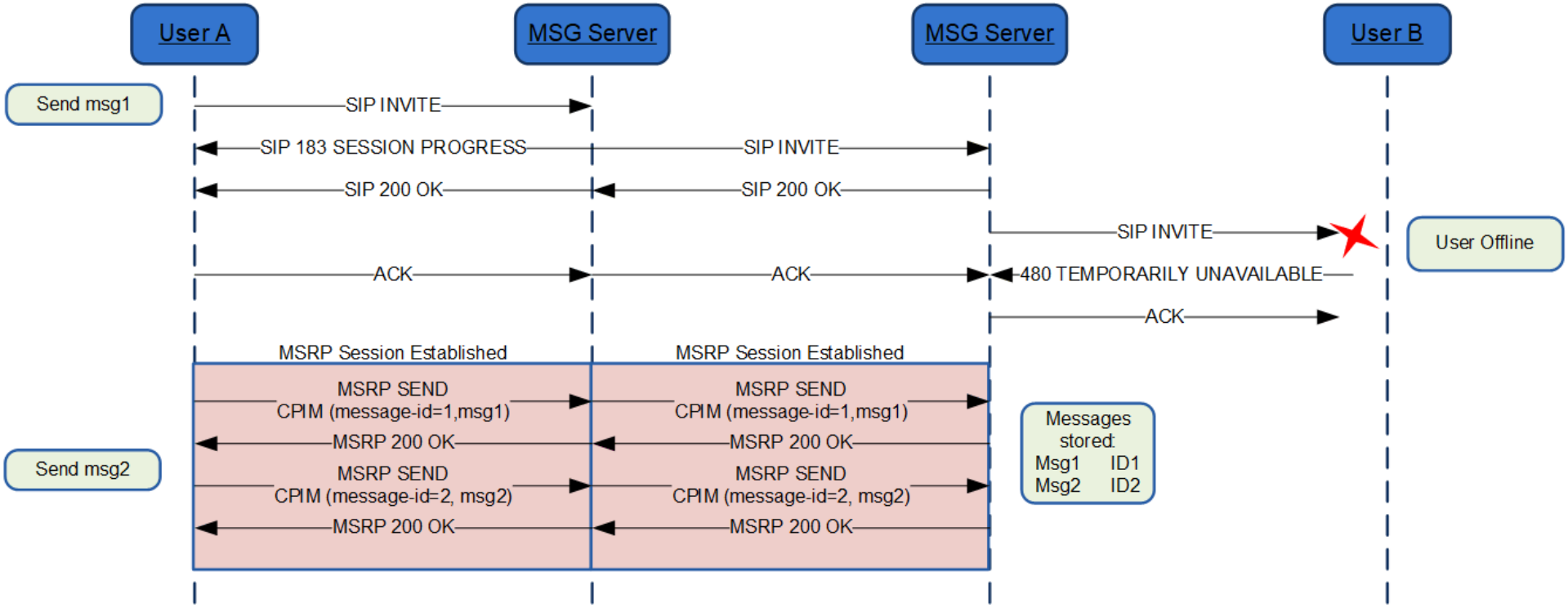


Figure 71: Store and forward: Receiver offline*

*: Check NOTES 1, 5 and 8 in section B.1.11.

B.1.2 Store and forward: Message deferred delivery with sender still on an active Chat session

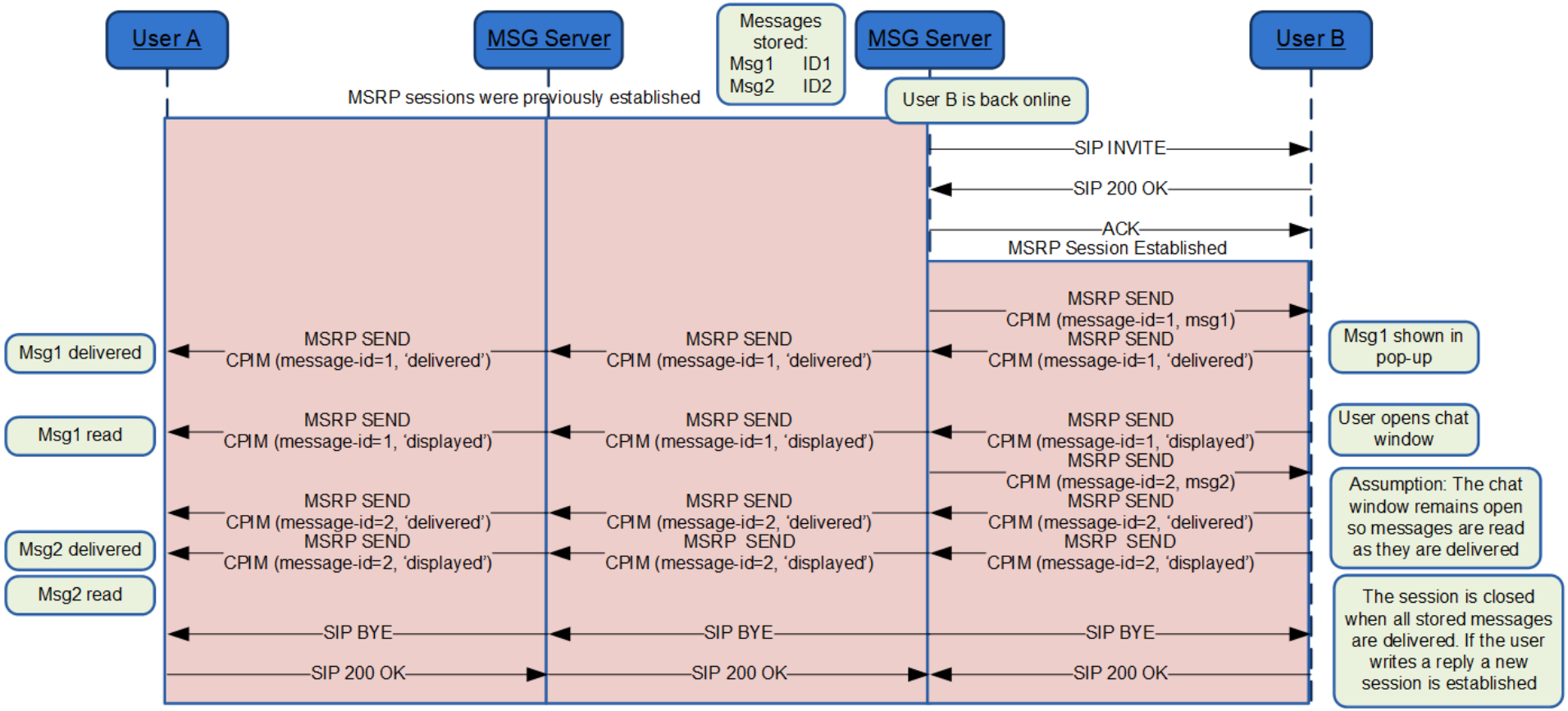


Figure 72: Store and forward: Message(s) deferred delivery with a sender still on an MSRP session*

*: Check NOTES 1, 2, 3, 6, 7 and 8 in section B.1.11

B.1.3 Store and forward: Message deferred delivery with sender online

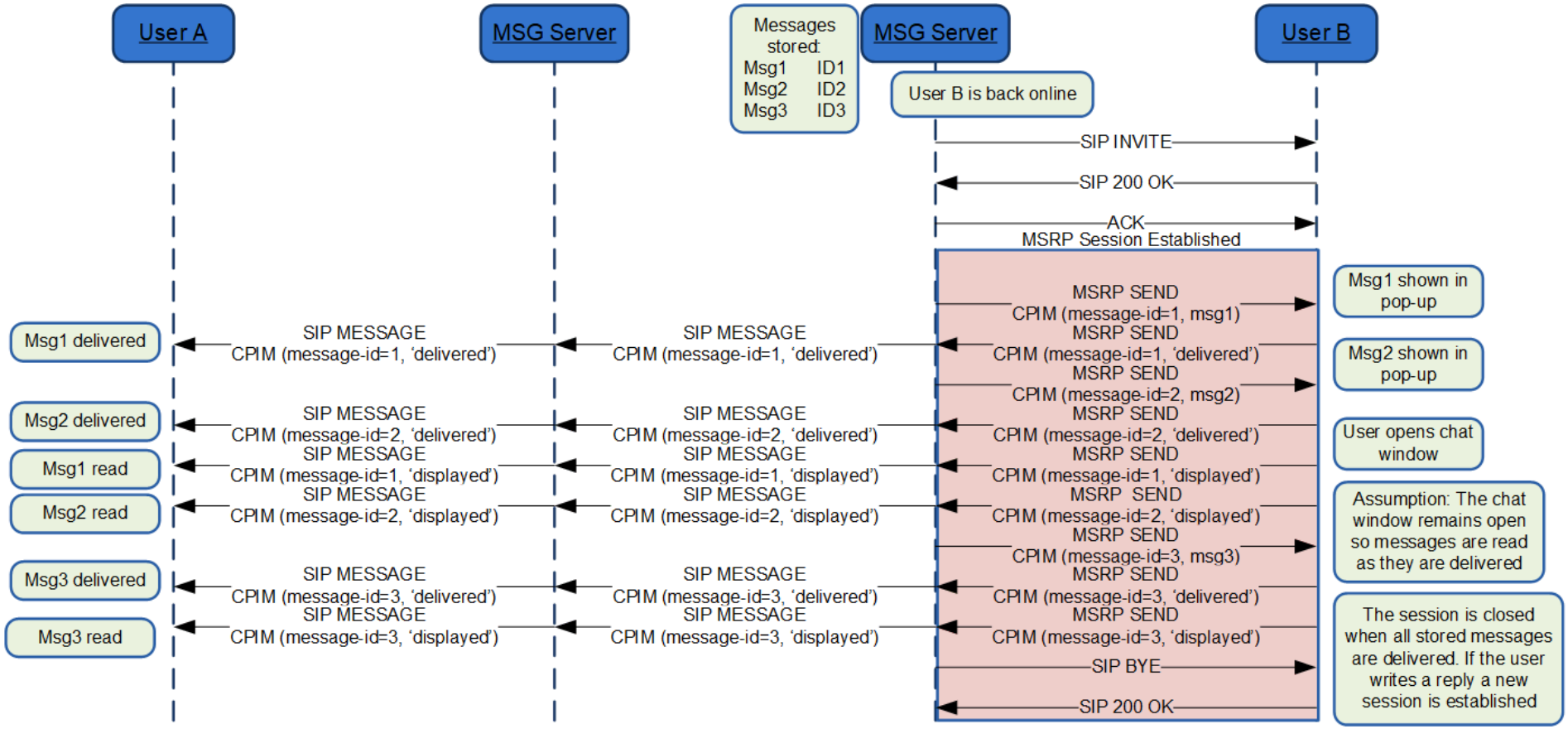


Figure 73: Store and forward: Message deferred delivery with sender online *

*: Check NOTES 1, 2, 3, 6, 7 and 8 in section B.1.11

B.1.4 Store and forward: Message deferred delivery with sender offline (delivery notifications)

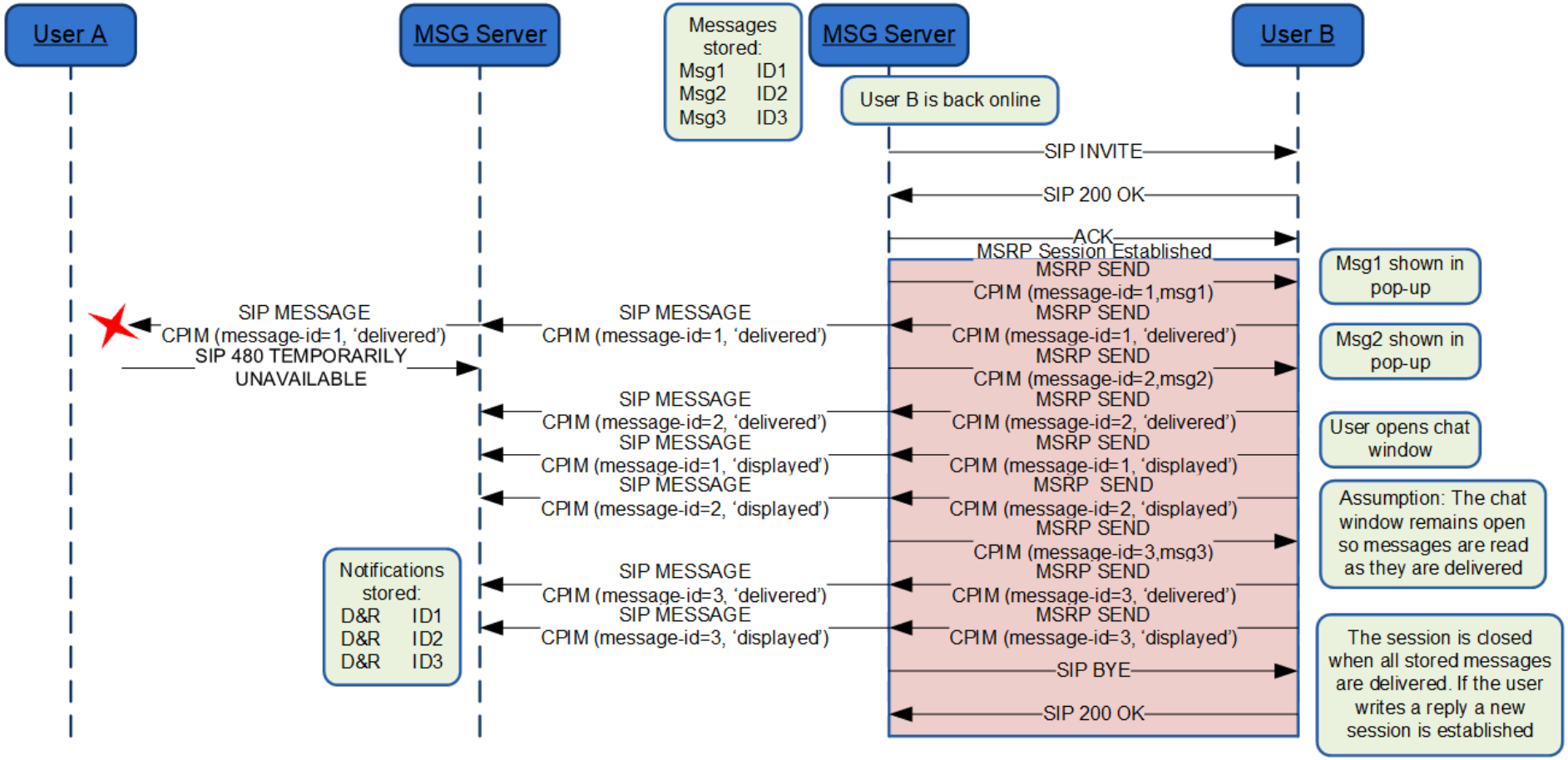


Figure 74: Store and forward: Message(s) deferred delivery with a sender offline (delivery notifications) *

*: Check NOTE 1, 4, 6, 7 and 8 in section B.1.11

B.1.5 Store and forward: Notifications deferred delivery

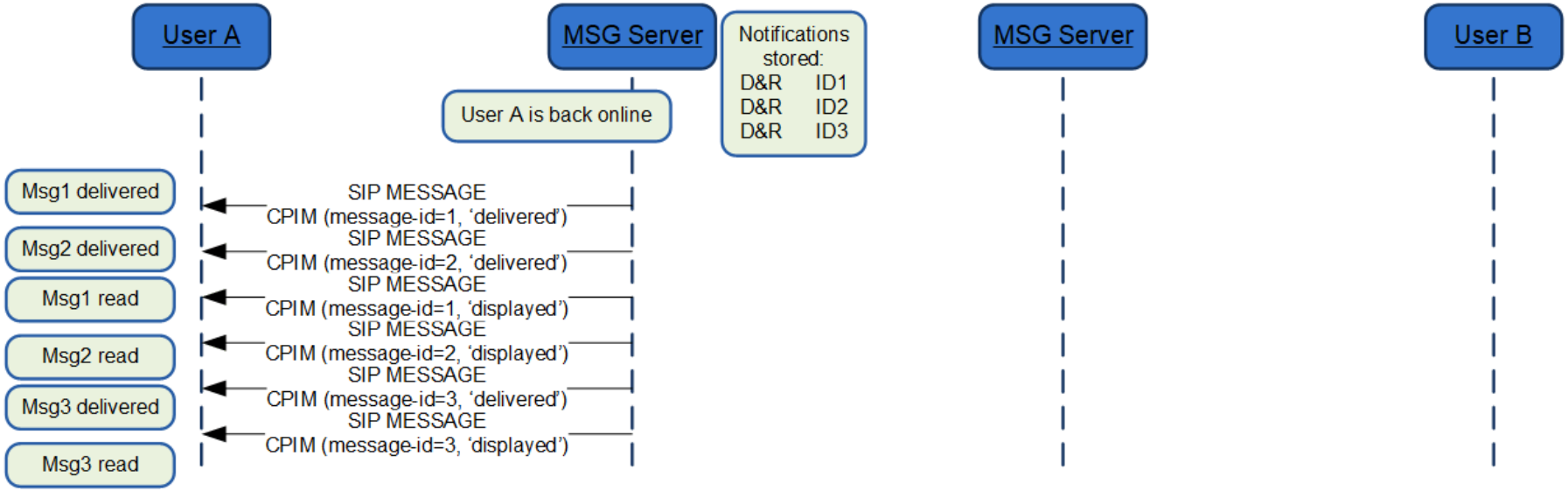


Figure 75: Store and forward: Notification(s) deferred delivery*

*: Check NOTES 1 and 8 in section B.1.11.

B.1.6 Network Interworking to SMS/MMS

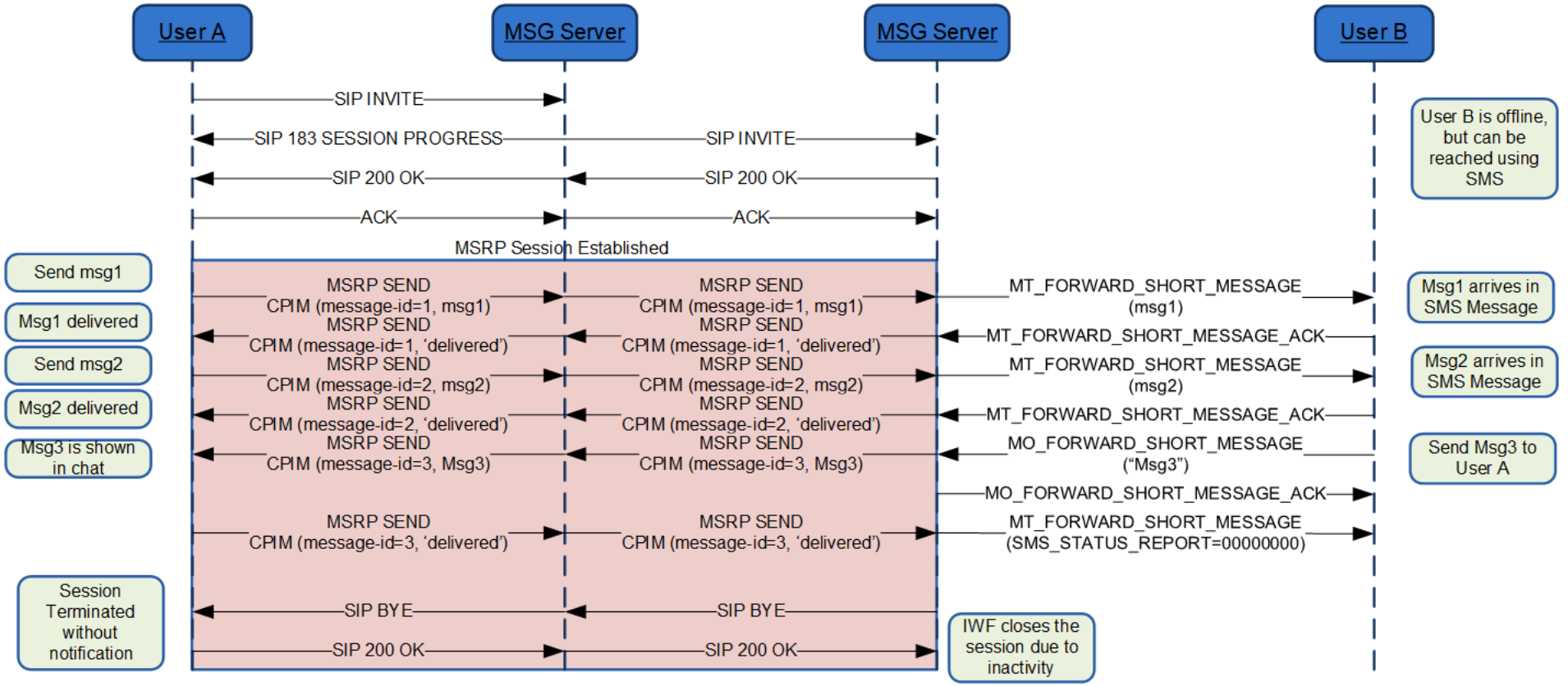


Figure 76: Interworking: Automatic acceptance on behalf of the SMS/MMS user*

*: Check NOTES 1, 8 and 9 in section B.1.11.

B.1.7 Message Revoke: Successful Request

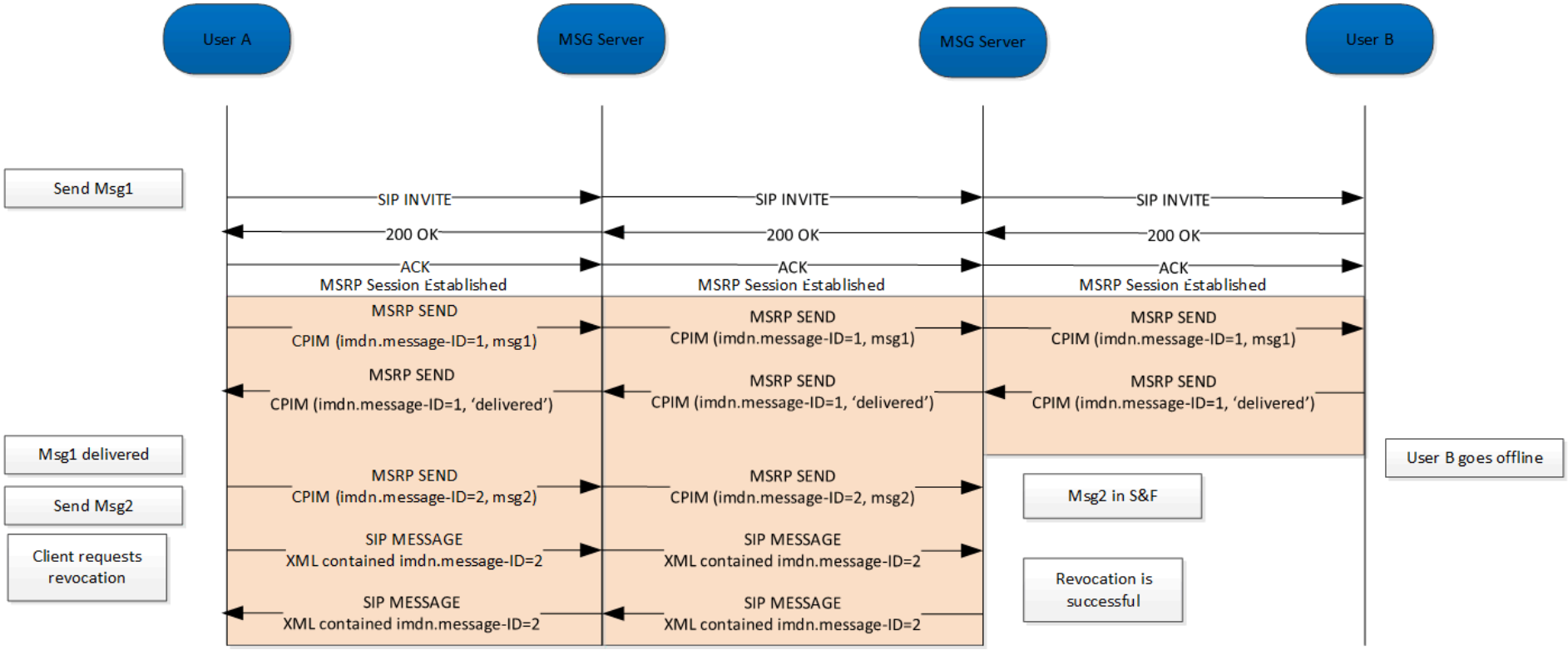


Figure 77: Message Revoke, Successful request*

*: Check NOTES 1 and 8 in section B.1.11

B.1.8 Message Revoke: Failed Request

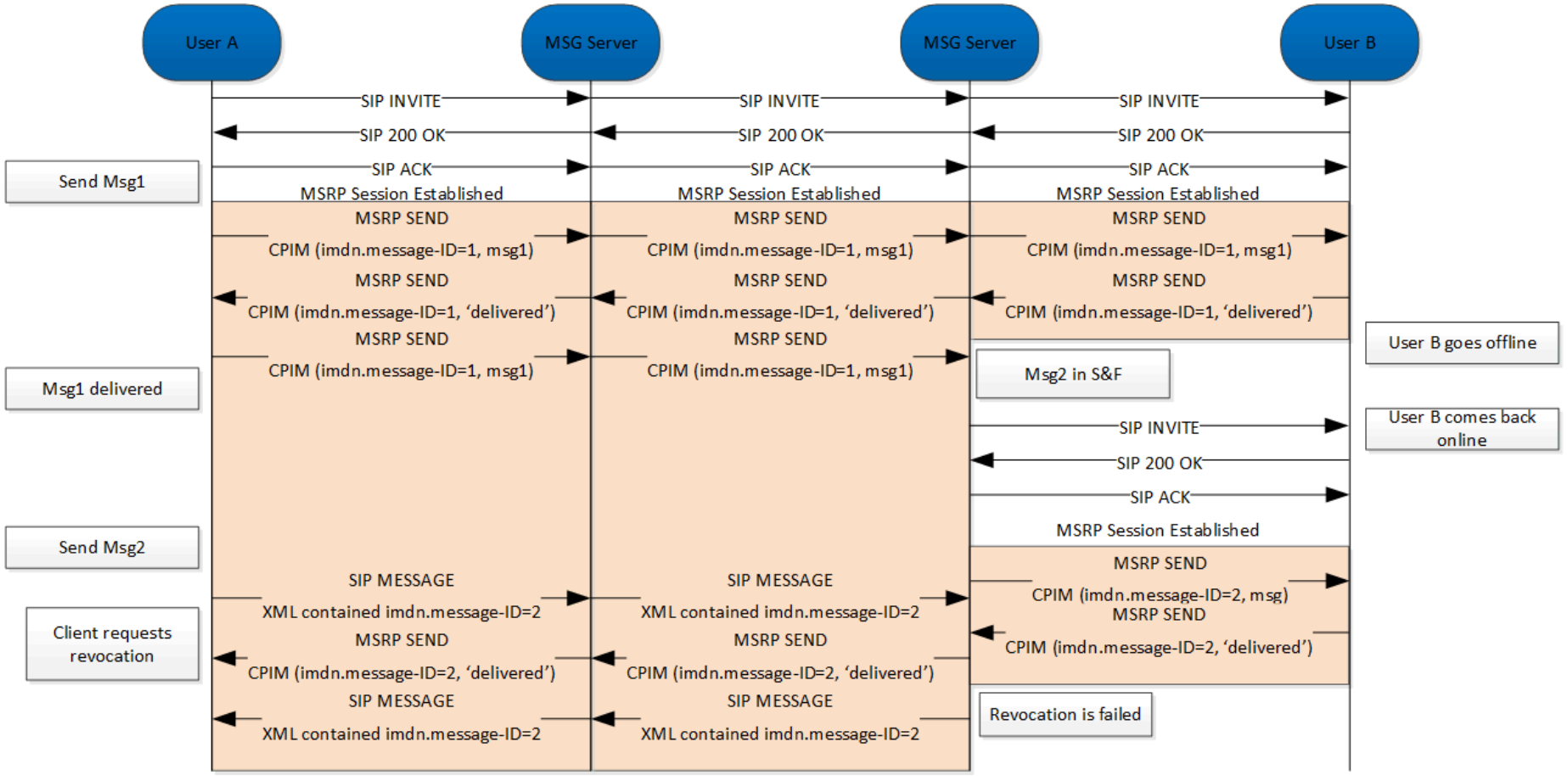


Figure 78: Message Revoke, Failed request*

*: Check NOTES 1 and 8 in section B.1.11

B.1.9 Deliver Stored Group Chat Messages while Chat is idle

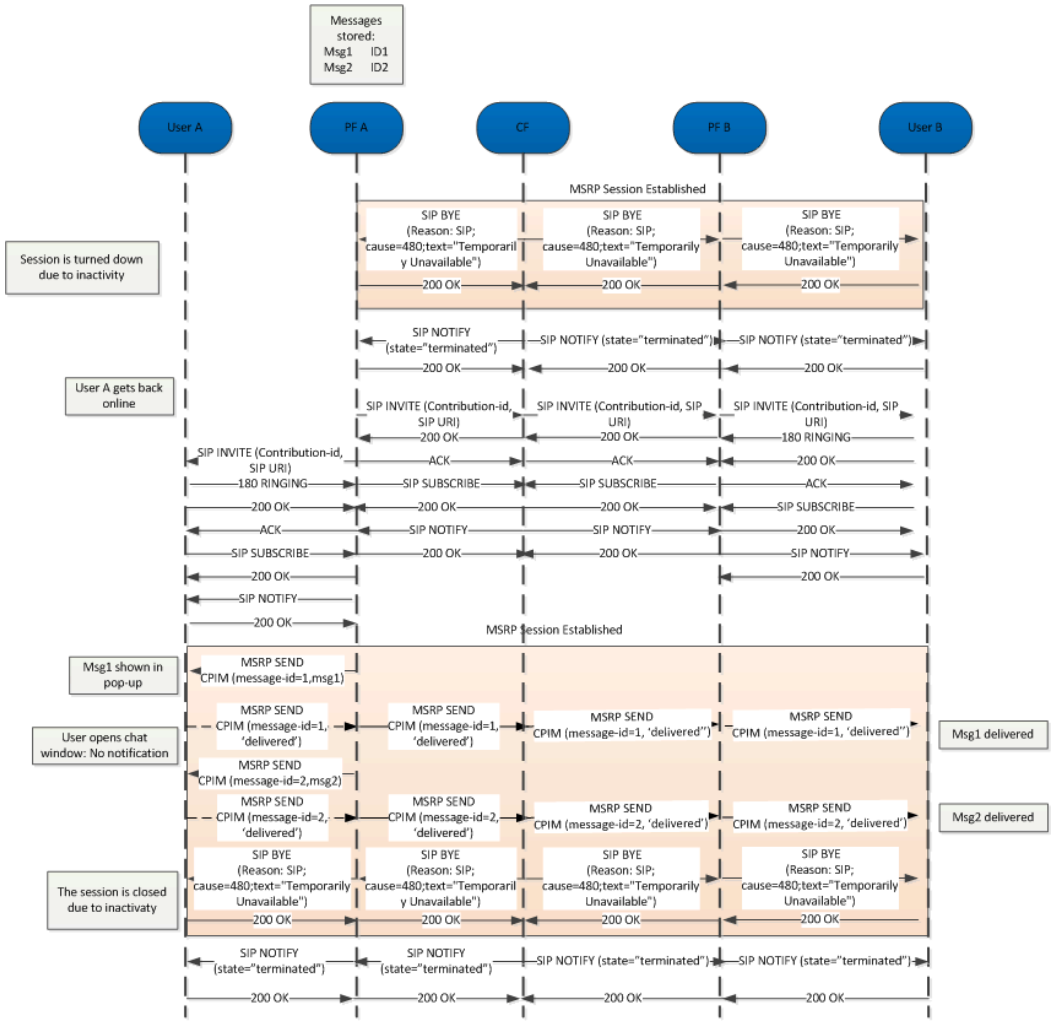


Figure 79: Deliver Group Chat Messages while Chat is idle*

*: Check NOTES 1, 8, 10, 11 and 12 in section B.1.11.

B.1.10 Multi-device

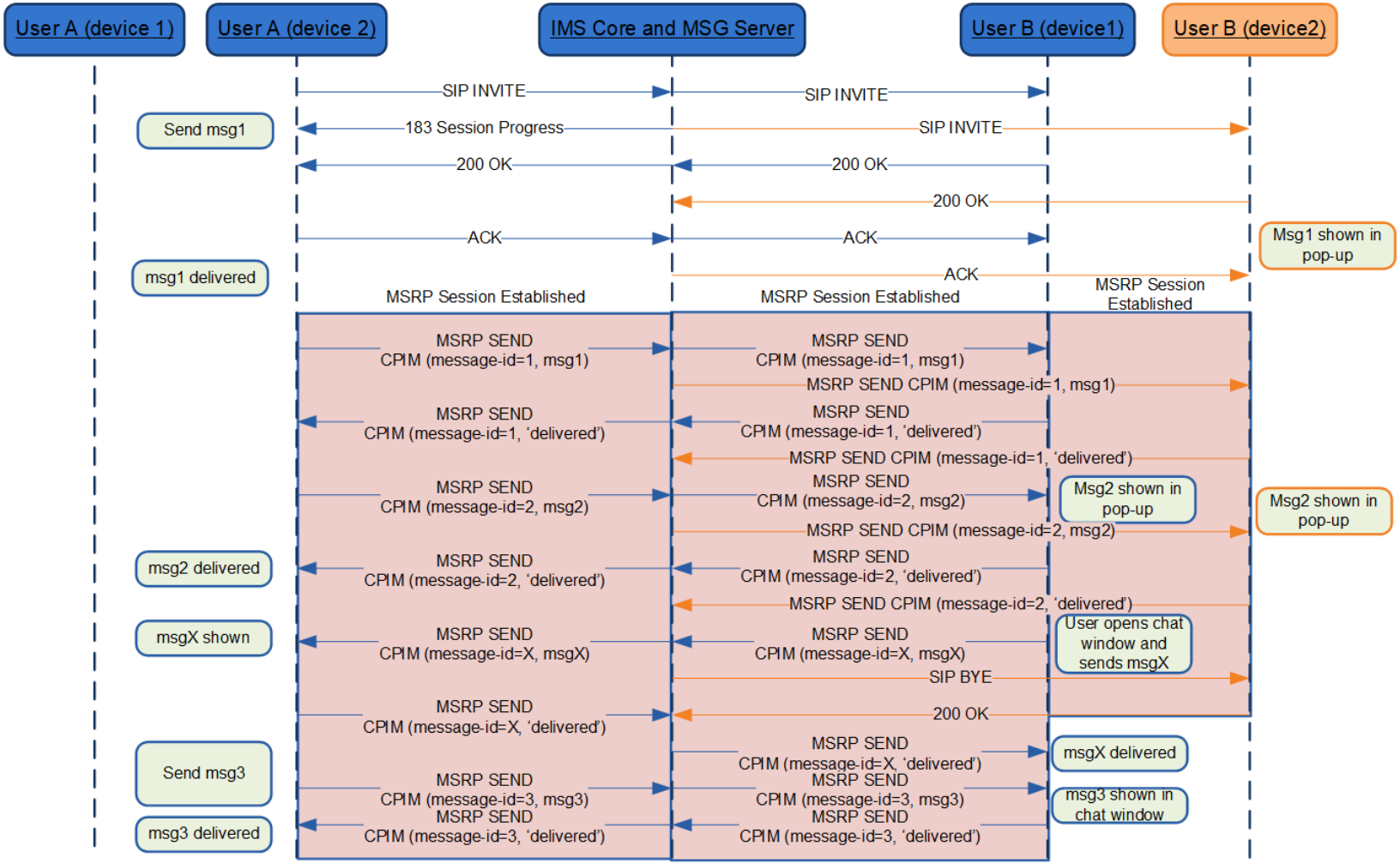


Figure 80: Rejoin in active Chat which is torn down due to inactivity*

*: Check NOTES 1, 8, 13, 14 and 15 in section B.1.11.

B.1.11 Chat and store and forward diagrams: Notes

Please note the following notes apply to diagrams in section B.1:

- NOTE 1 (B.1.1, B.1.2, B.1.3, B.1.4, B.1.5, B.1.6, B.1.7, B.1.8, B.1.9 and B.1.10): 200 OK responses to SIP MESSAGE and MSRP SEND messages are omitted for clarity.
- NOTE 2 (B.1.2 and B.1.3): In a multidevice scenario, a delivery notification received from User B might not end up on User A's device that sent the message. However this is not an issue, since all User A's devices will eventually receive the delivery notification upon synchronising with the Common Message Store.
- NOTE 3 (B.1.2 and B.1.3): B could have to handle two incoming INVITEs, one from the Messaging Server on behalf of A to deliver messages and notifications that were stored to be forwarded, and a second one directly from A who happens to want to chat with B at the same time. B should recognise the INVITE from the Messaging Server on behalf of A and not tear it down when the new INVITE directly from A arrives: The INVITE from the Messaging Server has a Referred-By header and no isfocus tag, and the INVITE directly from A does not have a Referred-By header. Please note that the same applies to the case in which the order in which the INVITEs arrive is reversed.
- NOTE 4 (B.1.4): The flow assumes that the Messaging Server does not attempt to deliver subsequent IMDNs if delivery of a first one failed shortly before.
- NOTE 5 (B.1.1): In the diagram we have represented one of the possible mechanisms to detect that the user is not online (wait for the 480 response), however, there are alternative mechanisms (triggers, 3rd party registration) that can be also used by the Messaging Server for the purpose.
- NOTE 6 (B.1.2, B.1.3 and B.1.4): Note that in the scenario where the MSRP socket is closed between the Messaging Server and the Terminating client (B) in a deferred message delivery (due for instance to a small connectivity loss with the PDP context remaining active) and no re-registration takes place, if there are notifications pending (delivery or displayed) and all the deferred messages have been sent to B already (no need to open a new MSRP session), SIP MESSAGE can be used to confirm the pending delivery/display notifications that could not be sent over MSRP.
- NOTE 7 (B.1.2, B.1.3 and B.1.4): The session established by the Messaging Server to deliver deferred messages or notifications should be terminated once the all the messages and notifications have been delivered. In more detail:
    - When delivering deferred messages, the session should be terminated (by sending a BYE) either (whatever is shorter) when the display notification corresponding to the last deferred message has been received by the Messaging Server or, after a timer started on the reception of the delivered notification for the last message expires. This timer is defined by the Service Provider.

- NOTE 8 (B.1.1, B.1.2, B.1.3, B.1.4, B.1.5, B.1.6, B.1.7, B.1.8, B.1.9 and B.1.10): As per [RFC5438], the message-id is conveyed in the messages via the imdn.Message-ID header and in the notifications via the value of the <message-id> element in the body of the IMDN.
- NOTE 9 (B.1.6): The flow shows interworking with SMS, but the flow in the SIP/MSRP part of the figure also applies when interworking with MMS.
- NOTE 10 (B.1.9): As per sections 3.2.4.10 and 3.2.4.15.
- NOTE 11 (B.1.9): The flow shows the Participating Function restarting the session before attempting the delivery. This is an implementation option to ensure that a session is established when the user sends content. The Participating Function may also choose to establish this session in parallel or only when there is actual content to be sent in the Chat.
- NOTE 12 (B.1.9): The flow assumes that no display notifications were requested.
- NOTE 13 (B.1.10): The diagram show that “delivered” notifications for messages for which such a notification was sent already, are suppressed by the network. As this cannot always be guaranteed, clients shall be prepared to receive such duplicate notifications and discard them silently. This holds also for display notifications and for notifications related to messages that were not sent by that client.
- NOTE 14 (B.1.10): To support this case forking in the terminating side needs to be done at the Messaging Server using the mechanisms defined in section 2.10.2 as forking in the IMS core will lead to a race condition.
- NOTE 15 (B.1.10): The behaviour shown to close the session to the other devices when there is activity from one receiving device is just one of the possible behaviours. Alternatively, the session to those devices could be maintained and all sent and received messages, notifications and events could be delivered to all devices with appropriate indications as described in [GSMA PRD-RCC.11].

B.2 Restful Message Store Flows (informative)

B.2.1 Client Initialization and Synchronization using RESTful Approach

There are two types of initialization that the client must consider:

1. First Time Sync - Running the application for the first time having either had no previous contact with MESSAGE STORE SERVER or having had to clear and reinitialize the local message store.
2. Steady State Sync – Ongoing sync with the Message Store Server. This happens during normal operation; it also covers the case when connections time out, network signal is lost, the application or phone is restarted etc.

The overall initialization process for a client may look as follows.

1. Client ensures there is a valid OAuth Token to use with MESSAGE STORE SERVER,
2. starts the Notification process and is listening for updates to MESSAGE STORE SERVER, initiates the First Time Sync process if appropriate
3. Waits for notifications to arrive, i.e. initiates the Steady State Sync process.
4. The Notification process on the client polls for notifications that have arrived from MESSAGE STORE SERVER. Note again that client ensures there is a valid OAuth token for use by both it and the Notification thread. This means that the Notification thread must wait if it gets an authentication error until the Worker thread has obtained a new OAuth token

The following sections discuss in more detail how the First Time and Steady State Sync functions work.

B.2.1.1 First Time Sync Process

First Time Sync performs the following to get the local store and MESSAGE STORE SERVER in sync with one another. This is in two stages and result in both stores having the same set of messages.

- Download existing messages from MESSAGE STORE SERVER to the handset.
- Upload any handset messages that are not present in MESSAGE STORE SERVER.

Once First Time Sync has completed then Steady State Sync can start.

Two key notes to consider:

1. Since First Time Sync can take a significant amount of time to complete, the process may be interrupted. The handset may lose network access or be stopped.
2. Any user interactions with messages on the handset such as reading or deleting messages need to be queued up until the message in question has been correlated with MESSAGE STORE SERVER.

Since messages are returned newest first, the handset may choose to synchronize only the most recent messages, saving local storage space.

Recommended Approach:

To ensure a graceful re-sync restart, the handset must store the following information persistently:

- The Index of the latest notification returned by a poll – to check for missed notifications.
- The latest RestartToken value – used to restart the polling process if interrupted.
- The latest /objects/operations/search cursor - used to restart the search process if interrupted.
- The time at which the existing subscription (if any) times out - used to know when the lifetime of the subscription needs updating.
- A list of the messages on the handset that have not been correlated with MESSAGE STORE SERVER.
- Whether First Time Sync message upload has completed or not - to decide whether Steady State Sync should run or not.

The client application should perform the following steps:

- Perform the user interaction needed to begin first time sync – gaining acceptance of terms, account provisioning, etc. After this point, all synchronization is now performed within the new threads.
- Start listening for Notifications and wait until it has an active long poll call to MESSAGE STORE SERVER.
- Submit a search request to the MESSAGE STORE SERVER with a blank cursor, or the stored cursor if restarting an interrupted sync.

  Begin processing the queue of the response to the search request:

- Use the request arrival time and number of downloaded messages to determine the pause before issuing another request. The overall rate for this sync should be one message/second to limit handset, network and MESSAGE STORE SERVER load.
- For a search request: download the messages using the objects/operations/search endpoint using the specified cursor.
  - The first search uses a blank cursor, and subsequent searches use the cursor supplied by the last search.
  - The number of messages to be downloaded in each search is specified using the maxEntries field – the recommended value is 10.

  On receiving a response to a search:

  - Iterate through the message headers and correlate with locally stored messages. If a correlationId is available for the message, use it to correlate message. Otherwise, correlate using the correlationTag.
    - If a correlating message is found, then update the local message, and remove from the list of uncorrelated messages.

- Be aware that in rare cases, when relaying on a correlationTag, multiple matches may occur. In this case, follow the guidance provided in Dealing with Hash Collisions.
    - If not found then store the message locally as an uncorrelated message.
- For small messages, such as SMS messages, all the information on the message will be retrieved in the search. For larger messages, the client will need to retrieve the content by downloading the payload parts of the message – but the client will not need to download the message headers as these are returned by the search.
- Add a new request to the worker queue. If the cursor is not blank, add a search request with the new cursor. If it is blank, messages have finished downloading; add an upload request for the first uncorrelated message.

- For a notification request, update the local message store with the changed objects. Event notifications contain new, changed or deleted objects. How the client processes such a notification depends on whether the message already exists on the client:
    - For messages that already exist on the local store, use the lastModSeq attribute to determine whether to update the local copy of the message.
    - Save the latest lastModSeq of every object and folder. If the network copy has a lastModSeq greater than the local one, the client needs to update the local copy to match.
    - This does not need any further requests; the updated information is contained in the poll response.
    - When updating the local copy of an uncorrelated message in response to a sync, remove that message from the list of uncorrelated messages.
    - For messages that do not have a local version, the client will need to download the message from the message store.

- For a request from the application thread to modify a message
    - If the message has been correlated with MESSAGE STORE SERVER, the thread should make the update to MESSAGE STORE SERVER.
    - If the message has not yet been correlated, the update must be delayed until the message has been correlated (which may only be after the First Time Sync process is complete).

B.2.1.2 Notification Flow

- The Client shall start a notification channel and subscription.
- Begin the Steady State long poll flow.
    - Issue poll request. This must be on a separate network connection (pipelining requests on the same HTTP connection is not recommended in long polling.)
    - If the poll returns with an empty notification list or no content, it has timed out - reissue the poll request.
    - Otherwise, the poll returns with one or more event notifications. In this case

        - Check the index of the notifications returned against the saved index. The smallest of the new indices should be one greater than the saved index; if this is not the case, the client has missed a notification.
        - Otherwise, reissue the poll.
        - Add the notifications returned to the worker queue for processing.

    - The client is responsible for ensuring that the subscription does not time out; the client must keep track of the remaining subscription duration and periodically extend it before it expires.
    - The MESSAGE STORE SERVER service refreshes notification channels automatically when the client performs a long poll. If the client experiences an outage, the notification channel and/or subscription it is using may expire; for details on how to handle this case, see section 7.3.1 of [CPM-MSGSTOR-REST].

B.2.1.3 Steady State Synchronization

The purpose of the Steady State Sync process is to update the handset with changes made to the subscriber's mailbox.

In overview, the client waits for notifications of incoming messages using a polling request, and may concurrently process already received notifications, uploading and downloading messages and performing all correlation of MESSAGE STORE SERVER with the local handset store.

Recommended Approach:

- For a notification request, the client shall update the local message store with the changed objects. Event notifications contain new, changed or deleted objects. How the client processes such a notification depends on whether the message already exists on the client:

    - For a ResetBox notification see [CPM-MSGSTOR-REST] section 5.1.4.2.1; in summary the MESSAGE STORE SERVER client application data (local message cache, counters, etc.) should be emptied and the client should perform a first time sync with MESSAGE STORE SERVER.
    - Perform correlation steps provided for objects with accompanied by a correlationId and correlation Tag. If a correlationId does not exist, in which case a correlationTag is provided. If both are provided, the client shall use the correlationId.
    - For messages that already exist on the local store (in other words, for message that are successfully correlated), client shall use the lastModSeq attribute to determine whether it needs to update its local copy of the message.

        - The client should save the latest lastModSeq of every object and folder. If the network copy has a lastModSeq greater than the local one, the client needs to update the local copy to match.
        - This does not need any further requests; the updated information is contained in the poll response.

- Outside of First Time Sync, a client should never download message content from the message store. The message may reside in the client's uncorrelated sync objects list, but the client should never present it to the user.

- For a request from the application thread to modify a message, the thread should also make the update to MESSAGE STORE SERVER.

B.2.2 RESTful Notification Events

For Notification about storage changes, the events in Table 274 are combined into an NmsEventList before being sent to clients with an appropriate subscription.

Element	Type	Optional	Description
deletedObject	DeletedObject	Choice	Reference to the user-deleted object
deletedFolder	DeletedFolder	Choice	Reference to the user-deleted folder
expiredObject	DeletedObject	Choice	Reference to the expired object
expiredFolder	DeletedFolder	Choice	Reference to the expired folder.
changedObject	ChangedObject	Choice	Reference to the new or changed object.
changedFolder	ChangedFolder	Choice	Reference to the new or changed folder.
resetBox	ResetBox	Choice	The box has been reset.

Table 274: RESTful Notification Events

B.2.3 Notification Channel Setup

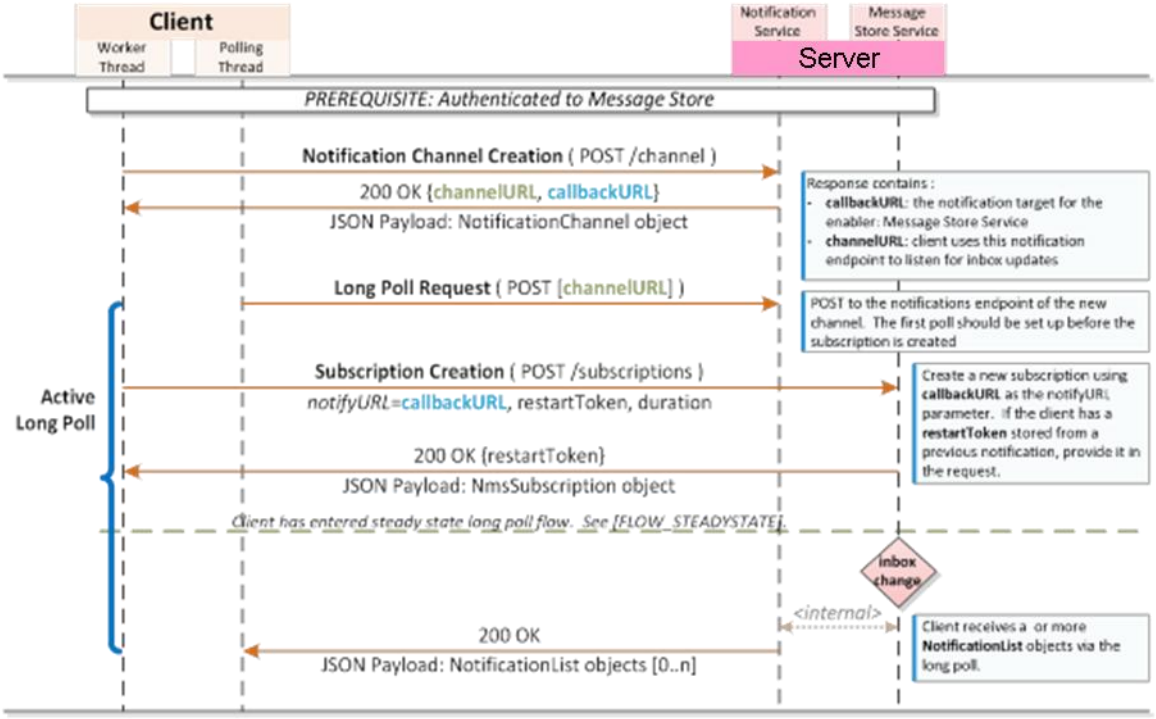


Figure 81: Message Store - Notification Channel Setup

B.2.4 Object Upload

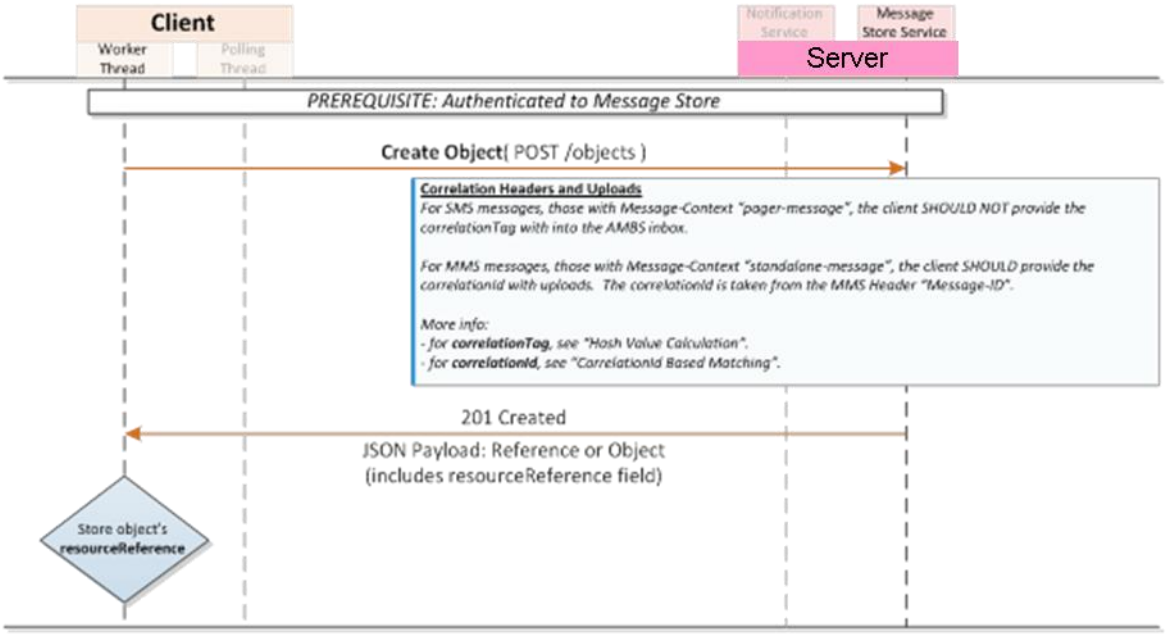


Figure 82: Message Store – Object Upload

B.2.5 Example Object Download

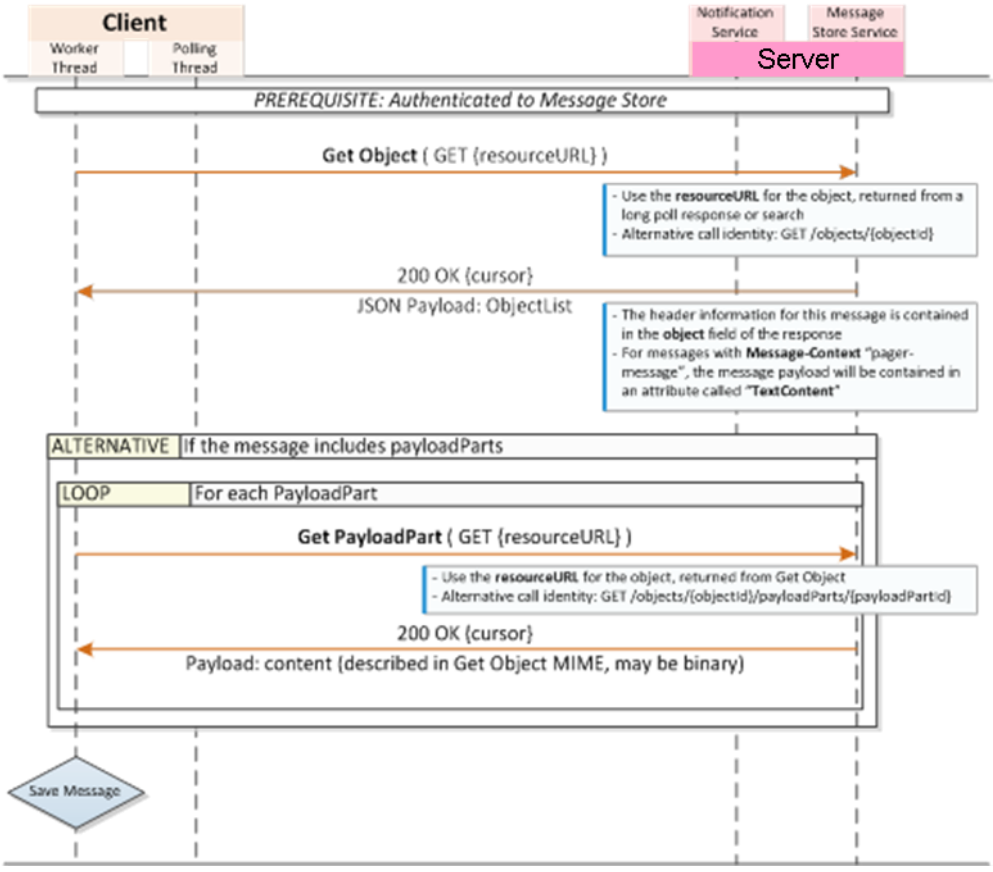


Figure 83: Message Store – Example Object Download

B.2.6 Example RESTful Search operation

B.2.6.1 /objects/operations/search

Resource for getting information about a selected set of objects in the storage supporting POST only

```
//{nmsHost}/nms/v1/base/{boxId}/objects/operations/search
```

Note that NMS supports filtering and therefore a client may receive filtered results based on search criteria that the client sets or that the server imposes (based on client capabilities): searchCriteria, searchScope nor sortCriterion, NMS supports the maxEntries and fromCursor elements of selectionCriteria.

Objects are returned in descending date order; the most recent object is returned first.

B.2.6.2 POST

This endpoint is specified in more detail at [CPM-MSGSTOR-REST] section 6.7.5. The examples below do not exactly match the NMS specification because this version of NMS contains a restricted implementation of the search endpoint.

Making a request:

```
POST http://nms-
sib01.si.enclab.mno.net/nms/v1/base/tel:+19717774171/objects/operations/s
earch HTTP/1.1
Accept-Encoding: gzip,deflate
MIME-Version: 1.0
x-MNO-clientId: MNO_NMSG001
x-MNO-clientVersion: 2.3.4
x-MNO-deviceId: 310410438258561
x-MNO-contextInfo: mdl=MNONMSTestClient,os=1.2.3,fw=11.22.333.4444
x-MSw-ClientType: caching
Accept: application/json
Content-Type: application/json
Authorization: Bearer
PAT_bceNdO3GSKwkHBkL1rnEceW2TTLx3ijGzbonv3qFvVuYHnocDopEZ2eyMfzea/2POmQrL
nYJch2VDeEA2ooRh22pDnYEHk9YsFr8WrKbsMrHt5IMHhVrq81krtmWacbV/rFV4oT/1ckjpa
IhYu+qh4YYLc3FGuYMl+fvipRoLBTYlyHUZSR7xJSVHqQtY1aIGU7J2xXr2r6tTIJ5eGpoRIH
DK7z3CL5c6ii1aI+TqyW5BBDqZtse8MTRfTva69ot9rv0ehYHoZ7T7Q/DfbY/DZ4ffnTReB6E
HHYxA62SOKJ+GiaAkVYZNhtuEAHTSAF6
Content-Length: 40
Host: nms-sib01.si.enclab.MNO.net
Proxy-Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.1.1 (java 1.5)
{"selectionCriteria": {"maxEntries": 3}}
```

Responses:

```
HTTP/1.1 200 OK
Server: nginx
Date: Thu, 22 Oct 2015 20:06:01 GMT
Content-Type: application/json; charset=UTF-8
Content-Length: 689
```

```
Cache-Control: proxy-revalidate
Proxy-Connection: Keep-Alive
Connection: Keep-Alive
Content-Encoding: gzip
Set-Cookie: SERVERID=882565574.20480.0000; path=/

{"objectList": {
   "object":    [
            {
         "parentFolder": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/folders/iw9%7Coz
AEM333XXXMnMn",
         "attributes": {"attribute":          [
                        {
               "name": "Date",
               "value": ["2015-04-01T14:30:30Z"]
            },
                        {
               "name": "TextContent",
               "value": ["this is a test SMS message"]
            },
                        {
               "name": "Message-Context",
               "value": ["pager-message"]
            },
                        {
               "name": "Is-CPM-Group",
               "value": ["no"]
            },
                        {
               "name": "Direction",
               "value": ["IN"]
            },
                        {
               "name": "From",
               "value": ["+19995551212"]
            },
                        {
               "name": "Subject",
               "value": ["Test Object"]
            },
                        {
               "name": "To",
               "value": ["+19717774171"]
            }
         ]},
         "flags": {"flag": []},
         "path": "/Default/tel%3A%2B19995551212/CiUh",
         "resourceURL": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/objects/CiUh",
         "lastModSeq": 1445540857000000000,
         "correlationTag": "40bef8ea4cad717d"
```

```
      },
            {
         "parentFolder": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/folders/iw9%7Coz
AEM333XXXMnMn",
         "attributes": {"attribute":          [
                        {
               "name": "Date",
               "value": ["2015-04-01T14:30:30Z"]
            },
                        {
               "name": "MultipartContentType",
               "value": ["application/vnd.wap.multipart.mixed"]
            },
                        {
               "name": "Message-Context",
               "value": ["standalone-message"]
            },
                        {
               "name": "Is-CPM-Group",
               "value": ["no"]
            },
                        {
               "name": "Direction",
               "value": ["IN"]
            },
                        {
               "name": "From",
               "value": ["+19995551212"]
            },
                        {
               "name": "Conversation-ID",
               "value": ["5EUINVAX8.CBHLZHDXR"]
            },
                        {
               "name": "Subject",
               "value": ["Test Object"]
            },
                        {
               "name": "To",
               "value": ["+19717774171"]
            }
         ]},
         "flags": {"flag": []},
         "path": "/Default/tel%3A%2B19995551212/wnhX",
         "resourceURL": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/objects/wnhX",
         "payloadPart":          [
                        {
               "contentType": "text/plain; name=sms",
               "size": 28,
```

```
                "href": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/objects/wnhX/pay
loadParts/M"
            },
                        {
                "contentType": "image/jpeg; name=testImage1.jpeg",
                "size": 130879,
                "href": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/objects/wnhX/pay
loadParts/n"
            }
         ],
         "lastModSeq": 0,
         "correlationId": "BNZ2HAFOV.70A9D1W5B"
      },
            {
         "parentFolder": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/folders/iw9%7Coz
AEM333XXXMnMn",
         "attributes": {"attribute":          [
                        {
               "name": "Date",
               "value": ["2015-04-01T14:30:30Z"]
            },
                        {
               "name": "TextContent",
               "value": ["this is a test SMS message"]
            },
                        {
               "name": "Message-Context",
               "value": ["pager-message"]
            },
                        {
               "name": "Is-CPM-Group",
               "value": ["no"]
            },
                        {
               "name": "Direction",
               "value": ["IN"]
            },
                        {
               "name": "From",
               "value": ["+19995551212"]
            },
                        {
               "name": "Subject",
               "value": ["Test Object"]
            },
                        {
               "name": "To",
               "value": ["+19717774171"]
            }
```

```
         ]},
         "flags": {"flag":          [
            "\\Seen",
            "\\Flagged"
         ]},
         "path": "/Default/tel%3A%2B19995551212/CiU7",
         "resourceURL": "http://nms-
sib01.si.enclab.MNO.net/nms/v1/base/tel%3A%2B19717774171/objects/CiU7",
         "lastModSeq": 1445535875000000000,
         "correlationTag": "40bef8ea4cad717d"
      }
   ],
   "cursor": "wAnhuA$CAiUXA$TAhiA$xA3XA$"
}}
```

Annex C Special Procedures

C.1 SIP/TCP and NAT traversal

As specified in section 2.7 when using SIP over TCP (or TLS), the client shall rely on the CRLF mechanism defined in [RFC6223]. However [RFC6223] does not provide the means to negotiate the direction in which these keep-alive requests are sent (it's always the party that initiated the SIP request that has to send keep-alive requests) and a device OS's scheduling policy may not always allow the client to meet the timing requirements for sending keep-alive requests. To overcome these limitations for clients running on such platforms a mechanism is provided in this annex which is also specified in an internet draft that has been submitted to the IETF (see [IETF-DRAFT-RKEEP]). This mechanism allows these clients to request to reverse the direction in which the keep-alive requests are sent (that is they will be sent from network to client) by including an 'rkeep' parameter in the Via header of the SIP request that is used in the same way as the 'keep' parameter defined in [RFC6223].

Like the server in [RFC6223], the client may include a proposed frequency (in seconds) of the keep-alive period by adding a value to the '*rkeep'* parameter (e.g. "rkeep=600"). This frequency shall not be set to a value smaller than 30 seconds. An Edge Proxy supporting this mechanism that receives requests that contain an 'rkeep' parameter in the top-most Via header can provide the following responses:

- If the *rkeep* value is provided by the client (e.g. rkeep=600) and it is acceptable according to the service provider policies, the registration response shall include the '*rkeep*' parameter in the top-most Via header when sending a reliable response on that request and shall remove the value (i.e. *rkeep* is sent back without a value).
- If the rkeep value is provided by the client but it is not acceptable based on the Service Provider policies, the Edge proxy shall include the '*rkeep*' parameter in the top-most Via header when sending a reliable response on that request and shall set the value to a default one (i.e. rkeep=180 [assuming 180 is the default value]).
- If the rkeep value is not provided by the client (e.g. rkeep without a specified value), the Edge Proxy shall provide a frequency value by setting a default value to the 'rkeep' parameter in its response (i.e. rkeep=180 [assuming 180 is the default value]).

Then it shall send double CRLF "ping" requests as defined in [RFC5626] to the client thereby complying to the specified interval and considering the connection as failed when no single CRLF "pong" response is received within 10 seconds.

An Edge proxy not supporting this mechanism shall not modify the rkeep parameter included by the client. The fact the value introduced by the client is not modified by the Edge Proxy shall be interpreted by the client as the Edge Proxy does not support the network initiated keep alive. Please note that this approach guarantees backwards compatibility.

NOTE1: It is highly recommended that clients not experiencing such scheduling limitations use the standard 'keep' mechanism defined in [RFC6223] and send the keep-alive requests themselves. For those clients the implementation of this section is therefore optional.

NOTE2: Alternatively a Service Provider could decide to rely on client platform specific notification mechanisms

NOTE3: The requirement to extend the keep-alive procedures to support network-initiated keep-alives has been brought into the IETF for standardisation (see [IETF-DRAFT-RKEEP]). The procedures here will be updated once that work is completed. In particular, this standardisation process should allow the client to detect that the network does not support network-initiated keep-alives as described above.

C.2 Errata for RFC 5438

The following errata have been reported for [RFC5438] in [RFC5438Errata] and is important to be taken into consideration for RCS with respect to messaging and chat services:

- Errata ID: 3013
- Status: Held for Document Update
- Type: Technical
- Reported By: Dan Price
- Date Reported: 2011-11-04
- Held for Document Update by: Robert Sparks
- Section 7.2.1.1 says:

  From: Bob <im:bob@example.com>
  To: Alice <im:alice@example.com>
  NS: imdn <urn:ietf:params:imdn>
  imdn.Message-ID: d834jied93rf
  Content-Type: message/imdn+xml
  Content-Disposition: notification
  Content-length: ...

- It should say:

  From: Bob <im:bob@example.com>
  To: Alice <im:alice@example.com>
  NS: imdn <urn:ietf:params:imdn>
  imdn.Message-ID: d834jied93rf

  Content-Type: message/imdn+xml
  Content-Disposition: notification
  Content-length: ...

- Notes:

  None of the examples in this RFC (Request For Comments) comply with the format of CPIM defined in RFC 3862, in which the message metadata headers are separated from the headers of the encapsulated MIME object by a blank line.

C.3 Definition of RCS CPIM Header Extensions

C.3.1 RCS CPIM Extension Name Space

CPIM header extensions make use of the extension framework defined in [RFC3862]. The RCS extensions make use of the RCS name space as defined in this section.

The RCS Namespace is defined as follows:

```
NS: rcs <http://www.gsma.com>
```

NOTE: The namespace is considered as a place holder for a final one to be defined by GSMA or other committees.

C.3.2 Definition of rcs.Service-Centre-Address header

The rcs.Service-Centre-Address header contains the Service Centre Address associated with a short message, see [3GPP TS 23.040]

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Service-Centre-Address	0	1

Table 275: rcs.Service-Centre-Address header

The field itself is defined in ABNF as follows:

```
service-centre-address  = "rcs.Service-Centre-Address:" service-
centre-value CRLF
service-centre-value    = "+" *15DIGIT
```

Example:

```
rcs.Service-Centre-Address: +491712020202
```

C.3.3 Definition of rcs.Reply-Path header

The rcs.Reply-Path header contains the indication whether a reply path exists for a short message, see [3GPP TS 23.040].

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Reply-Path	0	1

Table 276: rcs.Reply-Path header

The field itself is defined in ABNF as follows:

```
reply-path  = "rcs.Reply-Path:" reply-path-value CRLF
reply-path-value  = "0" / "1"
```

Example:

```
rcs.Reply-Path: 0
```

C.3.4 Definition of rcs.Replace-Short-Message-Type header

The rcs.Replace-Short-Message-Type header indicates the type number of a short message for replacement, see [3GPP TS 23.040]

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Replace-Short-Message-Type	0	1

Table 277: Replace-Short-Message-Type header

The field itself is defined in ABNF as follows:

```
rcs-replace-short-message-type = "rcs.Replace-Short-Message-Type:"
                  replace-type-value CRLF

replace-type-value      = "1" / "2" / "3" / "4" / "5" / "6" / "7"
```

Example:

```
rcs.Replace-Short-Message-Type: 1
```

C.3.5 Definition of rcs.Mms-Message-Class header

The rcs.Mms-Message-Class header indicates the class of a multimedia message, see [3GPP TS 23.140].

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Mms-Message-Class	0	1

Table 278: Mms-Message-Class header

The field itself is defined in ABNF as follows:

```
rcs-mms-message-class = "rcs.Mms-Message-Class: "
                  class-value CRLF

class-value = "Personal" / "Advertisement" / "Informational" / "Auto"
```

Example:

```
rcs.Mms-Message-Class: Personal
```

C.3.6 Definition of rcs.Message-Correlator header

The rcs.Message-Correlator header contains the message identification for message correlation, see section 3.2.4.7.2. The encoding of the message identification depends on the context defined in rcs.Message-Context parameter.

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Message-Correlator	0	1

Table 279: Message-Correlator header

The field itself is defined in ABNF as follows:

```
rcs-message-context = "rcs.Message-Correlator: "
                      message-correlator-value CRLF

                ; for encoding rules of "message-correlator-value"
                ; follow references in [CPM-MSGSTOR-REST]
```

Example:

```
rcs.Message-Correlator: Hello world
```

C.3.7 Definition of rcs.Message-Context header

The rcs.Message-Context header indicates the context and presentation characteristics of a message. It can be used by interpreters to derive the encoding rules for the value of the rcs.Message-Context header.

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Message-Context	0	1

Table 280: Message-Context header

The field itself is defined in ABNF as follows:

```
rcs-message-context = "rcs.Message-Context: "
                      message-context-class CRLF

                ; for encoding rules of " message-context-class"
                ; refer to [RFC3458]
```

Example:

```
rcs.Message-Context: "pager-message"
```

C.3.8 Definition of rcs.Reference-ID header

The rcs.Reference-ID header contains the imdn.Message-ID of the referenced message, which is defined in [RFC5438] and should conform to the definitions of [RFC5322].

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Reference-ID	0	1

Table 281: Reference-ID header

The field itself is defined in ABNF as follows:

```
rcs-message-context = "rcs.Reference-ID: " message-id CRLF
            ; for encoding rules of message-id refer to [RFC5322]
```

Example:

```
rcs.Reference-ID: bddee54e-f450-4fbf-a18d-24814f64e461
```

C.3.9 Definition of rcs.Reference-Type header

The rcs.Reference-Type header contains the type of Extended Messaging (see section 3.2.8).

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Reference-Type	0	1

Table 282: Reference-Type header

The field itself is defined in ABNF as follows:

```
rcs-message-context = "rcs.Reference-Type: " reference-type-value
            CRLF
reference-type-value = reaction-type-value / “Reply” / “Edit” /
            “Delete” / “Recall”
reaction-type-value = react-action (“Reaction” / “Custom-Reaction”)
react-action = “+” / “-”
```

Examples:

```
rcs.Reference-Type: Reply
rcs.Reference-Type: +Reaction
rcs.Reference-Type: -Custom-Reaction
```

C.3.10 Definition of rcs.advised-action header

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.advised-action	0	1

Table 283: advised-action header

The field itself is defined in ABNF as follows:

```
advised-action = "advised-action:" advised-action-type-value CRLF
advised-action-type-value = "warn" | "hide"
```

Example:

```
rcs.advised-action: hide
```

C.3.11 Definition of rcs.source header

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.source	0	1

Table 284: source header

The field itself is defined in ABNF as follows:

```
source = "source:" *ALPHA CRLF
```

Example:

```
rcs.source: mycarrier
```

C.3.12 Example CPIM Message with a Suspicion Indicator

The following CPIM message containing a spam indicator is provided by way of example:

```
From: <sip:anonymous@anonymous.invalid>
To: <sip:anonymous@anonymous.invalid>
DateTime: 2024-01-30T06:48:10Z
NS: imdn <urn:ietf:params:imdn>
NS: rcs <http://www.gsma.com/rcs/>
imdn.Message-ID: 2252a2757d
imdn.Disposition-Notification: positive-delivery,display
…
…
rcs.advised-action: warn
rcs.source: mycarrier
…
…
Content-Type: text/plain;charset=UTF-8
Content-Length: 12

Hello world!
```

C.3.13 Definition of rcs. Welcome-ID

The Welcome-ID header provides an identifier for a Chatbot Welcome Message.

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Welcome-ID	0	1

Table 285: Welcome-ID header

The field itself is defined in ABNF as follows:

```
Welcome-ID = "Welcome-ID:" *HEADERCHAR CRLF
            ; for HEADERCHAR definition refer to [RFC3862] CRLF
```

Example:

```
rcs.Welcome-ID: campaign123
```

C.3.14 Definition of rcs. Deep-Link-Context

The Deep-Link-Context header provides an identifier for a contextual Chatbot Welcome Message.

The header is defined as an extension to the [RFC3862] field definitions. The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.Deep-Link-Context	0	1

Table 286: Deep-Link-Context header

The field itself is defined in ABNF as follows:

```
Deep-Link-Context = "Deep-Link-Context:" *HEADERCHAR CRLF
            ; for HEADERCHAR definition refer to [RFC3862]
```

Example:

```
rcs.Deep-Link-Context: store99
```

C.3.15 Definition of rcs.User-Profile-Config-Data

The User-Profile-Config-Data header provides the Base64 encoded ProfileConfig protocol buffer defined as per section 3.2.11.3.

The limits for the occurrence of the field are defined in the following table:

Field	Min Number	Max Number
rcs.User-Profile-Config-Data	0	1

Table 287: User-Profile-Config-Data

The field itself is defined in ABNF as follows:

```
User-Profile-Config-Data = "User-Profile-Config-Data:" *HEADERCHAR
CRLF
            ; for HEADERCHAR definition refer to [RFC3862]
```

Example:

```
rcs.User-Profile-Config-Data:
ZGF0YTEsZGF0YTIsZGF0YTMsZGF0YTQsZGF0YTUsZGF0YTYsZGF0YTcsZGF0YTgsZGF0Y
TksZGF0Y
```

C.4 Definition of SIP Header Extensions

C.4.1 User-Agent and Server Header Extensions

The "User-Agent" and "Server" header fields and their formatting are defined in [RFC3261]. For use in GSMA context, the header field values shall be composed in accordance with the definitions in this section.

The User-Agent and Server headers ABNF are specified in [RFC3261] as follows:

```
Server = "Server" HCOLON server-val *(LWS server-val)
User-Agent = "User-Agent" HCOLON server-val *(LWS server-val)
server-val = product / comment
product =  token [SLASH product-version]
product-version = token
```

The "User-Agent" or "Server" header value shall contain product tokens to identifiy the user agent characteristics.

For the use in GSMA context, the following ABNF applies:

```
Server = "Server" HCOLON product-list
User-Agent = "User-Agent" HCOLON product-list
product-list = enabler *(LWS enabler)
                   *(LWS service)
                   [LWS terminal]
                   [LWS client]
                   [LWS OS]
                   *(LWS list-extension)
```

The following user agent characteristics are defined:

1. Enabler technology:

   This product token identifies the supported and activated enabler technologies of an entity. The values of the product tokens are defined in the corresponding enabler specifications, e.g. the CPM enabler. One or more product tokens can be present in the header value. For RCS, the presence of the CPM technology product token defined in Annex D of [GSMA PRD-RCC.11] is mandatory. If used for other services, at least one enabler needs to be specified as an extension of the enabler rule.

   ```
   enabler = CPM-Version / extension ; CPM version is defined in
                                     ; Annex D of
                                     ; [GSMA PRD-RCC.11]
   ```

2. Service:

   This product token identifies the supported and enabled services provided by an entity. Zero or more product tokens can be present in the header value. For RCS the service product token shall be provided by clients and may be provided by networks and Chatbot Platforms.

   ```
   service = RCS-product / extension
   ```

```
RCS-product = "RCS-" RCS-device-token SLASH RCS-profile
RCS-device-token = "client" ; header is included by a client
                  / "serv" ; header is included by an
                           ; application server
                  / "bot"  ; header is included by a botplatform
                  / token
RCS-profile = token ; same as rcs_profile defined in section 2.3.2.2
```

Where RCS-device-token shall be set to "client", "serv" and "bot" depending on whether the header field is included by a client, an application server or a Chatbot Platform respectively. RCS-profile shall be set to the same value as the one provided for the rcs_profile Configuration request parameter defined in section 2.3.2.2.

3. Terminal information:

   This product token identifies the terminal on which an entity resides. The value of the product token is determined by the terminal OEM. Zero or one product token can be present in the header value. This product token is only applicable for clients.

```
terminal = "term-" terminal-vendor SLASH terminal-model "-"
           terminal-SW-version
terminal-vendor = token      ; same as terminal_vendor defined in
                             ; section 2.4 of [GSMA PRD-RCC.14]
terminal-model = sf-token ; same as terminal_model defined
                             ; in section 2.4 of [GSMA PRD-RCC.14]
terminal-SW-version = token  ; same as terminal_sw_version defined
                             ; in section 2.4 of [GSMA PRD-RCC.14]
sf-token = 1*(alphanum / "." / "!" / "%" / "*" / "_" /
                       "+" / "`" / "'" / "~" )
```

   For RCS, the terminal-vendor, terminal-model and terminal-SW-version shall be set to the same value as the one provided for respectively the terminal_vendor, terminal_model and terminal_sw_version Configuration request parameters defined in section 2.4 of [GSMA PRD-RCC.14].

4. Client information:

   This product token identifies the client software. The value of the product token is determined by the client provider. Zero or one product token can be present in the header value. For RCS, the client information product token is only applicable for clients.

```
client = "client-" client-vendor SLASH client-version
client-vendor = token ; same as client_vendor defined in
                      ; section 2.3.2.2
client-version = token; same as client_version defined in
                      ; section 2.3.2.2
```

   For RCS, the client-vendor and client-version shall be set to the same value as the one provided for respectively the client_vendor and client_version Configuration request parameters defined in section 2.3.2.2.

5. Operating System information:

This product token identifies the operating System on which an entity resides. The value of the product token is determined by the terminal OS provider. Zero or one product token can be present in the header value. This product token is only applicable for clients.

```
OS = "OS-" OS-type (SLASH OS-version)
OS-type = "Android" / "IOS" / "SymbianOS" / "Windows" / "Other" /
          token
OS-version = 1*alphanum *4("." 1*alphanum)
```

Where OS-type and OS-version should be set to the most appropriate values for the Operating System on which the client is running.

6. Vendor specific and extensions:

   The extensibility of the product token list is maintained. Vendor specific information and extensions beyond the defined product token groups can be included in the list using the extension mechanism.

```
list-extension = extension
extension = extension-product / comment
extension-product = gsma-extension / other-extension
gsma-extension = tag "-" value [SLASH product-version]
tag = sf-token
value = token
product-version = token
other-extension = token [SLASH product-version]
```

NOTE: Because it cannot be guaranteed that the User-Agent and Server header fields are passed unmodified between entities supporting RCS, it is recommended that such entities do not depend on the presence of the described information. It is therefore recommended to handle this as purely informational.

Examples:

```
User-Agent: CPM-client/OMA2.2 RCS-client/UP_2.0 term-Vendor1/Model1-
XXXX client-CLN1/Software1234 OS-Android/8.1
Server: CPM-serv/OMA2.2 RCS-serv/UP_2.0
```

C.5 Void

C.6 IMDN Extensions

C.6.1 Definition of Delivery IMDN Extension

The delivery Notification IMDN schema in [GSMA PRD-RCC.11] is extended to identify failure reason as follows:

```
<define name="deliveryNotification">
  <element name="delivery-notification">
    <element name="status">
      <choice>
        <element name="delivered">
          <empty/>
```

```
          </element>
          <element name="failed">
            <choice>
              <element name="not-supported">
                <choice>
                  <element name="unsupported-content-type">
                    <empty/>
                  </element>
                  <element name="unsupported-content">
                    <empty/>
                  </element>
                </choice>
              </element>
              <ref name="commonDispositionStatus"></ref>
            </choice>
            <ref name="deliveryExtension"/>
          </element>
        </element>
</define>
```

C.6.2 Delivery IMDN Example

The following IMDN shows a failed delivery notification if provided by way of example:

```
<?xml version="1.0" encoding="UTF-8"?>
  <imdn xmlns="urn:ietf:params:xml:ns:imdn">
      <message-id>34jk324j</message-id>
      <datetime>2024-11-26T12:16:49-05:00</datetime>
      <delivery-notification>
        <status>
          <failed>
            <not-supported>
              <not-supported-content-type>
            </not-supported>
          </failed>
        </status>
      </delivery-notification>
    </imdn>
```

C.7 Additional MIME Headers Definitions

C.7.1 Content-Encoding

If present, this will communicate the compression algorithm that was applied to the content in the MIME body, and is defined using following ABNF form:

```
encoding = "Content-Encoding" ":" mechanism
mechanism = cpim-body-encoding
```

NOTE: allowed values for `cpim-body-encoding` are for further study.

C.7.2 Original-Content-Length

Original-Content-Length is used to communicate original length of the compressed payload.

```
original-content-length = "Original-Content-Length" ":" 1*DIGIT
```

Annex D Document Management

D.1 Document History

Version	Date	Brief Description of Change	Approval Authority	Editor / Company
1.0	13 August 2012	First version for RCS 5.1 based on approved RCS 5.0 specification version 1.0 Approved by DAG and PSMC	PSMC	Tom Van Pelt / GSMA
1.0	26 September 2012	Added RCC.07 number		Tom Van Pelt / GSMA
2.0	02 May 2013	Applied MCR1001 approved by DAG and PSMC	PSMC	Tom Van Pelt / GSMA
3.0	25 September 2013	Applied MCR1002 approved by DAG and PSMC	PSMC	Tom Van Pelt / GSMA
4.0	28 November 2013	Applied MCR1003 approved by DQR and Global Specification Group (GSG)	GSG	Tom Van Pelt / GSMA
5.0	07 May 2014	First version of the document for RCS 5.2: Include approved CR1004	GSG	Tom Van Pelt / GSMA
6.0	28 February 2015	First version of the document for RCS 5.3: Include approved CR1005	PSMC	Tom Van Pelt / GSMA
7.0	21 March 2016	First version of the document for RCS 6.0: Include approved CR1007	PSMC	Tom Van Pelt / GSMA
8.0	28 June 2017	First version of the document for RCS 7.0: Include approved CR1008	TG	Tom Van Pelt / GSMA
9.0	16 May 2018	Include approved CR1009	TG	Tom Van Pelt / GSMA
10.0	06 December 2018	Include approved CR1002	TG	Tom Van Pelt / GSMA
11.0	16 October 2019	Include approved CR1003	NG	Tom Van Pelt / GSMA
12.0	16 October 2020	Include approved CR1004	NG	Tom Van Pelt / GSMA
13.0	19 December 2022	Include approved CR1005	ISAG	Tom Van Pelt / GSMA
14.0	04 June 2024	Include approved CR1014	ISAG	Tom Van Pelt / GSMA

Version	Date	Brief Description of Change	Approval Authority	Editor / Company
15.0	28 February 2025	Include approved CR1015	ISAG	Tom Van Pelt / GSMA
16.0	18 July 2025	Include approved CR1016	ISAG	Tom Van Pelt / GSMA
17.0	19 February 2026	Include approved CR1017	ISAG	Tom Van Pelt / GSMA
18.0	17 July 2026	Include approved CR1018	ISAG	Tom Van Pelt / GSMA

D.2 Other Information

Type	Description
Document Owner	RCS Group
Editor / Company	Tom Van Pelt / GSMA

It is our intention to provide a quality product for your use. If you find any errors or omissions, please contact us with your comments. You may notify us at prd@gsma.com

Your comments or suggestions & questions are always welcome.